



考研计算机题型精练

(数据结构)

基础巩固篇 (第一轮复习使用)

进阶提高篇 (第二轮复习使用)

真题聚类篇 (第三轮复习使用)

万学海文考研

专业课教研中心

仅限内部使用

目 录

目 录	I
第一部分：基础巩固篇	1
1、绪论	1
1.1 基本概念	1
1.2 算法和复杂度计算	1
2、线性表	4
2.1 线性表的概念	4
2.2 线性表的顺序结构	4
2.3 线性表的链式结构	9
3、栈	20
4、队列	26
5、树	31
5.1 树的概念	31
5.2 二叉树的概念	32
5.3 二叉树的遍历	36
5.4 线索二叉树	41
5.5 树、森林	42
5.6 树与二叉树的应用	46
6、图	49
6.1 图的概念	49
6.2 图的存储	51
6.3 图的遍历	55
6.4 图的应用	59
7、查找	70
7.1 顺序查找	70
7.2 分块查找	71
7.3 折半查找	72
7.4 树型查找	75
7.5 B 树和 B+树	82
7.6 散列表	87
7.7 字符串模式匹配	92
7.8 查找算法的分析及应用	96
8、排序	99
8.1 排序的基本概念	99
8.2 插入类排序	99
8.3 交换类排序	101
8.4 选择类排序	103
8.5 分配类排序	105

8.6 归并类排序和外部排序	107
8.7 排序算法的分析与应用	110
9、多维数组和矩阵的压缩存储	118
9.1 多维数组	118
9.2 矩阵的压缩存储	118
第二部分：进阶提高篇	120
1、绪论	120
2、线性表	122
2.1 综合练习	122
2.2 线性表的基本操作算法专题	127
2.2.1 线性表的拆分与合并	127
2.2.2 线性表子集、交集	130
2.2.3 线性表的删除	134
2.3 线性表的应用算法专题	136
2.3.1 双指针策略	136
2.3.2 逆置策略	142
2.3.3 哈希策略	144
2.3.4 对称策略	145
2.3.5 旋转策略	147
2.4 线性表应用扩展算法专题	148
3、栈	150
4、队列	157
5、树	164
5.1 综合练习	164
5.2 算法专题	169
5.2.1 二叉树的遍历专题	169
5.2.2 树的存储专题	172
5.2.3 二叉树的删除专题	175
5.2.4 点数量计算专题	176
5.2.5 树的高度和宽度的计算	177
5.2.6 二叉树的交换专题	180
5.2.7 二叉树的路径与查找	180
5.2.8 二叉树的判定专题	184
5.2.9 线索二叉树	185
6、图	188
6.1 综合练习	188
6.2 算法专题	194
6.2.1 图的存储专题	194
6.2.2 图的遍历	194
6.2.3 出度和入度的计算	199
7、查找	201
7.1 综合练习	201

7.2 算法专题	213
7.2.1 折半查找	213
7.2.2 二叉排序树	213
7.2.3 平衡二叉树	216
7.2.4 串和子串	216
8、排序	223
8.1 综合练习	223
8.2 算法专题	228
8.2.1 插入排序算法	228
8.2.2 冒泡排序	229
8.2.3 快速排序	230
8.2.4 计数排序	232
8.2.5 简单选择排序	232
8.2.6 堆排序	233
9、多维数组和矩阵的压缩存储	235
9.1 综合练习	235
9.2 算法专题	236
9.2.1 数组运算	236
9.2.2 三元组	240
9.2.3 矩阵	243
第三部分：真题聚类篇	245
类型一：算法时间复杂度与空间复杂度	245
类型二：线性表的顺序结构和操作	247
类型三：链表的操作	247
类型四：静态链表	248
类型五：线性表算法应用	249
1. 双指针策略	249
2. 哈希策略	252
3. 单链表或者数组逆序	254
4. 单链表或者数组旋转	255
类型六：栈的基础概念和性质	256
类型七：栈的应用	258
类型八：队列的性质和应用	260
类型九：循环队列的概念和性质	262
类型十：树和二叉树的概念与性质	263
类型十一：二叉树的遍历	265
类型十二：线索二叉树的构造和性质	268
类型十三：树、二叉树和森林的转换	269
类型十四：哈夫曼树的构造和性质	271
类型十五：哈夫曼编码	272
类型十六：二叉树算法应用	275
类型十七：图的性质和概念	277
类型十八：图的存储	278

类型十九: 图的遍历	280
类型二十: 最小生成树的构造算法和性质	281
类型二十一: AOE 网、拓扑排序和关键路径	283
类型二十二: 最短路径	288
类型二十三: 图算法和综合应用	289
1. 图的基本算法	289
2. 图的应用	292
类型二十四: 顺序查找	295
类型二十五: 分块查找	296
类型二十六: 折半树的构造和查找	296
类型二十七: BST 的查找、插入、删除和判定	297
类型二十八: AVL 树的性质和调整	300
类型二十九: B-树和 B+树的性质	303
类型三十: B-树的插入和删除	305
类型三十一: 哈希函数构造和冲突解决方法	306
类型三十二: 串与 KMP 算法	309
类型三十三: 查找算法	311
类型三十四: 插入类排序	311
类型三十五: 交换类排序	313
类型三十六: 选择类排序	315
类型三十七: 基数类排序	317
类型三十八: 归并类排序	317
类型三十九: 外部排序	318
类型四十: 排序的比较分析	320
类型十四一: 排序算法题专题	322
类型四十二: 数组运算和矩阵压缩	323

第一部分：基础巩固篇

1、绪论

1.1 基本概念

1. 可以用 () 定义一个完整的数据结构。
A. 数据元素 B. 数据对象 C. 数据关系 D. 抽象数据类型
2. 以下关于数据结构的说法中, 正确的是 ()。
A. 数据的逻辑结构独立于其存储结构
B. 数据的存储结构独立于其逻辑结构
C. 数据的逻辑结构唯一决定其存储结构
D. 数据结构仅由其逻辑结构和存储结构决定
3. 在存储数据时, 通常不仅要存储各数据元素的值, 而且要存储 ()。
A. 数据的操作方法 B. 数据元素的类型
C. 数据元素之间的关系 D. 数据的存取方法
4. 从逻辑上可以把数据结构分为 () 两大类。
A. 动态结构、静态结构 B. 顺序结构、链式结构
C. 线性结构、非线性结构 D. 初等结构、构造型结构

【答案解析】

1.D。抽象数据类型 (ADT) 描述了数据的逻辑结构和抽象运算, 通常用 (数据对象, 数据关系, 基本操作集) 这样的三元组来表示, 从而构成一个完整的数据结构定义。

2.A。数据的逻辑结构是从面向实际问题的角度出发的, 只采用抽象表达方式, 独立于存储结构, 数据的存储方式有多种不同的选择; 而数据的存储结构是逻辑结构在计算机上的映射, 它不能独立于逻辑结构而存在。数据结构包括三个要素, 缺一不可。

3.C。在存储数据时, 不仅要存储数据元素的值, 而且要存储数据元素之间的关系。

4.C。数据结构的基本概念

1.2 算法和复杂度计算

1. 算法的计算量的大小称为算法的 ()。
A. 效率 B. 复杂度 C. 现实性 D. 难度
2. 算法的时间复杂度取决于 ()。
A. 问题的规模 B. 待处理数据的初态 C. A 和 B
3. 一个算法应该是 ()。
A. 程序 B. 问题求解步骤的描述
C. 要满足 5 个基本特性 D. A 和 C

4. 下面关于算法说法正确的是 ()。

- A. 算法最终必须由计算机程序实现
- B. 为解决某问题的算法同为该问题编写的程序含义是相同的
- C. 算法的可行性是指指令不能有二义性
- D. 以上几个都是错误的

5. 一个算法应该是 ()。

- A. 程序
- B. 问题求解步骤的描述
- C. 要满足五个基本特性
- D. A 和 C

6. 某算法的时间复杂度为 $O(n^2)$, 表明该算法的 ()。

- A. 问题规模是 n^2
- B. 执行时间等于 n^2
- C. 执行时间与 n^2 成正比
- D. 问题规模与 n^2 成正比

7. 以下算法的时间复杂度为 ()。

```
void fun(int n){
    int i=1;
    while(i<=n)
        i=i*2;}
```

- A. $O(n)$
- B. $O(n^2)$
- C. $O(n\log_2 n)$
- D. $O(\log_2 n)$

8. 有以下算法, 其时间复杂度为 ()。

```
void fun (int n ) {
    int i=0;
    while(i*i*i<=n)
        i++;}
```

- A. $O(n)$
- B. $O(n\log n)$
- C. $O(\sqrt[3]{n})$
- D. $O(\sqrt{n})$

9. 程序段如下:

```
for(i=n-1;i>1;i--)
    for(j=1;j<i;j++)
        if (A[j]>A[j+1]) //A[j]与 A[j+1]对换;
```

其中 n 为正整数, 则最后一行语句的频度在最坏情况下是 ()。

- A. $O(n)$
- B. $O(n\log n)$
- C. $O(n^3)$
- D. $O(n^2)$

10. 以下算法中加下画线的语句的执行次数为 ()。

```
int m=0,i,j;
for(i=1;i<=n;i++)
    for (j=1;j<=2*i;j++)
        m++;
```

- A. $n(n+1)$
- B. n
- C. $n+1$
- D. n

11. 在下面的程序段中, 对 x 的赋值语句的频度为 ()。

```
for(i=0;i<n; ++i)
    for(j=0;j<n; ++j)
        ++x;
```

- A. $O(2n)$
- B. $O(n)$
- C. $O(n^2)$
- D. $O(\log_2 n)$

12. 下面说法中, 错误的是 ()。

- I. 算法原地工作的含义是指不需要任何额外的辅助空间
- II. 在相同规模 n 下, 复杂度为 $O(n)$ 的算法在时间上总是优于复杂度为 $O(n^2)$ 的算法
- III. 所谓时间复杂度, 是指最坏情况下估算算法执行时间的一个上界
- IV. 同一个算法, 实现语言的级别越高, 执行效率越低

- A. I
- B. I, II
- C. I, IV
- D. III

【答案解析】

1.B. 本题考查算法时间复杂度的定义。算法中基本操作的重复执行次数就是算法, 将其大小作为算法的时间复杂度, 因此选 B。

2.C. 本题考查算法时间复杂度的定义。算法时间复杂度即为基本操作执行次数, 显然问题规模越大, 基本操作的次数越多, 因此时间复杂度与规模有关。在相同的规模下, 与数据初态也有关, 如两个数相乘, 有一个因子为 0 时的计算速度显然要比两个因子都非 0 的情况要快。因此本题选 C。

3.B. 算法的基本概念。

4.D. 本题考查算法的概念。选项 A, 计算机程序只是实现算法的一种手段, 手工也可以完成。选项 B, 算法可以理解为由基本运算及规定的运算顺序所构成的完整的解题步骤。程序是为实现特定目标或解决特定问题而用计算机语言编写的命令序列的集合。两者显然是不同的概念。选项 C, 明显错误。

5.B. 本题是中山大学某年的考研真题, 题目本身没有问题, 考查的是算法的定义。程序不一定满足有穷性, 如死循环、操作系统等, 而算法必须有穷。算法代表对问题求解步骤的描述, 而程序则是算法在计算机上的特定实现。不少读者认为 C 也对, 它只是算法的必要条件, 不能成为算法的定义。

6.C. 时间复杂度为 $O(n^2)$, 说明算法的时间复杂度 $T(n)$ 满足 $T(n) \leq cn^2$ (C 为比例常数), 即 $T(n) = O(n^2)$, 时间复杂度 $T(n)$ 是问题规模 n 的函数, 其问题规模仍然是 n 而不是 n^2 。

7.D. 找出基本运算 $i=i*2$, 设执行次数为 t , 则 $2^t \leq n$, 即 $t \leq \log_2 n$, 因此时间复杂度 $T(n) = O(\log_2 n)$ 。更直观的方法: 计算基本运算 $i=i*2$ 的执行次数 (每执行一次 i 乘 2), 其中判断条件可理解为 $2^t = n$, 即 $t = \log_2 n$, 则 $T(n) = O(\log_2 n)$ 。

8.C. 基本运算为 $i++$, 设执行次数为 t , 有 $t \times t \times t \leq n$, 即 $t^3 \leq n$ 。故有 $t \leq (\sqrt[3]{n})$, 则 $T(n) = O(\sqrt[3]{n})$ 。

9.D. 这是冒泡排序的算法代码, 考查最坏情况下的元素交换次数 (若觉得理解困难可在学完第 8 章后再回顾)。当所有相邻元素都为逆序时, 则最后一行的语句每次都会执行。此时,

$$T(n) = \sum_{i=2}^{n-1} \sum_{j=1}^{i-1} 1 = \sum_{i=2}^{n-1} i - 1 = \frac{(n-2)(n-1)}{2} = O(n^2)$$

所以在最坏情况下的该语句频度是 $O(n^2)$ 。

10.A. $m++$ 语句的执行次数为

$$\sum_{i=1}^n \sum_{j=1}^{2i} 1 = \sum_{i=1}^n 2i = 2 \sum_{i=1}^n i = n(n+1)$$

11.C. 本题考查算法时间复杂度的计算。 $f(n) = n^2$, 因此时间复杂度是 $O(n^2)$ 。

12.A. I, 算法原地工作是指算法所需的辅助空间是常量。II, 本项考查算法效率的理解, 时间复杂度是指渐近时间复杂度, 不要想当然地去给 n 赋予一个特殊值, 时间复杂度为 $O(n)$ 的算法必然优于时间复杂度为 $O(n^2)$ 的算法。III, 时间复杂度总是考虑最坏情况下的时间复杂度, 以保证算法的运行时间不会比它更长。IV 为严蔚敏教材中的原话, 该问题在论坛讨论过多年, 对于这种在语言层次上的效率问题, 建议不要以特例程序来解释其优劣, 此处认为该结论是正确的。

2、线性表

2.1 线性表的概念

- 线性表是具有 n 个 () 的有限序列。
A. 数据表 B. 字符 C. 数据元素 D. 数据项
- 以下 () 是一个线性表。
A. 由 n 个实数组成的集合 B. 由 100 个字符组成的序列
C. 所有整数组成的序列 D. 邻接表
- 在线性表中, 除开始元素外, 每个元素()。
A. 只有唯一的前趋元素 B. 只有唯一的后继元素
C. 有多个前趋元素 D. 有多个后继元素
- 线性表是具有 n 个 () 的有限序列。
A. 表元素 B. 字 符 C. 数据元素 D. 数据项
- 线性表是()。
A. 一个有限序列, 可以为空 B. 一个有限序列, 不可以为空
C. 一个无限序列, 可以为空 D. 一个无限序列, 不可以为空

【答案解析】

- C. 线性表是由具有相同数据类型的有限数据元素组成的, 数据元素是由数据项组成的。
- B. 线性表 定义的要求为: 相同数据类型、有限序列。选项 C 的元素个数是无穷个, 错误; 选项 A 集合中的元素没有前后驱关系, 错误; 选项 D 属于存储结构, 线性表是一种逻辑结构, 不要将二者混为一谈。只有选项 B 符合线性表定义的要求。
- A. 线性表中, 除最后一个 (或第一个) 元素外, 每个元素都只有一个后继 (或前驱) 元素。
- C. 本题考查线性表的定义。线性表是具有相同特性数据元素的一个有限序列。
- A. 本题考查线性表的定义。线性表是具有相同特性数据元素的一个有限序列。该序列中所含元素的个数叫作线性表的长度, 用 $n(n \geq 0)$ 表示。注意, n 可以等于零, 表示线性表是一个空表。空表也可以作为一个线性表。因此本题选 A。



2.2 线性表的顺序结构

(一) 选择题

- 下述各项中属于顺序存储结构优点的是 ()。
A. 存储密度大 B. 插入运算方便
C. 删除运算方便 D. 可方便地用于各种逻辑结构的存储表示
- 下面关于线性表的叙述中, 错误的是 ()。
A. 线性表采用顺序存储, 必须占用一片连续的存储单元
B. 线性表采用顺序存储, 便于进行插入和删除操作

5.B. 顺序表所占的存储空间=表长 $\times$ sizeof (元素的类型), 元素的类型显然会影响存储空间的大小。对于同一元素类型的顺序表, 表越长, 所占存储空间就越大。

6.D. 题干实际要求能最快存取第 $i-1$ 、 i 和 $i+1$ 个元素值。A、B、C 都只能从头结点依次顺序查找, 时间复杂度为 $O(n)$; 只有顺序表可以按序号随机存取, 时间复杂度为 $O(1)$ 。

7.A. 只有顺序表可以按序号随机存取, 且在最后进行插入和删除操作时不需要移动任何元素。

8.C. 需要将 $a_{i+1} \sim a_n$ 元素前移一位, 共移动 $n-(i+1)+1=n-i$ 个元素。

9.B. 线性表元素的序号是从 1 开始, 而在第 $n+1$ 个位置插入相当于在表尾追加。

10.D. 顺序存储需要连续的存储空间, 在申请时需申请 $n+m$ 个连续的存储空间, 然后将线性表原来的 n 个元素复制到新申请的 $n+m$ 个连续的存储空间的前 n 个单元。

(二) 综合应用题

1. 线性表可用顺序表或链表存储。试问:

(1) 如果有 n 个表同时并存, 并且在处理过程中各表的长度会动态发生变化, 表的总数也可能自动改变, 在此情况下, 应选用哪种存储表示? 为什么?

(2) 若表的总数基本稳定, 且很少进行插入和删除, 但要求以最快的速度存取表中的元素, 这时应采用哪种存储表示? 为什么?

2. 设计一个算法, 从一给定的顺序表 L 中删除下标 $i \sim j$ ($i \leq j$, 包括 i 、 j) 的所有元素, 假定 i 、 j 都是合法的。

3. 从顺序表中删除具有最小值的元素 (假设唯一) 并由函数返回被删元素的值。空出的位置由最后一个元素填补, 若顺序表为空, 则显示出错信息并退出运行。

4. 设计一个高效算法, 将顺序表 L 的所有元素逆置, 要求算法的空间复杂度为 $O(1)$ 。

5. 对长度为 n 的顺序表 L , 编写一个时间复杂度为 $O(n)$ 、空间复杂度为 $O(1)$ 的算法, 该算法删除线性表中所有值为 x 的数据元素。

6. 从有序顺序表中删除其值在给定值 s 与 t 之间 (要求 $s < t$) 的所有元素, 若 s 或 t 不合理或顺序表为空, 则显示出错信息并退出运行。

7. 从有序顺序表中删除所有其值重复的元素, 使表中所有元素的值均不同。

8. 试设计一个算法, 将数组 $A[0, \dots, n-1]$ 中所有奇数移到偶数之前。要求不另增加存储空间, 且时间复杂度为 $O(n)$ 。

【答案解析】

1. 答 (1) 采用链表。如果采用顺序表, 在多个表并存的情况下, 在问题求解的过程中, 一旦发现某个表有存满并溢出的情况, 很可能需要移动其他表以腾出位置为其扩充空间, 导致不断地把大片数据移来移去。不但时间耗费很多, 而且操作复杂, 容易出错。如果表的总数还要变化, 则还会带来需要不影响其他表工作的情况下开辟新表空间或者释放旧表空间的操作上的麻烦。如果采用链表就没有这些问题, 一般在内存空间足够的情况下, 各个表的空间分配或释放不受其他表的影响。

(2) 采用顺序表。若表的总数基本稳定, 且很少进行插入和删除, 则顺序表可以充分发挥其存取速度快、存储利用率高的优点。

2. 分析: 本题是顺序表删除算法的扩展, 可以采用如下方法解决: 从第 $j+1$ 个元素开始到最后一个元素为止, 用这之间的每个元素去覆盖从这个元素开始往前数第 $j-i+1$ 个元素, 即可完成删除 $i \sim j$ 的所有元素。

本题代码如下:

```
void delete(SqList &L, int i, int j) {           // L 要改变, 故用引用型
    int k, delta;
    delta = j - i + 1;                          // 元素要移动的距离
    for (k = j + 1; k < L.length; ++k)
        L.data[k - delta] = L.data[k];         // 用第 K 个元素去覆盖它前边的第 delta 个元素
    L.length -= delta;                          // 表长改变
}
```

说明: 有同学问: “从 $j+1$ 开始到表中最后一个元素为止, 如果这之间的元素个数小于 i 到 j 之间的元素个数, 岂不是无法完全覆盖导致删除失败?” 注意, 覆盖和删除没有关系, 只要落在表长范围之外的元素即为被删除元素, 也就是说, 如果需要删除表中的所有元素, 只需要让 `length` 赋值为 0 即可, 不需要拿某个元素来覆盖一遍。

3. 算法思想: 搜索整个顺序表, 查找最小值元素并记住其位置, 搜索结束后用最后一个元素填补空出的原最小值元素的位置。

本题代码如下:

```
bool Del_Min(SqList &L, ElemType &value) {
    // 删除顺序表 L 中最小值元素结点, 并通过引用型参数 value 返回其值
    // 若删除成功, 则返回 true; 否则返回 false
    if (L.length == 0)
        return false;                          // 表空, 中止操作返回
    value = L.data[0];
    int pos = 0;                               // 假定 0 号元素的值最小
    for (int i = 1; i < L.length; ++i) {       // 循环, 寻找具有最小值的元素
        if (L.data[i] < value) {               // 让 value 记忆当前具有最小值的元素
            value = L.data[i];
            pos = i;
        }
    }
    L.data[pos] = L.data[L.length - 1];        // 空出的位置由最后一个元素填补
    L.length--;
    return true;                               // 此时, value 即为最小值
}
```

注意: 本题也可用函数返回值返回, 两者的区别是: 函数返回值只能返回一个值, 而参数返回 (引用传参) 可以返回多个值。

4. 算法思想: 扫描顺序表 L 的前半部分元素, 对于元素 $L.data[i]$ ($0 \leq i < L.length/2$), 将其与后半部分的对应元素 $L.data[L.length-i-1]$ 进行交换。本题代码如下:

```
void Reverse(SqList &L) {
    ElemType temp;                             // 辅助变量
    for (i = 0; i < L.length / 2; ++i) {
        temp = L.data[i];                      // 交换 L.data[i] 与 L.data[L.length - i - 1]
        L.data[i] = L.data[L.length - i - 1];
        L.data[L.length - i - 1] = temp;
    }
```

```

        L.data[i] = L.data[L.length - i - 1];
        L.data[L.length - i - 1] = temp;
    }
}

```

5.解答：解法一：用 K 记录顺序表 L 中不等于 x 的元素个数（即需要保存的元素个数），边扫描 L 边统计 k，并将不等于 x 的元素向前移动 k 个位置，最后修改 L 的长度。本题代码如下：

```

void del_x_1(SqList &L, ElemType x) {
//本算法实现删除顺序表 L 中所有值为 x 的数据元素
    int k=0; //记录值不等于 x 的元素个数
    for(i=0; i<L.length; i++)
        if(L.data[i] != x) {
            L.data[k] = L.data[i];
            k++; //不等于 x 的元素增 1
        }
    L.length = k; //顺序表 L 的长度等于 k
}

```

解法二：用 K 记录顺序表 L 中等于 x 的元素个数，边扫描 L 边统计 k，并将不等于 x 的元素前移 k 个位置，最后修改 L 的长度。

本题代码如下：

```

void del_x_2(SqList &L, ElemType x) {
    int k = 0, i = 0; // k 记录值等于 x 的元素个数
    while (i < L.length) {
        if (L.data[i] == x)
            k++; // 当前元素前移 k 个位置
        else
            L.data[i - k] = L.data[i];
        i++;
    }
    L.length = L.length - k; // 顺序表 L 的长度递减
}

```

此外，本题还可以考虑设头、尾两个指针（ $i=1, j=n$ ），从两端向中间移动，在遇到最左端值 x 的元素时，直接将最右端值非 x 的元素左移至值为 x 的数据元素位置，直到两指针相遇。但这种方法会改变原表中元素的相对位置。

6 解答：注意：本题与上一题存在区别。因为是有序表，所以删除的元素必然是相连的整体。

算法思想：先寻找值大于等于 s 的第一个元素（第一个删除的元素），然后寻找值大于 t 的第一个元素（最后一个删除的元素的下一个元素），要将这段元素删除，只需直接将后面的元素前移。

本题代码如下：

```

bool Del_s_t(SqList &L, ElemType s, ElemType t) {
// 删除有序顺序表 L 中值在给定值 s 与 t 之间的所有元素
    int i, j;
    if (s >= t || L.length == 0)
        return false; // 如果 s >= t 或表为空，返回 false
    for (i = 0; i < L.length && L.data[i] < s; i++); // 寻找值大于等于 s 的第一个元素
    if (i >= L.length)
        return false; // 所有元素值均小于 s，返回 false
    for (j = i; j < L.length && L.data[j] <= t; j++); // 寻找值大于 t 的第一个元素
    for (; j < L.length; i++, j++) {
        L.data[i] = L.data[j]; // 前移，填补被删元素位置
    }
}

```

```

    }
    L.length = i;           // 更新顺序表的长度
    return true;
}

```

7 解答: 算法思想: 注意是有序顺序表, 值相同的元素一定在连续的位置上, 用类似于直接插入排序的思想, 初始时将第一个元素视为非重复的有序表。之后依次判断后面的元素是否与前面非重复有序表的最后一个元素相同, 若相同, 则继续向后判断, 若不同, 则插入前面的非重复有序表的最后, 直至判断到表尾为止。

本题代码如下:

```

bool Delete_Same(SeqList &L) {
    if (L.length == 0)
        return false;           // 如果顺序表为空, 返回 false
    int i, j;                     // i 存储第一个不相同的元素, j 为工作指针
    for (i = 0, j = 1; j < L.length; j++) {
        if (L.data[i] != L.data[j])
            L.data[++i] = L.data[j];   // 查找下一个与上个元素值不同的元素
    }                               // 找到后, 将元素前移
    L.length = i + 1;             // 更新顺序表的长度
    return true;
}

```

对于本题的算法, 请读者用序列 1, 2, 2, 2, 2, 3, 3, 3, 4, 4, 5 来手动模拟算法的执行过程, 在模拟过程中要标注 i 和 j 所指示的元素。

8. 算法分析: 设置两个指针 i 和 j, i 从数组的左边往右边遍历, j 从数组的右边往左边遍历, 当 i 指向偶数, j 指向奇数的时候, 将 A[i] 与 A[j] 交换; 当 i 大于等于 j 的时候, 算法结束。算法实现代码如下:

```

void divide(int A[], int n) {
    int i = 0, j = n - 1, temp;
    while (i < j) {
        while (A[i] % 2 == 1 && i < j)   // 循环扫描数组, 将 i 停在 A 中的偶数上
            ++i;
        while (A[j] % 2 == 0 && i < j)   // 循环扫描数组, 将 j 停在 A 中的奇数上
            --j;
        if (i < j) {                     // A[i] 与 A[j] 交换
            temp = A[i];
            A[i] = A[j];
            A[j] = temp;
            ++i;
            --j;
        }
    }
}

```

说明: 本题虽然是两层循环, 但实际上仅对数组进行了一次遍历, 因此时间复杂度为 $O(n)$

2.3 线性表的链式结构

(一) 选择题

1. 关于线性表的顺序存储结构和链式存储结构的描述中, 正确的是()。

I. 线性表的顺序存储结构优于其链式存储结构

- II. 链式存储结构比顺序存储结构能更方便地表示各种逻辑结构
- III. 若频繁使用插入和删除结点操作, 则顺序存储结构更优于链式存储结构
- IV. 顺序存储结构和链式存储结构都可以进行顺序存取
- A. I、II、III B. II、IV C. II、III D. III、IV
2. 某线性表中最常用的操作是在最后一个元素之后插入一个元素和删除第一个元素, 则采用 () 存储方式最节省运算时间。
- A. 单链表 B. 不带头结点的单循环链表
C. 双链表 D. 不带头结点且有尾指针的单循环链表
3. 静态链表中指针指示的是 ()。
- A. 内存地址 B. 数组下标
C. 链表中下一元素在数组中的地址 D. 左、右孩子地址
4. 链表不具有的特点是 ()。
- A. 插入、删除不需要移动元素 B. 可随机访问任一元素
C. 不必事先估计存储空间 D. 所需空间与线性长度成正比
5. 将两个有 n 个元素的有序表归并成一个有序表, 其最少比较次数为 ()。
- A. n B. $2n-1$ C. $2n$ D. $n-1$
6. 在一个具有 n 个结点的有序单链表中插入一个新结点仍然保持有序的时间复杂度是 ()。
- A. $O(1)$ B. $O(n)$ C. $O(n^2)$ D. $O(n\log_2 n)$
7. 在一个长度为 n ($n>1$) 的带头结点的单链表 h 上, 另设有尾指针 r (指向尾结点), 执行 () 操作与链表的长度有关。
- A. 删除单链表中的第一个结点 B. 删除单链表中的最后一个结点
C. 在单链表第一个元素前插入一个新结点 D. 在单链表最后一个元素后插入一个新结点
8. 线性表采用链表存储时, 其结点地址 ()。
- A. 必须是连续的 B. 一定是不连续的
C. 部分地址必须是连续的 D. 连续与否均可以
9. 线性表的静态链表存储结构与顺序存储结构相比, 其优点是 ()。
- A. 所有的操作算法实现简单 B. 便于随机存取
C. 便于插入和删除 D. 便于利用零散的存储空间
10. 设线性表有 n 个元素, 以下操作中, () 在顺序表上实现比在链表上实现效率更高。
- A. 输出第 i ($1 \leq i \leq n$) 个元素值 B. 交换第 1 个元素与第 2 个元素的值
C. 顺序输出这 n 个元素的值 D. 输出与给定值 x 相等的元素在线性表中的序号
11. 需要分配较大的连续空间, 插入和删除不需要移动元素的线性表, 其存储结构为 ()。
- A. 单链表 B. 静态链表 C. 顺序表 D. 双链表
12. 如果最常用的操作是取第 i 个元素的前驱结点, 则采用 () 存储方式最节省时间。
- A. 单链表 B. 双链表 C. 单循环链表 D. 顺序表
13. 与单链表相比, 双链表的优点之一是 ()。
- A. 插入、删除操作更简单 B. 可以进行随机访问
C. 可以省略表头指针或表尾指针 D. 访问前后相邻结点更灵活
14. 若某链表最常用的操作是在最后一个结点之后插入一个结点或删除最后一个结点, 则采用 () 存储结构最节省运算时间。
- A. 单链表 B. 给出表头指针的循环单链表

- C. 双链表 D. 带头结点的循环双链表
15. 线性表最常用的操作是在最后一个结点之后插入一个结点或删除第一个结点, 则采用 () 存储方式最节省时间。
- A. 单链表 B. 仅有头结点的单循环链表
C. 双链表 D. 仅有尾结点指针的单循环链表
16. 下列关于线性表说法中, 正确的是()。
- I. 顺序存储方式只能用于存储线性结构
II. 取线性表的第 i 个元素的时间与 i 的大小有关
III. 静态链表需要分配较大的连续空间, 插入和删除不需要移动元素
IV. 在一个长度为 n 的有序单链表中插入一个新结点并仍保持有序的时间复杂度 为
V. 若用单链表来表示队列, 则应该选用带尾指针的循环链表
- A. I、II B. I、III、IV、V C. IV、V D. III、IV、V
17. 设线性表中有 $2n$ 个元素, () 在单链表上实现要比在顺序表上实现效率更高。
- A. 删除所有值 为 x 的元素
B. 在最后一个元素的后面插入一个新元素
C. 顺序输出前 k 个元素
D. 交换第 i 个元素和第 $2n-i-1$ 个元素的值 ($i=0, \dots, n-1$)
18. 单链表中, 增加一个头结点的目的是 ()。
- A. 使单链表至少有一个结点 B. 标识表结点中首结点的位置
C. 方便运算的实现 D. 说明单链表是线性表的链式存储
19. 下面关于线性表的一些说法中, 正确的是 ()。
- A. 对一个设有头指针和尾指针的单链表执行删除最后一个元素的操作与链表长度无关
B. 线性表中每个元素都有一个直接前驱和一个直接后继
C. 为了方便插入和删除数据, 可以使用双链表存放数据
D. 取线性表第 i 个元素的时间与 i 的大小有关
20. 与单链表相比, 双链表的优点之一是 ()。
- A. 插入、删除操作更方便 B. 可以进行随机访问
C. 可以省略表头指针或表尾指针 D. 访问前后相邻结点更灵活
21. 设对 $n (n>1)$ 个元素的线性表的运算只有 4 种: 删除第一个元素; 删除最后一个元素; 在第一个元素之前插入新元素; 在最后一个元素之后插入新元素, 则最好使用()。
- A. 只有尾结点指针没有头结点指针的循环单链表
B. 只有尾结点指针没有头结点指针的非循环双链表
C. 只有头结点指针没有尾结点指针的循环双链表
D. 既有头结点指针又有尾结点指针的循环单链表

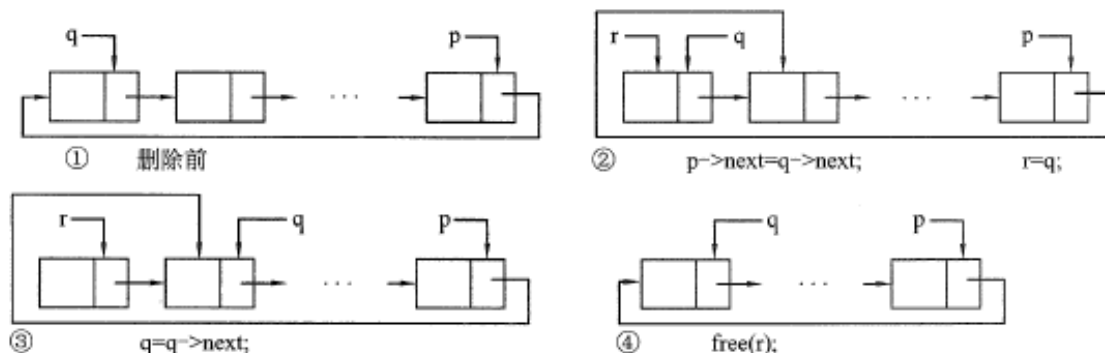
【答案解析】

1.B。两种存储结构有不同的适用场合, 不能简单地谁好谁坏, I 错误。链式存储用指针表示逻辑结构, 而指针的设置是任意的, 故可以很方便地表示各种逻辑结构; 顺序存储只能用物理上的邻接关系来表示逻辑结构, II 正确。在顺序存储中, 插入和删除结点需要移动大量元素, 效率较低, III 的描述刚好相反。顺序存储结构既能随机存取又能顺序存取, 而链式 结构只能进行顺序存取, IV 正确。

2.D。本题考查几种链表的插入和删除操作。

对于 A 选项,单链表要在最后一个元素后插入一个元素,需要遍历整个链表才能找到插入位置,时间复杂度是 $O(n)$ 。删除第一个元素时间复杂度是 $O(1)$ 。

对于 B 选项,不带头结点的单循环链表在最后一个元素之后插入一个元素的情况和单链表相同,时间复杂度是 $O(n)$ 。对于删除表中第一个元素的情况,同样需要找到终端结点,即删除结点的时间复杂度也是 $O(n)$,因为终端结点的指针 $next$ 指向开始结点,首先需要遍历整个链表找到终端结点,然后执行 $p \rightarrow next = q \rightarrow next$; $r = q$; $q = q \rightarrow next$; 和 $free(r)$; 4 句才满足删除要求(其中 p 指向终端结点, q 指向起始结点, r 用来帮助释放结点空间)。删除过程如下图所示



说明:本选项的讲解中顺便给出了不带头结点的单循环链表中删除开始结点的算法过程,供考生参考。

对于 C 选项,双链表的情况和单链表相同,一个为 $O(n)$,一个为 $O(1)$ 。

对于 D 选项,有尾指针(指向表中终端结点的指针),因为已经知终端结点的位置,所以省去了 B 选项中链表的遍历过程,因此删除和插入结点的时间复杂度为 $O(1)$ 。

综上所述,本题选 D。

3.B。静态链表是用数组实现链表的一种存储方法。在静态链表中,每个数组元素由两个域构成。

① data 域存放数据元素。

② next 域(也称游标)存放该元素的后继元素所在数组下标。

静态链表中指针表示的是后继元素在数组中的下标。

4.B。本题考查顺序表和链表的比较。显然随机存储是顺序表的特性,而不是链表的特性。

5.A。本题考查线性表的归并算法。在归并算法中,当 $L1$ 表中的所有元素均小于 $L2$ 表中的所有元素时,比较次数最少,为 n 。因为此时 $L1$ 中的元素在比较了 n 次后已经全部被并入顺序表,剩下的 $L2$ 中的元素就不需要再比较了。

注意:前些年有些同学提问:“如果第一个表中的第一个元素大于第二个表中的最后一个元素(假设递增有序),岂不是只需要比较一次即可?”没错,单单把两个有序表合并成一个有序表是可以出现这种情况。但是题目提到“归并”二字,在考研中,归并一般特指前边讲到的那个 merge 算法,而你的这句:“如果第一个表中的第一个元素大于第二个表中的最后一个元素”包含了一个归并算法中没有的判断操作,也就是说这样就不是标准的归并算法了。

补充:如果本题问比较次数最多是几次,又该选什么呢?答案是 $2n-1$ 次,此时 $L1$ 与 $L2$ 中元素轮流被并入顺序表,每并入一个就比较一次,当剩下最后两个元素的时候($L1$ 与 $L2$ 表各一个),已经比较了 $2n-2$ 次,这时还需进行一次比较就有一个表变为空表,至此所有比较结束,因此总的比较次数是 $2n-1$ 次。

6.B。本题考查链表的查找与插入操作。这里假设单链表递增有序(递减的情况同理),在插入

数据为 x 的结点之前, 先要在单链表中找到第一个大于 x 的结点的直接前驱 p , 在 p 之后插入该结点。查找过程的时间复杂度为 $O(n)$, 插入过程的时间复杂度为 $O(1)$, 整个过程的时间复杂度为 $O(n)$ 。

7.B。题考查链表的删除操作。在带有头结点的链表中要删除一个结点, 必须知道其前驱结点的位置。由题可知, 第一个结点的前驱结点位置已知, 为 h 所指的头结点, 因此无论链表多长, 删除其中第一个结点的操作都是 $p=h \rightarrow next$; $h \rightarrow next=p \rightarrow next$; 和 $free(p)$; 这 3 条语句。尾结点的前驱结点地址未知, 因此要删除链表中的最后一个结点, 需要从头结点开始遍历整个链表来找到最后一个结点的前驱结点, 链表越长, 执行操作的循环次数越多, 即删除最后一个结点的操作与链表长度有关。

要在某一个结点后插入一个结点, 则必须知道这个结点的地址, C 选项中在第一个元素前插入一个新结点, 等价于在头结点后插入一个新结点, 头结点地址已知, 插入操作与链表长度无关。同样, D 选项, 尾结点地址已知, 在其后插入新结点的操作与链表长度也无关。

8.D。本题考查链表存储结构的特点。一般链表结点在内存中是分散存在的, 因此可以是不连续的。当然, 如果连续地申请一片存储空间来存储链表结点也未尝不可。因此本题选 D。

9.C。本题考查静态链表与顺序表的特点。静态链表具有链表的插入和删除方便的特点, 也不需要移动较多的元素, 这是优于顺序表的地方, 因此本题选 C。

10.A。本题考查顺序表与链表的特点。

顺序表支持随机存储, 链表不支持, 因此顺序表输出第 i 个元素值的时间复杂度为 $O(1)$, 链表则为 $O(n)$, 因此 A 选项正确。

交换第 1 个与第 2 个元素的值, 对于顺序表与链表, 时间复杂度均为 $O(1)$, 因此 B 选项错误。输出 n 个元素的值, 两者时间复杂度均为 $O(n)$, 因此 C 选项错误。

输出与给定值 x 相等的元素在线性表中的序号, 对于顺序表和链表, 都需要搜索整个表, 因此时间复杂度均为 $O(n)$, D 选项错误。

11.B。本题考查静态链表的特点。静态链表用数组来表示, 因此需要分配较大的连续空间。静态链表同时还具有一般链表的特点, 即插入和删除操作不需要移动元素。因此本题选 B。

12.D。本题考查顺序表的特点。较之其他三种存储结构, 顺序表找到表中第 i 个元素的时间复杂度最小, 为 $O(1)$; 取第 i 个元素前驱的时间复杂度也是最小的, 为 $O(1)$ 。因此本题选 D。

13.D。本题考查双链表的特点。在双链表中可以快速访问任何一个结点的前后相邻结点, 而在单链表中只能快速访问任何一个结点的后继结点。因此本题选 D。

14.D。本题考查链表的基本操作。在链表中插入或删除一个结点, 需要修改相邻结点的指针域。如不特别指明, 通常只给出链表头结点的地址, 其他结点的地址只能从它的前驱或者后继得到。只有 D 选项能通过头结点指针直接获得最后一个结点的相邻结点的地址, 因此本题选 D。

15.D。本题考查链表的基本操作。在有尾结点指针 r 的循环单链表中, 在最后一个结点之后插入结点 s 的操作是 $s \rightarrow next=r \rightarrow next$; $r \rightarrow next=s$; $r=s$; , 删除第一个结点的操作是 $p=r \rightarrow next$; $r \rightarrow next=p \rightarrow next$; $free(p)$; , 其时间复杂度均为 $O(1)$ 。因此本题选 D。

16.D。顺序存储方式同样适合图和树, I 错误。线性表采用顺序存储时 II 错误。III 是静态链表的特点。有序单链表只能依次查找插入位置, 时间复杂度为 $O(n)$, IV 正确。队列需要在表头删除元素, 表尾插入元素, 采用带尾指针的循环链表较为方便, 插入和删除的时间复杂度都为 $O(1)$, V 正确。

17.A。对于 A, 在单链表和顺序表上实现的时间复杂度都为 $O(n)$, 但后者要移动很多元素, 因此在单链表上实现效率更高。对于 B 和 D, 顺序表的效率更高。C 无区别。

18.C。单链表设置头结点的目的是方便运算的实现, 主要好处体现在: 第一, 有头结点后, 插入

和删除数据元素的算法就统一了, 不再需要判断是否在第一个元素之前插入或删除第一个元素; 第二, 不论链表是否为空, 其头指针是指向头结点的非空指针, 链表的头指针不变, 因此空表和非空表的处理也就统一了。

19.C. 双链表能很方便地访问前驱和后继, 故删除和插入数据较为方便。A 显然错误。B 表中第一个元素和最后一个元素不满足题设要求。D 未考虑顺序存储的情况。

20.D 在插入和删除操作上, 单链表和双链表都不用移动元素, 都很方便, 但双链表修改指针的操作更为复杂, A 错误。双链表中可以快速访问任何一个结点的前驱和后继结点, D 正确。

21.C. 对于 A, 删除尾结点 *p 时, 需要找到 *p 的前一个结点, 时间复杂度为 $O(n)$ 。对于 B, 删除首结点 *p 时, 需要找到 *p 结点, 这里没有直接给出头结点指针, 而通过尾结点的 prior 指针找到 *p 结点的时间复杂度为 $O(n)$ 。对于 D, 删除尾结点 *p 时, 需要找到 *p 的前一个结点, 时间复杂度为 $O(n)$ 。对于 C, 执行这 4 种算法的时间复杂度均为 $O(1)$ 。

(二) 综合应用题

1. 为什么在单循环链表中设置尾指针比设置头指针更好?
2. 有一个递增非空单链表, 设计一个算法删除值域重复的结点。例如 {1, 1, 2, 3, 3, 3, 4, 4, 7, 7, 7, 9, 9, 9} 经过删除后变成 {1, 2, 3, 4, 7, 9}。
3. 设计一个算法删除单链表 L (有头结点) 中的一个最小值结点。
4. 有一个线性表, 采用带头结点的单链表 L 来存储。设计一个算法将其逆置。要求不能建立新结点, 只能通过表中已有结点的重新组合来完成。
5. 设计一个递归算法, 删除不带头结点的单链表 L 中所有值为 x 的结点。
6. 在带头结点的单链表 L 中, 删除所有值为 x 的结点, 并释放其空间, 假设值为 x 的结点不唯一, 试编写算法以实现上述操作。
7. 设在一个带头结点的单链表中所有元素结点的数据值无序, 试编写一个函数, 删除表中所有介于给定的两个值 (作为函数参数给出) 之间的元素的元素 (若存在)。
8. 设计一个算法, 将一个头结点为 A 的单链表 (其数据域为整数) 分解成两个单链表 A 和 B, 使得 A 链表只含有原来链表中 data 域为奇数的结点, 而 B 链表只含有原链表中 data 域为偶数的结点, 且保持原来的相对顺序。

【答案解析】

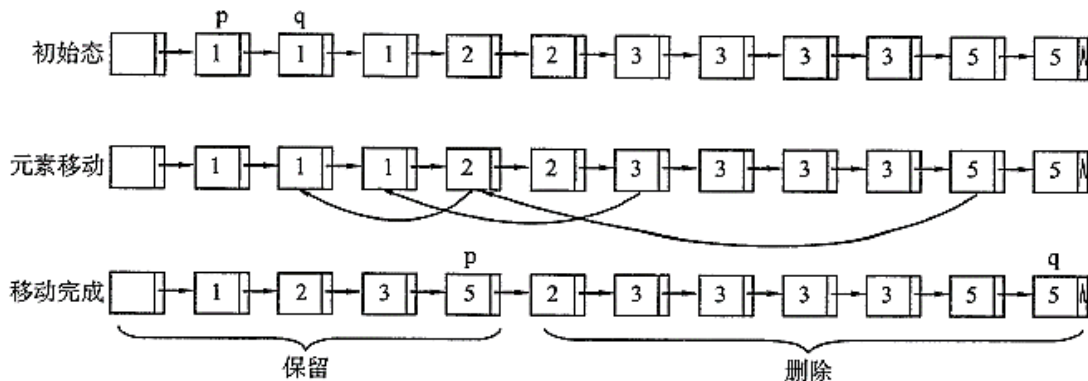
1. 答: 尾指针是指向终端结点的指针, 用它来表示单循环链表可以使得查找链表的开始结点和终端结点都很方便。设一个带头结点的单循环链表, 其尾指针是 rear, 则开始结点和终端结点分别为指针 rear 所指结点的后继结点的后继结点和指针 rear 所指的结点, 即 $rear \rightarrow next \rightarrow next$ 和 rear, 查找时间均为 $O(1)$ 。若用头指针来表示该链表, 则查找开始结点为 $O(1)$, 终端结点为 $O(n)$ 。

2.分析: 解法一: 定义指针 p 指向起始结点。将 p 所指的当前结点值域和其直接后继结点值域做比较。如果当前结点值域等于后继结点值域, 则删除后继结点; 否则 p 指向后继结点。重复以上过程, 直到 p 的后继结点为空。

本题代码如下:

```
void delsl1(LNode *L) {
    LNode *p=L->next,*q; // 初始化指针 p 为链表的第一个实际节点 (即头节点之后的节点)
    while (p->next!=NULL) { // 遍历链表, 直到 p 的下一个节点为空 (即到达链表末尾)
        if (p->data==p->next->data) { // 找到重复结点并删除
            q=p->next; // q 指向要删除的重复节点 (即 p 的下一个节点)
            p->next=q->next; // 将 p 的 next 指针指向 q 的下一个节点, 跳过 q
            free(q); // 释放被删除节点 q 占用的内存
        }
        else
            p=p->next; // 如果当前节点和下一个节点的数据不同, 则移动 p 到下一个节点
    }
}
```

解法二: 依次将原序列中每个连续相等子序列的第一个元素移动到表的前端, 将剩余的元素删除即可, 即如下图所示的过程。



令 p 指向起始结点。 q 从 p 的后继结点开始扫描, q 每来到一个新结点的时候进行检测: 当 $q->data$ 等于 $p->data$ 时, 什么也不做, q 继续往后走; 当两者不相等时, p 往后走一个位置, 然后用 $q->data$ 取代 $p->data$ 之后, q 继续往后扫描, 重复以上过程。当 q 为空时, 释放从 p 之后的所有结点空间。上述过程的实现代码如下:

```
void delsl2(LNode *L) {
    if (L == NULL || L->next == NULL) return; // 如果链表为空或只有一个节点, 直接返回
    LNode *p=L->next,*q=L->next->next,*r;
    while(q!=NULL) { // 遍历链表中的每个节点
        while (q!=NULL && q->data==p->data)
            q=q->next;
        if(q!=NULL) {
            p=p->next;
            p->data=q->data;
        }
        q=p->next;
        p->next=NULL;
        while(q!=NULL) {
            r=q;
            q=q->next;
            free(r);
        }
    }
}
```

}

3.分析: 用 p 从头至尾扫描链表, pre 指向 $*p$ 结点的前驱, 用 $minp$ 保存值最小的结点指针, $minpre$ 指向 $minp$ 的前驱。一边扫描, 一边比较, 将最小值结点放到 $minp$ 中。

本题代码如下:

```
void delminnode(LNode *L) {
    LNode *pre=L, *p=pre->next, *minp=p, *minpre=pre;
    while(p!=NULL) { //查找最小值结点 minp 以及前驱结点 minpre
        if (p->data < minp->data) {
            minp=p;
            minpre=pre;
        }
        pre=p;
        p=p->next;
    }
    minpre->next =minp->next; //删除*mi np 结点
    free (minp);
}
```

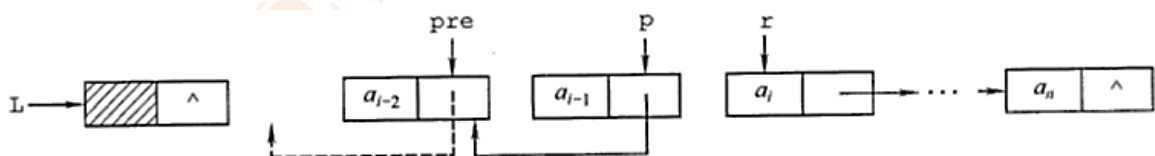
4.分析: 在前边讲过的算法基础中, 提到过关于逆序的问题, 那就是链表建立的头插法。头插法完成后, 链表中的元素顺序和原数组中元素的顺序相反。这里可以将 L 中的元素作为逆转后 L 的元素来源, 即将 $L \rightarrow next$ 设置为空, 然后将头结点后的一串结点用头插法逐个插入 L 中, 这样新的 L 中的元素顺序正好是逆序的。

本题代码如下:

```
void reversel(LNode *L) {
    LNode *p=L->next, *q;
    L->next=NULL;
    while(p!=NULL) { //p 结点始终指向旧的链表的开始结点
        q=p->next; //q 结点作为辅助结点来记录 p 的直接后继结点的位置
        p->next=L->next; //将 p 所指的结点插入新的链表中
        L->next=p;
        p=q; //因为后继结点已经存入 q 中, 所以 p 仍然可以找到后继
    } // (此时的新开始结点)
}
```

解法二: 大部分辅导书都只介绍解法一, 这对读者的理解和思维是不利的。为了将调整指针这个过程分析清楚, 我们借助图形来进行直观的分析。

假设 pre 、 p 和 r 指向 3 个相邻的结点, 如下图所示。假设经过若干操作后, $*pre$ 之前的结点的指针都已调整完毕, 它们的 $next$ 都指向其原前驱结点。现在令 $*p$ 结点的 $next$ 域指向 $*pre$ 结点, 注意到一旦调整指针的指向, $*p$ 的后继结点的链就会断开, 为此需要用 r 来指向原 $*p$ 的后继结点。处理时需要注意两点: 一是在处理第一个结点时, 应将其 $next$ 域置为 $NULL$, 而不是指向头结点 (因为它将作为新表的尾结点); 二是在处理完最后一个结点后, 需要将头结点的指针指向它。



本题代码如下:

```
LinkedList Reverse_2(LinkedList L) { // 依次遍历线性表 L, 并将结点指针反转
    LNode *pre, *p = L->next, *r = p->next;
```



```

pre = p; // 否则, pre 和 p 同步后移
p = p->next;
}
}
}

```

本算法是在无序单链表中删除满足某种条件的所有结点, 这里的条件是结点的值为 x 。实际上, 这个条件是可以任意指定的, 只要修改 if 条件即可。比如, 我们要求删除值介于 mink 和 maxk 之间的所有结点, 则只需将 if 语句修改为 $\text{if}(p \rightarrow \text{data} > \text{mink} \ \&\& \ p \rightarrow \text{data} < \text{maxk})$ 。

解法二: 采用尾插法建立单链表。用 p 指针扫描 L 的所有结点, 当其值不为 x 时, 将其链接到 L 之后, 否则将其释放。

本题代码如下:

```

void Del_X_2(Linklist &L, ElemType x){
//L 为带头结点的单链表, 本算法删除 L 中所有值为 x 的结点
    LNode *p=L->next, *r=L, *q;           //r 指向尾结点, 其初值为头结点
    while (p!=NULL) {
        if (p->data!=x) {                   // *p 结点值不为 x 时将其链接到 L 尾部
            r->next=p;
            r=p;
            p=p->next;                       //继续扫描
        }
        else{                               // *p 结点值为 x 时将其释放
            q=p;
            p=p->next;                       //继续扫描
            free(q);                         //释放空间
        }
    }
} //while
r->next=NULL;                             //插入结束后置尾结点指针为 NULL
}

```

上述两个算法扫描一遍链表, 时间复杂度为 $O(n)$, 空间复杂度为 $O(1)$ 。

7. 解答: 因为链表是无序的, 所以只能逐个结点进行检查, 执行删除。

本题代码如下:

```

void RangeDelete(LinkList &L, int min, int max) {
    LNode *pr = L, *p = L->link;          // p 是检测指针, pr 是其前驱
    while (p != NULL) {
        if (p->data > min && p->data < max) {
            pr->link = p->link;             // 删除 p 结点
            free(p);                       // 释放 p 结点的空间
            p = pr->link;                   // 更新 p 指针
        }
        else {                             // 否则继续寻找被删结点
            pr = p;
            p = p->link;
        }
    }
}

```

8. 解决思路为: 用指针 p 从头至尾扫描 A 链表, 当发现结点 data 域为偶数的结点则取下, 插入链表 B 中。因为题目要求保持原来数据元素的相对顺序, 所以要用尾插法来建立 B 链表。

本题代码如下:

```
void split2 (LNode *A,LNode *&B) {
    LNode *p,*q,*r;
    B=(LNode *) malloc (sizeof (LNode)); //申请链表 B 的头结点
    B->next=NULL; //每申请一个新结点的时候, 将其指针域 next 设置为 NULL
                //这样可以避免很多因链表的终端结点忘记置 NULL 而产生的错误

    r=B;
    p=A;
    while(p->next!=NULL){ //p 始终指向当前被判断结点的前驱结点, 这和删除结点类似
        //因为取下一个结点就是删除一个结点, 只是不释放这个结点的内存空间而已 */
        if (p->next->data%2 == 0 ) { //判断结点的 data 域是否为偶数, 是偶数则从链表中取下
            q=p->next; //q 指向要从链表中取下的结点
            p->next=q->next; //从链表中取下这个结点
            q->next=NULL;
            r->next=q;
            r=q;
        }
        else p=p->next; //p 后移一个位置, 即开始检查下一个结点
    }
}
```

3、栈

(一) 选择题

1. 栈和队列具有相同的()。
A. 抽象数据类型 B. 逻辑结构 C. 存储结构 D. 运算
2. 栈是()。
A. 顺序存储的线性结构 B. 链式存储的非线性结构
C. 限制存取点的线性结构 D. 限制存储点的非线性结构
3. ()不是栈的基本操作。
A. 删除栈顶元素 B. 删除栈底元素
C. 判断栈是否为空 D. 将栈置为空栈
4. 假定利用数组 $a[n]$ 顺序存储一个栈, 用 top 表示栈顶指针, 用 $top = -1$ 表示栈空, 并已知栈未
满, 当元素 x 进栈时所执行的操作为()。
A. $a[--top] = x$ B. $a[top--] = x$ C. $a[++top] = x$ D. $a[top++] = x$
5. 和顺序栈相比, 链栈有一个比较明显的优势, 即()。
A. 通常不会出现栈满的情况 B. 通常不会出现栈空的情况
C. 插入操作更容易实现 D. 删除操作更容易实现
6. 设链表不带头结点且所有操作均在表头进行, 则下列最不适合作为链栈的是()。
A. 只有表头结点指针, 没有表尾指针的双向循环链表
B. 只有表尾结点指针, 没有表头指针的双向循环链表
C. 只有表头结点指针, 没有表尾指针的单向循环链表
D. 只有表尾结点指针, 没有表头指针的单向循环链表
7. 栈操作数据的原则是()。
A. 先进先出 B. 后进先出 C. 后进后出 D. 不分顺序
8. 在做进栈运算时, 应先判别栈是否(①); 在做退栈运算时, 应先判别栈是否(②)。当栈中
元素为 n 个, 做进栈运算时发生上溢, 则说明该栈的最大容量为(③)。
①, ②: A. 空 B. 满 C. 上溢 D. 下溢
③: A. $n-1$ B. n C. $n+1$ D. $n/2$
9. 一个栈的输入序列为 $1, 2, 3, \dots, n$, 若输出序列的第一个元素是 n , 则输出序列的第 i ($1 \leq i \leq n$) 个元
素是()。
A. 不确定 B. $n-i+1$ C. i D. $n-i$
10. 若一个栈的输入序列为 $1, 2, 3, \dots, n$, 输出序列的第一个元素是 i , 则第 j 个输出元素是()。
A. $i-j-1$ B. $i-j$ C. $j-i+1$ D. 不确定的
11. 有 6 个元素以 $6, 5, 4, 3, 2, 1$ 的顺序进栈, ()不是合法的出栈序列。
A. $5, 4, 3, 6, 1, 2$ B. $4, 5, 3, 1, 2, 6$
C. $3, 4, 6, 5, 2, 1$ D. $2, 3, 4, 1, 5, 6$
12. 输入序列为 A, B, C , 当输出序列为 C, B, A 时, 经过的栈操作为()。
A. $push, pop, push, pop, push, pop$ B. $push, push, push, pop, pop, pop$
C. $push, push, pop, pop, push, pop$ D. $push, pop, push, push, pop, pop$
13. 若一个栈以向量 $V[1, \dots, n]$ 存储, 初始栈顶指针 top 为 $n+1$, 则 x 进栈的正确操作是()。
A. $top = top + 1; V[top] = x;$ B. $V[top] = x; top = top + 1;$

- C. $\text{top}=\text{top}-1; V[\text{top}]=x;$ D. $V[\text{top}]=x; \text{top}=\text{top}-1;$
14. 若栈采用顺序存储方式存储, 现两栈共享空间 $V[1, \dots, m]$, $\text{top}[i]$ 代表第 i ($i=1, 2$) 个栈的栈顶, 栈 1 的底在 $V[1]$, 栈 2 的底在 $V[m]$, 则栈满的条件是 ()。
- A. $|\text{top}[2]-\text{top}[1]|=0$ B. $\text{top}[1]+1=\text{top}[2]$
C. $\text{top}[1]+\text{top}[2]=m$ D. $\text{top}[1]=\text{top}[2]$
15. 栈在 () 中应用。
- A. 递归调用 B. 子程序调用 C. 表达式求值 D. A, B, C
16. 3 个不同元素依次进栈, 能得到 () 种不同的出栈序列。
- A. 4 B. 5 C. 6 D. 7
17. 设 a, b, c, d, e, f 以所给的次序进栈, 若在进栈操作时, 允许出栈操作, 则下面得不到的序列为 ()。
- A. fedcba B. bcafed C. dcefb a D. cabdef
18. 用 S 表示进栈操作, 用 X 表示出栈操作, 若元素的进栈顺序是 1234, 为了得到 1342 的出栈顺序, 相应的 S 和 X 的操作序列为 ()。
- A. $SXSXS:XX$ B. $SSSXXS:XX$ C. $SXSS:XXSX$ D. $SXSSXS:XX$
19. 若一个栈的输入序列是 $1, 2, 3, \dots, n$ 输出序列的第一个元素是 n , 则第 i 个输出元素是 ()。
- A. 不确定 B. $n-i$ C. $n-i-1$ D. $n-i+1$
20. 一个栈的输入序列为 $1, 2, 3, \dots, n$, 输出序列的第一个元素是 i , 则第 j 个输出元素是 ()。
- A. $i-j-1$ B. $i-j$ C. $j-i+1$ D. 不确定
21. 某栈的输入序列为 a, b, c, d , 下面的 4 个序列中, 不可能为其输出序列的是 ()。
- A. a, b, c, d B. c, b, d, a C. d, c, a, b D. a, c, b, d
22. 设栈的初始状态为空, 当字符序列 “n1_” 作为栈的输入时, 输出长度为 3, 且可用做 C 语言标识符的序列有 () 个。
- A. 4 B. 5 C. 3 D. 6
23. 采用共享栈的好处是 ()。
- A. 减少存取时间, 降低发生上溢的可能 B. 节省存储空间, 降低发生上溢的可能
C. 减少存取时间, 降低发生下溢的可能 D. 节省存储空间, 降低发生下溢的可能
24. 设有一个顺序共享栈 $\text{Share}[0:n-1]$, 其中第一个栈顶指针 top1 的初值为 -1, 第二个栈顶指针 top2 的初值为 n , 则判断共享栈满的条件是 ()。
- A. $\text{top2}-\text{top1}=1$ B. $\text{top1}-\text{top2}=1$ C. $\text{top1}=\text{top2}$ D. 以上都不对
25. 最不适合用作链栈的链表是 (以下链表没有头结点) ()。
- A. 只有表头指针、没有表尾指针的循环双链表
B. 只有表尾指针、没有表头指针的循环双链表
C. 只有表尾指针、没有表头指针的循环单链表
D. 只有表头指针、没有表尾指针的循环单链表
26. 如果以链表作为栈的存储结构, 则元素出栈的时候 ()。
- A. 必须判断链栈是否为满 B. 必须判断链栈元素的类型
C. 必须判断链栈是否为空 D. 无须做任何判断

【答案解析】

1. B。栈和队列的逻辑结构都是相同的, 都属于线性结构, 只是它们对数据的运算不同。
2. C。首先栈是一种线性表, 所以 B、D 错。按存储结构的不同可分为顺序栈和链栈, 但不可以把栈局限在某种存储结构上, 所以 A 错。栈和队列都是限制存取点的线性结构。

3. B。基本操作是指该结构最核心、最基本的运算,其他较复杂的操作可通过基本操作实现。删除栈底元素不属于栈的基本运算,但它可以通过调用栈的基本运算求得。

4. C。初始时 top 为 -1,则第一个元素入栈后, top 为 0,即指向栈顶元素,故入栈时应先将指针 top 加 1,再将元素入栈,只有选项 C 符合题意。

5. A。顺序栈采用数组存储,数组的大小是固定的,不能动态地分配大小。和顺序栈相比,链栈的最大优势在于它可以动态地分配存储空间,所以答案为 A。

6. C。对于双向循环链表,不管是表头指针还是表尾指针,都可以很方便地找到表头结点,方便在表头做插入或删除操作。而单循环链表通过尾指针可以很方便地找到表头结点,但通过头指针找尾结点则需要遍历一次链表。对于 C,插入和删除结点后,找尾结点需要花费 $O(n)$ 的时间。

7. B。本题考查栈的性质。栈的最大特点就是栈操作数据遵循后进先出的原则。

8. B, A, B。本题考查对栈概念的理解。

9. B。本题考查栈的性质。由栈的 FILO 性质可知,如果第一个出栈元素是 n ,则之前的 $1 \sim n-1$ 个元素都已经依次进栈,因此第 i 个出栈元素就是从第 n 个元素往前数 i 个元素,为 $n-i+1$ 。因此本题选 B。

10. D。本题考查栈的基本操作。假如输入序列为 1, 2, 3, 4, 5, 第一个出栈元素为 3, 如果进栈出栈操作为 IIIIOO II (I 代表进栈操作, O 代表出栈操作), 则第二个出栈元素为 2; 如果操作序列为 IIIIOIOL 则第二个出栈元素为 4。因此不确定。

说明: 本题的这个方法是解决数据结构选择题的一个十分有用的方法, 数据结构题目中有很多可以用特殊来推测一般, 本题就是令 n 等于 5, 举反例说明。

11. C。本题考查栈的基本操作。解这类题目, 就是按照题干所给序列去挨个检查选项。选项 C, 3 第一个出栈, 说明 6, 5, 4 都已进栈, 4 第二个出栈可以, 6 第三个出栈显然不可以, 因为如果 6 要出栈必须 5 已经出栈。其他选项自行检查, 都是可以出现的序列。因此本题选 C。

12. B。本题考查栈的基本操作。若要用栈使得输入序列 A, B, C 的输出序列为 C, B, A, 即将原序列逆序, 则只需将所有元素入栈, 然后全部出栈即可。

说明: 通过这个题目我们知道, 用栈可以实现原有序列的逆序输出, 这在很多程序设计题目中是有用处的。

13. C。本题考查栈基本概念的扩展。栈基本概念中规定 top 初始指针为 -1, 本题只不过是把 top 规定在了数组的另一端。因此进栈操作只需将栈基本算法中进栈操作的 $\text{top}++$ 改为 $\text{top}-$ 即可, 又因 top 初值在数组下标范围之外, 所以必须先变指针再进元素, 故本题选 C。

14. B。本题考查栈基本概念的扩展。两栈共享空间, 栈满的时候应该是两栈顶指针相遇的时候, 相遇但不能相等, 相等意味着两栈有共同的栈顶元素, 不合理。因此栈满条件为 $\text{top}_1+1=\text{top}_2$, 故本题选 B。

15. D。本题考查栈的应用。这里只说明一下 A 选项和 B 选项, 两者类似, A 是函数调用自己, B 是函数调用其他函数。只要调用函数, 就得在调用前保存现场, 在调用后恢复现场, 这就要用到栈 (前边讲过栈有记忆当前状态的功能)。因此本题选 D。

16. B。对于 n 个不同元素进栈, 出栈序列的个数为:

$$\frac{1}{n+1} C_{2n}^n = \frac{1}{n+1} \frac{(2n)!}{n! \times n!} = \frac{6 \times 5 \times 4}{4 \times 3 \times 2 \times 1} = 5$$

考题中给出的 n 值不会很大, 可以根据栈的特点, 若 X 已经出栈, 则 x 前面的尚未出栈的元素一定逆置有序地出栈, 因此可采用例举方法。如 a, b, c 依次进栈的出栈序列有 abc, acb, bac, bca, cba。另外, 在一些考题中可能会问符合某个特定条件的出栈序列有多少种, 比如此题中的问以 b 开头的出

栈序列有几种, 这种类型的题目一般都使用穷举法。

17. D。根据栈“先进后出”的特点, 且在进栈操作的同时允许出栈操作, 显然答案 D 中 c 最先出栈, 则此时栈内必定为 a 和 b, 但由于 a 先于 b 进栈, 故要晚出栈。对于某个出栈的元素, 在它之前进栈却晚出栈的元素必定是按逆序出栈的, 其余答案均是可能出现的情况。

此题也可采用将各序列逐个代入的方法来确定是否有对应的进出栈序列 (类似下题)。

18. D。采用排除法, 选项 A, B, C 得到的出栈序列分别为 1243, 3241, 1324。由 1234 得到 1342 的进出栈序列为: 1 进, 1 出, 2 进, 3 进, 3 出, 4 进, 4 出, 2 出, 故选 D。

19. D。第 n 个元素第一个出栈, 说明前 n - 1 个元素都已经按顺序入栈, 由“先进后出”的特点可知, 此时的输出序列一定是输入序列的逆序, 故答案选 D。

20. D。当第 i 个元素第一个出栈时, 则 i 之前的元素可以依次排在 i 之后出栈, 但剩余的元素可以在此时进栈并且也会排在 i 之前的元素出栈, 所以第 j 个出栈的元素是不确定的。

21. C。对于 A, 可能的顺序是 a 入, a 出, b 入, b 出, c 入, c 出, d 入, d 出。对于 B, 可能的顺序是 a 入, b 入, c 入, c 出, b 出, d 入, d 出, a 出。对于 D, 可能的顺序是 a 入, a 出, b 入, c 入, c 出, b 出, d 入, d 出。C 没有对应的序列。

另解: 若出栈序列的第一个元素为 d, 则出栈序列只能是 dcba。该思想通常也适用于出栈序列的局部分析: 如 12345 入栈, 问出栈序列 34152 是否正确? 如何分析? 若第一个出栈元素是 3, 则此时 12 必停留在栈中, 它们出栈的相对顺序只能是 21, 故 34152 错误。

22. C。标识符的第一个字符必须是大小写英文字母或下划线, 不能是数字。按照上述规定 $nl_$ 三个字符符合规定的标识符有 $nl_$, n_l , $ln_$, nl 四种形式。第一种: n 进栈再出栈, l 进栈再出栈, $_$ 进栈再出栈。第二种: n 进栈再出栈, l 进栈, $_$ 进栈, $_$ 出栈, l 出栈。第三种: n 进栈, l 进栈, $_$ 进栈, $_$ 出栈, l 出栈, n 出栈。而根据栈的操作特性, nl 这种情况不可能出现, 故选 C。

23. B。存取栈中的元素都只需要 $O(1)$ 的时间, 所以减少存取时间无从谈起。另外, 栈的插入和删除操作都是在栈顶进行的, 只可能发生上溢 (栈顶指针超出了最大范围), 因此本题答案为 B。

24. A。这种情况就是前面我们所描述的, 详细内容请参见本节考点精析部分对共享栈的讲解。另外, 读者可以思考若 $top1$ 的初值为 0, $top2$ 的初值为 $n-1$ 时栈满的条件。

注意: 栈顶、队头与队尾的指针的定义是不唯一的, 读者务必要仔细审题。

25. D。本题考查链栈的存储结构的选取。链栈在链表的头部进行操作 (插入和删除结点), 需要找到链表开始结点的前驱结点。只有表头指针、没有表尾指针的循环单链表, 不方便查找开始结点的前驱结点 (查找时间复杂度为 $O(n)$), 因此本题选 D。

26. C。本题考查栈的基本操作。若栈空则无法进行出栈操作, 因此本题选 C。

(二) 综合应用题

1. 有 5 个元素, 其入栈次序为 A, B, C, D, E, 在各种可能的出栈次序中, 第一个出栈元素为 C 且第二个出栈元素为 D 的出栈序列有几个?

2. 若元素的进栈序列为 A, B, C, D, E, 运用栈操作, 能否得到出栈序列 B, C, A, E, D 和 D, B, A, C, E? 为什么?

3. 假设以 1 和 0 分别表示入栈和出栈操作。栈的初态和终态均为空, 入栈和出栈的操作序列可表示为仅由 1 和 0 组成的序列, 可以操作的序列称为合法序列, 否则称为非法序列。

(1) 下面所示的序列中哪些是合法的?

- A. IOIOIOO B. IOOIOHO C. IIIIOIOO D. IIIOOIOO

(2) 通过对 1) 的分析, 写出一个算法, 判定所给的操作序列是否合法。若合法, 返回 `true`, 否则返回 `false` (假定被判定的操作序列已存入一维数组中)。

4. 铁路进行列车调度时, 常把站台设计成栈式结构, 试问:

(1) 设有编号为 1, 2, 3, 4, 5, 6 的 6 辆列车, 顺序开入栈式结构的站台, 则可能的出栈序列有多少种?

(2) 若进站的 6 辆列车顺序如上所述, 那么是否能够得到 435612、326541、154623 和 135426 的出站序列? 如果不能, 说明为什么不能; 如果能, 说明如何得到 (写出进栈或出栈的序列)。

5. 试证明: 若借助栈可由输入序列 $1, 2, 3, \dots, n$ 得到一个输出序列 $P_1, P_2, P_3, \dots, P_n$ (它是输入序列的某一种排列), 则在输出序列中不可能出现以下情况: 存在 $i < j < k$, 使得 $P_j < P_k < P_i$ (提示: 用反证法)。

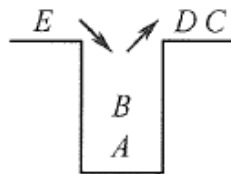
6. 有 5 个元素, 其入栈次序为 A, B, C, D, E, 在各种可能的出栈次序中, 以元素 C, D 最先出栈 (C 第一个且 D 第二个出栈) 的次序有哪几个?

【答案解析】

1. CD 出栈后的状态如图所示。

此时有如下 3 种操作: ① E 进栈后出栈, 出栈序列为 CDEBA; ② B 出栈, E 进栈后出栈, 出栈序列为 CDBEA; ③ B 出栈, A 出栈, E 进栈后出栈, 出栈序列为 CDBAE。

所以, 以 CD 开头的出栈序列有 CDEBA、CDBEA、CDBAE 三种。



2. 解答: 能得到出栈序列 BCAED。可由 A 进, B 进, B 出, C 进, C 出, A 出, D 进, E 进, E 出, D 出得到。不能得到出栈序列 DBACE。若出栈序列以 D 开头, 说明在 D 之前的入栈元素是 A、B 和 C, 三个元素中 C 是栈顶元素, B 和 A 不可能早于 C 出栈, 故不可能得到出栈序列 DBACE。

3. 解答: (1) A、D 合法, 而 B、C 不合法。在 B 中, 先入栈 1 次, 再连续出栈 2 次, 错误。在 C 中, 入栈和出栈次数不一致, 会导致最终的栈不空。A、D 均为合法序列, 请自行模拟。注意: 在操作过程中, 入栈次数一定大于等于出栈次数; 结束时, 栈一定为空。

(2) 设被判定的操作序列已存入一维数组 A 中。算法的基本设计思想: 依次逐一扫描入栈出栈序列 (即由 “I” 和 “O” 组成的字符串), 每扫描至任一位置均需检查出栈次数 (即 “O” 的个数) 是否小于入栈次数 (“I” 的个数), 若大于则为非法序列。扫描结束后, 再判断入栈和出栈次数是否相等, 若不相等则不合题意, 为非法序列。

```
int Judge(char A[]) {
    // 判断字符串 A 中的输入输出序列是否是合法序列。如是, 返回 true, 否则返回 false
    int i = 0; // i 为下标
    int j = 0, k = 0; // j 和 k 分别为字母 I 和 O 的个数
    while (A[i] != '\0') { // 未到字符串尾
        switch (A[i]) {
            case 'I': j++; break; // 入栈次数增 1
```



```

        case 'O': k++;
        if (k > j) {
            printf("序列非法\n");
            exit(0);
        }
        i++;           // 不论 A[i] 是 "I" 或 "O", 指针 i 均后移
    }
    if (j != k) {
        printf("序列非法\n");
        return false;
    }
    else {
        printf("序列合法\n");
        return true;
    }
}

```

另解：入栈后，栈内元素个数加 1；出栈后，栈内元素个数减 1，因此可将判定一组出入栈序列是否合法转化为一组由 +1、-1 组成的序列，它的任意前缀子序列的累加和不小于 0 (每次出栈或入栈操作后判断) 则合法；否则非法。

4. (1) 编号为 1, 2, 3, 4, 5, 6 的 6 辆列车顺序开入栈式结构站台，可能的不同出栈序列有

$$\frac{1}{n+1} C_{2n}^n = \frac{1}{6+1} C_{12}^6 = 132$$

(2) 不能得到 435612 和 154623 这样的出栈序列。因为若在 4, 3, 5, 6 之后再将 1, 2 出栈，则 1, 2 必须一直在栈中，此时 1 先进栈，2 后进栈，2 应在 1 的上面，不可能 1 先于 2 出栈。154623 也是这种情况。出栈序列 326541 和 135426 可以得到。

5. 证明：(1) 必要性。按照题意，当 $i < j < k$ 时，进栈顺序是 i, j, k ，这 3 个元素出栈的相对顺序是 P_i, P_j, P_k ，例如，当 $i=1, j=2, k=3$ 时，一个合理的出栈序列是 $P_i=2, P_j=3, P_k=1$ 。如果 $P_j < P_k < P_i$ 成立，意味着出栈顺序为 3、1、2，这恰恰是不可能的。当较大的数首先出栈时，那些较小的数都是降序压在栈内的，如 2、1，这些数不可能如 1、2 那样正序出栈。

(2) 充分性。如果 $P_j < P_k < P_i$ 成立，表明当 $i < j < k$ 时各元素进栈、出栈后的相对顺序为 P_j, P_k, P_i 。下面做具体分析。

当 $i < j$ 时， $P_j < P_i$ ，表明 P_j 在 P_i 进栈后进栈并压在 P_i 上面，并且 P_j 在 P_i 出栈前出栈。当 $j < k$ 时， $P_j < P_k$ ，表明 P_j 必须在 P_k 进栈之前就出栈，否则 P_j 就被压在 P_k 下面了。当 $i < k$ 时， $P_k < P_i$ ，表明 P_i 是先于 P_k 进栈的。

综上所述可知，这与正确的出栈顺序 $P_i < P_j < P_k$ 相矛盾。

6. 分析：C, D 最先出栈，并且 C 先于 D，则 A, B 的相对顺序已经确定，必为 B 先于 A。而 E 的位置可以在 B 前、B 与 A 中间或者 A 后。因此所得序列有 3 种，具体如下：

C, D, E, B, A; C, D, B, E, A; C, D, B, A, E

4、队列

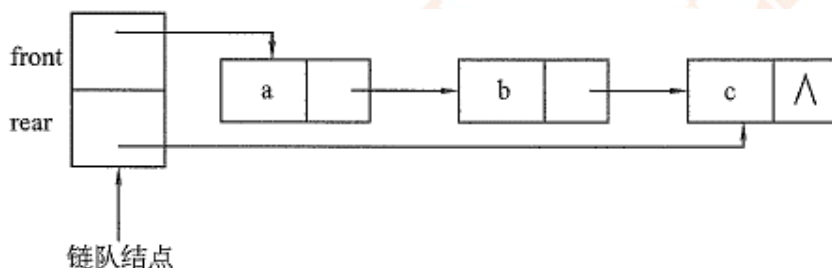
(一) 选择题

1. 栈和队列的主要区别在于 ()。
A. 它们的逻辑结构不一样 B. 它们的存储结构不一样
C. 所包含的元素不一样 D. 插入、删除操作的限定不一样
2. 队列的“先进先出”特性是指 ()。
I. 最后插入队列中的元素总是最后被删除; II. 当同时进行插入、删除操作时, 总是插入操作优先
III. 每当有删除操作时, 总要先做一次插入操作; IV. 每次从队列中删除的总是最早插入的元素
A. I B. I 和 IV C. II 和 III D. IV
3. 允许对队列进行的操作有 ()。
A. 对队列中的元素排序 B. 取出最近进队的元素
C. 在队列元素之间插入元素 D. 删除队头元素
4. 一个队列的入队顺序是 1, 2, 3, 4, 则出队的输出顺序是 ()。
A. 4, 3, 2, 1 B. 1, 2, 3, 4 C. 1, 4, 3, 2 D. 3, 2, 4, 1
5. 已知循环队列的存储空间为数组 $A[21]$, $front$ 指向队头元素的前一个位置, $rear$ 指向队尾元素, 假设当前 $front$ 和 $rear$ 的值分别为 8 和 3, 则该队列的长度为 ()。
A. 5 B. 6 C. 16 D. 17
6. 若用数组 $A[0..5]$ 来实现循环队列, 且当前 $rear$ 和 $front$ 的值分别为 1 和 5, 当从队列中删除一个元素, 再加入两个元素后, $rear$ 和 $front$ 的值分别为 ()。
A. 3 和 4 B. 3 和 0 C. 5 和 0 D. 5 和 1
7. 用链式存储方式的队列进行删除操作时需要 ()。
A. 仅修改头指针 B. 仅修改尾指针 C. 头尾指针都要修改 D. 头尾指针可能都要修改
8. 对于链队, 在进行出队操作时 ()。
A. 仅修改头指针 B. 仅修改尾指针
C. 头、尾指针都要修改 D. 头、尾指针可能都要修改
9. 用不带头结点的单链表存储队列时, 其队头指针指向队头结点, 其队尾指针指向队尾结点, 则在进行出队操作时 ()。
A. 仅修改队头指针 B. 仅修改队尾指针
C. 队头、队尾指针都要修改 D. 队头、队尾指针都可能要修改
10. 递归过程或函数调用时, 处理参数及返回地址要用一种称为 () 的数据结构。
A. 队列 B. 多维数组 C. 栈 D. 线性表
11. 若用一个大小为 6 的数组来实现循环队列, 并且当前 $rear$ 和 $front$ 的值分别为 0 和 3, 当从队列中删除一个元素, 再加入两个元素后, $rear$ 和 $front$ 的值分别为 ()。
A. 1 和 5 B. 2 和 4 C. 4 和 2 D. 5 和 1
12. 栈和队列的共同点是 ()。
A. 都是先进先出 B. 都是先进后出
C. 只允许在端点处插入和删除元素 D. 没有共同点
13. 栈和队都是 ()。
A. 顺序存储的线性结构 B. 链式存储的非线性结构
C. 限制存取点的线性结构 D. 限制存取点的非线性结构

14. 一个队列的入队序列为 1,2,3,4, 则可能的出队序列为 ()。
- A. 4, 3, 2, 1 B. 1, 2, 3, 4
C. 1, 4, 3, 2 D. 3, 2, 4, 1
15. 最适合用作链队的链表 (链表有头结点, 有队首指针则指向头结点, 有队尾指针则指向终端结点) 是 ()。
- A. 只带队首指针的循环单链表 B. 只带队尾指针的循环单链表
C. 只带队首指针的非循环单链表 D. 只带队尾指针的非循环单链表
16. 最不适合用作队列的链表是 ()。
- A. 只带队首指针的非循环双链表 B. 只带队首指针的循环双链表
C. 只带队尾指针的循环双链表 D. 只带队尾指针的循环单链表
17. (多选) 用单链表 (含有头结点) 表示的队列的队头在链表的 () 位置。
- A. 链头 B. 链尾 C. 链中 D. 以上都可以
18. 用单链表 (含有头结点) 表示的链队的队尾在链表的 () 位置。
- A. 链头 B. 链尾 C. 链中 D. 以上都可以

【答案解析】

- D。栈和队列的逻辑结构都是线性结构, 都可以采用顺序存储或链式存储, C 显然也错误。只有 D 才是栈和队列的本质区别, 限定表中插入和删除操作位置的不同。
- B。队列“先进先出”的特性表现在: 先进队列的元素先出队列, 后进队列的元素后出队列, 进队列对应的是插入操作, 出队列对应的是删除操作。I 和 IV 均正确。
- D。删除队头元素即出队, 是队列的基本操作之一, 故选 D。
- B。队列的入队顺序和出队顺序是一致的, 这是和栈不同的。
- C。队列的长度为 $(\text{rear} - \text{front} + \text{maxsize}) \% \text{maxsize} = (\text{rear} - \text{front} + 21) \% 21 = 16$ 。这种情况和 front 指向当前元素, rear 指向队尾元素的下一个元素的计算是相同的。
- B。循环队列中, 每删除一个元素, 队首指针 $\text{front} = (\text{front} + 1) \% 6$, 每插入一个元素, 队尾指针 $\text{rear} = (\text{rear} + 1) \% 6$ 。上述操作后, $\text{front} = 0, \text{rear} = 3$ 。
- D。队列用链式存储时, 删除元素从表头删除, 通常仅需修改头指针, 但若队列中仅有一个元素, 则尾指针也需要被修改, 当仅有一个元素时, 删除后队列为空, 需修改尾指针为 $\text{rear} = \text{front}$ 。
- D。本题考查链队的基本操作及其存储结构。下图所示为链队示意图。



由图可知, 如果出队一个或者两个元素, 则只需要修改头指针; 如果 3 个元素全部出队, 则头尾指针都变成 NULL, 即头尾指针都需要修改。因此本题选 D。

9. D。本题又是一个基本知识的扩展。用单链表来表示队列, 当队列中有多个元素的时候, 出队操作只改变头指针即可; 当队列中只有一个元素时, 头尾指针都指向这个元素, 出队的话显然两指针都要置 NULL。本题与 12 题类似, 因此本题选 D。

10. C。本题考查栈的应用。函数的递归调用过程要用到系统栈。

11. B。本题考查循环队列的基本操作。在循环队列中入队执行 $\text{rear}=(\text{rear}+1)\%\text{maxSize};$ ，出队执行 $\text{front}=(\text{front}+1)\%\text{maxSize};$ ，将本题数据代入其中，可以算出 rear 为 2， front 为 4，因此答案选 B。

12. C。本题考查栈和队列的比较。两者共同点即为只允许在端点处插入和删除元素。

13. C。本题考查栈和队列的概念与比较。栈和队列都是限制存取点的线性结构。

14. B。本题考查队列的性质。队列先进先出，由入队序列 1, 2, 3, 4 知出队序列必为 1, 2, 3, 4，因此本题选 B。

15. B。本题考查链表的操作及其在队列上的应用。

对于 A 选项，只带队首指针的循环单链表，入队操作需要找到队尾指针，时间复杂度为 $O(n)$ ；出队操作时间复杂度为 $O(1)$ 。

对于 B 选项，入队操作和出队操作时间复杂度都是 $O(1)$ 。因知道终端结点的位置，入队直接插入即可；出队时，只需队尾指针后移一位即可找到头结点，然后直接删除即可。

对于 C 选项，同 A 选项。

对于 D 选项，只有队尾指针，又是非循环链表，所以无法找到头结点，因此不能完成出队操作。

综上所述，本题应选 B。

16. A。本题考查链表的操作及其在队列上的应用。只带队首指针的非循环双链表在入队操作时的时间复杂度为 $O(n)$ ，而其他选项中，入队、出队操作的时间复杂度都是 $O(1)$ 。因此本题选 A。

17. A、C。本题考查链表在队列上的应用。当带有头结点的链表用作队列的存储结构时，链表的头结点不存储信息，链表的开始结点存有队头信息。因此，当队列中的元素多于 1 个时，队头在链表的链中位置（链中位置是指除去表头结点和表尾结点以外的结点位置）；当队列中的元素只有 1 个时，队头结点在链表的链尾位置。因此本题选 A、C。

18. B。本题考查链表在队列上的应用。当带有头结点的链表用作队列的存储结构时，链表的链尾即为队列的队尾，因此本题选 B。

（二）综合应用题

1. 如果允许在循环队列的两端都可以进行插入和删除操作，要求：

(1) 写出循环队列的类型定义。

(2) 分别写出从队尾删除和从队头插入的算法。

2. 设计一个循环队列，用 front 和 rear 分别作为队头和队尾指针，另外用一个标志 tag 表示队列是空还是非空，约定当 tag 为 0 时队空，当 tag 为 1 时队非空，这样就可以用 $\text{front}==\text{rear}$ 作为队满的条件。要求，设计队列的结构和相关基本运算算法（队列元素为 int 型）。

3. Q 是一个队列， S 是一个空栈，实现将队列中的元素逆置的算法。

【答案解析】

1. 分析：用一维数组 $\text{data}[0, \dots, \text{maxSize}-1]$ 实现循环队列，其中 maxSize 是队列长度。设置队头指针 front 和队尾指针 rear ，约定 front 指向队头元素的前一位置， rear 指向队尾元素。定义满足 $\text{front}==\text{rear}$ 时为队空。从队尾删除元素，则 rear 向着下标减小的方向行走；从队头插入元素， front

同样向着下标减小的方向行走。因此, 当满足 $\text{rear} == (\text{front} - 1 + \text{maxSize}) \% \text{maxSize}$ 时队满。

(1) 队列的结构体定义:

```
typedef struct{
    int data [max Size];           //假设 maxSize 为已定义的常量
    int front,rear ;
} cycqueue;
```

(2) 算法实现。

①出队算法:

```
int deQueue(cycqueue &Q, int &x ) {
    /*本算法实现从队尾删除, 若删除成功, 用 x 接纳删除元素, 返回 1, 否则返回 0*/
    if(Q.front==Q.rear)           //队空无法出队, 返回 0
        return 0;
    else{
        x=Q.data[Q.rear];
        Q.rear=(Q.rear-1+maxSize)%maxSize; //修改队尾指针
        return 1;                      //出队成功, 返回 1
    }
}
```

②入队算法:

```
int enQueue(cycqueue &Q,int x) {           /*本算法实现从队头插入元素 x*/
    if (Q.rear == (Q. front-1+maxSize)%maxSize) //队满
        return 0;
    else{
        Q. da t a [ Q. front] =x ;           // x 入队列
        Q. front= (Q.front-1+maxSize) %maxSize; //修改队头指针
        return 1;
    }
}
```

说明: 本题算法中用到了一个操作: $\text{Q. front} = (\text{Q. front} - 1 + \text{maxSize}) \% \text{maxSize};$, 如果把这一句放在一个循环中, front 指针则沿着 $\text{maxSize}-1, \text{maxSize}-2, \dots, 2, 1, 0, \text{maxSize}-1, \text{maxSize}-2 \dots$ 的无限循环数行走, 这个操作和 $\text{Q. front} = (\text{Q. front} + 1) \% \text{maxSize};$ 实现的效果正好相反。

2.分析: 本题为循环队列基本算法操作的扩展。在队列结构体定义中加入 tag, 用 tag 判断队列是否为空, 用 $\text{front} == \text{rear}$ 判断是否队满。具体过程如下:

(1) 队列的结构体定义:

```
typedef struct{
    int data [ max Size];
    int front,rear;
    int tag ;
} Queue;
```

定义一个队列:

Queue qu;

(2) 队列的各要素:

```
qu. tag = 0; qu. front =qu. rear ;           //初始时
qu . front==qu.rear&&qu.tag==0             //队空条件
qu. f ront==qu. rear&&qu.tag==1            //队满条件
```

(3) 算法实现:

```
void initQueue (Queue &qu) {                //初始化队列
    qu . front =qu.rear=0;
    qu.tag=0;
}
```

```

int isEmptyQueue(Queue qu) {           //判断队是否为空
    if (qu.front==qu.rear&&qu.tag==0)
        return 1;
    else
        return 0;
}
int QueueFull (Queue qu) {             //判断是否队满
    if (qu.tag==1&&qu.front==qu.rear)
        return 1;
    else
        return 0;
}
int enQueue (Queue &qu, int x) {        //元素进队
    if(QueueFull(qu)==1)
        return 0;
    else{
        qu.rear=(qu.rear+1)%maxSize;
        qu.data[qu.rear]=x;
        qu.tag=1;                       //只要进队就把 tag 设置为 1
        return 1;
    }
}
int deQueue (Queue &qu, int &x ) {       //元素出队
    if(isEmptyQueue(qu)==1)
        return 0;
    else{
        qu.front= (qu.front+1)%maxSize;
        x=qu.data[qu.front];
        qu.tag=0;                       //只要有元素出队, 就把 tag 设置为 0
        return 1;
    }
}

```

说明: 对于 tag 值的设置, 初始时一定为 0, 插入成功后应设置为 1, 删除成功后应设置为 0。因为只有插入操作后, 队列才有可能满, 在删除操作后, 队列才有可能空。tag 的值再配合上 front==rear 这一句的判断就能正确区分队满与队空。

3. 解答: 本题主要考查大家对队列和栈的特性与操作的理解, 由于对队列的一系列操作不可能将其中的元素逆置, 而栈可以将入栈的元素逆序提取出来, 因此我们可以让队列中的元素逐个地出队列, 入栈; 全部入栈后再逐个出栈, 入队列。

算法的实现如下:

```

void Inverser(Stack &S, Queue &Q) {    // 本算法实现将队列中的元素逆置
    while (!QueueEmpty(Q)) {
        x = DeQueue(Q);                 // 队列中全部元素依次出队
        Push(S, x);                     // 元素依次入栈
    }
    while (!StackEmpty(S)) {
        Pop(S, x);                       // 栈中全部元素依次出栈
        EnQueue(Q, x);                   // 再入队
    }
}

```

5、树

5.1 树的概念

(一) 选择题

1. 树最适合用来表示 () 的数据。
A. 有序 B. 无序
C. 任意元素之间具有多种联系 D. 元素之间具有分支层次关系
2. 一棵有 n 个结点的树的所有结点的度数之和为 ()。
A. $n-1$ B. n C. $n+1$ D. $2n$
3. 树的路径长度是从树根到每个结点的路径长度的 ()。
A. 总和 B. 最小值 C. 最大值 D. 平均值
4. 对于一棵具有 n 个结点、度为 4 的树来说, ()。
A. 树的高度至多是 $n-3$ B. 树的高度至多是 $n-4$
C. 第 i 层上至多有 $4(i-1)$ 个结点 D. 至少在某一层上正好有 4 个结点
5. 度为 4 高度为 h 的树, ()。
A. 至少有 $h+3$ 个结点 B. 至多有 $4h-1$ 个结点
C. 至多有 $4h$ 个结点 D. 至少有 $h+4$ 个结点
6. 假定一棵度为 3 的树中, 结点数为 50, 则其最小高度为 ()。
A. 3 B. 4 C. 5 D. 6

【答案解析】

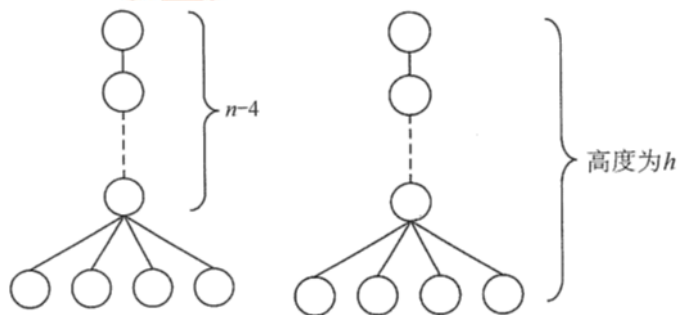
1. D。树是一种分层结构, 它特别适合组织那些具有分支层次关系的数据。
2. A。除根结点外, 其他每个结点都是某个结点的孩子, 因此树中所有结点的度数加 1 等于结点数, 也即所有结点的度数之和等于总结点数减 1。这是一个重要的结论, 做题时经常用到。
3. A。树的路径长度是指树根到每个结点的路径长的总和, 根到每个结点的路径长度的最大值应是树的高度减 1。注意与哈夫曼树的带权路径长度相区别。
4. A。要使得具有 n 个结点、度为 4 的树的高度最大, 就要使得每层的结点数尽可能少, 类似右图所示的树, 除最后一层外, 每层的结点数是 1, 最终该树的高度为 $n-3$ 。树的度为 4 只能说明存在某结点正好 (也最多) 有 4 个孩子结点, D 错误。

5. A。要使得度为 4、高度为 h 的树的总结点数最少, 需要满足以下两个条件:

- ①至少有一个结点有 4 个分支。
- ②每层的结数目尽可能少。

情况类以下图所示的树, 结点个数为 $h+3$ 。

要使得度为 4、高度为 h 的树的总结点数最多, 应使每个非叶结点的度均为 4, 即为满树, 总结点个数最多为 $1+4+4^2+\dots+4^{h-1}$ 。



6. C。要求满足条件的树, 那么该树是一棵完全三叉树。在度为 3 的完全三叉树中, 第 1 层有 1 个结点, 第 2 层有 $3^1=3$ 个结点, 第 3 层有 $3^2=9$ 个结点, 第 4 层有 $3^3=27$ 个结点, 因此结点数之和为 $1+3+9+27=40$, 第 5 层的结点数 $=50-40=10$ 个, 因此最小高度为 5。

(二) 综合应用题

1. 含有 n 个结点的三叉树的最小高度是多少?
2. 已知一棵度为 4 的树中, 度为 0, 1, 2, 3 的结点数分别为 14, 4, 3, 2, 求该树的结点总数 n 和度为 4 的结点个数, 并给出推导过程。

【答案解析】

1. 解答: 要求含有 n 个结点的三叉树的最小高度, 那么满足条件的一定是一棵完全三叉树, 设含有 n 个结点的完全三叉树的高度为 h , 第 h 层至少有 1 个结点, 至多有 3^{h-1} 个结点。则有:

$$1 + 3^1 + 3^2 + \dots + 3^{h-2} < n \leq 1 + 3^1 + 3^2 + \dots + 3^{h-2} + 3^{h-1}$$

即 $(3^{h-1} - 1)/2 < n \leq (3^h - 1)/2$, 得 $3^{h-1} < 2n + 1 \leq 3^h$, 也即 $h < \log_3(2n + 1) + 1$, $h \geq \log_3(2n + 1)$ 。

由于 h 只能为正整数, $h = \lceil \log_3(2n + 1) \rceil$, 故这样的三叉树的最小高度是 $\lceil \log_3(2n + 1) \rceil$ 。

2. 解答: 设树度为 i ($i=0, 1, 2, 3, 4$) 的结点数为 n_i , 那么结点总数 $n = n_0 + n_1 + n_2 + n_3 + n_4$, 即 $n = 23 + n_4$, 根据“树中所有结点的度数加 1 等于结点数”的结论, 有 $n = 0 + n_1 + 2n_2 + 3n_3 + 4n_4 + 1$, 即有 $n = 17 + 4n_4$ 。

综合两式得 $n_4 = 2$, $n = 25$ 。所以该树的结点总数为 25, 度为 4 的结点个数为 2。

5.2 二叉树的概念

(一) 选择题

1. 下列关于二叉树的说法中, 正确的是 ()。
 - A. 度为 2 的有序树就是二叉树
 - B. 含有 n 个结点的二叉树的高度为 $\lfloor \log_2 n \rfloor + 1$
 - C. 在完全二叉树中, 若一个结点没有左孩子, 则它必是叶结点
 - D. 在任意一棵非空二叉排序树中, 删除某结点后又将其插入, 则所得二叉排序树与删除前原二叉排序树相同
2. 以下说法中, 正确的是 ()。
 - A. 在完全二叉树中, 叶子结点的双亲的左兄弟 (若存在) 一定不是叶子结点
 - B. 任何一棵二叉树, 叶子结点个数为度为 2 的结点数减 1, 即 $n_0 = n_2 - 1$
 - C. 完全二叉树不适合顺序存储结构, 只有满二叉树适合顺序存储结构
 - D. 结点按完全二叉树层序编号的二叉树中, 第 i 个结点的左孩子的编号为 $2i$
3. 具有 10 个叶子结点的二叉树中有 () 个度为 2 的结点。
 - A. 8
 - B. 9
 - C. 10
 - D. 11
4. 按照二叉树的定义, 具有 3 个结点的二叉树有 () 种。
 - A. 3
 - B. 4
 - C. 5
 - D. 6
5. 有 10 个叶子结点的二叉树中有 () 个度为 2 的结点。
 - A. 8
 - B. 9
 - C. 10
 - D. 11
6. 如果一棵二叉树有 10 个度为 2 的结点, 5 个度为 1 的结点, 则度为 0 的结点个数为 ()。
 - A. 9
 - B. 11
 - C. 15
 - D. 不确定
7. 一棵完全二叉树上有 1001 个结点, 其中叶子结点的个数是 ()。
 - A. 250
 - B. 500
 - C. 505
 - D. 501

8. 一棵高度为 4 的完全二叉树至少有 () 个结点。
A. 15 B. 7 C. 8 D. 16
9. 一棵高度为 5 的完全二叉树至多有 () 个结点。
A. 16 B. 32 C. 31 D. 10
10. 假设高度为 h 的二叉树上只有度为 0 和度为 2 的结点, 则此类二叉树中所包含的结点数至少为 ()。
A. $2h$ B. $2h-1$ C. $2h+1$ D. $h+1$
11. 假设一棵二叉树的结点个数为 50, 则它的最小高度是 ()。
A. 4 B. 5 C. 6 D. 7
12. 设二叉树有 $2n$ 个结点, 且 $m < n$, 则不可能存在 () 的结点。
A. n 个度为 0 B. $2m$ 个度为 0 C. $2m$ 个度为 1 D. $2m$ 个度为 2
13. 设二叉树只有度为 0 和 2 的结点, 其结点个数为 15, 则该二叉树的最大深度为 ()。
A. 4 B. 5 C. 8 D. 9
14. 已知一棵完全二叉树的第 6 层 (设根为第 1 层) 有 8 个叶结点, 则完全二叉树的结点个数最少是 ()。
A. 39 B. 52 C. 111 D. 119
15. 若一棵深度为 6 的完全二叉树的第 6 层有 3 个叶子结点, 则该二叉树共有 () 个叶子结点。
A. 17 B. 18 C. 19 D. 20
16. 若一棵二叉树有 126 个结点, 在第 7 层 (根结点在第 1 层) 至多有 () 个结点。
A. 32 B. 64 C. 63 D. 不存在第 7 层
17. 一棵有 124 个叶子结点的完全二叉树, 最多有 () 个结点。
A. 247 B. 248 C. 249 D. 250
18. 一棵有 n 个结点的二叉树采用二叉链存储结点, 其中空指针数为 ()。
A. n B. $n+1$ C. $n-1$ D. $2n$
19. 在一棵完全二叉树中, 其根的序号为 1, () 可判定序号为 p 和 q 的两个结点是否在同一层。
A. $\lfloor \log_2 p \rfloor = \lfloor \log_2 q \rfloor$ B. $\log_2 p = \log_2 q$
C. $\lfloor \log_2 p \rfloor + 1 = \lfloor \log_2 q \rfloor$ D. $\lfloor \log_2 p \rfloor = \lfloor \log_2 q \rfloor + 1$
20. 假定一棵三叉树的结点数为 50, 则它的最小高度为 ()。
A. 3 B. 4 C. 5 D. 6

【答案解析】

1. C。在二叉树中, 若某个结点只有一个孩子, 则这个孩子的左右次序是确定的; 而在度为 2 的有序树中, 若某个结点只有一个孩子, 则这个孩子就无须区分其左右次序, A 错误。选项 B 仅当是完全二叉树时才有意义, 对于任意一棵二叉树, 高度可能为 $\lfloor \log_2 n \rfloor + 1 \sim n$ 。在二叉排序树中插入结点时, 一定插入在叶结点的位置, 故若先删除分支结点再插入, 则会导致二叉排序树的重构, 其结果就不再相同, D 错误。根据完全二叉树的定义, 在完全二叉树中, 若有度为 1 的结点, 则只可能有一个, 且该结点只有左孩子而无右孩子, 选项 C 正确。

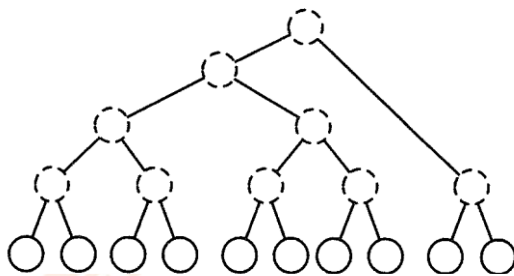
2. A。在完全二叉树中, 叶子结点的双亲的左兄弟的孩子一定在其前面 (且一定存在), 故双亲的左兄弟 (若存在) 一定不是叶结点, A 正确。 n_0 应等于 $n_2 + 1$, B 错误。完全二叉树和满二叉树均可以采用顺序存储结构, C 错误。第 i 个结点的左孩子不一定存在, D 错误。

选项 B 的这种通用公式适用于所有二叉树, 我们应能立即联想到采用特殊值代入法验证, 如画

一个只含 3 个结点的满二叉树的草图来验证是否满足条件。

3. B。由二叉树的性质 $n_0 = n_2 + 1$, 得 $n_2 = n_0 - 1 = 10 - 1 = 9$ 。

另解: 画出草图, 如下图所示。首先画出 10 个叶结点, 然后每 2 个结点向上合并, 构造一个新的度为 2 的分支结点, 直到构成如下图所示的二叉树, 其中度为 2 的分支结点数为 9。



4. C。本题考查二叉树的性质。含有 n 个结点的二叉树共有 $\frac{C_{2n}^n}{n+1}$ 种, 因此本题选 C。

5. B。

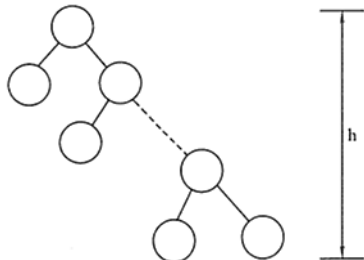
6. B。3、4 这两道题都考查二叉树的性质。叶子结点个数比度为 2 的结点的个数多 1, 可知本题选 B。

7. D。本题考查完全二叉树的性质。本题可分两种情况讨论: ①也没有单分支结点, 设叶子结点的个数 n , n 为整数, 则有 $n+n-1=1001$, 解得 $n=501$; ②有 1 个单分支结点, 则有 $n+n-1+1=1001$, n 无解。综上可知此题选 D。

8. C。本题考查完全二叉树结点个数的计算。其 1~3 层是一个满二叉树, 结点数为 7, 而第 4 层至少有 1 个结点, 因此本题选 C。

9. C。本题考查完全二叉树结点个数的计算。高度一定, 结点数最多的完全二叉树即为同高度的满二叉树。高度为 5 的完全二叉树结点最多有 $2^5 - 1 = 31$ 个, 因此本题选 C。

10. B。本题考查二叉树的基本性质。高度一定, 结点最少的情况如下图所示。除了最上层有一个结点以外, 其余 $h-1$ 层均有两个结点, 结点总数为 $2(h-1)+1=2h-1$ 。因此本题选 B。



11. C。要求满足条件的树, 分析可知, 当这 50 个结点构成一棵完全二叉树时高度最小, $h = \lfloor \log_2 n \rfloor + 1 = \lfloor \log_2 50 \rfloor + 1 = 6$ 。

另解: 第 1 层最多有 1 个结点, 第 2 层最多有 2^1 个结点, 第 3 层最多有 2^2 个结点, 第 4 层最多有 2^3 个结点, 以此类推, 可以得到 h 最少为 6。

12. C。由二叉树的性质 1 可知 $n_0 = n_2 + 1$, 结点总数 $= 2n = n_0 + n_1 + n_2 = n_1 + 2n_2 + 1$, 则 $n_1 = 2(n - n_2) - 1$, 所以 n_1 为奇数, 说明该二叉树中不可能有 $2m$ 个度为 1 的结点。

13. C。解题思路同第 4 题, 第一层有一个结点, 其余 $h-1$ 层上各有两个结点, 总结点数 $= 1 + 2(h-1) = 15$, $h = 8$ 。建议画出草图。

14. A。第 6 层有叶结点说明完全二叉树的高度可能为 6 或 7, 显然树高为 6 时结点最少。若第 6 层上有 8 个叶结点, 则前 5 层为满二叉树, 故完全二叉树的结点个数最少为 $2^5 - 1 + 8 = 39$ 个结点。

15. A。深度为 6 的完全二叉树, 第 5 层共有结点 $2^4 = 16$ 个。第 6 层最左边有 3 个叶子结点, 其对应的双亲结点为第 5 层最左边的两个结点, 所以第 5 层剩余的结点均为叶子结点, 共有 $16 - 2 = 14$ 个, 加上第 6 层的 3 个, 共 17 个叶子结点。

16. C。要使二叉树在第 7 层达到最多的结点个数, 其上面的 6 层必须是一个满二叉树, 深度为 6 的满二叉树有 $63(2^6 - 1)$ 个结点, 故第 7 层最多有 $126 - 63 = 63$ 个结点。

17. B。在非空的二叉树当中, 由度为 0 和 2 的结点数关系 $n_0 = n_2 + 1$ 可知 $n_2 = 123$; 总结点数 $n = n_0 + n_1 + n_2 = 247 + n_1$, 其最大值为 248 (n_1 的取值为 1 或 0, 当 $n_1 = 1$ 时结点最多), 注意, 由完全二叉树总结点数的奇偶性可以确定 m 的值, 但不能根据 n 来确定的 n_1 值。

另解: $124 < 2^7 = 128$, 故第 8 层没满, 前 7 层为完全二叉树, 由此可推算第 8 层可能有 120 个叶子结点, 第 7 层的最右 4 个为叶子结点, 考虑最多的情况, 这 4 个叶子结点中的最左边可以有 1 个左孩子 (不改变叶子结点数), 因此结点总数 $= 2^7 - 1 + 120 + 1 = 248$ 。

18. B。非空指针数 = 总分支数 = $n - 1$, 空指针数 = $2 \times$ 结点总数 - 非空指针数 = $2n - (n - 1) = n + 1$ 。

另解: 在树中, 1 个指针对应 1 个分支, n 个结点的树共有 $n - 1$ 个分支, 即 $n - 1$ 个非空指针, 每个结点有 2 个指针域, 故空指针数 = $2n - (n - 1) = n + 1$ 。

19. A。由完全二叉树的性质, 编号为 i ($i \geq 1$) 的结点所在的层次为 $\lfloor \log_2 i \rfloor + 1$, 若两个结点位于同一层, 则一定有 $\lfloor \log_2 p_i \rfloor + 1 = \lfloor \log_2 q_i \rfloor + 1$, 因此有 $\lfloor \log_2 p_i \rfloor = \lfloor \log_2 q_i \rfloor$ 成立。

20. C。分析可知, 满足条件的三叉树可以是完全三叉树, 这棵树的第 i ($i \geq 1$) 层最多有 3^{i-1} 个结点。设高度为 h , 则 $3^0 + 3^1 + \dots + 3^{h-1} = (3^h - 1)/2$ 是结点数上限, 问题是求解 $50 \leq (3^h - 1)/2$ 的最小 h 值, 即 $h \geq \log_3 101$, 有 $h = \lceil \log_3 101 \rceil = 5$ 。

(二) 综合应用题

1. 设二叉树根结点所在层次为 1, 树的深度 d 为距离根最远的叶结点所在的层次。

(1) 试精确给出深度为 d 的完全二叉树的不同二叉树棵数。

(2) 试精确给出深度为 d 的满二叉树的不同二叉树棵数。

2. 在一棵完全二叉树中, 含有 n_0 个叶子结点, 当度为 1 的结点数为 1 时, 该树的高度是多少? 当度为 1 的结点数为 0 时, 该树的高度是多少?

3. 一棵有 n 个结点的满二叉树有多少个分支结点和多少个叶子结点? 该满二叉树的高度是多少?

4. 已知完全二叉树的第 9 层有 240 个结点, 则整个完全二叉树有多少个结点? 有多少个叶子结点?

【答案解析】

1. 答: 1) 根据二叉树的性质, 在第 d 层最多有 2^{d-1} 个结点。因此, 深度为 d 的不同完全二叉树有 2^{d-1} 棵。

2) 深度为 d 的满二叉树只有 1 棵。

2. 解答: 在非空的二叉树中, 由度为 0 和度为 2 的结点之间的关系 $n_0 = n_2 + 1$, 可知 $n_2 = n_0 - 1$ 。因此总结点数 $n = n_0 + n_1 + n_2 = 2n_0 + n_1 - 1$ 。

① 当 $n_1 = 1$ 时, $n = 2n_0$, $h = \lceil \log_2(n+1) \rceil = \lceil \log_2(2n_0+1) \rceil$

② 当 $n_1 = 0$ 时, $n = 2n_0 - 1$, $h = \lceil \log_2(n+1) \rceil = \lceil \log_2(2n_0) \rceil = \lceil \log_2 n_0 \rceil + 1$

3.解答: 满二叉树中 $n_1 = 0$, 由二叉树的性质 1 可知 $n_0 = n_2 + 1$, 即 $n_2 = n_0 - 1$, $n = n_0 + n_1 + n_2 = 2n_0 - 1$, 则 $n_0 = (n + 1)/2$ 。分支结点个数 $n_2 = n - (n + 1)/2 = (n - 1)/2$ 。高度为 h 的满二叉树的结点数 $n = 1 + 2^1 + 2^2 + \dots + 2^{h-1} = 2^h - 1$, 即高度 $h = \log_2(n + 1)$ 。

4.解答: 在完全二叉树中, 若第 9 层是满的, 则结点数 $= 2^{9-1} = 256$, 而现在第 9 层只有 240 个结点, 说明第 9 层未滿, 是最后一层。1~8 层是满的, 所以总结点数 $= 2^8 - 1 + 240 = 495$ 。

因为第 9 层是最后一层, 所以第 9 层的结点都是叶子结点。且第 9 层的 240 个结点的双亲在第 8 层中, 其双亲个数为 120, 即第 8 层有 120 个分支结点, 其余为叶子结点, 所以第 8 层的叶子结点个数为 $2^{8-1} - 120 = 8$ 。因此, 总的叶子结点个数 $= 8 + 240 = 248$ 。

另解: 总结点数 $n = n_0 + n_1 + n_2$, $n_2 = n_0 - 1$, $n = n_0 + n_1 + n_2 = 2n_0 + n_1 - 1$ 。若 $n_1 = 1$, 则 $2n_0 + n_1 - 1 = 2n_0 = 495$, 不符合; 若 $n_1 = 0$, 则 $2n_0 + n_1 - 1 = 2n_0 - 1 = 495$, 则 $n_0 = 248$ 。注意: 对于本题, 应理解完全二叉树中只有最低一层的结点是不满的, 其他各层的结点是满的 (即第 i 层有 2^{i-1} 个结点)。

5.3 二叉树的遍历

(一) 选择题

- 在下列关于二叉树遍历的说法中, 正确的是()。
 - 若有一个结点是二叉树中某个子树的中序遍历结果序列的最后一个结点, 则它一定是该子树的前序遍历结果序列的最后一个结点
 - 若有一个结点是二叉树中某个子树的前序遍历结果序列的最后一个结点, 则它一定是该子树的中序遍历结果序列的最后一个结点
 - 若有一个叶子结点是二叉树中某个子树的中序遍历结果序列的最后一个结点, 则它一定是该子树的前序遍历结果序列的最后一个结点
 - 若有一个叶子结点是二叉树中某个子树的前序遍历结果序列的最后一个结点, 则它一定是该子树的中序遍历结果序列的最后一个结点
- 在任何一棵二叉树中, 若结点 a 有左孩子 b 、右孩子 c , 则在结点的先序序列、中序序列、后序序列中()。
 - 结点 b 一定在结点 a 的前面
 - 结点 a 一定在结点 c 的前面
 - 结点 b 一定在结点 c 的前面
 - 结点 a 一定在结点 b 的前面
- 设 n, m 为一棵二叉树上的两个结点, 在中序遍历时, n 在 m 前的条件是()。
 - n 在 m 右方
 - n 是 m 祖先
 - n 在 m 左方
 - n 是 m 子孙
- 设 n, m 为一棵二叉树上的两个结点, 在后序遍历时, n 在 m 前的条件是()。
 - n 在 m 右方
 - n 是 m 祖先
 - n 在 m 左方
 - n 是 m 子孙
- 在二叉树中有两个结点 m 和 n , 若 m 是 n 的祖先, 则使用()可以找到从 m 到 n 的路径。
 - 先序遍历
 - 中序遍历
 - 后序遍历
 - 层次遍历
- 在二叉树的前序序列、中序序列和后序序列中, 所有叶子结点的先后顺序()。
 - 都不相同
 - 完全相同
 - 前序和中序相同, 而与后序不同
 - 中序和后序相同, 而与前序不同
- 对二叉树的结点从 1 开始进行连续编号, 要求每个结点的编号大于其左、右孩子的编号, 同一结点的左、右孩子中, 其左孩子的编号小于其右孩子的编号, 可采用()次序的遍历实现编号。
 - 先序遍历
 - 中序遍历
 - 后序遍历
 - 层次遍历
- 前序为 A, B, C, 后序为 C, B, A 的二叉树共有()。

- A. 1 棵 B. 2 棵 C. 3 棵 D. 4 棵
9. 一棵非空的二叉树的先序遍历序列与后序遍历序列正好相反, 则该二叉树一定满足()。
- A. 所有的结点均无左孩子 B. 所有的结点均无右孩子
C. 只有一个叶结点 D. 是任意一棵二叉树
10. 设结点 X 和 Y 是 二叉树中任意的两个结点。在该二叉树的先序遍历序列中 X 在 Y 之前, 而在其后序遍历序列中 X 在 Y 之后, 则 X 和 Y 的关系是 ()。
- A. X 是 Y 的左兄弟 B. X 是 Y 的右兄弟
C. X 是 Y 的祖先 D. X 是 Y 的后裔
11. 若二叉树中结点的先序序列是... a...b ..., 中序序列是... b...a..., 则 ()。
- A. 结点 a 和结点 b 分别在某结点的左子树和右子树中
B. 结点 b 在结点 a 的右子树中
C. 结点 b 在结点 a 的左子树中
D. 结点 a 和结点 b 分别在某结点的两棵非空子树中
12. 一棵二叉树的前序遍历序列为 1234567, 它的中序遍历序列可能是 ()。
- A. 3124567 B. 1234567 C. 4135627 D. 1463572
13. 下列序列中, 不能唯一地确定一棵二叉树的是 ()。
- A. 层次序列和中序序列 B. 先序序列和中序序列
C. 后序序列和中序序列 D. 先序序列和后序序列
14. 一棵二叉树的先序遍历序列和其后序遍历序列正好相反, 则该二叉树一定是()。
- A. 空树或者只有一个结点 B. 完全二叉树
C. 二叉排序树 D. 高度等于其结点数
15. 有关二叉树下列说法正确的是 ()。
- A. 二叉树的度为 2 B. 一棵二叉树的度可以小于 2
C. 二叉树中至少有一个结点的度为 2 D. 二叉树中任何一个结点的度都为 2
16. 在一棵非空二叉树的中序遍历序列中, 根结点的右边()。
- A. 只有右子树上的所有结点 B. 只有右子树上的部分结点
C. 只有左子树上的部分结点 D. 只有左子树上的所有结点
17. 设 n 与 m 为一棵二叉树上的两个结点, 在中序遍历时, n 在 m 前的条件是 ()。
- A. n 在 m 的右方 B. n 是 m 的祖先
C. n 在 m 的左方 D. n 是 m 的子孙

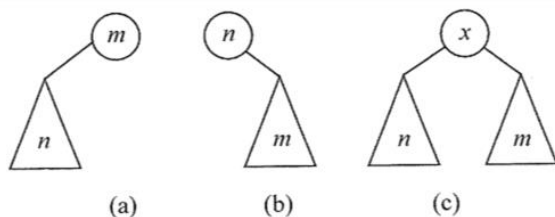
【答案解析】

1. C。二叉树中序遍历的最后一个结点一定是从根开始沿右子女指针链走到底的结点, 设用 p 指示。若结点 p 不是叶子结点 (其左子树非空), 则前序遍历的最后一个结点在它的左子树中, A、B 错; 若结点 p 是叶子结点, 则前序与中序遍历的最后一个结点就是它, C 正确。若中序遍历的最后一个结点 p 不是叶子结点, 它还有一个左子女 q, 结点 q 是叶子结点, 那么结点 q 是前序遍历后一个结点, 但不是中序遍历的最后一个结点, D 错。

2. C。这三种遍历方式中, 无论哪种遍历方式, 都先遍历左子树, 再遍历右子树, 所以结点 b 在结点 c 的前面访问。

3. C。中序遍历时, 先访问左子树, 再访问根结点, 后访问右子树。n 在 m 前的 3 种可能性如下图所示, 从中看出 n 总是在 m 的左方。另解: 设 n 和 m 的最近公共祖先 p, 则有以下可能: 情形

1. m 和 n 分别在 p 的左、右 (右、左) 分支上; 情形 2, m 或 n 为 p 结点, 另一结点在 p 的分支上。 n 和 m 分别处于 p 的左、右分支上, m 为祖先结点且 n 位于 m 的左分支, n 为祖先结点且 m 位于 n 的右分支, 符合题意。



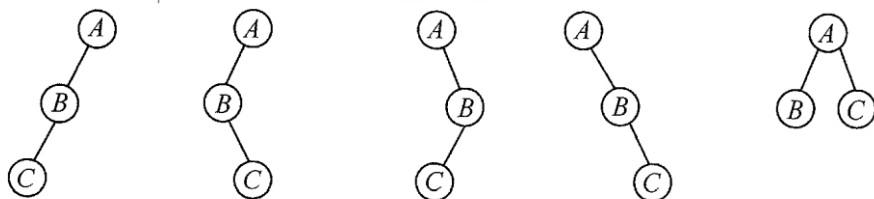
4. D。后序遍历的顺序是 LRN, 若 n 在 N 的左子树, m 在 N 的右子树, 则在后序遍历的过程中 n 在 m 之前访问; 若 n 是 m 的子孙, 设 m 在 N 的位置, 则 n 无论是在 m 的左子树还是在右子树, 在后序遍历的过程中 n 都在 m 之前访问。其他都不可以。C 要成立, 要加上两个结点位于同一层。

5. C。在后序遍历退回时访问根结点, 就可以从下向上把从 n 到 m 的路径上的结点输出, 若采用非递归的算法, 则当后序遍历访问到 n 时, 栈中把从根到 n 的父指针的路径上的结点都记忆下来, 也可以找到从 m 到 n 的路径。

6. B。在三种遍历方式中, 访问左右子树的先后顺序是不变的, 只是访问根结点的顺序不同, 因此叶子结点的先后顺序完全相同。此外, 读者可以采用特殊值法, 画一个结点数为 3 的满二叉树, 采用三种遍历方式来验证答案的正确性。

7. C。对每个顶点从 1 开始按序编号, 要求结点编号大于其左、右孩子编号, 并且左孩子编号小于右孩子编号。编号越大说明遍历顺序越靠后, 因此, 三者遍历顺序为先左子树、再右子树、后根结点, 4 个选项中仅后序遍历满足要求。

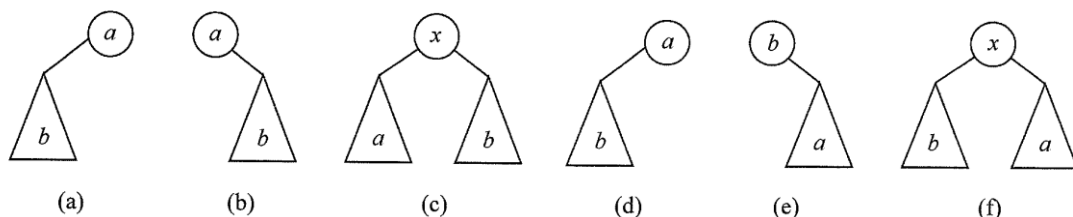
8. D。前序为 A、B、C 的不同二叉树共有 5 种, 其中后序为 C、B、A 的有 4 种 (前 4 种), 都是单支树, 如下图所示。



9. C。非空树的先序序列和后序序列相反, 即“根左右”与“左右根”顺序相反, 因此树只有根结点, 或者根结点只有左子树或右子树, 以此类推, 其子树有同样的性质。因此, 树中所有非叶结点的度均为 1, 即二叉树仅有一个叶结点。

10. C。设二叉树的前序遍历顺序为 NLR, 后序遍历顺序为 LRN。根据题意, 在前序遍历序列中 X 在 Y 之前, 在后序遍历序列中 X 在 Y 之后, 若设 X 在根结点的位置, Y 在其左子树或右子树中, 即满足要求。

11. C。先序序列是 $\cdots a \cdots b \cdots$, 因此 a 和 b 结点的 3 种情况如下图(a)~(c)所示。中序序列是 $\cdots b \cdots a \cdots$, 因此 a 和 b 结点的 3 种情况如下图(d)~(f)所示, 相同部分是 b 在 a 的左子树中。

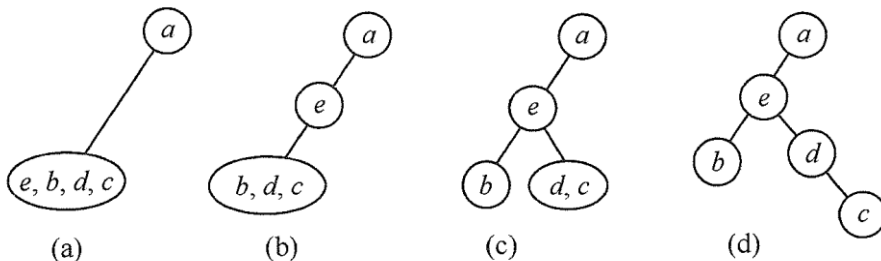


12. B。由题可知 1 为根结点。对于 A 选项, 3 应为 1 的左孩子, 先序序列应为 13……不符。类似的选项 C 也是错误的。对于 B 选项, 2 为 1 的右孩子, 3 为 2 的右孩子……满足题意。对于 D 选项, 463572 应为 1 的右子树, 2 为 1 的右孩子, 46357 为 2 的左子树, 3 为 2 的左孩子,

46 为 3 的左子树, 57 为 3 的右子树, 先序序列 4、6 应相连, 5、7 应相连, 不符。

另解: 前序遍历时需要借助栈。前序序列和中序序列的关系相当于以前序序列为入栈次序, 以中序序列为出栈次序。题中以 1234567 入栈: 选项 A, 第一个出栈的是 3, 故 1 不可能在 2 之前出栈, 错误。选项 C, 1 不可能在 3 之前出栈, 错误。选项 D, 6 第三个出栈, 此时栈顶元素是 5, 不是 3, 错误。故选 B。

13. D。前序序列和后序序列不能唯一确定一棵二叉树, 但可以确定二叉树中结点的祖先关系: 当两个结点的前序序列为 XY、后序序列为 YX 时, 则 X 为 Y 的祖先。考虑前序序列 a, e, b, d, c, 后序序列 b, c, d, e, a, 可知 a 为根结点, e 为 a 的孩子结点; 此外, 由 a 的孩子结点的前序序列 e, b, d, c 和后序序列 b, c, d, e, 可知 e 是 bed 的祖先, 故根结点的孩子结点只有 e。故选 D。



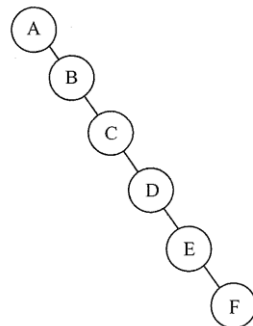
排除法: 显然 a 为根结点, 且确定 e 为 a 的孩子结点, 排除 D。各种遍历算法中左右子树的遍历次序是固定的, 若 b 也为 a 的孩子结点, 则在前序序列和后序序列中 e、b 的相对次序应是不变的, 故排除 B, 同理排除 C。

特殊法: 前序序列和后序序列对应多棵不同的二叉树树形, 我们只需画出满足该条件的任意一棵二叉树即可, 任意一棵二叉树必定满足正确选项的要求。

显然选 A, 最终得到的二叉树满足题设中前序序列和后序序列的要求。

14. D。本题考查二叉树的遍历。对于高度等于其结点数的二叉树, 每层只有一个结点, 假设从上向下分别为 A, B, C, D, E, F, 则其先序遍历序列为 A, B, C, D, E, F, 后序遍历序列为 F, E, D, C, B, A, 如右图所示。

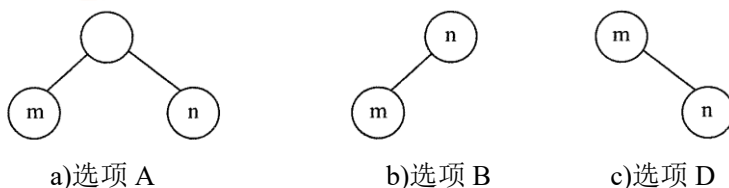
提示: 在数据结构的考题中, 尤其是树和图的选择題部分, 很多题目都可以用特殊情况验证的方法来解决。如本题, 可以画出一个简单的二叉树来排除其他情况。



15. B。本题考查树与二叉树的性质。由树的度的定义可知, 树中各个结点度的最大值为树的度, 单个结点也可以称为一棵树。因此, 对于一棵二叉树, 当只有一个结点的时候, 其度为 0。因此本题选 B。

16. A。本题考查二叉树的中序遍历。中序遍历的规则为: 先中序遍历左子树, 然后访问根结点, 最后中序遍历右子树。在中序遍历序列中, 根结点将左子树上的所有结点与右子树上的所有结点分割开来。根左边的序列包含左子树上的所有结点, 根右边的序列包含右子树上的所有结点。因此本题选 A。

17. C。本题考查二叉树的中序遍历。本题用排除法比较好。通过反例排除了 A、B、D 3 个选项, 如下图所示。



如图 a 所示, n 在 m 的右方, 遍历序列中 m 与 n 的相对位置为: $\dots m \dots n \dots$, 因此选项 A 错, 将其排除。

如图 b 所示, n 是 m 的祖先, 遍历序列中 m 与 n 的相对位置为: $\dots m \dots n \dots$, 因此选项 B 错, 将其排除。

如图 c 所示, n 是 m 的子孙, 遍历序列中 m 与 n 的相对位置为: $\dots m \dots n \dots$, 因此选项 D 错, 将其排除。

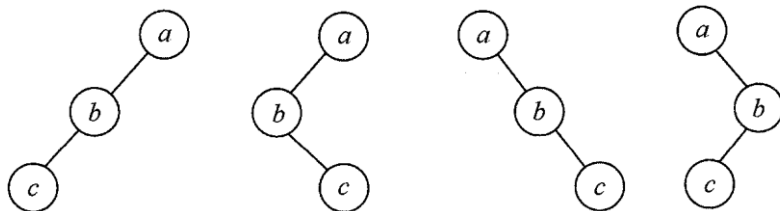
因此本题选 C。

(二) 综合应用题

1. 若某非空二叉树的先序序列和后序序列正好相反, 则该二叉树的形态是什么?
2. 若某非空二叉树的先序序列和后序序列正好相同, 则该二叉树的形态是什么?
3. 对于一个堆栈, 若其入栈序列为 $1, 2, 3, \dots, n$, 不同的出入栈操作将产生不同的出栈序列。其出栈序列的个数正好等于结点个数为 n 的二叉树的个数, 且与不同形态的二叉树一一对应。请简要叙述一种从堆栈输入 (固定为 $1, 2, 3, \dots, n$) 和输出序列对应一种二叉树形态的方法, 并以入栈序列 $1, 2, 3$ ($n=3$) 为例确定所有的二叉树。

【答案解析】

1. 解答: 二叉树的先序序列是 NLR, 后序序列是 LRN。要使 $NLR = NRL$ (后序序列反序) 成立, L 或 R 应为空, 这样的二叉树每层只有一个结点, 即二叉树的形态是其高度等于结点个数。以 3 个结点 a, b, c 为例, 其形态如下图所示。

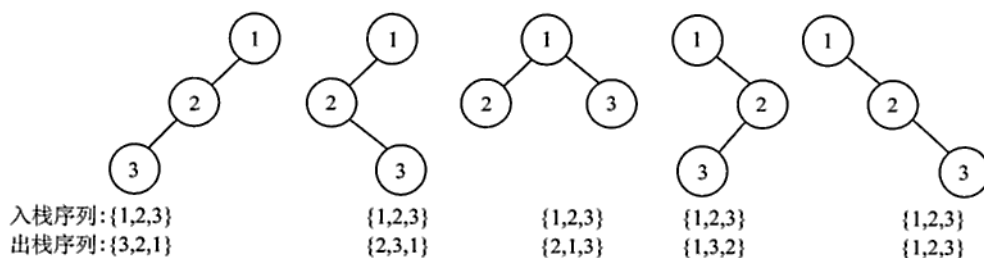


2. 解答: 二叉树的先序序列是 NLR, 后序序列是 LRN。要使 $NLR = LRN$ 成立, L 和 R 应均为空, 所以满足条件的二叉树只有一个根结点。

3. 二叉树的遍历都是递归的, 递归要用到系统栈。其实在二叉树的遍历序列中, 先序遍历序列即为各结点在入栈时进行打印所得的序列, 中序遍历序列即为各结点在出栈时进行打印所得的序列。明白了这一点, 本题就容易解决了。

1) 由于二叉树先序遍历序列和中序遍历序列可唯一确定一棵二叉树, 因此若入栈序列为 $1, 2, 3, \dots, n$, 相当于先序遍历序列是 $1, 2, 3, \dots, n$, 出栈序列就是对应的二叉树的中序遍历序列。有了进栈和出栈序列, 就相当于知道了一棵二叉树的先序和中序遍历序列, 即确定了对应的二叉树。

2) 当结点入栈序列为 $\{1, 2, 3\}$ 时, 出栈序列可能为 $\{3, 2, 1\}$ 、 $\{2, 3, 1\}$ 、 $\{2, 1, 3\}$ 、 $\{1, 3, 2\}$ 、 $\{1, 2, 3\}$ 。因此, 依次构造的二叉树如下图所示 (树下方为生成这棵树的两个序列)。



5.4 线索二叉树

- 线索二叉树是一种 () 结构。
 - 逻辑
 - 逻辑和存储
 - 物理
 - 线性
- n 个结点的线索二叉树上含有的线索数为 ()。
 - $2n$
 - $n-1$
 - $n+1$
 - n
- 判断线索二叉树中 $*p$ 结点有右孩子结点的条件是 ()。
 - $p! = \text{NULL}$
 - $p -> \text{rchild}! = \text{NULL}$
 - $p -> \text{rtag} == 0$
 - $p -> \text{rtag} == 1$
- 一棵左子树为空的二叉树在先序线索化后, 其中空的链域的个数是 ()。
 - 不确定
 - 0 个
 - 1 个
 - 2 个
- 在线索二叉树中, 下列说法不正确的是 ()。
 - 在中序线索树中, 若某结点有右孩子, 则其后继结点是其右子树的最左下结点
 - 在中序线索树中, 若某结点有左孩子, 则其前驱结点是它的左子树的最右下结点
 - 线索二叉树是利用二叉树的 $n+1$ 个空指针来存放结点的前驱和后继信息的
 - 每个结点通过线索都可以直接找到它的前驱和后继
- 二叉树在线索化后, 仍不能有效求解的问题是 ()。
 - 先序线索二叉树中求先序后继
 - 中序线索二叉树中求中序后继
 - 中序线索二叉树中求中序前驱
 - 后序线索二叉树中求后序后继
- 若 X 是二叉中序线索树中一个有左孩子的结点, 且 X 不为根, 则 X 的前驱为 ()。
 - X 的双亲
 - X 的右子树中最左的结点
 - X 的左子树中最右结点
 - X 的左子树中最右叶结点
- () 的遍历仍需要栈的支持。
 - 前序线索树
 - 中序线索树
 - 后序线索树
 - 所有线索树
- 若 X 是二叉中序线索树中一个有左孩子的结点, 且 X 不为根, 则 X 的前驱为 ()。
 - X 的双亲
 - X 的右子树中最左边的结点
 - X 的左子树中最右边的结点
 - X 的左子树中最右边的叶结点
- 引入二叉线索树的目的是 ()。
 - 加快查找结点的前驱或后继的速度
 - 为了能在二叉树中方便地进行插入与删除
 - 为了能方便地找到双亲
 - 使二叉树的遍历结果唯一

【答案解析】

1. C。二叉树是一种逻辑结构, 但线索二叉树是加上线索后的链表结构, 即它是二叉树在计算机内部的一种存储结构, 所以是一种物理结构。

2. C。 n 个结点共有链域指针 $2n$ 个, 其中, 除根结点外, 每个结点都被一个指针指向。剩余

的链域建立线索, 共 $2n - (n - 1) = n + 1$ 个线索。

3. C。线索二叉树中用 ltag /rtag 标识结点的左/右指针域是否为线索, 其值为 1 时, 对应指针域为线索, 其值为 0 时, 对应指针域为左/右孩子。

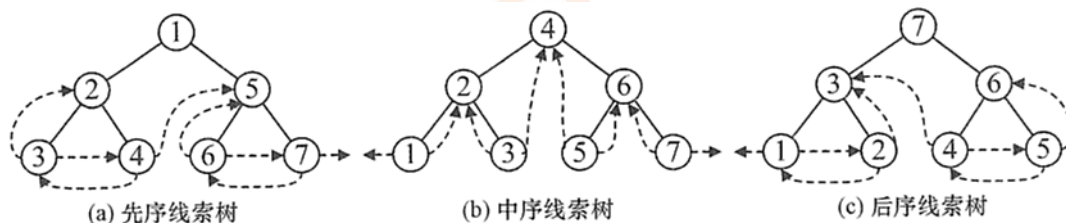
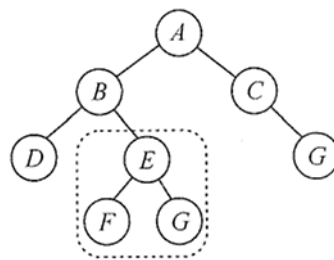
4. D。对左子树为空的二叉树进行先序线索化, 根结点的左子树为空并且也没有前驱结点 (先遍历根结点), 先序遍历的最后一个元素为叶结点, 左、右子树均为空且有前驱无后继结点, 故线索化后, 树中空链域有 2 个。

5. D。不是每个结点通过线索都可以直接找到它的前驱和后继。在先序线索二叉树中查找一个结点的先序后继很简单, 而查找先序前驱必须知道该结点的双亲结点。同样, 在后序线索二叉树中查找一个结点的后序前驱也很简单, 而查找后序后继也必须知道该结点的双亲结点, 二叉链表中没有存放双亲的指针。

6. D。后序线索二叉树不能有效解决求后序后继的问题。如右图所示, 结点 E 的右指针指向右孩子, 而在后序序列中 E 的后继结点为 B, 在查找 E 的后继时后序线索不能起到任何作用, 只能按常规方法来查找。

7. C。在二叉中序线索树中, 某结点若有左孩子, 则按照中序“左根右”的顺序, 该结点的前驱结点为左子树中最右的一个结点 (注意, 并不一定是最右叶子结点)。

8. C。后序线索树遍历时, 最后访问根结点, 若从右孩子 x 返回访问父结点, 则由于结点 x 的右孩子不一定为空 (右指针无法指向其后继), 因此通过指针可能无法遍历整棵树。如下图所示, 结点中的数字表示遍历的顺序, 图(c)中结点 6 的右指针指向其右孩子 5, 而不指向其后序后继结点 7, 因此后序遍历还需要栈的支持, 而图(a)和图(b)均可遍历。



9. C。本题考查二叉树的遍历以及线索二叉树的相关知识。在中序线索二叉树中, X 的前驱即为中序遍历序列中 X 的前驱。对于中序遍历, 要访问 X, 必须把 X 左子树中的结点都访问完才可以, 因此 X 左子树中最后一个被访问的结点即为 X 的前驱结点; 而 X 左子树中的最右边的结点即为左子树中最后一个被访问的结点。因此本题选 C。

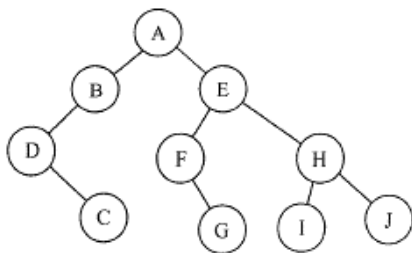
10. A。本题考查线索二叉树的作用。在一般的二叉树中, 我们只知道某个点的左、右孩子, 并不能知道某个结点的直接前驱和直接后继, 如果能够知道“前驱”和“后继”信息, 就可以把二叉树看作一个链表结构, 从而可以像遍历链表那样来遍历二叉树, 提高效率。因此本题选 A。

5.5 树、森林

(一) 选择题

1. 下列关于树的说法中, 正确的是()。
 - I. 对于有 n 个结点的二叉树, 其高度为 $\log_2 n$
 - II. 完全二叉树中, 若一个结点没有左孩子, 则它必是叶结点
 - III. 高度为 h ($h > 0$) 的完全二叉树对应的森林所含的树的个数一定是 h
 - IV. 一棵树中的叶子数一定等于与其对应的二叉树的叶子数

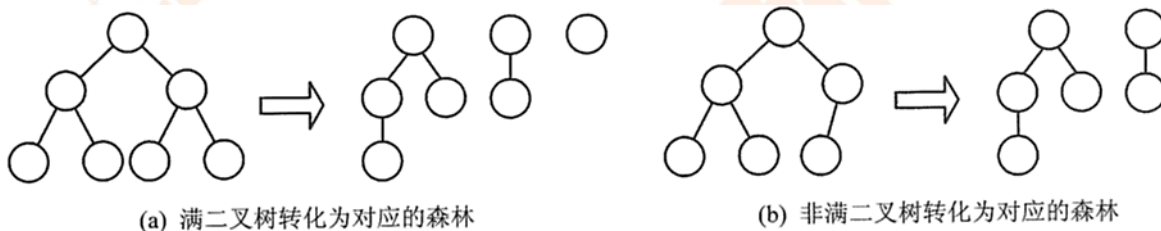
- A. I 和 III B. IV C. I 和 II D. II
2. 利用二叉链表存储森林时, 根结点的右指针是 ()。
- A. 指向最左兄弟 B. 指向最右兄弟 C. 一定为空 D. 不一定为空
3. 设森林 F 中有 3 棵树, 第一、第二、第三棵树的结点个数分别为 M_1 , M_2 和 M_3 。与森林 F 对应的二叉树根结点的右子树上的结点个数是 ()。
- A. M_1 B. $M_1 + M_2$ C. M_3 D. $M_2 + M_3$
4. 设森林 F 对应的二叉树为 B, 它有 m 个结点, B 的根为 p, p 的右子树结点个数为 n , 森林 F 中第一棵树的结点个数是 ()。
- A. $m-n$ B. $m-n-1$ C. $n+1$ D. 条件不足, 无法确定
5. 森林 $T=(T_1, T_2, \dots, T_m)$ 转化为二叉树 BT 的过程为: 若 $m=0$, 则 BT 为空, 若 $m \neq 0$, 则 ()。
- A. 将中间子树 T_{mid} ($mid=(1+m)/2$) 的根作为 BT 的根; 将 $(T_1, T_2, \dots, T_{mid-1})$ 转换为 BT 的左子树; 将 $(T_{mid+1}, \dots, T_m)$ 转换为 BT 的右子树。
- B. 将子树 T_1 的根作为 BT 的根; 将 T_1 的子树森林转换成 BT 的左子树; 将 $(T_2, T_3, \dots, T_m)$ 转换成 BT 的右子树。
- C. 将子树 T_1 的根作为 BT 的根; 将 T_1 的左子树森林转换成 BT 的左子树; 将 T_1 的右子树森林转换为 BT 的右子树; 其他以此类推。
- D. 将森林 T 的根作为 BT 的根; 将 $(T_1, T_2, \dots, T_m)$ 转化为该根下的结点, 得到一棵树, 然后将这棵树再转化为二叉树 BT。
6. 设 F 是一个森林, B 是由 F 变换来的二叉树。若 F 中有 n 个非终端结点, 则 B 中右指针域为空的结点有 () 个。
- A. $n-1$ B. n C. $n+1$ D. $n+2$
7. 若 T_1 是由有序树 T 转换而来的二叉树, 则 T 中结点的后根序列就是 T_1 中结点的 () 序列。
- A. 先序 B. 中序 C. 后序 D. 层序
8. 设 X 是树 T 中的一个非根结点, B 是 T 所对应的二叉树。在 B 中, X 是其双亲结点的右孩子, 下列结论中正确的是 ()。
- A. 在树 T 中, X 是其双亲结点的第一个孩子
- B. 在树 T 中, X 一定无右边兄弟
- C. 在树 T 中, X 一定是叶子结点
- D. 在树 T 中, X 一定有左边兄弟
9. 在森林的二叉树表示中, 结点 M 和结点 N 是同一父结点的左儿子和右儿子, 则在该森林中 ()。
- A. M 和 N 有同一双亲 B. M 和 N 可能无公共祖先
- C. M 是 N 的儿子 D. M 是 N 的左兄弟
10. 如果下图所示的二叉树是由森林转化而来的, 那么原森林有 () 个叶子结点。
- A. 4 B. 5 C. 6 D. 7



11. 设树 T 的度为 4, 其中度为 1、2、3 和 4 的结点个数分别为 4、2、1、1, 则 T 中的叶子数为 ()。
- A. 5 B. 6 C. 7 D. 8
12. 设森林 F 对应的二叉树为 B , 它有 m 个结点, B 的根为 p , p 的右子树结点数为 n , 森林 F 中第一棵树的结点个数是 ()。
- A. $m-n$ B. $m-n-1$ C. $n+1$ D. 条件不足, 无法确定
13. 在一棵三叉树中, 度为 3 的结点数为 2 个, 度为 2 的结点数为 1 个, 度为 1 的结点数为 2 个, 则度为 0 的结点数为 () 个。
- A. 4 B. 5 C. 6 D. 7
14. 树的后序遍历序列等同于该树对应的二叉树的 ()。
- A. 先序遍历序列 B. 中序遍历序列 C. 后序遍历序列

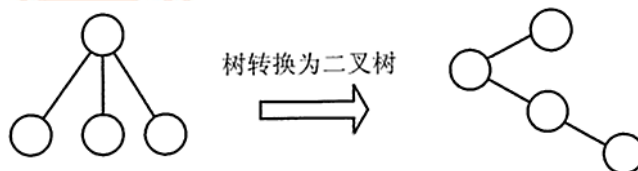
【答案解析】

1. D。若 n 个结点的二叉树是一棵单支树, 则其高度 n 。完全二叉树中最多存在一个度为 1 的结点且只有左孩子, 若不存在左孩子, 则一定也不存在右孩子, 因此必是叶结点, II 正确。只有满二叉树才具有性质 III, 如下图所示。



在树转换为二叉树时, 若有几个叶子结点具有共同的双亲, 则转换成二叉树后只有一个叶子结点 (最右边的叶子结点), 如下图所示, IV 错误。

注意, 若树中的任意两个叶子结点都不存在相同的双亲, 则树中的叶子数才有可能与其对应的二叉树中的叶子数相等。



2. D。森林与二叉树具有对应关系, 因此, 我们存储森林时应先将森林转换成二叉树, 转换的方法就是“左孩子右兄弟”, 与树不同的是, 若存在第二棵树, 则二叉链表的根结点的右指针指向的是森林中的第二棵树的根结点。若此森林只有一棵树, 则根结点的右指针为空。因此, 右指针可能为空也可能不为空。

3. D。森林与二叉树的转换规则同样是“左孩子右兄弟”, 不过与普通树不同, 森林中的每棵树是独立的, 因此先要将每棵树的根结点全部视为兄弟结点的关系。所以, 题中森林转换后, 树 2 作为树 1 的根结点的右子树, 树 3 作为树 2 的根结点的右子树。所以, 森林 F 对应的二叉树根结点的右子树上的结点个数是 $M_2 + M_3$ 。

4. A。森林转换成二叉树时采用孩子兄弟表示法, 根结点及其左子树为森林中的第一棵树。右子树为其他剩余的树。所以, 第一棵树的结点个数为 $m-n$ 。

5. B。将森林中每棵树的根结点视为兄弟结点的关系, 再按照“左孩子右兄弟”的规则来进行转化。

6. C。根据森林与二叉树转换规则“左孩子右兄弟”。二叉树 B 中右指针域为空代表该结点没有兄弟结点。森林中每棵树的根结点从第二个开始依次连接到前一棵树的根的右孩子，因此最后一棵树的根结点的右指针为空。另外，每个非终端结点，其所有孩子结点在转换之后，最后一个孩子的右指针也为空，故树 B 中右指针域为空的结点有 $n+1$ 个。

7. B。有序树 T 转换成二叉树 T_1 时，T 的后根序列是对应 T_1 的中序序列还是后序序列呢（显然树的后根序列不可能对应二叉树的先序序列和层序序列）？看下图所示的例子，在树 T 中，叶子结点 B 应最先访问，在 T 中，B 的右兄弟 C 转换为它的右孩子，若对应 T_1 的后序序列，则 C 应在 B 的前面访问，所以 T 的后根序列不可能对应 T_1 的后序序列。

8. D。在二叉树 B 中，X 是其双亲的右孩子，因此在树 T 中，X 必是其双亲结点的右兄弟，换句话说，X 在树中必有左兄弟。

9. B。在森林的二叉树表示中，当 M 和 N 的父结点是二叉树根结点时，M 和 N 在不同的树上。因此 M 和 N 可能无公共祖先。

10. C。本题考查森林和二叉树的转化。由二叉树与森林的转化方法可知其中 C、D、F、G、I、J 是叶子结点。因此本题选 C。

11. D。本题考查树的性质。直接套用二叉树性质 1 扩展部分所讲的公式： $n_0=1+n_2+2n_3+\dots+(m-1)n_m$ ，可得 T 中叶子结点数为 $1+2+2\times 1+3\times 1=8$ 。因此本题选 D。

12. A。本题考查森林和二叉树的转换。由转换规则可知，二叉树中除了左子树和根结点来源于原森林中第一棵树，其余结点来源于森林中的其他树，其他树的结点总数为 n ，则第一棵树的结点个数为 $m-n$ 。因此本题选 A。

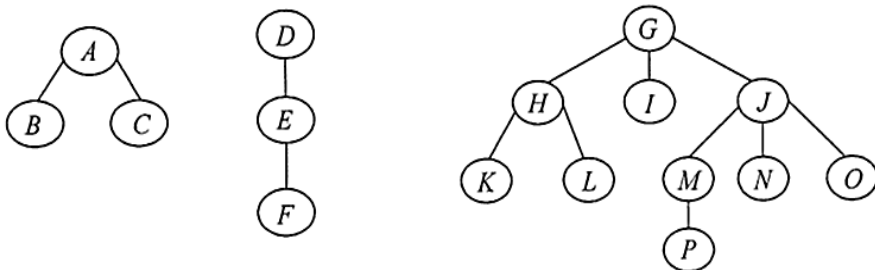
13. C。本题考查树的性质。由公式 $n_0=1+n_2+2n_3+\dots+(m-1)n_m$ 可知，度为 0 的结点个数为 $1+1+2\times 2=6$ 。因此本题选 C。

14. B。本题考查树的遍历与对应二叉树遍历的关系。树的先序遍历对应于二叉树的先序遍历。树的后序遍历对应于二叉树的中序遍历。因此本题选 B。

（二）综合应用题

1. 给定一棵树的先根遍历序列和后根遍历序列，能否唯一确定一棵树？若能，请举例说明；若不能，请给出反例。

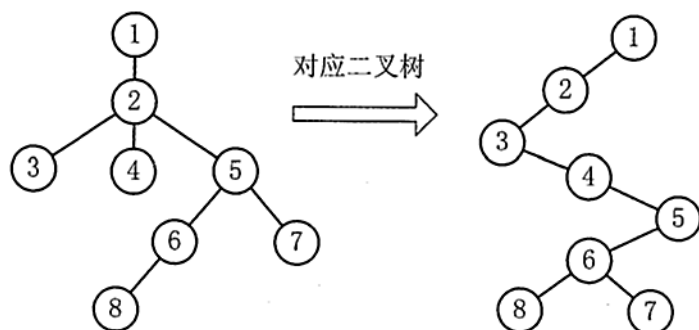
2. 将下面一个由 3 棵树组成的森林转换为二叉树。



【答案解析】

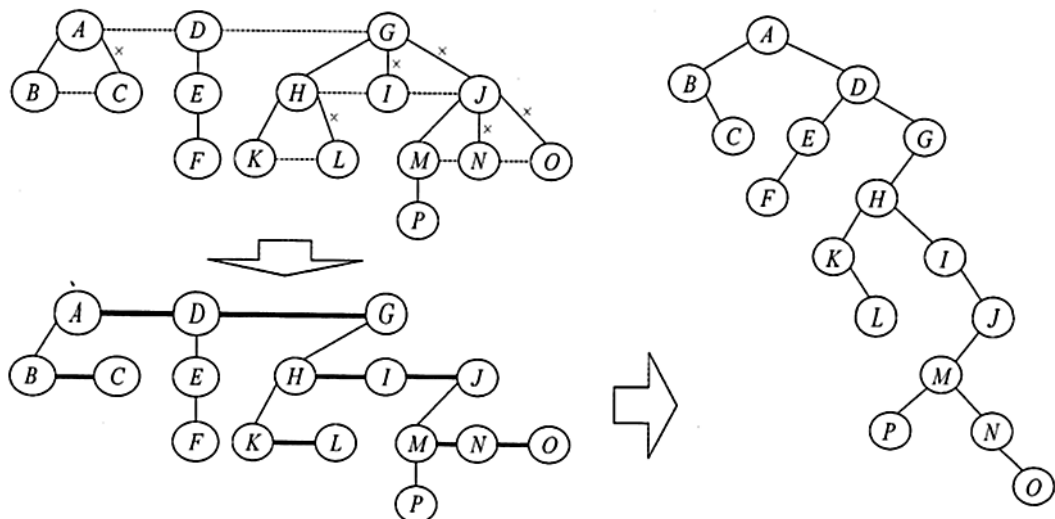
1. 一棵树的先根遍历结果与其对应二叉树的先序遍历结果相同，树的后根遍历结果与其对应二叉树表示的中序遍历结果相同。由于二叉树的先序序列和中序序列能够唯一地确定这棵二叉树，因此，根据题目给出的条件，利用树的先根遍历序列和后根遍历序列能够唯一地确定这棵树。例如，

对于下图所示的树, 对应二叉树的先序序列为 1,2,3,4,5,6,8,7, 中序序列为 3,4,8,6,7,5,2,1。原树的先根遍历序列为 1,2,3,4,5,6,8,7, 后根遍历序列为 3,4,8,6,7,5,2,1。



注意: 树的先根遍历、后根遍历与对应二叉树的前序遍历、中序遍历对应。

2. 根据树与二叉树“左孩子右兄弟”的转换规则, 将森林转换为二叉树的过程如下: ①将每棵树的根结点也视为兄弟关系, 在兄弟结点之间加一连线。②对每个结点, 只保留它与第一个子结点的连线, 与其他子结点的连线全部抹掉。③以树根为轴心, 顺时针旋转 45° 。结果如下图所示。



5.6 树与二叉树的应用

(一) 选择题

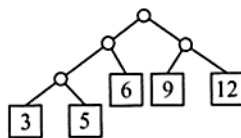
- 在有 n 个叶子结点的哈夫曼树中, 非叶子结点的总数是 ()。
A. $n-1$ B. n C. $2n-1$ D. $2n$
- 给定整数集合 $\{3, 5, 6, 9, 12\}$, 与之对应的哈夫曼树是 ()。



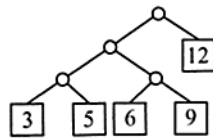
A.



B.



C.



D.

- 设哈夫曼编码的长度不超过 4, 若已对两个字符编码为 1 和 01, 则还最多可对 () 个字符编码。
A. 2 B. 3 C. 4 D. 5

4. 一棵哈夫曼树共有 215 个结点, 对其进行哈夫曼编码, 共能得到 () 个不同的码字。
A. 107 B. 108 C. 214 D. 215
5. 以下对于哈夫曼树的说法中, 错误的是()
A. 对应一组权值构造出来的哈夫曼树一般不是唯一的
B. 哈夫曼树具有最小的带权路径长度
C. 哈夫曼树中没有度为 1 的结点
D. 哈夫曼树中除了度为 1 的结点外, 还有度为 2 的结点和叶结点
6. 设有 13 个值, 用它们组成一棵赫夫曼树, 则赫夫曼树共有 () 个结点。
A. 13 B. 12 C. 26 D. 25
7. 假设赫夫曼二叉树中只有度为 0 或 2 的结点, 有 n 个叶子的赫夫曼树的结点总数为 ()。
A. 不确定 B. $2n$ C. $2n+1$ D. $2n-1$

【答案解析】

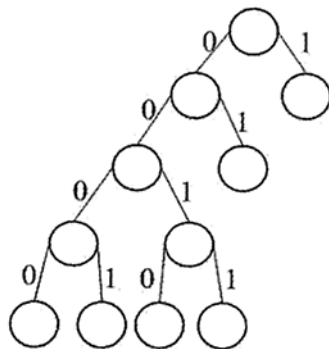
1. A。由哈夫曼树的构造过程可知, 哈夫曼树中只有度为 0 和 2 的结点。在非空二叉树中, 有 $n_0 = n_2 + 1$, 故 $n_2 = n - 1$ 。

另解: n 个结点构造哈夫曼树需要 $n - 1$ 次合并过程, 每次合并新建一个分支结点, 故选 A。

2. C。首先, 3 和 5 构造为一棵子树, 其根权值为 8, 然后该子树与 6 构造为一棵新子树, 根权值为 14, 再后 9 与 12 构造为一棵子树, 最后两棵子树共同构造为一棵哈夫曼树。

3. C。在哈夫曼编码中, 一个编码不能是任何其他编码的前缀。
3 位编码可能是 001, 对应的 4 位编码只能是 0000 和 0001。3 位编码也可能是 0, 对应的 4 位编码只能是 0010 和 0011。若全采开为 0000, 0001, 0010 和 0011。题中问的是最多, 故选 C。

另解: 若哈夫曼编码的长度只允许小于等于 4, 则哈夫曼树的高度最高是 5, 已知一个字符编码为 1, 另一个字符编码是 01, 这说明第二层和第三层各有一个叶子结点, 为使得该树从第 3 层起能够对尽可能多的字符编码, 余下的二叉树应该是满二叉树, 如右图所示, 底层可以有 4 个叶结点, 最多可以再对 4 个字符编码。



4. B。根据上题的结论, 叶子结点数为 $(215 + 1)/2 = 108$, 所以共有 108 个不同的码字。

另解: 在哈夫曼树中只有度为 0 和 2 的结点, 结点总数 $n = n_0 + n_2$, 且 $n_0 = n_2 + 1$, 由题知 $n = 215$, $n_0 = 108$ 。

5. D。哈夫曼树通常是指带权路径长度达到最小的扩充二叉树, 在其构造过程中每次选根的权值最小的两棵树, 一棵作为左子树, 一棵作为右子树, 生成新的二叉树, 新的二叉树根的权值应为其左右两棵子树根结点权值的和。至于谁做左子树, 谁做右子树, 没有限制, 所以构造的哈夫曼树是不唯一的。哈夫曼树只有度为 0 和 2 的结点, 度为 0 的结点是外结点, 带有权值, 没有度为 1 的结点。

6. D。本题考查赫夫曼树的性质。13 个值肯定是 13 个叶子结点, 又因为赫夫曼树没有单分支结点, 所以双分支结点的个数为 $13 - 1 = 12$ 个, 因此总结点数为 $13 + 12 = 25$ 个, 本题选 D。

7. D。本题考查赫夫曼树的性质。赫夫曼树中只有度为 2 与度为 0 的结点, 由二叉树性质 1 可知, n 个叶子结点的赫夫曼树有 $n - 1$ 个度为 2 的结点, 则总结点数为 $2n - 1$ 。因此本题选 D。

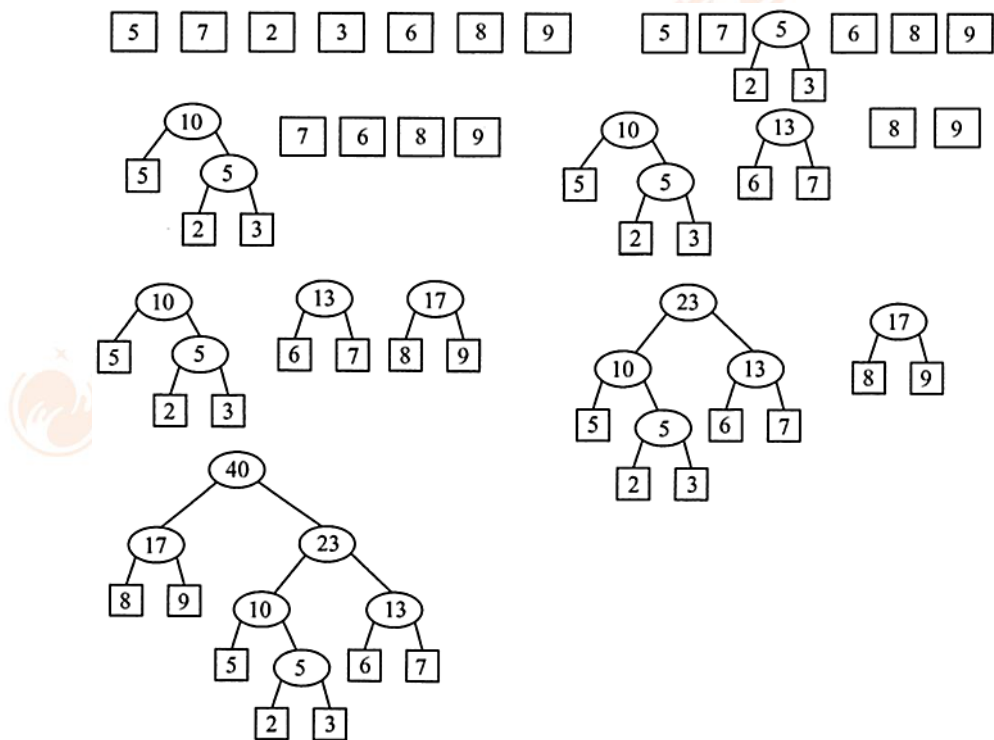
(二) 综合应用题

1. 设给定权集 $w = \{5, 7, 2, 3, 6, 8, 9\}$, 试构造关于 w 的一棵哈夫曼树, 并求其加权路径长度 WPL。

2. 以数据集合 $\{2, 5, 7, 9, 13\}$ 为权值构造一棵赫夫曼树, 并计算其带权路径长度。

【答案解析】

1. 根据哈夫曼树的构造方法, 每次从森林中选取两个根结点值最小的树合并成一棵树, 将原先的两棵树作为左、右子树, 且新根结点的值为左、右孩子关键字之和。构造过程如下图所示。



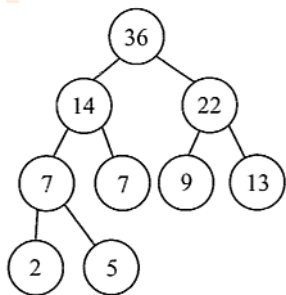
由构造出的哈夫曼树可得 $WPL = (2+3) \times 4 + (5+6+7) \times 3 + (8+9) \times 2 = 108$ 。

注意: 哈夫曼树并不唯一, 但带权路径长度一定是相同的。

2. 分析此题是简单题, 按照赫夫曼二叉树的构造过程直接构造即可, 如下图所示。

$$WPL = (2+5) \times 3 + (7+9+13) \times 2 = 79$$

说明: 在赫夫曼二叉树的构造过程中, 每次需要选取权值最小的两个结点来生成新结点。如果有 3 个以上的结点值相同, 则可以任取两个来生成新结点, 虽然得到的二叉树不同, 但是其带权路径长度总是相同的。



6、图

6.1 图的概念

(一) 选择题

- 图中有关路径的定义是()。
A. 由相邻顶点序偶所形成的序列 B. 由不同顶点所形成的序列
C. 由不同边所形成的序列 D. 上述定义都不是
- 设无向图的顶点个数为 n , 则该图最多有() 条边。
A. $n-1$ B. $n(n-1)/2$ C. $n(n+1)/2$ D. 0 E. n^2
- 含有 n 个顶点的连通无向图, 其边的个数至少为()。
A. $n-1$ B. n C. $n+1$ D. $n \log_2 n$
- 要连通具有 n 个顶点的有向图, 至少需要() 条边。
A. $n-1$ B. n C. $n+1$ D. $2n$
- n 个结点的完全有向图含有边的数目为()。
A. n^2 B. $n(n+1)$ C. $n/2$ D. $n(n-1)$
- 一个有 n 个结点的无向图, 最少有() 个连通分量, 最多有() 个连通分量。
A. 0 B. 1 C. $n-1$ D. n
- 在一个无向图中, 所有顶点的度数之和等于所有边数的() 倍; 在一个有向图中, 所有顶点的入度之和等于所有顶点出度之和的() 倍。
A. $1/2$ B. 2 C. 1 D. 4
- 一个有 n 个顶点和 n 条边的无向图一定是()。
A. 连通的 B. 不连通的 C. 无环的 D. 有环的
- 若从无向图的任意顶点出发 进行一次深度优先搜索即可访问所有顶点, 则该图一定是()。
A. 强连通图 B. 连通图 C. 有回路 D. 一棵树
- 以下关于图的叙述中, 正确的是()。
A. 图与树的区别在于图的边数大于等于顶点数
B. 假设有图 $G = \{V, \{E\}\}$, 顶点 $V' \subseteq V$, $E' \subseteq E$, 则 V' 和 $\{E'\}$ 构成 G 的子图
C. 无向图的连通分量是指无向图中的极大连通子图
D. 图的遍历就是从图中某一顶点出发访遍图中其余顶点
- 以下关于图的叙述中, 正确的是()。
A. 强连通有向图的任何顶点到其他所有顶点都有弧
B. 图的任意顶点的入度等于出度
C. 有向完全图一定是强连通有向图
D. 有向图的边集的子集和顶点集的子集可构成原有向图的子图
- 对于一个有 n 个顶点的图: 若是连通无向图, 其边的个数至少为(); 若是强连通有向图, 其边的个数至少为()。
A. $n-1, n$ B. $n-1, n(n-1)$ C. n, n D. $n, n(n-1)$
- 在有 n 个顶点的有向图中, 每个顶点的度最大可达()。
A. n B. $n-1$ C. $2n$ D. $2n-2$

【答案解析】

1. A。本题考查图的基本概念。在一个图中,一条路径是由相邻顶点序列所形成的序列。因此本题选 A。

2. B。本题考查图基本概念的应用。对于无向图,边数最多的情况就是任意两个顶点之间都有一条边,即 n 个顶点里面任取两个顶点的组合数: $C_n^2 = \frac{n(n-1)}{2}$ 此时的无向图也叫完全无向图。因此本题选 B。

3. A。本题考查图和树基本概念的应用。一个连通图边数最少的情况就是这个图中没有回路的情况,实际上这个图是一棵树。对于树,分支和结点的关系为:分支数比结点数少 1。因此本题应选 A。

4. B。本题考查图基本概念的应用。在有向图中,若从 V_i 到 V_j 有路径,则称 V_i 与 V_j 是连通的。有向图是连通图要求任意两个顶点之间都有路径,即 V_i 到 V_j 有路径,同时 V_j 到 V_i 也有路径。下图所示即为有向图是连通的且边的数目是最少的情况。从图中可以看出,有 n 个顶点的有向图正好需要 n 条边来实现整个图的连通。例如, V_1 到 V_n 有路径 $\langle V_1, V_2 \rangle, \langle V_2, V_3 \rangle, \dots, \langle V_{n-1}, V_n \rangle$, 而 V_n 到 V_1 也有路径 $\langle V_n, V_1 \rangle$ 。因此本题选 B。



5. D。本题考查图基本概念的应用。若图中的每两个顶点之间都存在一条边,则称该图为完全图。完全无向图有 $n(n-1)/2$ 条边,完全有向图有 $n(n-1)$ 条边。因此本题选 D。

6. B,D。本题考查图基本概念的应用。图的连通分量为图的极大连通子图,如果图中没有边,则单个顶点就是一个连通分量,此时有 n 个连通分量。如果图是连通的,则图只有 1 个连通分量,就是它本身。因此本题选 B、D。

7. B,C。本题考查图基本概念的应用。在无向图中,每一条边连接着两个顶点,这两个顶点因这条边而各有一个度,对于整个图,各个顶点度数的总和是图中边数总和的两倍。在有向图中,一条边使一个顶点产生一个出度,则必使另一个顶点产生一个入度,即出度和入度是成对出现的,因此两者总数相等。因此本题选 B、C。

8. D。若一个无向图有 n 个顶点和 $n-1$ 条边,可以使它连通但没有环(即生成树),但再加一条边,在不考虑重边的情形下,则必然会构成环。

9. B。强连通图是有向图,与题意矛盾,选项 A 错误;对无向连通图做一次深度优先搜索,可以访问到该连通图的所有顶点,选项 B 正确;有回路的无向图不一定是连通图,因为回路不一定包含图的所有结点,选项 C 错误;连通图可能是树,也可能存在环,选项 D 错误。

10. C。图与树的区别是逻辑上的区别,而不是边数的区别,图的边数也可能小于树的边数,故选项 A 错;若 E' 中的边对应的顶点不是 V 的元素, V 和 $\{E'\}$ 无法构成图,故选项 B 错;无向图的极大连通子图称为连通分量,选项 C 正确;图的遍历要求每个结点只能被访问一次,且若图非连通,则从某一顶点出发无法访问到其他全部顶点,选项 D 的说法不准确。

11. C。强连通有向图的任何顶点到其他所有顶点都有路径,但未必有弧;无向图任意顶点的入度等于出度,但有向图未必满足;若边集中的某条边对应的某个顶点不在对应的顶点集中,则有向图的边集的子集和顶点集的子集无法构成子图。

12. A。对于连通无向图,边最少即构成一棵树的情形;对于强连通有向图,边最少即构成一个有向环的情形。

13. D。在有向图中,顶点的度等于入度与出度之和。 n 个顶点的有向图中,任意一个顶点最多还可以与其他 $n-1$ 个顶点有一对指向相反的边相连。

(二) 综合应用题

1. 图 G 是一个非连通无向图, 共有 28 条边, 该图至少有多少个顶点?
2. 如何对无环有向图中的顶点号重新安排可使得该图的邻接矩阵中所有的 1 都集中到对角线以上?

【答案解析】

1. 答案: 由于图 G 是一个非连通无向图, 在边数固定时, 顶点数最少的情况是该图由两个连通子图构成, 且其中之一只含一个顶点, 另一个为完全图。其中只含一个顶点的子图没有边, 另一个完全图的边数为 $n(n-1)/2 = 28$, 得 $n = 8$ 。所以该图至少有 $1 + 8 = 9$ 个顶点。

2. 解答: 按各顶点的出度进行排序。 n 个顶点的有向图, 其顶点的最大出度是 $n-1$, 最小出度为 0。这样排序后, 出度最大的顶点编号为 1, 出度最小的顶点编号为 n 。之后, 进行调整, 即只要存在弧 $\langle i, j \rangle$, 就不管顶点 j 的出度是否大于顶点 i 的出度, 都把 i 编号在顶点 j 的编号之前, 因为只有 $i \leq j$, 弧 $\langle i, j \rangle$ 对应的 1 才能出现在邻接矩阵的上三角。

通过后面小节的学习, 会发现采用拓扑排序并依次编号是一种更为简便的方法。

6.2 图的存储

(一) 选择题

1. 关于图的存储结构, () 是错误的。
 - A. 使用邻接矩阵存储一个图时, 在不考虑压缩存储的情况下, 所占用的存储空间大小只与图中的顶点数有关, 与边数无关
 - B. 邻接表只用于有向图的存储, 邻接矩阵适用于有向图和无向图
 - C. 若一个有向图的邻接矩阵的对角线以下的元素为 0, 则该图的拓扑序列必定存在
 - D. 存储无向图的邻接矩阵是对称的, 故只需存储邻接矩阵的下(或上)三角部分
2. 若图的邻接矩阵中主对角线上的元素皆为 0, 其余元素全为 1, 则可以断定该图一定()。
 - A. 是无向图
 - B. 是有向图
 - C. 是完全图
 - D. 不是带权图
3. 在含有 n 个顶点和 e 条边的无向图的邻接矩阵中, 零元素的个数为 ()。
 - A. e
 - B. $2e$
 - C. $n^2 - e$
 - D. $n^2 - 2e$
4. 以下关于图的存储结构的叙述中, 正确的是()。
 - A. 一个图的邻接矩阵表示唯一, 邻接表表示唯一
 - B. 一个图的邻接矩阵表示唯一, 邻接表表示不唯一
 - C. 一个图的邻接矩阵表示不唯一, 邻接表表示唯一
 - D. 一个图的邻接矩阵表示不唯一, 邻接表表示不唯一
5. 用邻接表法存储图所用的空间大小()。
 - A. 与图的顶点数和边数有关
 - B. 只与图的边数有关
 - C. 只与图的顶点数有关
 - D. 与边数的平方有关
6. 若邻接表中有奇数个边表结点, 则 ()。
 - A. 图中有奇数个结点
 - B. 图中有偶数个结点
 - C. 图为无向图
 - D. 图为有向图
7. 在有向图的邻接表存储结构中, 顶点 v 在边表中出现的次数是 ()。
 - A. 顶点 v 的度
 - B. 顶点 v 的出度

- C. 顶点 v 的入度
- D. 依附于顶点 v 的边数
8. n 个顶点的无向图的邻接表最多有 () 个边表结点。
- A. n^2 B. $n(n-1)$ C. $n(n+1)$ D. $n(n-1)/2$
9. 假设有 n 个顶点、 e 条边的有向图用邻接表表示, 则删除与某个顶点 v 相关的所有边的时间复杂度为 ()。
- A. $O(n)$ B. $O(e)$ C. $O(n+e)$ D. $O(ne)$
10. 对邻接表的叙述中, () 是正确的。
- A. 无向图的邻接表中, 第 i 个顶点的度为第 i 个链表中结点数的两倍
- B. 邻接表比邻接矩阵的操作更简便
- C. 邻接矩阵比邻接表的操作更简便
- D. 求有向图结点的度, 必须遍历整个邻接表
11. 邻接多重表是 () 的存储结构。
- A. 无向图 B. 有向图 C. 无向图和有向图 D. 都不是
12. 十字链表是 () 的存储结构。
- A. 无向图 B. 有向图 C. 无向图和有向图 D. 都不是
13. () 的邻接矩阵是对称矩阵。
- A. 有向图 B. 无向图 C. AOV 网 D. AOE 网
14. 对于一个具有 n 个顶点的无向图, 若采用邻接矩阵表示, 则该矩阵的大小是 ()。
- A. n B. $(n-1)^2$ C. $n-1$ D. n^2
15. 无向图的邻接矩阵是一个 ()。
- A. 对称矩阵 B. 零矩阵 C. 上三角矩阵 D. 对角矩阵

【答案解析】

1. B。n 个顶点的图，若采用邻接矩阵表示，不考虑压缩存储，则存储空间大小为 $O(n^2)$ ，A 正确。邻接表可用于存储无向图，只是把每条边都视为两条方向相反的有向边，因此需要存储两次，B 错误。由于邻接矩阵中对角线以下的元素全为 0，若存在 $\langle i, l \rangle$ ，则必然有 $i < j$ ，由传递性可知图中路径的顶点编号是依次递增的，假设存在环 $k \rightarrow \dots \rightarrow j \rightarrow k$ ，由题设可知 $k < j < k$ ，矛盾，故不存在环，拓扑序列必定存在，C 正确。选项 D 显然正确。

注意：若邻接矩阵对角线以下（或以上）的元素全为 0，则图中必然不存在环，即拓扑序列一定存在，但这并不能说明拓扑序列是唯一的。

2. C。除主对角线上的元素外，其余元素全为 1，说明任意两个顶点之间都有边相连，因此该图一定是完全图。

3. D。无向图的邻接矩阵中，矩阵大小为 n^2 ，非零元素的个数为 $2e$ ，故零元素的个数 $n^2 - 2e$ 。读者应掌握此题的变体，即当无向图变为有向图时，能够求出零的个数和非零的个数。

4. B。邻接矩阵表示唯一是因为图中边的信息在矩阵中有确定的位置,邻接表不唯一是因为邻接表的建立取决于读入边的顺序和边表中的插入算法。

5. A。邻接表存储时，顶点数 n 决定了顶点表的大小，边数 e 决定了边表的个数，且每条边存储两次，总存储空间为 $O(n+2e)$ ，故选 A。而邻接矩阵只与图的顶点数有关，为 $O(n^2)$ 。

6. D。无向图采用邻接表表示时，每条边存储两次，所以其边表结点的个数为偶数。题中边表结点为奇数个，故必然是有向图，且有奇数条边。

7. C。题中的边表是不包括顶点表的。因为任何顶点 u 对应的边表中存放的都是以 u 为起点

的边所对应的另一个顶点 v 。从而 v 在边表中出现的次数也就是它的入度。

8. B。 n 个顶点的无向图最多有 $n(n-1)/2$ 条边，每条边在邻接表中存储两次，所以边表结点最多为 $n(n-1)$ 个。

9. C。删除与某顶点 v 相关的所有边的过程如下：先删除下标为 v 的顶点表结点的单链表，出边数最多为 $n-1$ ，对应时间复杂度为 $O(n)$ ；再扫描所有边表结点，删除所有的顶点 v 的入边，对应时间复杂度为 $O(e)$ 。故总的时间复杂度为 $O(n+e)$ 。

10. D。无向图的邻接表中，第 i 个顶点的度为第 i 个链表中的结点数，故 A 错。邻接表和邻接矩阵对于不同的操作各有优势，B 和 C 都不准确。有向图结点的度包括出度和入度，对于出度，需要遍历顶点表结点所对应的边表；对于入度，则需要遍历剩下的全部边表，故 D 正确。

11. A。邻接多重表是无向图的存储结构，选 A。

12. B。十字链表是有向图的存储结构，选 B。

13. B。本题考查图的存储结构。无向图采用邻接矩阵做存储结构，则邻接矩阵一定是对称的，无向图中两顶点间的边相当于有向图中两顶点间的两条边，即 (V_i, V_j) 等价于 $\langle V_i, V_j \rangle$ 并上 $\langle V_j, V_i \rangle$ ，因此反映在邻接矩阵中必定有：如果 $a_{ij}=1$ ，则 $a_{ji}=1$ 。A、C、D 3 个选项都是有向图，邻接矩阵不一定对称，因此本题选 B。

14. D。本题考查图的邻接矩阵存储结构。图的邻接矩阵规模为图中顶点数的二次，方顶点有 n 个，因此邻接矩阵大小为 n^2 。

15. A。本题考查图的邻接矩阵存储结构。图的邻接矩阵存储结构中，行中 1 的个数为该行所对应顶点的出度，列中 1 的个数为该列所对应顶点的入度。对于无向图，与顶点相连的每一条边相当于同时存在一条入边和一条出边，即无向图相当于每个顶点的入度等于出度的有向图。反映在邻接矩阵中即为每个顶点所对应的行中 1 的个数等于该顶点所对应的列中 1 的个数，这样的邻接矩阵必为对称矩阵。

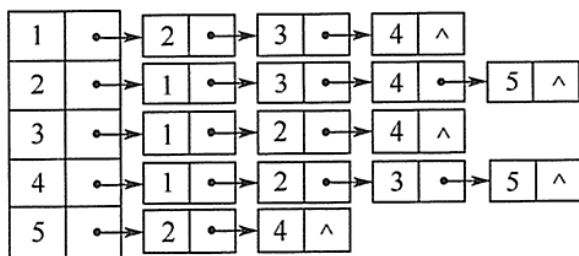
(二) 综合应用题

1. 用邻接矩阵表示图时，矩阵元素的个数与顶点个数是否有关？与边的条数是否有关？

2. 已知带权有向图 G 的邻接矩阵如下图所示，请画出该带权有向图 G 。

0	15	2	12	∞	∞	∞
∞	0	∞	∞	6	∞	∞
∞	∞	0	∞	8	4	∞
∞	∞	∞	0	∞	∞	3
∞	∞	∞	∞	0	∞	9
∞	∞	∞	5	∞	0	10
∞	4	∞	∞	∞	∞	0

3. 设图 $G=(V, E)$ 以邻接表存储，如下图所示。画出其邻接矩阵存储及图 G 。



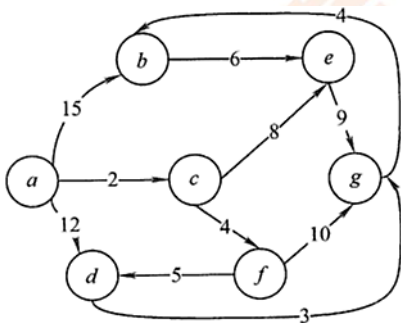
4. 对 n 个顶点的无向图和有向图，分别采用邻接矩阵和邻接表表示时，试问：

- (1) 如何判别图中有多少条边？
- (2) 如何判别任意两个顶点 i 和 j 是否有边相连？
- (3) 任意一个顶点的度是多少？

【答案解析】

1、答：用邻接矩阵表示图时，矩阵元素个数为 n^2 ，这个表达式中只涉及一个自变量 n ， n 是图中顶点个数，因此矩阵元素只与顶点个数有关，而与边数无关。

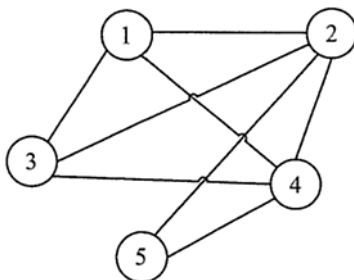
2. 解答：带权有向图 G 如下图所示：



3. 解答：其邻接矩阵存储如下所示：

$$\begin{bmatrix} 0 & 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 & 1 \\ 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 \end{bmatrix}$$

在邻接表中，每条边存储了 2 次，在没有特殊说明时，通常默认其为无向图（当然，无向图也可视为具有对边的有向图）。该邻接表对应的图 G 如下图所示。



4. 解答: (1) 对于邻接矩阵表示的无向图, 边数等于矩阵中 1 的个数除以 2; 对于邻接表表示的无向图, 边数等于边结点的个数除以 2。对于邻接矩阵表示的有向图, 边数等于矩阵中 1 的个数; 对于邻接表表示的有向图, 边数等于边结点的个数。

(2) 在邻接矩阵表示的无向图或有向图中, 对于任意两个顶 i 和 j , 邻接矩阵中 $arcs[i][j]$ 或 $arcs[j][i]$ 为 1 表示有边相连, 否则表示无边相连。在邻接表表示的无向图或有向图中, 对于任意两个顶点 i 和 j , 若从顶点表结点 i 出发找到编号为 j 的边表结点或从顶点表结点 j 出发找到编号为 i 的边表结点, 表示有边相连; 否则为无边相连。

(3) 对于邻接矩阵表示的无向图, 顶点 i 的度等于第 i 行中 1 的个数; 对于邻接矩阵表示的有向图, 顶点 i 的出度等于第 i 行中 1 的个数; 入度等于第 i 列中 1 的个数; 度数等于它们的和。对于邻接表表示的无向图, 顶点 i 的度等于顶点表结点 i 的单链表中边表结点的个数; 对于邻接表表示的有向图, 顶点 i 的出度等于顶点表结点 i 的单链表中边表结点的个数, 顶点 i 的入度等于邻接表中所有编号为 i 的边表结点数; 度数等于入度与出度之和。

6.3 图的遍历

(一) 选择题

1. 对于一个非连通无向图 G , 采用深度优先遍历访问所有顶点, 在 $DFS\text{Traverse}$ 函数 (见考点讲解 DFS 部分) 中调用 DFS 的次数正好等于 ()。

- A. 顶点数 B. 边数 C. 连通分量数 D. 不确定

2. 对一个有 n 个顶点、 e 条边的图采用邻接表表示时, 进行 DFS 遍历的时间复杂度为 (), 空间复杂度为 (); 进行 BFS 遍历的时间复杂度为 (), 空间复杂度为 ()。

- A. $O(n)$ B. $O(e)$ C. $O(n+e)$ D. $O(1)$

3. 对有 n 个顶点、 e 条边的图采用邻接矩阵表示时, 进行 DFS 遍历的时间复杂度为 (), 进行 BFS 遍历的时间复杂度为 ()。

- A. $O(n^2)$ B. $O(e)$ C. $O(n+e)$ D. $O(e^2)$

4. 无向图 $G=(V, E)$, 其中: $V=\{a, b, c, d, e, f\}$, $E=\{(a, b), (a, e), (a, c), (b, e), (c, f), (f, d), (e, d)\}$, 以顶点 a 为源点, 对该图进行深度优先遍历, 得到的顶点序列正确的是 ()。

- A. a, b, e, c, d, f B. a, c, f, e, b, d
C. a, e, b, c, f, d D. a, e, d, f, c, b

- A. a, b, e, c, d, f B. a, c, f, e, b, d C. a, e, b, c, f, d D. a, e, d, f, c, b

5. 无向图 $G=(V, E)$, 其中 $V=\{a, b, c, d, e, f\}$, $E=\{(a, b), (a, e), (a, c), (b, e), (c, f), (f, d), (e, d)\}$ 。对该图进行深度优先遍历, 不能得到的序列是 ()。

- A. $acfdbe$ B. $aebdfc$ C. $aedfcb$ D. $abcedf$

6. 判断有向图中是否存在回路, 除可以利用拓扑排序外, 还可以利用 ()。

- A. 求关键路径的方法 B. 求最短路径的 Dijkstra 算法
C. 深度优先遍历算法 D. 广度优先遍历算法

7. 设无向图 $G=(V, E)$ 和 $G'=(V', E')$, 若 G' 是 G 的生成树, 则下列说法中错误的是 ()。

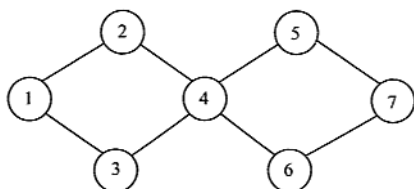
- A. G' 为 G 的子图 B. G' 为 G 的连通分量
C. G' 为 G 的极大连通子图且 $V=V'$ D. G' 是 G 的一个无环子图

8. 图的广度优先生成树的树高比深度优先生成树的树高 ()。

- A. 小或相等 B. 小 C. 大或相等 D. 大

9. 对有 n 个顶点、 e 条边且使用邻接表存储的有向图进行广度优先遍历, 其算法的时间复杂度是

- ()。
- A. $O(n)$ B. $O(e)$ C. $O(n+e)$ D. $O(ne)$
10. 下列说法不正确的是 ()。
- A. 图的遍历是从给定的源点出发, 每一个顶点仅被访问一次
B. 图的深度优先遍历不适用于有向图
C. 遍历的基本算法有两种: 深度优先搜索遍历和广度优先搜索遍历
D. 图的深度遍历是一个递归过程
11. 如果从无向图的任一顶点出发, 进行一次深度优先搜索即可访问所有顶点, 则该图一定是 ()。
- A. 完全图 B. 连通图 C. 有回路 D. 一棵树
12. 图的深度优先遍历算法类似于二叉树的 () 算法。
- A. 先序遍历 B. 中序遍历 C. 后序遍历 D. 层次遍历
13. 对图进行广度优先搜索遍历类似于二叉树的 () 算法。
- A. 先序遍历 B. 中序遍历 C. 后序遍历 D. 层次遍历
14. 对于下图进行从顶点 1 开始的深度优先搜索遍历, 可得到的顶点访问序列是 ()。
- A. 1, 2, 4, 3, 5, 7, 6 B. 1, 2, 4, 3, 5, 6, 7
C. 1, 2, 4, 5, 6, 3, 7 D. 1, 2, 4, 3, 5, 6, 7



15. 对上题 (14 题) 所示的图进行从顶点 1 开始的广度优先搜索遍历, 可得到的顶点访问序列为 ()。
- A. 1, 3, 2, 4, 5, 6, 7 B. 1, 2, 4, 3, 5, 6, 7
C. 1, 2, 4, 5, 6, 3, 7 D. 2, 5, 1, 4, 7, 3, 6

【答案解析】

1. C。DFS (或 BFS) 可用来计算图的连通分量数, 因为一次遍历必然会将一个连通图中的所有顶点都访问到, 而对于已被访问的顶点将不再调用 DFS, 故计算图的连通分量数正好是 DFSTraverse () 中 DFS 被调用的次数。

2. C、A、C、A。深度优先遍历时, 每个顶点表结点和每个边表结点均查找一次, 每个顶点递归调用一次, 要借助一个递归工作栈; 而广度优先遍历时, 也是每个顶点表结点和每个边表结点均查找一次, 每个顶点进入队列一次。故都是选 C, A。

3. A、A。采用邻接矩阵表示时, 查找一个顶点所有出边的时间复杂度为 $O(n)$, 共有 n 个顶点, 故时间复杂度均为 $O(n^2)$ 。

4. D。本题考查图的遍历。依照题目给出的顶点和边的信息将图画出, 然后根据深度优先搜索的过程可知序列 a, e, d, f, c, b 是深度优先搜索的结果, 因此答案选 D。

5. D。画出 V 和 E 对应的图 G, 然后根据搜索算法求解。

这里应注意: 为什么本题序列是不唯一的, 而上题却是唯一的呢?

因为上题给出了具体的存储结构, 此时就必须按照算法的过程来执行, 每个顶点的邻接点的顺序已固定, 但本题中每个顶点的邻接点的顺序是非固定的。

6. C。利用深度优先遍历可以判断图 G 中是否存在回路。对于无向图来说,若深度优先遍历过程中遇到了回边,则必定存在环;对于有向图来说,这条回边可能是指向深度优先森林中另一棵生成树上的顶点的弧;但是,从有向图的某个顶点 v 出发进行深度优先遍历时,若在 $\text{DFS}(v)$ 结束之前出现一条从顶点 u 到顶点 v 的回边,且 u 在生成树上是 v 的子孙,则有向图必定存在包含顶点 v 和顶点 u 的环。

7. B。连通分量是无向图的极大连通子图,其中极大的含义是将依附于连通分量中顶点的所有边都加上,所以连通分量中可能存在回路,这样就不是生成树了。

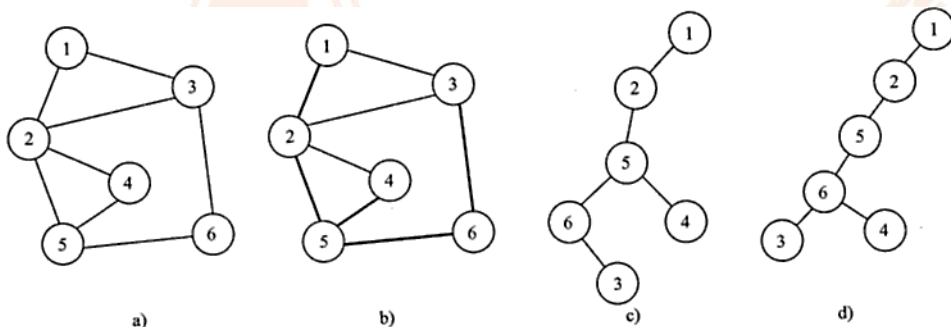
8. A。对于无向图的广度优先搜索生成树,起点到其他顶点的路径是图中对应的最短路径,即所有生成树中树高最小。此外,深度优先总是尽可能“深”地搜索图,因此其路径也尽可能长,故深度优先生成树的树高总是大于等于广度优先生成树的树高。

9. C。广度优先遍历需要借助队列实现。采用邻接表存储方式对图进行广度优先遍历时,每个顶点均需入队一次(顶点表遍历),故时间复杂度为 $O(n)$,在搜索所有顶点的邻接点的过程中,每条边至少访问一次(出边表遍历),所以时间复杂度为 $O(e)$,算法总的时间复杂度为 $O(n+e)$ 。

10. B。本题考查图的遍历。本章中所讲两种图的遍历方法——广度优先搜索遍历和深度优先搜索遍历,对于任何图都是适用的,并没有限制是针对有向图还是无向图的遍历。

11. B。本题考查图的遍历及连通图的性质。图的一次深度优先搜索遍历,可以遍历完图中一个连通分量中所有的顶点。如果图是连通的,则图只含有一个连通分量,即图本身,这样一次深度优先搜索遍历即可遍历完图中所有的顶点。因此本题选 B。无向完全图相当于在连通图上加上了更严格的条件,即任意两个顶点间都存在边。对于满足本题的要求不需要完全图,条件到达连通图的强度就足够了。

12. A。本题考查图与二叉树遍历方式的理解与联系。下图 a)所示是一个无向图,下图 b)所示为对图 a) 进行深度优先搜索的过程,得深度优先搜索遍历序列为 1, 2, 5, 6, 3, 4。下图 c)所示为对应的深度优先搜索生成树,将其转化为二叉树如下图 d)所示。对图 d)进行先序遍历,可以得到一个与原图进行深度优先搜索遍历相同的序列: 1, 2, 5, 6, 3, 4。这就是图的深度优先搜索与树的先序遍历的相似之处之一。



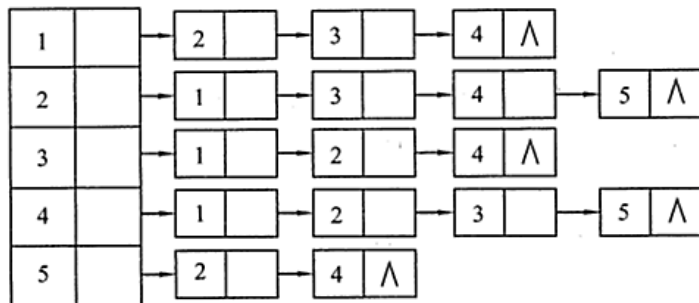
13. D。本题考查图与二叉树遍历方式的理解与联系。此题类似于第 29 题, 答案为 D。

14. A。本题考查图的深度优先搜索遍历。图的深度优先搜索遍历并不一定唯一,因此解此类题目的快速做法为:当遍历完图中某一顶点之后,参照题目中的选项来选择下一顶点进行访问。选项中对于 1、2 两顶点已经确定。从 2 开始,下一步只有一个顶点 4 可以选择,因此排除选项 D。访问完 4 之后,待选顶点为 3、5、6,此时 A、B、C 3 选项都可以,因 A、B 两选项在 4 之后都选择了 3 进行访问(如果访问 3 是错误的,则可以同时排除两个选项),因此此步访问 3。3 访问过之后,3 的邻接点都已经被访问,退回 4,此时只有 5、6 两个待选顶点,同上一步,选择 5 进行访问。5 访问完之后,只有 7 可供选择,而 B 选项中在 5 之后却选择了访问 6,因此排除 B 选项。A 选项中选择了 7 进行访问,满足要求。之后访问最后一个顶点 6,完成遍历。

15. A。本题考查图的广度优先搜索遍历。本题做法类似于第 14 题。

(二) 综合应用题

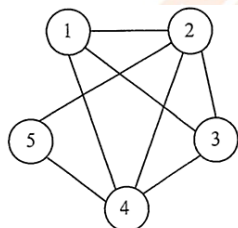
1. 设 $G=(V, E)$ 以邻接表存储, 如下图所示, 试画出图的深度优先和广度优先生成树。



【答案解析】

1、分析: 在图的遍历过程中, 由图中顶点作为树的结点, 所经历的边作为树的分支, 所构成的树叫作按某种方式遍历生成树, 有深度优先生成树和广度优先生成树。因此要画出图的生成树, 只需按照某种方式遍历图, 然后记下遍历过程中所经历的边, 根据边及其所连顶点就可以画出生成树。

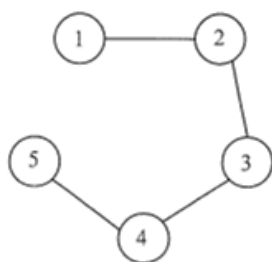
解: 根据邻接表可画出下图。



①深度优先遍历序列, 以边来表示为:

$\langle 1, 2 \rangle, \langle 2, 3 \rangle, \langle 3, 4 \rangle, \langle 4, 5 \rangle$

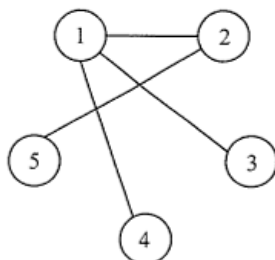
由边的序列可画出深度优先生成树, 如下图所示:



②广度优先遍历序列, 以边来表示为:

$\langle 1, 2 \rangle, \langle 1, 3 \rangle, \langle 1, 4 \rangle, \langle 2, 5 \rangle$

由边的序列可画出广度优先生成树, 如下图所示。



6.4 图的应用

(一) 选择题

1. 设有无向图 $G=(V, E)$ 和 $G'=(V', E')$, 若 G' 是 G 的生成树, 则下列不正确的是 ()。
 - I. G' 为 G 的连通分量
 - II. G' 为 G 的无环子图
 - III. G' 为 G 的极小连通子图且 $V=V'$

A. I、II B. 只有 III C. II、III D. 只有 I
2. 若具有 n 个顶点的图是一个环, 则它有 () 棵生成树。

A. n^2 B. n C. $n-1$ D. 1
3. 若一个具有 n 个顶点、 e 条边的无向图是一个森林, 则该森林中必有 () 棵树。

A. n B. e C. $n-e$ D. 1
4. 任何一个无向连通图的最小生成树 ()。

A. 有一棵或多棵 B. 只有一棵 C. 一定有多棵 D. 可能不存在
5. 用 Prim 算法和 Kruskal 算法构造图的最小生成树, 所得到的最小生成树 ()。

A. 相同 B. 不相同

C. 可能相同, 可能不同 D. 无法比较
6. 以下叙述中, 正确的是 ()。
 - A. 只要无向连通图中没有权值相同的边, 则其最小生成树唯一
 - B. 只要无向图中有权值相同的边, 则其最小生成树一定不唯一
 - C. 从 n 个顶点的连通图中选取 $n-1$ 条权值最小的边, 即可构成最小生成树
 - D. 设连通图 G 含有 n 个顶点, 则含有 n 个顶点、 $n-1$ 条边的子图一定是 G 的生成树
7. 以下叙述中, 正确的是 ()。
 - A. 最短路径一定是简单路径
 - B. Dijkstra 算法不适合求有回路的带权图的最短路径
 - C. Dijkstra 算法不适合求任意两个顶点的最短路径
 - D. Floyd 算法求两个顶点的最短路径时, $path_{k-1}$ 一定是 $path_k$ 的子集
8. 以下关于拓扑排序的说法中, 错误的是 ()。
 - I. 若某有向图存在环路, 则该有向图一定不存在拓扑排序
 - II. 在拓扑排序算法中为暂存入度为零的顶点, 可以使用栈, 也可以使用队列
 - III. 若有向图的拓扑有序序列唯一, 则图中每个顶点的入度和出度最多为 1

A. I、III B. II、III C. II D. III
9. 若一个有向图的顶点不能排成一个拓扑序列, 则判定该有向图 ()。

A. 含有多个出度为 0 的顶点 B. 是个强连通图

C. 含有多个入度为 0 的顶点 D. 含有顶点数大于 1 的强连通分量
10. 若一个有向图具有有序的拓扑排序序列, 则它的邻接矩阵必定为 ()。

A. 对称 B. 稀疏 C. 三角 D. 一般
11. 下列关于图的说法中, 正确的是 ()。
 - I. 有向图中顶点 V 的度等于其邻接矩阵中第 V 行中 1 的个数
 - II. 元向图的邻接矩阵一定是对称矩阵, 有向图的邻接矩阵一定是非对称矩阵
 - III. 在图 G 的最小生成树 G_1 中, 某条边的权值可能会超过未选边的权值

- IV. 若有向无环图的拓扑序列唯一, 则可以唯一确定该图
- A. I、II 和 III B. III 和 IV C. III D. IV
12. 下面关于求关键路径的说法中, 不正确的是()。
- A. 求关键路径是以拓扑排序为基础的
- B. 一个事件的最早发生时间与以该事件为始的弧的活动的最早开始时间相同
- C. 一个事件的最迟发生时间是以该事件为尾的弧的活动的最迟开始时间与该活动的持续时间的差
- D. 关键活动一定位于关键路径上
13. 使用 DFS 算法递归地遍历一个无环有向图, 并在退出递归时输出相应顶点, 这样得到的顶点序列是()。
- A. 逆拓扑有序 B. 拓扑有序 C. 无序的 D. 都不是
14. (多选题) () 方法可以判断出一个有向图是否有环。
- A. 深度优先遍历 B. 拓扑排序 C. 求最短路径 D. 求关键路径
15. 下列关于广度优先算法的说法中, 正确的是()。
- I. 当各边的权值相等时, 广度优先算法可以解决单源最短路径问题
- II. 当各边的权值不等时, 广度优先算法可用来解决单源最短路径问题
- III. 广度优先遍历算法类似于树中的后序遍历算法
- IV. 实现图的广度优先算法时, 使用的数据结构是队列
- A. I、IV B. II、III、IV C. II、IV D. I、III、IV
16. 当各边上的权值() 时, BFS 算法可用来解决单源最短路径问题。
- A. 均相等 B. 均互不相等 C. 不一定相等
17. 求解最短路径的弗洛伊德算法的时间复杂度为()。
- A. $O(n)$ B. $O(n+e)$ C. $O(n^2)$ D. $O(n^3)$
18. 若一个有向图的邻接矩阵中, 主对角线以下的元素均为零, 则该图的拓扑有序序列()。
- A. 一定存在 B. 不一定存在 C. 一定不存在
19. 一个有向无环图的拓扑排序序列() 是唯一的。
- A. 一定 B. 不一定
20. 在有向图 G 的拓扑序列中, 若顶点 v_i 在顶点 v_j 之前, 则下列情形不可能出现的是()。
- A. G 中有边 $\langle v_i, v_j \rangle$ B. G 中有一条从 v_i 到 v_j 的路径
- C. G 中没有边 $\langle v_i, v_j \rangle$ D. G 中有一条从 v_j 到 v_i 的路径
21. 关键路径是事件结点图中()。
- A. 从源点到汇点的最长路径 B. 从源点到汇点的最短路径
- C. 最长回路 D. 最短回路
22. 对于含有 n 个顶点的带权连通图, 它的最小生成树是指图中任意一个()。
- A. 由 $n-1$ 条权值最小的边构成的子图
- B. 由 $n-1$ 条权值之和最小的边构成的子图
- C. 由 $n-1$ 条权值之和最小的边构成的连通子图
- D. 由 n 个顶点构成的边的权值之和最小的连通子图
23. 若一个有向图中的顶点不能排成一个拓扑序列, 则断定该有向图()。
- A. 含有多个出度为 0 的顶点 B. 是个强连通图
- C. 含有多个入度为 0 的顶点 D. 含有顶点数目大于 1 的强连通分量

【答案解析】

1. D。一个连通图的生成树是一个极小连通子图，显然它是无环的，故 II、III 正确。极大连通子图称为连通分量，G' 连通但非连通分量。这里再补充一下“极大连通子图”：如果图本来就不是连通的，那么每个子部分若包含它本身的所有顶点和边，则它就是极大连通子图。

2. B。因为 n 个顶点构成的环共有 n 条边，去掉其中任意一条便是一棵生成树，所以共有 n 种情况。

3. C。 n 个结点的树有 $n-1$ 条边，假设森林中有 x 棵树，将每棵树的根连到一个添加的结点，则成为一棵树，结点数是 $n+1$ ，边数是 $e+x$ ，从而可知 $x=n-e$ 。

另解：设森林中有 x 棵树，则再用 $x-1$ 条边就能把所有的树连接成一棵树，此时，边数+1=顶点数，即 $e+(x-1)+1=n$ ，故 $x=n-e$ 。

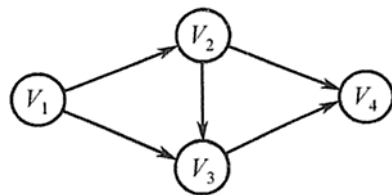
4. A。当无向连通图存在权值相同的多条边时，最小生成树可能是不唯一的；另外，由于这是一个无向连通图，故最小生成树必定存在，从而选 A。

5. C。由于无向连通图的最小生成树不一定唯一，所以用不同算法生成的最小生成树可能不同，但当无向连通图的最小生成树唯一时，不同算法生成的最小生成树必定是相同的。

6. A。选项 A 显然正确；选项 B，若无向图本身就是一棵树，则最小生成树就是它本身，这时就是唯一的；选项 C，选取的 $n-1$ 条边可能构成回路；选项 D，含有 n 个顶点、 $n-1$ 条边的子图可能构成回路，也可能不连通。

7. A。选项 A 正确，见严蔚敏的教材《数据结构》。Dijkstra 算法适合求解有回路的带权图的最短路径，也可以求任意两个顶点的最短路径，不适合求带负权值的最短路径问题。在用 Floyd 算法求两个顶点的最短路径时，当最短路径发生更改时， $path_{k+1}$ 就不是 $path_k$ 的子集。

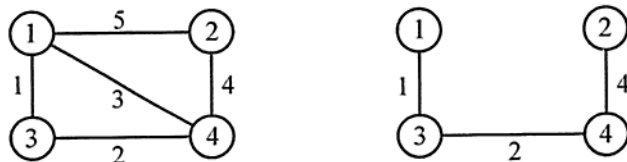
8. D。I 中，对于一个存在环路的有向图，使用拓扑排序算法运行后，肯定会出现有环的子图，在此环中无法再找到入度为 0 的结点，拓扑排序也就进行不下去。II 中，注意，若两个结点之间不存在祖先或子孙关系，则它们在拓扑序列中的关系是任意的（即前后关系任意），因此使用栈和队列都可以，因为进栈或队列的都是入度为 0 的结点，此时入度为 0 的所有结点是没有关系的。III 是难点，若拓扑有序序列唯一，则很自然地让人联想到一个线性的有向图（错误），下图的拓扑序列也是唯一的，但度却不满足条件。



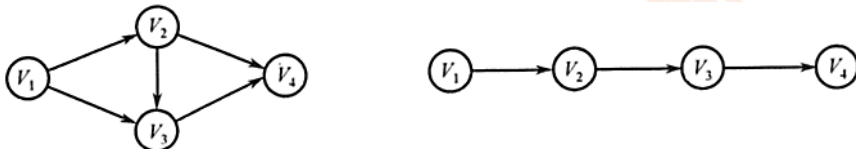
9. D。一个有向图中的顶点不能排成一个拓扑序列，表明其中存在一个顶点数目大于 1 的回路（环，）该回路构成一个强连通分量，从而答案选 D。

10. C。此题一直以来争议较大，因为有些书中漏掉了“有序”二字。可以证明，对有向图中的顶点适当地编号，使其邻接矩阵为三角矩阵且主对角元全为零的充分必要条件是，该有向图可以进行拓扑排序。若这个题目把“有序”二字去掉，显然应该选 D。但此题题干中已经指出是“有序的拓扑序列”，因此应该选 C。需要注意的是，若一个有向图的邻接矩阵为三角矩阵（对角线上元素为 0），则图中必不存在环，因此其拓扑序列必然存在。

11. C。有向图邻接矩阵的第 V 行中 1 的个数是顶点 V 的出度，而有向图中顶点的度为入度与出度之和，I 错。无向图的邻接矩阵一定是对称矩阵，但当有向图中任意两个顶点之间有边相连，且是两条方向相反的有向边（无向图也可视为有两条方向相反的有向边的特殊有向图）时，有向图的邻接矩阵也是一个对称矩阵，II 错。最小生成树中的 $n-1$ 条边并不能保证是图中权值最小的 $n-1$ 条边，因为权值最小的 $n-1$ 条边并不一定能使图连通。在下图中，左图的最小生成树如右图所示，权值为 3 的边并不在其最小生成树中。



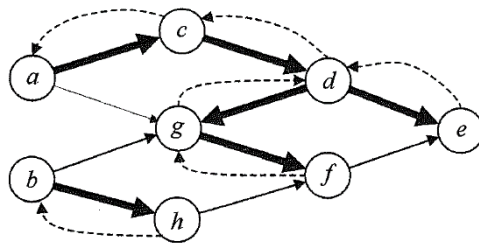
有向无环图的拓扑序列唯一并不能唯一确定该图。在下图所示的两个有向无环图中，拓扑序列都为 V_1, V_2, V_3, V_4 ，错。注意，很多辅导书对该命题的判断是错误的。



12. C。一个事件的最迟发生时间等于 $\text{Min}\{\text{以该事件为尾的弧的活动的最迟开始时间, 最迟结束时间与该活动的持续时间的差}\}$ 。

13. A。对一个有向图做深度优先遍历，并未专门判断有向图是否有环（有向回路）存在，无论图中是否有环，都得到一个顶点序列。若无环，在退出递归过程中输出的应是逆拓扑有序序列。对有向无环图利用深度优先搜索进行拓扑排序的例子如下：

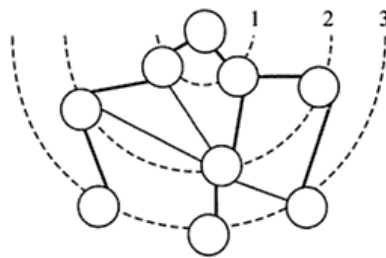
如下图所示，退出 DFS 栈的顺序为 $efgdcahb$ ，此图的一个拓扑序列为 $bhacd gfe$ 。该方法的每一步均是先输出当前无后继的结点，即对每个结点 V ，先递归地求出 v 的每个后继的拓扑序列。对于一个 AOV 网，按此方法输出的序列是一个逆拓扑序列。



14. A, B。本题考查深度优先搜索遍历和拓扑排序。用两者都可以判断有向图中是否有回路。用深度优先搜索的方法，如果从有向图上某个顶点 v 出发的遍历，在 $\text{dfs}(v)$ 结束之前出现一条从顶点 j 到 v 的边，由于 j 在生成树上是 v 的子孙，则图中必定存在包含 v 和 j 的环，因此 A 正确。用拓扑排序的方法，在拓扑排序过程中，每次要删去一个没有前驱的顶点，如果最后图中所有的顶点都被删除，则表示没有环，否则有环，因此 B 正确。

15. A。广度优先搜索以起始结点为中心，一层一层地向外层扩展遍历图的顶点，因此无法考虑到边权值，只适合求边权值相等的图的单源最短路径。广度优先搜索相当于树的层序遍历，III 错误。广度优先搜索需要用到队列，深度优先搜索需要用到栈，IV 正确。

16. A。本题考查广度优先搜索遍历。对于无权图，广度优先搜索总是按照距离源点由近到远来遍历图中每个顶点（这里的距离是指当前顶点到源点路径上顶点的个数），如下图所示。图中各顶点分布在 3 个层上，同一层上的顶点距离源点的距离是相同的。广度优先搜索就是沿着从 1 到 3 的层次顺序来遍历各个顶点，并在遍历的过程中形成了一棵树，称为广度优先搜索生成树，树的分支总是连接不同层上的顶点，如图中粗线所连。由源点沿生成树分支到达其余顶点的距离都是最近的（可以用层号来描述其远近）。对于无权图，可用广度优先搜索遍历的方法来求最短路径。而对于有权图，当图中各个边的权值相同的时候，就可以类比为无权图（无权图可理解为各边权值 1）因为各边没有了权的大小之分，同样可以用广度优先搜索遍历的方式来求最短路径，因此本题选 A。



17. D。本题考查弗洛伊德算法。以下代码是弗洛伊德算法的主体部分，可知基本操作为最内层循环中 if 语句内的部分，基本操作外有 3 层 n 次循环，因此时间复杂度为 $O(n^3)$ ，故本题选 D。

```
for(k=0;k<g.n;++k)
    for(i=0;i<g.n;++i)
        for(j=0;j<g.n;++j)
            if(A[i][j]>A[i][k]+A[k][j]) {
                A[i][j]=A[i][k]+A[k][j];
                path[i][j]=k;
            }
```

18. A。本题考查邻接矩阵存储结构和拓扑排序。拓扑排序序列存在的条件是图中不存在环。如果邻接矩阵主对角线以下的元素均为零,则代表图中只存在由编号小的顶点指向编号大的顶点的边,不存在由编号大的顶点指向编号小的顶点的边,这样图中就不可能出现环。如邻接矩阵:

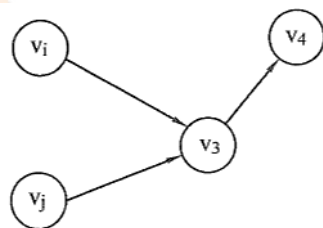
$$A = \begin{pmatrix} 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

所表示的图中的边为 $\langle 0, 1 \rangle$, $\langle 0, 2 \rangle$, $\langle 0, 3 \rangle$, $\langle 1, 2 \rangle$, $\langle 1, 3 \rangle$, $\langle 2, 3 \rangle$,

图中不存在环,因此一定存在拓扑排序序列(通过本题可以联想到在操作系统中,给资源编号,按序分配,可以破坏环路,不会产生死锁)。

19. B。本题考查拓扑排序的排序过程。如果在拓扑排序过程中,当删除一个入度为 0 的顶点时,出现了多个入度为 0 的顶点,这时其中任何一个入度为 0 的顶点都可以作为当前删除的顶点在拓扑排序序列中的后继顶点,因此拓扑排序序列不一定是唯一的。

20. D。本题考查拓扑排序的排序过程。在拓扑排序过程中,在一条路径上的顶点,总是按照由路径尾到路径头(假如有路径 $\langle a, b \rangle$ 、 $\langle b, c \rangle$ 、 $\langle c, d \rangle$,则规定 a 是路径尾, d 是路径头)的顺序依次出现在拓扑排序序列中,因为拓扑排序每次都是选出入度为 0 的顶点,并在图中删除这个顶点及和它相连的边。只有路径尾先被选出来,后序结点才可能变成入度为 0 的结点,才有可能被选出来。由此可知,对于一条路径,路径尾结点总是先于路径头结点出现在拓扑排序序列中。A、B 两选项的描述是可能出现的, D 选项的描述是不可能出现的, C 选项也是可能出现的。如下图所示,拓扑排序序列可以是 V_i, V_j, V_3, V_4 , 而 V_i 与 V_j 之间没有边。



21. A。本题考查关键路径的定义。在 AOE 网中,从源点到汇点的所有路径中,具有最大路径长度的路径称为关键路径。完成整个工程的最短时间就是网中关键路径的长度,也就是网中关键路径上各种活动持续时间的总和。把关键路径上的活动称为关键活动。

22. D。本题考查最小生成树的定义。一个连通图的最小生成树一定包含图中的 n 个顶点, A、B、C 3 选项不一定包含 n 个顶点。

23. D。本题考查拓扑排序。若有向图中的顶点不能排成一个拓扑序列,则该有向图中一定含有环。顶点数目大于 1 的强连通图一定含有环,因此本题选 D。A、C 两选项显然不对。对于 B 选项顶点数目为 1 的图也是强连通图,这是一种特例。

(二) 综合应用题

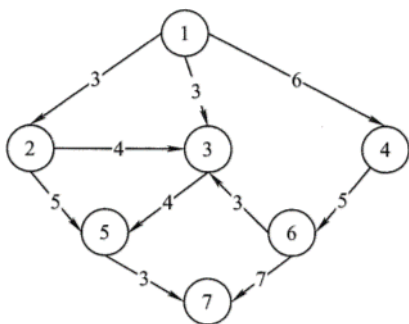
1. 已知有向图如下图所示。

(1) 写出该图的邻接矩阵表示并据此给出从顶点 1 出发的深度优先遍历序列。

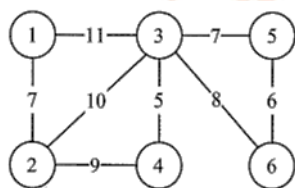
(2) 求该有向图的强连通分量的数目。

(3) 给出该图的任意两个拓扑序列。

4) 若将该图视为无向图,分别用 Prim 算法和 Kruskal 算法求最小生成树。

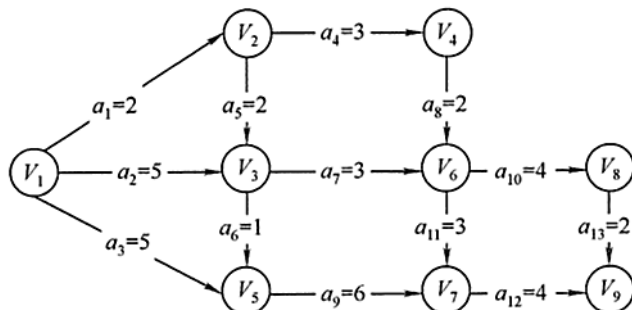


2. 对下图所示的无向图，按照 Dijkstra 算法，写出从顶点 1 到其他各个顶点的最短路径和最短路径长度（顺序不能颠倒）。



3. 下图所示为一个用 AOE 网表示的工程。

- 1) 画出此图的邻接表表示。
- 2) 完成此工程至少需要多少时间？
- 3) 指出关键路径。
- 4) 哪些活动加速可以缩短完成工程所需的时间？

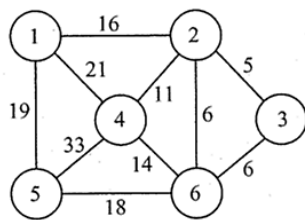


4. 下表给出了某工程各工序之间的优先关系和各工序所需的时间（其中“—”表示无先驱工序），请完成以下各题：

- (1) 画出相应的 AOE 网。
- (2) 列出各事件的最早发生时间和最迟发生时间。
- (3) 求出关键路径并指明完成该工程所需的最短时间。

工序代号	A	B	C	D	E	F	G	H
所需时间	3	2	2	3	4	3	2	1
先驱工序	—	—	A	A	B	A	C、E	D

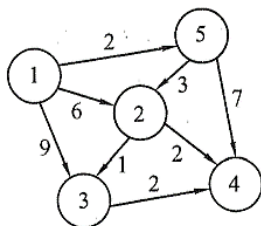
5. 下图所示为一个地区的通信网，边表示城市间的通信线路，边上的权表示架设线路花费的代价，如何选择能沟通每个城市且总代价最省的 $n-1$ 条线路，画出所有可能的选择。



6. 有一图的邻接矩阵 A 如下。顶点编号从 0 开始, 试给出用弗洛伊德算法求各点间最短距离的矩阵序列 A_0, A_1, A_2, A_3 , 其中矩阵下标代表当前所取的中间顶点, 如 A_2 即为将顶点 2 作为中间顶点时已求出的最短路径矩阵。

$$A = \begin{pmatrix} 0 & 2 & \infty & \infty \\ \infty & 0 & 1 & 6 \\ 5 & \infty & 0 & 4 \\ 3 & \infty & \infty & 0 \end{pmatrix}$$

7. 对于下图所示的带权有向图, 采用迪杰斯特拉算法求出从顶点 1 到其他顶点的最短路径, 要求给出求解过程。



【答案解析】

1. 解答: 1) 该图的邻接矩阵为:

$$A = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \end{matrix} & \begin{bmatrix} 0 & 3 & 3 & 6 & \infty & \infty & \infty \\ \infty & 0 & 4 & \infty & 5 & \infty & \infty \\ \infty & \infty & 0 & \infty & 4 & \infty & \infty \\ \infty & \infty & \infty & 0 & \infty & 5 & \infty \\ \infty & \infty & \infty & \infty & 0 & \infty & 3 \\ \infty & \infty & 3 & \infty & \infty & 0 & 7 \\ \infty & \infty & \infty & \infty & \infty & \infty & 0 \end{bmatrix} \end{matrix}$$

得到的深度优先遍历序列为 1, 2, 3, 5, 7, 4, 6。

2) 解题思路: 当某个顶点只有出弧而没有入弧时, 其他顶点无法到达这个顶点, 不可能与其他顶点和边构成强连通分量 (这个单独的顶点构成一个强连通分量)。

① 顶点 1 无入弧构成第一个强连通分量。删除顶点 1 及所有以之为尾的弧。

② 顶点 2 无入弧构成一个强连通分量。删除顶点 2 及所有以之为尾的弧。

③

以此类推, 最后得到每个顶点都是一个强连通分量, 故强连通分量数目为 7。

3) 该图的两个拓扑序列如下:

① 1, 2, 4, 6, 3, 5, 7

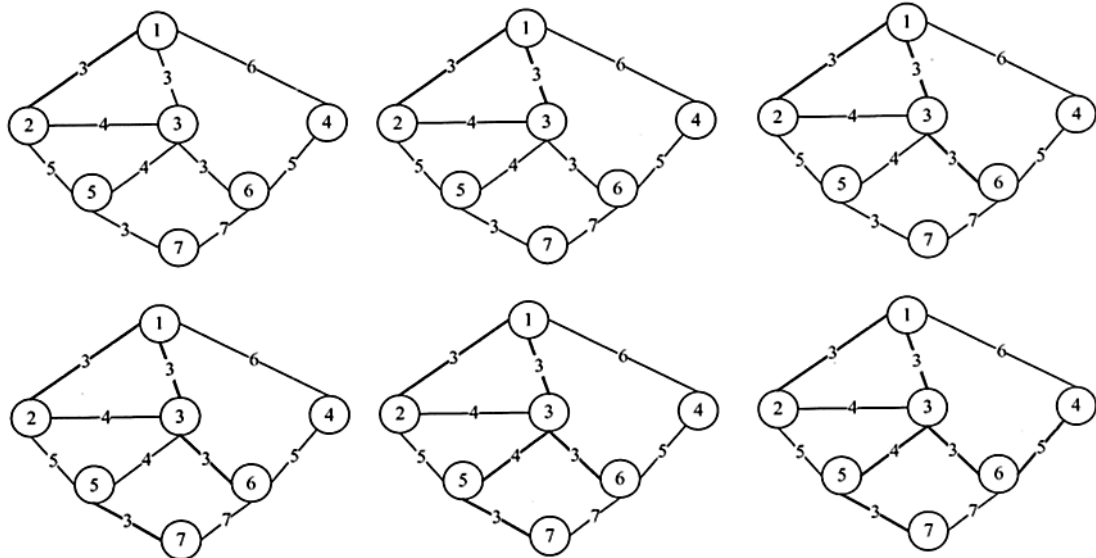
② 1, 4, 2, 6, 3, 5, 7

4) 若视该图为无向图:

用 Prim 算法生成最小生成树的过程如下:

1—2, 1—3, 3—6, 3—5, 5—7, 6—4 (图略)。

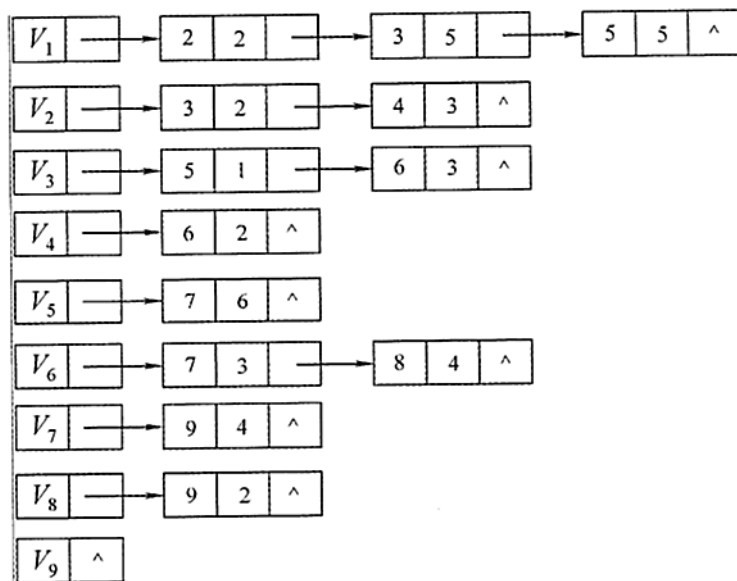
用 Kruskal 算法生成最小生成树的过程如下图所示。



2. 解答: 根据 Dijkstra 算法, 求从顶点 1 到其余各顶点的最短路径如下表所示。

顶点	从顶点 1 到各终点的 dist 值				
	第 1 轮	第 2 轮	第 3 轮	第 4 轮	
2	7 V ₁ , V ₂				
3	11 V ₁ , V ₃	11 V ₁ , V ₃			
4	∞	16 V ₁ , V ₂ , V ₄	16 V ₁ , V ₂ , V ₄		
5	∞	∞	18 V ₁ , V ₃ , V ₅	18 V ₁ , V ₃ , V ₅	
6	∞	∞	19 V ₁ , V ₃ , V ₆	19 V ₁ , V ₃ , V ₆	19 V ₁ , V ₃ , V ₆
集合 S	{1,2}	{1,2,3}	{1,2,3,4}	{1,2,3,4,5}	{1,2,3,4,5,6}

3. 1) 该图的邻接表表示如下图所示。



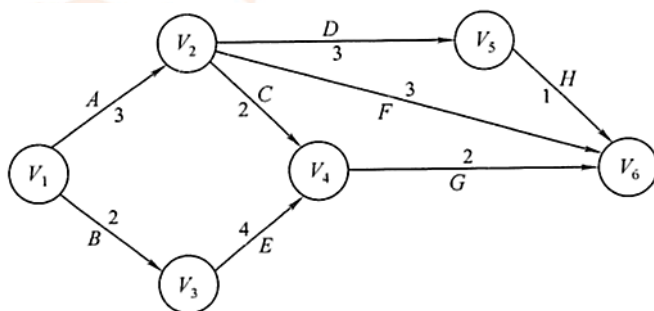
求关键路径的算法如下:

- ① 输入 e 条弧 $\langle j, k \rangle$, 建立 AOE 网的存储结构。
- ② 从源点 v_1 出发, 令 $v_e(l) = 0$, 求 $v_e(j)$, $2 \leq j \leq n$ 。
- ③ 从汇点 v_n 出发, 令 $v_l(n) = v_e(n)$, 求 $v_l(i)$, $1 \leq i \leq n-1$ 。
- ④ 根据各顶点的 v_e 和 v_l 值, 求每条弧 s (活动) 的最早开始时间 $e(s)$ 和最晚开始时间 $l(s)$,

其中 $e(s) = l(s)$ 为关键活动。

- 2) 根据以上算法可以得到至少需要时间 16。
- 3) 关键路径为 $(V_1, V_3, V_5, V_7, v_9)$
- 4) 活动 a_2, a_6, a_9, a_{12} 加速, 可以缩短工程所需的时间。

4. 1) 根据题表可以画出 AOE 网如下图所示。



求解各事件和活动的最早发生时间与最迟发生时间公式分别如下:

- ① $v_e(\text{源点}) = 0$, $v_e(k) = \text{Max}\{v_e(j) + \text{Weight}(v_j, v_k)\}$, $\text{Weight}(v_j, v_k)$ 表示从 v_j 指向 v_k 的弧的权值。
- ② $v_l(\text{汇点}) = v_e(\text{汇点})$, $v_l(j) = \text{Min}\{v_l(k) - \text{Weight}(v_j, v_k)\}$, $\text{Weight}(v_j, v_k)$ 表示从 v_j 指向 v_k 的弧的权值。
- ③ 若边 $\langle v_k, v_j \rangle$ 表示活动 a_i , 则有 $e(i) = v_e(k)$ 。
- ④ 若边 $\langle v_k, v_j \rangle$ 表示活动 a_i , 则有 $l(i) = v_l(j) - \text{Weight}(v_k, v_j)$ 。
- ⑤ $d(i) = l(i) - e(i)$ 。

关键路径即由 $d(i) = 0$ 的 i 构成。

2) 根据上述公式, 各事件的最早发生时间 v_e 和最迟发生时间 v_l 如下表所示。

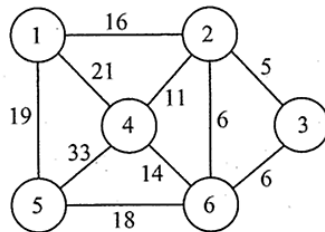
	V_1	V_2	V_3	V_4	V_5	V_6
$V_e(i)$	0	3	2	6	6	8
$V_l(i)$	0	4	2	6	7	8

3) 根据上述公式, 各活动最早发生时间 e 、最迟发生时间 l 和时间余量 $d(i) = l(i) - e(i)$ 如下表所示。

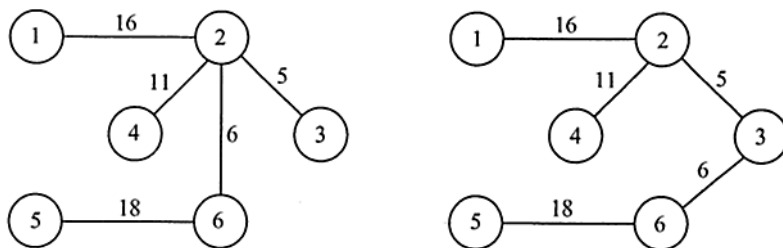
	A	B	C	D	E	F	G	H
$e(i)$	0	0	3	3	2	3	6	6
$l(i)$	1	0	4	4	2	5	6	7
$l(i) - e(i)$	1	0	1	1	0	2	0	1

所以关键路径为 B、E、G, 完成该工程最少需要 8 (单位依题意而定)。

5、本题为最小生成树问题, 可以采取普里姆或者克鲁斯卡尔算法来求最小生成树, 这里采用克鲁斯卡尔算法。题目中要求画出所有可能的情况, 通过观察可知, 下图中除了权值为 6 的边之外, 其余边都各不相同。权值为 6 的边有两条, 并且在求解过程中, 这两条边不可能同时被选为最小生成树的分支, 因此最小生成树有两种可能, 差别只在权值 6 的边这里。



解: 由题可知最小生成树有两种可能, 依照克鲁斯卡尔算法, 得到最小生成树如下图所示。



6、按照弗洛伊德算法, 求解过程如下。

(1) 将顶点 0 作为中间顶点, 检查所有顶点对, 有:

$A[2][1] \Rightarrow A[2][0] + A[0][1] = 5 + 2 = 7$, 因此 $A[2][1]$ 改为 7。

$A[3][1] \Rightarrow A[3][0] + A[0][1] = 3 + 2 = 5$, 因此 $A[3][1]$ 改为 5。

因此:

$$A_0 = \begin{pmatrix} 0 & 2 & \infty & \infty \\ \infty & 0 & 1 & 6 \\ 5 & 7 & 0 & 4 \\ 3 & 5 & \infty & 0 \end{pmatrix}$$

(2) 将顶点 1 作为中间顶点, 检查所有顶点对, 有:

$A[0][2] \Rightarrow A[0][1] + A[1][2] = 2 + 1 = 3$, 因此 $A[0][2]$ 改为 3。

$A[0][3] \Rightarrow A[0][1] + A[1][3] = 2 + 6 = 8$, 因此 $A[0][3]$ 改为 8。

$A[3][2] \Rightarrow A[3][1] + A[1][2] = 5 + 1 = 6$, 因此 $A[3][2]$ 改为 6。

因此:

$$A_1 = \begin{pmatrix} 0 & 2 & 3 & 8 \\ \infty & 0 & 1 & 6 \\ 5 & 7 & 0 & 4 \\ 3 & 5 & 6 & 0 \end{pmatrix}$$

(3) 将顶点 2 作为中间顶点, 检查所有顶点对, 有:

$A[0][3]=8 > A[0][2]+A[2][3]=3+4=7$, 因此 $A[0][3]$ 改为 7。

$A[1][0]=\infty > A[1][2]+A[2][0]=1+5=6$, 因此 $A[1][0]$ 改为 6。

$A[1][3]=6 > A[1][2]+A[2][3]=1+4=5$, 因此 $A[1][3]$ 改为 5。

因此:

$$A_2 = \begin{pmatrix} 0 & 2 & 3 & 7 \\ 6 & 0 & 1 & 5 \\ 5 & 7 & 0 & 4 \\ 3 & 5 & 6 & 0 \end{pmatrix}$$

4) 将顶点 3 作为中间顶点, 检查所有顶点对后, 可知本步无须修改。

因此:

$$A_3 = \begin{pmatrix} 0 & 2 & 3 & 7 \\ 6 & 0 & 1 & 5 \\ 5 & 7 & 0 & 4 \\ 3 & 5 & 6 & 0 \end{pmatrix}$$

7、用数组 $dist[]$ 存放最短路径长度, 用数组 $path[]$ 存放最短路径。具体求解过程见下表。

已进入的顶点	dist[]					Path[]				
	1	2	3	4	5	1	2	3	4	5
1	0	6	9	∞	2	-1	1	1	-1	1
1,5	0	5	9	∞	2	-1	5	1	5	1
1,2,5	0	5	6	9	2	-1	5	2	2	1
1,2,3,5	0	5	6	7	2	-1	5	2	2	1
1,2,3,4,5	0	5	6	7	2	-1	5	2	2	1

由上表可知:

顶点 1 到顶点 2 的最短路径长度为 5, 最短路径为 1-5-2;

顶点 1 到顶点 3 的最短路径长度为 6, 最短路径为 1-5-2-3;

顶点 1 到顶点 4 的最短路径长度为 7, 最短路径为 1-5-2-4;

顶点 1 到顶点 5 的最短路径长度为 2, 最短路径为 1-5。

7、查找

7.1 顺序查找

(一) 选择题

- 顺序查找适合于存储结构为 () 的线性表。
A. 顺序存储结构或链式存储结构 B. 散列存储结构
C. 索引存储结构 D. 压缩存储结构
- 由 n 个数据元素组成的两个表: 一个递增有序, 一个无序。采用顺序查找算法, 对有序表从头开始查找, 发现当前元素已不小于待查元素时, 停止查找, 确定查找不成功, 已知查找任一元素的概率是相同的, 则在两种表中成功查找 ()。
A. 平均时间后者小 B. 平均时间两者相同
C. 平均时间前者小 D. 无法确定
- 对长度为 n 的有序单链表, 若查找每个元素的概率相等, 则顺序查找表中任一元素的查找成功的平均查找长度为 ()。
A. $n/2$ B. $(n+1)/2$ C. $(n-1)/2$ D. $n/4$
- 若查找每个关键字的概率均等, 则在具有 n 个关键字的顺序表中采用顺序查找法查找一个记录, 其查找成功的平均查找长度 ASL 为 ()。
A. $(n-1)/2$ B. $n/2$ C. $(n+1)/2$ D. n
- 顺序查找法适合于存储结构为 () 的线性表。
A. 散列存储 B. 顺序存储或者链式存储
C. 压缩存储 D. 索引存储

【答案解析】

- A。顺序查找是指从表的一端开始向另一端查找。它不要求查找表具有随机存取的特性, 可以是顺序存储结构或链式存储结构。
- B。对于顺序查找, 不管线性表是有序的还是无序的, 成功查找第一个元素的比较次数为 1, 成功查找第二个元素的比较次数为 2, 以此类推, 即每个元素查找成功的比较次数只与其位置有关 (与是否有序无关), 因此查找成功的平均时间两者相同。
- B。在有序单链表上做顺序查找, 查找成功的平均查找长度与在无序顺序表或有序顺序表上做顺序查找的平均查找长度相同, 都是 $(n+1)/2$ 。
- C。本题考查顺序查找法的平均查找长度。由题干知查找 n 个关键字的概率是相等的, 因此对于每个关键字的查找概率均为 $1/n$ 。在表中第一个关键字的比较次数为 1, 以后的每个关键字的比较次数比前一个多 1, 一共 n 个记录, 因此由等差数列求和公式可得所有记录比较次数的总和为 $(1+n)n/2$, 则平均查找长度为 $(1/n) \times (1+n)n/2 = (n+1)/2$ 。
- B。本题考查顺序查找所使用的存储结构。顺序查找方法适应于线性表, 用顺序表或者链表来表示的线性表它都适应, 因此本题选 B。

(二) 综合应用题

- 若对于大小均为 n 的有序顺序表和无序顺序表分别进行顺序查找, 试在下列 3 种情况下分

别讨论两者在等概率时平均查找长度是否相同（在表的尾部有一个查找结束标记关键字）。

- 1) 查找不成功，即表中没有关键字等于给定值 Key。
- 2) 查找成功，且表中只有一个关键字等于给定值 Key。
- 3) 查找成功，且表中有若干关键字等于给定值 Key，要求找出所有这些记录。

【答案解析】

1.分析：1) 查找长度不同。原因：扫描有序表，只要发现第一个比关键字大的值，即可结束扫描，无须扫描整个表，因此平均查找长度与无序表不同。

2) 查找成功时，对于两种表都是逐个扫描关键字，直到找到与给定的 Key 相同的关键字为止，因此平均查找长度均为 $(n+1)/2$ 。

3) 有序表中相同关键字在表中的位置一定是相邻的，而无序表中相同关键字是随机散落在表中的。因此，对于有序表，找到第一个与 Key 相同的元素后，只需继续扫描，找到最后一个与 Key 不相同的元素即可；而对于无序表，则需要扫描整个表。对于无序表的平均查找长度为 $n+1$ ，而对于有序表则小于 $n+1$ 。

答：1) 不同，有序顺序表为 $n/2+n/(n+1)$ ，无序顺序表为 $n+1$ 。

2) 相同，有序顺序表为 $(n+1)/2$ ，无序顺序表也为 $(n+1)/2$ 。

3) 不同，有序顺序表小于 $n+1$ ，无序顺序表为 $n+1$ 。

7.2 分块查找

1. 已知一个有序表 (13, 18, 24, 35, 47, 50, 62, 83, 90, 115, 134)，当二分查找值为 90 的元素时，查找成功的比较次数为()。

- A. 1 B. 2 C. 4 D. 6

2. 下列关于分块查找的说法中，正确的是()。

- A. 数据分成大小相等的若干块，块内数据有序
B. 数据分成若干块，每块（除最后一块外）中数据个数需相同
C. 数据分成若干块，块内数据必须有序，块间不必有序
D. 数据分成若干块，块内数据不必有序，但块间必须有序

3. 采用分块查找时，数据的组织方式为()。

- A. 数据分成若干块，每块内数据有序
B. 数据分成若干块，每块内数据不必有序，但块间必须有序，每块内最大（或最小）的数据组成索引块
C. 数据分成若干块，每块内数据有序，每块内最大（或最小）的数据组成索引块
D. 数据分成若干块，每块（除最后一块外）中数据个数需相同

4. 对有 2500 个记录的索引顺序表（分块表）进行查找，最理想的块长为()。

- A. 50 B. 125 C. 500 D. $\lceil \log 2500 \rceil$

【答案解析】

1.B. 开始时 low 指向 13，high 指向 134，mid 指向 50，比较第一次 $90 > 50$ ，所以将 low 指向 62，high 指向 134，mid 指向 90，第二次比较找到 90。

2.D. 分块查找把数据分为若干个块，每个块内的元素不需要有序，而块间元素保证有序，每一块选出最大（或最小）元素作为这块的代表元素。故分块查找要保证块间有序，而块内可以无序，

每一块的大小不一定需要相同, 故答案为选项 D。

3.B. 通常情况下, 在分块查找的结构中, 不要求每个索引块中的元素个数都相等。

4.A. 设块长为 b , 索引表包含 n/b 项, 索引表的 $ASL = (n/b + 1)/2$, 块内的 $ASL = (b + 1)/2$, 总 $ASL =$ 索引表的 $ASL +$ 块内的 $ASL = (b + n/b + 2)/2$, 其中对于 $b + n/b$, 由均值不等式知 $b = n/b$ 时有最小值, 此时 $b = \sqrt{n}$ 。则最理想块长为 $\sqrt{2500} = 50$ 。

7.3 折半查找

(一) 选择题

- 下列关于二分查找的叙述中, 正确的是()。
 - 表必须有序, 表可以顺序方式存储, 也可以链表方式存储
 - 表必须有序且表中数据必须是整型、实型或字符型
 - 表必须有序, 而且只能从小到大排列
 - 表必须有序, 且表只能以顺序方式存储
- 下面关于折半查找的叙述正确的是()。
 - 表必须有序, 表可以顺序方式存储, 也可以链表方式存储
 - 表必须有序, 而且只能从小到大排列
 - 表必须有序, 且表中关键字必须是整型、实型或字符型
 - 表必须有序, 且表只能以顺序方式存储
- 对于同一个表, 用二分(折半)法查找表的元素的速度比用顺序法()。
 - 必然快
 - 必然慢
 - 相等
 - 不能确定
- 折半查找的时间复杂度为()。
 - $O(n^2)$
 - $O(n)$
 - $O(n \log_2 n)$
 - $O(\log_2 n)$
- 对线性表进行二分查找时, 要求线性表必须()。
 - 以顺序方式存储
 - 以链表方式存储
 - 以顺序方式存储, 且关键字要有序
 - 以链表方式存储, 且关键字要有序
- 有一个有序表为 $\{1, 3, 9, 12, 32, 41, 45, 62, 75, 77, 82, 95, 99\}$, 采用二分查找法, 找到 82 共进行()比较。
 - 1 次
 - 2 次
 - 4 次
 - 8 次
- 对有 18 个元素的有序表 $R[1, \dots, 18]$ 进行二分查找, 则查找 $R[3]$ 的比较序列下标为()。
 - 1, 2, 3
 - 9, 5, 2, 3
 - 9, 5, 3
 - 9, 4, 2, 3
- 折半查找过程所对应的判定树是一棵()。
 - 最小生成树
 - 平衡二叉树
 - 完全二叉树
 - 满二叉树
- 在有 11 个元素的有序表 $A[1, 2, \dots, 11]$ 中进行折半查找 ($\lfloor (low + high) / 2 \rfloor$), 查找元素 $A[11]$ 时, 被比较的元素下标依次是()。
 - 6, 8, 10, 11
 - 6, 9, 10, 11
 - 6, 7, 9, 11
 - 6, 8, 9, 11
- 对表长为 n 的有序表进行折半查找, 其判定树的高度为()。
 - $\lceil \log_2(n + 1) \rceil$
 - $\lfloor \log_2(n + 1) \rfloor - 1$
 - $\lceil \log_2 n \rceil$
 - $\lfloor \log_2 n \rfloor - 1$
- 已知一个长度为 16 的顺序表, 其元素按关键字有序排列, 若采用折半查找算法查找一个不存在的元素, 则比较的次数至少是(), 至多是()。
 - 4
 - 5
 - 6
 - 7

【答案解析】

1. D。二分查找通过下标来定位中间位置元素,故应采用顺序存储,且二分查找能够进行的前提是查找表是有序的,但具体是从大到小还是从小到大的顺序则不做要求。

2. D。本题考查折半查找的适用情况。

选项 A,折半查找是基于随机存储方式的算法,必须用顺序表而不能用链表,因此 A 错。

选项 B,折半查找表要求元素有序,但没有要求其表中元素是递增有序还是递减有序。由折半查找的判定树也可以看出,只要根结点能将树中结点分为两部分,一部分结点值大于根结点值,一部分结点值小于根结点值即可,而没有左子树结点值必须小于右子树结点值这样的特殊规定,因此 B 错。

选项 C,折半查找方法只要求关键字有序,而对关键字的数据类型没有规定,因此 C 错。

3. D。本题考查折半查找法与顺序查找法的性能比较。对于同一个,表如果找的是表中的第一个元素,则用顺序查找方法一定比折半查找快;如果找的是表中的最后一个元素,则用顺序查找方法一定比折半查找方法慢。因此本题选 D。

4. D。本题考查折半查找算法的时间复杂度。由第 2 题的讲解可知,对规模为 n 的有序表进行折半查找的平均查找长度不大于 $\lceil \log_2 n \rceil + 1$ 。在不做特殊说明的情况下,算法的时间复杂度就是最坏情况下基本运算的执行次数 $f(n)$ 。查找中基本运算为关键字的比较,因此对于折半查找,最坏情况下的比较次数 $f(n) = \lceil \log_2 n \rceil + 1$,折半查找算法的时间复杂度为 $O(\log_2 n)$ 。因此本题选 D。

5. C。本题考查二分查找法所适用的表的特点。二分查找要求查找对象采用顺序存储结构,并且元素有序,因此本题选 C。

6. C。本题考查二分查找的过程。解决此类题目最保险的方法是构造二分查找判定树,虽然这样速度有点慢,但是不容易出错,本题的判定树如右图所示。

由右图所示的判定树可知,找到 82 时,共进行了 4 次比较,因此本题选 C。

7. D。本题考查二分查找的过程。本题可以通过构造二分查找判定树来解决,这里采用另一种方法,直接计算比较序列下标。第一次: $\lfloor (1+18)/2 \rfloor = 9$, 下一次查找子区间为 1~8; 第二次: $\lfloor (1+8)/2 \rfloor = 4$, 下一次查找子区间为 1~3; 第三次: $\lfloor (1+3)/2 \rfloor = 2$, 下一次查找子区间为 3~3; 第四次: $\lfloor (3+3)/2 \rfloor = 3$ 。因此,比较序列下标为 9, 4, 2, 3, 故本题选 D。

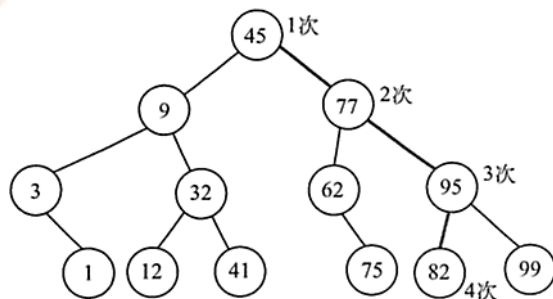
8. B。A 显然排除。对于选项 C, 考点精析示例中的判定树就不是完全二叉树。由选项 C 也可排除选项 D, 且满二叉树对结点数有要求。只可能选 B。事实上, 由折半查找的定义不难看出, 每次把一个数组从中间结点分割时, 总是把数组分为结点数相差最多不超过 1 的两个子数组, 从而使得对应的判定树的两棵子树高度差的绝对值不超过 1, 所以应是平衡二叉树。

9. B。依据折半查找算法的思想, 第一次 $\text{mid} = \lfloor (1+11)/2 \rfloor = 6$, 第二次 $\text{mid} = \lfloor ((6+1)+11)/2 \rfloor = 9$, 第三次 $\text{mid} = \lfloor ((9+1)+11)/2 \rfloor = 10$, 第四次 $\text{mid} = 11$ 。

10. A。对 n 个结点的判定树, 设结点总数 $n = 2^h - 1$, 则 $h = \lceil \log_2(n+1) \rceil$ 。

另解: 特殊值代入法。直接将 $n=1$ 和 $n=2$ 的情况代入, 仅有 A 满足要求。

11. A、B。对于此类题, 有两种做法: 一种方法是, 画出查找过程中构成的判定树, 让最小的分支高度对应于最少的比较次数, 让最大的分支高度对应于最多的比较次数, 出现类似于长度为 15 的顺序表时, 判定树刚好是一棵满树, 此时最多比较次数与最少比较次数相等; 另一种方法是, 直接用公式求出最小的分支高度和最大分支高度, 从前面的讲解不难看出最大分支高度为 $H = \lceil \log_2(n+1) \rceil = 5$, 这对应的就是最多比较次数, 然后由于判定树不是一棵满树, 所以至少应该是 4 (由判定树



的各分支高度最多相差 1 得出)。

注意,若是求查找成功或查找失败的平均查找长度,则需要画出判定树进行求解。此外,对长度为 n 的有序表,采用折半查找时,查找成功和查找失败的最多比较次数相同,均为 $\lceil \log_2(n+1) \rceil$ 。

(二) 综合应用题

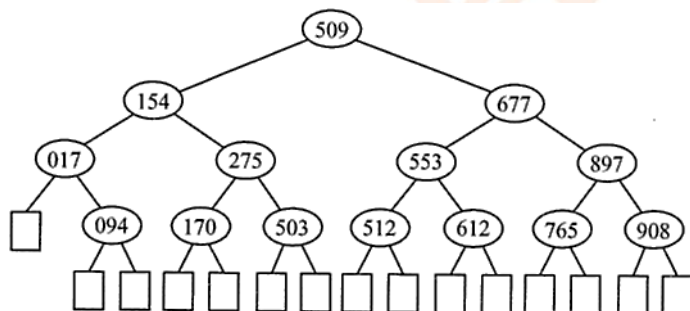
1. 设有序顺序表中的元素依次为 017, 094, 154, 170, 275, 503, 509, 512, 553, 612, 677, 765, 897, 908。

- 1) 试画出对其进行折半查找的判定树。
- 2) 若查找 275 或 684 的元素,将依次与表中的哪些元素比较?
- 3) 计算查找成功的平均查找长度和查找不成功的平均查找长度。

2. 类比二分查找算法,设计 K 分查找算法 (k 为大于 2 的整数) 如下: 首先检查 n/k 处 (n 为查找表的长度) 的元素是否等于要搜索的值, 然后检查 $2n/k$ 处的元素……这样, 或者找到要查找的元素, 或者把集合缩小到原来的 $1/k$, 若未找到要查找的元素, 则继续在得到的集合上进行 k 分查找; 如此进行, 直到找到要查找的元素或查找失败。试求查找成功和查找失败的时间复杂度。

【答案解析】

1 答: 1) 判定树如下图所示:



2) 若查找 275, 依次与表中元素 509, 154, 275 进行比较, 共比较 3 次。若查找 684, 依次与表中元素 509, 677, 897, 765 进行比较, 共比较 4 次。

3) 在查找成功时, 会找到图中的某个圆形结点, 其平均查找长度为:

$$ASL_{\text{成功}} = \frac{1}{14} \sum_{i=1}^{14} C_i = \frac{1}{14} (1 + 2 \times 2 + 3 \times 4 + 4 \times 7) = \frac{45}{14}$$

在查找失败时, 会找到图中的某个方形结点, 但这个结点是虚构的, 最后一位的比较元素为其父结点 (圆形结点), 故其平均查找长度为

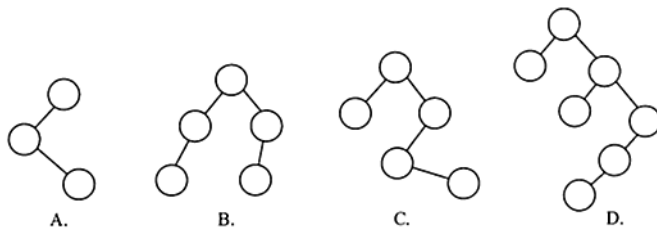
$$ASL_{\text{不成功}} = \frac{1}{15} \sum_{i=0}^{14} C'_i = \frac{1}{15} (3 \times 1 + 4 \times 14) = \frac{59}{15}$$

2. 解答: 与二分查找类似, K 分查找法可用 K 叉树来描述。 K 分查找法在查找成功时进行比较的关键字的个数最多不超过树的深度, 而具有 n 个结点的 K 叉树的深度为 $\lfloor \log_k n \rfloor + 1$, 所以 K 分查找法在查找成功时和给定值进行比较的关键字个数至多为 $\lfloor \log_k n \rfloor + 1$, 即时间复杂度为 $O(\log_k n)$ 。同理, 查找不成功时, 和给定值进行比较的关键字个数也至多为 $\lfloor \log_k n \rfloor + 1$, 故时间复杂度也为 $O(\log_k n)$ 。

7.4 树型查找

(一) 选择题

1. 在下图所示的树中, 属于平衡二叉树的是 ()。

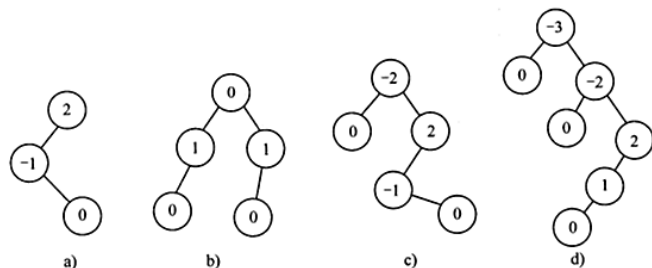


2. 在一棵平衡二叉树中, 每个结点的平衡因子的取值范围是 ()
A. -1~1 B. -2~2 C. 1~2 D. 0~1
3. 对于二叉排序树, 下面的说法中, () 是正确的。
A. 二叉排序树是动态树表, 查找失败时插入新结点, 会引起树的重新分裂和组合
B. 对二叉排序树进行层序遍历可得到有序序列
C. 用逐点插入法构造二叉排序树, 若先后插入的关键字有序, 二叉排序树的深度最大
D. 在二叉排序树中进行查找, 关键字的比较次数不超过结点数的 1/2
4. 按 () 遍历二叉排序树得到的序列是一个有序序列。
A. 先序 B. 中序 C. 后序 D. 层次
5. 在二叉排序树中进行查找的效率与 () 有关。
A. 二叉排序树的深度 B. 二叉排序树的结点的个数
C. 被查找结点的度 D. 二叉排序树的存储结构
6. 在常用的描述二叉排序树的存储结构中, 关键字值最大的结点 ()。
A. 左指针一定为空 B. 右指针一定为空
C. 左右指针均为空 D. 左右指针均不为空
7. 从空树开始, 依次插入元素 52, 26, 14, 32, 71, 60, 93, 58, 24 和 41 后构成了一棵二叉排序树。在该树查找 60 要进行比较的次数为 ()。
A. 3 B. 4 C. 5 D. 6
8. 在含有 n 个结点的二叉排序树中查找某个关键字的结点时, 最多进行 () 次比较。
A. $n/2$ B. $\log_2 n$ C. $\log_2 n + 1$ D. n
9. 构造一棵具有 n 个结点的二叉排序树时, 最理想情况下的深度为 ()。
A. $n/2$ B. n C. $\lfloor \log_2(n+1) \rfloor$ D. $\lceil \log_2(n+1) \rceil$
10. 不可能生成如下图所示的二叉排序树的关键字序列是 ()。
A. {4, 2, 1, 3, 5} B. {4, 2, 5, 3, 1} C. {4, 5, 2, 1, 3} D. {4, 5, 1, 2, 3}
11. 含有 20 个结点的平衡二叉树的最大深度为 ()。
A. 4 B. 5 C. 6 D. 7
12. 具有 5 层结点的 AVL 至少有 () 个结点。
A. 10 B. 12 C. 15 D. 17
13. 在含有 12 个结点的平衡二叉树上, 查找关键字 35 (树中存在该结点) 的结点, 则依次比较的关键字序列有可能是 ()。
A. 46, 36, 18, 20, 28, 35 B. 47, 37, 18, 27, 36
C. 27, 48, 39, 43, 37 D. 15, 25, 55, 35
14. 在含有 15 个结点的平衡二叉树上, 查找关键字为 28 的结点, 则依次比较的关键字可能是 ()。

- A. 30,36 B. 38, 48, 28 C. 48, 18, 38, 28 D. 60, 30, 50, 40, 38, 28
15. 一棵高度为 K 的平衡二叉树, 其每个非叶子结点的平衡因子均为 0, 则该树共有 () 个结点。
A. $2^{k-1}-1$ B. 2^{k-1} C. $2^{k-1}+1$ D. 2^k-1
16. 下列关于查找的说法中, 错误的是 ()。
I. 如果完全二叉树从根结点开始按层次遍历的输入序列为 $\{1, 2, 3, 4, 5, 6, 7\}$, 则该完全二叉树是二叉排序树。
II. 设有关键字构成二叉排序树, 每个关键字查找的概率相等, 查找成功的 ASL 最大是 n 。
III. 若把堆看成是一棵完全二叉树, 则该树一定是一棵二叉排序树。
A. I和II B. I、II和III C. II和III D. I和III
17. 下列关于红黑树和 AVL 树的描述中, 不正确的是 ()
A、两者都属于自平衡二叉树
B、两者插入、删除、查找的时间复杂度都相同
C、红黑树插入和删除过程至多有 2 次旋转操作
D、红黑树的任一结点的左右子树高度之差不超过 2 倍
18. 下列关于红黑树的说法中, 正确的是 ()
A、红黑树是一种特殊的平衡树
B、如果红黑树的所有结点都是黑色的, 那么它一定是一棵满二叉树
C、红黑树的任何一个分支结点都有两个非空孩子结点
D、红黑树的子树也一定是红黑树
19. 对于结点个数为 n 的红黑树, 在最坏的情况下, 红黑树的删除结点操作的时间复杂度是 ()。
A. $O(\log n)$ B. $O(n)$ C. $O(n^{1/2})$ D. $O(n^2)$
20. 红黑树在处理过程中红黑结点会产生冲突, 在下列操作的解决冲突中, 正确的是 ()。
A. 插入操作时, 解决红黑冲突 B. 删除操作时, 解决黑黑冲突
C. 插入操作时, 解决红红冲突 D. 删除操作时, 解决黑黑冲突
21. 下列关于红黑树的描述中, 错误的是 ()。
A. 从某结点到其后代叶结点的所有简单路径中, 最长的一条是最短一条的至多两倍
B. 从根到叶结点 (不包含根) 的任一条简单路径上至少有一半的节点必是黑色的
C. 从任意结点出发的所有下降路径都有相同的红结点数
D. 在红黑树上的查找操作可以在 $O(\log n)$ 时间内完成
22. 下列关于红黑树的说法中, 错误的是 ()。
A. 每个结点只能是红色或者黑色
B. 每个叶子结点是黑色
C. 如果一个结点是黑色, 则它的孩子结点必须是红色
D. 一棵 4 阶 B 树可以转换成对应的红黑树

【答案解析】

1. B。本题考查平衡因子的定义以及平衡二叉树的定义。计算出各结点的平衡因子如右图所示。
由各结点平衡因子可知选项 B 中的二叉树为平衡二叉树。



2. A。本题考查平衡因子的定义以及平衡二叉树的定义。结点的平衡因子为其左子树与右子树的高度之差。平衡二叉树其左、右子树都是平衡二叉树,且左、右子树高度之差的绝对值不超过 1。因此本题选 A。

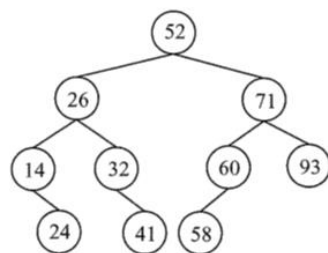
3. C。二叉排序树插入新结点时不会引起树的分裂组合。对二叉排序树进行中序遍历可得到有序序列。当插入的关键字有序时,二叉排序树会形成一个长链,此时深度最大。在此种情况下进行查找,有可能需要比较每个结点的关键字,超过总结点数的 $1/2$ 。

4. B。由二叉排序树的定义不难得出中序遍历二叉树得到的序列是一个有序序列。

5. A。二叉排序树的查找路径是自顶向下的,其平均查找长度主要取决于树的高度。

6. B。在二叉排序树的存储结构中,每个结点由三部分构成,其中左(或右)指针指向比结点的关键字值小(或大)的结点。关键字值最大的结点位于二叉排序树的最右位置,因此它的右指针一定为空(注意,有可能不是叶子结点)。

7. A。以第一个元素为根结点,依次将元素插入树,生成的二叉排序树如右图所示。进行查找时,先与根结点比较,然后根据比较结果,继续在左子树或右子树上进行查找。比较的结点依次为 52, 71, 60。

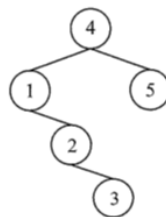


7 题图

8. D。当输入序列是一个有序序列时,构造的二叉排序树是一个单支树,当查找一个不存在的关键字值或最后一个结点的关键字值时,需要 n 次比较。

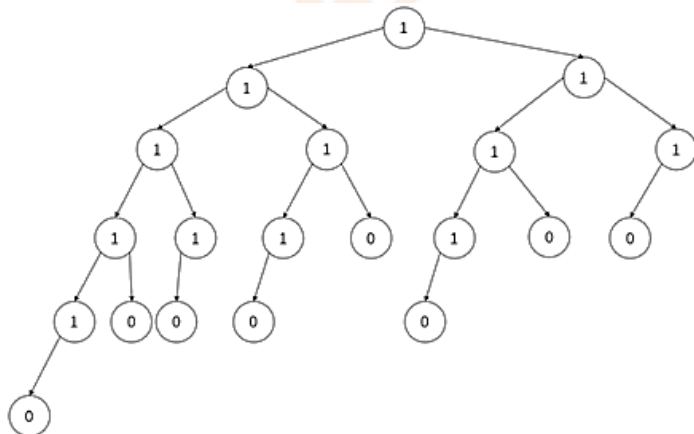
9. D。当二叉排序树的叶子结点全部都在相邻的两层内时,深度最小。理想情况是从第一层到倒数第二层为满二叉树。类比完全二叉树,可得深度为 $\lceil \log_2(n+1) \rceil$ 。

10. D。选项 D 中,插入 1 后,再插入 2, 2 应作为 1 的右孩子结点,再插入 3, 3 应作为 2 的右孩子结点。故选 D。选项 D 对应的二叉排序树如右图所示。



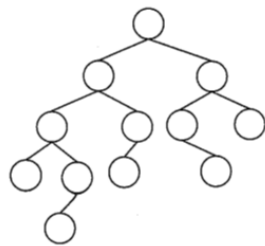
10 题图

11. C。平衡二叉树结点数的递推公式为 $n_0=0, n_1=1, n_2=2, n_h=1+n_{h-1}+n_{h-2}$ (h 为平衡二叉树高度, n_h 为构造此高度的平衡二叉树所需的最少结点数)。通过递推公式可得,构造 5 层平衡二叉树至少需 12 个结点,构造 6 层至少需要 20 个结点。



12. B。设 n_h 表示高度为 h 的平衡二叉树中含有的最少结点数,则有 $n_1=1, n_2=2, n_h=n_{h-1}+n_{h-2}+1$, 由此求出 $n_5=12$, 对应的 AVL 如右图所示。

13. D。本题为平衡二叉树知识的扩展。由第 12 题的公式 $N_0=0, N_1=1, N_2=2, \dots, N_h=N_{h-1}+N_{h-2}+1$ 可以算出 $N_3=4, N_4=7, N_5=12$ 。也就是说, 12 个结点的平衡二叉树中叶子结点的最小层数为 3, 最大层数为 5, 又由于 35 是存在的, 因此最多查找 5 次就一定能找到, 可以排除 A、B、C 三项。



12 题图

14. C。本题为平衡二叉树知识的扩展。根据第 12 题的公式有 $N_3=4, N_4=7, N_5=12, N_6=20>15$ 。也就是说, 高度为 6 的平衡二叉树最少有 20 个结点, 因此 15 个结点的平衡二叉树的最大高度为

说明：在第 29、30、31 三个题中，介绍了一个公式，即 N_h 表示高度为 h 的平衡二叉树中含有的最少结点数，有

$$N_0=0, \quad N_1=1, \quad N_2=2, \quad \dots, \quad N_h=N_{h-1}+N_{h-2}+1$$

叶子结点所在最小层为3

叶子结点所在最大层为5

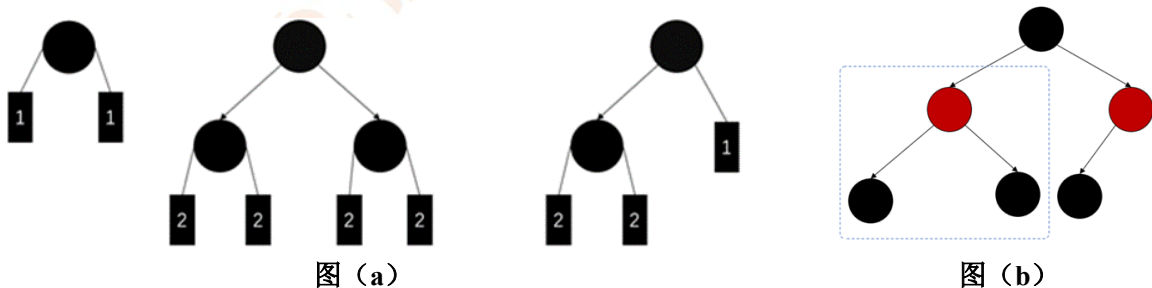
16. B。完全二叉树层次遍历序列为 $\{1, 2, 3, 4, 5, 6, 7\}$ ，其中序遍历序列无序，为 $\{4, 2, 5, 1, 6, 3, 7\}$ ，因此该完全二叉树不是二叉排序树，说法 I 错误。每个关键字查找的概率相等，最坏情况下二叉排序树退化为线性结构，此时查找成功的 ASL 与顺序表查找成功的 ASL 相等，为 $(n+1)/2$ ，因此说法 II 错误。所有双亲结点的值都大于（或小于）孩子结点的值，满足此特性的完全二叉树称为大（小）根堆，而堆不一定满足中序序列有序的特性，因此说法 III 错误。故答案为选项 B。

对于 B，两者插入、删除、查找的时间复杂度都是 $O(\log n)$ 。

对于 D，因为红黑树中，根节点到任意叶结点的路径上经过的黑色结点个数都相等，且除了根节点和第二层的叶结点之间，不允许有红红或黑黑的情况出现，红黑必须交叉出现，因此如果高度之差超过 2，必定会出现红红或黑黑的情况或者是经过黑色结点数不相等的情况。

对于 B，因为红黑树的叶结点必须是黑色的，根节点是黑色的，除了第一层了第二层可以连着出现黑黑的情况，其他不允许出现黑黑和红红的情况，且到达叶结点的路径上经过的黑色结点数必须相等，综上所述，所有结点都是黑色的红黑树只能有下图（a）左边两种情况，全都是满二叉树。下图（a）最右边的图可以看到不满足“到达叶结点的路径上经过的黑色结点数必须相等”。

对于 D，红黑树子树的根结点可能是红色的，因此就不是红黑树，如下图（b），虚线框里的左子树就不是红黑树。



19.A. n 个结点的红黑树高度 h 约等于 $\log n$, 删除结点时首先定位到结点, 查找操作的时间复杂度为 $O(\log n)$; 为维护红黑树的特性, 删除过程中可能会包含调整操作, 时间复杂度也为 $O(\log n)$, 故红黑树的删除结点操作的时间复杂度为 $O(\log n)$ 。

20.C. 红黑树的一条性质: 红色结点不连续, 所以当插入新结点时, 将结点颜色初始化为红色, 检查任意结点到空叶结点的所有路径是否包含相同数量的黑色结点; 若不满足, 则检查与该结点对立面的结点颜色是黑色还是红色, 若为红色, 只需回溯改变结点的颜色; 若为黑色, 则需要回溯调整。

21.C. 从某结点到其后代叶结点的所有简单路径中, 黑结点出现次数相等, 最短路径显然均为黑结点; 路径中不能连续出现两个红结点, 最长路径为黑红交替出现, 最长路径至多为最短路径的 2 倍, 因此选项 A 正确。路径中不能连续出现两个红结点, 若黑结点个数小于红结点的 $1/2$, 会出现两个红结点相邻的情况, 因此选项 B 正确。从任意结点出发的所有下降路径都有相同的黑结点个数, 并非红结点个数, 因此选项 C 错误。含有 n 个结点的红黑树高度 h 约为 $\log n$, 查找次数与树高为同一数量级, 查找操作均可以在 $O(\log n)$ 时间内完成, 因此选项 D 正确。

22.C. 红黑树性质: ①每个结点只能是红色或黑色; ②根结点是黑色; ③叶结点是黑色, 且不存在的外部结点; ④红色结点的儿子只能是黑色; ⑤对每个结点, 从该结点到任一叶结点的简单路径上, 所含黑结点数相同。选项 A 符合①, 正确。选项 B 符合③, 正确。选项 C 错误, 黑色结点的孩子可以是红色也可以是黑色。选项 D, 红黑树和 4 阶 B 树 (也称 2-3-4 树) 有对应关系, 可以互相转化, 正确。故答案为选项 C。

(二) 综合应用题

1. 一棵二叉排序树按先序遍历得到的序列为 (50, 38, 30, 45, 40, 48, 70, 60, 75, 80), 试画出该二叉排序树, 并求出等概率下查找成功和查找失败的平均查找长度。
2. 按照序列 (40, 72, 38, 35, 67, 51, 90, 8, 55, 21) 建立一棵二叉排序树, 画出该树, 并求出在等概率的情况下, 查找成功的平均查找长度。
3. 依次把结点(34, 23, 15, 98, 115, 28, 107)插入初始状态为空的平衡二叉排序树, 使得在每次插入后保持该树仍然是平衡二叉树。请依次画出每次插入后所形成的平衡二叉排序树。
4. 给定一个关键字集合 {25, 18, 34, 9, 14, 27, 42, 51, 38}, 假定查找各关键字的概率相同, 请画出其最佳二叉排序树。
5. 画出一个二叉树, 使得它既满足大根堆的要求又满足二叉排序树的要求。
6. 将二叉排序树 T 的前序序列中的关键字依次插入到一棵空的二叉排序树中, 得到二叉排序树 T' , T' 与 T 是否相同? 为什么?
7. 设二叉排序树中的关键字互不相同, 则其中的最小元素必无左子女, 最大元素必无右子女。此命题是否正确? 最小元素和最大元素一定是叶结点吗? 一个新元素总是作为叶结点插入二叉排序树吗?

【答案解析】

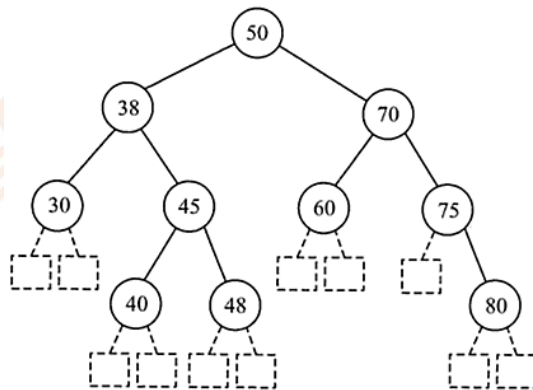
1.解答: 先序序列为(50, 38, 30, 45, 40, 48, 70, 60, 75, 80), 二叉树的中序序列是一个有序序列, 故为(30,38, 40, 45, 48, 50, 60, 70, 75, 80), 由先序序列和中序序列可以构造出对应的二叉树, 如下图所示。

查找成功的平均查找长度为

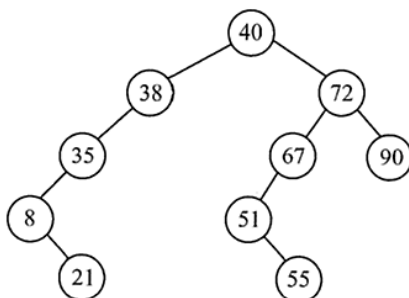
$$ASL = (1 \times 1 + 2 \times 2 + 3 \times 4 + 4 \times 3) / 10 = 2.9$$

图中的方块结点为虚构的查找失败结点, 其查找路径为从根结点到其父结点 (圆形结点) 的结点序列, 故对应的查找失败平均长度为

$$ASL = (3 \times 5 + 4 \times 6) / 11 = 39 / 11$$



2.解答: 根据二叉排序树的定义, 该序列所对应的二叉排序树如下图所示。

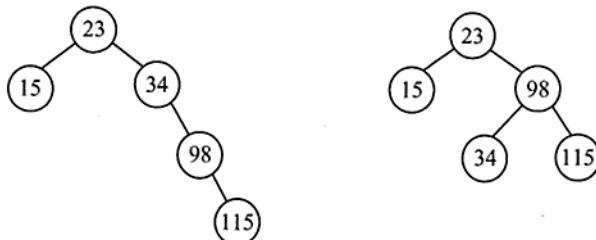


平均查找长度为 $ASL = (1 + 2 \times 2 + 3 \times 3 + 4 \times 2 + 5 \times 2) / 10 = 3.2$ 。

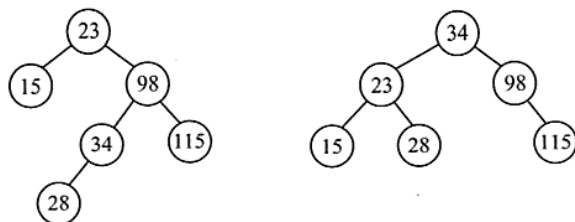
3.解答:



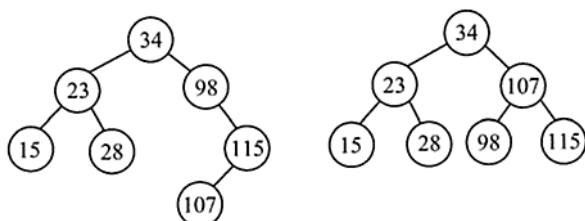
第一步: 插入结点 34, 23, 15 后, 需要根结点 34 的子树做 LL 调整。



第二步：插入结点 98, 115 后，需要根结点 34 的子树做 RR 调整。

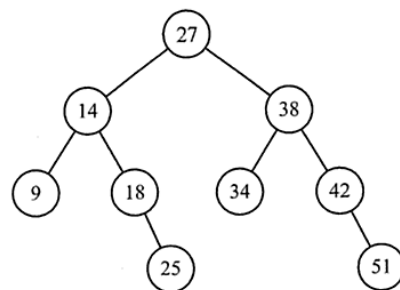


第三步：插入结点 28 后，需要根结点 23 的子树做 RL 调整。

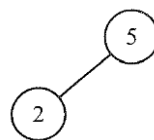


第四步：插入结点 107 后，需要根结点 98 的子树做 RL 调整。

4. 解答：当各关键字的查找概率相等时，最佳二叉排序树应是高度最小的二叉排序树。构造过程分两步走：首先对各关键字按值从小到大排序，然后仿照折半查找的判定树的构造方法构造二叉排序树。这样得到的就是最佳二叉排序树，结果如右图所示。



5. 解答：大根堆要求根结点的关键字值既大于等于左子女的关键字值，又大于等于右子女的关键字值。二叉排序树要求根结点的关键字值大于左子女的关键字值，同时小于右子女的关键字值。两者的交集是：根结点的关键字值大于左子女的关键字值。这意味着它是一棵左斜单支树，但大根堆要求是完全二叉树，因此最后得到的只能是如右图所示的两个结点的二叉树。读者也可能会注意到，当只有一个结点时，显然是满足题意的，但我们不举一个结点的例子是为了体现出排序树与大根堆的区别。



6. 答：构造出来的二叉排序树与原来的相同。

因为二叉排序树是二叉树，它的前序序列的第一个元素一定是二叉排序树的根，而对应前序序列的根后面的所有元素分为两组：从根的后一元素开始，其值小于根的值的一组元素就是树的左子树的结点的前序序列，剩下的元素的值大于根的值，即为树的右子树的结点的前序序列。在把前序序列的元素依次插入初始为空的二叉排序树时，第一个元素就成为树的根，它后面第一组元素的值都小于根结点的值，可以递归建立根的左子树；第二组元素的值都大于根结点的值，可以递归地建立根的右子树。最后插入的结果就是一棵与原来二叉排序树相同的二叉排序树。

7. 答：1) 最小元素必无左子女，最大元素必无右子女，此命题正确。

根据二叉排序树的定义，二叉排序树的根结点的关键字值一定大于左子树（若非空）上所有结点的关键字值，同时一定小于右子树（若非空）上所有结点的关键字值，而根结点的左、右子树也是二叉排序树。这是一个递归的定义。因此寻找最小元素的过程可以是一个递归的过程，逐层查找树（或子树）根结点的左子树，直至根结点的左子树空为止，这个子树的根结点就是最小元素所在结点，它无左子女；同样，寻找最大元素的过程也可以是一个递归的过程，逐层查找树（或子树）根结点的右子树，直至根结点的右子树空为止，这个子树的根结点就是最大元素所在结点，它无右子女。

2) 最小元素和最大元素不一定是叶结点。

具有最小关键字值的结点可以有右子树, 具有最大关键字值的结点可以有左子树。

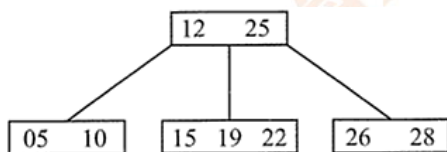
3) 一个新元素总是作为叶结点插入二叉排序树的。

在插入新元素时, 插入算法总是先调用查找算法, 从根开始查找是否具有相等关键字值的结点, 如果查找成功, 则不插入; 如果查找失败, 则新结点作为叶结点插入到刚刚查找失败的位置。

7.5 B 树和 B+树

(一) 选择题

1. 下图所示是一棵()。



A. 4 阶 B 树 B. 4 阶 B+树 C. 3 阶 B 树 D. 3 阶 B+ 树

2. 下列关于 m 阶 B 树的说法中, 错误的是()。

- A. 根结点至多有 m 棵子树
- B. 所有叶结点都在同一层次上
- C. 非叶结点至少有 $m/2$ (m 为偶数) 或 $(m+1)/2$ (m 为奇数) 棵子树
- D. 根结点中的数据是有序的

3. 在一棵 m 阶 B 树中做插入操作前, 若一个结点中的关键字个数等于(), 则必须分裂成两个结点; 向一棵 m 阶的 B 树做删除操作前, 若一个结点中的关键字个数等于(), 则可能需要同它的左兄弟或右兄弟结点合并成一个结点。

A. $m, \lceil m/2 \rceil - 2$ B. $m-1, \lceil m/2 \rceil - 1$ C. $m+1, \lceil m/2 \rceil$ D. $m/2, \lceil m/2 \rceil + 1$

4. 具有 n 个关键字的 m 阶 B 树, 应有() 个叶结点。

A. $n+1$ B. $n-1$ C. mn D. $nm/2$

5. 含有 n 个非叶结点的 m 阶 B 树中至少包含() 个关键字。

A. $n(m+1)$ B. n C. $n(\lceil m/2 \rceil - 1)$ D. $(n-1)(\lceil m/2 \rceil - 1) + 1$

6. 下面关于 m 阶 B 树说法正确的是()。

- ①每个结点至少有两棵非空子树; ②树中每个结点至多有 $m-1$ 个关键字
- ③所有叶子在同一层上; ④当插入一个数据项引起 B-树结点分裂后, 树长高一层

A. ①②③ B. ②③ C. ②③④ D. ③

7. 下面关于 B-树和 B+树的叙述中, 不正确的是()。

- A. B-树和 B+树都是平衡的多叉树
- B. B-树和 B+树都可用于文件的索引结构
- C. B-树和 B+树都能有效地支持顺序查找
- D. B-树的叶子结点不包含关键字, 而 B+树的叶子结点包含关键字

8. m 阶 B-树是一棵()。

- A. m 叉查找树 B. m 叉平衡查找树
- C. $m-1$ 叉平衡查找树 D. $m+1$ 叉平衡查找树

9. 在一棵 m 阶的 B-树中, 除根结点以外的结点的分支数 S 应满足()。

A. $\lceil m/2 \rceil \leq S \leq m$ B. $\lceil m/2 \rceil - 1 \leq S \leq m-1$ C. $\lceil rn/2 \rceil \leq S \leq rn-1$ D. $\lceil rn/2 \rceil - 1 \leq S \leq rn$

【答案解析】

1. A。关键字数目比子树数目少, 所以不是 B+树, 而是 B 树。又因为 m 阶 B 树结点关键字数最多为 $m-1$, 有一个结点关键字个数为 3, 所以不可能为 3 阶。

2. C。除根结点外的所有非终端结点至少有 $\lceil m/2 \rceil$ 棵子树。对于根结点, 最多有 m 棵子树, 若其不是叶结点, 则至少有 2 棵子树。

3. A。由于 B 树中每个结点内的关键字个数最多为 $m-1$, 所以当关键字个数大于 $m-1$ 时, 就应该分裂。而每个结点内的关键字个数至少为 $\lceil m/2 \rceil - 1$ 个, 所以当关键字个数少于 $\lceil m/2 \rceil - 1$ 时, 应与其他结点合并。若将本题题干改为 B+树, 请读者自行思考上述问题的解答。

4. A。B 树的叶结点对应查找失败的情况, 对有 n 个关键字的查找集合进行查找, 失败可能性有 $n+1$ 种。

5. D。除根结点外, m 阶 B 树中的每个非叶结点至少有 $\lceil m/2 \rceil - 1$ 个关键字, 根结点至少有一个关键字, 所以总共包含的关键字最少个数 $= (n-1)(\lceil m/2 \rceil - 1) + 1$ 。

注意: 由以上题目可知 B 树和 B+树的定义与性质尤为重要, 需要熟练掌握。

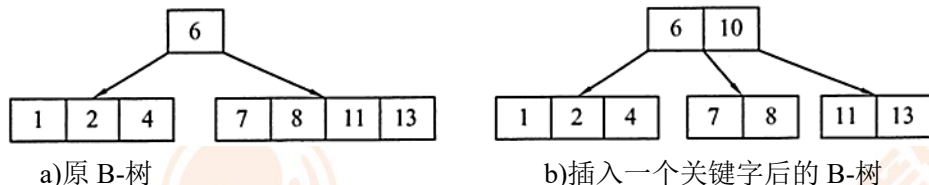
6. B。本题考查 B- 树的定义及插入操作。

m 阶 B- 树的根结点至少有两棵子树, 并且这两颗子树可以是空树, 其余结点至少有 $\lceil m/2 \rceil$ 个分支, 即 $\lceil m/2 \rceil$ 棵子树, 因此①不对。

每个结点中关键字的个数比分支数少 1, m 阶 B- 树的一个结点中至多有 m 个分支, 因此最多有 $m-1$ 个关键字, 因此②正确。

B- 树是平衡的多路查找树, 叶子结点均在同一层上, 因此③正确。

发生结点分裂的时候不一定会使树长高。例如, 向下图 a) 中的 B- 树插入一个关键字 10, 变成下图 b) 中的 B- 树, 使得第二层右端的一个结点分裂成两个, 但是树并没有长高, 因此④不对。



7. C。本题考查 B- 树和 B+树的性质。由定义知选项 A、B、D 正确。B+树所有的叶子结点包含了全部的关键字信息, 且叶子结点本身依照关键字的大小自小而大顺序连接, 可以进行顺序查找, 而 B-树则不可以, 因此选项 C 错误。

8. B。本题考查 B- 树的定义。m 阶 B-树说明每个结点至多有 m 个分支, B-树的所有叶子结点都在同一层上。因此, m 阶 B- 树是一棵 m 叉平衡排序树, 故本题选 B。

9. A。本题考查 B- 树的定义。在 m 阶 B- 树中, 除了根结点以外, 其余结点的分支数为 $\lceil m/2 \rceil \leq S \leq m$, 因此本题选 A。

(二) 综合应用题

1. 设有一棵空的 3 阶 B- 树, 依次插入关键字 {30, 20, 10, 40, 80, 58, 47, 50, 29, 22, 56, 98, 99}, 请画出该树。

2. 高度为 h 的 m 阶 B- 树至少有多少个结点?

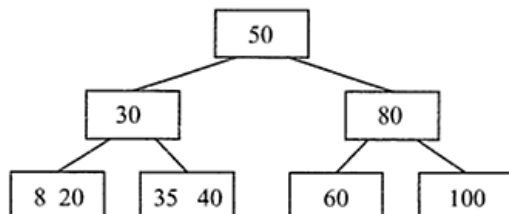
3. 直接在平衡二叉树中查找关键字 Key 与在对其进行中序遍历输出的有序序列中查找关键字

Key, 其效率是否相同? 按关键字有序序列输入关键字, 构造一棵二叉排序树, 然后对此树进行查找, 其效率如何? 为什么?

4. 给定一组关键字 {20, 30, 50, 52, 60, 68, 70}, 给出创建一棵 3 阶 B 树的过程。

5. 对如下图所示的 3 阶 B 树, 依次执行下列操作, 画出各步操作的结果。

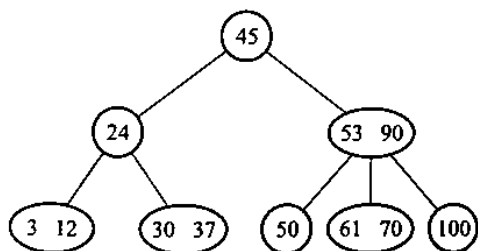
1) 插入 90 2) 插入 25 3) 插入 45 4) 删除 60 5) 删除 80



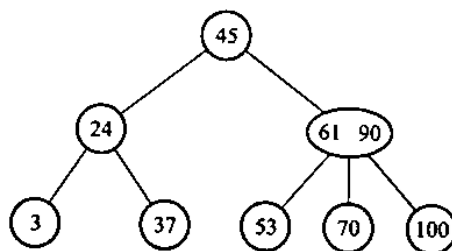
6. 已知 2 棵 3 阶 B 树如下图所示。

(1) 对如图 (a) 所示树 1, 请分别画出先后插入 26 和 85 两个新结点后的树形。

(2) 对如图 (b) 所示树 2, 请分别画出先后删除 53 和 37 两个结点后的树形。



(a) 树 1

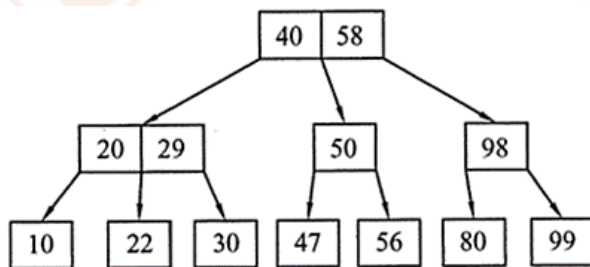


(b) 树 2

【答案解析】

1. 分析: 本题考查一般的 B-树建立, 按照例题中所讲的步骤解题即可。

3 阶 B-树的结点中, 关键字个数范围为 1~2。按照关键字序列逐个插入后所得的 B-树如下图所示。



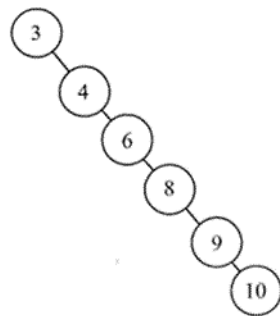
2. 分析: 由 B-树的定义可知, 根结点最少有两个分支, 其余结点分支数为 $\lceil m/2 \rceil \sim m$ 个。因此, 第一层有 1 个结点, 第二层至少有 2 个结点, 第三层有 $2 \times \lceil m/2 \rceil$ 个结点, 第四层有 $2 \times \lceil m/2 \rceil^2$ 个结点……第 $h+1$ 层至少有 $2 \times \lceil m/2 \rceil^{h-1}$ 个结点 ($h \geq 2$)。结点总数为:

$$N = 1 + 2 + 2 \times \lceil m/2 \rceil + 2 \times \lceil m/2 \rceil^2 + \dots + 2 \times \lceil m/2 \rceil^{h-1} = 2 \times (1 - \lceil m/2 \rceil^h) / (1 - \lceil m/2 \rceil) + 1$$

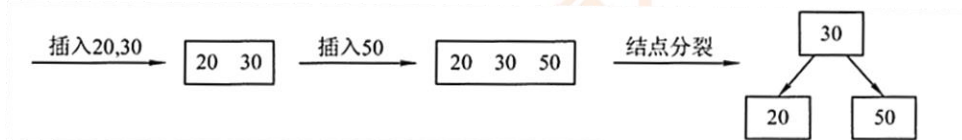
注意: 高度为 h 的 B-树有 $h+1$ 层, 因本书规定高度不包含叶子结点层, 这在不同的书中有不同的规定。有的书中高度包含叶子结点层, 则高度为 h 的 B-树有 h 层。这需要考生根据自己目标学校的指定参考书或者历年考卷来确定其具体规定。

3.分析: 在平衡二叉树上查找关键字 Key, 走了一条从根到叶子的路径, 时间复杂度可以通过树的高度来反映, 是 $O(\log_2 n)$ 。在中序遍历输出的序列中查找关键字 Key, 如果采用顺序查找, 则时间复杂度是 $O(n)$; 如果采用折半查找, 则时间复杂度是 $O(\log_2 n)$ 。

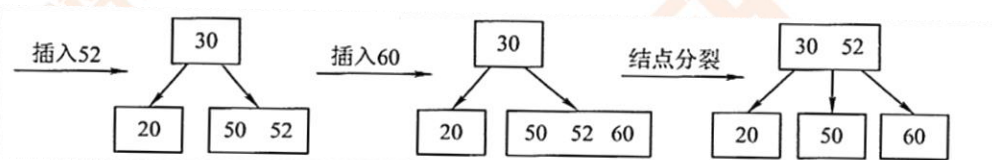
按序输入建立的二叉排序树, 因为插入元素是有序的, 因此所有的插入操作都会发生在最左边的叶子结点的左指针上, 或者最右边的叶子结点的右指针上, 这样所形成的二叉排序树蜕变为单枝树, 折半查找也蜕变成了顺序查找, 如下图所示, 其平均查找长度是 $(n+1)/2$, 时间复杂度是 $O(n)$ 。



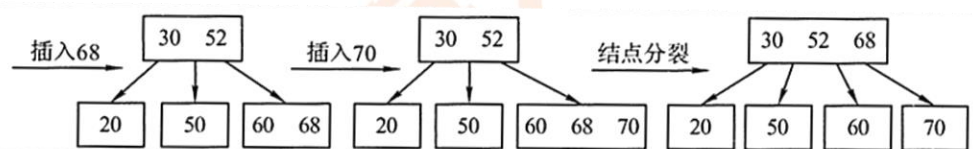
4. 解答: $m=3$, 因此除根结点外, 非叶子结点关键字个数为 1~2。



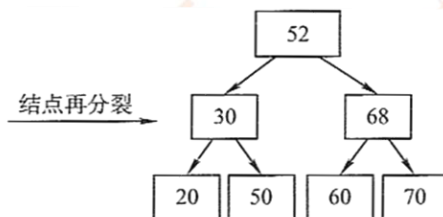
如上图所示, 首先插入 20, 30, 结点内关键字个数不超过 $\lceil m/2 \rceil = 2$, 不会引起分裂; 插入 50, 插入 20, 30 所在的结点, 引起分裂, 结点内第 $\lceil m/2 \rceil$ 个关键字 30 上升为父结点。



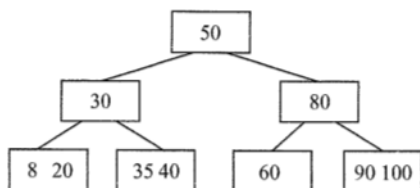
如上图所示, 插入 52, 插入 50 所在的结点, 不会引起分裂; 继续插入 60, 插入 50, 52 所在的结点, 引起分裂, 52 上升到父结点中, 不会引起父结点的分裂。



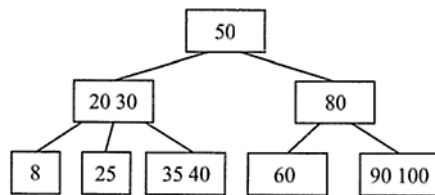
如上图所示, 插入 68, 插入 60 所在的结点, 不会引起分裂; 继续插入 70, 插入 60, 68 所在的结点, 引起分裂, 68 上升为新的父结点, 68 上升到 30, 52 所在的结点后, 会继续引起该结点的分裂, 故 52 上升为新的根结点。最后得到的 B 树如下图所示。



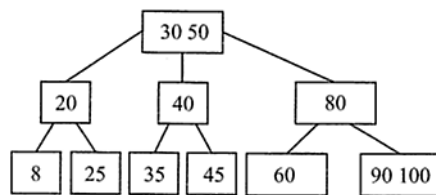
5. 解答: (1) 插入 90: 将 90 插入 100 所在的结点, 插入 90 后该结点中的元素个数不超过 $\lceil 3/2 \rceil = 2$, 不会引起结点的分裂, 插入后的 B 树如下图所示。



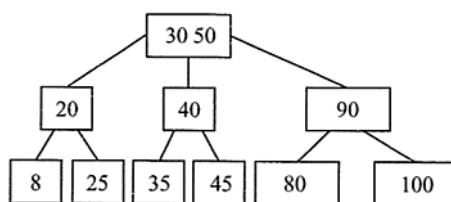
(2) 插入 25: 将 25 插入 8, 20 所在的结点, 插入后结点内的元素个数为 3, 引起分裂。故将结点内的中间元素 20 上升到父结点中, 此时父结点中的元素个数为 2 (元素 20 和 30), 不会引起继续分裂, 插入 25 后的 B 树如下图所示。



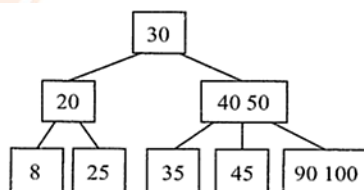
(3) 插入 45: 将 45 插入 35, 40 所在的结点, 引起分裂, 中间元素 40 上升到父结点 (20, 30 所在的结点) 中, 引起父结点分裂, 中间元素 30 上升到父结点 (50 所在的结点) 中, 两次分裂后的 B 树如下图所示。



(4) 删除 60: 删除 60 后, 其所在的结点元素为空, 从而导致借用右兄弟结点的元素, 调整后的 B 树如下图所示。

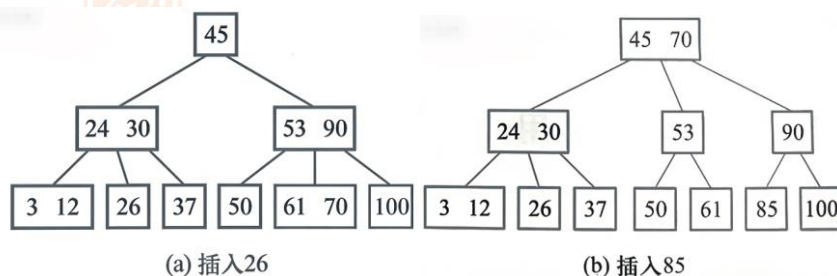


(5) 删除 80: 删除 80 后, 导致 80 所在结点的父结点与其右兄弟结点合并, 这时父结点元素个数为 0, 再次对父结点进行调整。将 50 与 40 合并成一个新结点, 则 90, 100 所在结点为这个结点的子结点。从而构造的 B 树如下图所示。注意, 这次调整的过程实际上包含多次调整过程, 希望读者对照考点讲解中的删除过程仔细思考。

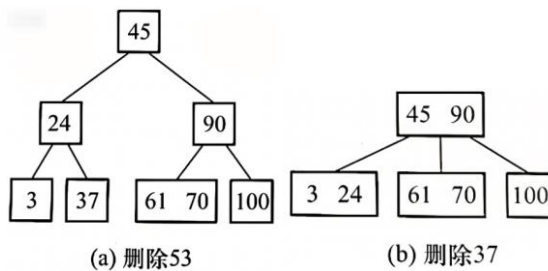


注意: B 树中结点的插入、删除操作 (特别是插入、删除后的结点分裂与合并) 是本节的重点, 也是难点, 请读者务必熟练掌握。

6. (1) 插入结点 26 和 85 后的树形如下图所示:



(2) 删除结点 53 和 37 后的树形如下图所示。



7.6 散列表

(一) 选择题

- 只能在顺序存储结构上进行的查找方法是()。
 - 顺序查找法
 - 折半查找法
 - 树型查找法
 - 散列查找法
- 散列查找一般适用于()的情况下的查找。
 - 查找表为链表
 - 查找表为有序表
 - 关键字集合比地址集合大得多
 - 关键字集合与地址集合之间存在对应关系
- 下列关于散列表的说法中, 正确的是()。
 - 若散列表的填装因子 $a < 1$, 则可避免碰撞的产生
 - 散列查找中不需要任何关键字的比较
 - 散列表在查找成功时平均查找长度与表长有关
 - 若在散列表中删除一个元素, 不能简单地将该元素删除
 - I 和 IV
 - II 和 III
 - III
 - IV
- 在开放定址法中散列到同一个地址而引起的“堆积”问题是由于()引起的。
 - 同义词之间发生冲突
 - 非同义词之间发生冲突
 - 同义词之间或非同义词之间发生冲突
 - 散列表“溢出”
- 下列关于散列冲突处理方法的说法中, 正确的有()。
 - 采用再散列法处理冲突时不易产生聚集
 - 采用线性探测法处理冲突时, 所有同义词在散列表中一定相邻
 - 采用链地址法处理冲突时, 若限定在链首插入, 则插入任一个元素的时间是相同的
 - 采用链地址法处理冲突易引起聚集现象
 - I 和 III
 - I、II 和 III
 - III 和 N
 - I 和 W
- 设有一个含有 200 个表项的散列表, 用线性探测法解决冲突, 按关键字查询时找到一个表项的平均探测次数不超过 1.5, 则散列表项应能够容纳()个表项(设查找成功的平均查找长度为 $ASL = [1 + 1/(1 - a)]/2$, 其中 a 为装填因子)。
 - 400
 - 526
 - 624
 - 676
- 对包含 n 个元素的散列表进行查找, 平均查找长度()。
 - 为 $O(\log_2 n)$
 - 为 $O(1)$
 - 不直接依赖于 n
 - 直接依赖于表长 m
- 采用开放定址法解决冲突的散列查找中, 发生聚集的原因主要是()。
 - 数据元素过多
 - 负载因子过大
 - 散列函数选择不当
 - 解决冲突的方法选择不当
- 在采用链地址法处理冲突所构成的散列表上查找某一关键字, 则在查找成功的情况下, 所探

测的这些位置上的键值 () ; 若采用线性探测法, 则()。

- A. 一定都是同义词 B. 不一定都是同义词
C. 都相同 D. 一定都不是同义词

10. 设散列表长 $m = 14$, 散列函数为 $H(\text{key}) = \text{key} \% 11$, 表中仅有 4 个结点 $H(15) = 4$, $H(38) = 5$, $H(61) = 6$, $H(84) = 7$, 若采用线性探测法处理冲突, 则关键字为 49 的结点地址是()。

- A. 8 B. 3 C. 5 D. 9

11. 设有一组记录的关键字为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}, 用链地址法构造 Hash 表, Hash 函数为 $H(\text{key}) = \text{key} \bmod 13$, 散列地址为 1 的链中有 () 个关键字。

- A. 1 B. 2 C. 3 D. 4

12. 若采用链地址法构造散列表, Hash 函数为 $H(\text{key}) = \text{key} \bmod 17$, 则需①中的 () 个链表。这些链的链首指针构成一个指针数组, 数组的下标范围为②中的()。

- ① A. 17 B. 13 C. 16 D. 任意
② A. 0~17 B. 1~17 C. 0~16 D. 1~16

13. 假定有 K 个关键字互为同义词, 若用线性探查法把这 K 个关键字存入散列表中, 至少要进行 () 探测。

- A. $k-1$ 次 B. k 次 C. $k+1$ 次 D. $k(k+1)/2$ 次

14. Hash 函数有一个共同的性质, 即函数值应当尽量以 () 取其值域的每个值。

- A. 最大概率 B. 最小概率 C. 平均概率 D. 同等概率

15. Hash 表的地址区间为 0~16, Hash 函数为 $H(K) = K \bmod 17$ 。采用线性探查法处理冲突, 并将关键字序列 {26, 25, 72, 38, 8, 18, 59} 依次存储到 Hash 表中。

(1) 关键字 59 存放在 Hash 表中的地址是 ()。

- A. 8 B. 9 C. 10 D. 11

(2) 存放关键字 59 需要探查的次数是 ()。

- A. 2 B. 3 C. 4 D. 5

16. 将 10 个元素散列到 100 000 个单元的 Hash 表中, 则 () 产生冲突。

- A. 一定会 B. 一定不会 C. 仍可能会 D. 不能确定

17. 设有 n 个关键字, Hash 查找法的时间复杂度是 ()。

- A. $O(1)$ B. $O(n)$ C. $O(\log_2 n)$ D. $O(n^2)$

18. 已知一个线性表 {1, 13, 12, 34, 38, 33, 27, 22}, 假定采用 $h(k) = k \% 11$ 计算散列地址进行散列存储, 若用链地址法处理冲突, 则查找成功的平均查找长度为 ()。

- A. 1 B. 9/8 C. 13/11 D. 13/8

19. 在采用链地址法处理冲突所构成的散列表上查找某一关键字, 则在查找成功的情况下, 所探测的这些位置上的键值 ()。

- A. 一定都是同义词 B. 不一定都是同义词 C. 都相同 D. 一定都不是同义词

20. 下列关于哈希 (散列) 查找的说法中正确的是 ()。

- A. 哈希函数构造得越复杂越好, 因为这样随机性好, 冲突小
B. 除留余数法是所有哈希函数中最好的
C. 不存在特别好与坏的哈希函数, 要视情况而定
D. 若需在哈希表中删去一个元素, 不管用何种方法解决冲突都只要简单地将该元素删去即可

21. 在构造散列表方面, 下列说法中正确的是 ()。

- A. 再散列法在处理冲突时不会产生聚集
B. 散列表的装载因子越大, 说明空间利用率越好, 因此应使装载因子尽量大

- C. 散列函数选择得当可以减少冲突现象
D. 对于任何关键字都不可能找到不产生冲突的散列函数

【答案解析】

1. B。顺序查找可以是顺序存储或链式存储；折半查找只能是顺序存储且要求关键字有序；树形查找法要求采用树的存储结构，既可以采用顺序存储也可以采用链式存储；散列查找中的链地址法解决冲突时，采用的是顺序存储与链式存储相结合的方式。

2. D。关键字集合与地址集合之间存在对应关系时，通过散列函数表示这种关系。这样，查找以计算散列函数而非比较的方式进行查找。

3. D。冲突（碰撞）是不可避免的，与装填因子无关，因此需要设计处理冲突的方法，I 错误。散列查找的思想是计算出散列地址来进行查找，然后比较关键字以确定是否查找成功，II 错误。散列查找成功的平均查找长度与装填因子有关，与表长无关，III 错误。在开放定址的情形下，不能随便删除散列表中的某个元素，否则可能会导致搜索路径被中断（因此通常的做法是在要删除的地方做删除标记，而不是直接删除），IV 正确。

4. C。在开放定址法中散列到同一个地址而产生的“堆积”问题，是同义词冲突的探查序列和非同义词之间不同的探查序列交织在一起，导致关键字查询需要经过较长的探测距离，降低了散列的效率。因此要选择好的处理冲突的方法来避免“堆积”。

5. A。利用再散列法处理冲突时，按一定的距离，跳跃式地寻找“下一个”空闲位置，减少了发生聚集的可能，I 正确。散列地址 i 的关键字，为解决冲突形成的某次探测地址为 i 的关键字，都争夺地址 $i, i+1, \dots$ ，因此不一定相邻，II 错误。III 正确。同义词冲突不等于聚集，链地址法处理冲突时将同义词放在同一个链表中，不会引起聚集现象，IV 错误。

6. A。若有 200 个表项要放入散列表，采用线性探测法解决冲突，限定查找成功的平均查找长度不超过 1.5，则：

$$ASL_{成功} = \frac{1}{2} \left(1 + \frac{1}{1-a} \right) \leq 1.5 \Rightarrow a = \frac{200}{m} \Rightarrow m \geq 400$$

7. C。在散列表中，平均查找长度与装填因子 a 直接相关，表的查找效率不直接依赖于表中已有表项个数 n 或表长 m 。若散列表中存放的记录全部是某个地址的同义词，则平均查找长度为 $O(n)$ 而非 $O(1)$ 。

8. D。聚集是因选取不当的处理冲突的方法，而导致不同关键字的元素对同一散列地址进行争夺的现象。用线性再探测法时，容易引发聚集现象。

9. A, B。因为在链地址法中，映射到同一地址的关键字都会链到与此地址相对应的链表上，所以探测过程一定是在此链表上进行的，从而这些位置上的关键字均为同义词；但在线性探测法中出现两个同义关键字时，会把该关键字对应地址的下一个地址也占用掉，两个地址分别记为 $Addr$ 、 $Addr+1$ ，查找一个满足 $H(key) = Addr+1$ 的关键字 key 时，显然首次探测到的不是 key 的同义词。

10. A。线性探测法的公式为 $H_i = (H(key) + d_i) \% m$ ，其中 $d_i = 1, 2, 3, \dots, m-1$ 。 $H(49) = 49 \% 11 = 5$ ，发生冲突； $H_1 = (H(49) + 1) \% 14 = 6$ ，发生冲突； $H_2 = (H(49) + 2) \% 14 = 7$ ，发生冲突； $H_3 = (H(49) + 3) \% 14 = 8$ ，无冲突。选 A。

11. D。本题考查用除留余数法的 Hash 函数以及链地址法的冲突解决方法来建立 Hash 表的过程。只需用各个关键字与 13 做取余运算，结果为 1 的就是地址为 1 的链中的关键字，经运算可知 14、1、27、79 将被插入到地址为 1 的链中，因此本题选 D。

12. A, C。本题考查用链地址法处理冲突。由 Hash 函数可知，地址范围为 0~16，有 17 个地

址, 需要 17 个链表, 因此本题选 A、C。

13. D。本题考查 Hash 冲突解决方法中的线性探查法。在不发生堆积现象的时候, K 个互为同义词的关键字中, 第一个关键字需要探查一次, 以后的关键字均比前一个关键字要多探查一次, 则每个关键字需要探查的次数构成一个首项为 1、公差为 1、项数为 K 的等差数列, 因此总共需要 $k(k+1)/2$ 次探查。如果在这个过程中发生了堆积现象 (如下图所示, 发生了堆积, a 占据了 k_{i+1} 的地址, 则 k_{i+1} 的探查次数将比没有发生堆积现象的时候多 1 次), 则会发生更多的探查, 因此至少需要探查 $k(k+1)/2$ 次, 故本题选 D。

.....	K_{i-1}	k_i	a	K_{i+1}	
-------	-----------	-------	---	-----------	-------

14. D。本题考查构造 Hash 函数应遵循的原则。这个原则在知识点讲解中并没有明确提到, 但是应该知道, Hash 函数要尽可能地减少冲突的发生来提高查找的效率。如何减少冲突的发生呢? 所谓冲突, 通俗来讲就是关键字在表中扎堆, 即关键字经 Hash 函数处理后落在表中各个位置上的概率不相等。因此要减少冲突, 应当使得函数值以等概率取其值域中的每个值。这种题目就需要根据已有知识来推理得出答案, 考生在学习的过程中要重在理解。

15. D, C。本题考查 Hash 表的建立以及冲突处理。先用 59 之前的关键字建表过程如下: $H(26)=9$, 不冲突; $H(25)=8$, 不冲突; $H(72)=4$, 不冲突; $H(38)=4$, 冲突, 冲突处理后地址为 5; $H(8)=8$, 冲突, 冲突处理后地址为 10; $H(18)=1$, 不冲突。当放入关键字 59 时, $H(59)=8$, 冲突, 后移一位得地址 9 也冲突, 继续后移得地址 10 也冲突, 继续后移得地址 11 不冲突。因此 59 在表中的地址为 11, 共探查 4 次, 故本题选 D、C。

16. C。本题考查装填因子的性质。装填因子越大, 则越可能发生冲突 (这是显而易见的, 装填因子大, 证明表中的关键字多, 而空位置少, 所以冲突容易发生); 装填因子越小, 则冲突发生的可能性越小。但是不论什么情况都不能绝对避免冲突, 因此本题选 C。

17. A。本题考查 Hash 表的查找效率。Hash 查找法是由关键字结合 Hash 函数和冲突处理方法直接算出关键字在表中的位置, 与表的长度 n 无关, 其查找的时间复杂度对于 n 为常量级, 即 $O(1)$ 。

18. D。查找成功的平均查找长度 $ASL = (4 + 1 + 3 + 2 + 1 + 3) / 8 = 13/8$, 故答案为选项 D。

19. A。采用开放定址法时, 如线性探测法, 空闲表项向同义词和非同义词都开放; 采用链地址法处理冲突时, 表项仅向同义词开放, 相同位置上的关键字一定是同义词, 故选项 A 正确。

20. C。理想的散列函数应尽可能地减少冲突, 与散列函数构造的复杂性无关, 选项 A 错误。除留余数法构造计算简单, 但最好的散列函数要根据具体的需求和数据特点来选择, 选项 B 错误。散列函数的好坏要根据具体情况而定, 相同的散列函数在不同情况下的性能不尽相同, 选项 C 正确。从散列表中删除元素时, 如果采用拉链法, 可以直接将元素删去; 若散列表对非同义词也开放, 例如线性探测法, 则不能简单地将元素物理删除, 而是需要标记删除或其他处理方式, 选项 D 错误。

21. C。再散列法在处理冲突时会产生“聚集”现象, 选项 A 错误。散列表的装载因子等于表中记录数/表大小, 装载因子越大, 越容易产生冲突, 应避免装载因子过大, 选项 B 错误。好的散列函数可以减少冲突现象, 选项 C 正确。对于某些特定的关键字 (注意这里只是针对某种特殊情况而言), 存在不冲突的散列函数, 选项 D 错误。

(二) 综合应用题

1. 若要在散列表中删除一个记录, 应如何操作? 为什么?

2. 假定把关键字 key 散列到有 n 个表项 (从 0 到 n-1 编址) 的散列表中。对于下面的每个函数 $H(key)$ (key 为整数), 这些函数能够当作散列函数吗? 若能, 它是一个好的散列函数吗? 说明

理由。设函数 $\text{random}(n)$ 返回一个 0 到 $n-1$ 之间的随机整数 (包括 0 与 $n-1$ 在内)。

- 1) $H(\text{key}) = \text{key} / n$ 。
- 2) $H(\text{key}) = 1$ 。
- 3) $H(\text{key}) = (\text{key} + \text{random}(n)) \% n$ 。
- 4) $H(\text{key}) = \text{key} \% p(n)$; 其中 $p(n)$ 是不大于 n 的最大素数。

3. 设散列表为 $HT[0 \dots 12]$, 即表的大小为 $m = 13$ 。现采用双散列法解决冲突, 散列函数和再散列函数分别为:

$$H_0(\text{key}) = \text{key} \% 13 \quad \text{注: \% 是求余数运算 (MOD)}$$

$$H_i = (H_{i-1} + \text{REV}(\text{key} + 1) \% 11 + 1) \% 13; \quad i = 1, 2, 3, \dots, m-1$$

其中, 函数 $\text{REV}(x)$ 表示颠倒十进制数 x 的各位, 如 $\text{REV}(37) = 73$, $\text{REV}(7) = 7$ 等。若插入的关键码序列为 (2, 8, 31, 20, 19, 18, 53, 27), 请回答:

- 1) 画出插入这 8 个关键码后的散列表。
- 2) 计算查找成功的平均查找长度 ASL。

4. 已知一组关键字为 {26, 36, 41, 38, 44, 15, 68, 12, 6, 51, 25}, 用链地址法解决冲突。假设装填因子 $\alpha = 0.75$, Hash 函数的形式为 $H(\text{Key}) = \text{Key} \bmod P$ 。

- 1) 构造 Hash 函数。
- 2) 计算等概率情况下查找成功时的平均查找长度 ASL_1 。
- 3) 计算等概率情况下查找失败时的平均查找长度 ASL_2 (将与关键字的比较次数计算在内)。

5. 已知记录关键字集合为 {53, 17, 19, 61, 98, 75, 79, 63, 46, 49}, 要求用除留余数法将关键字散列到地址区间 {100, 101, 102, 103, 104, 105, 106, 107, 108, 109} 内, 若产生冲突, 则用开放定址法的线性探查法解决。要求写出选用的 Hash 函数及形成的 Hash 表, 计算出查找成功时的平均查找长度 (设等概率情况)。

【答案解析】

1. 解答: 在散列表中删除一个记录, 在拉链法情况下可以物理地删除。但在开放定址法情况下, 不能物理地删除, 只能做删除标记。该地址可能是该记录的同义词查找路径上的地址, 物理地删除就中断了查找路径, 因为查找时碰到空地址就认为是查找失败。

2. 解答: 1) 不能作为散列函数, 因为 key/n 可能大于 n , 这样就无法找到适合的位置。
- 2) 能够作为散列函数, 但不是一个好的散列函数, 因为所有关键字都映射到同一位置, 造成大量的冲突机会。
- 3) 不能当作散列函数, 因为该函数的返回值不确定, 这样无法进行正常的查找。
- 4) 能够作为散列函数, 是一个好的散列函数。

3. 解答: 1) $H_0(2) = 2$, $H_0(8) = 8$, $H_0(31) = 5$, $H_0(20) = 7$, $H_0(19) = 6$, 没有冲突。 $H_0(18) = 5$, 发生冲突, $H_1(18) = (H_0(18) + \text{REV}(18+1) \% 11 + 1) \% 13 = (5+3+1) \% 13 = 9$, 没有冲突。 $H_0(53) = 1$, 没有冲突。 $H_0(27) = 1$, 发生冲突, $H_1(27) = (H_0(27) + \text{REV}(27+1) \% 11 + 1) \% 13 = (1+5+1) \% 13 = 7$, 发生冲突, $H_2(27) = (H_1(27) + \text{REV}(27+1) \% 11 + 1) \% 13 = 0$, 没有冲突。构造的散列表如下:

散列地址	0	1	2	3	4	5	6	7	8	9	10	11	12
------	---	---	---	---	---	---	---	---	---	---	----	----	----

关键字	27	53	2			31	19	20	8	18			
比较次数	3	1	1			1	1	1	1	2			

2)由 1)中散列表的构造过程, 各个关键字查找成功的比较次数如上表所示, 故有

$$ASL_{成功} = (3 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 2) / 8 = 11/8$$

4.分析: 此题为简单的 Hash 构造题目, 按照之前所讲的方法直接构造即可。

1)由装填因子 $\alpha=0.75$, 表中元素个数 $n=11$, 得表长为 $\lceil n/\alpha \rceil=15$ 。p 取不大于表长的最大素数, 因此 p 为 13。

所构造的 Hash 函数为 $H(Key)=Key \text{ Mod } 13$ 。

2)由 Hash 函数求各关键字的 Hash 地址如下: {26, 36, 41, 38, 44, 15, 68, 12, 6, 51, 25}

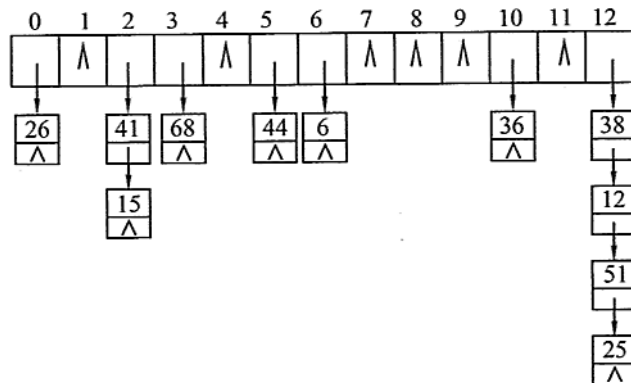
关键字	26	36	41	38	44	15	68	12	6	51	25
散列地址	0	10	2	12	5	2	3	12	6	12	12

$H(26)=0, H(36)=10, H(41)=2, H(38)=12, H(44)=5, H(15)=2, H(68)=3, H(12)=12, H(6)=6, H(51)=12, H(25)=12$ 。

由此可以构造出用链地址法处理冲突的 Hash 表, 如下图所示。

可知, $ASL_{成功} = (1+1+2+1+1+1+1+1+2+3+4) / 11 = 18/11$ 。

3) $ASL_{失败} = (1+0+2+1+0+1+1+0+0+0+1+0+4) / 13 = 11/13$ 。



5.分析: 本题是一般的除留余数法的变形, 可以先将关键字散列到 0~9 的范围内, 然后向右平移 100 个单位, 即可散列到 100~109 内。

设 Hash 函数为 $H(key)=key \text{ Mod } p+100$, 因表长为 10, p 取不大于表长的最大素数即 7, 因此得 Hash 函数为 $H(key)=key \text{ Mod } 7+100$ 。

所得的 Hash 表见下表所示

散列地址	100	101	102	103	104	105	106	107	108	109
关键字	98	63	79	17	53	19	61	75	46	49
比较次数	1	2	1	1	1	1	2	3	5	10

用线性探测在散列解决冲突时, 查找成功时的平均查找长度 $ASL = (1+2+1+1+1+1+2+3+5+10) / 10 = 27/10$ 。

7.7 字符串模式匹配

(一) 选择题

1. 空格串与空串是相同的, 这种说法 ()。
A. 正确 B. 错误
2. 串是一种特殊的线性表, 其特殊性体现在 ()。

- A. 可以顺序存储 B. 数据元素是一个字符
- C. 可以链式存储 D. 数据元素是多个字符
3. 设有两个串 p 和 q , 求 q 在 p 中首次出现的位置的运算称为 ()。
A. 连接 B. 模式匹配 C. 求子串 D. 求串长
4. 设串 s_1 为 "ABCDEFGH", s_2 为 "PQRST", 函数 $con(x,y)$ 返回 x 和 y 串的连接串, $subs(s,i,j)$ 返回 s 的从序号 i ($0 \leq i \leq len(s)-1$) 处字符开始的 j 个字符组成的子串, $len(s)$ 返回串 s 的长度, 则 $con(subs(s_1,1,len(s_2)),subs(s_1,len(s_2)-1,2))$ 的结果是 ()。
A. BCDEF B. BCDEFG C. BCPQRST D. BCDEFEF
5. 串的两种最基本的存储方式是 ()
A. 顺序存储方式和链式存储方式 B. 顺序存储方式和堆存储方式
C. 堆存储方式和链式存储方式 D. 堆存储方式和数组存储方式
6. 两个串相等的充分必要条件是 ()
A. 两串长度相等 B. 两串所包含的字符集合相等
C. 两串长度相等且对应位置的字符相等 D. 两串长度相等且所包含的字符集合相等
7. 空格串是 (①), 其长度等于 (②), 正确选项是 ()。
A. ①空串, ②0
B. ①由一个或者多个空格组成的字符串, ②其包含的空格个数
C. ①空串, ②未定义
D. ①由一个或者多个空格组成的字符串, ②未定义
8. 设字符串 str 为 "I_AM_A_TEACHER" (其中 "_" 代表空格字符), 则 str 的长度为 ()。
A. 13 B. 14 C. 15 D. 16
9. 设 str_1 为 "Demon", str_2 为 "_", str_3 为 "Hunter", 则 str_1 、 str_2 和 str_3 依次连接后的结果是 ()
A. "DemonHunter" B. "DemonHunter" C. "DemonHunter " D. "Demon Hunter"
10. 若串 $S = \text{"software"}$, 则其子串的数目是 (), 其中空串和 S 串本身这两个字符串也算作 S 的子串。
A. 8 B. 37 C. 36 D. 9
11. 设有两个串 S_1 和 S_2 , 求 S_2 在 S_1 中首次出现的位置的运算称为 ()。
A. 求子串 B. 判断是否相等 C. 模式匹配 D. 连接
12. KMP 算法的特点是在模式匹配时指示主串的指针 ()。
A. 不会变大 B. 不会变小 C. 都有可能 D. 无法判断
13. 设主串的长度为 n , 子串的长度为 m , 则简单的模式匹配算法的时间复杂度为 (), KMP 算法的时间复杂度为 ()。
A. $O(m)$ B. $O(n)$ C. $O(mn)$ D. $O(m+n)$
14. 已知串 $S = \text{'aaab'}$, 其 $next$ 数组值为 ()。
A. 0123 B. 0112 C. 0231 D. 1211

【答案解析】

1. B。空串在内存中通常只存储一个结束标记 '\0', 不占用存储实际字符的空间。空格串: 在内存中需要存储实际的空格字符数据以及结束标记。
2. B。串是限定了数据元素是字符的线性表, 串的数据元素必须是单个字符, 因此选 B。
3. B。根据 4.2 节所讲解的模式匹配算法可知本题选 B。
4. D。本题只需将各个中间结果展开即可很容易地得到答案:

len(s2)返回 5。

subs(s1, 1, len(s2))=>subs(s1, 1, 5)返回 "BCDEF"。

subs(s1, len(s2)-1, 2)=>subs(s1, 4, 2)返回 "EF"。

con(subs(s1, 1, len(s2)), subs(s1, len(s2)-1, 2))=>con("BCDEF", "EF")返回 "BCDEFEF"。

因此答案选 D。

5. A。串的两种最基本存储方式是顺序存储方式和链式存储方式。

6. C。

7. B。由空格串的定义可知本题选 B，注意空格串和空串的区别。

8. C。注意空格字符也是串中的一个字符，因此本题选 C。

9. D。

10. B。设待求串长度为 n，则：

长度为 0 的子串，1 个，即空串；

长度为 n 的子串，即 S 串本身，1 个；

长度为 1 的子串， $n-(1-1)=n$ 个；

长度为 2 的子串， $n-(2-1)=n-1$ 个；

长度为 3 的子串， $n-(3-1)=n-2$ 个；

...

长度为 n-1 的子串， $n-(n-1-1)=2$ 个；

因此，所有子串个数为 $1+1+2+\dots+n-2+n-1+n=n(n+1)/2+1$ 个。综上可知本题选 B。

11. C。求子串操作是从串 S 中截取第 i 个字符起长度为 1 的子串，A 错。BD 明显错。选 C。

12. B。在 KMP 算法的比较过程中，主串不会回溯，所以主串的指针不会变小。选 B。

13. C、D。尽管实际应用中，一般情况下简单的模式匹配算法的时间复杂度近似为 $O(m+n)$ ，但它的理论时间复杂度还是 $O(mn)$ ，选 C。和 KMP 算法的时间复杂度为 $O(m+n)$ ，选 D。

14. A。

1) 设 $next[1]=0$, $next[2]=1$ 。

编号	1	2	3	4
S	a	a	a	b
Next	0	1		

2) $j=3$ 时 $k=next[j-1]=next[2]=1$ ，观察 $S[j-1](S[2])$ 与 $S[k](S[1])$ 是否相等，

$S[2]=a, S[1]=a, S[2]=S[1]$ ，所以 $next[j]=k+1=2$ 。

$\downarrow j-1=2$

a a a b

a a a b

$\uparrow k=1$

3) $j=4$ 时 $k=next[j-1]=next[3]=2$ ，观察 $S[j-1](S[3])$ 与 $S[k](S[2])$ 是否相等，

$S[3]=a, S[2]=a, S[3]=S[2]$ ，所以 $next[j]=k+1=3$ 。

$\downarrow j-1=3$

a a a b

a a a b

$\uparrow k=1$

最后的结果如下，选 A。

编号	1	2	3	4
S	a	a	a	b
Next	0	1	2	3

本题采用 next 的推理原理求解，如果字符串较长，求解过程会比较烦琐。

(二) 综合应用题

1. 在字符串模式匹配的 KMP 算法中, 求模式的 next 数组值的定义如下:

$$\text{next}[j] = \begin{cases} 0, & j=1 \\ \max\{k | 1 < k < j \text{ 且 } 'p_1 \cdots p_{k-1}' = 'p_{j-k+1} \cdots p_{j-1}'\}, & \text{此集合不空时} \\ 1, & \text{其他情况} \end{cases}$$

- 1) 当 $j=1$ 时, 为什么要取 $\text{next}[1]=0$?
- 2) 为什么要取 $\max\{k\}$, k 最大是多少?
- 3) 其他情况是什么情况, 为什么取 $\text{next}[j]=1$?

2. 设有字符串 $S = \text{'aabaabaabaac'}$, $P = \text{'aabaac1'}$ 。

- 1) 求出 P 的 next 数组。
- 2) 若 S 作主串, P 作模式串, 试给出 KMP 算法的匹配过程。

【答案解析】

1. 解答: 1) 当模式串中的第一个字符与主串的当前字符比较不相等时, $\text{next}[1]=0$, 表示模式串应右移一位, 主串当前指针后移一位, 再和模式串的第一字符进行比较。

2) 当主串的第 i 个字符与模式串的第 j 个字符失配时, 主串 i 不回溯, 则假定模式串的第 k 个字符与主串的第 i 个字符比较, K 值应满足条件 $1 < k < j$ 且 $'p_1 \cdots p_{k-1}' = 'p_{j-k+1} \cdots p_{j-1}'$, 即 k 为模式串的下次比较位置。 K 值可能有多个, 为了不使向右移动丢失可能的匹配, 右移距离应该取最小, 由于 $j-k$ 表示右移的距离, 所以取 $\max\{k\}$ 。

3) 除上面两种情况外, 发生失配时, 主串指针 i 不回溯, 在最坏情况下, 模式串从第 1 个字符开始与主串的第 i 个字符比较。

2. 解答: (1) $P = \text{'aabaac'}$, 按照 next 数组生成算法, 对于 P 有:

① 设 $\text{next}[1] = 0, \text{next}[2] = 1$ 。

编号	1	2	3	4	5	6
S	a	a	b	a	a	c
next	0	1				

② $j=3$ 时 $k = \text{next}[j-1] = \text{next}[2] = 1$, 观察 $S[j-1](S[2])$ 与 $S[k](S[1])$ 是否相等, $S[2]=a, S[1]=a, S[2]=S[1]$, 所以 $\text{next}[j] = k+1 = 2$ 。

↓ $j-1=2$

a a b a a c

a a b a a c

↑ $k=1$

③ $j=4$ 时 $k = \text{next}[j-1] = \text{next}[3] = 2$, 观察 $S[j-1](S[3])$ 与 $S[k](S[2])$ 是否相等, $S[3]=b, S[2]=a, S[3] \neq S[2]$ 。

↓ $j-1=3$

a a b a a c

a a a b a a c

↑ $k=2$

此时 $k = \text{next}[k] = 1$, 观察 $S[3]$ 与 $S[k](S[1])$ 是否相等, $S[3]=b, S[1]=a, S[3] \neq S[1]$ 。 $k = \text{next}[k] = 0$, 因为 $k=0$, 所以 $\text{next}[j] = 1$ 。

↓ $j-1=3$

a a b a a c

a a b a a c

↑ $k=0$

④ $j = 5$ 时 $k = \text{next}[j-1] = \text{next}[4] = 1$, 观察 $S[j-1]$ ($S[4]$) 与 $S[k]$ ($S[1]$) 是否相等, $S[4]=a$, $S[1]=a$, $S[4]=S[1]$, 所以 $\text{next}[j] = k+1 = 2$ 。

$\downarrow j-1=4$

a a b a a c

a a b a a c

$\uparrow k=1$

⑤ $j = 6$ 时 $k = \text{next}[j-1] = \text{next}[5] = 2$, 观察 $S[j-1]$ ($S[5]$) 与 $S[k]$ ($S[2]$) 是否相等, $S[5]=a$, $S[2]=a$, $S[5]=S[2]$, 所以 $\text{next}[j] = k+1 = 3$ 。

$\downarrow j-1=5$

a a b a a c

a a b a a c

$\uparrow k=2$

最后的结果为

编号	1	2	3	4	5	6
S	a	a	b	a	a	c
next	0	1	2	1	2	3

也可以通过求部分匹配值的方法来求 next 数组。

(2) 利用 KMP 算法的匹配过程如下。

第一趟: 从主串和模式串的第一个字符开始比较, 失配时 $i=6$ 、 $j=6$ 。

主串	a	a	b	a	a	b	a	a	b	a	a	c
	a	a	b	a	a	c						

第二趟: $\text{next}[6]=3$, 主串当前位置和模式串的第 3 个字符继续比较, 失配时 $i=9$ 、 $j=6$ 。

主串	a	a	b	a	a	b	a	a	b	a	a	c
				a	a	b	a	a	c			

第三趟: $\text{next}[6]=3$, 主串当前位置和模式串的第 3 个字符继续比较, 匹配成功。

主串	a	a	b	a	a	b	a	a	b	a	a	c
							a	a	b	a	a	c

7.8 查找算法的分析及应用

1. 顺序查找法的平均查找长度为 (), 二分法查找的平均查找长度为 ()。假定 n 为线性表中关键字的个数, 且每次查找都是成功的。

A. $n+1$ B. $2\log_2 n$ C. $\log_2 n$ D. $(n+1)/2$ E. $n\log_2 n$ F. n^2

2. 折半查找和二叉排序树的时间性能()。

A. 相同 B. 有时不相同 C. 完全不同 D. 无法比较

3. 对动态查找有高效率的查找表组织结构是 ()。

A. 有序表 B. 二叉排序树 C. 链表 D. B-树

4. 查找效率最高的二叉排序树为 ()。

A. 所有结点的左子树都为空的二叉排序树 B. 所有结点的右子树都为空的二叉排序树
C. 平衡二叉树 D. 没有左子树的二叉排序树

5. 下列排序树中删除效率最高的是 ()。

A. 平衡二叉树 B. 二叉排序树 C. 没有左子树的二叉排序树 D. 红黑树

6. 查找效率最低的数据结构是 ()。

- A. 有序顺序表 B. 二叉排序树 C. 堆 D. 平衡二叉树

7. 下列关于红黑树和 AVL 树的说法中, 正确的是 ()。

- A. 红黑树查找比 AVL 树快 B. 红黑树插入和删除操作比 AVL 树慢
C. 红黑树的结构比 AVL 树更加平衡
D. 红黑树和 AVL 树的插入、删除操作的时间复杂度都是 $O(\log n)$

8. 下列关于查找的说法中, 错误的是 ()。

I. 在索引顺序表中, 实现分块查找, 在等概率查找情况下, 其平均查找长度不仅与表中元素个数有关, 而且与每块中元素个数有关。

II. 适用于对动态查找表进行高效率查找的组织结构是分块有序表。

III. 对于满足折半查找和分块查找条件的文件而言, 无论它存放在何种介质上, 均能进行顺序查找、折半查找和分块查找。

IV. 折半查找法的查找速度一定比顺序查找法快。

- A. II和III B. III和IV C. II、III和IV D. I、II和IV

【答案解析】

1. D, C。本题考查顺序查找法和折半查找法的平均查找长度。

1) 顺序查找法的平均查找长度在第 1 题中已经讲过, 答案选 D。

2) 根据折半查找法的执行过程可以做出一棵二叉树, 即折半查找判定树, 如果将这个二叉树的最底层结点去掉, 则这棵二叉树是一棵满二叉树, 而查找过程就是走了从根结点到树中某一结点的一条路径, 路径上结点的个数就是关键字的比较次数。对于这棵二叉树, 其高度为 $\lceil \log_2 n \rceil + 1$, 这是表中关键字的最大比较次数, 则折半查找的平均查找长度应该不大于 $\lceil \log_2 n \rceil + 1$, 由此可知答案选 C。

2.B。折半查找的性能分析可以用二叉判定树来衡量, 平均查找长度和最大查找长度都是 $O(\log_2 n)$; 二叉排序树的查找性能与数据的输入顺序有关, 最好情况下的平均查找长度与折半查找相同, 但最坏情况即形成单支树时, 其查找长度为 $O(n)$ 。

3.D。有序表插入、删除元素时间开销大, 链表查找效率低, 二叉排序树极端情况下有可能退化成一条链 (效率低)。B 树是一种平衡的多路查找树, 常用于文件系统中, 查找、插入、删除都很方便。故答案为选项 D。

4.C。本题考查平衡二叉树的查找效率。二叉排序树的查找效率取决于其高度。对于结点个数相同的二叉排序树, 平衡二叉树的高度最小, 因此效率最高。

5.D。平衡二叉树删除结点后要保持平衡的特性, 可能需要多次调整; 二叉排序树若删除的是非叶结点, 需要转换成删除叶结点的情况; 没有左子树的二叉排序树退化成链表结构, 删除元素的时间复杂度为 $O(n)$; 红黑树是在平衡二叉树基础上修改得到的, 较平衡二叉树减少了删除元素时的调整次数。故答案为选项 D。

6.C。选项 A、B 和 C 都数据有序, 是查找结构, 可以根据当前结点的值进行查找, 平均时间复杂度是 $O(\log n)$ (二叉排序树在最极端情况下会达到 $O(n)$) ; 而堆是排序结构, 只有双亲间的大小关系, 查找某元素需要遍历整个堆 (平均时间复杂度为 $O(n)$) , 效率最低。因此, 答案为选项 C。

7.D。红黑树和 AVL 树查找的时间复杂度都是 $O(\log n)$, 但关键字一样时红黑树一般比 AVL 树更高, 所以一般红黑树比 AVL 树查找更慢, 因此选项 A 错误。在插入和删除时红黑树比 AVL 树更快, 因此选项 B 错误。AVL 树对于平衡的要求比红黑树更高, 因此选项 C 错误。红黑树和

AVL 树的插入和删除操作的时间复杂度都是 $O(\log n)$ ，因此选项 D 正确。故答案为选项 D。

8. B。分块查找时，将表均匀分为 b 块，每块 s 个记录，等概率查找情况下平均查找长度 $ASL = (b+1)/2 + (s+1)/2 = (s_2 + 2s + n)/2s$ ，与表中元素个数 n 和每块中元素个数 s 均有关，因此说法 I 正确。分块有序表能对动态查找表进行高效率查找，因此说法 II 正确。若文件存储在磁带上，仅能顺序读取，不能进行折半查找和分块查找，即只能进行顺序查找，因此说法 III 错误。折半查找的时间复杂度低于顺序查找，若查找元素位于序列开始部分（或者说序列总长度 n 很小，如 $n=2$ 时），则顺序查找快于折半查找，因此说法 IV 错误。故答案为选项 B。

8、排序

8.1 排序的基本概念

1. 排序算法的稳定性是指 ()。
 - A. 经过排序后, 能使关键字相同的元素保持原顺序中的相对位置不变
 - B. 经过排序后, 能使关键字相同的元素保持原顺序中的绝对位置不变
 - C. 排序算法的性能与被排序元素个数关系不大
 - D. 排序算法的性能与被排序元素的个数关系密切
2. 下列关于排序的叙述中, 正确的是 ()。
 - A. 稳定的排序方法优于不稳定的排序方法
 - B. 对同一线性表使用不同的排序方法进行排序, 得到的排序结果可能不同
 - C. 排序方法都是在顺序表上实现的, 在链表上无法实现排序方法
 - D. 在顺序表上实现的排序方法在链表上也可以实现

【答案解析】

1. A。注意, 这里的绝对位置是指若在排序前元素 R 在位置 i, 则绝对位置就是 i, 即排序后 R 的位置不发生变化, 显然 B 是不对的。C、D 与题目要求无关。
2. B。算法的稳定性与算法优劣无关, A 排除。使用链表也可以进行排序, 只是有些排序算法不再适用, 因为这时定位元素只能顺序逐链查找, 如折半插入排序。

8.2 插入类排序

(一) 选择题

1. 对 5 个不同的数据元素进行直接插入排序, 最多需要进行的比较次数是 ()。
 - A. 8
 - B. 10
 - C. 15
 - D. 25
2. 对有 n 个元素的顺序表采用直接插入排序算法进行排序, 在最坏情况下所需的比较次数是 (); 在最好情况下所需的比较次数是 ()。
 - A. $n-1$
 - B. $n+1$
 - C. $n/2$
 - D. $n(n-1)/2$
3. 折半插入排序算法的时间复杂度为 ()。
 - A. $O(n)$
 - B. $O(n\log_2 n)$
 - C. $O(n^2)$
 - D. $O(n^3)$
4. 对序列 {98, 36, -9, 0, 47, 23, 1, 8, 10, 7} 采用希尔排序, 下列序列 () 是增量为 4 的一趟排序结果。
 - A. {10, 7, -9, 0, 47, 23, 1, 8, 98, 36}
 - B. {-9, 0, 36, 98, 1, 8, 23, 47, 7, 10}
 - C. {36, 98, -9, 0, 23, 47, 1, 8, 7, 10}
 - D. 以上都不对
5. 直接插入排序在最好的情况下的时间复杂度为 ()。
 - A. $O(n)$
 - B. $O(n\log_2 n)$
 - C. $O(\log_2 n)$
 - D. $O(n^2)$

【答案解析】

1. B。直接插入排序在最坏的情况下要做 $n(n-1)/2$ 次关键字的比较, 当 $n=5$ 时, 关键字的比较次数为 10 (未考虑与哨兵的比较)。
2. D、A。待排序表为反序时, 直接插入排序需要进行 $n(n-1)/2$ 次比较 (从前往后依次需要比

较 $1, 2, \dots, n-1$ 次)；待排序表为正序时，只需进行 $n-1$ 次比较。

3. C。虽然折半插入排序是对直接插入排序的改进，但它改进的只是比较的次数，而移动次数未发生变化，时间复杂度仍为 $O(n^2)$ 。

4. A。增量为 4 意味着所有相距为 4 的记录构成一组，然后在组内进行直接插入排序，经观察，只有选项 A 满足要求。

5. A。本题考查直接插入排序的时间复杂度。当序列已经有序时，直接插入排序的时间复杂度为 $O(n)$ ，因此本题选 A。

(二) 综合应用题

- 给出关键字序列 $\{4, 5, 1, 2, 6, 3\}$ 的直接插入排序过程。
- 给出关键字序列 $\{50, 26, 38, 80, 70, 90, 8, 30, 40, 20\}$ 的希尔排序过程（取增量序列为 $d = \{5, 3, 1\}$ ，排序结果为从小到大排列）。
- 给出关键字序列 $\{4, 5, 1, 2, 8, 6, 7, 3, 10, 9\}$ 的希尔排序过程，对于每一个增量写出一次排序后的序列。
- 已知关键字序列为 $\{53, 87, 12, 61, 90, 10, 97, 25, 53, 46\}$ ，设增量序列为 $\{5, 3, 1\}$ ，请画出希尔排序（递增排序）过程的分析图。

【答案解析】

1. 解答：直接插入排序过程如下。

初始序列： 4, 5, 1, 2, 6, 3

第一趟： 4, 5, 1, 2, 6, 3 (将 5 插入 $\{4\}$)

第二趟： 1, 4, 5, 2, 6, 3 (将 1 插入 $\{4, 5\}$)

第三趟： 1, 2, 4, 5, 6, 3 (将 2 插入 $\{1, 4, 5\}$)

第四趟： 1, 2, 4, 5, 6, 3 (将 6 插入 $\{1, 2, 4, 5\}$)

第五趟： 1, 2, 3, 4, 5, 6 (将 3 插入 $\{1, 2, 4, 5, 6\}$)

2. 解答：原始序列： 50, 26, 38, 80, 70, 90, 8, 30, 40, 20

第一趟（增量 5）: 50, 8, 30, 40, 20, 90, 26, 38, 80, 70

第二趟（增量 3）: 26, 8, 30, 40, 20, 80, 50, 38, 90, 70

第三趟（增量 1）: 8, 20, 26, 30, 38, 40, 50, 70, 80, 90

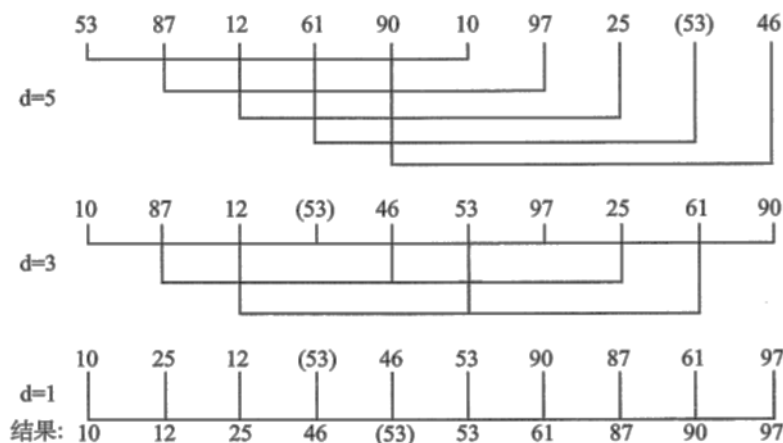
3. 解由待排序列知增量序列为 5, 2, 1。对应于每个增量，排序序列为：

增量 $d=5$: 4, 5, 1, 2, 8, 6, 7, 3, 10, 9

增量 $d=2$: 1, 2, 4, 3, 7, 5, 8, 6, 10, 9

增量 $d=1$: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

4. 解希尔排序过程如下图所示。



8.3 交换类排序

(一) 选择题

- 对 n 个不同的元素利用冒泡法从小到大排序, 在 () 情况下元素交换的次数最多。
 - 从大到小排列好的
 - 从小到大排列好的
 - 元素无序
 - 元素基本有序
- 若用冒泡排序算法对序列 {10, 14, 26, 29, 41, 52} 从大到小排序, 则需进行 () 次比较。
 - 3
 - 10
 - 15
 - 25
- 一组记录的关键码为 (46, 79, 56, 38, 40, 84), 则利用快速排序的方法, 以第一个记录为基准, 从小到大得到的一次划分结果为 ()。
 - (38, 40, 46, 56, 79, 84)
 - (40, 38, 46, 79, 56, 84)
 - (40, 38, 46, 56, 79, 84)
 - (40, 38, 46, 84, 56, 79)
- 快速排序算法在 () 情况下最不利于发挥其长处。
 - 要排序的数据量太大
 - 要排序的数据中含有多个相同值
 - 要排序的数据个数为奇数
 - 要排序的数据已基本有序
- 对数据序列 {8, 9, 10, 4, 5, 6, 20, 1, 2} 采用冒泡排序 (从后向前次序进行, 要求升序), 需要进行的趟数至少是 ()。
 - 3
 - 4
 - 5
 - 8
- 对下列 4 个序列, 以第一个关键字为基准用快速排序算法进行排序, 在第一趟过程中移动记录次数最多的是 ()。
 - 92, 96, 88, 42, 30, 35, 110, 100
 - 92, 96, 100, 110, 42, 35, 30, 88
 - 100, 96, 92, 35, 30, 110, 88, 42
 - 42, 30, 35, 92, 100, 96, 88, 110
- 对 n 个关键字进行快速排序, 最大递归深度为 (), 最小递归深度为 ()。
 - 1
 - n
 - $\log_2 n$
 - $n \log_2 n$
- 下列序列中, () 是执行第一趟快速排序后所得的序列。
 - {68, 11, 18, 69, 23, 93, 73}
 - {68, 11, 69, 23, 18, 93, 73}
 - {93, 73, 68, 11, 69, 23, 18}
 - {68, 11, 69, 23, 18, 73, 93}
- 对关键字序列 {28, 16, 32, 12, 60, 2, 5, 72} 进行递增快速排序, 第一趟划分的结果为 ()。
 - {2, 5, 12, 16} 28 {60, 32, 72}
 - {5, 16, 2, 12} 28 {60, 32, 72}
 - {2, 16, 12, 5} 28 {60, 32, 72}
 - {5, 16, 2, 12} 28 {32, 60, 72}

【答案解析】

1. A。通常情况下,冒泡排序最少进行 1 次冒泡,最多进行 $n-1$ 次冒泡。初始序列为逆序时,进行 $n-1$ 次冒泡,并且需要交换的次数最多。初始序列为正序时,进行 1 次冒泡(无交换)就可以终止算法。

2. C。冒泡排序始终在调整“逆序”,因此交换次数为排列中逆序的个数。对逆序序列进行冒泡排序,每个元素向后调整时都需要进行比较,因此共需要比较 $5+4+3+2+1=15$ 次。

3. C。以 46 为基准元素,首先从后向前扫描比 46 小的元素,并与之进行交换,而后从前向后扫描比 46 大的元素并将 46 与该元素交换,得到(40, 46, 56, 38, 79, 84)。此后,继续重复从后向前扫描与从前往后扫描的操作,直到 46 处于最终位置,答案选 C。

4. D。当待排序数据为基本有序时,每次选取第 n 个元素为基准,会导致划分区间分配不均匀,不利于发挥快速排序算法的优势。相反,当待排序数据分布较为随机时,基准元素能将序列划分为两个长度大致相等的序列,这时才能发挥快速排序的优势。

5. C。从后向前“冒泡”的过程为,第 1 趟{1, 8, 9, 10, 4, 5, 6, 20, 2},第 2 趟{1, 2, 8, 9, 10, 4, 5, 6, 20},第 3 趟{1, 2, 4, 8, 9, 10, 5, 6, 20},第 4 趟{1, 2, 4, 5, 8, 9, 10, 6, 20},第 5 趟{1, 2, 4, 5, 6, 8, 9, 10, 20},经过第 5 趟冒泡后,序列已经全局有序,故选 C。实际每趟冒泡发生交换后可以判断是否会导致新的逆序对,如果不会产生,则本趟冒泡之后序列全局有序,所以最少 5 趟即可。

6. B。对各序列分别执行一趟快速排序,可做如下分析(以 A 为例):由于枢轴值为 92,因此 35 移动到第一个位置,96 移动到第六个位置,30 移动到第二个位置,再将枢轴值移动到 30 所在的单元,即第五个位置,所以 A 中序列移动的次数为 4。同样,可以分析出 B 中序列的移动次数为 8, C 中序列的移动次数为 4, D 中序列的移动次数为 2。

7. B、C。快速排序过程构成一个递归树,递归深度即递归树的高度。枢轴值每次都子表等分时,递归树的高为 $\log_2 n$;枢轴值每次都是子表的最大值或最小值时,递归树退化为单链表,树高为 n 。

8. C、D。本题考查快速排序过程。快速排序第一趟结束后,整个序列会出现以下特点:在序列中一定存在这样一个关键字 a ,比 a 大的关键字与比 a 小的关键字分别出现在 a 的两边。据此来检查各个选项, C 与 D 符合,本题选 C、D。

9. B。本题考查快速排序的过程。根据划分方法进行一趟排序,结果如选项 B 所示。

(二) 综合应用题

1. 在使用非递归方法实现快速排序时,通常要利用一个栈记忆待排序区间的两个端点。能否用队列来实现这个栈?为什么?

2. 有如下快速排序算法,指出该算法是否正确,若不正确,请说明错误的原因。

```
void QuickSort(int R [ ], int low, int high ) {
    int i=low,j=high;
    int temp;
    if(low<high) {
        temp=low;
        while(i!=j) {
            while (j>i&&R[j] >R [temp]) --j;
            if(i<j) {
                R[i]=R.[ j ];
                ++i;
            }
            while (i<j&&R[i] <R[temp]) ++i;
        }
    }
}
```

```

        if ( i < j ) {
            R [ j ] = R [ i ] ;
            --j;
        }
    }
    R [ i ] = R [ temp ] ;
    QuickSort ( R, low, i - 1 ) ;
    QuickSort ( R, i + 1, high ) ;
}
}

```

【答案解析】

1.答。可以用队列来代替栈。在快速排序的过程中,通过一趟划分,可以把一个待排序区间分为两个子区间,然后分别对这两个子区间施行同样的划分。栈的作用是在处理一个子区间时,保存另一个子区间的上界和下界(排序过程中可能产生新的左、右子区间),待该区间处理完后再从栈中取出另一子区间的边界,对其进行处理。这个功能用队列也可以实现,只不过处理子区间的顺序有所变动而已。

2.答:这个算法是错误的。与正确的快速排序算法进行比较发现,本算法将原来 temp 保存划分关键字的值改为保存划分关键字的下标,由于在后边的比较移动过程中可能改变该 temp 下标所指位置上的关键字,造成了 R[i]=R[temp]; 这句赋值错误,从而引起排序失败。

8.4 选择类排序

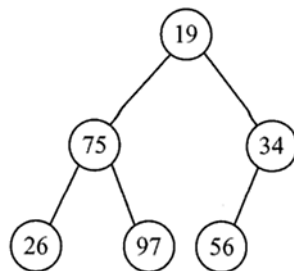
(一) 选择题

- 下列 () 是一个堆。
 - 19, 75, 34, 26, 97, 56
 - 97, 26, 34, 75, 19, 56
 - 19, 56, 26, 97, 34, 75
 - 19, 34, 26, 97, 56, 75
- 有一组数据(15, 9, 7, 8, 20, -1, 7, 4), 用堆排序的筛选方法建立的初始小根堆为 ()。
 - 1, 4, 8, 9, 20, 7, 15, 7
 - 1, 7, 15, 7, 4, 8, 20, 9
 - 1, 4, 7, 8, 20, 15, 7, 9
 - A、B、C 均不对
- 在含有 n 个关键字的小根堆中, 关键字最大的记录有可能存储在 () 位置。
 - n/2
 - n/2 + 2
 - 1
 - n/2 - 1
- 向具有 n 个结点的堆中插入一个新元素的时间复杂度为 (), 删除一个元素的时间复杂度为 ()
 - O(1)
 - O(n)
 - O(log₂n)
 - O(nlog₂n)
- 构建 n 个记录的初始堆, 其时间复杂度为 ()
 - O(n)
 - O(n²)
 - O(log₂n)
 - O(nlog₂n)
- 下列 4 个序列中, () 是堆。
 - {75, 65, 30, 15, 25, 45, 20, 10}
 - {75, 65, 45, 10, 30, 25, 20, 15}
 - {75, 45, 65, 30, 15, 25, 20, 10}
 - {75, 45, 65, 10, 25, 30, 20, 15}
- 以下序列不是堆的是 ()。
 - {100, 85, 98, 77, 80, 60, 82, 40, 20, 10, 66}
 - {100, 98, 85, 82, 80, 77, 66, 60, 40, 20, 10}
 - {10, 20, 40, 60, 66, 77, 80, 82, 85, 98, 100}
 - {100, 85, 40, 77, 80, 60, 66, 98, 82, 10, 20}
- 对 n 个关键字进行堆排序, 最坏情况下的时间复杂度为 ()。
 - O(log₂n)
 - O(n)
 - O(nlog₂n)
 - O(n²)
- 采用简单选择排序, 比较次数与交换次数满足 ()。

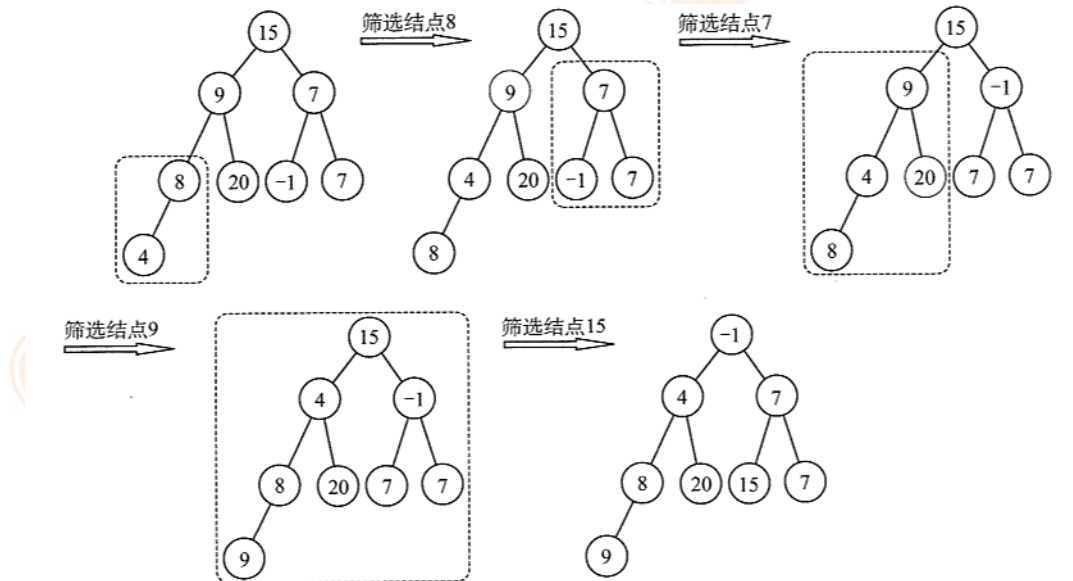
- A. $O(n), O(\log_2 n)$ B. $O(\log_2 n), O(n^2)$ C. $O(n^2), O(n)$ D. $O(n \log_2 n), O(n)$

【答案解析】

1. D. 可将每个选项中的序列表示成完全二叉树, 再看父结点与子结点的关系是否全部满足堆的定义。例如, 选项 A 中序列对应的完全二叉树如右图所示。显然, 最小元素 19 在根结点, 因此可能是小根堆, 但 75 与 26 的关系却不满足小根堆的定义, 所以选项 A 中的序列不是一个堆。其他选项采用类似的过程分析。



2. C. 从 $\lfloor n/2 \rfloor - 1$ 依次筛选堆的过程如下图所示, 显然选 C。

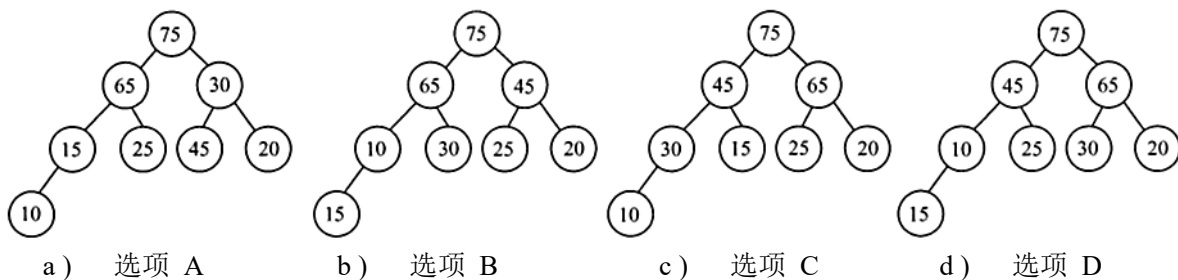


3. B. 这是小根堆, 关键字最大的记录一定存储在这个堆所对应的完全二叉树的叶子结点中; 又因为二叉树中的最后一个非叶子结点存储在 $\lfloor n/2 \rfloor$ 中, 所以关键字最大记录的存储范围为 $\lfloor n/2 \rfloor + 1 \sim n$, 所以应该选 B。

4. C、C. 在向有 n 个元素的堆中插入一个新元素时, 需要调用一个向上调整的算法, 比较次数最多等于树的高度减 1, 由于树的高度为 $\lfloor \log_2 n \rfloor + 1$, 所以堆的向上调整算法的比较次数最多等于 $\lfloor \log_2 n \rfloor$ 。此处需要注意, 调整堆和建初始堆的时间复杂度是不一样的, 读者可以仔细分析两个算法的具体执行过程。

5. A. 建堆过程中, 向下调整的时间与树高 h 有关, 为 $O(h)$ 。

6. C. 本题 考查堆的定义。题目中 4 个选项所表示的 完全二叉树如下图所示, 由图可知选项 C 满足堆的定义。



7. D. 本题考查堆的定义。假设序列存在数组 $a[]$ 中, 序列首关键字存在 $a[1]$ 上, 如果是大顶堆, 则满足 $a[i] > a[2*i]$ 且 $a[i] > a[2*i+1]$; 如果是小顶堆, 则满足 $a[i] < a[2*i]$ 且 $a[i] < a[2*i+1]$ 。根据这两条检查各个选项, 可知只有选项 D 不是堆。

8. C。本题考查堆排序算法的时间复杂度。堆排序算法每次调整堆, 相当于从一棵完全二叉树的根走到叶子结点的过程, 时间复杂度为 $O(\log_2 n)$, 初始建堆需要 n 次调整, 每趟排序需要一次调整, 有 n 趟排序, 因此基本操作的执行次数为 $2n \times O(\log_2 n)$, 即时间复杂度为 $O(n \log_2 n)$ 。初始序列如何, 对排序过程都没有影响, 因此任何情况下堆排序的时间复杂度都是 $O(n \log_2 n)$ 。

9. C。本题考查简单选择排序的时间复杂度。简单选择排序算法, 如果每趟挑出的最小关键字不是在待排序列第一的位置上, 则对每个关键字都需要交换一次, 关键字个数为 n , 交换次数也是 n , 因此交换次数满足 $O(n)$; 每交换一次之前, 查找无序序列中的最小关键字所需的比较次数满足 $O(n)$, 有 n 个关键字, 则总的比较次数满足 $n \times O(n)$, 即 $O(n^2)$ 。

(二) 综合应用题

1. 指出堆和二叉排序树的区别?

2. 若只想得到一个序列中第 $k(k \geq 5)$ 个最小元素之前的部分排序序列, 则最好采用什么排序方法?

3. 有 n 个元素已构成一个小根堆, 现在要增加一个元素 K_{n+1} , 请用文字简要说明如何在 $\log_2 n$ 的时间内将其重新调整为一个堆。

【答案解析】

1. 解答: 以小根堆为例, 堆的特点是双亲结点的关键字必然小于等于该孩子结点的关键字, 而两个孩子结点的关键字没有次序规定。在二叉排序树中, 每个双亲结点的关键字均大于左子树结点的关键字, 均小于右子树结点的关键字, 也就是说, 每个双亲结点的左、右孩子关键字有次序关系。这样, 当对两种树执行中序遍历后, 二叉排序树会得到一个有序的序列, 而堆则不一定能得到一个有序的序列。

2. 解答: 在基于比较的排序方法中, 插入排序、快速排序和归并排序只有在将元素全部排完后, 才能得到前 k 小的元素序列, 算法的效率不高。

冒泡排序、堆排序和简单选择排序可以, 因为它们在每一趟中都可以确定一个最小的元素。采用堆排序最合适, 对于 n 个元素的序列, 建立初始堆的时间不超过 $4n$, 取得第 k 个最小元素之前的排序序列所花的时间为 $k \log_2 n$, 总时间为 $4n + k \log_2 n$; 冒泡和简单选择排序完成此功能所花时间为 kn , 当 $k \geq 5$ 时, 通过比较可以得出堆排序最优。

注意: 从本题可以得出结论, 只需得到前 k 小元素的顺序排列可采用的排序算法有冒泡排序、堆排序和简单选择排序。

3. 解答: 将 K_{n+1} 插入数组的第 $n+1$ 个位置 (即作为一个树叶插入), 然后将其与双亲比较, 若它大于其双亲则停止调整, 否则将 K_{n+1} 与其双亲交换, 重复地将 K_{n+1} 与其新的双亲比较, 算法终止于 K_{n+1} 大于等于其双亲或 K_{n+1} 本身已上升为根。

8.5 分配类排序

(一) 选择题

1. 对 {05, 46, 13, 55, 94, 17, 42} 进行基数排序, 一趟排序的结果是 ()。

A. 05, 46, 13, 55, 94, 17, 42

B. 05, 13, 17, 42, 46, 55, 94

C. 42, 13, 94, 05, 55, 46, 17

D. 05, 13, 46, 55, 17, 42, 94

【答案解析】

1. C。比较各数的个位数，按各数的个位数进行排序，可知选 C。

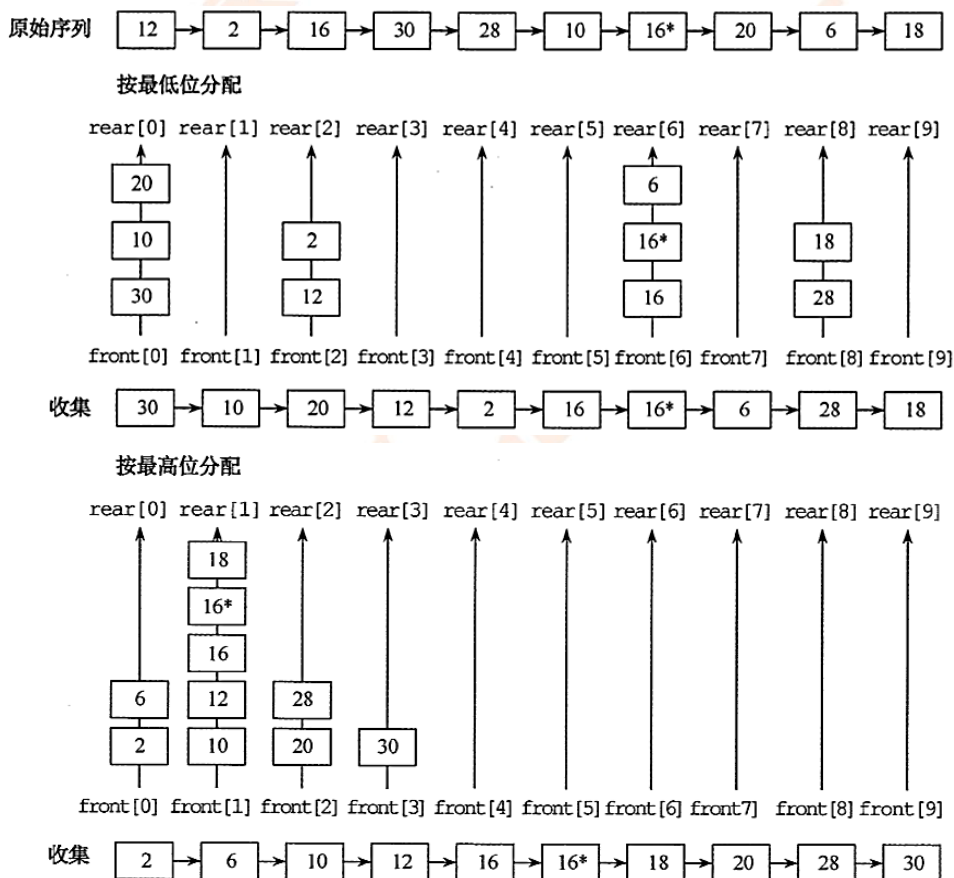
(二) 综合应用题

1. 设待排序的排序码序列为{12,2,16,30,28,10,16*,20,6,18}，试写出使用基数排序方法每趟排序后的结果，并说明做了多少次排序码比较。

2. 给出关键字序列{321, 156, 57, 46, 28, 7, 331, 33, 34, 63}，采用最低位优先算法对其进行基数排序，列出每趟分配和收集的结果。

【答案解析】

1. 解答：使用链式队列的基数排序的排序过程如下图所示。



需要通过 2 次“分配”和“收集”完成排序。

2. 解：将不足 3 位的关键字前边补 0，使各关键字位数相同。

原始序列链：

321 — 156 — 057 — 046 — 028 — 007 — 331 — 033 — 034 — 063

第 1 趟：

分配（按最低位）结果为：

0	1	2	3	4	5	6	7	8	9
	331		063	034		046	007	028	
	321		033			156	057		

收集: 321-331-033-063-034-156-046-057-007-028

第 2 趟:

分配 (按中间位, 依照第 1 趟中结果) 结果为:

0	1	2	3	4	5	6	7	8	9
007		028	034	046	057	063			
		321	033		156				
			331						

收集: 007-321-028-331-033-034-046-156-057-063

第 3 趟:

分配 (按最高位, 依照第 2 趟中结果) 结果为:

0	1	2	3	4	5	6	7	8	9
063	156		331						
057			321						
046									
034									
033									
028									
007									

收集: 007-028-033-034-046-057-063-156-321-331

因此最终排序结果为: 7, 28, 33, 34, 46, 57, 63, 156, 321, 331。

8.6 归并类排序和外部排序

(一) 选择题

1. 归并排序中归并的趟数为()。
 - A. n
 - B. $\lceil \log_2 n \rceil$
 - C. $n \log_2 n$
 - D. n^2
2. 外排序是指()。
 - A. 在外存上进行的排序算法
 - B. 不需要使用内存的排序算法
 - C. 数据量大, 需要人工干预的排序算法
 - D. 排序前后记录在外存, 排序时记录调入内存的排序算法
3. 文件有 m 个初始归并段, 采用 k 路归并时, 所需的归并趟数是()。
 - A. $\log_k k$
 - B. $\log_2 m$
 - C. $\log_k m$
 - D. $\lceil \log_k m \rceil$
4. 归并排序有两个基本阶段, 第一个阶段是()
 - A. 生成初始归并段
 - B. 从归并序列选出最小值 (最大值)
 - C. 将记录读入内存
 - D. 进行多遍归并
5. 若对 27 个元素只进行三趟多路归并排序, 则选取的归并路数最少为()。
 - A. 2
 - B. 3
 - C. 4
 - D. 5
6. 2 路归并排序中, 归并趟数的数量级是()。
 - A. $O(n)$
 - B. $O(\log_2 n)$
 - C. $O(n \log_2 n)$
 - D. $O(n^2)$
7. 将两个各有 N 个元素的有序表合并成一个有序表, 最少的比较次数是(), 最多的比较次数是()。
 - A. N
 - B. $2N-1$
 - C. $2N$
 - D. $N-1$
8. 一组经过第一趟 2 路归并排序后的记录的关键字为 {25, 50, 15, 35, 80, 85, 20, 40, 36, 70}, 其中包

含 5 个长度为 2 的有序表, 用 2 路归并排序方法对该序列进行第二趟归并后的结果为()。

- A. 15, 25, 35, 50, 80, 20, 85, 40, 70, 36 B. 15, 25, 35, 50, 20, 40, 80, 85, 36, 70
C. 15, 25, 50, 35, 80, 85, 20, 36, 40, 70 D. 15, 25, 35, 50, 80, 20, 36, 40, 70, 85

9. 设在磁盘上存放有 375000 个记录, 做 5 路平衡归并排序, 内存工作区能容纳 600 个记录, 为把所有记录排好序, 需要做() 趟归并排序。

- A. 3 B. 4 C. 5 D. 6

10. 最佳归并树在外部排序中的作用是()。

- A. 完成 m 路归并排序 B. 设计 m 路归并排序的优化方案
C. 产生初始归并段 D. 与锦标赛树的作用类似

11. 在下列关于外部排序过程输入/输出缓冲区作用的叙述中, 不正确的是()。

- A. 暂存输入/输出记录 B. 内部归并的工作区
C. 产生初始归并段的工作区 D. 传送用户界面的消息

12. 多路平衡归并的作用是()。

- A. 减少归并趟数 B. 减少初始归并段的个数
C. 便于实现败者树 D. 以上都对

13. 在由 k 路归并构建的败者树中选取一个关键字最小的记录, 则所需时间为(C)。

- A. $O(1)$ B. $O(k)$ C. $O(\log k)$ D. 以上都不对

14. 下列关于小顶堆和败者树的说法中, 正确的是(C)。

- I. 败者树从下往上维护, 每上一层, 只需和失败结点比较 1 次
II. 败者树的每次维护, 必定要从叶结点一直走到根结点, 不可能从中间停止
III. 堆从上往下维护, 每下一层, 若其左右孩子均不为空, 则需比较 2 次
IV. 堆的每次维护, 必定要从根结点一直走到叶结点, 不可能从中间停止

- A. I、III B. II、III C. I、II、III D. I、III、IV

15. 一个无序文件的 n 个记录采用置换-选择排序产生 m 个有序段, 则 m 和 n 的关系是(D)。

- A. m 与 n 成正比 B. $m = \log_2 n$ C. m 与 n 成反比 D. 以上都不对

16. 在由 m 个初始归并段构建的 k 阶最佳归并树中, 不需要补充虚段, 则度为 k 的结点个数是(C)。

- A. $(m-1)/k$ B. m/k C. $(m-1)/(k-1)$ D. 无法确定

【答案解析】

1. B. 本题考查归并排序的时间复杂度分析。由本章归并排序时间复杂度分析可知, 归并排序中排序趟数为 $\lceil \log_2 n \rceil$ 。

2. D. 本题考查外部排序的基本概念。CPU 不能直接操作外存, 因此选项 A、B 不对。选项 C 需要人工干预不对。

3. D. 本题考查归并排序时间复杂度相关问题 (前面知识点讲解中已经涉及)。

4. A. 本题考查归并排序处理过程 (前面知识点讲解中已经涉及)。

5. B. 利用上述公式, 这里要求的是 k, 代入可得 $k=3$ 。

6. B. 对于 N 个元素进行 k 路归并排序时, 排序的趟数 m 满足 $k^m = N$, 所以 $m = \lceil \log_k N \rceil$, 本题中即为 $\lceil \log_2 n \rceil$ 。

7. A、B. 注意到当一个表中的最小元素比另一个表中的最大元素还大时, 比较的次数是最少的, 仅比较 N 次; 而当两个表中的元素依次间隔地比较时, 即 $a_1 < b_1 < a_2 < b_2 < \dots < a_n < b_n$ 时, 比较的次数是最多的, 为 $2N-1$ 次。

归并排序代码比选择插入排序更复杂, 前者的空间复杂度是 $O(n)$, 后者的是 $O(1)$ 。但前者的时间复杂度是 $O(n\log n)$, 后者的是 $O(n^2)$ 。所以选项 B 正确。

8. B。由于这里采用 2 路归并排序算法, 而且是第二趟排序, 所以每 4 个元素放在一起归并, 可将序列划分为 $\{25, 50, 15, 35\}$, $\{80, 85, 20, 40\}$ 和 $\{36, 70\}$, 分别对它们进行排序后有 $\{15, 25, 35, 50\}$, $\{20, 40, 80, 85\}$ 和 $\{36, 70\}$, 故选 B。

9. B。初始归并段的个数 $r = 375000/600 = 625$, 因此, 归并趟数 $S = \lceil \log_m r \rceil = \lceil \log_5 625 \rceil = 4$ 。第一趟把 625 个归并段归并成 $625/5 = 125$ 个; 第二趟把 125 个归并段归并成 $125/5 = 25$ 个; 第三趟把 25 个归并段归并成 $25/5 = 5$ 个; 第四趟把 5 个归并段归并成 $5/5 = 1$ 个。

10. B。最佳归并树在外部排序中的作用是设计 m 路归并排序的优化方案, 仿照构造哈夫曼树的方法, 以初始归并段的长度为权值, 构造具有最小带权路径长度的 m 叉哈夫曼树, 可以有效地减少归并过程中的读写记录数, 加快外部排序的速度。

11. D。在外部排序过程中输入/输出缓冲区就是排序的内存工作区, 例如做 m 路平衡归并需要 m 个输入缓冲区和 1 个输出缓冲区, 用以存放参加归并的和归并完成的记录。在产生初始归并段时也可用作内部排序的工作区。它没有传送用户界面的消息的任务。

12. A。A: 正确。减少趟数意味着减少数据在内存和外存(如磁盘)之间的读写次数(I/O 操作), 这是外部排序性能优化的关键。

B: 错误。初始归并段的个数 m 是由内部排序阶段生成的, 与后续采用几路归并无关。

C: 错误。败者树是为了优化多路归并中“选出最小值”的效率而引入的数据结构(复杂度从 $O(k)$ 降为 $O(\log_2 k)$)。是因为采用了多路归并才需要败者树来辅助, 而不是为了实现败者树。

13. C。败者树是多路平衡归并中用于优化选择最小元素效率的数据结构。每选出一个最小元素并读入新元素后, 重构败者树所需的比较次数等于树的高度量级, 因此, 时间复杂度为 $O(\log k)$ 。

14. C。I: 正确。败者树的节点存储的是“失败者”, 而胜者继续向上比赛。当某个叶子节点的值发生变化(读入新记录)时, 它需要向上调整。在每一层, 它只需要与当前节点存储的“失败者”进行 1 次比较。如果新值胜出, 则新值继续向上, 原失败者留在当前节点; 如果新值失败, 则新值留在当前节点, 原失败者继续向上。

II: 正确。在败者树中, 即使新上来的元素在某一层“输”了, 变成了失败者并留在那一层, 原本在那一层的“旧失败者”, 现在变成了胜者必须继续向上走, 去挑战更高层的节点, 从而改变更高层的胜负关系。调整过程必须一直遍历到根节点, 无法像堆那样在满足堆序性时提前终止。

III: 正确。在小顶堆的向下调整中, 若当前节点有左右两个孩子, 为了维持堆性质, 必须先比较左右孩子, 共需 2 次比较。

IV: 错误。堆的维护不一定走到底, 最佳情况下, 如根节点本身就是最小的, 则比较次数为 0, 平均情况也远小于树高, 这与败者树必须走到底不同。

15. D。A: 错误。在随机分布下, m 与 n 成正比, 但在特殊情况下, 若数据正序, $m=1$ (与 n 无关); 若数据逆序, $m \approx n/M$ (M 是内存缓冲区)。

B: 错误。 $\log_2 n$ 通常与归并的趟数有关。

C: 错误。 n 越大, 归并段个数通常越多。

16. C。 m 个初始归并段对应 m 个叶子结点, k 阶归并树则每个内部结点度为 k , 即合并 k 个段生成 1 个新段, 同时消耗 k 个叶子/子树。因此, 每增加一个内部结点, 当前的可用段总数净减少 $k-1$ 个。

设度为 k 的结点个数为 y 。需要的归并次数(即内部结点数): $y(k-1) = m-1$ 。解得: $y = (m-1)/(k-1)$ 。

(二) 综合应用题

1. 已知序列 {503,87,512,61,908,170,897,275,653,462}, 采用 2 路归并排序法对该序列做升序排序时需要几趟排序? 给出每一趟的结果。

2. 多路平衡归并排序是外部排序的主要方法, 试问多路平衡归并排序包括哪两个相对独立的阶段? 每个阶段完成何种工作?

3. 若某个文件经内部排序得到 80 个初始归并段, 试问:

- 1) 若使用多路平衡归并执行 3 趟完成排序, 则应取得的归并路数至少应为多少?
- 2) 若操作系统要求一个程序同时可用的输入/输出文件的总数不超过 15 个, 则按多路归并至少需要几趟可以完成排序? 若限定趟数, 可取的最低路数是多少?

【答案解析】

1. 解答: $n = 10$, 需要排序的趟数 $= \lceil \log_2 10 \rceil = 4$, 各趟的排序结果如下:
初始序列: 503,87,512,61,908,170,897,275,653,462

第一趟: 87,503,61,512, 170,908,275,897,462,653 (长度为 2)

第二趟: 61,87,503,512, 170,275,897,908,462,653 (长度为 4)

第三趟: 61,87, 170,275,503,512,897,908,462,653 (长度为 8)

第四趟: 61,87, 170,275,462,503,512,653,897,908 (长度为 10)

2. 解答: 多路平衡归并排序由两个相对独立的阶段组成: 生成初始归并段阶段和多趟归并排序阶段。生成初始归并段阶段根据内存工作区的大小, 将有 n 个记录的磁盘文件分批输入内存, 采用有效的内部排序方法分别进行排序, 生成若干有序的子文件, 即初始归并段。多趟归并排序阶段采用多路归并方法将这些归并段逐趟归并, 最后归并成一个有序文件。

3. 解答: 1) 设归并路数为 m , 初始归并段个数 $r = 80$, 根据归并趟数计算公式 $S = \lceil \log_m r \rceil = \lceil \log_m 80 \rceil = 3$, 得 $\log_m 80 \leq 3$, $m^3 \geq 80$ 。由此解得 $m \geq 5$, 即应取的归并路数至少为 5。

2) 设多路归并的归并路数为 m , 需要 m 个输入缓冲区和 1 个输出缓冲区。一个缓冲区对应一个文件, 有 $m + 1 = 15$, 因此 $m = 14$, 可做 14 路归并。由 $S = \lceil \log_m r \rceil = \lceil \log_{14} 80 \rceil = 2$, 即至少需要 2 趟归并可完成排序。若限定趟数为 2, 由 $S = \lceil \log_m 80 \rceil = 2$, 有 $80 \leq m^2$, 可取的最低路数为 9。即要在 2 趟内完成排序, 进行 9 路归并排序即可。

8.7 排序算法的分析与应用

(一) 选择题

1. 下列排序算法中, () 是稳定的。

I. 直接插入排序; II. 快速排序; III. 堆排序; IV. 归并排序

A. I 和 II B. II 和 III C. III 和 IV D. I 和 IV

2. 下列排序算法中, 平均时间复杂度为 $O(n \log_2 n)$ 的有 ()。

I. 起泡排序; II. 快速排序; III. 堆排序; IV. 简单选择排序

A. I 和 II B. II 和 III C. III 和 IV D. IV 和 I

3. 若要求尽可能快地对序列进行稳定的排序, 应该选择 ()。

- A. 快速排序 B. 归并排序 C. 冒泡排序 D. 堆排序
4. 在下面的排序算法中, 辅助空间为 $O(n)$ 的是 ()。
- A. 直接插入排序 B. 堆排序 C. 快速排序 D. 归并排序
5. 对一个初始状态为递增的序列进行按递增顺序的排序, 用 () 最省时, 用 () 最费时。
- A. 直接插入排序 B. 堆排序 C. 快速排序 D. 归并排序
6. 下列排序算法中, () 可能出现这种情况: 在最后一趟开始前, 所有关键字都不在其最终位置上。
- A. 堆排序 B. 冒泡排序 C. 快速排序 D. 折半插入排序
7. 排序趟数与序列的原始状态有关的排序算法是 ()。
- A. 直接插入排序 B. 简单选择排序 C. 冒泡排序 D. 基数排序
8. 下面给出的 4 种排序算法中, 排序过程中的比较次数与初始序列无关的是 ()。
- A. 简单选择排序 B. 直接插入排序 C. 快速排序 D. 冒泡排序
9. 对一组关键字 {84, 47, 25, 15, 21} 排序, 关键字的排列次序在排序过程中的变化为:
- (1) {84, 47, 25, 15, 21} (2) {15, 47, 25, 84, 21}
- (3) {15, 21, 25, 84, 47} (4) {15, 21, 25, 47, 84}
- 则采用的排序是 ()。
- A. 简单选择排序 B. 冒泡排序 C. 快速排序 D. 直接插入排序
10. 关键字序列 {8, 9, 10, 4, 5, 6, 20, 1, 2} 只能是 () 算法中两趟排序后的结果。
- A. 简单选择排序 B. 冒泡排序 C. 直接插入排序 D. 堆排序
11. 对于关键字序列 {15, 9, 7, 8, 20, -1, 4} 进行排序, 进行一趟后关键字序列变为 {9, 15, 7, 8, 20, -1, 4}, 则采用的是 () 算法。
- A. 简单选择排序 B. 冒泡排序 C. 直接插入排序 D. 堆排序
12. 以下排序算法中, () 不能保证每趟排序至少能将一个关键字放在其最终位置上。
- A. 快速排序 B. 希尔排序 C. 堆排序 D. 冒泡排序
13. 在以下排序算法中, 关键字比较的次数与初始排列次序无关的是 ()。
- A. 希尔排序 B. 起泡排序 C. 插入排序 D. 简单选择排序
14. 以下算法中, () 算法在关键字基本有序的时候效率最高。
- A. 快速排序 B. 冒泡排序 C. 堆排序 D. 简单选择排序
15. 下列排序算法中, 辅助空间为 $O(n)$ 的是 ()。
- A. 希尔排序 B. 冒泡排序 C. 堆排序 D. 归并排序
16. 下述排序方法中, 不属于内部排序方法的是 ()。
- A. 插入排序 B. 选择排序 C. 拓扑排序 D. 冒泡排序
17. 在待排序的元素序列基本有序的前提下, 效率最高的排序方法是 ()。
- A. 直接插入排序 B. 简单选择排序 C. 快速排序 D. 归并排序
18. 数据序列 {8, 10, 13, 4, 6, 7, 22, 2, 3} 只能是 () 两趟排序后的结果。
- A. 简单选择排序 B. 起泡排序 C. 直接插入排序 D. 堆排序
19. 希尔排序属于 ()。
- A. 插入排序 B. 交换排序 C. 选择排序 D. 归并排序
20. 对序列 {15, 9, 7, 8, 20, -1, 4}, 经一趟排序后序列变成 {9, 15, 7, 8, 20, -1, 4}, 则采用的是下列的 ()。
- A. 选择排序 B. 快速排序 C. 直接插入排序 D. 冒泡排序
21. 有些排序算法在每趟排序过程中, 都会有一个元素被放置到其最终位置上, () 算法不会出

现此种情况。

- A. 希尔排序 B. 堆排序 C. 冒泡排序 D. 快速排序
22. 以下排序算法中, 不稳定的是()。
- A. 冒泡排序 B. 直接插入排序 C. 希尔排序 D. 归并排序
23. 以下排序算法中, 稳定的是()。
- A. 快速排序 B. 堆排序 C. 直接插入排序 D. 简单选择排序
24. 就平均性能而言, 目前最好的内部排序方法是()。
- A. 冒泡排序 B. 直接插入排序 C. 希尔排序 D. 快速排序
25. 数据序列 $F = \{2, 1, 4, 9, 8, 10, 6, 20\}$ 只能是下列排序算法中的() 两趟排序后的结果。
- A. 快速排序 B. 冒泡排序 C. 选择排序 D. 插入排序
26. 在以下排序算法中, 每次从未排序的记录中选取最小关键字的记录, 加入已排序记录的 末尾, 该排序方法是()。
- A. 简单选择排序 B. 冒泡排序 C. 堆排序 D. 直接插入排序
27. 若只想得到 1000 个元素组成的序列中第 10 个最小元素之前的部分排序的序列, 用() 方法最快。
- A. 冒泡排序 B. 快速排序 C. 希尔排序 D. 堆排序
28. 以下排序方法中, () 在一趟结束后不一定能选出一个元素放在其最终位置上。
- A. 简单选择排序 B. 冒泡排序 C. 归并排序 D. 堆排序
29. 以下排序算法中, () 不需要进行关键字的比较。
- A. 快速排序 B. 归并排序 C. 基数排序 D. 堆排序
30. 在下列排序算法中, 平均情况下空间复杂度为 $O(n)$ 的是(); 最坏情况下空间复杂度为 $O(n)$ 的是()。
- I. 希尔排序; II. 堆排序; III. 冒泡排序
IV. 归并排序; V. 快速排序; VI. 基数排序
- A. I、IV、VI B. II、V C. IV、V D. IV
31. 下列排序方法中, 排序过程中比较次数的数量级与序列初始状态无关的是()。
- A. 归并排序 B. 插入排序 C. 快速排序 D. 冒泡排序
32. 若将中国人按照生日(不考虑年份, 只考虑月、日)未排序, 则使用下列排序算法时, 最快的是()。
- A. 归并排序 B. 希尔排序 C. 快速排序 D. 基数排序
33. 置换-选择排序的作用是()。
- A. 用于生成外部排序的初始归并段
B. 完成将一个磁盘文件排序成有序文件的有效的外部排序算法
C. 生成的初始归并段的长度是内存工作区的 2 倍
D. 对外部排序中输入/归并/输出的并行处理
34. 若要求排序是稳定的, 且关键字为实数, 则在下列排序方法中应选()。
- A. 直接插入排序 B. 选择排序 C. 基数排序 D. 快速排序
35. 以下排序方法中时间复杂度为 $O(n \log_2 n)$ 且稳定的是()。
- A. 堆排序 B. 快速排序 C. 归并排序 D. 直接插入排序
36. 设被排序的结点序列共有 N 个结点, 在该序列中的结点已十分接近有序的情况下, 用直接插入排序、归并排序和快速排序对其进行排序, 这些算法的时间复杂度应为()。

- A. $O(N)$, $O(N)$, $O(N)$ B. $O(N)$, $O(N\log_2 N)$, $O(N\log_2 N)$
 C. $O(N)$, $O(N\log_2 N)$, $O(N^2)$ D. $O(N^2)$, $O(N\log_2 N)$, $O(N^2)$
37. 就排序算法所用的辅助空间而言, 堆排序、快速排序和归并排序的关系是 ()。
- A. 堆排序 < 快速排序 < 归并排序 B. 堆排序 < 归并排序 < 快速排序
 C. 堆排序 > 归并排序 > 快速排序 D. 堆排序 > 快速排序 > 归并排序
38. 排序趟数与序列的原始状态无关的排序方法是 ()。
- I. 直接插入排序; II. 简单选择排序; III. 冒泡排序; IV. 基数排序
- A. I、III B. I、II、IV C. I、II、III D. I、IV
39. 若序列的原始状态为 {1, 2, 3, 4, 5, 10, 6, 7, 8, 9}, 要想使得排序过程中的元素比较次数最少, 则应该采用 () 方法。
- A. 插入排序 B. 选择排序 C. 希尔排序 D. 冒泡排序

【答案解析】

1. D. 本题考查稳定性。还记得“心情不稳定, 快些选一堆好友来聊天吧”这句话吗? “快”“堆”在题目中出现了, 因为不稳定, 予以排除, 故选 D。

2. B. 本题考查时间复杂度。请教官出场, 台词是“快些归队”。“快”“队”(堆)出现, 于是可立即锁定答案为选项 B。

3. B. 本题综合考查时间复杂度和稳定性。首先根据稳定性, 排除选项 A 和选项 D, 然后根据时间, 归并为 $O(n\log_2 n)$, 冒泡为 $O(n^2)$ 。因此本题选 B。

4. D. 本题考查空间复杂度。

前三个通过实现代码很容易看出来是 $O(1)$, 最后一个归并排序, 写出代码如下:

```
void mergesort(int i, int j) {
    int m;
    if(i!=j) {
        m=(i+j)/2;           //(1)
        mergesort(i,m);       //(2)
        mergesort(m+1,j);     //(3)
        merge(i, j, m);       //(4) //本函数的时间复杂度为  $O(n)$ 
    }
}
```

第(4)句中需要一个额外的数组来辅助归并的完成, 长度为 $j-i$ 。第(2)和(3)句虽有内部空间消耗, 但是: 每次递归入口函数返回的时候, 开辟的临时辅助空间都被系统回收; 每次递归入口函数内开辟的辅助空间长度均比最外层的 $j-i$ 要小。因此, 整个归并排序过程中所出现的最大辅助空间开销为 $j-i$, 属于 $O(n)$ 的空间复杂度。

5. A, C. 本题考查适用场景。题意是原始序列已经有序, 那么直接插入排序是最好的, 快速排序达到最坏情况。因此第一个选 A, 第二个选 C。

6. D. 本题考查排序原理。直接插入排序和折半插入排序可能出现这种情况: 例如, 直接插入和折半插入都是在一个已经有序的序列中插入一个新的关键字, 那么最后一趟排序前的序列可能是 2, 3, 4, 5, 1, 和最终有序的序列 1, 2, 3, 4, 5 相比较, 每个关键字都不在其最终的位置上。所以选 D。

7. C. 本题考查排序原理。交换类的排序, 其趟数和原始序列状态有关, 故选 C。顺便分析一下其他选项。选项 A 为直接插入排序, 每趟排序都是插入一个关键字, 所以排序趟数固定为 $n-1$ (n 为关键字数); 选项 B 为简单选择排序, 每趟排序都是选出一个最小(或最大)的关键字, 所以排序趟数固定为 $n-1$ (n 为关键字数); 选项 D 为基数排序, 每趟排序都要进行“分配”和“收集”,

排序趟数固定为 d (d 为关键字位数)。

8. A。本题考查排序原理。比较次数与序列原始状态无关的是简单选择排序, 因为无论序列初始状态如何, 每趟排序选择最小值 (或最大值) 时, 都要顺序扫描序列, 依次用当前最小值和序列中的当前关键字进行比较。所以选 A。

9. A。本题考查排序原理。这类题是已知排序结果求排序算法, 可以采用两种方法: 一种是直接观察序列, 另一种是将序列用每个选项的排序算法执行, 看哪个选项符合。第二种方法较好, 因为有时候直接观察是困难的, 当然若能直接观察出来更好, 这样更快。代入 A 简单选择排序, 按流程执行, 发现是符合的, 所以选 A。另外提一点, 做这种题, 优先考虑代入简单的排序算法, 如直接插入、冒泡、简单选择, 因为这些排序手工执行快, 有符合的立即选出, 节省时间。如果几个简单的都不符合, 用排除法也可以确定是剩下的一个选项。

10. C。本题考查排序过程。已知排序结果求排序算法, 没有告诉初始序列, 只告诉排序结果, 根据排序原理即可解答。还是采用代入选项的方法。选项 A 简单选择排序, 两趟后应该选出两个最小的在最前面, 或者两个最大的在最后面, 观察序列发现不符合。选项 B 冒泡排序, 两趟后应该有最大的两个数沉底, 或者最小的两个数上浮, 观察序列发现不符合。选项 C 直接插入排序, 两趟后插入两个数, 那么前三个数应该是有序的, 观察序列发现符合, 故选 C。选项 D 堆排序, 和简单选择排序一样, 两趟后应该选出两个最大或最小的数, 序列也不符合。

11. C。本题考查排序算法的特点。本题类似于第 13 题, 答案为 C。

12. B。本题考查各种排序算法的特点。

选项 A, 快速排序, 每趟排序后将当前子序列划分为两部分的那个关键字 (“枢轴”) 到达了其最终位置。

选项 C, 堆排序, 每趟排序后, 当前待排序列中的最大或者最小关键字到达其最终位置。

选项 D, 冒泡排序, 每趟排序后, 当前待排序列中的最大或者最小关键字到达其最终位置。

选项 B, 希尔排序是直接插入排序的改进, 每趟排序后不能保证一定有关键字到达最终位置。

因此本题选 B。

13. D。本题考查各种排序算法的特点。A、B、C 三项都是原始序列越接近有序, 比较次数越少。简单选择排序每次从无序序列中挑出一个最小的, 不管原始序列如何, 比较次数皆为无序序列中关键字的个数, 因此比较次数与初始序列无关。

14. B。本题考查排序算法的特点。堆排序和简单选择排序的时间复杂度都与初始序列无关。而快速排序在序列有序的情况下退化起成起泡排序, 即序列越接近有序, 快速排序越慢。

15. D。

16. C 拓扑排序是将有向图中所有结点排成一个线性序列, 虽然也是在内存中进行的, 但它不属于我们这里所提到的内部排序范畴, 也不满足前面排序的定义。

17. A。由于这里的序列基本有序, 使用直接插入排序算法的时间复杂度接近 $O(n)$, 而使用其他算法的时间复杂度均大于 $O(n)$ 。

18. C。选项 A、B、D 的三种排序算法, 在每趟排序后均会使一个记录存放在最终位置上。选项 C 在经过两趟排序后, 原序列中的前 3 个元素会排成局部有序序列。

19. A。希尔排序是对直接插入排序算法改进后提出来的, 本质上仍属于插入排序的范畴。

20. C。前两个元素局部已经有序, 很明显一趟直接插入排序算法有效。再排除其他算法即可。

21. A。由于希尔排序是基于插入排序算法而提出的, 它不一定在每趟排序过程后将某一元素放置到最终位置上。

22. C。希尔排序是一种复杂的插入排序方法, 它是一种不稳定的排序方法。

23. C。基于插入、交换、选择的三类排序方法中,通常简单方法是稳定的(直接插入、折半插入、冒泡),但有一个例外就是简单选择,复杂方法都是不稳定的(希尔、快排、堆排)。

24. D。这里问的是平均性能,A、B 的平均性能都会达到 $O(n^2)$,而希尔排序虽然大大降低了直接插入排序的时间复杂度,但其平均性能不如快速排序。另外,虽然众多排序算法的平均时间复杂度也是 $O(n\log_2 n)$,但快速排序算法的常数因子是最小的。

25. A。若为插入排序,则前三个元素应该是有序的,显然不对。而冒泡排序和选择排序经过两趟排序后应该有两个元素处于最终位置(最左/右端),无论是按从小到大还是从大到小排序,数据序列中都没有两个满足这样的条件的元素,因此只可能选 A。

另解:先写出排好序的序列,并和题中的序列做对比。

题中序列: 2 1 4 9 8 10 6 20

已排好序序列: 1 2 4 6 8 9 10 20

在已排好序的序列中,与题中序列相同元素的有 4、8 和 20,最左和最右两个元素与题中的序列不同,故不可能是冒泡排序、选择排序或插入排序。

26. A。

27. D。希尔排序和快速排序要等排序全部完成之后才能确定最小的 10 个元素。冒泡排序需要从后向前执行 10 趟冒泡才能得到 10 个最小的元素,而堆排序只需调整 10 次小根堆,调整时间与树高成正比。显然堆排序所需的时间更短。

通常,取一大堆数据中的 K 个最大(最小)的元素时,都优先采用堆排序。

28. C。前面我们知道插入排序不能保证在一趟结束后一定有元素放在最终位置上。事实上,归并排序也不能保证。例如,序列 {6, 5, 7, 8, 2, 1, 4, 3} 进行一趟 2 路归并排序(从小到大)后为 {5, 6, 7, 8, 1, 2, 3, 4},显然它们都未被放在最终位置上。

29. C。基数排序是基于关键字各位的大小进行排序的,而不是基于关键字的比较进行的。

30. D、C。归并排序算法在平均情况下和最坏情况下的空间复杂度都会达到 $O(n)$,快速排序只在最坏情况下才会达到 $O(n)$,平均情况下为 $O(\log_2 n)$ 。所以归并排序算法可视为本章所有算法中占用辅助空间最多的排序算法。

31. A。前面已讲过选择排序的比较次数与序列初始状态无关,归并排序也与序列的初始状态无关,读者还应能从算法的原理方面来考虑为什么和初始状态无关。

32. D。按照所有中国人的生日排序,一方面 N 是非常大的,另一方面关键字所含的排序码数为 2,且一个排序码的基数为 12,另一个排序码的基数为 31,都是较小的常数值,因此采用基数排序可以在 $O(N)$ 内完成排序过程。

33. A。置换-选择排序是外部排序中生成初始归并段的方法,用此方法得到的初始归并段的长度是不等长的,其长度平均是传统等长初始归并段的 2 倍,从而使得初始归并段数减少到原来的近二分之一。但是,置换-选择排序不是一种完整的生成有序文件的外部排序算法。

34. A。采用排除法。由于题目要求是稳定排序,排除选项 B 和 D,又由于基数排序不能对 float 和 double 类型的实数进行排序,故排除选项 C。

35. C。堆排序和快速排序不是稳定排序方法,而直接插入排序算法的时间复杂度为 $O(n^2)$ 。

36. C。读者应熟练掌握各种排序算法的时间和空间复杂度、稳定性等,详见表 8.1。

37. A。由于堆排序的空间复杂度为 $O(1)$,快速排序的空间复杂度在最坏情况下为 $O(n)$,平均空间复杂度为 $O(\log_2 n)$,归并排序的空间复杂度为 $O(n)$,所以不难得出正确选项是 A。

38. B。交换类的排序,其趟数和原始序列状态有关,故冒泡排序与初始序列有关。直接插入排序:每趟排序都插入一个元素,所以排序趟数固定为 $n-1$;简单选择排序:每趟排序都选出一个最

小(或最大)的元素,所以排序趟数固定为 $n-1$;基数排序:每趟排序都要进行“分配”和“收集”排序趟数固定为 d 。

39. A。选择排序和序列初态无关,直接排除。初始序列基本有序时,插入排序比较次数较少。本题中,插入排序仅需比较 $n+4$ 次,而希尔排序和冒泡排序的比较次数均远大于此。

(二) 综合应用题

1. 设关键字序列为{3, 7, 6, 9, 7, 1, 4, 5, 20}, 对其进行排序的最小交换次数是多少?
2. 在堆排序、快速排序和归并排序中:
 - (1) 若只从存储空间考虑,则应首选哪种排序算法,其次选取哪种排序算法,最后选取哪种排序算法?
 - (2) 若只从排序结果的稳定性考虑,应选取哪种排序算法?
 - (3) 若只从平均 情况下 排序最快考虑,应选取哪种排序算法?
 - (4) 若只从最坏情况下排序最快并且要节省内存考虑,应选取哪种排序算法?
3. 给出一组关键字: {29, 18, 25, 47, 58, 12, 51, 10}, 分别写出按下列各种排序算法进行排序时的变化过程。
 - (1) 归并排序,每归并一次书写一个次序。
 - (2) 快速排序,每划分一次书写一个次序。
 - (3) 堆排序,先建成一个堆(写出一个序列),然后每从堆顶取下一个关键字后,将堆调整一次(每次都写出一个序列),直到排序完成。

【答案解析】

1. 答案: 由于关键字序列数较小,采用直接插入排序或简单选择排序,直接插入排序的交换次数更多,选择简单选择排序。

初始序列: 3, 7, 6, 9, 7, 1, 4, 5, 20

第一次: 1, 7, 6, 9, 7, 3, 4, 5, 20 交换 1, 3

第二次: 1, 3, 6, 9, 7, 7, 4, 5, 20 交换 3, 7

第三次: 1, 3, 4, 9, 7, 7, 6, 5, 20 交换 4, 6

第四次: 1, 3, 4, 5, 7, 7, 6, 9, 20 交换 5, 9

第五次: 1, 3, 4, 5, 6, 7, 7, 9, 20 交换 6, 7

所以最小交换次数为 5 (注意这里是交换次数,不是移动次数或比较次数)。

2. 解: (1) 若只从存储空间考虑,则应首选堆排序 ($O(1)$),其次选取快速排序 ($O(\log_2 n)$),最后选取归并排序 ($O(n)$)。

(2) 若只从排序结果的稳定性来考虑,则应该选取归并排序。

(3) 若只从平均情况下排序最快考虑,则应选取快速排序。

说明: 从平均情况来看,在时间复杂度同为 $O(n \log_2 n)$ 的所有算法中,快速排序的基本操作执行次数是最少的,虽然数量级是一样的,因此它是最快的。

(4) 若只从最坏情况下排序最快并且节省内存来考虑,则应选取堆排序。

3. 解: (1) 二路归并。

第一趟: 18, 29, 25, 47, 12, 58, 10, 51

第二趟: 18, 25, 29, 47, 10, 12, 51, 58

第三趟: 10, 12, 18, 25, 29, 47, 51, 58

(2) 快速排序。

第一趟: 10, 18, 25, 12, 29, 58, 51, 47

第二趟: 10, 18, 25, 12, 29, 47, 51, 58

第三趟: 10, 12, 18, 25, 29, 47, 51, 58

(3) 堆排序。

建大堆: 58, 47, 51, 29, 18, 12, 25, 10

排序:

① 51, 47, 25, 29, 18, 12, 10, 58

② 47, 29, 25, 10, 18, 12, 51, 58

③ 29, 18, 25, 10, 12, 47, 51, 58

④ 25, 18, 12, 10, 29, 47, 51, 58

⑤ 18, 10, 12, 25, 29, 47, 51, 58

⑥ 12, 10, 18, 25, 29, 47, 51, 58

⑦ 10, 12, 18, 25, 29, 47, 51, 58

9、多维数组和矩阵的压缩存储

9.1 多维数组

- 对于数组的操作, 最常见的两种是 ()。
A. 建立与删除 B. 索引和修改 C. 查找和修改 D. 查找与索引
- 数组 A 中, 每个元素的长度为 3B, 行下标 i 从 0~7, 列下标 j 从 0~9, 从首地址开始连续存放在存储器内, 则存放该数组至少需要的单元数是 ()。
A. 80 B. 100 C. 240 D. 270
- 在一个二维数组 A 中, 假设每个数组元素的长度为 3 个存储单元, 行下标 i 为 0~8, 列下标 j 为 0~9, 从首地址 SA 开始连续存放。在这种情况下, 元素 A[8][5] 的起始地址为 ()。
A. SA+141 B. SA+144 C. SA+222 D. SA+255

【答案解析】

- C。数组最适合且最常用的操作是利用其随机存取特性进行的读取(查找)和更新(修改)。
- C。行数=7-0+1=8 行, 列数=9-0+1=10 列, 每个元素的长度为 3B, 总字节数=80×3=240 B。
- D。二维数组计算地址(按行优先顺序)的公式为:

$$LOC(i, j) = LOC(O, 0) + (ixm + j) \times L$$

其中, $LOC(O, 0) = SA$, 是数组存放的首地址; $L = 3$ 是每个数组元素的长度; $m = 9 - 0 + 1 = 10$ 是数组的列数。因此有 $LOC(8, 5) = SA + (8 \times 10 + 5) \times 3 = SA + 255$, 故选 D。

9.2 矩阵的压缩存储

- 对特殊矩阵采用压缩存储的主要目的是 ()。
A. 表达变得简单 B. 对矩阵元素的存取变得简单
C. 去掉矩阵中的多余元素 D. 减少不必要的存储空间
- 对 n 阶对称矩阵压缩存储时, 需要表长为 () 的顺序表。
A. n/2 B. n×n/2 C. n(n+1)/2 D. n(n-1)/2
- 有一个 n×n 的对称矩阵 A, 将其下三角部分按行存放在一维数组 B 中, 而 A[0][0] 存放于 B[0] 中, 则第 i+1 行的对角元素 A[i][i] 存放于 B 中的 () 处。
A. (i+3)i/2 B. (i+1)i/2 C. (2n-i+1)i/2 D. (2n-i-1)i/2
- 将三对角矩阵 A[1..100][1..100] 按行优先存入一维数组 B[1..298] 中, A 中元素 A[66][65] 在数组 B 中的位置 k 为 ()。
A. 198 B. 195 C. 197 D. 196
- 稀疏矩阵一般的压缩存储方法有 () 两种。
A. 二维数组和三维数组 B. 三元组和散列
C. 三元组和十字链表 D. 散列和十字链表
- 设矩阵 A 是一个对称矩阵, 为了节省存储空间, 将其下三角部分按照行优先存放在一维数组 B[0, ..., n(n+1)/2-1] 中, 对于下三角部分中的任一元素 a_{ij} ($i \geq j$, i 和 j 从 1 开始取值), 在一维数组 B 中的下标 k 的值是 ()。
A. i(i-1)/2+j-1 B. i(i+1)/2+j C. i(i+1)/2+j-1 D. i(i-1)/2+j

【答案解析】

1. D。特殊矩阵中含有很多相同元素或零元素，故可采用压缩存储，以节省存储空间。
2. C。只需要存储其上三角或下三角部分（含对角线），元素个数为 $n+(n-1)+(n-2)+\dots+1=n(n+1)/2$ 。

3. A。此题要注意 3 个细节：矩阵的最小下标为 0；数组下标也是从 0 开始的；矩阵按行优先存在数组中。注意到此三点，答案不难得到为 A。此外，本类题建议采用特殊值代入法求解，例如， $A[1][1]$ 对应的下标应为 2，代入后只有 A 满足条件。

技巧：对于特殊三角矩阵压缩存储的题，心中应有“平移”搬动的思想，并结合草图，这样会比较形象，在计算时再注意矩阵和数组的起始下标，就不容易出错。

4. B。对于三对角矩阵，将 $A[1..n][1..n]$ 压缩至 $B[1..3n-2]$ 时， a_{ij} 与 b_k 的对应关系为 $k=2i+j-2$ 。则 A 中的元素 $A[66][65]$ 在数组 B 中的位置 k 为 $2\times 66+65-2=195$ 。

5. C。稀疏矩阵指非零元素远少于零元素的矩阵，为节省空间常采用压缩存储。

三元组顺序表（顺序存储）：将非零元素表示为（行，列，值）的三元组，按顺序存储。适合静态存储，结构简单。

十字链表（链式存储）：每个非零元素结点包含行、列、值及两个指针（指向同行下一个、同列下一个）。适合动态操作（如插入、删除）及复杂矩阵运算。

6. A。矩阵的行号 i 和列号 j 从 1 开始，一维数组 B 的下标 k 从 0 开始。

1) 计算前 i-1 行的元素总数，由于只存储下三角部分：

第 1 行有 1 个元素 a_{11} ，第 2 行有 2 个元素 a_{21}, a_{22} ，第 i-1 行有 i-1 个元素，则前 i-1 行的元素总和为等差数列求和：

$$\text{sum} = 1 + 2 + \dots + (i-1) = (i-1) \times i / 2$$

2) 计算第 i 行内的偏移量，在第 i 行中，存储的元素是 a_{ij} 。 a_{ij} 是第 i 行的第 j 个元素。

3) 计算数组下标 k。数组下标 $k = (\text{前 } i-1 \text{ 行的总元素个数}) + (\text{当前元素在第 } i \text{ 行的序号}-1)$ ，因为数组 B 下标从 0 开始，第 j 个元素对应的偏移量是 j-1。

$$k = \frac{i \times (i-1)}{2} + j - 1$$

第二部分：进阶提高篇

1、绪论

(一) 选择题

- 下述 () 与数据的存储结构无关。
A. 栈 B. 双向链表 C. 散列表 D. 线索树 E. 循环队列
- 以下数据结构中, () 是非线性数据结构。
A. 树 B. 队 C. 栈
- 以下属于逻辑结构的是 ()。
A. 顺序表 B. 散列表 C. 有序表 D. 单链表
- 以下数据结构中, () 是非线性数据结构。
A. 树 B. 字符串 C. 队 列 D. 栈
- 以下属于逻辑结构的是 ()。
A. 顺序表 B. 哈希表 C. 有序表 D. 单链表
- 以下与数据的存储结构无关的术语是 ()。
A. 循环队列 B. 链表 C. 哈希表 D. 栈
- 链式存储设计时, 结点内的存储单元地址 ()。
A. 一定连续 B. 一定不连续 C. 不一定连续 D. 部分连续, 部分不连续

【答案解析】

1. A。选项 A, 栈是逻辑结构。选项 B, 双向链表也说明线性表是以链式结构存储的。选项 C, 散列是算法, 散列存储方法本质上是顺序存储方法的扩展。散列表本质上是顺序表的扩展。选项 D, 线索树是在链式存储结构的基础上对树进行线索, 与链式存储结构有关。选项 E, 循环队列是建立在顺序存储结构上的。

说明: 这种题目还有一种比较直观的解法, 要判断是否与数据的存储结构无关, 只需要看这种结构到底有没有具体到使用顺序存储还是链式存储, 如果已经具体到了那就一定和数据的存储结构有关。例如, A 选项中的栈并没有说明是用顺序栈还是用链栈来实现, 所以是逻辑结构。D 选项中的线索树很明显是要用链式存储来实现, 故与数据的存储结构有关, 以此类推。

- A。树是一种分支结构, 显然不属于线性结构。
- C。有序表指出了表中数据是根据一定逻辑顺序排列的, 是一种逻辑结构。
- A。树和图是典型的非线性数据结构, 其他选项都属于线性数据结构。
- C。顺序表、哈希表和单链表是三种不同的数据结构, 既描述逻辑结构, 又描述存储结构和数据运算。而有序表是指关键字有序的线性表, 仅描述元素之间的逻辑关系, 它既可以链式存储, 又可以顺序存储, 故属于逻辑结构。
- D。数据的存储结构有顺序存储、链式存储、索引存储和散列存储。循环队列 (易错点) 是用顺序表表示的队列, 是一种数据结构。栈是一种抽象数据类型, 可采用顺序存储或链式存储, 只表示逻辑结构。
- A。链式存储设计时, 各个不同结点的存储空间可以不连续, 但结点内的存储单元地址必须

连续。

(二) 综合应用题

1. 有下列运行时间函数:

$$(1) f_1(n)=1000; \quad (2) f_2(n)=n^2+1000n; \quad (3) f_3(n)=3n^3+100n^2+n+1;$$

分别写出相应的以 O 表示的运算时间。

2. 一个算法所需时间由下述递归方程表示, 试求出该算法的时间复杂度的级别 (或阶)。

$$T(n)=\begin{cases} 1 & n=1 \\ 2T(n/2)+n, & n>1 \end{cases}$$

式中, n 是问题的规模, 为简单起见, 设 n 是 2 的整数次幂。

3. 分析以下各程序段, 求出算法的时间复杂度。

① $i=1; k=0;$

```
while(i<n-1){
    k=k+10*i;
    i++;
}
```

② $y=0;$

```
while((y+1)*(y+1)<=n)
    y=y+1;
```

③ $\text{for}(i=1; i \leq n; i++)$

```
     $\text{for}(j=1; j \leq i; j++)$ 
         $\text{for}(k=1; k \leq j; k++)$ 
             $x++;$ 
```

④ $\text{for}(i=0; i < n; i++)$

```
     $\text{for}(j=0; j < m; j++)$ 
         $a[i][j]=0;$ 
```

【答案解析】

1. (1) $T_1(n)=O(1000/1000)=O(1)$ 。(2) $T_2(n)=O(n^2/1)=O(n^2)$ 。(3) $T_3(n)=O(3n^3/3)=O(n^3)$ 。

2. 时间复杂度为 $O(n \log_2 n)$ 。

设 $n=2^k$ ($k \geq 0$), 根据题目所给定义有 $T(2^k)=2T(2^{k-1})+2^k=2^2T(2^{k-2})+2 \times 2^k$, 由此可得一般递推公式 $T(2^k)=2iT(2^{k-i})+i \times 2^k$, 进而得 $T(2^k)=2^kT(2^0)+k \times 2^k=(k+1)2^k$, 即 $T(n)=2^{\log_2 n+1} \log_2 n \times n=n(\log_2 n+1)$, 也就是 $O(n \log_2 n)$ 。

3. 解答:

① 基本语句 $k=k+10*i$ 共执行了 $n-2$ 次, 所以 $T(n)=O(n)$ 。

② 设循环体共执行 t 次, 每循环一次, 循环变量 y 加 1, 最终 $t=y$ 。故 $t^2 \leq n$, 得 $T(n)=O(n^{1/2})$ 。

③ 基本语句 $x++$ 的执行次数为

$$T(n)=O\left(\sum_{i=1}^n \sum_{j=1}^i \sum_{k=1}^j 1\right)=O\left(\frac{1}{6}n^3\right)=O(n^3)$$

④ 内循环执行 m 次, 外循环执行 n 次, 根据乘法原理, 共执行了 $m \times n$ 次, 故 $T(m, n)=O(m \times n)$ 。

2、线性表

2.1 综合练习

(一) 选择题

- 在 n 个元素的线性表的数组表示中, 时间复杂度为 $O(1)$ 的操作是 ()。
 - 访问第 i ($1 \leq i \leq n$) 个结点和求第 i ($2 \leq i \leq n$) 个结点的直接前驱
 - 在最后一个结点后插入一个新的结点
 - 删除第 1 个结点
 - 在第 i ($1 \leq i \leq n$) 个结点后插入一个结点

A. I B. II、III C. I、II D. I、II、III
- 设线性表有 n 个元素, 严格说来, 以下操作中, () 在顺序表上实现要比在链表上实现的效率高。
 - 输出第 i ($1 \leq i \leq n$) 个元素值
 - 交换第 3 个元素与第 4 个元素的值
 - 顺序输出这 n 个元素的值

A. I B. I、III C. I、II D. II、III
- 对于顺序表, 访问第 i 个位置的元素和在第 i 个位置插入一个元素的时间复杂度为 ()。

A. $O(n), O(n)$ B. $O(n), O(1)$ C. $O(1), O(n)$ D. $O(1), O(1)$
- 对于顺序存储的线性表, 其算法时间复杂度为 $O(1)$ 的运算应该是 ()。
 - 将 n 个元素从小到大排序
 - 删除第 i ($1 \leq i \leq n$) 个元素
 - 改变第 i ($1 \leq i \leq n$) 个元素的值
 - 在第 i ($1 \leq i \leq n$) 个元素后插入一个新元素
- 单链表 L (带头结点) 为空的判断条件是 ()。

A. $L == \text{NULL}$ B. $L \rightarrow \text{next} == \text{NULL}$ C. $L \rightarrow \text{next} != \text{NULL}$ D. $L != \text{NULL}$
- 在一个单链表中, 已知 q 所指结点是 p 所指结点的前驱结点, 若在 q 和 p 之间插入结点 s , 则执行 ()。
 - $s \rightarrow \text{next} = p \rightarrow \text{next}; p \rightarrow \text{next} = s;$
 - $p \rightarrow \text{next} = s \rightarrow \text{next}; s \rightarrow \text{next} = p;$
 - $q \rightarrow \text{next} = s; s \rightarrow \text{next} = p;$
 - $p \rightarrow \text{next} = s; s \rightarrow \text{next} = q;$
- 给定有 n 个元素的一维数组, 建立一个有序单链表的最低时间复杂度是 ()。

A. $O(1)$ B. $O(n)$ C. $O(n^2)$ D. $O(n \log_2 n)$
- 将长度为 n 的单链表链接在长度为 m 的单链表后面, 其算法的时间复杂度采用大 O 形式表示应该是 ()。

A. $O(1)$ B. $O(n)$ C. $O(m)$ D. $O(n + m)$
- 对于一个头指针为 head 的带头结点的单链表, 判定该表为空表的条件是 (); 对于不带头结点的单链表, 判定空表的条件为 ()。
 - $\text{head} == \text{NULL}$
 - $\text{head} \rightarrow \text{next} == \text{NULL}$
 - $\text{head} \rightarrow \text{next} == \text{head}$
 - $\text{head} != \text{NULL}$
- 在长度为 n 的有序单链表中插入一个新结点, 并仍然保持有序的时间复杂度是 ()。

A. $O(1)$ B. $O(n)$ C. $O(n^2)$ D. $O(n \log_2 n)$
- 带头结点的双循环链表 L 为空的条件是 ()。
 - $L \rightarrow \text{prior} == L \&\& L \rightarrow \text{next} == \text{NULL}$
 - $L \rightarrow \text{prior} == \text{NULL} \&\& L \rightarrow \text{next} == \text{NULL}$
 - $L \rightarrow \text{prior} == \text{NULL} \&\& L \rightarrow \text{next} == L$
 - $L \rightarrow \text{prior} == L \&\& L \rightarrow \text{next} == L$

12. 某线性表用带头结点的循环单链表存储, 头指针为 head, 当 head->next->next= head 成立时, 线性表长度可能是 ()。
- A. 0 B. 1 C. 2 D. 可能为 0 或 1
13. 在一个双链表中, 在 p 结点之后插入结点 q 的操作是 ()。
- A. q->prior=p; p->next=q; p->next->prior=q; q->next=p->next;
B. q->next=p->next; p->next->prior=q; p->next=q; q->prior=p;
C. p->next=q; q->prior=p; q->next=p->next; p->next->prior=q;
D. q->prior=p; p->next=q; q->next=p->next; p->next->prior=q;
14. 在一个双链表中, 在 p 结点之前插入 q 结点的操作是 ()。
- A. p->prior=q; q->next=p; p->prior->next=q; q->prior=p->prior;
B. q->prior=p->prior; p->prior->next=q; q->next=p; p->prior=q->next;
C. q->next=p; p->next=q; q->prior->next=q; q->next=p;
D. p->prior->next=q; q->next=p; q->prior=p->prior; p->prior=q;
15. 在一个双链表中, 删除 p 结点的操作是 () (结点空间释放语句省略)。
- A. p->prior->next=p->next; p->next->prior=p->prior;
B. p->prior=p->prior->prior; p->prior->prior=p;
C. p->next->prior=p; p->next=p->next->next;
D. p->next=p->prior->prior; p->prior=p->prior->prior;
16. 非空的单循环链表 L (带头结点) 的终端结点 (由 p 所指向) 满足 ()。
- A. p->next==NULL B. p==L
C. p->next==L D. p->next==L&&p!=L
17. 带头结点的双循环链表 L 为空的条件是 ()。
- A. L==NULL B. L->next->prior==NULL
C. L->prior==NULL D. L->prior==L&&L->next==L
18. 对于一个线性表, 既要求能够快速地进行插入和删除, 又要求存储结构能够反映数据元素之间的逻辑关系, 则应采用 () 存储结构。
- A. 顺序 B. 链式 C. 散列 (Hash 表)
19. 在顺序表中插入一个元素的时间复杂度为 ()。
- A. $O(1)$ B. $O(\log_2 n)$ C. $O(n)$ D. $O(n^2)$
20. 在顺序表中删除一个元素的时间复杂度为 ()。
- A. $O(1)$ B. $O(\log_2 n)$ C. $O(n)$ D. $O(n^2)$
21. 对于一个具有 n 个元素的线性表, 建立其单链表的时间复杂度为 ()。
- A. $O(1)$ B. $O(\log_2 n)$ C. $O(n)$ D. $O(n^2)$
22. 设有两个长度为 n 的单链表 (带头结点), 结点类型相同, 若以 h1 头结点指针的链表是非循环的, 以 h2 为头结点指针的链表是循环的, 则 ()。
- A. 对于两个链表来说, 删除开始结点的操作, 其时间复杂度分别为 $O(1)$ 和 $O(n)$
B. 对于两个链表来说, 删除终端结点的操作, 其时间复杂度都是 $O(n)$
C. 循环链表要比非循环链表占用更多的内存空间
D. h1 和 h2 是不同类型的变量
23. 若设一个顺序表的长度为 n。那么, 在表中顺序查找一个值为 x 的元素时, 在等概率的情况下, 查找成功的数据平均比较次数为 ()。
- A. n B. n/2 C. (n+1)/2 D. (n-1)/2
24. 若设一个顺序表的长度为 n。在向表中第 i ($1 \leq i \leq n+1$) 个元素位置插入一个新元素时, 为保持

插入后表中原有元素的相对次序不变, 需要从后向前依次后移 () 个元素。

- A. $n-i$ B. $n-i+1$ C. $n-i-1$ D. i

25. 若设一个顺序表的长度为 n 。那么, 在删除表中第 i ($1 \leq i \leq n$) 个元素时, 为保持删除后表中原有元素的相对次序不变, 需要从前向后依次前移 (③) 个元素。

- A. $n-i$ B. $n-i+1$ C. $n-i-1$ D. i

【答案解析】

1.C. I 解析略; II 中, 在最后位置插入新结点不需要移动元素, 时间复杂度为 $O(1)$; III 中, 被删结点后的结点需依次前移, 时间复杂度为 $O(n)$; IV 中, 需要后移 $n-i$ 个结点, 时间复杂度为 $O(n)$ 。

2.C. 对于 II, 顺序表仅需 3 次交换操作; 链表则需要分别找到两个结点前驱, 第 4 个结点断链后再插入到第 2 个结点后, 效率较低。对于 III, 需依次顺序访问每个元素, 时间复杂度相同。

3.C. 在第 i 个位置插入一个元素, 需要移动 $n-i+1$ 个元素, 时间复杂度为 $O(n)$ 。

4..C. 对 n 个元素进行排序的时间复杂度最小也要 $O(n)$ (初始有序时), 通常为 $O(n \log_2 n)$ 或 $O(n^2)$, 通过第 7 章学习后会更理解。B 和 D 显然错误。顺序表支持按序号的随机存取 (读写) 方式。

5.B. 本题考查带头结点的单链表和不带头结点的单链表基本操作的区别。带头结点的单链表的判空条件是看 $head \rightarrow next$ 是否为 NULL, 不带头结点的单链表是看头指针 $head$ 是否为 NULL。

6.C. s 插入后, q 成为 s 的前驱, 而 p 成为 s 的后继, 选项 C 符合。

7.D. 若先建立链表, 然后依次插入建立有序表, 则每插入一个元素就需遍历链表寻找插入位置, 即直接插入排序, 时间复杂度为 $O(n^2)$ 。若先将数组排好序, 然后建立链表, 建立链表的时间复杂度为 $O(n)$, 数组排序的最好时间复杂度为 $O(n \log_2 n)$, 总时间复杂度为 $O(n \log_2 n)$ 。故选 D。

8.C. 先遍历长度为 m 的单链表, 找到该单链表的尾结点, 然后将其 $next$ 域指向另一个单链表的首结点, 其时间复杂度为 $O(m)$ 。

9.B, A. 在带头结点的单链表中, 头指针 $head$ 指向头结点, 头结点的 $next$ 域指向第一个元素结点, $head \rightarrow next = NULL$ 表示该单链表为空。在不带头结点的单链表中, $head$ 直接指向第一个元素结点, $head = NULL$ 表示该单链表为空。

10.B. 设单链表递增有序, 首先要在单链表中找到第一个大于 x 的结点的直接前驱 p , 在 p 之后插入该结点。查找的时间复杂度为 $O(n)$, 插入的时间复杂度为 $O(1)$, 总时间复杂度为 $O(n)$ 。

11.D. 循环双链表 L 判空的条件是头结点 (头指针) 的 $prior$ 和 $next$ 域都指向它自身。

12.D. 对一个空循环单链表, 有 $head \rightarrow next = head$, 推理 $head \rightarrow next \rightarrow next = head \rightarrow next = head$ 。对含有 1 个元素的循环单链表, 头结点 (头指针 $head$ 指示) 的 $next$ 域指向该唯一元素结点, 该元素结点的 $next$ 域指向头结点, 因此也有 $head \rightarrow next \rightarrow next = head$ 。故选 D。

13.B. A 选项执行完第二句时, p 的后继结点地址丢失, 插入不能完成。C 选项执行完第一句时, p 的后继结点地址丢失, 插入不能完成。D 选项执行完第二句时, p 的后继结点地址丢失, 插入不能完成。

14.D. A 选项执行完第一句时, p 的前驱结点地址丢失, 插入不能完成。B 选项最后一句 $p \rightarrow prior = q \rightarrow next$; 应该改为 $p \rightarrow prior = q$ 。C 选项破坏了 p 和其后继结点的联系。

15. A. B 选项第一句执行完之后, p 的后继结点和前边的所有结点都失去联系。C 选项第一句为废操作, 第二句执行完, p 的后继结点地址丢失。D 选项将 p 结点的 $next$ 指针指向了其前驱的前驱, 不满足删除要求。

16.D. 带头结点的单循环链表中没有空指针域, 因此 A 选项错误。当终端结点与头结点为同一个结点, 即满足 $p = L$ 时, 链表为空, 因此 B 选项错误。当链表为空时, p 指向头结点, 同时也是

终端结点, 此时一定满足 $p \rightarrow next = L$ 。当链表不空时, P 指向终端结点, 同样满足 $p \rightarrow next = L$, 因此 C 选项恒成立, 不能作为判断链表非空的条件。因此本题答案为 D 。

17.D. 本题考查双循环链表的判空条件。当双循环链表为空时, 头结点体现如下图所示的形态。可见当满足 $L \rightarrow prior = L \& \& L \rightarrow next = L$ 时, 链表为空, 并且循环双链表同循环单链表一样, 没有空指针域, 因此本题选 D 。

18.B. 本题考查链表的特点。链表较之本题选项中其他两种存储结构的最大优点就是能够快速地进行插入和删除操作。当然, 链表也能反映数据元素之间的逻辑关系, $Hash$ 表中各元素之间不存在逻辑关系, 因此本题选 B 。

19.C. 本题考查顺序表插入算法的效率。顺序表插入算法的基本操作是元素的移动。在一个长度为 n 的顺序表中的位置 i 上插入一个元素, 需要进行元素移动的次数大约为 $n-i$ 次, 插入算法元素平均移动次数大约为 $n/2$ 次, 因此时间复杂度为 $O(n)$ 。

20.C. 本题考查顺序表删除算法的效率。顺序表删除算法的基本操作是元素的移动。在一个长度为 n 的顺序表中的位置 i 上删除一个元素, 需要进行元素移动的次数大约为 $n-i$ 次, 删除算法平均元素移动次数大约为 $n/2$ 次, 因此时间复杂度为 $O(n)$ 。

21.C. 本题考查链表的建立。链表建立的过程即将结点逐个插入链表的过程, 因此链表建立的时间复杂度即为链表规模 n 乘以链表插入操作的时间复杂度 $O(1)$, 即 $n \times O(1)$, 即 $O(n)$ 。因此本题选 C 。

22.B. 本题考查链表的综合知识。对于两个链表来说, 要删除开始结点, 其时间复杂度都为 $O(1)$ 。因为已知开始结点的前驱结点地址, 即头结点地址。

对于两个链表来说, 要删除终端结点, 都需要从头结点开始找到终端结点的前驱结点, 其时间复杂度都是 $O(n)$ 。

两个链表占用内存空间相同。不要误以为循环链表比非循环链表多一个指向头结点的指针, 其实非循环链表也有这个指针, 只是它没有指向头结点而已。

$h1$ 和 $h2$ 指向相同类型的结点, 因此是相同类型的变量。单链表和循环单链表 结点类型相同, 只是表中结点的组织方式不同。

23.C. 本题考查线性表中与查找、插入、删除操作效率相关的问题。

$$ASL = \frac{1}{n} \sum_{i=1}^n i = \frac{1}{n} (1 + 2 + \dots + n) = \frac{1}{n} \frac{(1+n)n}{2} = \frac{n+1}{2}$$

24.B. 在有 n 个元素的顺序表中的第 i 个元素位置插入一个新元素时, 需把表中从第 n 个元素到第 i 个元素全部后移一个元素位置, 以空出第 i 个元素位置供新元素插入, 需要移动的元素有 $n-i+1$ 个, 前面的 $i-1$ 个元素没有移动。

25.A. 想要在有 n 个元素的顺序表中删除第 i 个元素, 需把表中从第 $i+1$ 个元素到第 n 个元素的所有元素前移, 以填补原来第 i 个元素, 需要移动 $n-(i+1)+1=n-i$ 个元素。

(二) 综合应用题

1. 线性表 $(a_1, a_2, a_3, \dots, a_n)$ 中的元素递增有序且按顺序存储于计算机内。要求设计一个算法, 完成用最少时间在表中查找数值为 x 的元素, 若找到, 则将其与后继元素位置相交换, 若找不到, 则将其插入表中并使表中元素仍递增有序。

2. 有一个带头结点的单链表 L , 设计一个算法使其元素递增有序。

3. 设头指针为 L 的带有表头结点的非循环双向链表, 其每个结点中除有 $pred$ (前驱指针)、 $data$ (数据) 和 $next$ (后继指针) 域外, 还有一个访问频度域 $freq$ 。在链表被启用前, 其值均初始化为零。每当在链表中进行一次 $Locate(L, x)$ 运算时, 令元素值为 x 的结点中 $freq$ 域的值增 1, 并使得此链表中结点保持按访问频度非增 (递减) 的顺序排列, 同时最近访问的结点排在频度相同的结点前面, 以便使频繁访问的结点总是靠近表头。试编写符合上述要求的 $Locate(L, x)$ 运算的算法, 该运算为函数过程, 返回找到结点的地址, 类型为指针型。

【答案解析】

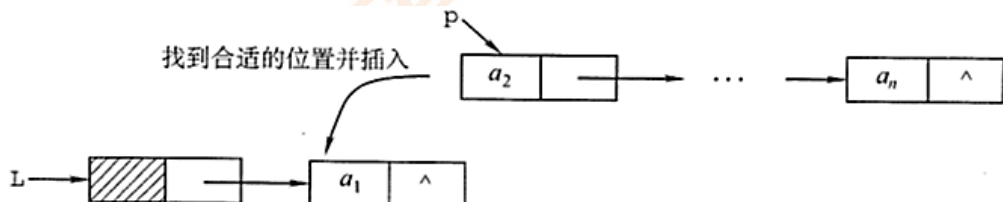
1. 算法思想: 顺序存储的线性表递增有序, 可以顺序查找, 也可以折半查找。题目要求“最少的时间在表中查找数值为 x 的元素”, 这里应使用折半查找法。

本题代码如下:

```
void SearchExchangeInsert(ElemType A[], ElemType x) {
    int low = 0, high = n - 1, mid; // low 和 high 指向顺序表下界和上界的下标
    while (low <= high) {
        mid = (low + high) / 2; // 找中间位置
        if (A[mid] == x) break; // 找到 x, 退出 while 循环
        else if (A[mid] < x) low = mid + 1; // 到中点 mid 的右半部去查
        else high = mid - 1; // 到中点 mid 的左半部去查
    } // 下面两个 if 语句只会执行一个
    if (A[mid] == x && mid != n - 1) { // 若最后一个元素与 x 相等, 则不存在与其后继交换的操作
        t = A[mid];
        A[mid] = A[mid + 1];
        A[mid + 1] = t;
    }
    if (low > high) { // 查找失败, 插入数据元素 x
        for (i = n - 1; i > high; i--) A[i + 1] = A[i]; // 后移元素
        A[i + 1] = x; // 插入 x
    } // 结束插入
}
```

本题的算法也可写成三个函数: 查找函数、交换后继函数与插入函数。写成三个函数的优点是逻辑清晰、易读。

2. 算法思想: 采用直接插入排序算法的思想, 先构成只含一个数据结点的有序单链表, 然后依次扫描单链表中剩下的结点 $*p$ (直至 $p = \text{NULL}$ 为止), 在有序表中通过比较查找插入 $*p$ 的前驱结点 $*pre$, 然后将 $*p$ 插入到 $*pre$ 之后, 如下图所示。



本题代码如下:

```
void Sort(LinkList &L) { // 本算法实现将单链表 L 的结点重排, 使其递增有序
    LNode *p = L->next, *pre;
    LNode *r = p->next; // r 保持 *p 后继结点指针, 以保证不断链
    p->next = NULL; // 构造只含一个数据结点的有序表
    p = r;
```

```
while (p != NULL) {
    r = p->next;           // 保存 *p 的后继结点指针
    pre = L;
    while (pre->next != NULL && pre->next->data < p->data)
        pre = pre->next;   // 在有序表中查找插入 *p 的前驱结点 *pre
    p->next = pre->next;     // 将 *p 插入到 *pre 之后
    pre->next = p;
    p = r;                 // 扫描原单链表中剩下的结点
}
}
```

细心的读者会发现该算法的时间复杂度为 $O(n^2)$ ，为达到最佳的时间性能，可先将链表的数据复制到数组中，再采用时间复杂度为 $O(n\log_2 n)$ 的排序算法进行排序，然后将数组元素依次插入到链表中，此时的时间复杂度为 $O(n\log_2 n)$ ，显然这是以空间换时间的策略。

3. 算法思想：首先在双向链表中查找数据值为 x 的结点，查到后，将结点从链表上摘下，然后顺着结点的前驱链查找该结点的插入位置（频度递减，且排在同频度的第一个，即向前找到第一个比它的频度大的结点，插入位置为该结点之后），并插入到该位置。

本题代码如下：

```
DLinkedList Locate(DLinkedList &L, ElemType x) { // 先查找数据 x，查找成功时结点的访问频度域增 1
    // 最后将该结点按频度递减插入链表中适当位置（同频度最近访问的在前面）
    DNode *p = L->next, *q;           // p 为工作指针，q 为 p 的前驱，用于查找插入位置
    while (p && p->data != x)
        p = p->next;                 // 查找值为 x 的结点
    if (!p) {
        printf("不存在值为 %d 的结点\n", x);
        exit(0);
    }
    else {
        p->freq++;                    // 令元素值为 x 的结点的 freq 域加 1
        if (p->next != NULL) p->next->pred = p->pred;
        p->pred->next = p->next;       // 将 p 结点从链表上摘下
        q = p->pred;                  // 以下查找 p 结点的插入位置
        while (q != L && q->freq <= p->freq)
            q = q->pred;
        p->next = q->next;
        q->next->pred = p;
        p->pred = q;
        q->next = p;                  // 将 p 结点插入，一定是排在同频率的第一个
    }
    return p;                         // 返回值为 x 的结点的指针
}
```

2.2 线性表的基本操作算法专题

2.2.1 线性表的拆分与合并

1. 将两个有序顺序表合并为一个新的有序顺序表，并由函数返回结果顺序表。
2. 将一个带头结点的单链表 A 分解为两个带头结点的单链表 A 和 B，使得 A 表中含有原表中序号为奇数的元素，而 B 表中含有原表中序号为偶数的元素，且保持其相对顺序不变。

3. 设 $C = \{a_1, b_1, a_2, b_2, \dots, a_m, b_m\}$ 为线性表, 采用带头结点的 hc 单链表存放, 设计一个就地算法, 将其拆分为两个线性表, 使得 $A = \{a_1, a_2, \dots, a_n\}$, $B = \{b_n, \dots, b_2, b_1\}$ 。

4. 假设有两个按元素值递增次序排列的线性表, 均以单链表形式存储。请编写算法将这两个单链表归并为一个按元素值递减次序排列的单链表, 并要求利用原来两个单链表的结点存放归并后的单链表。

5. 有两个循环单链表, 链表头指针分别为 h1 和 h2, 编写一个函数将链表 h2 链接到链表 h1 之后, 要求链接后的链表仍保持循环链表形式。

【答案解析】

1. 算法思想: 首先, 按顺序不断取下两个顺序表表头较小的结点存入新的顺序表中。然后, 哪个表还有剩余, 将剩下的部分加到新的顺序表后面。

本题代码如下:

```
bool Merge (SeqList A, SeqList B, SeqList &C) { //将有序顺序表 A 与 B 合并为一个新的有序顺序表 C
    if (A.length + B.length > C.maxSize) //大于顺序表的最大长度
        return false;
    int i=0, j=0, k=0;
    while(i<A.length && j<B.length) { //循环, 两两比较, 小者存入结果表
        if(A.data[i] <= B.data[j])
            C.data[k++] = A.data[i++];
        else
            C.data[k++] = B.data[j++];
    }
    while(i<A.length) //还剩一个没有比较完的顺序表
        C.data[k++] = A.data[i++];
    while(j<B.length)
        C.data[k++] = B.data[j++];
    C.length = k;
    return true;
}
```

注意: 本算法的方法非常典型, 需牢固掌握。

2. 解答: 算法思想: 设置一个访问序号变量 (初值为 0), 每访问一个结点序号自动加 1, 然后根据序号的奇偶性将结点插入到 A 表或 B 表中。重复以上操作直到表尾。

本题代码如下:

```
LinkedList DisCreat_1(LinkedList &A) { // 将表 A 中结点按序号的奇偶性分解到表 A 或表 B 中
    int i = 0; // i 记录表 A 中结点的序号
    LinkedList B = (LinkedList)malloc(sizeof(LNode)); // 创建 B 表表头
    B->next = NULL; // B 表的初始化
    LNode *ra = A, *rb = B; // ra 和 rb 将分别指向将创建的 A 表和 B 表的尾结点
    LNode *p = A->next; // p 为链表工作指针, 指向待分解的结点
    A->next = NULL; // 置空新的 A 表
    while (p != NULL) {
        i++; // 序号加 1
        if (i % 2 == 0) { // 处理序号为偶数的链表结点
            rb->next = p; // 在 B 表尾插入新结点
            rb = p;
        }
        p = p->next;
    }
}
```



```

    }
    else {
        ra->next = p;
        ra = p;
    }
    p = p->next;
}
ra->next = NULL; // 断开 A 表的最后一个结点的 next 指针
rb->next = NULL; // 断开 B 表的最后一个结点的 next 指针
return B; // 返回 B 表的头结点
}

```

为了保持原来结点中的顺序, 本题采用尾插法建立单链表。此外, 本算法完全可以不用设置序号变量。while 循环中的代码改为将结点插入到表 A 中并将下一结点插入到表 B 中, 这样 while 中第一处理的结点就是奇数号结点, 第二处理的结点就是偶数号结点。

3. 解答: 算法思想: 采用上题的思路, 不设序号变量。二者的差别仅在于对 B 表的建立不采用尾插法, 而是采用头插法。

本题代码如下:

```

LinkedList DisCreat_2(LinkedList &A) {
    LinkedList B = (LinkedList)malloc(sizeof(LNode)); // 创建 B 表表头
    B->next = NULL; // B 表的初始化
    LNode *p = A->next,*q; // p 为工作指针
    LNode *ra = A; // ra 始终指向 A 的尾结点
    while (p != NULL) {
        ra->next = p; // 将 *p 链到 A 的表尾
        ra = p;
        p = p->next;
        if (p != NULL) {
            q = p->next; // 头插后, *p 将断链, 因此用 q 记忆 *p 的后继
            p->next = B->next; // 将 *p 插入到 B 的前端
            B->next = p;
            p = q;
        }
    }
    ra->next = NULL; // A 尾结点的 next 域置空
    return B;
}

```

该算法特别需要注意的是, 采用头插法插入结点后, *p 的指针域已改变, 若不设变量保存其后继结点, 则会引起断链, 从而导致算法出错。

4. 算法思想: 两个链表已经按元素值递增次序排序, 将其合并时, 均从第一个结点起进行比较, 将小的结点链入链表中, 同时后移工作指针。该问题要求结果链表按元素值递减次序排列, 故新链表的建立应该采用头插法。比较结束后, 可能会有一个链表非空, 此时用头插法将剩下的结点依次插入新链表中即可。

本题代码如下:

```

void MergeList(LinkedList &La, LinkedList &Lb) {
    // 合并两个递增有序链表 (带头结点), 并使合并后的链表递减排列
    LNode *r, *pa = La->next, *pb = Lb->next; // 分别是表 La 和 Lb 的工作指针
    La->next = NULL; // La 作为结果链表的头指针, 先将结果链表初始化为空
}

```

```

while (pa && pb) {
    if (pa->data <= pb->data) {
        r = pa->next;           // r 暂存 pa 的后继结点指针
        pa->next = La->next;     // 将 pa 结点链于结果表中, 同时逆置 (头插法)
        La->next = pa;
        pa = r;                 // 恢复 pa 为当前待比较结点
    }
    else {
        r = pb->next;           // r 暂存 pb 的后继结点指针
        pb->next = La->next;     // 将 pb 结点链于结果表中, 同时逆置 (头插法)
        La->next = pb;
        pb = r;                 // 恢复 pb 为当前待比较结点
    }
}
if (pa)
    pb = pa;                    // 通常情况下会剩一个链表非空, 处理剩下的部分
while (pb) {
    r = pb->next;               // 处理剩下的一个非空链表
    pb->next = La->next;
    La->next = pb;              // 依次插入到 La 中 (头插法)
    pb = r;
}
free(Lb);
}

```

5. 算法思想: 先找到两个链表的尾指针, 将第一个链表的尾指针与第二个链表的头结点链接起来, 再使之成为循环的。

本题代码如下:

```

LinkedList Link(LinkedList &h1, LinkedList &h2) {
    // 将循环链表 h2 链接到循环链表 h1 之后, 使之仍保持循环链表的形式
    LNode *p, *q;
    p = h1; // 分别指向两个链表的尾结点
    while (p->next != h1) // 寻找 h1 的尾结点
        p = p->next;
    q = h2;
    while (q->next != h2) // 寻找 h2 的尾结点
        q = q->next;
    p->next = h2; // 将 h2 链接到 h1 之后
    q->next = h1; // 令 h2 的尾结点指向 h1
    return h1;
}

```

2.2.2 线性表子集、交集

1. 设 A 和 B 是两个单链表 (带头结点), 其中元素递增有序。设计一个算法从 A 和 B 中的公共元素产生单链表 C, 要求不破坏 A、B 的结点。

2. 已知递增有序的单链表 A, B (A, B 中元素个数分别为 m, n 且 A, B 都带有头结点) 分别存储了一个集合, 请设计算法以求出两个集合 A 和 B 的差集 A-B (即仅由在 A 中出现而不在 B 中出现的元素所构成的集合)。将差集保存在单链表 A 中, 并保持元素的递增有序性。

(1) 给出算法的基本设计思想。

- (2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。
- (3) 说明你所设计算法的时间复杂度。

3. 设有 n 个不全为负的整型元素存储在一维数组 $A[n]$ 中, 它包含很多连续的子数组, 例如数组 $A=\{1, -2, 3, 10, -4, 7, 2, -5\}$, 请设计一个时间上尽可能高效的算法, 求出数组 A 的子数组之和的最大值 (例如数组 A 的最大的子数组为 $\{3, 10, -4, 7, 2\}$, 因此输出为该子数组的和 18)。

要求:

- (1) 给出算法的基本设计思想。
- (2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。
- (3) 说明你所设计算法的时间复杂度和空间复杂度。

【答案解析】

1. 算法思想: 表 A 、 B 都有序, 可从第一个元素起依次比较 A 、 B 两表的元素, 若元素值不等, 则值小的指针往后移, 若元素值相等, 则创建一个值等于两结点的元素值的新结点, 使用尾插法插入到新的链表中, 并将两个原表指针后移一位, 直到其中一个链表遍历到表尾。

本题代码如下:

```
void Get_Common(LinkList A, LinkList B) {    // 采用单链表 A 和 B 的公共元素的单链表 C
    LNode *p = A->next, *q = B->next, *r, *s;
    LinkList C = (LinkList)malloc(sizeof(LNode));    // 建立表 C
    r = C;    // r 始终指向 C 的尾结点
    while (p != NULL && q != NULL) {    // 循环跳出条件
        if (p->data < q->data)
            p = p->next;    // 若 A 的当前元素较小, 后移指针
        else if (p->data > q->data)
            q = q->next;    // 若 B 的当前元素较小, 后移指针
        else {
            s = (LNode*)malloc(sizeof(LNode));    // 找到公共元素结点
            s->data = p->data;    // 复制产生结点 *s
            r->next = s;    // 将 *s 链接到 C 上 (尾插法)
            r = s;
            p = p->next;    // 表 A 和 B 继续向后扫描
            q = q->next;
        }
    }
    r->next = NULL;    // 置 C 尾结点指针为空
}
```

2. 算法思想: 采用归并的思想, 设置两个工作指针 pa 和 pb , 对两个链表进行归并扫描, 同时出现在两集合中的元素才链接到结果表中且仅保留一个, 其他的结点全部释放。当一个链表遍历完毕后, 释放另一个表中剩下的全部结点。

本题代码如下:

```
LinkList Union(LinkList &la, LinkList &lb) {
    pa = la->next;    // 设工作指针分别为 pa 和 pb
    pb = lb->next;
    pc = la;    // 结果表中当前合并结点的前驱指针
    while (pa && pb) {
        if (pa->data == pb->data) {    // 交集并入结果表中
```

```

    pc->next = pa;                // A 中结点链接到结果表
    pc = pa;
    pa = pa->next;
    u = pb; // B 中结点释放
    pb = pb->next;
    free(u);
}
else if (pa->data < pb->data) {    // 若 A 中当前结点值小于 B 中当前结点值
    u = pa;
    pa = pa->next;
    free(u);                      // 释放 A 中当前结点
}
else {
    u = pb;
    pb = pb->next;
    free(u);                      // 释放 B 中当前结点
} // while 结束
while (pa) {
    u = pa;
    pa = pa->next;
    free(u);                      // 释放 A 中剩余结点
}
while (pb) {
    u = pb;
    pb = pb->next;
    free(u);                      // 释放 B 中剩余结点
}
pc->next = NULL;                 // 置结果链表尾指针为 NULL
free(lb);                       // 释放 B 表的头结点
return la;
}

```

链表归并类型的试题在各学校历年真题中出现的频率很高, 故应扎实掌握解决此类问题的思想。该算法的时间复杂度为 $O(\text{len1} + \text{len2})$, 空间复杂度为 $O(1)$ 。

3. 采用动态规划法。若记 $b[j] = \max \sum_{k=i}^j a[k]$, $1 \leq i \leq j \leq n$, 则所求最大子段和为: $\max \sum_{k=i}^j a[k] = \max \sum_i^j a[k] = \max[j]$ 。由 $b[j]$ 的定义可知, 当 $b[j-1] > 0$ 时, $b[j] = b[j-1] + a[j]$, 否则 $b[j] = a[j]$ 。由此可计算机 $b[j]$ 的动态规划递归式: $b[j] = \max\{b[j-1] + a[j], a[j]\}$, $1 \leq j \leq n$, 据此, 可设计出求最大子段和的算法如下:

```

int MaxSum(int n, int *a) {
    int sum = 0, b = 0;
    for (int i = 1; i <= n; i++) {
        if (b > 0)
            b += a[i];
        else
            b = a[i];
        if (b > sum)
            sum = b;
    }
    return sum;
}

```

算法的时间复杂度为 $O(n)$, 空间复杂度为 $O(1)$ 。

【另解 1】 (1) 算法的基本思想:

采用分治法。数组(A[0], A[1], ..., A[n-1])分为长度相等的两段数组(A[0], ..., A[n/2]) 以及 (A[n/2+1], ..., A[n-1]), 分别求出这两段数组各自的最大子段和, 则原数组(A[0], A[1], ..., A[n-1])的最大子段和分为以下 3 种情况:

- 1) (A[0], A[1], ..., A[n-1])的最大子段与(A[0], ..., A[n/2]) 的最大子段相同。
- 2) (A[0], A[1], ..., A[n-1])的最大子段与(A[n/2+1], ..., A[n-1])的最大子段相同。
- 3) (A[0], A[1], ..., A[n-1])的最大子段跨过(A[0], ..., A[n/2])与(A[n/2+1], ..., A[n-1])。如果数组元素全部为负, 结果返回 0。

①设置 left, right。初始化为原数组的开始和结束位置。设置 mid=(left+right)/2, 指向数组的中间位置。

②如果 left==right, 返回元素值和 0 的较大者。

③计算 A[left...mid]中包含 A[mid]的最大连续数组及其值 Lmax。计算 A[mid+1...right]中包含 A[mid+1]的最大连续数组及其值 Rmax。求出跨过中间元素时的最大子段及其最大值 Lmax+Rmax。递归求出 A[left...mid]中的最大连续子数组及其最大值, 与 A[mid+1...right]的最大连续子数组及其最大值。返回三者之中的最大值。

(2) 算法的实现如下:

```
int MaxSum(int *A,int left,int right){
    if(left==right)                //递归退出条件, 只有一个元素
        return max(A[left],0);    //返回元素值与 0 较大者。
    int mid=(left+right)/2;        //mid 是数组的中间位置, 分治开始
    int Lmax=0;                    //求(A[left],...,A[mid])中包含 A[mid]子数组的最大值
    int Lsum=0;                    //Lmax 是左边最大和, Lsum 是累加和
    for(int i=mid;i>=left;i--){
        Lsum+=A[i];                //从 A[mid]往左累加
        if(Lsum>Lmax)              //比较
            Lmax = Lsum;
    }
    int Rmax=0;                    //求(A[mid+1],...,A[right])中包含 A[mid+1]子数组的最大值
    int Rsum=0;                    //Rmax 是右边最大和, Rsum 是累加和
    for(int i=mid+1;i<=right;i++){
        Rsum+=A[i];                //从 A[mid+1]往右累加
        if(Rsum>Rmax)              //比较
            Rmax = Rsum;
    }
    //递归求 1)2)情况下的连续子数组最大和。并返回 1)2)3)种情况下的最大值。
    //Lmax+Rmax 为第三种情况下连续子数组最大和。
    //MaxSum(A,left,mid)递归求 A[left...mid]的连续子数组最大和。
    //MaxSum(A,mid+1,right)递归求 A[mid+1,right]连续子数组最大和。
    return max(Lmax+Rmax,max(MaxSum(A,left,mid),MaxSum(A,mid+1,right)));
}
```

(3) 时间复杂度的计算公式为 $T(n)=2*T(n/2)+n$, 因此时间复杂度为 $O(n*\log_2 n)$ 。递归树的高度为 $\log_2 n$, 每层空间辅助变量为常数, 因此空间复杂度为 $O(\log_2 n)$ 。

【另解 2】使用暴力破解。假设最大的一段数组为 A[i], ..., A[j], 则对 $i=0\sim n-1$ $j=i\sim n-1$, 遍历一遍, 求出最大的 Sum(i,j)即可。长度为 n 的数组有 $O(n^2)$ 个子数组, 求一个长 n 的数组和的时间复杂度为 $O(n)$, 因此时间复杂度为 $O(n^3)$, 因此性能较差, 空间复杂度为 $O(1)$ 。

2.2.3 线性表的删除

1. 从顺序表中删除其值在给定值 s 与 t 之间 (包含 s 和 t , 要求 $s < t$) 的所有元素, 若 s 或 t 不合理或顺序表为空, 则显示出错信息并退出运行。

2. 给定一个带头结点的单链表, 设 $head$ 为头指针, 结点结构为 $(data, next)$, $data$ 为整型元素, $next$ 为指针, 试写出算法: 按递增次序输出单链表中各结点的数据元素, 并释放结点所占的存储空间 (要求: 不允许使用数组作为辅助空间)。

3. 设有一个带头结点的循环单链表, 其结点值均为正整数。设计一个算法, 反复找出单链表中结点值最小的结点并输出, 然后将该结点从中删除, 直到单链表空为止, 再删除表头结点。

4. 设一个长度为 $n(n>1)$ 的单链表 L , 从第一个结点开始计数, 当计数到 $m(m>1)$ 时, 将这第 m 个结点从单链表上摘除, 然后从被摘除的下一个结点开始重新计数; 当计数到表尾时, 接着表的第一个结点继续计数。试设计一个在时间和空间两方面都尽可能高效的算法, 完成上述过程, 要求:

- (1) 给出算法的基本设计思想。
- (2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。
- (3) 说明你所设计算法的时间复杂度和空间复杂度。

【答案解析】

1. 算法思想: 从前向后扫描顺序表 L , 用 K 记录下元素值在 s 到 t 之间元素的个数 (初始时 $k=0$)。对于当前扫描的元素, 若其值不在 s 到 t 之间, 则前移 K 个位置; 否则执行 $k++$ 。由于这样每个不在 s 到 t 之间的元素仅移动一次, 因此算法效率高。

本题代码如下:

```
bool Del_s_t(SqList &L, ElemType s, ElemType t) {
    // 删除顺序表 L 中值在给定值 s 与 t (要求 s < t) 之间的所有元素
    int i, k = 0;
    if (L.length == 0 || s >= t)
        return false; // 线性表为空或 s、t 不合法, 返回 false
    for (i = 0; i < L.length; i++) {
        if (L.data[i] >= s && L.data[i] <= t)
            k++; // 当前元素前移 k 个位置
        else
            L.data[i - k] = L.data[i]; // 当前元素前移 k 个位置
    }
    L.length -= k; // 长度减小
    return true;
}
```

注意: 本题也可从后向前扫描顺序表, 每遇到一个值在 s 到 t 之间的元素, 则删除该元素, 其后的所有元素全部前移。但移动次数远大于前者, 效率不够高。

2. 算法思想: 对链表进行遍历, 在每次遍历中找出整个链表的最小值元素, 输出并释放结点所占空间; 再查找次小值元素, 输出并释放空间, 如此下去, 直至链表为空, 最后释放头结点所占存储空间。该算法的时间复杂度为 $O(n^2)$ 。

本题代码如下:

```
void Min_Delete(LinkList &head) {
```

```
// head 是带头结点的单链表的头指针, 本算法按递增顺序输出单链表中的数据元素
while (head->next != NULL) {           // 循环到仅剩头结点
    LNode *pre = head;                  // pre 为元素最小值结点的前驱结点的指针
    LNode *p = pre->next;                // p 为工作指针
    while (p->next != NULL) {
        if (p->next->data < pre->next->data)
            pre = p;                    // 记住当前最小值结点的前驱
        p = p->next;
    }
    print(pre->next->data);               // 输出元素最小值结点的数据
    LNode *u = pre->next;                // 删除元素值最小的结点, 释放结点空间
    pre->next = u->next;
    free(u);
} // while
free(head); // 释放头结点
}
```

若题设不限制数组辅助空间的使用, 则可先将链表的数据复制在数组中, 再采用时间复杂度为 $O(n\log_2 n)$ 的排序算法进行排序, 然后将数组元素输出, 时间复杂度为 $O(n\log_2 n)$ 。

3. 解答: 对于循环单链表 L , 在不空时循环: 每循环一次查找一个最小结点 (由 minp 指向最小值结点, minpre 指向其前驱结点) 并删除它。最后释放头结点。

本题代码如下:

```
void Del_All(LinkList &L) {
    // 本算法实现每次删除循环单链表中的最小元素, 直到链表空为止
    LNode *p, *pre, *minp, *minpre;
    while (L->next != L) {           // 表不空, 循环
        p = L->next;                  // p 为工作指针, pre 指向其前驱
        pre = L;
        minp = p;
        minpre = pre;
        while (p != L) {              // 循环一趟, 查找最小值结点
            if (p->data < minp->data) { // 找到值更小的结点
                minp = p;
                minpre = pre;
            }
            pre = p;                  // 查找下一个结点
            p = p->next;
        }
        printf("%d", minp->data);      // 输出最小值结点元素
        minpre->next = minp->next;     // 最小值结点从表中“断”开
        free(minp);                  // 释放空间
    }
    free(L);                          // 释放头结点
}
```

4. 解答: (1) 从逻辑上可以把题中的单链表看成是一个循环单链表, 因此在第一趟遍历时, 可以将单链表改造成循环单链表。算法的基本设计思想如下:

① 设置一个计数变量 i , 初始时工作指针 p 指向第一个结点, 故初始置 i 为 1。

② 第一趟访问时, 令尾结点的指针 (初始为 NULL) 指向头指针 L (注意: 未改造前, 单链表中仅有尾结点的 next 域为 NULL , 且改造后表中不存在 next 域为 NULL 的结点)。

③如果 i 等于 m : 则摘下、输出并删除这个计数值等于 m 的结点, 工作指针 p 指向被删除的下一个结点, 计数变量重新置为 1。

④如果 i 不等于 m : 继续访问单链表的下一个结点, 计数值加 1。

重复过程③④, 直到单链表中所有的结点均已删除, 即 L 指针等于 $NULL$ 为止。

(2) 算法的实现如下:

```
typedef struct LNode{                                //链表结点的结构定义
    ElemType data;                                    //结点数据
    struct LNode *next;                               //结点链接指针
} *LinkList;
void Loop_Del(LinkList &L,int n,int m){
    int i=1;                                           //计数, 初始 p 指向第 1 个元素
    LNode *pre,*p=L,*pDel;                           //p 工作指针, pDel 指向待删除结点
    while(L){                                         //L 不空则循环
        if(p->next==NULL)                             //第一趟, 将尾结点指向 L
            p->next=L;
        if(i==m){                                     //计数到第 m 个结点
            pDel=p;                                    //摘下这个结点
            pre->next=p->next;                          //断链
            p=p->next;
            Output(pDel->data);                        //输出并销毁
            free(pDel);                                //重新开始计数
            i=1;
        }
        else{                                         //非第 m 个结点, 则继续计数
            pre=p;                                     //逐链访问
            p=p->next;
            i++;                                       //计数值加 1
        }
    } //while(L)
}
```

(3) 在单链表中共有 n 个结点, 每删除一个结点需要遍历 m 次, 故总的时间复杂度为 $O(m*n)$ 。若 m 为常量, 则时间复杂度为 $O(n)$ 。算法的空间复杂度为 $O(1)$

2.3 线性表的应用算法专题

2.3.1 双指针策略

1. 给定两个单链表, 编写算法找出两个链表的公共结点。

2. 两个整数序列 $A = a_1, a_2, a_3, \dots, a_m$ 和 $B = b_1, b_2, b_3, \dots, b_n$ 已经存入两个单链表中, 设计一个算法, 判断序列 B 是否是序列 A 的连续子序列。

3. 设计一个算法完成以下功能: 判断一个链表是否有环, 如果有, 找出环的入口点并返回, 否则返回 $NULL$ 。

4. 已知线性表 $(a_1, a_2, a_3, \dots, a_n)$ 存放在一维数组 A 中。试设计一个在时间和空间两方面都尽可能高效的算法, 将所有奇数号元素移到所有偶数号元素前, 并且不得改变奇数号 (或偶数号) 元素之间的相对顺序, 要求:

- (1) 给出算法的基本设计思想。
- (2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。
- (3) 说明你所设计算法的时间复杂度和空间复杂度。

5. 设有一元素为整数的线性表 L, 存放在一维数组 A[0, ..., n-1] 中, 设计一个算法, 以 A[n-1], 为参考量, 将该数组分为左右两个部分, 其中左半部分的元素值均小于等于 A[n-1], 右半部分的元素值均大于 A[n-1], A[n-1] 则位于这两部分之间。要求结果仍存放在数组 A 中。

6. 有一个顺序表 L, 其元素为整型数据, 设计一个算法, 将 L 中所有小于表头元素的整数放在前半部分, 大于表头元素的整数放在后半部分。

【答案解析】

1. 解答: 两个单链表有公共结点, 即两个链表从某一结点开始, 它们的 next 都指向同一个结点。由于每个单链表结点只有一个 next 域, 因此从第一个公共结点开始, 之后它们所有的结点都是重合的, 不可能再出现分叉。所以两个有公共结点而部分重合的单链表, 拓扑形状看起来像 Y, 而不可能像 X。

本题极易联想到“蛮”方法: 在第一个链表上顺序遍历每个结点, 每遍历一个结点, 二个链表上顺序遍历所有结点, 若找到两个相同的结点, 则找到了它们的公共结点。显然, 法的时间复杂度为 $O(len1 \times len2)$ 。

接下来我们试着去寻找一个线性时间复杂度的算法。先把问题简化: 如何判断两个单向链表有没有公共结点? 应注意到这样一个事实: 若两个链表有一个公共结点, 则该公共结点之后的所有结点都是重合的, 即它们的最后一个结点必然是重合的。因此, 我们判断两个链表是不是有重合的部分时, 只需要分别遍历两个链表到最后一个结点。若两个尾结点是一样的, 则说明它们有公共结点, 否则两个链表没有公共结点。

然而, 在上面的思路中, 顺序遍历两个链表到尾结点时, 并不能保证在两个链表上同时到达尾结点。这是因为两个链表长度不一定一样。但假设一个链表比另一个长 k 个结点, 我们先在长的链表上遍历 k 个结点, 之后再同步遍历, 此时我们就能保证同时到达最后一个结点。由于两个链表从第一个公共结点开始到链表的尾结点, 这一部分是重合的, 因此它们肯定也是同时到达第一个公共结点的。于是在遍历中, 第一个相同的结点就是第一个公共的结点。

根据这一思路中, 我们先要分别遍历两个链表得到它们的长度, 并求出两个长度之差。在长的链表上先遍历长度之差个结点之后, 再同步遍历两个链表, 直到找到相同的结点, 或者一直到链表结束。此时, 该方法的时间复杂度为 $O(len1 + len2)$ 。

本题代码如下:

```
LinkedList Search_1st_Common(LinkedList L1, LinkedList L2) {
    // 本算法实现在线性的时间内找到两个单链表的第一个公共结点
    int len1 = Length(L1), len2 = Length(L2); // 计算两个链表的表长
    LinkedList longList, shortList;
    if (len1 > len2) { // L1 表长较长
        longList = L1->next;
        shortList = L2->next;
        dist = len1 - len2; // 表长之差
    }
    else { // L2 表长较长
        longList = L2->next;
```

```

        shortList = L1->next;
        dist = len2 - len1;           // 表长之差
    }
    while (dist--)
        longList = longList->next;    // 表长的链表先遍历到第 dist 个结点, 然后同步
    while (longList != NULL) {        // 同步寻找共同结点
        if (longList == shortList)
            return longList;          // 找到第一个公共结点
        else {
            longList = longList->next;
            shortList = shortList->next;
        }
        // 继续同步寻找
    }
    return NULL;                      // 如果没有找到公共结点, 返回 NULL
}

```

2.算法思想: 因为两个整数序列已存入两个链表中, 操作从两个链表的第一个结点开始, 若对应数据相等, 则后移指针; 若对应数据不等, 则 A 链表从上次开始比较结点的后继开始, B 链表仍从第一个结点开始比较, 直到 B 链表到尾表示匹配成功。A 链表到尾而 B 链表未到尾表示失败。操作中应记住 A 链表每次的开始结点, 以便下次匹配时好从其后继开始。

本题代码如下:

```

int Pattern(LinkList A, LinkList B) {
    // A 和 B 分别是数据域为整数的单链表, 本算法判断 B 是否是 A 的子序列
    LNode *p = A;           // p 为 A 链表的工作指针, 本题假定 A 和 B 均无头结点
    LNode *pre = p;         // pre 记住每趟比较中 A 链表的开始结点
    LNode *q = B;           // q 是 B 链表的工作指针
    while (p && q) {
        if (p->data == q->data) { // 结点值相同
            p = p->next;
            q = q->next;
        }
        else {
            pre = pre->next;      // A 链表新的开始比较结点
            p = pre;
            q = B;               // q 从 B 链表第一个结点开始
        }
    }
    if (q == NULL)             // B 已经比较结束
        return 1;             // B 是 A 的子序列
    else
        return 0;             // B 不是 A 的子序列
}

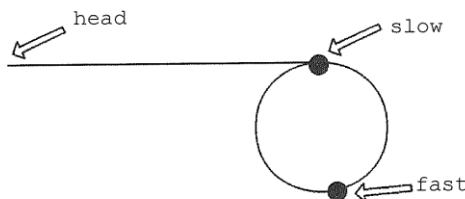
```

注意: 该题其实是字符串模式匹配的链式表示形式, 读者应该结合字符串模式匹配的内容重新考虑能否优化该算法。

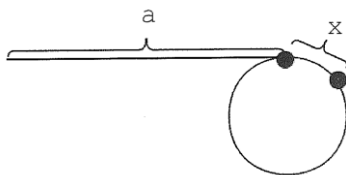
3.解答: (1) 算法的基本设计思想

设置快慢两个指针分别为 fast 和 slow, 初始时都指向链表头 head。slow 每次走一步, 即 slow=slow->next; fast 每次走两步, 即 fast=fast->next->next。由于 fast 比 slow 走得快, 如果有环, 那么 fast 一定会先进入环, 而 slow 后进入环。当两个指针都进入环后, 经过若干操作后两个指针定能在环上相遇。这样就可以判断一个链表是否有环。

如下图所示, 当 slow 刚进入环时, fast 早已进入环。因为 fast 每次比 slow 多走一步且 fast 与 slow 的距离小于环的长度, 所以 fast 与 slow 相遇时, slow 所走的距离不超过环的长度。



如下图所示, 设头结点到环的入口点的距离为 a , 环的入口点沿着环的方向到相遇点的距离为 x , 环长为 r , 相遇时 fast 绕过了 n 圈。



则有 $2(a+x) = a + n \cdot r + x$, 即 $a = n \cdot r - x$ 。显然从头结点到环的入口点的距离等于 n 倍的环长减去环的入口点到相遇点的距离。因此可设置两个指针, 一个指向 head, 一个指向相遇点, 两个指针同步移动 (均为一次走一步), 相遇点即为环的入口点。

2) 本题代码如下:

```

LNode* FindLoopStart(LNode *head) {
    LNode *fast = head, *slow = head; // 设置快慢两个指针
    while (slow != NULL && fast->next != NULL) {
        slow = slow->next; // 每次走一步
        fast = fast->next->next; // 每次走两步
        if (slow == fast) break; // 相遇
    }
    if (slow == NULL || fast == NULL || fast->next == NULL)
        return NULL; // 没有环, 返回 NULL
    LNode *p1 = head, *p2 = slow; // 分别指向开始点、相遇点
    while (p1 != p2) {
        p1 = p1->next;
        p2 = p2->next;
    }
    return p1; // 返回入口点
}
    
```

4. 本题考查顺序表的应用。如果不要求相对顺序, 则可以设置两个指针, 一个指针从前往后找偶数号元素, 另一个指针从后往前找奇数号元素, 将两者交换, 直到指针相遇。此外本题容易联系到, 先将偶数号元素复制到一个辅助空间, 然后整理数组剩下的奇数号元素, 最后将辅助空间中的元素复制到数组的后半部分, 但这种思路的空间复杂度为 $O(n)$ 。

以数组 $A[1 \sim 10]$ 为例, 为了思路方便, 我们画出如下草图:

数组: 1 2 3 4 5 6 7 8 9 10

目标: 1 3 5 7 9 2 4 6 8 10

由上面的草图, 下划线的元素像气泡一样依次、顺序地上浮, 不难联想到冒泡排序, 只是每交换一次元素, 气泡增大一个元素 (如下草图所示)。大体的思路如下:

第一趟冒泡: 从数组尾部找到第一个奇数号结点, 和其前面的偶数号结点交换, 这样就形成了

两个前后相连的奇数号元素 (将其看成一个整体一块)。

第二趟冒泡: 如果这个一块前存在元素, 将这个一块与其前面的元素交换 (只要一块前存在元素, 则必然是偶数号元素), 这样就形成了三个前后相连的奇数号元素。

第三趟冒泡: 依次类推, 直到这个整体一块前面没有元素为止。

第 1 趟 1 2 3 4 5 6 7 8 9 10

第 2 趟 1 2 3 4 5 6 7 9 8 10

第 3 趟 1 2 3 4 5 7 9 6 8 10

第 4 趟 1 2 3 5 7 9 4 6 8 10

第 5 趟 1 3 5 7 9 2 4 6 8 10

【提示】块前存在元素, 说明气泡未浮出水面, 底层的气泡上浮一层, 当遇到上面有气泡, 则气泡会合并。当最底层气泡将其他分散的气泡聚合, 且上浮到水面, 算法完成。

【解答】(1) 算法的基本设计思想:

①在数组尾部从后往前, 找到第一个奇数号元素, 将此元素与其前面的偶数号元素交换。这样, 就形成了两个前后相连且相对顺序不变的奇数号元素一块。

②暂存①中一块前面的偶数号元素, 将一块内奇数号结点依次前移, 然后将暂存的偶数号结点复制到空出来的数组单元中。就形成了三个连续的奇数号元素一块。

③暂存②中一块前面的偶数号元素, 将一块内奇数号结点依次前移, 然后将暂存的偶数号结点复制到空出来的数组单元中。就形成了四个连续的奇数号元素一块。

④如此继续, 直到前一步的一块前没有元素为止。

(2) 算法的设计如下:

```
void Bubble_Swap(ElemType A[], int n){
    int i=n, v=1;           //i 为工作指针, 初始假设 n 为奇数, v 为“块”的大小
    ElemType temp;          //辅助变量
    if(n%2==0) i=n-1;       //若 n 为偶数, 则令 i 为 n-1
    while(i>1){             //假设数组从 1 开始存放。当 i=1 时, 气泡浮出水面
        temp=A[i-1];        //将“块”前的偶数号元素暂存
        for(int j=0; j<v; j++) //将大小为 v 的“块”整体向前平移
            A[i-1+j]=A[i+j]  //从前往后依次向前平移
        A[i+v-1]=temp;      //暂存的奇数号元素复制到平移后空出的位置
        i--; v++;           //指针向前, 块大小增 1
    }
}
```

(3) 一共进行了 $n/2$ 次交换, 每次交换的元素个数从 $1 \sim n/2$, 因此时间复杂度为 $O(n^2)$ 。虽然时间复杂度为 $O(n^2)$, 但因 n^2 前的系数很小, 实际达到的效率是很高的。算法的空间复杂度为 $O(1)$ 。

【说明】从宏观上看 (将交换的一块看成一个整体), 虽然仅进行了一趟冒泡, 但是交换的对象的一块, 需要逐个复制, 因此时间复杂度不能想当然的认为是 $O(n)$ 。

5. 算法分析: 本题属于快速排序算法划分部分。

```
void divide (int A [ ], int n) {
    int temp ;
    int i=0, j=n-1;
    temp=A[i];
    A [i] =A [j] ;
    A[j]=temp;
    temp=A[i];
    /*下面这个循环将数组中小于 temp 的元素放在左边, 大于 temp 的元素放在右边*/
    while(i!=j) {
```



```

while (j > i && A[j] > temp) --j;    //从右往左扫描找到一个小于 temp 的元素
if(i < j) {
    A[j] = A[i];                    //放在 temp 左边
    ++i;                           //i 指针右移一位
}
while (i < j && A[i] < temp) ++i;    //从左往右扫描, 找到一个
//个大于 temp 的元素
if(i < j) {
    A[j] = A[i];                    //放在 temp 右边
    --j;                           //j 指针左移一位
}
}
A[i] = temp;                        //将 temp 放在最终位置
}

```

6. 本题解决方法为: 先将 L 的第一个元素存于变量 temp 中, 然后定义两个整型变量 i 和 j。i 从左往右扫描, j 从右往左扫描, 边扫描边交换。具体的执行过程如下图所示。

各步的解释如下:

① 开始状态 temp=2, i=0, j=L.length-1。

② 检查 j 所指元素是否比 2 小, 此时发现 -1 比 2 小, 则执行 L.data[i]=L.data[j]; ++i; (i 中元素已经被存入 temp, 所以可以直接覆盖), 并且 i 后移一位, 准备开始 i 的扫描, 此时发现 1 比 2 小, 因此什么都不做。

③ i 继续从左往右移动, 边移动边检测, 看是否 i 所指元素比 2 大, 此时发现 -7 比 2 小, 因此 i 在此位置时什么都不做。

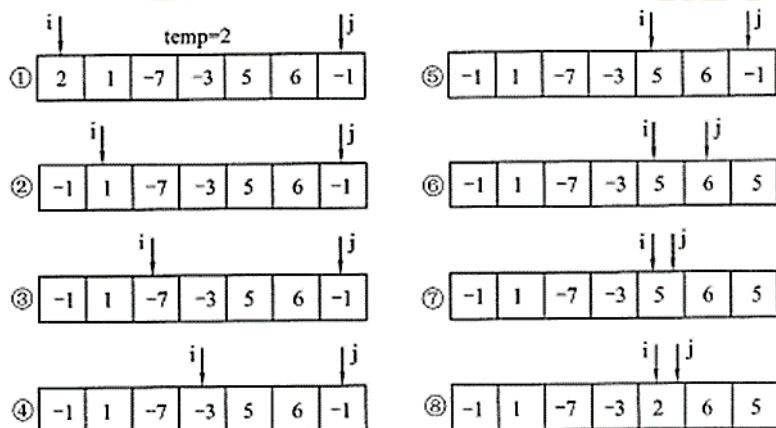
④ i 继续往右移动, 此时 i 所指元素为 -3, 也比 2 小, 此时什么都不做。

⑤ i 继续往右移动, 此时 i 所指元素为 5, 比 2 大, 因此执行 L.data[j]=L.data[i]; --j; (j 中元素已被保存, j 前移一位, 准备开始 j 的扫描)。

⑥ j 往左移动, 此时 j 所指元素为 6, 比 2 大, j 在此位置时什么都不做。

⑦ j 继续往左移动, 此时 j=i, 说明扫描结束。

⑧ 执行 L.data[i]=temp; , 此时整个过程结束, 所有比 2 小的元素都被移到了 2 的前边, 所有比 2 大的元素都被移到了 2 的后边。



以上过程要搞清楚两点:

1) i 和 j 是轮流移动的, 即当 i 找到比 2 大的元素时, 将 i 所指元素放入 j 所指位置, i 停在当前位置不动, j 开始移动。当 j 找到比 2 小的元素时, 将 j 所指元素放在 i 所指位置, j 停在当前位置

不动, i 开始移动, 如此交替, 直到 $i=j$ 。

2) 每次元素覆盖 (如执行 $L.data[i]=L.data[j]$;) 不会造成元素丢失, 因为在这之前被覆盖位置的元素已经存入其他位置。

由以上分析可写出如下算法:

```
void move (SqList &L) {           //L 要改变, 所以用引用型
    int temp;
    int i=0, j = L.length - 1;
    temp=L.data[i];
    while(i<j) { /*关键步骤开始*/
        while (i<j&&L.data[j]>temp)—j; /* j 从右往左扫描, 当来到第一个比 temp,小的元素时停止, 并且每走一步都要判断 i 是否小于 j 这个判断容易遗漏 */
        if(i<j) { /*检测看是否已满足 i<j,这一步同样
            L.data[i]=L.data[j]; //移动元素
            ++i; //i 右移一位
        }
        while (i < j &&L.data[i] <=temp)++i; //与上边的处理类似
        if(i<j) { //与上边的处理类似
            L.data[j]=L.data[i]; //与上边的处理类似
            --j;
        } /*关键步骤结束*/
    }
    L.data[i]=temp; //将表首元素放在最终位置
}
```

说明: 在这里之所以如此详细地讲解此题的执行过程, 是因为此题是后边排序章节中快速排序的关键步骤, 因此考生务必熟练掌握其执行过程以及算法。

2.3.2 逆置策略

1. 已知在一维数组 $A[m+n]$ 中依次存放两个线性表 $(a_1, a_2, a_3, \dots, a_m)$ 和 $(b_1, b_2, b_3, \dots, b_n)$ 。试编写一个函数, 将数组中两个顺序表的位置互换, 即将 $(b_1, b_2, b_3, \dots, b_n)$ 放在 $(a_1, a_2, a_3, \dots, a_m)$ 的前面。

2. 假设有一带头结点的循环双链表表示的线性表 $SL = (a_1, a_2, \dots, a_{n-1}, a_n)$ 。设计在时间和空间上都尽可能高效的算法, 将线性表 L 改造成 $SL = (a_1, a_3, \dots, a_n, \dots, a_4, a_2)$ 。要求:

- (1) 给出算法的基本设计思想。
- (2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。
- (3) 说明你所设计算法的时间复杂度与空间复杂度。

【答案解析】

1. 算法思想: 先将数组 $A[m+n]$ 中的全部元素 $(a_1, a_2, a_3, \dots, a_m, b_1, b_2, b_3, \dots, b_n)$ 原地逆置为 $(b_n, b_{n-1}, b_{n-2}, \dots, b_1, a_m, a_{m-1}, a_{m-2}, \dots, a_1)$, 再对前 n 个元素和后 m 个元素分别使用逆置算法, 即可得到 $(b_1, b_2, b_3, \dots, b_n, a_1, a_2, a_3, \dots, a_m)$, 从而实现顺序表的位置互换。

```
typedef int DataType;
void Reverse(DataType A[], int left, int right, int arraySize) {
    // 逆转 (aleft, aleft+1, aleft+2, ..., aright) 为 (aright, aright-1, ..., aleft)
    if (left >= right || right >= arraySize)
        return;
    int mid = (left + right) / 2;
    for (int i = 0; i <= mid - left; i++) {
```

```

        DataType temp = A[left + i];
        A[left + i] = A[right - i];
        A[right - i] = temp;
    }
}

void Exchange(DataType A[], int m, int n, int arraySize) {
/* 数组 A[m+n] 中, 从 0 到 m-1 存放顺序表 (a1, a2, a3, ..., am), 从 m 到 m+n-1 存放顺序表 (b1, b2, b3, ..., bn), 算法将这两个表的位置互换 */
    Reverse(A, 0, m + n - 1, arraySize);
    Reverse(A, 0, n - 1, arraySize);
    Reverse(A, n, m + n - 1, arraySize);
}

```

2. (1) 基本设计思想: 从改造后的线性表 L 可以看出, 前部分结点为奇序号结点, 并且递增; 后部分结点为偶序号结点, 并且递减。所以可以考虑建立两个新的循环双链表, 一个带头结点的循环双链表 L (由原先的 L 改造而来), 保存原先线性表 L 的奇数号结点, 可以采取尾插法, 让其序号顺序递增; 而另外一个不带头结点的循环双链表 s, 保存原先线性表 L 的偶数号结点, 可以采取头插法, 让其序号顺序递减。最后, 将 L 和 s 两个循环双链表连接成一个循环双链表 L, 并且 L 为其头结点指针。

```

void modify(struct node *head) {
    struct node *s = NULL;
    struct node *L = head;
    struct node *p = L->next, *pl;
    L->next = L->prev = NULL;
    for (; p != L; p = pl) {
        if (p->next != L) {
            pl = p->next;
            p->next = pl->next;
            pl->next->prev = p;
            // 把偶数结点插入 s
            if (s == NULL) {
                s = pl;
                pl->next = pl->prev = pl;
            }
            else {
                pl->next = s;
                pl->prev = s->prev;
                s->prev->next = pl;
                s->prev = pl;
                s = s->prev;
            }
        }
        pl = p->next;
        L->prev->next = p;
        p->next = L;
        p->prev = L->prev;
        L->prev = p;
    }
    if (s != NULL) {
        p = s->prev;
        p->next = L;
    }
}

```

// 初始化一个新的链表指针 s
// L 指向原链表的头节点
// p 指向头节点的下一个节点
// 将头节点的 next 和 prev 设为 NULL
// 遍历链表, 直到 p 等于 L
// 删除偶数结点
// pl 指向 p 的下一个节点
// 更新 p 的 next 指向 pl 的下一个节点
// 更新 pl 的下一个节点的 prev 指向 p
// 如果 s 为空
// s 指向 pl
// 创建一个循环链表, 只有 pl 节点
// 如果 s 不为空
// pl 的 next 指向 s
// pl 的 prev 指向 s 的前一个节点
// s 的前一个节点的 next 指向 pl
// s 的 prev 指向 pl
// 更新 s 指向 pl
// 更新 pl 指向 p 的下一个节点
// 更新 L 的前一个节点的 next 指向 p
// 更新 p 的 next 指向 L
// 更新 p 的 prev 指向 L 的前一个节点
// 更新 L 的 prev 指向 p
// 合并两个链表
// p 指向 s 的最后一个节点
// 更新 s 的最后一个节点的 next 指向 L

```

        L->prev->next = s;           // 更新 L 的前一个节点的 next 指向 s
        s->prev = L->prev;          // 更新 s 的 prev 指向 L 的前一个节点
        L->prev = p;                // 更新 L 的 prev 指向 p
    }
}

```

(3) 空间复杂度分析: 除去链表本身的空间外, 额外的空间消耗为 $O(1)$ 。其实本题可以看成是原来链表的重新组合, 并没有开辟新的空间。

时间复杂度分析: 整个过程相当于把链表遍历了一遍, 所以时间复杂度为 $O(n)$ 。

2.3.3 哈希策略

1. 给定一字符串, 该字符串中存在若干对相同的字符, 设计一个在时间和空间上尽可能高效的算法, 找出一对相同字符在该字符串中的最大距离。例如: “KLabcLdecL”, 其中第一个“L”和最后一个“L”相距最远, 它们在原字符串中的位置相差 8, 要求:

- (1) 给出算法的基本设计思想。
- (2) 根据设计思想, 采用 C、C++ 语言描述算法, 关键之处给出注释。
- (3) 说明你所设计算法的时间复杂度和空间复杂度。

【答案解析】

1. (1) 设计思想: 在遍历字符数组的过程中, 对已经访问过的字符进行标记, 同时记下它们第一次出现在原字符串中的位置, 当以后再次遍历到此字符时, 根据当前的位置和第一次出现的位置, 求出它们之间的距离, 然后用得到的距离和当前最大距离相比较。为此需要设置一个数组用来存放已经访问过的字符, 为了能加快搜索, 采用字符的 ASCII 码作为数组的下标来支持随机访问, 通过对应下标中的数组元素的访问标记来判断它是否被访问过, 同时获取它第一次出现的位置, 每遍历一个字符就开始判断, 并用 max 来保存当前最大距离。

(2) 代码:

```

int getMaxLength(char str[], int n) {
    int max = 0, i;           // 初始化最大长度为 0
    /*创建一个大小为 128 的数组 (ASCII 字符集的大小), 用来存储每个字符最后出现的位置。
    -1 表示该字符还没有在字符串中出现过。*/
    int isChInArray[128];
    for (i = 0; i < 128; ++i)
        isChInArray[i] = -1;   // 将所有元素初始化为-1
    for (i = 0; i < n; ++i) {   // 遍历字符串中的每一个字符
        if (isChInArray[str[i]] == -1) // 如果字符第一次出现, 则记录它的位置
            isChInArray[str[i]] = i;
        else {                  // 如果字符之前已经出现过, 则计算从上次出现到现在的距离
            int tempL = i - isChInArray[str[i]];
            if (max < tempL) // 更新最大长度
                max = tempL;
            isChInArray[str[i]] = i; // 更新最近出现的字符位置
        }
    }
    return max;               // 返回找到的最大长度
}

```

(3) 时间复杂度 $O(n)$ 。空间复杂度 $O(1)$ 。

2.3.4 对称策略

1. 已知长度为 n ($n > 1$) 的单链表, 表头指针为 L , 结点结构由 $data$ 和 $next$ 两个域构成, 其中 $data$ 域为字符型。试设计一个在时间和空间两方面都尽可能高效的算法, 判断该单链表是否中心对称 (例如 xyx 、 $xyyyxx$ 都是中心对称的), 要求:

- (1) 给出算法的基本设计思想。
- (2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。
- (3) 说明你所设计算法的时间复杂度和空间复杂度。

2. 设计一个算法用于判断带头结点的循环双链表是否对称。

【答案解析】

1. 本题考查单链表的应用。思路 1 (借助栈, 空间复杂度高): 将表的前半部分依次进栈, 依次访问后半部分时, 从栈中弹出一个元素, 进行比较。

思路 2 (类似折纸的思想, 算法复杂): 找到中间位置的元素, 将后半部分的链表就地逆置, 然后前半部分从前往后、后半部分从后往前比较比较结束后再恢复 (题中没有说不能改变链, 故可不恢复)。

(1) 算法的基本设计思想:

① 借助辅助栈, 将链表的前一半元素依次进栈。注意 n 为奇数时要特殊处理。

② 在处理链表的后一半元素时, 当访问到链表的一个元素后, 就从栈中弹出一个元素, 两元素比较, 若相等, 则将链表中下一元素与栈中再弹出元素比较, 直至链表到尾。

③ 若栈是空栈, 则得出链表中心对称的结论; 否则, 当链表中一元素与栈中弹出元素不等时, 结论为链表非中心对称。

(2) 算法的实现如下:

//代码 1: 采用栈的操作方式

```
typedef struct LNode{
    ElemType data;
    struct LNode *next;
} *LinkList;
int Str_Sym(LinkList L,int n){
    Stack s;initstack(s);
    LNode *q,*p=L->next;
    for(int i=1;i<n/2;i++){
        Push(p);
        p=p->next;
    }
    if(n%2==1)
        p=p->next;
    while(p!=null){
        q=pop(s);
        if(q->data==p->data)
            p=p->next;
        else
            break;
    }
    if(empty(s))
        return 1;
```

// 链表结点的结构定义

// 结点数据

// 结点链接指针

// 本算法判断带头结点的单链表是否是中心对称

// 初始化栈

// q 指向出栈元素, p 工作指针

// 前一半结点入栈

// 若 n 为奇数, 需要特殊处理

// 后一半表依次和前一半表比较

// 出栈一个结点

// 相等则继续比较下一个结点

// 不等则跳出循环

// 栈空, 则说明对称

```

else
    return 0;                // 否则不对称
}
//代码 2: 采用字符数组的操作方式
int dc(LinkList L, int n) {    // L 是带头结点的 n 个元素单链表, 本算法判断链表是否是中心对称
    int i;
    char s[n/2];              // s 字符栈
    LNode *p = L->next;       // p 是链表的工作指针, 指向待处理的当前元素
    for (i = 0; i < n/2; i++) { // 链表前半一半元素进栈
        s[i] = p->data;
        p = p->next;
    }
    i--;                      // 恢复最后的 i 值
    if (n % 2 == 1)           // 若 n 是奇数, 后移过中心结点
        p = p->next;
    while (p != NULL && s[i] == p->data) { // 检测是否中心对称
        i--;                  // i 充当栈顶指针
        p = p->next;
    }
    if (i == -1)               // 栈为空栈
        return 1;             // 链表中心对称
    else
        return 0;             // 链表不中心对称
}

```

算法先将“链表的前一半”元素(字符)进栈。当 n 为偶数时, 前一半和后一半的个数相同; 当 n 为奇数时, 链表中心结点字符不必比较, 移动链表指针到下一字符开始比较。比较过程中遇到不相等时, 立即退出 while 循环, 不再进行比较。

本题也可以先将单链表中的元素全部入栈, 然后扫描单链表 L 并比较, 直到比较到单链表尾为止, 但算法需要两次扫描单链表 L , 效率不及上述算法高。

(3) 算法的时间复杂度为 $O(n)$, 空间复杂度为 $O(n)$ 。

2. 算法思想: 让 p 从左向右扫描, q 从右向左扫描, 直到它们指向同一结点($p == q$, 当循环双链表中结点个数为奇数时)或相邻($p \rightarrow next = q$ 或 $q \rightarrow prior = p$, 当循环双链表中结点个数为偶数时)为止, 若它们所指结点值相同, 则继续进行下去, 否则返回 0。若比较全部相等, 则返回 1。

本题代码如下:

```

int Symmetry(DLinkList L) {
    // 本算法从两头扫描循环双链表, 以判断链表是否对称
    DNode *p = L->next, *q = L->prior; // 两头工作指针
    while (p != q && q->next != p) { // 循环跳出条件
        if (p->data == q->data) { // 所指结点值相同则继续比较
            p = p->next;
            q = q->prior;
        }
        else
            return 0; // 否则, 返回 0
    }
    return 1; // 比较结束后返回 1
}

```

注意: while 循环第二个判断条件易误写成 $p \rightarrow next != q$, 分析这样会产生什么问题。

2.3.5 旋转策略

1. 将一个数组最开始的若干个元素搬到数组的末尾，称之为数组的旋转。输入一个已排好序数组的一个旋转，求该旋转数组的最小元素。如，数组{3, 4, 5, 1, 2}为有序数组{1, 2, 3, 4, 5}的一个旋转数组，该数组的最小值为 1。

- (1) 给出算法的基本设计思想。
- (2) 根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。
- (3) 说明你所设计算法的时间复杂度和空间复杂度。

【答案解析】

1. (1) 算法的基本设计思想：注意到旋转之后的数组实际上可以划分成两个排序的子数组，且前面的子数组的元素都大于或等于后面子数组的元素，而最小的元素刚好是这两个子数组的分界线。我们试着用二元查找的思路寻找这个最小的元素：

定义两个指针，分别指向数组的第一个元素和最后一个元素。按照题目旋转的规则，第一个元素应该是大于或等于最后一个元素的。再定义一个指针指向中间的元素，如果该中间元素位于前面的递增子数组，那么它应大于或等于第一个指针指向的元素，此时最小的元素位于右子数组，然后把第一指针指向该中间元素，这样可以在缩小的范围内继续寻找。同样，如果该中间元素位于后面的递增子数组，思路和上面是类似的。

按照上述思路，第一个指针总是指向前面递增数组的元素，而第二个指针总是指向后面递增数组的元素。最后，第一个指针将指向前面子数组的最后一个元素，而第二个指针会指向后面子数组的第一个元素，此时两个指针相邻，而第二个指针指向的正好是最小元素。这就是循环结束的条件。

(2) 算法的实现如下：

```
int Min(int *numbers, int length) { // 检查输入数组是否为空或长度小于等于 0，如果是则返回 0
    if (numbers == 0 || length <= 0)
        return 0;
    int index1 = 0;                // 第一个指针，指向数组的起始位置
    int index2 = length - 1;       // 第二个指针，指向数组的末尾位置
    int indexMid = index1;         // 中间指针，初始时指向起始位置
    while (numbers[index1] >= numbers[index2]) { // 当左指针的值大于等于右指针的值时进入循环
        if (index2 - index1 == 1) { // 如果左右指针相邻，则最小值一定是右指针指向的元素
            indexMid = index2;
            break;
        }
        indexMid = (index1 + index2) / 2; // 计算中间位置
        // 如果中间值大于等于左指针的值，说明最小值在右半部分
        if (numbers[indexMid] >= numbers[index1])
            index1 = indexMid; // 将左指针移动到中间位置
        // 如果中间值小于等于右指针的值，说明最小值在左半部分（包括中间位置）
        else if (numbers[indexMid] <= numbers[index2])
            index2 = indexMid; // 将右指针移动到中间位置
    }
    return numbers[indexMid]; // 返回最小值，即右指针指向的元素
}
```

每次都把寻找的范围缩小了一半，时间复杂度为 $O(\log_2 N)$ 、空间复杂度为 $O(1)$ 。

解法二：本题最直观的解法并不难。从头到尾遍历数组一次，就能找出最小元素，时间复杂度显然是 $O(N)$ 。但这个思路没有利用输入数组的特性。

2.4 线性表应用扩展算法专题

1. C/C++中的 `int` 类型能表示的范围是 $-2^{31} \sim 2^{31}-1$ 。`unsigned` 类型能表示的范围是 $0 \sim 2^{32}-1$, 即 $0 \sim 4294967295$ 。所以, `int` 和 `unsigned` 类型变量, 都不能保存超过 10 位的整数。有时我们需要参与运算的数, 可能会远远不止 10 位, 例如要求 $100!$ 的精确值。

设有两个用来存储大整数的链表 `p` 和 `q` 按照输入各位数字的逆序存放。设计一个算法, 实现两个大整数的加法。

【答案解析】

1. 算法描述:

```
#define maxSize 100
typedef struct LNode {
    int data;
    struct LNode* next;
} LNode;
// 头插法建表: 根据数组 a 创建一个链表, 链表中的元素顺序与数组 a 的顺序相反
void createListF(LNode*& head, int a[], int n) {
    head = (LNode*)malloc(sizeof(LNode)); // 创建头结点, 初始化 next 为 NULL
    head->next = NULL;
    for (int i = 0; i < n; ++i) {
        LNode* temp = (LNode*)malloc(sizeof(LNode)); // 遍历数组 a, 依次将每个元素插入到链表头部
        temp->data = a[i]; // 创建新结点
        temp->next = head->next; // 将新结点插入到链表头部
        head->next = temp;
    }
}
// 大整数相加: 将两个链表表示的大整数相加, 结果存储在 result 中
void addLargeInt(LNode* p, LNode* q, LNode*& result) {
    LNode* tempP = p->next; // 初始化指针, 指向两个链表的有效部分 (跳过头结点)
    LNode* tempQ = q->next;
    result = (LNode*)malloc(sizeof(LNode));
    result->next = NULL; // 创建结果链表的头结点, 并初始化 next 为 NULL
    LNode* r = result; // r 用于遍历结果链表
    int carry = 0; // 进位标志
    while (tempP != NULL && tempQ != NULL) { // 同时遍历两个链表, 逐位相加
        LNode* temp = (LNode*)malloc(sizeof(LNode)); // 创建新结点存储当前位的和
        temp->next = NULL;
        // 计算当前位的和及进位
        temp->data = (tempP->data + tempQ->data + carry) % 10;
        carry = (tempP->data + tempQ->data + carry) / 10;
        // 将新结点插入到结果链表中
        r->next = temp;
        r = r->next;
        // 移动指针到下一个结点
        tempP = tempP->next;
        tempQ = tempQ->next;
    }
    // 如果其中一个链表还有剩余结点, 继续处理
    if (tempP == NULL)
        tempP = tempQ; // 如果 p 已经结束, 切换到 q
    while (tempP != NULL) { // 处理剩余的结点
```



```
    LNode* temp = (LNode*)malloc(sizeof(LNode));
    temp->next = NULL;
    // 计算当前位的和及进位
    temp->data = (tempP->data + carry) % 10;
    carry = (tempP->data + carry) / 10;
    // 将新结点插入到结果链表中
    r->next = temp;
    r = r->next;
    // 移动指针到下一个结点
    tempP = tempP->next;
}
if (carry > 0) { // 如果最后有进位, 创建一个新结点存储进位值
    LNode* temp = (LNode*)malloc(sizeof(LNode));
    temp->next = NULL;
    temp->data = carry;
    r->next = temp; // 将新结点插入到结果链表中
}
}
// 递归打印链表 (逆序打印)
void tempPrintLinkList(LNode* p) {
    if (p != NULL) {
        tempPrintLinkList(p->next); // 先递归打印后续结点, 再打印当前结点的数据
        cout << p->data << " ";
    }
}
// 打印大整数相加的结果
void rePrintLinkList(LNode* head) {
    cout << "两个大整数相加的结果: " << endl;
    tempPrintLinkList(head->next); // 调用递归函数打印链表中的数据
    cout << endl;
}
```

3、栈

(一) 选择题

- 栈的应用不包括 ()。
A. 递归 B. 进制转换 C. 迷宫求解 D. 缓冲区
- 表达式 $a*(b+c)-d$ 的后缀表达式是 ()。
A. $abed*+-$ B. $abc+*d-$ C. $abc*+d-$ D. $-+*abed$
- 设计一个判别表达式中左、右括号是否配对出现的算法, 采用 () 数据结构最佳。
A. 线性表的顺序存储结构 B. 队列
C. 线性表的链式存储结构 D. 栈
- 设有一个空栈, 栈顶指针为 1000H, 每个元素需要一个存储单元, 执行 Push、Push、Pop、Push、Pop、Push、Pop、Push 操作后, 栈顶指针的值为 ()。
A. 1002H B. 1003H C. 1004H D. 1005H
- 向一个栈顶指针为 top 的链栈 (不带头结点) 中插入一个 x 结点, 则执行 ()。
A. $top \rightarrow next = x$ B. $x \rightarrow next = top \rightarrow next; top \rightarrow next = x$
C. $x \rightarrow next = top; top = x$ D. $x \rightarrow next = top, top = top \rightarrow next$
- 链栈 (不带头结点) 执行 Pop 操作, 并将出栈的元素存在 x 中, 应该执行 ()。
A. $x = top; top = top \rightarrow next$ B. $x = top \rightarrow data$
C. $top = top \rightarrow next; x = top \rightarrow data$ D. $x = top \rightarrow data; top = top \rightarrow next$
- 经过以下栈的操作后, 变量 x 的值为 ()。
InitStack (st); Push(st,a); Push (st,b); Pop(st, x); Top(st, x);
A. a B. b C. NULL D. FALSE
- 若一个栈的输入序列是 $P_1, P_2, \dots, P_n$, 输出序列是 1, 2, 3, ..., n, 若 $P_3 = 1$, 则 P_1 的值 ()。
A. 可能是 2 B. 一定是 2 C. 不可能是 2 D. 不可能是 3
- 若已知一个栈的入栈序列是 1, 2, 3, 4, 其出栈序列为 P_1, P_2, P_3, P_4 , 则 P_2P_4 不可能是 ()。
A. 2, 4 B. 2, 1 C. 4, 3 D. 3, 4
- 表达式 $a*(b+c)-d$ 的后缀表达式是 ()。
A. $abed*+-$ B. $abc+*d-$ C. $abc*+d-$ D. $-+*abed$
- 利用栈求表达式的值时, 设立运算数栈 OPEN。假设 OPEN 只有两个存储单元, 则在下列表达式中, 不会发生溢出的是 ()。
A. $A-B*(C-D)$ B. $(A-B)*C-D$ C. $(A-B \text{ 式})-D$ D. $(A-B)*(C-D)$
- 对于一个问题的递归算法求解和其相对应的非递归算法求解, ()。
A. 递归算法通常效率高一些 B. 非递归算法通常效率高一些
C. 两者相同 D. 无法比较
- 执行函数时, 其局部变量一般采用 () 进行存储。
A. 树形结构 B. 静态链表 C. 栈结构 D. 队列结构

【答案解析】

- D。缓冲区是用队列实现的, A、B、C 都是栈的典型应用。
- B。本题考查中缀表达式转化成后缀表达式的方法。
第一步: $b+c \rightarrow bc+$ (令 $x="bc+"$)。
第二步: $a*x \rightarrow ax*$ (令 $y="ax*"$)。
第三步: $y-d \rightarrow yd-$ 。

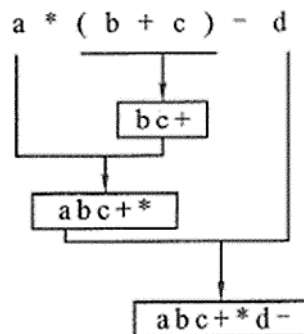
将 xy 还原后得到 $abc+*d-$ ，因此本题选 B。

补充知识点：中缀表达式转换成后缀表达式的另一种方式。

解析：可以通过手工添加或删除括号来将中缀表达式转换成后缀表达式，其过程为：先根据中缀表达式的求值次序加上括号，将右括号用相应的运算符替换，再删除所有的左括号。

例如，中缀表达式 $5+2*(1+6)-8/2$ 转换成后缀表达式的过程如下：

手工判断该表达式的计算过程。首先是先计算 $2*(1+6)$ ，加上括号变为 $5+(2*(1+6))-8/2$ ；再计算除法 $8/2$ ，加上括号变为 $5+(2*(1+6))-(8/2)$ ；接着进行加法运算，加上括号变为 $(5+(2*(1+6)))-(8/2)$ ；最后进行减法运算，加上括号变为 $((5+(2*(1+6)))-(8/2))$ 。运算符和右括号的对应关系如下图所示，将右括号用对应的运算符替换，变为 $((5(2(16+*+)(82/-))$ ，最后删除所有左括号得到的后缀表达式为 $5216+*+82/-$ 。



不妨试试将中缀表达式 $a*(b+c)-d$ 转换成后缀表达式。

第一步：进行乘法运算，加括号变为 $(a*(b+c))-d$ 。

第二步：进行减法运算，加括号变为 $((a*(b+c))-d)$ 。

第三步：找出运算符和右括号的对应关系，将右括号用对应的运算符替换变为 $((a(bc+*d-$ 。

$((5+(2*(1+6)))-(8/2))$



第四步：最后删除所有左括号得到的后缀表达式为 $abc+*d-$ 。

3. D。本题考查栈的应用。在栈的众多应用中，括号匹配是一个重要的体现。

4. A。每个元素需要 1 个存储单元，所以每入栈一次 top 加 1，出栈一次 top 减 1。指针 top 的值依次为 1001H, 1002H, 1001H, 1002H, 1001H, 1002H, 1001H, 1002H。

5. C。链栈采用不带头结点的单链表表示时，进栈操作在首部插入一个结点 x (即 $x \rightarrow next = top$)，插入完后需将 top 指向该插入的结点 x 。请思考当链栈存在头结点时的情况。

6. D。这里假设栈顶指针指向的是栈顶元素，所以选 D；而 A 中首先将 top 指针赋给 x ，错误；B 中没有修改 top 指针的值；C 为 top 指针指向栈顶元素的上一个元素时的答案。

7. A。执行前 3 句后，栈 st 内的值为 a, b ，其中 b 为栈顶元素；执行第 4 句后，栈顶元素 b 出栈， x 的值为 b ；执行最后一句，获取栈顶元素的值， x 的值为 a 。

8. C。入栈序列是 $P_1, P_2, \dots, P_n$ 。由于 $P_3=1$ ，即 P_1, P_2, P_3 连续入栈后，第一个出栈元素是 P_3 ，说明 P_1, P_2 已经按序进栈，根据先进后出的特点可知，凡必定在凡之前出栈，而第二个出栈元素是 2，而此时 P_1 不是栈顶元素，因此 P_1 的值不可能是 2。思考：哪些 P_i 可能是 2？

9. C。逐个判断每个选项可能的入栈出栈顺序。对于 A，可能的顺序是 1 入，1 出，2 入，2 出，3 入，3 出，4 入，4 出。对于 B，可能的顺序是 1 入，2 入，3 入，3 出，2 出，4 入，4 出，1 出。对于 D，可能的顺序是 1 入，1 出，2 入，3 入，3 出，2 出，4 入，4 出。C 没有对应的序列。

10. B。后缀表达式中，每个计算符号均直接位于其两个操作数的后面，按照这样的方式逐步根据计算的优先级将每个计算式进行变换，即可得到后缀表达式。

另解：将两个直接操作数用括号括起来，再将操作符提到括号后，最后去掉括号。如下：① $a*(ab+c)-d$ ，提出操作符并去掉括号后，可得后缀表达式为 $abc+*d-$ 。

学完二叉树后，可将表达式画成二叉树的形式，再用后序遍历即可求得后缀表达式。

11. B。利用栈求表达式的值时，可以分别设立运算符栈和运算数栈，其原理不变。选项 B 中 A 入栈，B 入栈，计算得 R1，C 入栈，计算得 R2，D 入栈，计算得 R3，由此得栈深为 2。A、C、D 依次计算得栈深为 4、3、3。因此选 B。

12. B。通常情况下，递归算法在计算机实际执行的过程中包含很多的重复计算，所以效率会低。

13. C。调用函数时，系统会为调用者构造一个由参数表和返回地址组成的活动记录，并将记录压入系统提供的栈中，若被调用函数有局部变量，也要压入栈中。

(二) 综合应用题

1. 假设以 I 和 O 分别表示入栈和出栈操作。若栈的初态和终态均为空，入栈和出栈的操作序列可表示为仅由 I 和 O 组成的序列，则称可以操作的序列为合法序列，否则称为非法序列。

(1) 试指出判别给定序列是否合法的一般规则。

(2) 两个不同的合法序列 (对两个具有同样元素的输入序列) 能否得到相同的输出元素序列? 如能得到，请举例说明。

(3) 写出一个算法，判定所给的操作序列是否合法。若合法，返回 1，否则返回 0 (假定被判定的操作序列已存入一维 char 型数组 ch[] 中，操作序列以 "\0" 为结束符)。

2. 写出下列中缀表达式的后缀形式。

(1) $A*B*C$

(2) $-A+B-C+D$

(3) $C-A*B$

(4) $(A+B)*D+E/(F+A*D)+C$

(5) $(A\&\&B)\text{II}(! (E>F))$

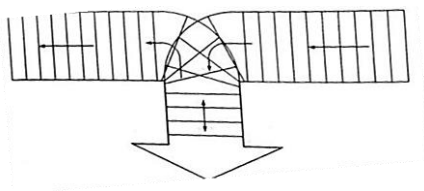
(6) $!(A\&\&(! ((B<C)\text{II}(C>D))))\text{II}(C<E)$

说明：单目运算符操作，如 !A 的后缀表达式为 A!。

3. 设有两个栈 s1、s2 都采用顺序栈方式，并共享一个存储区 $[0, \dots, \text{maxsize}-1]$ ，为了尽量利用空间，减少溢出的可能，可采用栈顶相向、迎面增长的存储方式。试设计 s1、s2 有关入栈和出栈的操作算法。

4. 假设一个算术表达式中包含圆括号、方括号和花括号 3 种类型的括号，编写一个算法未判别表达式中的括号是否配对，以字符 "\0" 作为算术表达式的结束符。

5. 按下图所示铁道进行车厢调度 (注意，两侧铁道均为单向行驶道，火车调度站有一个用于调度的“栈道”)，火车调度站的入口处有 n 节硬座和软座车厢 (分别用 H 和 S 表示) 等待调度，试编写算法，输出对这 n 节车厢进行调度的操作 (即入栈或出栈操作) 序列，以使所有的软座车厢都被调整到硬座车厢之前。



6. 利用一个栈实现以下递归函数的非递归计算;

$$|x| = \begin{cases} 1, & n = 0 \\ 2x, & n = 1 \\ 2xP_{n-1}(x) - 2(n-1)P_{n-2}(x), & n > 1 \end{cases}$$

【答案解析】

1.答: (1) 通常有两条规则, 第一条是给定序列中, I 的个数和 O 的个数相等; 第二条是从给定序列的开始到给定序列中的任一位置, I 的个数要大于或等于 O 的个数。

(2) 可以得到相同的输出元素序列。例如, 输入元素为 A, B, C, 则两个输入序列 A、B、C 和 B、A、C 均可得到输出元素序列 A、B、C。对于输入序列 A、B、C, 我们使用本题约定的 IOIOIO 操作序列; 对于输入序列 B、A、C, 我们使用 IIOOIO 操作序列。

(3) 由 (1) 中分析可以写出以下代码:

```
int judge(char ch[]){
/*判断字符数组 ch[] 中的序列是否是合法序列, 如果是, 返回 1, 否则返回 0*/
int i=0;
int I=0,O=0;          //I 和 O 分别为字符 "I" 和 "O" 的个数
while(ch[i]!='\0'){
    if(ch[i]=='I')
        ++I;
    if(ch[i]=='O')
        ++O;
    if(O>I)
        return 0;      //扫描过程中出现 O 的个数大于 I 的情况, 则一定不合法
    ++i;
}
if(I!=O)
    return 0;          //I 的总数和 O 不相等, 不合法, 返回 0
else
    return 1;          //合法返回 1
}
```

2.解: (1) AB*C*

(2) A-B+C-D+

(3) CAB*-

(4) AB+D*EFAD*+/-C+

(5) AB&&EF>||

(6) ABC<CD>||&&!CE<||

前 5 道题都很简单, 这里只讲第 (6) 题, 如下表。注意按照括号层次, 从内到外的转化过程以及括号消除过程。表中方框内为本步要转换的表达式, 右边为其转换中间结果, 将中间结果置换原被转换表达式并结合新的待转换项即为下一步要转换的表达式。

步数	结果	中间结果
1	!(A&&(!(B<C) C>D)))(C<E)	BC<,CD>,CE<
2	!A&&(!(BC<) C>D)))(CE<)	BC<CD>
3	!A&&(!(BC<CD>))(CE<)	BC<CD> !
4	!(A&&(BC<CD> !))(CE<)	ABC<CD> !&&
5	!(ABC(BC<CD> !))(CE<)	ABC<CD> !&&!
6	(ABC<CD> !&&!)(CE<)	ABC<CD> !&&!CE<
7	ABC<CD> !&&!CE<	

3.解答: 两个栈共享向量空间, 将两个栈的栈底设在向量两端, 初始时, s1 栈顶指针为 -1, s2 栈顶指针为 maxsize。两个栈顶指针相邻时为栈满。两个栈顶相向、迎面增长, 栈顶指针指向栈顶元素。

#define maxsize 100 // 两个栈共享顺序存储空间所能达到的最多元素数, 初始化为 100

#define elemtp int // 假设元素类型为整型

typedef struct {

elemtp stack[maxsize]; // 栈空间

int top[2]; // top 为两个栈顶指针

} stk;

stk s; // s 是如上定义的结构类型变量, 为全局变量

本题的关键在于, 两个栈入栈和退栈时的栈顶指针的计算。s1 栈是通常意义下的栈; 而 s2 栈入栈操作时, 其栈顶指针左移 (减 1), 退栈时, 栈顶指针右移 (加 1)。

此外, 对于所有栈的操作, 都要注意“入栈判满、出栈判空”的检查。

(1) 入栈操作

int push(int i, elemtp x) { /* 入栈操作。i 为栈号, i=0 表示左边的 s1 栈, i=1 表示右边的 s2 栈, x 是入栈元素入栈成功返回 1, 否则返回 0 */

if (i < 0 || i > 1) {
printf("栈号输入不对\n");
exit(0);
}

if (s.top[1] - s.top[0] == 1) {
printf("栈已满\n");
return 0;
}

switch (i) {
case 0:
s.stack[++s.top[0]] = x;
return 1;
break;
case 1:
s.stack[--s.top[1]] = x;
return 1;
}

}

(2) 退栈操作

elemtp pop(int i) { /* 退栈算法。i 代表栈号, i=0 时为 s1 栈, i=1 时为 s2 栈
退栈成功返回退栈元素, 否则返回 -1 */

if (i < 0 || i > 1) {
printf("栈号输入错误\n");
exit(0);
}

switch (i) {
case 0:
if (s.top[0] == -1) {
printf("栈空\n");
return -1;
}
else
return s.stack[s.top[0]--];
break;
case 1:
if (s.top[1] == maxsize) {
printf("栈空\n");
return -1;
}

```

        else
            return s.stack[s.top[1]++];
        break;
    } // switch
}

```

4. 解答：括号匹配是栈的一个典型应用，给出这道题是希望读者好好掌握栈的应用。算法的基本思想是扫描每个字符，遇到花、中、圆的左括号时进栈，遇到花、中、圆的右括号时检查栈顶元素是否为相应的左括号，若是，退栈，否则配对错误。最后栈若不为空也为错误。

```

bool BracketsCheck(char *str) {
    InitStack(S); // 初始化栈
    int i = 0;
    while (str[i] != '\0') {
        switch (str[i]) { // 左括号入栈
            case '(': Push(S, '('); break;
            case '[': Push(S, '['); break;
            case '{': Push(S, '{'); break;
            case ')': // 遇到右括号，检测栈顶
                Pop(S, e);
                if (e != '(') return false;
                break;
            case ']':
                Pop(S, e);
                if (e != '[') return false;
                break;
            case '}':
                Pop(S, e);
                if (e != '{') return false;
                break;
            default:
                break;
        } // switch
        i++;
    } // while
    if (!IsEmpty(S)) {
        printf("括号不匹配\n");
        return false;
    }
    else {
        printf("括号匹配\n");
        return true;
    }
}

```

5. 解答：两侧的铁道均为单向行驶道，且两侧不相通。所有车辆都必须通过“栈道”进行调度。算法的基本设计思想：所有车厢依次前进并逐一检查，若为硬座车厢则入栈，等待最后调度。检查完后，所有的硬座车厢已全部入栈道，车道中的车厢均为软座车厢，此时将栈道的车厢调度出来，调整到软座车厢之后。算法的实现如下：

```

void Train_Arrange(char *train) { // 用字符串 train 表示火车，H 表示硬座，S 表示软座
    char *p = train, *q = train, c;
    stack s;
    InitStack(s); // 初始化栈结构
    while (*p) {

```

```

    if (*p == 'H')
        Push(s, *p);           // 把 H 存入栈中
    else
        *(q++) = *p;           // 把 S 调到前部
    p++;
}
while (!StackEmpty(s)) {
    Pop(s, c);
    *(q++) = c; // 把 H 接在后部
}

```

6. 解答:

算法思想: 设置一个栈用于保存 n 和对应的 $P_n(x)$ 值, 栈中相邻元素的 $P_n(x)$ 有题中关系。然后边出栈边计算 $P_n(x)$, 栈空后该值就计算出来了。算法的实现如下:

```

double p(int n, double x) {
    struct stack {
        int no;           // 保存 n
        double val;       // 保存  $P_n(x)$  值
    } st[MaxSize];       // 定义一个结构体数组作为栈
    int top = -1, i;      // top 为栈 st 的下标变量
    double fv1 = 1, fv2 = 2 * x; //  $n=0, n=1$  时的初值
    for (i = n; i >= 2; i--) {
        top++; // 入栈
        st[top].no = i;
    }
    while (top >= 0) {
        st[top].val = 2 * x * fv2 - 2 * (st[top].no - 1) * fv1;
        fv1 = fv2;
        fv2 = st[top].val;
        top--; // 出栈
    }
    if (n == 0)
        return fv1;
    return fv2;
}

```


4、队列

(一) 选择题

- 下面 () 用到了队列。
A. 括号匹配 B. 迷宫求解 C. 页面替换算法 D. 递归
- 执行 () 操作时, 需要使用队列作为辅助存储空间。
A. 查找散列 (哈希) 表 B. 广度优先搜索图
C. 前序 (根) 遍历二叉树 D. 深度优先搜索图
- 下列说法中, 正确的是 ()。
A. 消除递归不一定需要使用栈
B. 对同一输入序列进行两组不同的合法入栈和出栈组合操作, 所得的输出序列也一定相同
C. 通常使用队列来处理函数或过程调用
D. 队列和栈都是运算受阻的线性表, 只允许在表的两端进行运算
- 循环队列存储在数组 $A[0, \dots, m]$ 中, 则入队时的操作为 ()。
A. $rear=rear+1$ B. $rear=(rear+1)\%(m-1)$
C. $rear=(rear+1)\%m$ D. $rear=(rear+1)\%(m+1)$
- 最大容量为 $maxSize$ 的循环队列, 队尾指针是 $rear$, 队头是 $front$, 则队空的条件是 ()。
A. $(rear+1)\%maxSize==front$ B. $rear==front$
C. $rear+1==front$ D. $(rear-1)\%maxSize==front$
- 元素 A, B, C, D 依次进顺序栈后, 栈顶元素是 (), 栈底元素是 ()。
A. A B. B C. C D. D
- 经过以下栈的操作后, x 的值为 ()。
 $initStack(st); push(st,a); push(st,b); pop(st,x); getTop(st,x);$
A. a B. b C. 1 D. 0
- 经过以下栈的操作后, $isEmpty(st)$ 的返回值为 ()。
 $initStack(st); push(st,a); push(st,b); pop(st,x); pop(st,y);$
A. a B. b C. 1 D. 0
- 设 n 个元素进栈序列是 $1, 2, 3, \dots, n$, 其输出序列是 $P_1, P_2, P_3, \dots, P_n$ 。若 $P_1=3$, 则 P_2 的值 ()。
A. 一定是 2 B. 一定是 1 C. 不可能是 1 D. 以上都不对
- 设循环队列的下标范围是 $0 \sim n-1$, 其头、尾指针分别为 f 和 r , 则其元素个数为 ()。
A. $r-f$ B. $r-f-1$ C. $(r-f)\%n+1$ D. $(r-f+n)\%n$
- 假设一个循环队列 $Q[MaxSize]$ 的队头指针为 $front$, 队尾指针为 $rear$, 队列的最大容量为 $MaxSize$, 除此之外, 该队列再没有其他数据成员, 则判断该队的列满条件是 ()。
A. $Q.front==Q.rear$ B. $Q.front+Q.rear \geq MaxSize$
C. $Q.front==(Q.rear+1)\%MaxSize$ D. $Q.rear=(Q.front+1)\%MaxSize$
- 在一个链队列中, 假设队头指针为 $front$, 队尾指针为 $rear$, x 所指向的元素需要入队, 则需要执行的操作为 ()。
A. $front=x, front=front->next$ B. $x->next=front->next, front=x$
C. $rear->next=x, rear=x$ D. $rear->next=x, x->next=null, rear=x$
- 假设循环单链表表示的队列长度为 n , 队头固定在链表尾, 若只设头指针, 则进队操作的时间复杂度为 ()。
A. $O(n)$ B. $O(1)$ C. $O(n^2)$ D. $O(n\log_2 n)$

14. 若以 1,2,3,4 作为双端队列的输入序列,则既不能由输入受限的双端队列得到,又不能由输出受限的双端队列得到的输出序列是 ()。

- A. 1,2,3,4 B. 4,1,3,2 C. 4,2,3,1 D. 4,2,1,3

【答案解析】

1. C。页面替换算法中的 FIFO 用到了队列。其余的都只用到了栈。
2. B。本题涉及第 5 章和第 6 章的内容,图的广度优先搜索类似于树的层序遍历,都要借助于队列。
3. A。使用栈可以模拟递归的过程,以此来消除递归,但对于单向递归和尾递归而言,可以用迭代的方式来消除递归,A 对;不同的进栈和出栈组合操作,会产生许多不同的输出序列,B 错;通常使用栈来处理函数或过程调用,C 错;队列和栈都是操作受限的线性表,但只有队列允许在表的两端进行运算,而栈只允许在栈顶方向进行操作,D 错。
4. D。本题考查队列的基本操作。入队时队尾变化为 $rear=(rear+1)\%maxSize$,其中 $maxSize$ 是数组元素最大个数,题干中数组最大个数为 $m+1$,因下标从 0 开始。因此答案为 D。
5. B。本题考查循环队列的判空条件。当首尾指针重合时队空。因此本题选 B。
6. D, A。本题考查栈的性质。A, B, C, D 依次进栈,则从栈顶到栈底的元素序列为 D, C, B, A, 因此本题选 D、A。
7. A。本题考查栈的基本操作。执行前 3 句后,栈 st 内的值为 a、b,其中 b 为栈顶元素;执行第 4 句后, x 的值为 b;执行最后一句后, x 的值为 a。因此本题选 A。
8. C。本题考查栈的基本操作。由题意可知,先经过两次进栈操作,后经过两次出栈操作,此时栈必为空,因此判断栈空函数 $isEmpty()$ 的返回值为 1。故本题选 C。
9. C。本题考查根据入栈序列对出栈序列的判断。当 $P1=3$ 时,说明 1,2,3 都已进栈。立即出栈 3,然后可能出栈的为 2;也可能 4 或者后面的元素进栈,再出栈。因此, $P2$ 可能是 2,也可能是 4, ..., n 中的任何一个,但一定不是 1。所以答案为 C。
10. D。本题考查循环队列的性质。要计算当前队列内的元素个数,可以分两种情况讨论:①当 r 大于 f 时,栈内元素为 $r-f$;②当 $r < f$ 时,栈内元素为 $n-(f-r)$ (其中, $f-r$ 为数组内空位置的个数,则 $n-(f-r)$ 为数组内元素的个数)。将①②合并得元素个数为 $(r-f+n)\%n$,因此本题选 D。
11. C。既然不能附加任何其他数据成员,只能采用牺牲一个存储单元的方法来区分是队空还是队满,约定以“队列头指针在队尾指针的下一位置作为队满的标志”,因此选 C。选项 A 是判断队列是否空的条件,选项 B 和 D 都是干扰项。
12. D。插入操作时,先将结点 x 插入到链表尾部,再让 $rear$ 指向这个结点 x 。C 的做法不够严密,因为是队尾,所以队尾 $x->next$ 必须置为空。
13. A。依题意,进队操作是在队尾进行,即链表表头。题中已明确说明链表只设头指针,也没有头结点和尾指针,进队后,循环单链表必须保持循环的性质,在只带头指针的循环单链表中寻找表尾结点的时间复杂度为 $O(n)$,故进队的时间复杂度为 $O(n)$ 。
14. C。使用排除法。先看可由输入受限的双端队列产生的序列:设右端输入受限,1,2,3,4 依次左入,则依次左出可得 4,3,2,1,排除 A;右出、左出、右出、右出可得到 4,1,3,2,排除 B;再看可由输出受限的双端队列产生的序列:设右端输出受限,1,2,3,4,依次左入、左入、右入、左入,依次左出可得到 4,2,1,3,排除 D。

(二) 综合应用题

1. 假设以带头结点的循环链表表示队列, 并且只设一个指针指向队尾结点, 但不设头指针, 请写出相应的入队列和出队列算法。

2. 编写一个算法, 将一个非负的十进制整数 N 转换为一个二进制数。

3. 试编写一个算法, 检查一个程序中的花括号、方括号和圆括号是否配对, 若全部配对, 则返回 1, 否则返回 0。对于程序中出现的一对单引号或双引号内的字符不进行括号配对检查。39 为单引号的 ASCII 值, 34 为双引号的 ASCII 值, 单引号和双引号如果出现 则必成对出现。

假设 `stack` 是已经定义的顺序栈结构体。可以直接调用的元素进栈/出栈、取栈顶元素、判断栈空的函数定义如下:

```
void push (stack &s, char ch);
void pop(stack &s, char &ch);
void getTop(stack S, char &ch);
int is Empty (stack S);           //若栈 s 空, 则返回 1, 否则返回 0
```

4. 利用两个栈 S_1, S_2 来模拟一个队列, 已知栈的 4 个运算定义如下:

```
Push(S,x);           //元素 x 入栈 s
Pop(S,x);             //S 出栈并将出栈的值赋给 x
StackEmpty(S);        //判断栈是否为空
StackOverflow(S);     //判断栈是否满
```

如何利用栈的运算来实现该队列的 3 个运算 (形参由读者根据要求自己设计)?

```
Enqueue;             //将元素 x 入队
Dequeue;              //出队, 并将出队元素存储在 x 中
QueueEmpty;          //判断队列是否为空
```

5. 某汽车轮渡口, 过江渡船每次能载 10 辆车过江, 过江车辆分为客车类和货车类, 上渡船有如下规定: 同类车先到先上船; 客车先于货车上船, 且每上 4 辆客车, 才允许放上 1 辆货车; 若等待客车不足 4 辆, 则以货车代替; 若无货车等待, 允许客车都上船。试设计一个算法模拟渡口管理。

【答案解析】

1. 分析: 本题是链队基本操作的扩展, 知道尾指针后, 要实现元素入队, 则直接用链表的插入操作即可。要实现出队操作, 则需要根据尾指针找出头结点和开始结点, 然后进行删除。要注意的是, 尾指针应始终指向终端结点, 并且当删除结点后队列为空时, 必须特殊处理。具体算法如下。

(1) 入队:

```
void enQueue(LNode *&rear, int x) {
/*rear 是带头结点的循环链队的尾指针, 本算法将元素 x 插入到队尾*/
    LNode *s = (LNode *) malloc (sizeof (LNode)); //申请结点空间
    s->data=x;
    s->next=rear->next;           //将 s 结点链入队尾
    rear->next=s;
    rear=s;                       //rear 指向新队尾
}
```

(2) 出队:

```
int deQueue(LNode *&rear, int &x) {
/*rear 是带头结点的循环链队的尾指针, x 接收出队元素, 操作成功返回 1, 否则返回 0*/
```

```

LNode *s;
if(rear->next == rear) return 0;
else{
    s=rear->next->next;           //s 指向开始结点
    rear->next->next=s->next ;     //队头元素出队
    x=s->data;
    if (s==rear)                 //如果元素出队后队列为空，则需要特殊处理
        rear=rear->next;         //将 rear 指向头结点
    free(s);                     //释放队结点空间
    return 1;                    //操作成功，返回 1
}
}

```

2.分析:

可以利用栈解决数制转换问题。例如, $(49)_{10} = 1 \times 2^5 + 1 \times 2^4 + 1 \times 2^0 = (110001)_2$, 其转换规则是

$$N = \sum_{i=0}^{\lfloor \log_2 N \rfloor} 2^i b_i \quad 0 \leq b_i \leq 1$$

式中, b_i 表示二进制数的第 i 位上的数字。这样, 十进制数 N 可以用长度为 $\lfloor \log_2 N \rfloor + 1$ 位的二进制数表示成 $b_{\lfloor \log_2 N \rfloor} \dots b_2 b_1 b_0$ 。若令 $j = \lfloor \log_2 N \rfloor$, 则有

$$\begin{aligned}
 N &= 2^j b_j + 2^{j-1} b_{j-1} + \dots + 2^1 b_1 + 2^0 b_0 \\
 &= (2^{j-1} b_j + 2^{j-2} b_{j-1} + \dots + b_1) \times 2 + b_0 \\
 &= (N/2) \times 2 + N \% 2 \quad ("/" \text{表示整除运算})
 \end{aligned}$$

因此, 可以先通过 $N \% 2$ 求出 b_0 , 然后令 $N = N/2$, 再对新的 N 做除 2 求模运算, 可求出 $b_1, \dots$, 如此重复, 直到某个 N 等于零结束。这个计算过程是从低位到高位逐个进行的, 但输出过程是从高位到低位逐个打印的, 因此需要利用栈来实现。实现代码如下:

```

int BaseTrans (int N) {
    int i, result=0;
    int stack[maxSize], top = -1;
    while(N!=0) { //定义并初始化栈, 其中 maxSize 是已定义的常量, 其大小足够处理本题数据
        i=N%2;
        N=N/2;
        Stack[++top] = i;
    }
    while (top != -1) {
        i = stack[top];
        --top;
        result = result*10 + i;
    }
    return result;
}

```

3.分析: 在算法中, 扫描程序中的每一个字符, 当扫描到每个左花括号、左方括号、左圆括号时, 令其进栈; 当扫描到右花括号、右方括号、右圆括号时, 则检查栈顶是否为相应的左括号, 若是则做退栈处理, 若不是则表明出现了语法错误, 返回 0。当扫描到程序文件结尾后, 若栈为空, 则表明没有发现括号配对错误, 返回 1; 否则表明栈中还有未配对的括号, 返回 0。另外, 对于一对单引号或双引号内的字符不进行括号配对检查。

由此可以写出以下代码:

```

int brackets Check (char f[]) { //对由字符数组 f 所存字符串中的文本进行括号配对检查
    Stack S; char ch;           //定义一个栈
}

```



```

char* p=f;
while (*p!='\0') { //顺序扫描串中的每一个字符
    if (*p=='39') {
        ++p; //跳过第一个单引号
        while (*p!=39) //39 为单引号的 ASCII 值
            ++p;
        ++p; //跳过最后一个单引号
    }
    else if (*p=='34') { //双引号内的字符不参与配对比较
        ++p; //跳过第一个双引号
        while (*p!=34) //34 为双引号的 ASCII 值
            ++p;
        ++p; //跳过最后一个双引号
    }
    else{
        switch(*p) {
            case '{':
            case '[':
                case '(': push (S, *p); //出现左括号: "{“、“ [”和“(“进栈
                break;
            case '}': getTop (S, ch);
                if (ch=='{') //栈顶的左花括号出栈
                    pop(S,ch);
                else
                    return 0;
                break;
            case ']': getTop ( S, ch );
                if (ch=='[') //栈顶的左方括号出栈
                    pop(S,ch);
                else
                    return 0;
                break;
            case ')': getTop(S,ch);
                if (ch=='(') //栈顶的左圆括号出栈
                    pop(S,ch);
                else
                    return 0;
        }
        ++p;
    }
}
if (isEmpty (s))
    return 1;
else
    return 0;
}

```

4. 解答: 利用两个栈 S1 和 S2 来模拟一个队列, 当需要向队列中插入一个元素 时, 用 S1 来存放已输入的元素, 即 S1 执行入栈操作。当需要出队时, 则对 S2 执行出栈操作。由于从栈中取出元素的顺序是原顺序的逆序, 所以必须先将 S1 中的所有元素全部出栈并入栈到 S2 中, 再在 S2 中执行出栈操作, 即可实现出队操作, 而在执行此操作前必须判断 S2 是否为空, 否则会导致顺序混乱。当栈 S1 和 S2 都为空时队列为空。

总结如下:

(1) 对 S2 的出栈操作做出队, 若 S2 为空, 则先将 S1 中的所有元素送入 S2。

(2) 对 S1 的入栈操作作入队, 若 S1 满, 必须先保证 S2 为空, 才能将 S1 中的元素全部插入 S2 中。

入队算法:

```
int EnQueue(Stack &S1, Stack &S2, ElemType e) {
    if (!StackOverflow(S1)) {
        Push(S1, e);
        return 1;
    }
    if (StackOverflow(S1) && !StackEmpty(S2)) {
        printf("队列满");
        return 0;
    }
    if (StackOverflow(S1) && StackEmpty(S2)) {
        while (!StackEmpty(S1)) {
            Pop(S1, x);
            Push(S2, x);
        }
    }
    Push(S1, e);
    return 1;
}
```

出队算法:

```
void DeQueue(Stack &S1, Stack &S2, ElemType &x) {
    if (!StackEmpty(S2))
        Pop(S2, x);
    else if (StackEmpty(S1))
        printf("队列为空");
    else {
        while (!StackEmpty(S1)) {
            Pop(S1, x);
            Push(S2, x);
        }
        Pop(S2, x);
    }
}
```

判断队列为空的算法:

```
int QueueEmpty(Stack S1, Stack S2) {
    if (StackEmpty(S1) && StackEmpty(S2))
        return 1;
    else
        return 0;
}
```

5. 算法思想: 假设数组 q 的最大下标为 10, 恰好是每次载渡的最大量。假设客车的队列为 q1, 货车的队列为 q2。若 q1 充足, 则每取 4 个 q1 元素后再取一个 q2 元素, 直到 q 的长度为 10。若 q1 不充足, 则直接用 q2 补齐。算法的实现如下:

```
Queue q; // 定义一个队列 q, 用于过江渡船载渡队列
Queue q1; // 定义一个队列 q1, 用于客车队列
Queue q2; // 定义一个队列 q2, 用于货车队列
void manager() {
    int i = 0, j = 0; // j 表示渡船上的总车辆数
    while (j < 10) { // 当渡船上的总车辆数小于 10 时, 继续循环
        if (!QueueEmpty(q1) && i < 4) { // 如果客车队列不为空, 并且客车数量少于 4 辆
```

```

        DeQueue(q1, x);           // 从客车队列中出队一辆车
        EnQueue(q, x);           // 将客车 x 加入渡船队列 q
        i++;                     // 客车数量加一
        j++;
    }
    else if (i == 4 && !QueueEmpty(q2)) { // 如果客车已满 4 辆且货车队列不为空
        DeQueue(q2, x);           // 从货车队列中出队一辆车
        EnQueue(q, x);           // 将货车 x 加入渡船队列 q
        j++;                     // 渡船上的总车辆数加一
        i = 0;                   // 每上一辆货车, i 重新计数
    }
    else {                       // 其他情况 (客车队列空或货车队列空)
        // 当渡船上的总车辆数小于 10 且客车数量少于 4 辆且货车队列不为空时
        while (j < 10 && i < 4 && !QueueEmpty(q2)) {
            DeQueue(q2, x);       // 从货车队列中出队一辆车
            EnQueue(q, x);       // 将货车 x 加入渡船队列 q
            i++;                 // 货车数量加一
            j++;                 // 渡船上的总车辆数加一
        }
        i = 0;                   // 重置客车数量计数器
    }
    if (QueueEmpty(q1) && QueueEmpty(q2)) // 如果客车队列和货车队列都为空
        j = 11;                 // 表示渡船上的车辆总数达到 10 辆, 结束循环
    }
}

```

5、树

5.1 综合练习

(一) 选择题

- 若二叉树采用二叉链表存储结构, 要交换其所有分支结点左右子树的位置, 利用 () 遍历方法最合适。
A. 前序 B. 中序 C. 后序 D. 按层次
- 一棵高度为 h 的完全二叉树至少有 () 个结点。
A. 2^{h-1} B. $2^{h-1}-1$ C. 2^{h-1} D. 2^h
- 一棵高度为 h 的完全二叉树至多有 () 个结点。
A. 2^{h-1} B. $2^{h-1}-1$ C. 2^{h-1} D. 2^h
- 一棵具有 1025 个结点的二叉树高度 h 为 ()。
A. 11 B. 10 C. 11~1025 D. 12~1024
- 对一棵满二叉树, 共有 n 个结点和 m 个叶子结点, 高度为 h , 则 ()。
A. $n=h+m$ B. $h+m=2h$ C. $m=h-1$ D. $n=2^{h-1}$
- 一棵二叉树的先序遍历序列为 A, B, C, D, E, F, 中序遍历序列为 C, B, A, E, D, F, 则后序遍历序列为 ()。
A. C, B, E, F, D, A B. F, E, D, C, B, A C. C, B, E, D, F, A D. 不确定
- 一棵二叉树的后序遍历序列为 D, A, B, E, C, 中序遍历序列为 D, E, B, A, C, 则先序遍历序列为 ()。
A. A, C, B, E, D B. D, E, C, B, A C. D, E, A, B, C D. C, E, D, B, A
- 一棵二叉树的先序遍历序列为 E, F, H, I, G, J, K, 中序遍历序列为 H, F, I, E, J, K, G 则该二叉树根结点的右孩子为 ()。
A. E B. F C. G D. H
- 已知一棵二叉树的层次序列为 ABCDEF, 中序序列为 BADCFE, 则先序序列为 ()。
A. ACBEDF B. ABCDEF C. BDFECA D. FCEDBA
- 已知一算术表达式的中缀形式为 $A+B \times C-D/E$, 后缀形式为 $ABC \times + DE / -$, 其前缀形式为 ()。
A. $-A+B \times C/DE$ B. $-A+B \times CD/E$ C. $- \times ABC/DE$ D. $- \times A \times BC/DE$
- 判断线索二叉树中 p 结点有右孩子结点的条件是 ()。
A. $p != \text{NULL}$ B. $p \rightarrow \text{rchild} != \text{NULL}$ C. $p \rightarrow \text{rtag} == 0$ D. $p \rightarrow \text{rtag} == 1$
- 某二叉树结点的中序序列为 BDAECF, 后序序列为 D BEFCA, 则该二叉树对应的森林包括 () 棵树。
A. 1 B. 2 C. 3 D. 4
- 下列编码中, () 不是前缀码。
A. {00, 01, 10, 11} B. {0, 1, 00, 11} C. {0, 10, 110, 111} D. {10, 110, 1110, 1111}
- 若度为 m 的哈夫曼树中, 叶子结点个数为 n , 则非叶子结点的个数为 ()。
A. $n-1$ B. $\lfloor n/m \rfloor - 1$ C. $\lceil (n-1)/(m-1) \rceil$ D. $\lceil n/(m-1) \rceil - 1$
- 根据使用频率为 5 个字符设计的赫夫曼编码, 不可能的是 ()。
A. 111, 110, 10, 01, 00 B. 000, 001, 010, 011, 1
C. 100, 11, 10, 1, 0 D. 001, 000, 01, 11, 10
- 假设赫夫曼二叉树中只有度为 0 或 2 的结点, 根据使用频率为 5 个字符设计的赫夫曼编码, 不可能的是 ()。

- A. 000, 001, 010, 011, 1 B. 0000, 0001, 001, 01, 1
C. 000, 001, 01, 10, 11 D. 00, 100, 101, 110, 111

【答案解析】

1. C。本题考查二叉树遍历的应用。要实现交换所有分支结点左右子树的位置，可以递归地进行如下操作：先递归交换左子树中的所有分支结点，再递归交换右子树中的所有分支结点，最后对根结点交换其所有子树位置。这对应了先遍历左子树，再遍历右子树，最后访问根结点的后序遍历方式。因此本题选 C。

2. C。本题考查完全二叉树的性质。高度为 h 的完全二叉树，结点最少时即为最底层只有 1 结点的情况，因此只需求前 $h-1$ 层的结点数，然后对结果加 1 即可。前 $h-1$ 层的结点数为 $2^{h-1}-1$ 个，因此高度为 h 的完全二叉树，结点最少为 2^{h-1} 个。

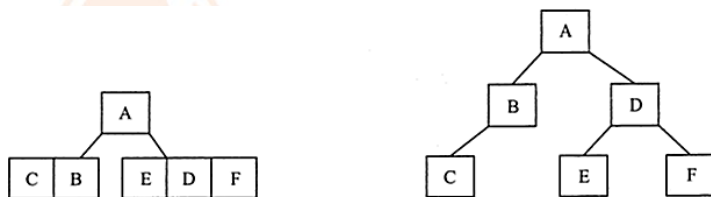
3. A。本题考查完全二叉树的性质。高度为 h 的完全二叉树，结点最多时，其结点数等于同高度的满二叉树的结点数，因此高度为 h 的完全二叉树结点最多为 2^h-1 个。

4. C。本题考查二叉树的性质。当每层只有一个结点时，二叉树高度最高，为 1025；当每层结点数与同高度的完全二叉树对应层的结点数相同时，二叉树高度最低。完全二叉树结点数与高度的关系为 $h = \lfloor \log_2(n+1) \rfloor + 1 = \lfloor \log_2(1025+1) \rfloor + 1 = 11$ 。因此本题选 C。

5. D。本题考查完全二叉树的性质。对于高度为 h 的满二叉树， $n=2^0+2^1+2^2+\dots+2^{h-1}=2^h-1$ ， $m=2^{h-1}$ 。因此本题选 D。

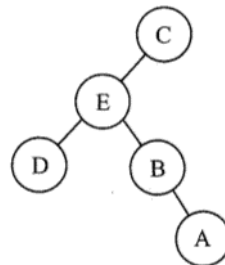
6. A。本题考查根据遍历序列确定二叉树。根据题目中的两个遍历序列确定二叉树的过程如下：

1) 先序序列第一个结点即为整棵树的根结点。在中序序列中，根结点左边为左子树上的结点，右边为右子树上的结点。由此得到如左下图所示的结果。



根据图可知，后序遍历序列为 C, B, E, F, D, A，因此本题选 A。

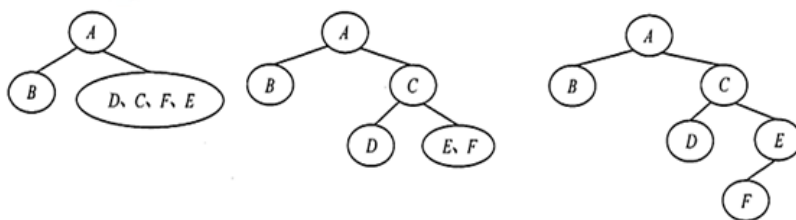
7. D。本题考查根据遍历序列确定二叉树。后序序列中的最后一个结点为整棵树的根结点，根据中序序列判断左、右子树中结点的情况。最终做出的二叉树如右图所示。



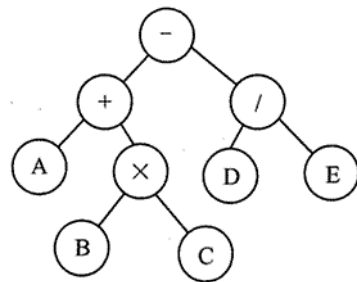
由图可知，先序遍历序列为 C, E, D, B, A，因此本题选 D。

8. C。本题考查根据遍历序列确定二叉树。

9. B。可构造出二叉树如下图所示。因此，先序序列为 ABCDEF。



10. D。本题考查由二叉树的遍历序列确定二叉树。平时用的中缀式其实是一棵以运算符为双分支结点,以操作数为叶子结点的二叉树按照中序遍历所得的序列。对应的后缀式是这个二叉树按照后序遍历所得的序列,相应的前缀式即为其先序遍历所得的序列。根据两序列做出相应的二叉树,对其进行先序遍历,就可得到其前缀式的形式。所做的二叉树如下图所示。



对这棵二叉树进行先序遍历得 $- + A \times B C / D E$, 因此本题选 D。

说明: 之前讲过由中缀式不一定能得出唯一的后缀式或唯一的前缀式, 原因就在于只知道中缀式不能唯一地确定一棵二叉树。考生在学了本章后应能通过二叉树来深入理解第 3 章中讲过的中缀式与前缀式或后缀式的转换问题。

11. C。本题考查线索二叉树的定义。

在线索二叉树中:

1) 如果 lTag 等于 0, 则表示 lchild 为指针, 指向结点的左孩子; 如果 lTag 等于 1, 则表示 lchild 为线索, 指向结点的直接前驱。

2) 如果 rTag 等于 0, 则表示 rchild 为指针, 指向结点的右孩子; 如果 rTag 等于 1, 则表示 rchild 为线索, 指向结点的直接后继。

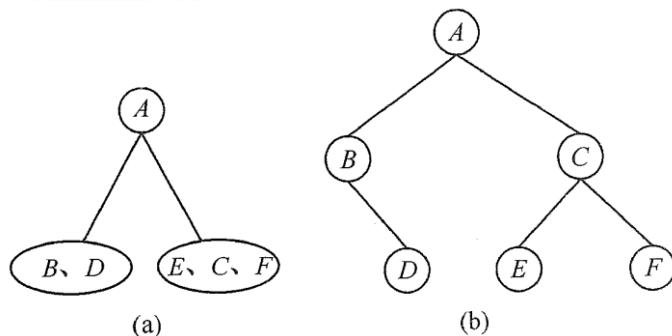
因此本题选 C。

12. C。根据二叉树的前序遍历序列和中序遍历序列可以唯一确定一棵二叉树。

根据后序遍历序列, A 是二叉树的根结点。根据中序遍历序列, 二叉树的形态一定如右图中的(a)所示。

对于 A 的左子树, 由后序遍历序列可知, 因为 B 比 D 后被访问, 因此 B 必为 D 的父结点, 又由中序遍历序列可知, D 是 B 的右儿子。对于 A 的右子树, 同理可确定结点 E、C、F 的关系。此二叉树的形态如右图中的(b)所示。

再根据二叉树与森林的对应关系, 森林中树的棵数即为其对应二叉树中根结点 A 的兄弟数, 可知此森林中有 3 棵树, 根结点分别为 A, C 和 F。



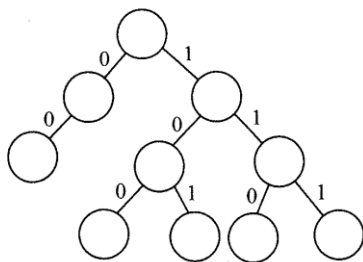
13. B。若没有一个编码是另一个编码的前缀, 则称这样的编码为前缀编码。B 选项中, 0 是 00 的前缀, 1 是 11 的前缀。

14. C。一棵度为 m 的哈夫曼树应只有度为 0 和 m 的结点, 设度为 m 的结点有 n_m 个, 度为 0 的结点有 n_0 个, 又设结点总数为 n , $n = n_0 + n_m$ 。因有 n 个结点的哈夫曼树有 $n-1$ 条分支, 则 $mn_m = n-1 = n_m + n_0 - 1$, 整理得 $(m-1)n_m = n_0 - 1$, $n_m = (n-1)/(m-1)$ 。

15. C。本题考查赫夫曼二叉树的应用。赫夫曼编码为前缀码, 即编码中任何一个序列都不是另一个序列的前缀。选项 C 中, 10 是 100 的前缀, 不是赫夫曼编码。

16. D。本题考查赫夫曼树的性质。赫夫曼树中只有度为 0 或 2 的结点, 由选项 D 可以做出

如下图所示的二叉树。



由赫夫曼树的性质知道，树中不含单分支结点，因此 D 选项不可能。

(二) 综合应用题

1. 已知一棵度为 m 的树中，有 n_1 个度为 1 的结点，有 n_2 个度为 2 的结点……有 n_m 个度为 m 的结点，问该树有多少个叶子结点？

2. 一棵高度为 h 的满 m 叉树有如下性质：根结点所在层次为第 1 层，第 h 层上的结点都是叶结点，其余各层上每个结点都有 m 棵非空子树，若按层次自顶向下，同一层自左向右，顺序从 1 开始对全部结点进行编号，试问：

- 1) 各层的结点个数是多少？
- 2) 编号为 i 的结点的双亲结点（若存在）的编号是多少？
- 3) 编号为 i 的结点的第 K 个孩子结点（若存在）的编号是多少？
- 4) 编号为 i 的结点有右兄弟的条件是什么？其右兄弟结点的编号是多少？

3. 已知某二叉树的先序序列和中序序列分别为 ABDEHCFIMGJKL 和 DBHEAIFCGKLJ，请画出这棵二叉树，并画出二叉树对应的森林。

4. 假设某密文仅有 8 个字母 C1、C2、C3、C4、C5、C6、C7、C8 组成，各个字母在电文中出现的频率分别为 5，25，3，6，10，11，36，4，试回答以下问题：

- (1) 构造出相对应的哈夫曼树。
- (2) 写出该 8 个字母的哈夫曼编码。
- (3) 试分别计算该哈夫曼树的带权路径长度和平均带权路径长度。

【答案解析】

1. 解答：根据“树中所有结点的度数加 1 等于结点数”的结论，有：

$$n = \sum_{i=0}^m i n_i = n_1 + 2n_2 + 3n_3 + \cdots + mn_m + 1$$

又有 $n = n_0 + n_1 + n_2 + \cdots + n_m$ ，所以

$$\begin{aligned} n_0 &= (n_1 + 2n_2 + 3n_3 + \cdots + mn_m + 1) - (n_1 + n_2 + \cdots + n_m) \\ &= n_2 + 2n_3 + \cdots + (m-1)n_m + 1 \\ &= 1 + \sum_{i=2}^m (i-1)n_i \end{aligned}$$

注意：综合以上几题，常用于求解树结点与度之间关系的有：

- ① 总结点数 = $n_0 + n_1 + n_2 + \cdots + n_m$
- ② 总分支数 = $n_1 + 2n_2 + \cdots + mn_m$ （度为 m 的结点引出 m 条分支）。
- ③ 总结点数 = 总分支数 + 1。

这个性质经常在选择题中出现, 读者对于以上关系应当熟练掌握并灵活应用。

2. 1) 第 1 层有 $m^0=1$ 个结点, 第 2 层有 m^1 个结点, 第 3 层有 m^2 个结点……一般地, 第 i 层有 m^{i-1} 个结点($1 \leq i \leq h$)。

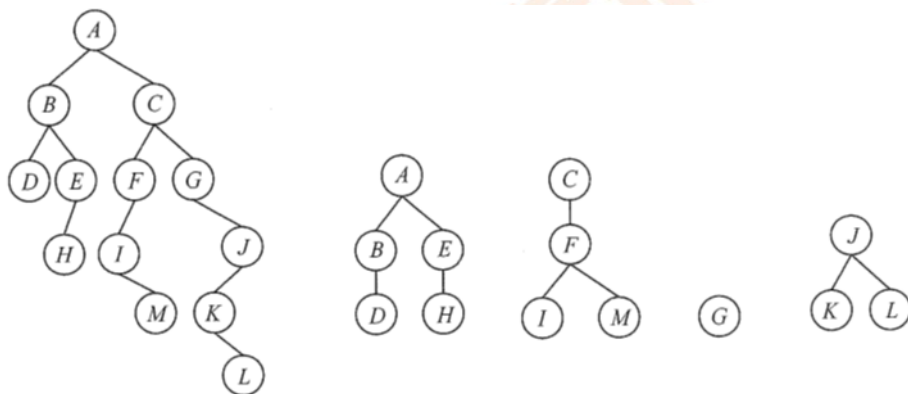
2) 在 m 叉树的情形下, 结点 i 的第 1 个子女编号为 $j = (i-1) \times m + 2$, 反过来, 结点 i 的双亲的编号是 $\lfloor (i-2)/m \rfloor + 1$, 根结点没有双亲, 所以要求 $i > 1$ 。

3) 因为结点 i 的第 1 个子女编号为 $(i-1)m + 2$, 若设该结点子女的序号为 $k = 1, 2, \dots, m$, 则第 k 个子女结点的编号为 $(i-1)m + k + 1$ ($1 \leq k \leq m$)。

4) 结点 i 不是其双亲的第 m 个子女时才有右兄弟。设其双亲编号为 j , 可得 $j = \lfloor (i + m - 2)/m \rfloor$, 结点 j 的第 m 个子女的编号为 $(j-1)m + m + 1 = jm + 1 = \lfloor (i + m - 2)/m \rfloor \times m + 1$, 所以当结点的编号 $i \leq \lfloor (i + m - 2)/m \rfloor \times m + 1$ 时才有右兄弟, 右兄弟的编号为 $i + 1$ 。或者, 对于任一双亲结点 j , 其第 m 个子女结点的编号是 $jm + 1$, 故若不为第 m 的子女结点, 则 $(i-1) \% m \neq 0$ 。

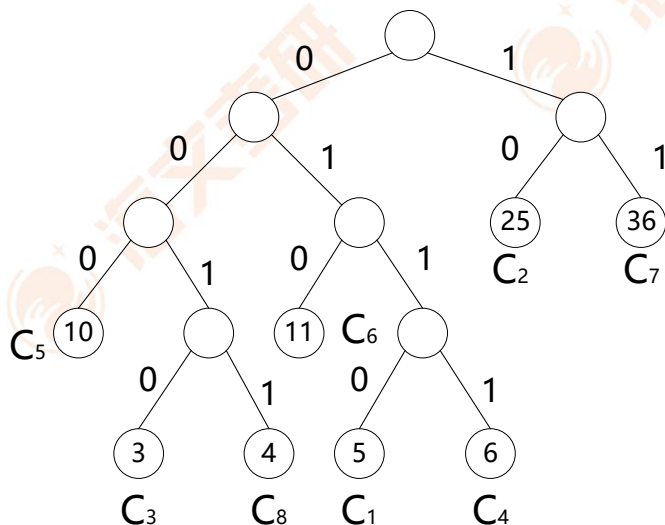
3. 知道二叉树的先序和中序遍历后, 可以唯一确定这棵树的结构。然后把二叉树转换到树和森林的方式是, 若结点 x 是双亲 y 的左孩子, 则把 x 的右孩子、右孩子的右孩子……都与 y 用连线连起来, 最后去掉所有双亲到右孩子的连线。

最后得到的二叉树及对应的森林如下图所示:



4. 解析: 此题考查哈夫曼树的构造过程;

(1) 根据构造哈夫曼树的算法可知 (详细过程省略), 构造出的哈夫曼树如下图所示:



(2) 由构造出的哈夫曼树可知, 8 个字母的编码如下:

C1: 0110; C2: 10; C3: 0010; C4: 0111; C5: 000; C6: 010; C7: 11; C8: 0011

(3) 由公式可知, 带权路径长度=各个字母的使用频率 X 该字母的码长。

带权路径长度 = $5*4 + 25*2 + 3*4 + 6*4 + 10*3 + 11*3 + 36*2 + 4*4 = 257$

平均带权路径长度= $257/8=32.125$

注意: 哈夫曼树的带权路径长度是唯一的, 但是哈夫曼树的形态有可能是不唯一的。

5.2 算法专题

5.2.1 二叉树的遍历专题

1. 假设满二叉树 b 的先序遍历序列已经存在于数组中 (在解题过程中, 此数组名称可自定义, 长度为 n), 设计一个算法将其转换为后序遍历序列。

2. 编写后序遍历二叉树的非递归算法。

3. 试给出二叉树的自下而上、从右到左的层次遍历算法。

4. 假设二叉树采用二叉链存储结构存储, 设计一个算法, 求先序遍历序列中第 k ($1 \leq k \leq$ 二叉树中结点个数) 个结点的值。

5. 设有一棵满二叉树 (所有结点值均不同), 已知其先序序列为 pre , 设计一个算法求其后序序列 $post$ 。

6. 二叉树的双序遍历是指: 对于二叉树的每一个结点来说, 先访问这个结点, 再按双序遍历它的左子树, 然后再一次访问这个结点, 接下来按双序遍历它的右子树。试写出执行这种双序遍历的算法。

【答案解析】

1. 分析: 本题已知先序遍历序列, 则序列中第一个元素即为根结点。将除去第一个元素之外的元素序列分成前后相等长度的两半, 前半为左子树上的结点, 后半为右子树上的结点。只需将根结点移动到整个序列的末尾, 然后分别递归地去处理序列的前一半和后一半即可。

假设原序列在 $pre[]$ 数组内, 转化后的序列存在 $post[]$ 数组内。本题算法如下:

```
void change(char pre [], int L1, int R1, char post [], int L2, int R2) {
    if (L1 <= R1) {
        post[R2] = pre[L1]; // 将 pre [] 中的第一个元素放在 post [] 的末尾
        /* 递归地处理 pre [] 中的前半序列, 将其存在 post [] 数组中对应的前一半位置 */
        change(pre, L1 + 1, (L1 + 1 + R1) / 2, post, L2, (L2 + R2 - 1) / 2);
        /* 递归地处理 pre [] 中的后半序列, 将其存在 post [] 数组中对应的后一半位置 */
        change(pre, (L1 + 1 + R1) / 2 + 1, R1, post, (L2 + R2 - 1) / 2 + 1, R2 - 1);
    }
}
```

2. 算法思想: 后序非递归遍历二叉树是先访问左子树, 再访问右子树, 最后访问根结点。结合图来分析: ①沿着根的左孩子, 依次入栈, 直到左孩子为空。此时栈内元素依次为 $A B D$ 。②读栈顶元素: 若其右孩子不空且未被访问过, 将右子树转执行①; 否则, 栈顶元素出栈并访问。栈顶 D 的右孩子为空, 出栈并访问, 它是后序序列的第一个结点; 栈顶 B 的右孩子不空且未被访问过, E 入

栈, 栈顶 E 的左右孩子均为空, 出栈并访问; 栈顶 B 的右孩子不空但已被访问, B 出栈并访问; 栈顶 A 的右孩子不空且未被访问过, C 入栈, 栈顶 C 的左右孩子均为空, 出栈并访问; 栈顶 A 的右孩子不空但已被访问, A 出栈并访问。由此得到后序序列 DEBC A。

在上述思想的第②步中, 必须分清返回时是从左子树返回的还是从右子树返回的, 因此设定一个辅助指针 r, 指向最近访问过的结点。也可在结点中增加一个标志域, 记录是否已被访问。

```
void PostOrder(BiTree T) {
    InitStack(S);           // 初始化栈 S
    p = T;                  // 设置当前节点指针 p 为根节点 T
    r = NULL;               // 初始化记录最近访问过的节点指针 r 为 NULL
    while (p || !IsEmpty(S)) { // 当 p 不为空或栈不为空时循环
        if (p) {
            push(S, p);      // 将当前节点压入栈中
            p = p->lchild;    // 移动到左子节点
        }
        else {
            GetTop(S, p);    // 获取栈顶元素 (但不弹出)
            if (p->rchild && p->rchild != r) // 如果右子节点存在且未被访问过
                p = p->rchild; // 移动到右子节点
            else {
                pop(S, p);    // 弹出栈顶元素
                visit(p->data); // 访问该节点的数据
                r = p;        // 记录最近访问过的节点
                p = NULL;     // 重置 p 指针
            }
        }
    }
}
```

注意: 每次出栈访问完一个结点就相当于遍历完以该结点为根的子树, 需将 p 置 NULL。

3. 解答: 一般的二叉树层次遍历是自上而下、从左到右, 这里的遍历顺序恰好相反。算法思想: 利用原有的层次遍历算法, 出队的同时将各结点指针入栈, 在所有结点入栈后再从栈顶开始依次访问即为所求的算法。具体实现如下:

- 1) 把根结点入队列。
- 2) 把一个元素出队列, 遍历这个元素。
- 3) 依次把这个元素的左孩子、右孩子入队列。
- 4) 若队列不空, 则跳到 2), 否则结束。

算法实现如下:

```
void InvertLevel(BiTree bt) {
    Stack s;
    Queue Q;
    if (bt != NULL) {
        InitStack(s); // 栈初始化, 栈中存放二叉树结点的指针
        InitQueue(Q); // 队列初始化, 队列中存放二叉树结点的指针
        EnQueue(Q, bt); // 将根节点入队
        while (IsEmpty(Q) == FALSE) { // 从上而下层次遍历
            DeQueue(Q, p); // 出队
            Push(s, p); // 入栈
            if (p->lchild) // 若左子女不空, 则入队列
                EnQueue(Q, p->lchild);
            if (p->rchild) // 若右子女不空, 则入队列
                EnQueue(Q, p->rchild);
        }
    }
}
```

```

        EnQueue(Q, p->rchild);
    }
    while (IsEmpty(Q) == FALSE) {           // 自下而上、从右到左的层次遍历
        Pop(s, p);
        visit(p->data);                     // 访问该结点的数据
    }                                       // 自下而上、从右到左的层次遍历
}
}

```

4. 解答：设置一个全局变量 i 记录已访问过的结点的序号，其初值是根结点在先序序列中的序号，即 1。当二叉树 b 为空时返回特殊字符 '#'，当 $i=k$ 时，表示找到了满足条件的结点，返回 $b->data$ ；当 $i \neq k$ 时，递归地在左子树中查找，若找到则返回该值，否则继续递归地在右子树中查找，并返回其结果。算法实现如下：

```

int i = 1; // 遍历序号的全局变量
ElemType PreNode(BiTree b, int k) {        // 本算法查找二叉树先序遍历序列中第 k 个结点的值
    if (b == NULL)
        return '#'; // 空结点，则返回特殊字符
    if (i == k)
        return b->data; // 相等，则当前结点即为第 k 个结点
    i++; // 下一个结点
    ch = PreNode(b->lchild, k); // 左子树中递归寻找
    if (ch != '#') // 在左子树中，则返回该值
        return ch;
    ch = PreNode(b->rchild, k); // 右子树中递归寻找
    return ch;
}

```

本题实质上就是一个遍历算法的实现，只不过用一个全局变量来记录访问的序号，求其他遍历序列的第 k 个结点也采用相似的方法。二叉树的遍历算法可以引申出大量的算法题，因此考生务必要熟练掌握二叉树的遍历算法。

5. 解答：对一般二叉树，仅根据先序或后序序列，不能确定另一个遍历序列。但对满二叉树，任意一个结点的左、右子树均含有相等的结点数，同时，先序序列的第一个结点作为后序序列的最后一个结点，由此得到将先序序列 $pre[l1 \dots h1]$ 转换为后序序列 $post[l2 \dots h2]$ 的递归模型如下：

```

f(pre, l1, h1, post, l2, h2) = 不做任何事情           //  $h1 < l1$  时
f(pre, l1, h1, post, l2, h2) = post[h2] = pre[l1]      // 其他情况
    取中间位置 half = (h1 - l1) / 2;
    将 pre[l1+1, l1+half] 左子树转换为 post[l2, l2+half-1],
    即 f(pre, l1+1, l1+half, post, l2, l2+half-1);
    将 pre[l1+half+1, h1] 右子树转换为 post[l2+half, h2-1],
    即 f(pre, l1+half+1, h1, post, l2+half, h2-1)。

```

其中， $post[h2] = pre[l1]$ 表示后序序列的最后一个结点（根结点）等于先序序列的第一个结点（根结点）。相应的算法实现如下：

```

void PreToPost(ElemType pre[], int l1, int h1, ElemType post[], int l2, int h2) {
    int half;
    if (h1 >= l1) {
        post[h2] = pre[l1]; // 将前序遍历的第一个元素赋值给后序遍历的最后一个元素
        half = (h1 - l1) / 2; // 计算中间位置
        PreToPost(pre, l1 + 1, l1 + half, post, l2, l2 + half - 1); // 转换左子树
        PreToPost(pre, l1 + half + 1, h1, post, l2 + half, h2 - 1); // 转换右子树
    }
}

```

}
例如, 有以下代码:

```
ElemType *pre = "ABCDEFGH";           // 前序遍历数组
ElemType post[MaxSize];               // 后序遍历数组
PreToPost(pre, 0, 6, post, 0, 6);     // 调用函数进行转换
printf("后序序列: ");
for(int i = 0; i <= 6; i++) {
    printf("%c", post[i]);
}
printf("\n");
执行结果如下:
后序序列:  C D B F G E A
```

6.解: 只要稍改一下二叉树的前序或中序遍历算法即可。算法代码如下:

```
void Double_order(BTNode *t) {
    if(t!=NULL) {
        Visit(t);           //Visit() 是已经定义的访问 t 的函数
        Double_order(t->lchild);
        Visit(t);
        Double_order(t->rchild);
    }
}
```

5.2.2 树的存储专题

1.假设二叉树采用二叉链存储结构, 设计一个算法, 利用结点的右孩子指针 rchild 将一棵二叉树的叶子结点按照从左往右的顺序串成一个单链表(在题目中定义两个指针 head 与 tail, 其中 head 指向第一个叶子结点, head 初值为 NULL, tail 指向最后一个叶子结点)。

2.设一棵二叉树中各结点的值互不相同, 其先序遍历序列和中序遍历序列分别存于两个一维数组 A[1...n] 和 B[1...n]中, 试编写算法建立该二叉树的二叉链表。

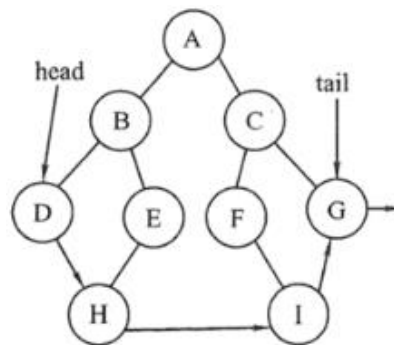
3.设计一个算法将二叉树的叶结点按从左到右的顺序连成一个单链表, 表头指针为 head。二叉树按二叉链表方式存储, 链接时用叶结点的右指针域来存放单链表指针。

4.已知一棵树的层次序列及每个结点的度, 编写算法构造此树的孩子一兄弟链表。

【答案解析】

1.分析。右图所示的二叉树即为链接完成后的二叉树。要想解决本题, 显然需要遍历这棵二叉树。通过遍历此二叉树, 能访问到每一个叶子结点, 并在访问时对其 rchild 指针进行修改, 以达到将叶子结点串成一条单链表的目的。通过重要技巧提示可以看出, 不论哪种遍历方式都可以对其叶子结点按照从左到右的顺序进行访问(这点在结点的先序、中序、后序遍历序列中也可以看出, 不管哪种遍历序列, 叶子结点在序列中的相对位置总是不变的)。

我们所需要做的只是在访问每个结点的过程中, 判断此结点是否是叶子结点, 如果是, 就对 rchild 指针修改, 如果不是, 就什么都不做。这里采用先序遍历方式来解决此题。



经过以上分析后, 此题已经基本解决, 剩下的就是应用单链表中的知识了。因为题目中提到要用 head 和 tail 指针分别来指示 遍历过程中第一个叶子结点和最后一个叶子结点, 所以要对第一个叶子结点进行一下区分, 并让 head 指向它, 而且 tail 始终指向当前链表中的最后一个结点, 即每当一个新结点来到时, 通过 tail 可以将这个结点链接到表尾, 并将 tail 指向它。

本题代码如下:

```
void link(BTNode *p, BTNode *&head, BTNode *&tail) {
    if (p!=NULL) { /*关键步骤开始*/
        if (p->lchild == NULL && p->rchild == NULL) { //判断是不是叶子结点
            if (head==NULL) { /*看 head 是不是为 NULL, 如果是, 说明当前
                                是第一个叶子结点, 则将 head 和 tail 都指向它*/
                head=p;
                tail =p;
            }
            else { /*如果 head 不为 NULL, 说明 head 已经指向第一个叶子结点, 因此这
                    不是第一个叶子结点, 则将此结点连接到链表的尾部, 并且将 tail 指向它, 即 指向新的表尾结点*/
                tail->rchild =p;
                tail=p;
            }
        } /*关键步骤结束*/
        link(p->lchild, head, tail); // (1)
        link(p->rchild, head, tail); // (2)
    }
}
```

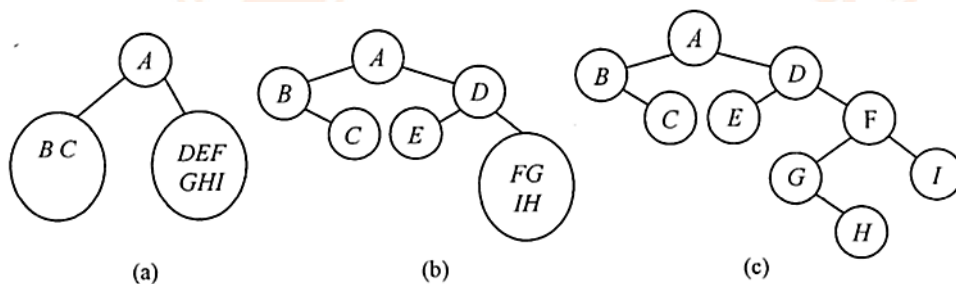
说明: 经过以上的分析可以知道, 上述程序段的关键步骤部分还可以放在 (1) 与 (2) 之间, 或者之后, 即中序、后序遍历一样可以得到同样的结果。

2. 解答: 由先序序列和中序序列可以唯一确定一棵二叉树。算法的实现步骤如下。

1) 根据先序序列确定树的根结点。

2) 根据根结点在中序序列中划分出二叉树的左、右子树包含哪些结点, 然后根据左、右子树结点在先序序列中的次序确定子树的根结点, 即回到步骤 1)。

如此重复上述步骤, 直到每棵子树仅有一个结点 (该子树的根结点) 为止, 如下图所示。



算法实现如下:

```
BiTree PreInCreat(ElemType A[], ElemType B[], int l1, int h1, int l2, int h2) {
    // l1, h1 为先序的第一个和最后一个结点下标, l2, h2 为中序的第一个和最后一个结点下标
    // 初始调用时, l1 = l2 = 1, h1 = h2 = n
    root = (BiTNode*)malloc(sizeof(BiTNode)); // 建根结点
    root->data = A[l1]; // 根结点
    for (i = l2; B[i] != root->data; i++); // 根结点在中序序列中的划分
    llen = i - l2; // 左子树长度
    rlen = h2 - i; // 右子树长度
    if (llen)
        PreInCreat(A, B, l1+1, h1, l2, i);
    if (rlen)
        PreInCreat(A, B, i+1, h1, i+1, h2);
}
```

```

    root->lchild = PreInCreat(A, B, l1 + 1, l1 + llen, l2, l2 + llen - 1); // 递归建立左子树
else
    root->lchild = NULL; // 左子树为空
if (rlen)
    root->rchild = PreInCreat(A, B, h1 - rlen + 1, h1, h2 - rlen + 1, h2); // 递归建立右子树
else
    root->rchild = NULL; // 右子树为空
return root; // 返回根结点指针
}

```

3. 解答：通常我们所用的先序、中序和后序遍历对于叶结点的访问顺序都是从左到右，这里我们选择中序递归遍历。算法思想：设置前驱结点指针 `pre`，初始为空。第一个叶结点由指针 `head` 指向，遍历到叶结点时，就将其前驱的 `rchild` 指针指向它，最后一个叶结点的 `rchild` 为空。算法实现如下：

```

LinkedList head, pre = NULL; // 全局变量
LinkedList InOrder(BiTree bt) {
    if (bt) {
        InOrder(bt->lchild); // 中序遍历左子树
        if (bt->lchild == NULL && bt->rchild == NULL) { // 叶结点
            if (pre == NULL) {
                head = bt;
                pre = bt;
            }
            else {
                pre->rchild = bt;
                pre = bt;
            }
        }
        InOrder(bt->rchild); // 中序遍历右子树
        pre->rchild = NULL; // 设置链表尾
    }
    return head;
}

```

上述算法的时间复杂度为 $O(n)$ ，辅助变量使用 `head` 和 `pre`，栈空间复杂度为 $O(n)$ 。

4. 解答：本题与树的层次序列有关。可设立一个辅助数组 `pointer[]` 存储新建树的各结点的地址，再根据层次序列与每个结点的度，逐个链接结点。算法描述如下：

```

#define maxNodes 15
void createCSTree_Degree(CSTree &T, DataType e[], int degree[], int n) {
    // 根据树结点的层次序列 e[] 和各结点的度 degree[] 构造树的孩子-兄弟链表
    // 参数 n 是树结点个数
    CSNode *pointer = new CSNode[maxNodes]; // 判断 pointer[i] 为空的语句未写
    int i, j, d, k = 0;
    for (i = 0; i < n; i++) { // 初始化
        pointer[i]->data = e[i];
        pointer[i]->lchild = pointer[i]->rsibling = NULL;
    }
    for (i = 0; i < n; i++) {
        d = degree[i]; // 结点 i 的度数
        if (d) {
            k++; // k 为子女结点序号
            pointer[i]->lchild = pointer[k]; // 建立 i 与子女 k 间的链接
            for (j = 2; j <= d; j++) {

```

```

        k++;
        pointer[k - 1]->rsibling = pointer[k];
    }
}
T = pointer[0];
delete[] pointer;
}

```

5.2.3 二叉树的删除专题

1. 已知二叉树以二叉链表存储, 编写算法完成: 对于树中每个元素值为 x 的结点, 删去以它为根的子树, 并释放相应的空间。

【答案解析】

1. 解答: 删除以元素值 x 为根的子树, 只要能删除其左、右子树, 就可以释放值为 x 的根结点, 因此宜采用后序遍历。算法思想: 删除值为 x 的结点, 意味着应将其父结点的左(右)子女指针置空, 用层次遍历易于找到某结点的父结点。本题要求删除树中每个元素值为 x 的结点的子树, 因此要遍历完整棵二叉树。算法实现如下:

```

void DeleteXTree(BiTree &bt) {                // 删除以 bt 为根的子树
    if (bt) {
        DeleteXTree(bt->lchild);              // 删除 bt 的左子树
        DeleteXTree(bt->rchild);              // 删除 bt 的右子树
        free(bt);                             // 释放被删结点所占的存储空间
    }
}
// 在二叉树上查找所有以 x 为元素值的结点, 并删除以其为根的子树
void Search(BiTree bt, ElemType x) {
    BiTree Q[];                               // Q 是存放二叉树结点指针的队列, 容量足够大
    if (bt) {
        if (bt->data == x) {                  // 若根结点值为 x, 则删除整棵树
            DeleteXTree(bt);
            exit(0);
        }
        InitQueue(Q);                        // 初始化队列
        EnQueue(Q, bt);                      // 将根节点入队
        while (!IsEmpty(Q)) {
            DeQueue(Q, p);                   // 出队
            if (p->lchild) {                  // 若左子女非空
                if (p->lchild->data == x) {   // 左子树符合则删除左子树
                    DeleteXTree(p->lchild);
                    p->lchild = NULL;        // 父结点的左子女置空
                }
            } else
                EnQueue(Q, p->lchild);        // 左子树入队列
            if (p->rchild) {                  // 若右子女非空
                if (p->rchild->data == x) {   // 右子树符合则删除右子树
                    DeleteXTree(p->rchild);
                    p->rchild = NULL;        // 父结点的右子女置空
                }
            } else
        }
    }
}

```

```

        EnQueue(Q, p->rchild); // 右子树入队列
    }
}

```

5.2.4 点数量计算专题

1. 假设二叉树采用二叉链存储结构, 设计一个算法, 计算一棵给定二叉树的所有结点数。
2. 假设二叉树采用二叉链存储结构, 设计一个算法, 计算一棵给定二叉树的所有叶子结点数。
3. 假设二叉树采用二叉链表存储结构存储, 试设计一个算法, 计算一棵给定二叉树的所有双分支结点数。
4. 编程求以孩子兄弟表示法存储的森林的叶子结点数。

【答案解析】

1. 解法一: 给出一棵二叉树, 怎样求它含有的结点数呢? 最直接的想法就是去遍历这棵二叉树。可以设置一个全局变量 n , 初值为 0, 每当经过一个结点, 就让全局变量自动增 1, 这样遍历结束后, n 中就保存了整个树中结点的个数。任何一种遍历方式都可以对树中所有结点各访问一次, 因此采用任何一种遍历方式都可以解决此题。这里采用先序遍历, 代码如下:

```

int n=0; //初值设为 0
void count (BTNode* p) {
    if(p!=NULL) {
        ++n; //来到一个结点的时候, n 自动增 1
        count(p->lchild);
        count(p->rchild);
    }
}

```

解法二: 我们换一种角度想问题, 给出一棵二叉树, 可以分几步来找出其结点数。一种可行的方法是, 如果这棵二叉树是空树, 则直接得出其结点数为 0; 如果二叉树非空, 则先数出其左子树的结点数 n_1 , 再数出其右子树的结点数 n_2 , 最后得出 n_1+n_2+1 就是整棵树的结点数, 这对应了树的后序遍历方式。将此过程翻译成程序语言, 即可写出以下代码:

```

int count(BTNode* p) {
    int n1,n2;
    if(p==NULL)
        return 0;
    else{
        n1=count(p->lchild); //数左子树中的结点个数并返回给 n1
        n2=count(p->rchild); //数右子树中的结点个数并返回给 n2
        return n1+n2+1; //返回总的结点个数
    }
}

```

2. 叶子结点必须满足左子树和右子树都为空。

代码:

```

int count(BTNode* p) {
    int n1,n2;
    if (p==NULL)

```



```

    return 0;           //如果空树，则返回 0
else if (p->lchild == NULL && p->rchild == NULL) //如果左、右子树为空，则返
                                                    //回 1，代表有 1 个叶子结点

    return 1;
else{
    n1=count(p->lchild);           //求出左子树的叶子结点数并返回给 n1
    n2=count(p->rchild);           //求出右子树的叶子结点数并返回给 n2
    return n1+n2;                 //返回 n1 与 n2 的和，即为总的叶子结点数
}
}

```

3. 解答：计算一棵二叉树 b 中所有双分支结点个数的递归模型 $f(b)$ 如下：

```

f(b)=0           若 b=NULL
f(b)=f(b->lchild)+f(b->rchild)+1  若 *b 为双分支结点
f(b)=f(b->lchild)+f(b->rchild)    其他情况 (*b 为单分支结点或叶子结点)

```

具体算法实现如下：

```

int DsonNodes(BiTree b) {
    if (b == NULL)
        return 0; // 如果节点为空，返回 0
    else if (b->lchild != NULL && b->rchild != NULL) // 如果节点是双分支节点
        return DsonNodes(b->lchild) + DsonNodes(b->rchild) + 1; //递归计算左子树和右子树，并加 1
    else
        return DsonNodes(b->lchild) + DsonNodes(b->rchild); // 递归计算左子树和右子树
}

```

当然，本题也可以设置一个全局变量 $Nurn$ ，每遍历到一个结点时，判断每个结点是否为分支结点（左、右结点都不为空，注意是双分支），若是则 $Nurn++$ 。

4. 解答：当森林（树）以孩子兄弟表示法存储时，若结点没有孩子($fch=null$)，则它必是叶子，总的叶子结点个数是孩子子树(fch)上的叶子数和兄弟子树($nsib$)上的叶结点个数之和。

算法代码如下：

```

typedef struct node {
    ElemType data;           // 数据域
    struct node *fch, *nsib; // 孩子与兄弟域
} *Tree;
int Leaves(Tree t) {        // 计算以孩子兄弟表示法存储的森林的叶子节点数
    if (t == NULL)
        return 0;           // 树空返回 0
    if (t->fch == NULL)
        return 1 + Leaves(t->nsib); // 若结点无孩子，则该结点必是叶子
    else
        return Leaves(t->fch) + Leaves(t->nsib); // 孩子子树和兄弟子树中叶子数之和
}

```

5.2.5 树的高度和宽度的计算

1. 假设二叉树采用二叉链表存储结构，设计一个非递归算法求二叉树的高度。
2. 假设二叉树采用二叉链表存储结构，设计一个算法，求非空二叉树 b 的宽度（即具有结点数最多的那一层的结点数）。

3. 以孩子兄弟链表为存储结构, 请设计递归算法求树的深度。

【答案解析】

1. 解答: 采用层次遍历的算法, 设置变量 `level` 记录当前结点所在的层数, 设置变量 `last` 指向当前层的最右结点, 每次层次遍历出队时与 `last` 指针比较, 若两者相等, 则层数加 1, 并让 `last` 指向下一层的最右结点, 直到遍历完成。`level` 的值即为二叉树的高度。

算法实现如下:

```
int Btdepth(BiTree T) {           // 采用层次遍历的非递归方法求解二叉树的高度
    if (!T)                        // 树空, 高度为 0
        return 0;
    int front = -1, rear = -1;
    int last = 0, level = 0;        // last 指向当前层的最右结点
    BiTree Q[MaxSize];             // 设置队列 Q, 元素是二叉树结点指针且容量足够
    Q[++rear] = T;                 // 将根结点入队
    while (front < rear) {          // 队不空, 则循环
        p = Q[++front];            // 队列元素出队, 即正在访问的结点
        if (p->lchild)              // 左孩子入队
            Q[++rear] = p->lchild;
        if (p->rchild)              // 右孩子入队
            Q[++rear] = p->rchild;
        if (front == last) {        // 处理该层的最右结点
            level++;                // 层数增 1
            last = rear;            // last 指向下层
        }
    }
    return level; // 返回层数
}
```

求某层的结点个数、每层的结点个数、树的最大宽度等, 都采用与此题类似的思想。当然, 此题可编写递归算法, 其实现如下:

```
int Btdepth2(BiTree T) {          // 采用递归方法求解二叉树的高度
    if (T == NULL)                 // 空树, 高度为 0
        return 0;
    int ldep = Btdepth2(T->lchild); // 左子树高度
    int rdep = Btdepth2(T->rchild); // 右子树高度
    if (ldep > rdep)                // 树的高度为左子树最大高度加根结点
        return ldep + 1;
    else                           // 树的高度为右子树最大高度加根结点
        return rdep + 1;
}
```

2. 解答: 采用层次遍历的方法求出所有结点的层次, 并将所有结点和对应的层次放在一个队列中。然后通过扫描队列求出各层的结点总数, 最大的层结点总数即为二叉树的宽度。算法实现如下:

```
typedef struct {                  // 保存队列中的结点指针
    BiTree data[MaxSize];        // 保存 data 中相同下标结点的层次
    int level[MaxSize];
    int front, rear;
} Qu;
int BTWidth(BiTree b) {
    BiTree p;
```

```

int k, max, i, n;
Qu.front = Qu.rear = -1;           // 队列为空
Qu.rear++;
Qu.data[Qu.rear] = b;              // 根节点指针入队
Qu.level[Qu.rear] = 1;             // 根节点层次为 1
while (Qu.front < Qu.rear) {
    Qu.front++;                    // 出队
    p = Qu.data[Qu.front];         // 出队结点
    k = Qu.level[Qu.front];        // 出队结点的层次
    if (p->lchild != NULL) {       // 左孩子进队列
        Qu.rear++;
        Qu.data[Qu.rear] = p->lchild;
        Qu.level[Qu.rear] = k + 1;
    }
    if (p->rchild != NULL) {       // 右孩子进队列
        Qu.rear++;
        Qu.data[Qu.rear] = p->rchild;
        Qu.level[Qu.rear] = k + 1;
    }
}
//while
max = 0;                           //max 保存同一层最多的结点个数
i = 0;
k = 1;                             //k 表示从第一层开始查找
while (i <= Qu.rear) {             //i 扫描队中所有元素
    n = 0;                          //n 统计第 k 层的结点个数
    while (i <= Qu.rear && Qu.level[i] == k) {
        n++;
        i++;
    }
    k = Qu.level[i];
    if (n > max)
        max = n;                  // 保存最大的 n
}
return max;
}

```

注意：本题队列中的结点，在出队后仍需要保留在队列中，以便求二叉树的宽度，所以设置的队列采用非环形队列，否则在出队后可能被其他结点覆盖，无法再求二叉树的宽度。

3. 解答：由孩子兄弟链表表示的树，求高度的算法思想如下：采用递归算法，若树为空，高度为零；否则，高度为第一子女树高度加 1 和兄弟子树高度的大者。其非递归算法使用队列，逐层遍历树，取得树的高度。算法代码如下：

```

int Height(CSTree bt) {            // 递归求以孩子兄弟链表表示的树的深度
    int hc, hs;
    if (bt == NULL)
        return 0;                 // 树为空返回 0
    else {                         // 否则，高度取子女高度+1 和兄弟子树高度的大者
        hc = height(bt->firstchild); // 第一子女树高
        hs = height(bt->nextsibling); // 兄弟树高
        if (hc + 1 > hs)
            return hc + 1;
        else
            return hs;
    }
}

```

}

5.2.6 二叉树的交换专题

1. 设树 B 是一棵采用链式结构存储的二叉树, 编写一个把树 B 中所有结点的左、右子树进行交换的函数。

【答案解析】

1. 解答: 采用递归算法实现交换二叉树的左、右子树, 首先交换 b 结点的左孩子的左、右子树, 然后交换 b 结点的右孩子的左、右子树, 最后交换 b 结点的左、右孩子, 当结点为空时递归结束 (后序遍历的思想)。

算法实现如下:

```
void swap(BiTree b) { // 本算法递归地交换二叉树的左、右子树
    if (b) {
        swap(b->lchild); // 递归地交换左子树
        swap(b->rchild); // 递归地交换右子树
        BiTree temp = b->lchild; // 交换左、右孩子结点
        b->lchild = b->rchild;
        b->rchild = temp;
    }
}
```

5.2.7 二叉树的路径与查找

1. 已知一棵二叉树按顺序存储结构进行存储, 设计一个算法, 求编号分别为 i 和 j 的两个结点的最近的公共祖先结点的值。

2. 在二叉树的二叉链存储结构中, 增加一个指向双亲结点的 parent 指针, 设计一个算法, 给这个指针赋值, 并输出所有结点到根结点的路径。

3. 假设二叉树采用二叉链存储结构, 设计一个算法, 求二叉树 b 中值为 x 的结点的层号。

4. 在二叉树中查找值为 x 的结点, 试编写算法 (用 C 语言) 打印值为 x 的结点的所有祖先, 假设值为 x 的结点不多于一个。

5. 设一棵二叉树的结点结构为 (LLINK, INFO, RLINK), ROOT 为指向该二叉树根结点的指针, p 和 q 分别为指向该二叉树中任意两个结点的指针, 试编写算法 ANCESTOR (ROOT, p, q, r), 找到 p 和 q 的最近公共祖先结点 r。

【答案解析】

1. 首先, 必须明确二叉树中任意两个结点必然存在最近的公共祖先结点, 最坏的情况下是根结点 (两个结点分别在根结点的左右分支中), 而且从最近的公共祖先结点到根结点的全部祖先结点都是公共的。由二叉树顺序存储的性质可知, 任一结点 i 的双亲结点的编号为 $i/2$ 。求解 i 和 j 最近公共祖先结点的算法步骤如下 (设从数组下标 1 开始存储):

1) 若 $i > j$, 则结点 i 所在层次大于等于结点 j 所在层次。结点 i 的双亲结点为结点 $i/2$, 若 $i/2 = j$, 则结点 $i/2$ 是原结点 i 和结点 j 的最近公共祖先结点, 若 $i/2 \neq j$, 则令 $i = i/2$, 即以该结点 i 的

双亲结点为起点, 采用递归的方法继续查找。

2) 若 $j > i$, 则结点 j 所在层次大于等于结点 i 所在层次。结点 j 的双亲结点为结点 $j/2$, 若 $j/2 = i$, 则结点 $j/2$ 是原结点 i 和结点 j 的最近公共祖先结点, 若 $j/2 \neq i$, 则令 $j = j/2$ 。重复上述过程, 直到找到它们最近的公共祖先结点为止。

本题代码如下:

```
ElemType Comm_Ancesor(SqTree T, int i, int j) {
    // 本算法在二叉树中查找结点 i 和结点 j 的最近公共祖先结点
    if (T[i] != '#' && T[j] != '#') {           // 结点存在
        while (i != j) {                          // 两个编号不同时循环
            if (i > j)                             // 向上找 i 的祖先
                i = i / 2;
            else                                    // 向上找 j 的祖先
                j = j / 2;
        }
        return T[i];                             // 返回最近公共祖先结点
    }
}
```

由解题中算法的步骤描述可知, 本题也很容易地联想到采用递归的方法求解。

2. 本题需要分 3 步解答: 第 1) 步, 修改二叉链结点的数据结构; 第 2) 步, 将所有结点的 parent 指针指向其双亲结点; 第 3) 步, 打印所有结点到根结点的路径。

(1) 修改数据结构。

```
typedef struct BTreeNode {
    char a;
    struct BTreeNode *parent;           // 此句是新增部分
    struct BTreeNode *lchild;
    struct BTreeNode *rchild;
} BTreeNode;
```

(2) 给各个结点的 parent 赋值。

因为要访问所有结点, 显然要用遍历来做, 修改遍历模板即可。在每访问到一个新结点时, 都要知道其双亲结点的地址, 才可以将其 parent 指针指向它, 为此需要一个指针来指向其双亲结点。代码如下:

```
void triBtree(BTreeNode *p, BTreeNode *q) {    /* 此处参数 q 始终指向当前访问结点 p 的双亲结点。
                                                当 p 为根结点时, q 应为 NULL */
    if (p != NULL) {
        p->parent = q;           // 将当前所访问的结点的 parent 指针指向 q 即指向其双亲结点
        q = p;                  /* 将 q 指向 p, 因为下边将要对 p 的左孩子和右孩子的
                                parent 指针进行赋值, 显然 p 是其双亲结点 */
        triBtree(p->lchild, q);  // 修改其左子树中所有结点的 parent 指针
        triBtree(p->rchild, q);  // 修改其右子树中所有结点的 parent 指针
    }
}
```

3) 打印路径。

上一步已经完成了 parent 的赋值, 这样就可以很容易地打印出各个结点的路径了。可以分两步来完成:

① 任给一个结点, 怎样打印出这个结点到根结点的路径呢? 只需打印出此结点, 然后通过 parent 找到其双亲结点。如此重复, 直到 parent 为 NULL (上一步中已经提到, 根结点的 parent 为 NULL) 即可。

代码如下:

```
void printPath (BTNode *p) {
    while(p!=NULL){ //p 不为空时就打印其 data 域值
        cout<<p->data<<" "<<endl;
        p=p->parent; //找到其双亲结点
    }
}
```

②怎样打印出所有路径? 要打印所有路径, 必须找到所有结点并逐一打印, 因此这里又用到了遍历, 显然任何一种遍历方式都可以。

代码如下:

```
void allPath (BTNode *p) {
    if(p!=NULL) {
        printPath (p); //每到一个结点的时候就调用句中的函数 prinPath ()
        //来打印这个结点到根结点的路径
        allPath (p->lchild );
        allPath (p->rchild );
    }
}
```

3.分析: 本题可以用层次遍历中例题所讲述的方法来解决, 这里我们用另一种方法。题目中要求给出值为 x 的结点所在的层号, 这里就包含一个查找操作, 即要在二叉树中找到这个结点。因此我们自然而然地想到要用遍历来做。题目又要求给出其层次号, 因此在遍历过程中, 当访问到一个新结点时, 就要算出其层号。这里还要用到重要技巧提示中的路线图以及对应的程序模板。

根据图中指针 p 的行走路径的特点可以发现, p 总是由上层走向下层, 一直走到最底层, 然后返回上层, 如此往复。因此, 可以用一个变量 L 来记录当前 p 所在的层次号, 当 p 由上层走向下层的时候, $++L$; 当 p 由下层走向上层的时候, $--L$ 。怎样实现 L 随着 p 的走向不同 而自增或者自减呢? 对于图中的程序模板, 在(1)处添加代码, 代表 p 第一次来到这个结点或者 p 将要由这个结点走向下层结点时要做的事情在(3)处添加代码, 代表 p 第三次来到这个结点或者 p 将要由这个结点返回上层结点的时候要做的事情。因此, 在(1)处添加 $++L$, 在(3)处添加 $--L$, 就可以实现由 L 来指出当 p 的层号。

对图中的程序模板进行修改, 得出以下代码:

```
int L=1; //L 是全局变量, 初值为 1, 用来记录当前所访问的结点层号
void leno(BTNode *p,char x) {
    if (p!=NULL) {
        if (p->data==x) /*一次来到这个结点时, 对其 data 域进行检查, 看是否等于 X,
            如果相等, 则打印出其层号 L */
            cout<<L<<endl;
        ++L; /* 打印完层号之后, 指针 p 将要进入下一层结点所示, 因此 L 要
            自增 1,代表下一层的层号 */
        leno(p->lchild,x);
        leno(p->rchild,x);
        --L; //p 指针将要由下一层返回上一层, 因此 L 要自减 1, 代表上一层号
    }
}
```

4. 解答: 算法思想: 采用非递归后序遍历, 最后访问根结点, 访问到值为 x 的结点 时, 栈中所有元素均为该结点的祖先, 依次出栈打印即可。算法实现如下:

```
typedef struct {
```

```

BiTree t;           // 二叉树节点指针
int tag;            // 标记: tag=0 表示左子女被访问, tag=1 表示右子女被访问
} stack;
void Search(BiTree bt, ElemType x) { // 在二叉树 bt 中, 查找值为 x 的结点, 并打印其所有祖先
    stack s[];           // 栈容量足够大
    int top = 0;
    while (bt != NULL || top > 0) {
        while (bt != NULL && bt->lchild != x) {
            s[++top].t = bt;           // 结点入栈
            s[top].tag = 0;           // 标记左子女被访问
            bt = bt->lchild;           // 沿左分支向下
        }
        if (bt->data == x) {
            printf("所查结点的所有祖先结点的值为:\n");           // 找到 x
            for (int i = 1; i <= top; i++)
                printf("%d ", s[i].t->data);           // 输出祖先值后结束
            exit(1);
        }
        while (top != 0 && s[top].tag == 1)
            top--;           // 退栈 (空遍历)
        if (top != 0) {
            s[top].tag = 1;           // 标记右子女被访问
            bt = s[top].t->rchild;           // 沿右分支向下遍历
        }
    }
}

```

因为查找的过程就是后序遍历的过程, 因此使用的栈的深度不超过树的深度。

5. 解答: 后序遍历最后访问根结点, 即在递归算法中, 根是压在栈底的。本题要找 p 和 q 的最近公共祖先结点 r , 不失一般性, 设 p 在 q 的左边。算法思想: 采用后序非递归算法, 栈中存放二叉树结点的指针, 当访问到某结点时, 栈中所有元素均为该结点的祖先。后序遍历必然先遍历到结点 p , 栈中元素均为 p 的祖先。先将栈复制到另一辅助栈中。继续遍历到结点 q 时, 将栈中元素从栈顶开始逐个到辅助栈中去匹配, 第一个匹配 (即相等) 的元素就是结点 p 和 q 的最近公共祖先。算法实现如下:

```

typedef struct {
    BiTree t;
    int tag;           // tag=0 表示左子女已被访问, tag=1 表示右子女已被访问
} stack;
stack s[], s1[];      // 栈, 容量足够大
BiTree Ancestor(BiTree ROOT, BiTNode *p, BiTNode *q) {
    // 本算法求二叉树中 p 和 q 指向结点的最近公共结点
    top = 0;
    bt = ROOT;
    while (bt != NULL || top > 0) {
        while (bt != NULL) {
            s[++top].t = bt;
            s[top].tag = 0;
            bt = bt->lchild; // 沿左分支向下
        }
        while (top != 0 && s[top].tag == 1) {
            // 假定 p 在 q 的左侧, 遇到 p 时, 栈中元素均为 p 的祖先
            if (s[top].t == p) {

```

```

        for (i = 1; i <= top; i++)
            s1[i] = s[i];
        topl = top;
    }
    if (s[top].t == q) { // 找到 q 结点
        for (i = top; i > 0; i--) {
            for (j = topl; j > 0; j--) {
                if (s1[j].t == s[i].t)
                    return s[i].t;
            }
        }
        top--;
    } // while
}
if (top != 0) {
    s[top].tag = 1;
    bt = s[top].t->rchild;
}
return NULL;
}

```

// 将栈 s 的元素转入辅助栈 s1 保存

// 将栈中元素的树结点到 s1 中去匹配

// p 和 q 的最近公共祖先已找到

// 退栈

// 沿右分支向下遍历

// p 和 q 无公共祖先

5.2.8 二叉树的判定专题

1. 二叉树按二叉链表形式存储, 写一个判别给定二叉树是否是完全二叉树的算法。

2. 试设计判断两棵二叉树是否相似的算法。所谓二叉树 T_1 和 T_2 相似, 指的是 T_1 和 T_2 都是空的二叉树或都只有一个根结点; 或 T_1 的左子树和 T_2 的左子树是相似的, 且 T_1 的右子树和 T_2 的右子树是相似的。

【答案解析】

1. 解答: 根据完全二叉树的定义, 具有 n 个结点的完全二叉树与满二叉树中编号从 $1 \sim n$ 的结点一一对应。算法思想: 采用层次遍历算法, 将所有结点加入队列 (包括空结点)。遇到空结点时, 看其后是否有非空结点。若有, 则二叉树不是完全二叉树。

算法实现如下:

```

bool IsComplete(BiTree T) {
    InitQueue(Q);
    if (!T)
        return 1;
    EnQueue(Q, T);
    while (!IsEmpty(Q)) {
        DeQueue(Q, p);
        if (p) {
            EnQueue(Q, p->lchild);
            EnQueue(Q, p->rchild);
        }
        else { // 如果节点为空
            while (!IsEmpty(Q)) {
                DeQueue(Q, p);
                if (p)
                    return 0;
            }
        }
    }
}

```

// 本算法判断给定二叉树是否为完全二叉树

// 初始化队列 Q

// 空树为满二叉树

// 将根节点入队

// 当队列不为空时循环

// 出队一个节点 p

// 如果节点非空

// 将左子节点入队

// 将右子节点入队

// 检查其后是否有非空节点

// 出队一个节点 p

// 如果节点非空, 则二叉树为非完全二叉树


```

    }
}
return 1; // 如果所有节点都满足条件, 则返回 1, 表示是完全二叉树
}

```

2. 本题采用递归的思想求解, 若 T_1 和 T_2 都是空树, 则相似; 若有一个为空另一个不空, 则必然不相似; 否则递归地比较它们的左、右子树是否相似。递归函数的定义如下:

- 1) $f(T_1, T_2) = 1$; 若 $T_1 = T_2 = \text{NULL}$ 。
- 2) $f(T_1, T_2) = 0$; 若 T_1 和 T_2 之一为 NULL , 另一个不为 NULL 。
- 3) $f(T_1, T_2) = f(T_1 \rightarrow \text{lchild}, T_2 \rightarrow \text{lchild}) \& \& f(T_1 \rightarrow \text{rchild}, T_2 \rightarrow \text{rchild})$; 若 T_1 和 T_2 均不为 NULL 。

因此, 算法实现如下:

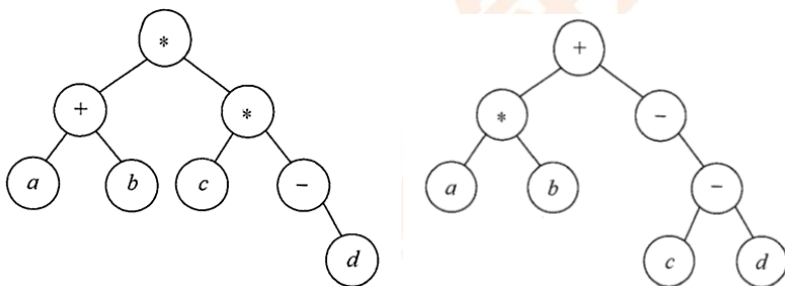
```

int similar(BiTree T1, BiTree T2) {           // 采用递归的算法判断两棵二叉树是否相似
    int leftS, rightS;
    if (T1 == NULL && T2 == NULL)             // 两树皆空
        return 1;
    else if (T1 == NULL || T2 == NULL)         // 只有一树为空
        return 0;
    else {                                     // 递归判断
        leftS = similar(T1->lchild, T2->lchild);
        rightS = similar(T1->rchild, T2->rchild);
        return leftS && rightS;
    }
}

```

5.2.9 线索二叉树

1. 写出在中序线索二叉树里查找指定结点在后序的前驱结点的算法。



输出的等价中缀表达式分别为 $(a + b) * (c * (-d))$ 和 $(a * b) + (-(c - d))$ 。二叉树结点定义如下:

```

typedef struct node {
    char data[10]; // 存储操作数或操作符
    struct node *left, *right;
} BTree;

```

要求:

- 1) 给出算法的基本设计思想。
- 2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。

2. 设中序线索二叉树的类型为 $\text{TBTNode} * \text{InThTree}$;

- 1) 设计算法, 在一棵中序线索二叉树中寻找结点 t 的子树上中序下的最后一个结点。
- 2) 设计算法, 在一棵中序线索二叉树中寻找结点 t 的中序下的前驱。
- 3) 设计算法, 在一棵中序线索二叉树中寻找结点 t 的前序下的后继。

【答案解析】

1. 算法思想: 在后序序列中, 若结点 p 有右子女, 则右子女是其前驱, 若无右子女而有左子女, 则左子女是其前驱。若结点 p 左、右子女均无, 设其中序左线索指向某祖先结点 f (p 是 f 右子树中按中序遍历的第一个结点), 若 f 有左子女, 则其左子女是结点 p 在后序下的前驱; 若 f 无左子女, 则顺其前驱找双亲的双亲, 一直找到双亲有左子女 (这时左子女是 p 的前驱)。还有一种情况, 若 p 是中序遍历的第一个结点, 则结点 p 在中序和后序下均无前驱。

算法代码如下:

```
BiThrTree InPostPre(BiThrTree t, BiThrTree p) {
    // 在中序线索二叉树 t 中, 求指定结点 p 在后序下的前驱结点 q
    BiThrTree q;
    if (p->rtag == 0)                // 若 p 有右子女, 则右子女是其后序前驱
        q = p->rchild;
    else if (p->ltag == 0)            // 若 p 只有左子女, 则左子女是其后序前驱
        q = p->lchild;
    else if (p->lchild == NULL)       // p 是中序序列第一结点, 无后序前驱
        q = NULL;
    else {                            // 顺左线索向上找 p 的祖先, 若存在, 再找祖先的左子女
        while (p->ltag == 1 && p->lchild != NULL)
            p = p->lchild;
        if (p->ltag == 0)              // p 结点的祖先的左子女是其后序前驱
            q = p->lchild;
        else
            q = NULL;                // 仅有单支树 (p 是叶子), 已到根结点, p 无后序前驱
    }
    return q;
}
```

2. 解: 1) 在一棵中序线索二叉树中, 寻找某结点 t 为根的子树上中序的最后一个结点。

算法的思路: 沿结点 t 的右子树链一直走下去, 直到遇到其右指针为右线索的结点为止, 该结点即为所求。

算法代码如下:

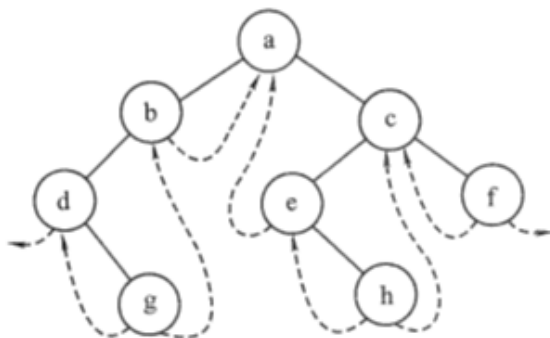
```
TBTNode* inLast(TBTNode *t) {
    TBTNode *p=t;
    while (p && !p->rtag)
        p=p->rchild;
    return p;
}
```

2) 在一棵中序线索二叉树中, 寻找某结点 t 的中序下的前驱。

算法的思路: 若结点 t 有左线索, 则其左线索所指结点即为其中序前驱; 若无左线索, 则其左子树中中序的最后一个结点即为它的中序前驱。

算法代码如下:

```
TBTNode* inPrior(TBTNode *t) {
    TBTNode *p=t->lchild;
    if (p && !t->ltag)
        p=inLast(p);
    return p;
}
```



3) 在一棵中序线索二叉树中, 寻找结点 t 的前序下的后继。下图所示的二叉树是一棵中序线索二叉树, 前序遍历的结果是 $abdgcehf$ 。分析这个示例可知:

- ① a 有左子女 b , a 的前序下的后继为 b 。
- ② d 无左子女但有右子女 g , d 的前序下的后继是 g 。
- ③ g 既无左子女又无右子女, g 的前序下的后继为 C , C 可从 g 沿右线索链到 a , a 的右子女即为 g 的前序下的后继。

算法代码如下:

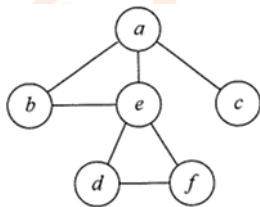
```
TBTNode* treNext(TBTNode *t) {
    TBTNode *p;
    if (! t->ltag)
        p=t->lchild;
    else if (!t->rtag)
        p=t->rchild;
    else{
        p=t;
        while (p && p->rtag)
            p=p->rchild;
        if(p)
            p=p->rchild;
    }
    return p;
}
```

6、图

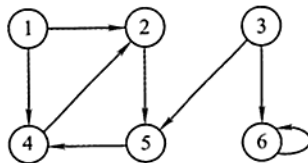
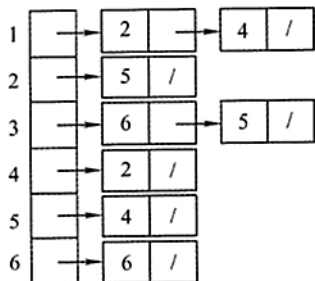
6.1 综合练习

(一) 选择题

1. 一个有 28 条边的非连通无向图至少有 () 个顶点。
A. 7 B. 8 C. 9 D. 10
2. 无向图 G 有 23 条边, 度为 4 的顶点有 5 个, 度为 3 的顶点有 4 个, 其余都是度为 2 的顶点, 则图 G 最多有 () 个顶点。
A. 11 B. 12 C. 15 D. 16
3. 具有 6 个顶点的元向图, 当有 () 条边时能确保是一个连通图。
A. 8 B. 9 C. 10 D. 11
4. 带权有向图 G 用邻接矩阵存储, 则 v_i 的入度等于邻接矩阵中 ()。
A. 第 i 行非 ∞ 的元素个数 B. 第 i 列非 ∞ 的元素个数
C. 第 i 行非 ∞ 且非 0 的元素个数 D. 第 i 列非 ∞ 且非 0 的元素个数
5. 一个有 n 个顶点的图用邻接矩阵 A 表示, 若图为有向图, 顶点 v_i 的入度是 () ; 若图为无向图, 顶点 v_i 的度是 ()。
A. $\sum_{i=1}^n A[i][j]$ B. $\sum_{j=1}^n A[j][i]$ C. $\sum_{i=1}^n A[j][i]$ D. $\sum_{j=1}^n A[j][i]$ 或 $\sum_{j=1}^n A[i][j]$
6. 用有向无环图描述表达式 $(A+B)*((A+B)/A)$, 至少需要顶点的数目为 ()。
A. 5 B. 6 C. 8 D. 9
7. 如下图所示, 在下面的 5 个序列中, 符合深度优先遍历的序列个数是 ()。

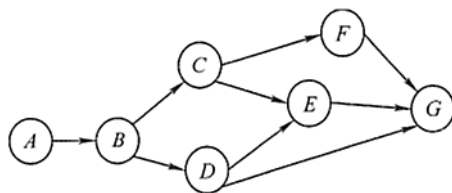


- aebfdc 2. acfdeb 3. aedfcb 4. aefdbc 5. aecfdb
- A. 5 B. 4 C. 3 D. 2
8. 一个有向图 G 的邻接表存储如下图所示, 从顶点 1 出发, 对图 G 调用深度优先遍历所得顶点序列是 () ; 按广度优先遍历所得顶点序列是 ()。



- A. 125436 B. 124536 C. 124563 D. 362514
9. 已知带权连通无向图 $G=(V, E)$, 其中 $V=\{v_1, v_2, v_3, v_4, v_5, v_6, v_7\}$, $E=\{(v_1, v_2)10, (v_1, v_3)2, (v_3, v_4)2, (v_3, v_6)11, (v_2, v_5)1, (v_4, v_5)4, (v_4, v_6)6, (v_5, v_7)7, (v_6, v_7)3\}$ (注: 顶点偶对括号外的数据表示边上的权值), 从源点 v_1 到顶点 v_7 的最短路径上经过的顶点序列是 ()。
A. v_1, v_2, v_5, v_7 B. v_1, v_3, v_4, v_6, v_7 C. v_1, v_3, v_4, v_5, v_7 D. $v_1, v_2, v_5, v_4, v_6, v_7$

10. 下图所示有向图的所有拓扑序列共有 () 个。



- A. 4 B. 6 C. 5 D. 7

11. 若某带权图为 $G=(V, E)$, 其中 $V= \{v_1, v_2, v_3, v_4, v_5, v_6, v_7, v_8, v_9, v_{10}\}$, $E=\{<v_1, v_2>5, <v_1, v_3>6, <v_2, v_5>3, <v_3, v_5>6, <v_3, v_4>3, <v_4, v_5>3, <v_4, v_7>1, <v_4, v_8>4, <v_5, v_6>4, <v_5, v_7>2, <v_6, v_{10}>4, <v_7, v_9>5, <v_8, v_9>2, <v_9, v_{10}>2\}$ (注: 边括号外的数据表示边上的权值), 则 G 的关键路径的长度为 ()。

- A. 19 B. 20 C. 21 D. 22

12. 下列关于关键路径的说法中, 正确的是 ()。

- I. 改变网上某一关键路径上的任一关键活动后, 必将产生不同的关键路径
II. 在 AOE 图中, 关键路径上活动的时间延长多少, 整个工程的时间也就随之延长多少
III. 缩短关键路径上任意一个关键活动的持续时间可缩短关键路径长度
IV. 缩短所有关键路径上共有的任意一个关键活动的持续时间可缩短关键路径长度
V. 缩短多条关键路径上共有的任意一个关键活动的持续时间可缩短关键路径长度

- A. II 和 V B. I、II 和 IV C. II 和 IV D. I 和 IV

13. 由邻接矩阵 $A= \begin{pmatrix} 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \end{pmatrix}$ 可以看出, 该图共有①中的 () 个顶点; 如果是有向图, 该图共有②中的 () 条边; 如果是无向图, 则共有③中的 () 条边。

- ①: A. 9 B. 3 C. 6 D. 1 E. 以上答案均不正确
②: A. 5 B. 4 C. 3 D. 2 E. 以上答案均不正确
③: A. 5 B. 4 C. 3 D. 2 E. 以上答案均不正确

14. 当一个顶点数为 n 的有向图用邻接矩阵 A 表示时, 顶点 i 的度是 ()。

- A. $\sum_{i=1}^n A[i, j]$ B. $\sum_{j=1}^n A[i, j]$ C. $\sum_{i=1}^n A[j, i]$ D. $\sum_{j=1}^n A[i, j] + \sum_{j=1}^n A[j, i]$

15. 已知有向图 $G=(V, E)$, 其中 $V=\{V_1, V_2, V_3, V_4, V_5, V_6, V_7\}$, $E=\{<V_1, V_2>, <V_1, V_3>, <V_1, V_4>, <V_2, V_5>, <V_3, V_5>, <V_3, V_6>, <V_4, V_6>, <V_5, V_7>, <V_6, V_7>\}$, G 的拓扑序列是 ()。

- A. $V_1, V_3, V_4, V_6, V_2, V_5, V_7$ B. $V_1, V_3, V_2, V_6, V_4, V_5, V_7$
C. $V_1, V_3, V_4, V_5, V_2, V_6, V_7$ D. $V_1, V_2, V_5, V_3, V_4, V_6, V_7$

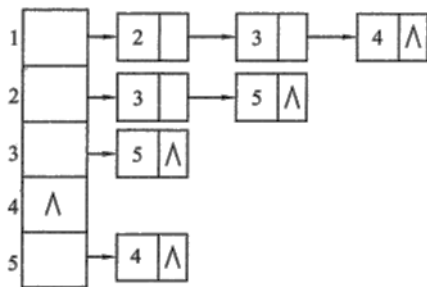
16. 下列关于 AOE 网的叙述中, 不正确的是 ()。

- A. 关键活动延期完成就会影响整个工程的完成时间
B. 任何一个关键活动提前完成, 那么整个工程将会提前完成
C. 所有的关键活动提前完成, 那么整个工程将会提前完成
D. 某些关键活动提前完成, 那么整个工程可能提前完成

17. 带权有向图 G 用邻接矩阵 A 存储, 则顶点 i 的入度等于 A 中 ()。

- A. 第 i 行非 ∞ 的元素之和 B. 第 i 列非 ∞ 的元素之和
C. 第 i 行非 ∞ 且非 0 元素个数 D. 第 i 列非 ∞ 且非 0 的元素个数

18. 一个有向图 G 的邻接表存储结构如下图所示, 现按深度优先搜索遍历, 从 1 出发, 所得到的顶点序列是 ()。



- A. 1, 2, 3, 4, 5 B. 1, 2, 3, 5, 4
C. 1, 2, 4, 5, 3 D. 1, 2, 5, 3, 4

【答案解析】

1. C。此题的解题思路和第 7 题的恰好相反。考查至少有多少个顶点的情形，我们考虑该非连通图最极端的情况，即它由一个完全图加一个独立的顶点构成，此时若再加一条边，则必然使图变成连通图。在 $28 = n(n-1)/2 = 8*7/2$ 条边的完全无向图中，总共有 8 个顶点，再加上 1 个不连通的顶点，共 9 个顶点。

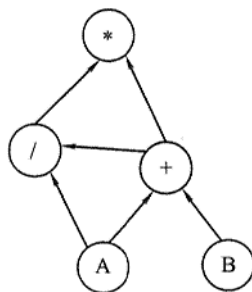
2. D。由于在具有 n 个顶点、 e 条边的无向图中，有 $\sum_{i=1}^n TD(v_i) = 2e$ ，故可求得度为 2 的顶点数为 7，从而最多有 16 个顶点。

3. D。解题思路与第 7 题类似。5 个顶点构成一个完全无向图，需要 10 条边；再加上 1 条边后，能保证第 6 个顶点必然与此完全无向图构成一个连通图，故共需 11 条边。

4. D。有向图的邻接矩阵中，0 和 ∞ 表示的都不是有向边，而入度是由邻接矩阵的列中元素计算出来的；出度是由邻接矩阵的行中元素计算出来的。

5. B、D。有向图的入度是其第 i 列的非 0 元素之和，无向图的度是第 i 行或第 i 列的非 0 元素之和。

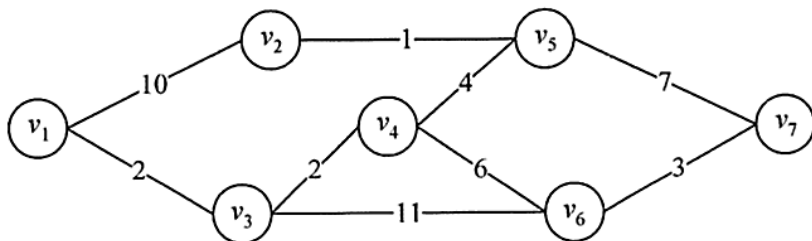
6. A。本题考查图的应用。用图可以表示表达式，式图中顶点表示参与运算的一种操作数和运算符（注意是一种而不是一个），用边来确定各种运算以及运算优先顺序。 $(A+B)*((A+B)/A)$ 表达式中有 3 种运算符： $+$ 、 $*$ 、 $/$ ，有两种操作数： A 、 B ，因此图中顶点数至少为 5。表示表达式的图如下图所示，图中 A 与 B 结合运算符“ $+$ ”做运算，将所得结果与“ A ”结合运算符“ $/$ ”做运算，上两步的结果再结合运算符“ $*$ ”做运算，得到最终结果。本题比较灵活，属于在掌握基础后的能力扩展。



7. D。仅 1 和 4 正确。以 2 为例，遍历到 c 之后，与 c 邻接且未被访问的结点为空集，所以 a 的邻接点 b 或 e 入栈，显然 2 不符合这种情况。

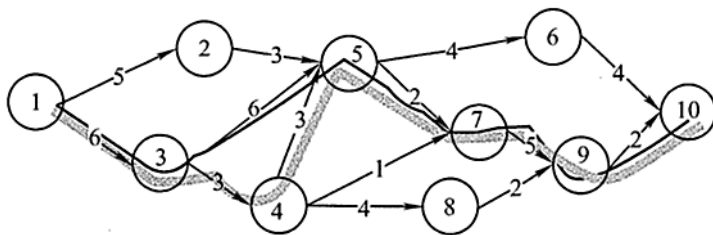
8. A、B。DFS 序列产生的路径为 $\langle 1, 2 \rangle$, $\langle 2, 5 \rangle$, $\langle 5, 4 \rangle$, $\langle 3, 6 \rangle$; BFS 序列产生的路径为 $\langle 1, 2 \rangle$, $\langle 1, 4 \rangle$, $\langle 2, 5 \rangle$, $\langle 3, 6 \rangle$ 。

9. B。题干内容所述的图 G 如下图所示。 A, B, C, D 对应的路径长度分别为 18, 13, 15, 24。应用 Dijkstra 算法不难求出最短路径为 $v_1 \rightarrow v_3 \rightarrow v_4 \rightarrow v_6 \rightarrow v_7$ 。



10. C。本图的拓扑排序序列有 ABCFDEG, ABCDFEG, ABCDEFG, ABDCFEG 和 ABDCEFG。读者应能把这一类经典习题的拓扑序列全部写出来。

11. C。画出题目所表示的图如下, 可得到关键路径的长度为 21。图中所示的两条路径都是关键路径。



12. C。若改变的是所有关键路径上的公共活动, 则不一定会产生不同的关键路径 (延长必然不会导致, 只有缩短才有可能导致)。根据关键路径的定义, 可知 II 正确。关键路径是源点到终点的最长路径, 只有所有关键路径的长度都缩短时, 整个图的关键路径才能有效缩短, 但也不能任意缩短, 一旦缩短到一定程度, 该关键活动就可能会变成非关键活动。

13. B, B, D。本题考查图的存储结构。对于邻接矩阵, 行数或者列数代表其所表示的图的顶点数, 因此①处选 B。邻接矩阵如果是对称的, 则它所表示的既可能是有向图, 又可能是无向图。表示有向图的时候, 邻接矩阵中 1 的个数就是有向图中的边数; 表示无向图的时候, 邻接矩阵中 1 的个数正好是图中边数的两倍, 因此②处选 B, ③处选 D。

14. D。本题考查图的存储结构。有向图中, 一个顶点的度是它的入度和出度之和, 反映在邻接矩阵上, 顶点 i 的出度就是第 i 行中 1 的个数, 入度就是第 i 列中 1 的个数, 第 i 行中 1 的个数为 $\sum_{j=1}^n A[i, j]$, 第 i 列中 1 的个数为 $\sum_{j=1}^n A[j, i]$, 因此本题选 D。

15. A。本题考查拓扑排序的排序过程。这类题的解法是, 根据题目中给出的顶点和边的关系画出图, 然后按照拓扑排序的过程算出拓扑排序序列。要注意, 拓扑排序序列不一定唯一, 因此需要结合题目, 在排序过程中排除错误选项, 这样能提高做题速度。

16. B。本题考查关键路径的性质。关键活动组成了关键路径, 关键路径是图中的最长路径, 关键路径长度代表整个工期的最短完成时间。关键活动延期完成, 必将导致关键路径长度增加, 即整个工期的最短完成时间增加, 因此 A 选项正确。关键路径并不唯一, 当有多条关键路径存在时, 其中一条关键路径上的关键活动时间缩短, 只能导致本条关键路径变成非关键路径, 而无法缩短整个工期, 因为其他关键路径没有变化, 因此 B 选项不正确。对于 A、B 两选项要搞懂的是, 任何一条关键路径上的关键活动变长了, 都会使这条关键路径变成更长的关键路径, 并且导致其他关键路径变成非关键路径 (如果关键路径不唯一), 因此整个工期延长。而某些关键活动缩短, 则不一定缩短整个工期。理解了 A、B 两选项, C、D 两选项就很容易理解了。

17. D。本题考查图的邻接矩阵存储结构。有向图的邻接矩阵, 中 0 和 ∞ 表示的都不是有向边, 而入度是由邻接矩阵的列中元素计算出来的。某一列中, 非 0 和非 ∞ 元素的个数即为该列所表示的顶点的入度。

18. B。本题考查以邻接表为存储结构的图的深度优先搜索遍历。本题遍历过程如: 下起点为 1, 1 先输出; 与 1 的第一个邻接点为 2, 2 输出; 与 2 的第一个邻接点为 3, 3 输出; 与 3 的第一个邻接点为 5, 5 输出; 与 5 的第一个邻接点为 4, 4 输出; 所有顶点遍历完全。因此遍历序列为 1, 2, 3, 5, 4。

注意: 此题不可以将图画出后进行深度优先搜索遍历, 然后找出答案。因一般图中对于每个顶点的邻接点是没有次序规定的, 如图中顶点 1 的 3 个邻接点 2、3、4, 反映在一般的图中, 没有先

后次序之分, 当访问完顶点 1 之后, 可任选 2、3、4 之中的一个进行访问。图用邻接表进行存储之后, 则对顶点的邻接点进行了次序的规定。如图在进行遍历时, 访问完顶点 1 之后必须访问顶点 2, 而不能先访问顶点 3 或 4。由此还可以知道, 对于一般的图, 深度优先搜索遍历序列不一定是唯一的, 但如果图用邻接表进行存储, 则其深度优先搜索遍历序列则被唯一确定。

(二) 综合应用题

1. 下面是一种称为“破圈法”的求解最小生成树的方法:

所谓“破圈法”, 是指“任取一圈, 去掉圈上权最大的边”, 反复执行这一步骤, 直到没有圈为止。

试判断这种方法是否正确。若正确, 说明理由; 若不正确, 举出反例 (注: 圈就是回路)。

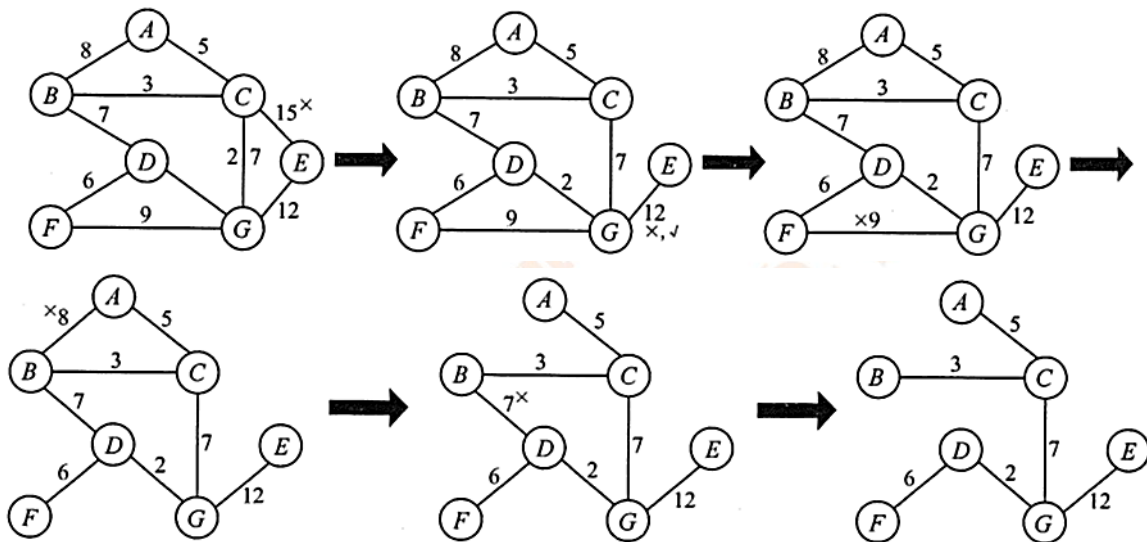
2. 试说明利用 DFS 如何实现有向无环图拓扑排序。

3. 一连通无向图, 边非负权值, 问用 Dijkstra 最短路径算法能否给出一棵生成树, 该树是否一定是最小生成树? 说明理由。

【答案解析】

1 解答: 这种方法是正确的。

由于经过“破圈法”之后, 最终没有回路, 故一定可以构造出一棵生成树。下面证明这棵生成树是最小生成树。记“破圈法”生成的树为 T , 假设 T 不是最小生成树, 则必然存在最小生成树 T_0 , 使得它与 T 的公共边尽可能地多, 则将 T_0 与 T 取并集, 得到一个图, 此图中必然存在回路, 由于“破圈法”的定义就是从回路中去除权最大的边, 此时生成的 T 的权必然是最小的, 这与原假设 T 不是最小生成树矛盾, 从而 T 是最小生成树。下图说明了“破圈法”的过程:



2. 解答: 对于有向无环图 G 中的任意结点 u, v , 它们之间的关系必然是下列三种之一:

1) 假设结点 u 是结点 v 的祖先, 则在调用 DFS 访问 u 的过程中, 必然会在这个过程结束之前递归地对 v 调用 DFS 访问, 即 v 的 DFS 函数结束时间先于 u 的 DFS 结束时间。从而可以考虑在 DFS 调用过程中设定一个时间标记, 在 DFS 调用结束时, 对各结点计时。因此, 祖先的结束

时间必然大于子孙的结束时间。

2) 若 u 是结点 v 的子孙, 则 v 为 u 的祖先, 按上述思路, v 的结束时间大于 u 的结束时间。

3) 若 u 和 v 没有关系, 则 u 和 v 在拓扑序列的关系任意。从而按结束时间从大到小, 可以得到一个拓扑序列。

下面给出利用 DFS 求各结点结束时间的代码。至于拓扑序列, 将结束时间从大到小排序即可得到 (实际上和深度优先遍历算法完全相同, 只不过加入了 time 变量)。

```
bool visited[MAX_VERTEX_NUM];           // 访问标记数组
void DFS_Traverse(Graph G) {             // 对图 G 进行遍历, 访问函数为 visit()
    for (v = 0; v < G.vexnum; ++v)
        visited[v] = FALSE;             // 初始化访问标记数组
    time = 0;
    for (v = 0; v < G.vexnum; ++v)       // 本代码中是从 v=0 开始遍历
        if (!visited[v]) DFS(G, v);
}
void DFS(Graph G, int v) {
    visited[v] = TRUE;
    visit(v);
    for (w = FirstNeighbor(G, v); w >= 0; w = NextNeighbor(G, v, w))
        if (!visited[w])                // w 为 u 的尚未访问的邻接顶点
            DFS(G, w);
    time = time + 1;
    finishTime[v] = time;
}
```

3. 解答: 不一定。参考下面的对比后, 请考生自己举出反例。可以对比 Dijkstra 算法和 Prim 算法 (基于邻接矩阵 $G[N][N]$)。

不同的地方在中间两栏的更新 Min 算法处, 其余地方一致。

```
bool closed[N] = {false};                // 将 closed 表初始置为 false
int Min[N] = {INF};                      /* 对应 Dijkstra 中的从 start 点出发到其余各点的最短路径或加入
Prim 算法中最小生成树的边。初始化的时候, 都置为正无穷。*/
closed[start] = true;                    // 表示从 start 结点出发执行 Dijkstra 算法或 Prim 算法
Min[start] = 0;
for (int i = 1; i < N; i++) {             // 执行 N-1 次, 即开始连接其余的 N-1 个结点
    int k = -1;                          /* 保存尚未求解出的结点中与起点距离最短的结点或者到已求出来的
最小生成树中距离最小的那个结点*/
    for (int j = 0; j < N; j++) {
        if (!closed[j] && (k == -1 || Min[k] > Min[j]))
            k = j;
    }
    closed[k] = true;
    // 得到了 k, 这里考虑了图是连通的, 所以认为 k 一定是存在的, 而不加判定条件
    for (int j = 0; j < N; j++) {
        // Dijkstra 算法对应的更新 Min 算法
        if (Min[j] > Min[k] + G[k][j])
            Min[j] = Min[k] + G[k][j];
        // Prim 算法对应的更新 Min 算法
        if (Min[j] > G[k][j])
            Min[j] = G[k][j];
    }
}
```

6.2 算法专题

6.2.1 图的存储专题

1. 写出从图的邻接表表示转换成邻接矩阵表示的算法。
2. 写出从图的邻接表表示转换成邻接矩阵表示的算法，图为无权图。

【答案解析】

1. 解答：算法的基本思想：设图的顶点分别存储在数组 $v[n]$ 中。首先初始化邻接矩阵。遍历邻接表，在依次遍历顶点 $V[i]$ 的边链表时，修改邻接矩阵的第 i 行的元素值。若链表边结点的值为 j ，则置 $arcs[i][j]=1$ 。遍历完邻接表时，整个转换过程结束。此算法对于无向图、有向图均适用。算法实现如下：

```
void Convert(ALGraph &G, int arcs[M][N]) { //此算法将邻接表方式表示的图 G 转换为邻接矩阵 arcs
    for (i = 0; i < n; i++) {                // 依次遍历各项点表结点为头的边链表
        p = (G->v[i]).firstarc;              // 取出顶点 i 的第一条出边
        while (p != NULL) {                  // 遍历边链表
            arcs[i][p->data] = 1;             // 将对应的邻接矩阵元素置为 1
            p = p->nextarc;                   // 取下一条出边
        }
    }
}
```

2、分析：本题只需扫描邻接表中所有的边结点，然后根据其所连接的顶点号，将邻接矩阵对应位置设为 1 即可。

由此可写出以下算法代码：

```
void AGraphToMGraph(MGraph &gl, AGraph* g2) {
    ArcNode *p;
    int i, j;
    for(i=0; i<gl.n; ++i) /*这个双重循环将 gl 的邻接矩阵清零*/
        for(j=0; j<gl.n; ++j)
            gl.edges[i][j]=0;
    for(i=0; i<g2->n; ++i) /*这个循环实现了用 p 指针扫描邻接表中的所有边结点*/
        p=g2->adjlist[i].firstarc;
        while(p) {
            gl.edges[i][p->adjvex]=1;
            p=p->nextarc;
        }
    gl.n=g2->n;
    gl.e=g2->e;
}
```

6.2.2 图的遍历

1. 试设计一个算法，判断一个无向图 G 是否为一棵树。若是一棵树，则算法返回 true，否则返回 false。
2. 写出图的深度优先搜索 DFS 算法的非递归算法（图采用邻接表形式）。

3. 分别采用基于深度优先遍历和广度优先遍历算法判别以邻接表方式存储的有向图中是否存在由顶点 V_i 到顶点 V_j 的路径 ($i \neq j$)。注意, 算法中涉及的图的基本操作必须在此存储结构上实现。
4. 假设图用邻接表表示, 设计一个算法, 输出从顶点 E 到顶点 E 的所有简单路径。
5. 请回答关于图的下列问题:
 - (1) 有 n 个顶点的有向强连通图最多有多少条边? 最少有多少条边?
 - (2) 对于一个有向图, 不用拓扑排序, 实现判断图中是否存在经过给定顶点 V_0 的环的算法。
6. 写出以邻接表为存储结构的图的深度优先搜索 DFS 算法的非递归算法。

【答案解析】

1. 解答: 一个无向图 G 是一棵树的条件是, G 必须是无回路的连通图或有 $n - 1$ 条边的连通图。这里采用后者作为判断条件。对连通的判定, 可用能否遍历全部顶点来实现。可以采用深度优先搜索算法在遍历图的过程中统计可能访问到的顶点个数和边的条数, 若一次遍历就能访问到 n 个顶点和 $n - 1$ 条边, 则可断定此图是一棵树。算法实现如下:

```
bool isTree(Graph &G) {
    for (i = 1; i <= G.vexnum; i++)
        visited[i] = FALSE;           // 访问标记 visited[] 初始化
    int Vnum = 0, Enum = 0;           // 记录顶点数和边数
    DFS(G, 1, Vnum, Enum, visited);
    if (Vnum == G.vexnum && Enum == 2 * (G.vexnum - 1)) // 符合树的条件
        return true;
    else
        return false;
}                                     // 不符合树的条件

void DFS(Graph &G, int v, int &Vnum, int &Enum, int visited[]) {
    // 深度优先遍历图 G, 统计访问过的顶点数和边数, 通过 Vnum 和 Enum 返回
    visited[v] = TRUE;                // 作访问标记, 顶点计数
    Vnum++;
    int w = FirstNeighbor(G, v);      // 取 v 的第一个邻接顶点
    while (w != -1) {                 // 当邻接顶点存在
        Enum++;                       // 边存在, 边计数
        if (!visited[w])              // 当该邻接顶点未访问过
            DFS(G, w, Vnum, Enum, visited);
        w = NextNeighbor(G, v, w);
    }
}
```

2. 解答: 在深度优先搜索的非递归算法中使用了一个栈 S 来记忆下一步可能访问的顶点, 同时使用了一个访问标记数组 $visited[i]$ 来记忆第 i 个顶点是否在栈内或曾经在栈内, 若是则它以后不能再进栈。图采用邻接表形式, 算法的实现如下:

```
void DFS_Non_RC(AGraph &G, int v) {
    // 从顶点 v 开始进行深度优先搜索, 一次遍历一个连通分量的所有顶点
    int w;                               // 顶点序号
    InitStack(S);                         // 初始化栈 S
    for (i = 0; i < G.vexnum; i++)
        visited[i] = FALSE;              // 初始化 visited
```

```

Push(S, v); visited[v] = TRUE;           // v 入栈并置 visited[v]
while (!IsEmpty(S)) {
    k = Pop(S);                          // 栈中退出一个顶点
    visit(k);                            // 先访问, 再将其子结点入栈
    for (w = FirstNeighbor(G, k); w >= 0; w = NextNeighbor(G, k, w)) { // k 所有邻接点
        if (!visited[w]) {               // 未进过栈的顶点进栈
            Push(S, w);
            visited[w] = true;           // 作标记, 以免再次入栈
        } // if
    } // for
} // while
} // DFS_Non_RC

```

注意: 由于使用了栈, 使得遍历的方式从右端到左端进行, 不同于常规的从左端到右端, 但仍然是深度优先遍历, 读者可以用实例模拟验证。

3. 解答: 两个不同的遍历算法都采用从顶点 v_i 出发, 依次遍历图中每个顶点, 直到搜索到顶点 V_j , 若能够搜索到 V_j , 则说明存在由顶点 V_i 到顶点 V_j 的路径。

深度优先遍历算法的实现如下:

```

int visited[MAXSIZE] = {0};             // 访问标记数组
void DFS(ALGraph G, int i, int j, bool &can_reach) {
    // 深度优先判断有向图 G 中顶点  $v_i$  到顶点  $v_j$  是否有路径, 用 can_reach 来标识
    if (i == j) {
        can_reach = true;
        return;                          // i 就是 j
    }
    visited[i] = 1;                      // 置访问标记
    for (int p = FirstNeighbor(G, i); p >= 0; p = NextNeighbor(G, i, p)) {
        if (!visited[p] && !can_reach)    // 递归检测邻接点
            DFS(G, p, j, can_reach);
    }
}

```

广度优先遍历算法的实现如下:

```

int visited[MAXSIZE] = {0};             // 访问标记数组
int BFS(ALGraph G, int i, int j) {
    // 广度优先判断有向图 G 中顶点  $v_i$  到顶点  $v_j$  是否有路径, 是则返回 1, 否则返回 0
    InitQueue(Q); // 初始化队列 Q
    EnQueue(Q, i);                      // 顶点 i 入队
    while (!IsEmpty(Q)) {                // 非空循环
        DeQueue(Q, u);                  // 队头顶点出队
        visited[u] = 1;                 // 置访问标记
        if (u == j) return 1;           // 如果 u 就是 j, 则查找成功
        for (int p = FirstNeighbor(G, u); p >= 0; p = NextNeighbor(G, u, p)) { // 检查所有邻接点
            if (p == j) return 1;        // 若 p == j, 则查找成功
            if (!visited[p]) {           // 否则, 顶点 p 入队
                EnQueue(Q, p);
                visited[p] = 1;
            }
        } // for
    } // while
    return 0; // 如果没有找到路径, 则返回 0
}

```

本题也可以这样解答: 调用以 i 为参数的 DFS (G, i) 或 BFS (G, i), 执行结束后判断 visited[j]

是否为 TRUE, 若是则说明 v_j 已被遍历, 图中必存在由 v_i 到 v_j 的路径。但此种解法每次都耗费最坏时间复杂度对应的时间, 需要遍历与 v_i 连通的所有顶点。

4. 解答: 本题采用基于递归的深度优先遍历算法, 从结点 u 出发, 递归深度优先遍历图中结点, 若访问到结点 v , 则输出该搜索路径上的结点。为此, 设置一个 $path$ 数组来存放路径上的结点 (初始为空), d 表示路径长度 (初始为-1)。查找从顶点 u 到 v 的简单路径过程说明如下 (假设查找函数名为 FindPath()):

1) FindPath($G, u, v, path, d$): $d++$; $path[d]=u$; 若找到 u 的未访问过的相邻结点 u_1 , 则继续下去, 否则置 $visited[u]=0$ 并返回。

2) FindPath($G, u_1, v, path, d$): $d++$; $path[d]=u_1$; 若找到 u_1 的未访问过的相邻结点 u_2 , 则继续下去, 否则置 $visited[u_1]=0$ 。

3) 以此类推, 继续上述递归过程, 直到 $u_i=v$, 输出 $path$ 。

算法实现如下:

```
void FindPath(AGraph *G, int u, int v, int path[], int d) {
    int w, i;
    ArcNode *p;
    d++; // 路径长度增 1
    path[d] = u; // 将当前顶点添加到路径中
    visited[u] = 1; // 置已访问标记
    if (u == v) // 找到一条路径则输出
        print(path[]); // 输出路径上的结点
    p = G->adjlist[u].firstarc; // p 指向 u 的第一个相邻点
    while (p != NULL) {
        w = p->adjvex; // 若顶点 w 未访问, 递归访问它
        if (visited[w] == 0)
            FindPath(G, w, v, path, d);
        p = p->nextarc; // p 指向 u 的下一个相邻点
    }
    visited[u] = 0; // 恢复环境, 使该顶点可重新使用
}
```

5、分析: (1) 强连通图的定义: 图中任意两个顶点 V_i 和 V_j 之间都连通, 即从 V_i 到 V_j 和从 V_j 到 V_i 都有路径, 则称图为强连通图。在保证图是连通的情况下, n 个顶点的有向图中, 边最多的情况就是每两个顶点之间都有两条边, 最少的情况是图中所有顶点通过边串成一个环。

答: 最多有 $n(n-1)$ 条边, 最少有 n 条边。

(2) 由深度优先搜索算法可知, 在一个顶点 v 的所有边都被检查过之后, 则要返回顶点 v , 进而退回上一个顶点, 重新开始检查上一个顶点的其他边。假如图中不存在回路, 则在 v 处遍历 v 的其他邻接顶点的过程结束前, 不会出现一条指向 v 的边, 可以通过检查是否有这样一条边来判断是否存在回路, 实现算法如下:

```
bool DFSForCircle(AGraph* G, int v, bool visited[]) {
    bool flag;
    ArcNode* p;
    visited[v] = true;
    for (p = G->adjList[v].first; p != NULL; p = p->next) {
        if (visited[p->adjV] == true)
            return true;
        else
            flag = DFSForCircle(G, p->adjV, visited);
    }
}
```

```

        if(flag==true) return true;
        visited[p->adjV] = 0;
    }
    return false;
}

```

// 递归返回时抹除遍历痕迹

函数调用之前 visited []数组已被全部初始化为 0。

调用格式:

isCircle =DFSFor Circle (&AG, v0, visited);

其中 isCircle 是定义好的 bool 型变量, 用来接收函数返回结果, 返回 false 代表无环, 返回 true 代表有环。

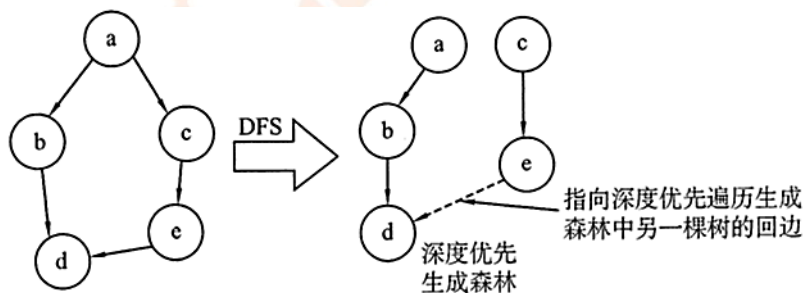
说明:

1) 对于无向图来说, 若深度优先遍历过程中遇到了回边, 即检测到 visit[i]不为 0, 此时可能有两种情况:

①存在环。

②存在双向边, 无向图中任一条无向边 $\langle a, b \rangle$ 都可以等效为存在两条有向边 $\langle a, b \rangle$ 和 $\langle b, a \rangle$, 此时如果题目规定双向边不构成环, 即 $a \rightarrow b \rightarrow a$ 不是环时, 则需要将其过滤掉, 具体可以这么做: 访问某个顶点 a 时, 给它编号为 i, 给它的孩子结点 b 编号 i+1 (经过一条边从 a 访问到 b, 称 b 为 a 的孩子结点, b 从未访问过), 如果访问过程中来到了一个结点 c, 即将访问它的 孩子结点 d, 而 d 已经被访问过, 则判断 c 的编号是否等于 d 的编号值+1, 如果相等则证明是双向边, 把它过滤掉即可。

2) 对于有向图有特殊情况, 当检测到“回边”的时候, 这条“回边”可能是指向深度优先生成森林中另一棵树上的顶点的边。例如, 下图所示即为由深度优先遍历产生一个深度优先生成森林, 森林中有两棵树。虽然图中没有回路, 但是在遍历过程中, 当遍历到顶点 e 时同样会检测到“回边” $\langle e, d \rangle$, 即指向生成森林中另一棵树的“回边”, 但不是真正的回边, 不存在环, 因此要正确地判断是否有环, 需要在算法中排除这种情况, 本题答案注释中的“递归返回时抹除遍历痕迹”即为排除这种情况的操作。



补充: 这里要说一下深度优先搜索生成树和生成森林。对于一个图, 如上图中的左图, 若将边 $\langle e, d \rangle$ 删去, 则可以看作原图的一个深度优先搜索生成树, 这是遍历整个图的结果。如果以每一次 DFS 为单位都做一棵生成树, 则原图就会产生一个深度优先生成森林, 如上图中的右图。

6 分析: 递归的本质就是调用了系统已有的栈来帮助我们解决问题, 题目要求用非递归来完成 DFS 算法, 因此本题关键在于自己写一个栈来代替系统栈, 除此之外还要知道栈在 DFS 算法中到底起什么作用。为了说明这点, 可以看如图 7-51 所示的例子, 图中反映了在以顶点 1 为起点的 DFS 算法中, 游历图中各顶点的过程。从顶点 1 开始, 沿着路线①一直走到顶点 4, 然后沿着路线②走到顶点 5, 最后沿着路线③返回到起点。在这个过程中, 每当来到一个新顶点的时候要去做两件事情: 第一, 对这个顶点访问 (如输出顶点等操作), 并且将顶点标记为已访问; 第二, 看看有没有可以通往下一个顶点的边 (这里指的是所指顶点没有被访问过的边), 如果有则找出一条, 沿着这条边走

向下一个顶点, 如果没有则原路返回。

如果当前被访问的顶点还有其他边可以通往其他顶点, 这就说明即便是已经沿着选出的边走向另一个顶点, 对于当前顶点的处理依然没有完成。如图 7-51 中的顶点 2, 即便已经离开了顶点 2 走向了顶点 3, 但是对于顶点 2 的另一个邻接顶点顶点 5 还没有访问。所以必须记下当前顶点的位置, 待之后的顶点处理完后再返回来处理, 这就要用到栈。当问题执行到一个状态, 以现有的条件无法完全解决时, 必须先记下当前状态, 然后继续往下进行, 等条件成熟之后再返回来解决。这一类的问题可以用栈来解决, 这一点之前已经重点讲解过。

由此可以写出以下算法代码:

```
void DFSI(AGraph *g,int v) {
    ArcNode *p;
    int stack[maxSize], top=-1;           //定义一个栈来记录访问过程中的顶点
    int i,k;
    int visit[maxSize];                   //顶点访问标记数组
    for(i=0;i<g->n;++i)
        visit[i]=0;
    Visit(v);                             //假设此函数已经声明, 包含对顶点访问的各种操作
    visit[v]=1;                           //标记起始顶点已被访问
    stack[++top]=v;                       //起始顶点入栈
    /*以下是本算法的关键部分*/
    while (top!=-1) {
        k=stack[top];                     //取栈顶元素
        p=g->adjlist[k].firstarc;         //p 指向该顶点的第一条边
        /*下面这个循环是 p 沿着边行走并 将图中经过的顶点入栈的过程*/
        while (p!=NULL&&visit [p->adjvex] ==1)
            p=p->nextarc;                 /*找到当前顶点第一个没访问过的邻接顶点或者 p 走到当前链表尾部时, while 循环停止*/
        if(p==NULL)                      //如果 p 到达当前链表尾部, 则说明当前顶点的所有点都
            --top;                        //访问完毕, 当前顶点出栈
        else{                             //否则访问当前邻接点并入栈
            Visit(p->adjvex);
            visit[p->adjvex]=1;
            stack[++top]=p->adjvex;
        }
    }
}
```

6.2.3 出度和入度的计算

1. 设有向图用邻接表表示, 图有 n 个顶点, 图采用简化的表示方法, 顶点信息与其在数组中的下标相同, 表示为 $0 \sim n-1$, 试写一个算法求顶点 k 的入度。

【答案解析】

1、分析: 要求顶点 k 的入度, 必须检查表中所有的边结点, 看边结点所指顶点是否为 k , 若是则证明 K 有一个入度, 做一次统计。因此, 还要设置一个计数器 sum 来统计 k 的入度。

由以上分析可写出如下算法代码:

```
int count (AGraph* g , int k) {
    ArcNode *p;                          //p 指针用来扫描每个顶点所发出的边
    int i,sum;
    sum=0;                                //计数器清零
    for(i=0;i<g->n;++i) {
```

```
p=g->adjlist[i].firstarc c ;
while(p!=NULL) {
    if(p->adjvex==k){ //若顶点 i 所发出的边的另一端是顶点 k,则计数器加 1
        ++sum;
        break;
    }
    p=p->nextarc;
}
return sum; //返回 k 的入度
}
```

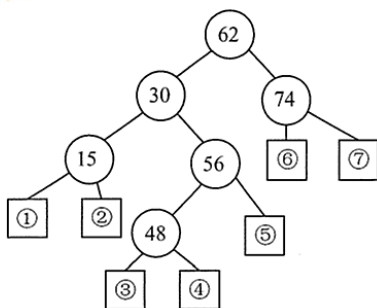
说明：在函数的参数定义中，如 $f(\text{变量类型变量名})$ ，当调用 $f(a)$ 的时候（将 a 作为参数传入 f ）， f 函数会在内部复制一个 a 出来，函数内所有对 a 的操作实际上都施加在了 a 的副本上。这样如果 a 是构造得比较大的结构体类型，则每次执行 f 时所伴随的对 a 的复制操作将是很大的一笔开销。因此对于函数参数的定义，我们可以定义成 a 变量同类型的指针型参数，调用函数的格式为 $f(\&a)$ ；或者定义成 a 变量的引用型，调用函数的格式为 $f(a)$ 。就可以避免对图这种大型结构体的复制操作，提高了效率。

7、查找

7.1 综合练习

(一) 选择题

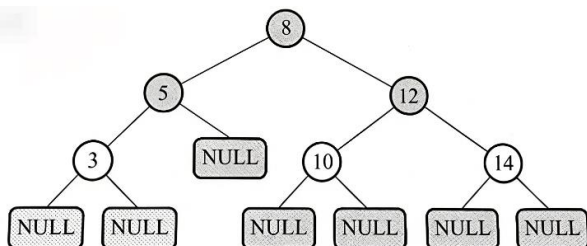
- 对长度为 3 的顺序表进行查找, 若查找第一个元素的概率为 $1/2$, 查找第二个元素的概率为 $1/3$, 查找第三个元素的概率为 $1/6$, 则查找任一元素的平均查找长度为 ()。
A. $5/3$ B. 2 C. $7/3$ D. $4/3$
- 设顺序存储的某线性表共有 123 个元素, 按分块查找的要求等分为 3 块。若对索引表采用顺序查找法来确定子块, 且在确定的子块中也采用顺序查找法, 则在等概率情况下, 分块查找成功的平均查找长度为 ()。
A. 21 B. 23 C. 41 D. 62
- 为提高查找效率, 对有 65025 个元素的有序顺序表建立索引顺序结构, 在最好情况下查找到表中已有元素最多需要执行 () 次关键字比较。
A. 10 B. 14 C. 16 D. 21
- 具有 12 个关键字的有序表, 对每个关键字的查找概率相同, 折半查找成功的平均查找长度 ASL 为 ()。
A. $37/12$ B. $35/12$ C. $39/12$ D. $43/12$
- 具有 12 个关键字的有序表中, 对每个关键字的查找概率相同, 折半查找查找失败的平均查找长度为 ()。
A. $37/12$ B. $35/12$ C. $39/13$ D. $49/13$
- 设二叉排序树中关键字由 1 到 1000 的整数构成, 现要查找关键字为 363 的结点, 下述关键字序列中, 不可能是在二叉排序树上查找的序列是 ()。
A. 2, 252, 401, 398, 330, 344, 397, 363 B. 924, 220, 911, 244, 898, 258, 362, 363
C. 925, 202, 911, 240, 912, 245, 363 D. 2, 399, 387, 219, 266, 382, 381, 278, 363
- 分别用以下序列构造二叉排序树, 与用其他 3 个序列所构造的结果不同的是 ()。
A. {100, 80, 90, 60, 120, 110, 130} B. {100, 120, 110, 130, 80, 60, 90}
C. {100, 60, 80, 90, 120, 110, 130} D. {100, 80, 60, 90, 120, 130, 110}
- 在平衡二叉树中插入一个结点后造成了不平衡, 设最小不平衡子树根为 A, 并已知 A 的左孩子的平衡因子为 0, 右孩子的平衡因子为 1, 则应做 () 型调整以使其平衡。
A. LL B. LR C. RL D. RR
- 在一棵二叉排序树上, 查找关键字为 35 的结点, 依次比较的关键字有可能是 ()。
A. 28, 36, 18, 46, 35 B. 18, 36, 28, 46, 35 C. 46, 28, 18, 36, 35 D. 46, 36, 18, 28, 35
- 下图所示的二叉排序树查找成功时的平均查找长度为 (), 查找不成功时的平均查找长度为 (只将与关键字的比较次数计算在内) ()。



- A. 21/7 B. 28/7 C. 15/6 D. 21/6
11. 高度为 5 的 3 阶 B 树至少有 () 个结点, 至多有 () 个结点。
A. 32 B. 31 C. 120 D. 121
12. 已知一棵 5 阶 B 树中共有 53 个关键字, 则树的最大高度为 (), 最小高度为 ()。
A. 2 B. 3 C. 4 D. 5
13. 已知一棵 3 阶 B 树中有 2047 个关键字, 则此 B 树的最大高度为 (), 最小高度为 ()。
A. 11 B. 10 C. 8 D. 7
14. 关于 Hash 查找以下说法不正确的有 () 个。
①采用链地址法解决冲突时, 查找任一个元素的时间都是相同的
②采用链地址法解决冲突时, 若插入规定总是在链首, 则插入任一个元素的时间都是相同的
③用链地址法解决冲突易引起堆积现象
④线性探查法不易产生堆积现象
A. 1 B. 2 C. 3 D. 4
15. 设 Hash 表长为 14, Hash 函数是 $H(\text{key}) = \text{key} \% 11$ 表中已有数据的关键字为 15、27、39、51, 共 4 个, 现要将关键字为 49 的结点添加到表中, 用二次探测再散列法解决冲突, 则放入的位置是 ()。
A. 8 B. 3 C. 5 D. 9
16. 在用 KMP 算法进行模式匹配时, 模式串 "ababaaababaa" 的 next 数组值为 ()。
A. 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 9, 9 B. 0, 1, 2, 1, 2, 1, 1, 1, 2, 1, 2
C. 0, 1, 1, 2, 3, 4, 2, 2, 3, 4, 5, 6 D. 0, 1, 2, 3, 0, 1, 2, 3, 2, 2, 3, 4
17. 在用 KMP 算法进行模式匹配时, 模式串 "ababaaababaa" 的 nextval 数组值为 ()。
A. 0, 1, 0, 1, 0, 4, 2, 1, 0, 1, 0, 4 B. 0, 1, 0, 1, 1, 4, 1, 1, 0, 1, 0, 2
C. 0, 1, 0, 1, 1, 2, 0, 1, 0, 1, 0, 2 D. 0, 1, 1, 1, 0, 2, 1, 1, 0, 1, 0, 4
18. 下列说法中正确的是 ()。
A. 先序遍历二叉排序树的结点就可以得到排好序的结点序列
B. 任一二叉排序树的平均查找时间都小于顺序查找法查找同样结点的线性表的平均查找时间
C. 对具有相同关键字集合的任一插入序列, 得到的二叉排序树的形态都是相同的
D. 采用分块查找方法, 既能实现较快地查找线性表, 又能适应动态变化的要求
19. 在构造散列表方面, 下列说法中正确的是 ()。
A. 链地址法在处理冲突时会产生聚集 B. 线性探测再散列法在处理冲突时会产生聚集
C. 好的哈希函数可以完全避免冲突 D. 在哈希表中进行查找是不需要关键字的比较的
20. 下列关于查找的说法中, 错误的是 ()。
I. 如果线性表的数据元素保持有序, 则搜索时就可以采用折半查找法;
II. 折半查找与二叉查找树的时间复杂度在最坏的情况下是相同的;
III. 用向量和单链表表示的有序表均可使用折半查找法来提高查找速度;
IV. 在一维数组中有 n 个数, 进行顺序查找时, 这 n 个数的排列有序或无序其平均查找长度不同。
A. I、II 和 III B. I、II、III 和 IV C. III 和 IV D. I、II 和 IV
21. 下列关于二叉树的说法中, 错误的是 ()。
I. 对一个堆, 按二叉树层次进行遍历可以得到一个有序序列;
II. 以同一组数的不同顺序来构造平衡二叉树, 可能会得到不同的树;
III. 向平衡二叉树中插入新结点 p , 若结点 p 的父结点 q 原来的平衡因子不为 0, 则必引起平衡旋转;
IV. 平衡二叉树中, 若某个结点的左孩子和右孩子的平衡因子均为 0, 则该结点的平衡因子一定是 0。
A. I、III 和 IV B. III 和 IV C. II 和 III D. I、II 和 IV

22. 如下图所示的红黑树中将结点 2 插入到 3 的左孩子且染成红色后, 下一步操作是 ()。

- A. 左旋 B. 右旋 C. 变色 D. 无须调整



23. 红黑树是一种比较平衡的二叉搜索树, 它的平衡是指 ()。

- A. 左子树和右子树的高度差最多为 1 B. 左子树和右子树的结点个数相等
C. 红色结点和黑色结点的个数相等 D. 从根结点到叶结点的每条路径上, 黑色结点个数相等

24. 下列关于红黑树的描述中, 错误的是 ()。

- A. 从某结点到其后代叶结点的所有简单路径中, 最长的一条是最短一条的至多两倍
B. 从根到叶结点 (不包含根) 的任一条简单路径上至少有一半的结点必是黑色的
C. 从任意结点出发的所有下降到叶结点路径都有相同的红结点个数
D. 在红黑树上的查找操作可以在 $O(\log n)$ 时间内完成

25. 假设散列表长度为 11, 散列函数 $H(\text{key}) = \text{key} \% 7$, 冲突处理方法为二次探测再散列法。按关键字序列 (32, 17, 9, 27) 构造散列表, 现要将关键字为 45 的记录加入散列表中, 则放入的位置是 ()。

- A. 1 B. 0 C. 5 D. 7

【答案解析】

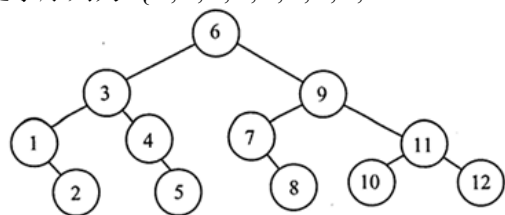
1.A。在长度为 3 的顺序表中, 查找第一个元素的查找长度为 1, 查找第二个元素的查找长度为 2, 查找第三个元素的查找长度为 3, 故有

$$ASL_{\text{成功}} = \frac{1}{2} \times 1 + \frac{1}{3} \times 2 + \frac{1}{6} \times 3 = \frac{5}{3}$$

2.B。根据公式 $ASL = L1 + LS = \frac{b+1}{2} + \frac{s+1}{2} = \frac{s^2+2s+n}{2s}$, 其中 $b = n/s$, $s = 123/3$, $n = 123$, 代入不难得出 ASL 为 23。故选 B。另一方面, 可根据穷举法来一步步模拟。对于 A 块中的元素, 查找过程的第一步是先找到 A 块, 由于是顺序查找, 找到 A 块只需一步, 然后在 A 块中顺序查找。因此, A 块内各元素查找长度分别为 2, 3, 4, ..., 42。对于 B 块, 采用类似的方法, 但查找到 B 块要比查找到 A 块多一步, 因此 B 块内各元素查找长度为 3, 4, 5, ..., 43。同理, C 块中各个元素查找长度为 4, 5, 6, ..., 44。所以平均查找长度为 $(2+3+4+\dots+42+3+4+5+\dots+43+4+5+6+\dots+44)/123=23$ 。

3.C。为使查找效率最高, 每个索引块的大小应是 $\sqrt{65025}=255$, 为每个块建立索引, 则索引表中索引项的个数为 255。若对索引项和索引块内部都采用折半查找, 则查找效率最高, 为 $\log_2(255+1) + \log_2(255+1)=16$ 。

4.A。本题考查折半查找平均查找长度的计算。我们可以假设关键字序列为 {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12}, 只要表中的关键字个数相同, 就一定会生成形状相同的判定树。这里的形状相同是指两棵树的结构是一样的, 但同一位置上的结点的值可以不一样。判定树只要形状相同, 平均查找长度就相同, 含有 12 个关键字的有序表必然可以生成与右图所示的树形状相同的判定树。



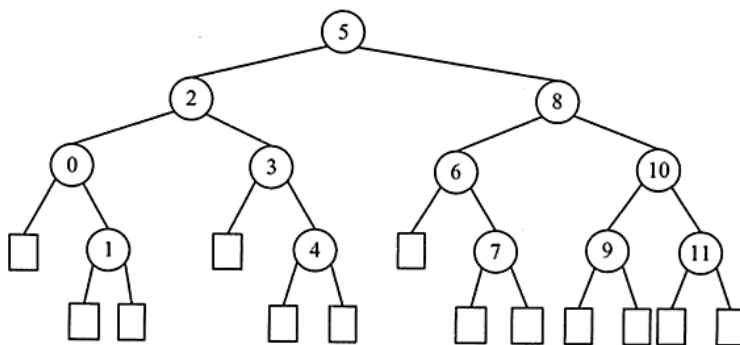
由右图所示的判定树可以得出各关键字的比较次数, 关键字的比较次数即为关键字在树中的层

数。由此可以得出:

关键字	1	2	3	4	5	6	7	8	9	10	11	12
比较次数	3	4	2	3	4	1	3	4	2	4	3	4

因此 $ASL = (3+4+2+3+4+1+3+4+2+4+3+4)/12 = 37/12$, 故本题选 A。

5.D. 假设有序表中元素为 $A[0...11]$, 不难画出对它进行折半查找的判定树如下图所示, 圆圈是查找成功结点, 方形是虚构的查找失败结点。从而可以求出查找失败的 $ASL = (3 \times 3 + 4 \times 10)/13$ 。注意: 对于本类题目, 应先根据所给 n 的值, 画出如右图的折半查找判定树。另外, 查找失败结点的 ASL 不是图中的方形结点, 而是方形结点上层的圆形结点。

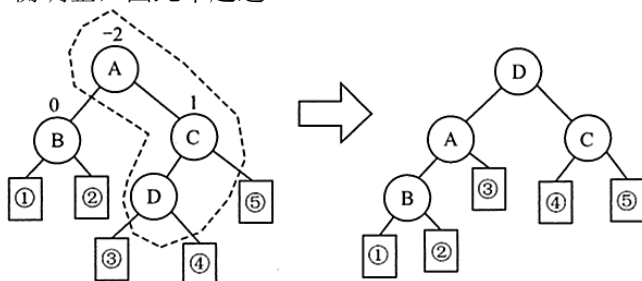


6.C. 在二叉排序树上查找时, 先与根结点值进行比较, 若相同, 则查找结束, 否则根据比较结果, 沿着左子树或右子树向下继续查找。根据二叉排序树的定义, 有左子树结点值三根结点值三右子树结点值。C 序列中, 比较 911 关键字后, 应转向其左子树比较 240, 左子树中不应出现比 911 更大的数值, 但 240 竟有一个右孩子结点值为 912, 所以不可能是正确的序列。

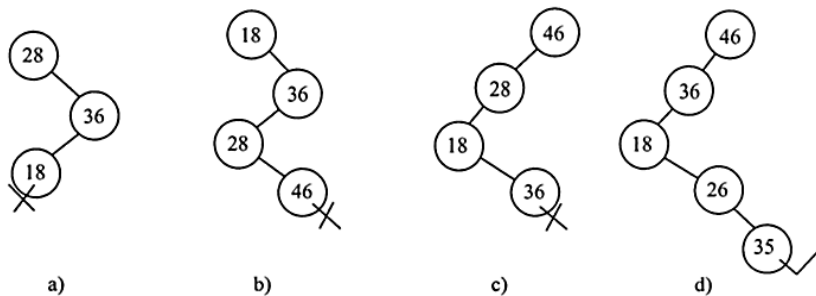
7.C. 对于这类选择题, 其实不需要对每个选项都画出相应的二叉排序树之后再进行比较, 直接观察选项即可。经过观察可知, 选项 A、B、D 中根结点的左孩子结点为 80, 而选项 C 中根结点的左孩子结点为 60, 至此即可判断选项 C 所构造的二叉排序树一定和其他 3 项不同, 因此本题选 C。

8.C. 由题意可知, 在插入前, A 的平衡因子为 -1, A 的右孩子 (下图左图中的结点 C) 的平衡因子为 0; 插入结点后 (新结点插入在 D 的子树上, 图中没有画出) A 的平衡因子变为 -2, C 的变为 1。结点必插入到 C 的左子树上, A、C、D 是需要进行平衡调整的 3 个结点。下图中虚线框内所示的 A、C、D 3 个结点构成的子树需要进行 RL 型平衡调整, 因此本题选 C。

结点 D 中关键字大于 A 小于 C, 因此将 D 作为根, A、C 分别为其左、右子树。此时 A 空出来一个右指针, C 空出来一个左指针, 而③、④变成了两棵独立的树。③中关键字都小于④中关键字, 即在最终树中, ③应该在④的左边。因此, ③应该挂在 A 的右指针上, ④应该挂在 C 的左指针上, 最终结果如右图所示。



9.D. 本题考查二叉排序树的查找过程。可以根据选项画出查找路线上的结点, 根据二叉排序树的规定来排除不满足条件的选项。根据题目选项所得查找路线如下图所示。



选项 A 中 28 的右子树中出现了小于它的 18, 不满足二叉排序树规定, 排除。
选项 B 中 36 的左子树中出现了大于它的 46, 不满足二叉排序树规定, 排除。
选项 C 中 28 的左子树中出现了大于它的 36, 不满足二叉排序树规定, 排除。

因此本题选 D。

10. C, A。本题考查二叉排序树平均查找长度的计算。由题意进行如下分析： 查找成功：

关键字	62	30	74	15	56	48
比较次数	1	2	2	3	3	4

查找成功时的平均查找长度 $= (1+2+2+3+3+4)/6 = 15/6$ 。查找失败：

失败位置	1	2	3	4	5	6	7
比较次数	3	3	4	4	3	2	2

查找失败时的平均查找长度 $= (3+3+4+4+3+2+2)/7 = 21/7$

11. B、D。由 m 阶 B 树的性质可知，根结点至少有 2 棵子树；根结点外的所有非终端结点至少有 $\lceil m/2 \rceil$ 棵子树，结点数最少时，3 阶 B 树形状至少类似于一棵满二叉树，即高度为 5 的 B 树至少有 $2^5 - 1 = 31$ 个结点。由于每个结点最多有 m 棵子树，所以当结点数最多时，3 阶 B 树形状类似于满三叉树，结点数为 $(3^5 - 1)/2 = 121$ (注意，这里求的是结点数而非关键字数，若求的是关键字数，则还应把每个结点中关键字数的上下界确定出来)。

12. C、B。5 阶 B 树中共有 53 个关键字，由最大高度公式 $H \leq \log \lceil m/2 \rceil ((n+1)/2) + 1$ 得最大高度 $H \leq \log_3 [(53+1)/2] + 1 = 4$ ，即最大高度为 4；由最小高度公式 $h \geq \log_m (n+1)$ 得最小高度 $h \geq \log_5 54 = 2.5$ ，从而最小高度为 3。

13. A、D。利用前面的公式即最小高度 $h \geq \log_m (n+1)$ 和最大高度 $H \leq \log \lceil m/2 \rceil [(n+1)/2] + 1$ ，易算出最大高度 $H \leq \log_2 [(2047+1)/2] + 1 = 11$ ，最小高度 $h \geq \log_3 2048 = 6.9$ ，从而最小高度取 7 (注意，有些辅导书针对本题算出的高度要比这里给出的答案多 1，因为它们在对 B 树的高度定义中，把最低层不包含任何关键字的叶结点也算进去了)。

14. C。本题考查 Hash 冲突的处理方法。

如果两个元素在同一个链表中，查找时间肯定不相同，因此也不正确。

若插入规定在链首，则插入操作不需要查找插入位置即可直接进行，因此插入任何一个元素的时间均相同，因此②正确。这是相对于链表的尾插法而言的，在尾插法中，需要找到链表的尾部，因此链表的长度决定了插入元素的执行时间。

所谓堆积问题，即在 Hash 表的建立过程中，某些 Hash 地址是由冲突处理产生的，而不是由 Hash 函数直接产生的，这就可能造成原本 Key_1 与 Key_2 虽然不是同义词，但是最后却得出了相同的 Hash 地址。例如，在线性探查法处理冲突的过程中，设第一个同义词占用单元 d ，这之后连续的若干同义词将占用 Hash 表的 d 、 $d+1$ 、 $d+2$ 等单元，此时随后任何 $d+1$ 、 $d+2$ 等单元上的 Hash 映射都会由于前面的同义词堆积而产生冲突，尽管随后的这些关键字并没有同义词，因此④不正确。

显然链地址法不会产生堆积现象，因为多个同义词只会占用表中的一个地址，因此③不正确。

说明：在考研数据结构的考题中，如果问链地址法会不会产生堆积现象，答案是不会；如果问堆积现象可不可以完全避免，答案是不可以。

15. D。哈希表如下表所示。ADDR(49)=5，关键字 27 保存在下标为 5 的存储单元内，采用二次探测再散列处理冲突， $d_i = +1^2, -1^2, +2^2, -2^2, +3^2$ 等。 $d_1 = 1$ ，下标 $5+1=6$ 所指存储单元保存关键字 39，发生冲突； $d_2 = -1$ ，下标 $5-1=4$ 所指存储单元保存有关键字 15，发生冲突；下标 $5+4=9$ 所指存储单元为空，将关键字 49 存储在下标 9 所指存储单元，关键字 49 的地址是 9。故答案为选项 D。

0	1	2	3	4	5	6	7	8	9	10	11	12	13
				15	27	39	51						

16. C。这道题采用手工求 next 数组的方法。先求串 $S = 'ababaaababa'$ 的部分匹配值：

‘a’的前后缀都为空，最长相等前后缀长度为 0。

‘ab’的前缀 { a } $\cap$ 后缀 { b } = $\emptyset$ ，最长相等前后缀长度为 0。

‘aba’的前缀{ a, ab } $\cap$ 后缀{ a, ba } = {a}, 最长相等前后缀长度为 1。

‘abab’的前缀{ a, ab, aba } $\cap$ 后缀{ b, ab, bab } = { ab }, 最长相等前后缀长度为 2。

依次求出的部分匹配值如下表第三行所示, 将其整体右移一位, 低位用-1 填充, 如下表第四行所示。

	1	2	3	4	5	6	7	8	9	10	11	12
S	a	b	a	b	a	a	a	b	a	b	a	a
PM	0	0	1	2	3	1	1	2	3	4	5	6
next	-1	0	0	1	2	3	1	1	2	3	4	5

选项中 next [1] 等于 0, 故将 next 数组整体加 1, 答案选 C。

17. A。写出模式串、对应字符下标和 next 数组的值, 见表

模式串	a	b	a	b	a	a	a	b	a	b	a	a
j	1	2	3	4	5	6	7	8	9	10	11	12
next	0	1	1	2	3	4	2	2	3	4	5	6

nextval[1]=0, 特殊标记;

P₂ 为 b, P_{next[2]} 为 a, 两者不等, nextval[2]=next[2]=1;

p₃ 为 a, P_{next[3]} 为 a, 两者相等, nextval[3]=nextval[next[3]]=0;

p₄ 为 b, P_{next[4]} 为 b, 两者相等, nextval[4]=nextval[next[4]]=1;

P₅ 为 a, P_{next[5]} 为 a, 两者相等, nextval[5]=nextval[next[5]]=0;

P₆ 为 a, P_{next[6]} 为 b, 两者不等, nextval[6]=next[6]=4;

P₇ 为 a, P_{next[7]} 为 b, 两者不等, nextval[7]=next[7]=2;

P₈ 为 b, P_{next[8]} 为 b, 两者相等, nextval[8]=nextval[next[8]]=1;

P₉ 为 a, P_{next[9]} 为 a, 两者相等, nextval[9]=nextval[next[9]]=0;

P₁₀ 为 b, P_{next[10]} 为 b, 两者相等, nextval[10]=nextval[next[10]]=1;

P₁₁ 为 a, P_{next[11]} 为 a, 两者相等, nextval[11]=nextval[next[11]]=0;

P₁₂ 为 a, P_{next[12]} 为 a, 两者相等, nextval[12]=nextval[next[12]]=4。

在第 5 步的推理中, P₅=P_{next[5]} = a, 按前面的讲解部分, 应该继续让 p₃ 和 P_{next[3]} 比较 (恰好 p₃=P_{next[3]}=1), 注意到此时 nextval[3] 的值已存在, 故直接将 nextval[5] 赋值为 nextval[3]。

说明: 本题存储串的数组下标从 1 开始, 有的题中从 0 开始, 这两种设定下的 next 数组值相差 1。

18.D。二叉排序树按照右根左 (左根右) 的原则构造, 中序遍历可以得到有序序列, 先序遍历序列无序, 因此选项 A 错误。当二叉排序树退化成线性结构, 即树中所有非叶结点只有左孩子或右孩子时, 二叉排序树与线性表的平均查找时间相等, 因此选项 B 错误。不同插入序列会影响二叉排序树的形态: 若插入序列有序, 二叉排序树会退化成线性结构, 与插入序列无序时得到的二叉排序树形态不同, 因此选项 C 错误。分块查找块间有序块内无序, 查找元素时先确定元素所在块, 然后在块内顺序查找, 时间性能较好。由于分块查找对块内元素无排序要求, 能够适应动态变化的要求, 因此选项 D 正确。

19.B。链地址法 (拉链法) 把哈希函数值相等的关键字存储在链表中, 表头指针数组同开放, 不会产生聚集现象, 因此选项 A 的说法错误。线性探测再散列处理冲突时会产生聚集现象, 因此选项 19.B。的说法正确。好的哈希函数应尽可能减少冲突, 并不能完全避免, 因此选项 C 的说法错误。哈希表虽然能尽量节省, 但是由于表项可能对同义词或非同义词同开放故, 导致在查找过程中不可避免地要进行关键字的比较, 因此选项 D 的说法错误。

20.B。折半查找基于有序顺序表的随机访问特性, 每次比较表中间元素和待查元素的相对大小, 同时要求表有序和随机访问, 链表没有随机访问的特性, 不能折半检索有序链表, 因此说法 I 错误。折半查找最坏情况下比较次数为 O(log n), 二叉查找树最坏情况下退化成线性结构, 每个非叶结点只有左孩子或右孩子, 比较次数为 O(n), 因此说法 II 错误。单链表表示的有序表不能使用折半查找方法, 因此说法 III 错误。顺序查找有序和无序数组, 查找成功时的平均查找长度相等, ASL=(1+2+...+n)/n = (n+1)/2; 查找失败时, 无序顺序表每次都要遍历完整个表才停止, 平均查找长度

ASL = $n+1$ ，而有序顺序表遍历到大于待查元素即可停止，平均查找长度 $ASL = (1+2+\dots+n+1+n)/n = (n+1)/2$ ，因此说法 IV 错误。

21.A. 所有双亲结点的值都大于(或小于)孩子结点的值，满足此特性的完全二叉树称为大(小)根堆；堆并没有规定左右孩子的相对大小，层次遍历序列不一定有序，因此说法 I 错误。平衡二叉树的最终形态与构造序列有关，因此说法 II 正确。若新结点插入在平衡因子为 1 的结点右孩子上，插入后平衡因子变为 0，插入没有改变树高，不会引起旋转，因此说法 III 错误。平衡二叉树中，平衡因子 = 左子树高度 - 右子树高度，左、右孩子的平衡因子为 0 只能说明左、右孩子结点的左子树和右子树高度相等，但不能说明该结点的左子树和右子树高度相等，因此说法 IV 错误。

22.B. 插入结点 2 并将其染成红色后违反“不红红”原则，且叔叔结点是黑色，应该按照平衡二叉树的方式旋转，将结点 3 右旋旋转到结点 5 的位置，结点 2 和结点 5 成为结点 3 的左右结点，然后将结点 3 染成黑色，结点 2 和结点 5 染成红色。所以第一步应该是右旋，答案为选项 B。

23.D. 红黑树的性质：从根结点到叶结点的每条路径上，黑色结点个数相等。

24.C. 从某结点到其后代叶结点的所有简单路径中，黑结点出现次数相等，当没有红结点时该路径是最短路径；路径中不能连续出现两个红结点，路径最长时黑、红结点交替出现，最长路径至多为最短路径的 2 倍，选项 A 正确。路径中不能连续出现两个红结点，若黑结点个数小于红结点的 1/2，会出现两个红结点相邻的情况，选项 B 正确。从任意结点出发的所有下降路径都有相同的黑结点个数，并非红结点个数，选项 C 错误。含有 n 个结点的红黑树高度数量级为 $O(\log n)$ ，查找次数最大值与树高相同，查找操作均可以在 $O(\log n)$ 时间内完成，选项 D 正确。

25.D. 二次探测法： $H_i = (H(\text{key}) + d_i) \% 11$ ，其中 $d_i = 0^2, 1^2, -1^2, 2^2, -2^2$ ，将 $\text{key}=5$ 代入， $H(\text{key}) = \text{key} \% 7 = 3$ 。

$H_0 = (3 + 0) \% 11 = 3$ ，与关键字 17 冲突。

$H_1 = (3 + 1^2) \% 11 = 4$ ，与关键字 32 冲突。

$H_2 = (3 - 1^2) \% 11 = 2$ ，与关键字 9 冲突。

$H_3 = (3 + 2^2) \% 11 = 7$ ，不冲突，放入的位置是 7。散列表如下表所示。

地址	0	1	2	3	4	5	6	7	8	9	10	11
关键字序列			9	17	32		27					
查找长度			1	1	1		1					

(二) 综合应用题

1. 已知一个有序顺序表 $A[0 \dots 8n-1]$ 的表长为 $8n$ ，并且表中没有关键字相同的数据元素。假设按下述方法查找一个关键字值等于给定值 X 的数据元素：先在 $A[7], A[15], A[23], \dots, A[8k-1], \dots, A[8n-1]$ 中进行顺序查找，若查找成功，则算法报告成功位置并返回；若不成功，当 $A[8k-1] < X < A[8(k+1)-1]$ 时，则可确定一个缩小的查找范围 $A[8k] \sim A[8(k+1)-2]$ ，然后可在这个范围内执行折半查找。特殊情况：若 $X > A[8n-1]$ 的关键字，则查找失败。

1) 画出描述上述查找过程的判定树。

2) 计算相等查找概率下查找成功的平均查找长度。

2. 线性表中各结点的检索概率不等时，可用如下策略提高顺序检索的效率：若找到指定的结点，则将该结点和其前驱结点（若存在）交换，使得经常被检索的结点尽量位于表的前端。试设计在顺序结构和链式结构的线性表上实现上述策略的顺序检索算法。

3. 已知长度为 11 的表 {xal, wan, wil, zol, yo, xul, yum, wen, wim, zi, yon}, 按表中元素顺序依次插入一棵初始为空的平衡二叉排序树 (单词的大小按字典顺序来决定, 即在字典中靠前的小、靠后的大), 画出插入完成后的平衡二叉树, 并求其在等概率的情况下查找成功时的平均查找长度。

4. 利用 B 树做文件索引时, 若假设磁盘页块的大小是 4000B (实际应是 2 的次幂, 此处是为了计算方便), 指示磁盘地址的指针需要 5B。现有 20000000 个记录构成的文件, 每个记录为 200B, 其中包括关键字 5B。

试问在这个采用 B 树作索引的文件中, B 树的阶数应为多少? 假定文件数据部分未按关键字有序排列, 则索引部分需要占用多少磁盘页块?

5. 使用散列函数 $H(\text{key}) = \text{key} \% 11$, 把一个整数值转换成散列表下标, 现要把数据 {1, 13, 12, 34, 38, 33, 27, 22} 依次插入散列表。

- 1) 使用线性探测法来构造散列表。
- 2) 使用链地址法构造散列表。

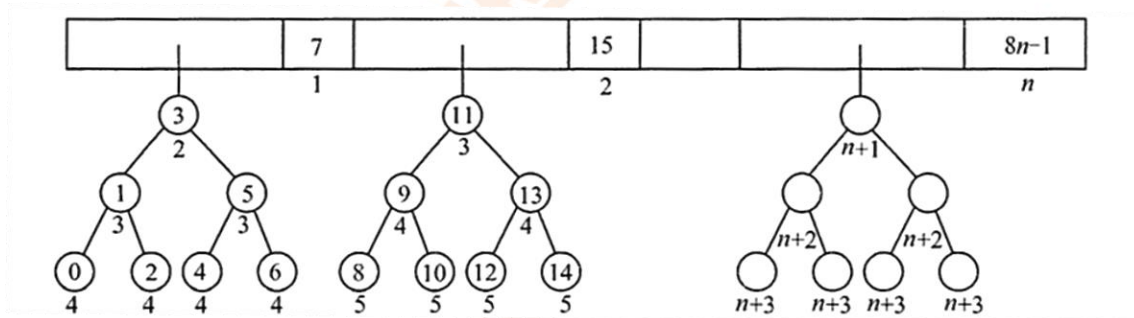
试针对这两种情况, 分别确定查找成功所需的平均查找长度, 及查找不成功所需的平均查找长度。

6. 从空树开始建立 3 阶 B 树, 要求:

- (1) 画出按序列 (58, 36, 50, 20, 54, 95, 30, 45, 40) 插入关键字后的 B 树。
- (2) 对 (1) 中的 B 树, 画出删除 58 和 36 后的 B 树及过程。

【答案解析】

1. 解答: 1) 先在 $A[7], A[15], \dots, A[8n-1]$ 内顺序查找, 再在区间内折半查找。相应的判定树如下图所示。其中, 每个关键字下的数字为其查找成功时的关键字比较次数。



2) 等查找概率下, 平均每个关键字查找成功的概率为 $1/8n$; 0~7 之间的关键字, 顺序比较 1 次后, 进行折半查找, 查找成功的平均查找长度为 $2 + 3 \times 2 + 4 \times 4$; 8~15 之间的关键字, 先顺序比较 2 次后, 再进入折半查找; 以此类推, $8(n-1) \sim 8n-1$ 之间的关键字, 先顺序比较 n 次, 再进入折半查找, 如上图所示。故查找成功的平均查找长度为:

$$\begin{aligned} ASL_{\text{成功}} &= \frac{1}{8n} \sum_{i=0}^{8n-1} Ci = \frac{1}{8n} \left(\sum_{i=1}^n i + \sum_{i=2}^{n+1} i + 2 \sum_{i=3}^{n+2} i + 4 \sum_{i=4}^{n+3} i \right) \\ &= \frac{1}{8n} \left(\sum_{i=1}^n (i + (i+1) + 2(i+2) + 4(i+3)) \right) \\ &= \frac{1}{8n} \left(\sum_{i=1}^n 8i + 17 \right) = \frac{1}{n} \sum_{i=1}^n i + \frac{17}{8} = \frac{n+1}{2} + \frac{17}{8} \end{aligned}$$

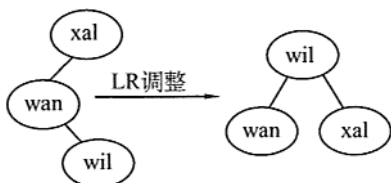
2. 解答: 算法的基本思想: 检索时可先从表头开始向后顺序扫描, 若找到指定的结点, 则将该结点和其前趋结点 (若存在) 交换。采用顺序表存储结构的算法实现如下:

```
int SeqSrch(RcdType R[], ElemType k) {           // 顺序查找线性表, 找到后和其前面的元素交换
    int i = 0;
    while ((R[i].key != k) && (i < n))
        i++; // 从前向后顺序查找指定结点
    if (i < n && i > 0) {                         // 若找到, 则交换
        temp = R[i];
        R[i] = R[i - 1];
        R[i - 1] = temp;
        return --i;                               // 交换成功, 返回交换后的位置
    }
    else
        return -1;                               // 交换失败
}
```

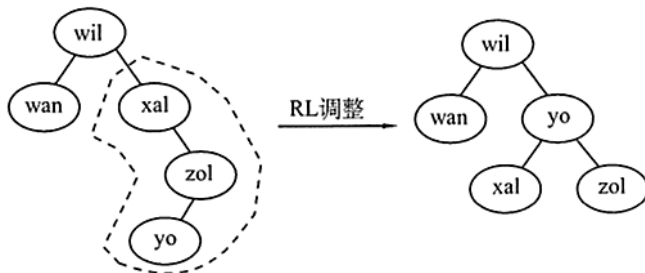
注意, 链表方式实现的基本思想与上述思想相似, 但要注意用链表实现时, 在交换两个结点之前需要保存指向前一结点的指针。

3. 分析: 按照二叉排序树的插入算法将关键字逐个插入树中, 在发生不平衡现象的时候进行平衡调整, 最后即可得到平衡二叉树, 操作步骤如下:

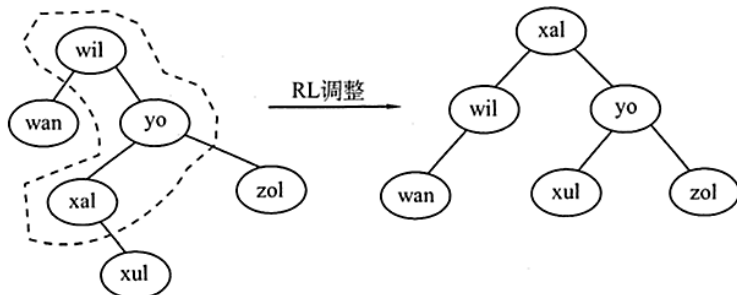
1) 插入关键字 xal、wan、wil 后发生不平衡现象, 进行调整, 如下图所示。



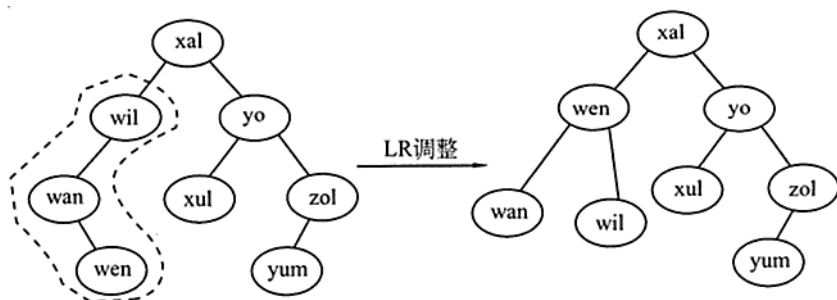
2) 插入关键字 zol、yo 后发生不平衡现象, 进行调整, 如下图所示。



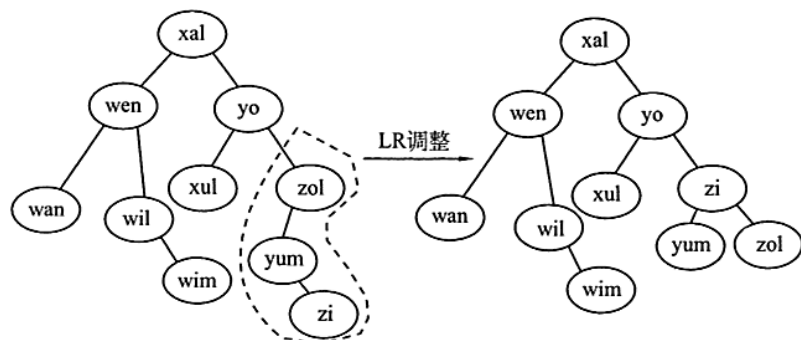
3) 插入关键字 xul 后发生不平衡现象, 进行调整, 要特别注意结点 xul 的连接, 调整后 wil 和 yo 两个结点都有空闲指针, xul 应该连接在 yo 的左指针上, 因为 xul 大于 xal, 结果如下图所示。



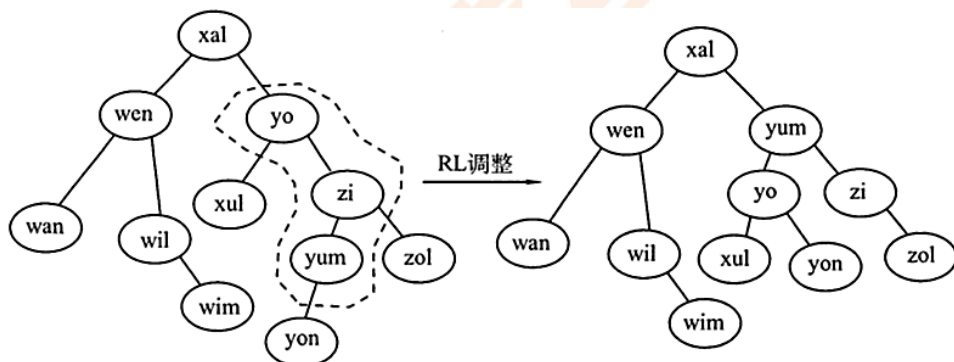
4) 插入关键字 yum、wen 后发生了不平衡现象, 调整后如下图所示。



5) 插入关键字 zi 后发生了不平衡现象, 调整后如下图所示。



6) 插入最后一个关键字 yon, 发生了不平衡现象, 进行调整。要特别注意的是, 调整后 yon 应该连接在 yo 的右指针上, 而不是 zi 的左指针上, 因为 yon 比 yum 小。调整后的最终结果如下图所示。



由上图右图可得各个关键字的比较次数为:

关键字	xal	wan	wil	zol	yo	xul	yum	wen	wim	zi	you
比较次数	1	3	3	4	3	4	2	2	4	3	4

则查找成功时的平均查找长度 $ASL = (1 + 3 + 3 + 4 + 3 + 4 + 2 + 2 + 4 + 3 + 4) / 11 = 33 / 11 = 3$ 。

说明: 本题为了让读者容易理解, 整个过程写得比较详细, 篇幅较大。如果题目不要求写出中间过程, 则只画出最终的平衡二叉树即可。

4. 解答: 根据 B 树的概念, 一个索引结点应适应操作系统一次读写的物理记录大小, 其大小应取不超过但最接近一个磁盘页块的大小。假设 B 树为 m 阶, 一个 B 树结点最多存放 m-1 个关键字 (5B) 和对应的记录地址 (5B)、m 个子树指针 (5B) 和 1 个指示结点中的实际关键字个数的整数 (2B), 则:

$$(2 \times (m - 1) + m) \times 5 + 2 \leq 4000$$

计算结果为 $m \leq 267$ 。

一个索引结点最多可以存放 $m - 1 = 266$ 个索引项, 最少可以存放 $\lceil m/2 \rceil - 1 = 133$ 个索引项。全部有 $n = 20000000$ 个记录, 每个记录占用空间 200B, 每个页块可以存放 $4000 / 200 = 20$ 个记录,

则全部记录分布在 $20000000/20=1000000$ 个页块中, 最多需要占用 $1000000/133=7519$ 个磁盘页块作为 B 树索引, 最少需要占用 $1000000/266=3760$ 个磁盘页块作为 B 树索引 (注意 B 树与 B+ 树的不同, B 树所有对数据记录的索引项分布在各个层次的结点中, B+ 树所有对数据记录的索引项都在叶结点中)。

5.分析: 由散列函数可知散列地址的范围为 0~10。

采用线性探测法构造散列表时, 首先应计算出关键字对应的散列地址, 然后检查散列表中对应的地址是否已经有元素。若没有元素, 则直接将该关键字放入散列表对应的地址中; 若有元素, 则采用线性探测的方法查找下一个地址, 从而决定该关键字的存放位置。

采用链地址法构造散列表时, 在直接计算出关键字对应的散列地址后, 将关键字结点插入此散列地址所在的链表。

解答:

(1) 线性探测法。

$H(1)=1$, 无冲突, 地址 1 存放关键字 1。 $H(13)=2$, 无冲突, 地址 2 存放关键字 13。 $H(12)=1$, 发生冲突, 根据线性探测法: $H1=2$, 发生冲突, 继续探测 $H2=3$, 无冲突, 于是 12 存放在地址为 3 的表项中。 $H(34)=1$, 发生冲突, 根据线性探测法: $H1=2$, 发生冲突, $H2=3$, 发生冲突, 凡 $=4$, 没有冲突, 于是 34 存放在地址为 4 的表项中。

同理, 可以计算其他的数据存放情况, 最后结果如下表所示。

散列地址	0	1	2	3	4	5	6	7	8	9	10
关键字	33	1	13	12	34	38	27	22			
冲突次数	0	0	0	2	3	0	1	7			

下面计算平均查找长度:

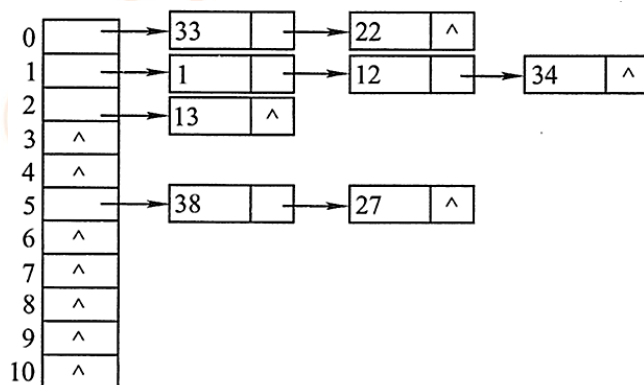
查找成功时, 显然查找每个元素的概率都是 $1/8$ 。对于 33, 由于冲突次数为 0, 所以仅需 1 次比较便可查找成功; 对于 22, 由于计算出的地址为 0, 但需要 8 次比较才能查找成功, 所以 22 的查找长度为 8; 其他元素的分析类似。因此有

$$ASL_{成功} = (1 + 1 + 1 + 3 + 4 + 1 + 2 + 8)/8 = 21/8$$

查找失败时, 由于 $H(key) = 0 \sim 10$, 因此对每个位置查找的概率都是 $1/11$, 对于计算出的地址为 0 的关键字 key_0 , 只有探测完 0~8 号地址后才能确定该元素不在表中, 比较次数为 9; 对于计算出的地址为 1 的关键字 key_1 , 只有探测完 1~8 号地址后, 才能确定该元素不在表中, 比较次数为 8, 以此类推。而对于计算出的地址为 8, 9, 10 的关键字, 这些单元中没有存放元素, 所以只需比较 1 次便可确定查找失败, 因此有

$$ASL_{失败} = (9 + 8 + 7 + 6 + 5 + 4 + 3 + 2 + 1 + 1 + 1)/11 = 47/11$$

(2) 链地址法构造的表如下:



在链地址表中查找成功时, 查找关键字为 33 的记录需进行 1 次比较, 查找关键字为 22 的记录需进行 2 次比较, 以此类推。因此有

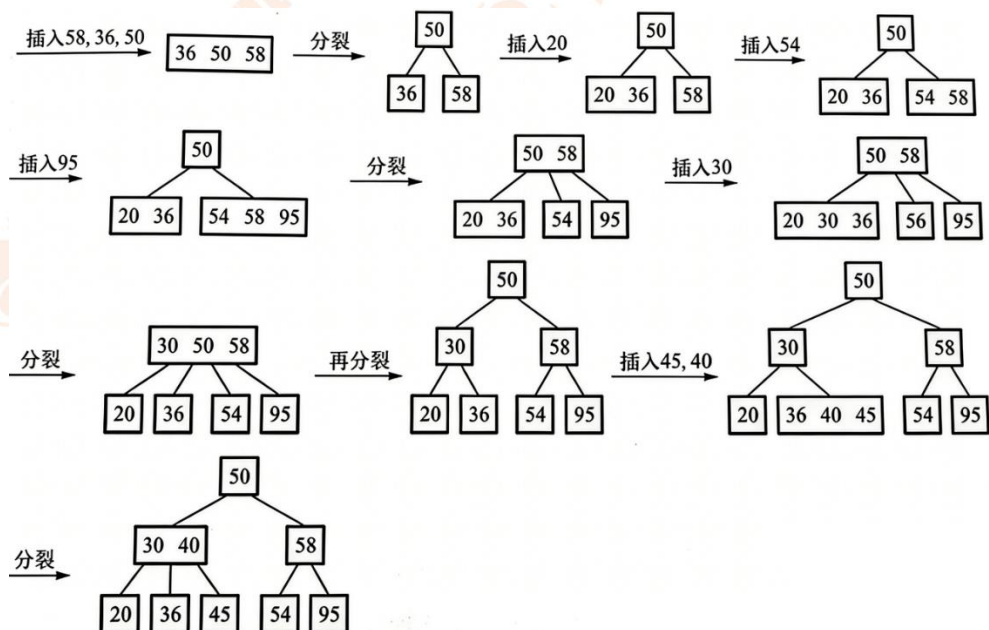
$$ASL_{成功} = (1 \times 4 + 2 \times 3 + 3) / 8 = 13/8$$

查找失败时, 对于地址 0, 比较 3 次后确定元素不在表中 (空指针算 1 次), 所以其查找长度为 3; 对于地址 1, 其查找长度为 4; 对于地址 2, 查找长度为 2; 以此类推。因此有

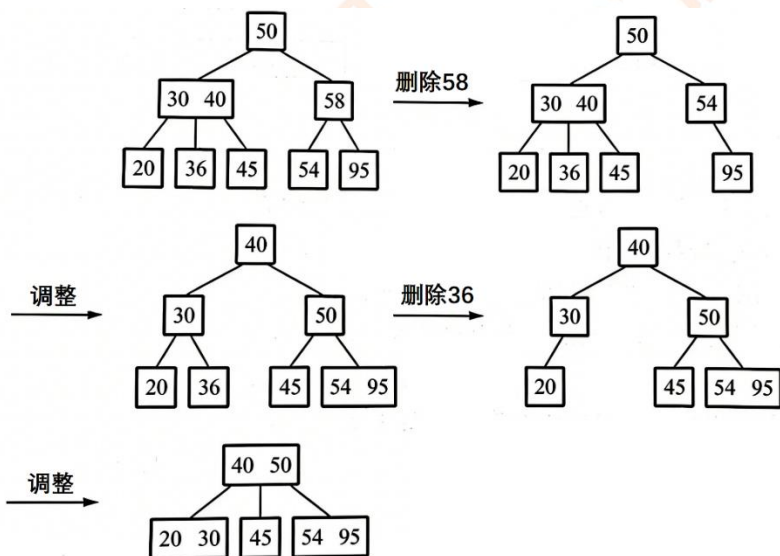
$$ASL_{失败} = (3 + 4 + 2 + 1 + 1 + 3 + 1 + 1 + 1 + 1 + 1) / 11 = 19/11$$

值得注意的是, 求查找失败的平均查找长度有两种观点: 其一, 认为比较到空结点才算失败, 所以比较次数等于冲突次数加 1; 其二, 认为只有与关键字的比较才算比较次数。

6. (1) 3 阶 B 树结点所含关键字个数范围为 [1, 2], 插入过程中结点关键字个数大于 2 时, 位于结点中间位置的关键字要向上分裂, 分裂操作可能进行多次, 直到满足 3 阶 B 树特性为止, 如下图:



(2) 删除结点时, 若删除的关键字在非叶结点内, 用前驱或后继替代删除的关键字。这样就转换为删除结点在终端结点的情况: 若删除后结点关键字个数为 0, 需要向其他结点借关键字, 若兄弟够则优先向兄弟结点借; 若兄弟不够借, 则向父结点借。如下图所示为删除关键字 58 和 36 的过程及删除后的 B 树。



7.2 算法专题

7.2.1 折半查找

1. 写出折半查找的递归算法。初始调用时, low 为 1, high 为 ST.length。

【答案解析】

1. 解答: 算法的基本思想: 根据查找的起始位置和终止位置, 将查找序列一分为二, 判断所查找的关键字在哪一部分, 然后用新的序列的起始位置和终止位置递归求解。

算法代码如下:

```
typedef struct {                //查找表的数据结构
    ElemType *elem;            // 存储空间基址, 建表时按实际长度分配, 0 号留空
    int length;                // 表的长度
} SearchTable;                // 查找表的数据结构
int BinSearchRec(SSTable ST, ElemType key, int low, int high) {
    // 在有序表中递归折半查找其关键字为 key 的元素, 返回其在表中序号
    if (low > high)
        return 0;              // 如果 low 大于 high, 表示没有找到, 返回 0
    mid = (low + high) / 2;      // 取中间位置
    if (key > ST.elem[mid])      // 向后半部分查找
        return BinSearchRec(ST, key, mid + 1, high);
    else if (key < ST.elem[mid]) // 向前半部分查找
        return BinSearchRec(ST, key, low, mid - 1);
    else                        // 查找成功
        return mid;
}
```

算法把规模为 n 的复杂问题经过多次递归调用转化为规模减半的子问题求解。时间复杂度为 $O(\log_2 n)$, 算法中用到了一个递归工作栈, 其规模与递归深度有关, 也是 $O(\log_2 n)$ 。

7.2.2 二叉排序树

1. 试编写一个算法, 判断给定的二叉树是否是二叉排序树。
2. 设计一个算法, 求出指定结点在给定二叉排序树中的层次。
3. 利用二叉树遍历的思想编写一个判断二叉树是否是平衡二叉树的算法。
4. 设计一个算法, 求出给定二叉排序树中最小和最大的关键字。
5. 设计一个算法, 从大到小输出二叉排序树中所有值不小于 K 的关键字。

6. 编写一个递归算法, 在一棵有 n 个结点的、随机建立起来的二叉排序树上查找第 k ($1 \leq k \leq n$) 小的元素, 并返回指向该结点的指针。要求算法的平均时间复杂度为 $O(\log_2 n)$ 。二叉排序树的每个结点中除 data, lchild, rchild 等数据成员外, 增加一个 count 成员, 保存以该结点为根的子树上的结点个数。

【答案解析】

1. 解答: 对二叉排序树来说, 其中序遍历序列为一个递增有序序列。因此, 对给定的二叉树进行

中序遍历, 若始终能保持前一个值比后一个值小, 则说明该二叉树是一棵二叉排序树。算法如下:

```
KeyType predt = -32767;           // predt 为全局变量, 保存当前结点中序前驱的值, 初值为-∞
int JudgeBST(BiTree bt) {
    int b1, b2;
    if (bt == NULL) {              // 空树
        return 1;
    }
    else {
        b1 = JudgeBST(bt->lchild); // 判断左子树是否是二叉排序树
        if (b1 == 0 || predt >= bt->data) // 若左子树返回值为 0 或前驱大于等于当前结点
            return 0;                // 则不是二叉排序树
        predt = bt->data;            // 保存当前结点的关键字
        b2 = JudgeBST(bt->rchild);   // 判断右子树
        return b2;                  // 返回右子树的结果
    }
}
```

2. 算法思想: 设二叉树采用二叉链表存储结构。在二叉排序树中, 查找一次就下降一层。因此, 查找该结点所用的次数就是该结点在二叉排序树中的层次。采用二叉排序树非递归查找算法, 用 n 保存查找层次, 每查找一次, n 就加 1, 直到找到相应的结点。算法如下:

```
int level(BiTree bt, BSTNode *p) { // 本算法计算给定结点*p 在二叉排序树中的层次
    int n = 0;                     // 统计查找次数
    BiTree t = bt;
    if (bt != NULL) {
        n++;
        while (t->data != p->data) {
            if (p->data < t->data) // 在左子树中查找
                t = t->lchild;
            else                  // 在右子树中查找
                t = t->rchild;
            n++;                  // 层次加 1
        }
    }
    return n;
}
```

3. 解答: 设置二叉树的平衡标记 $balance$, 标记返回二叉树 bt 是否为平衡二叉树, 若为平衡二叉树, 则返回 1, 否则返回 0; h 为二叉树 bt 的高度。采用后序遍历的递归算法:

①若 bt 为空, 则高度为 0, $balance = 1$ 。

②若 bt 仅有根结点, 则高度为 1, $balance = 1$ 。

③否则, 对 bt 的左、右子树执行递归运算, 返回左、右子树的高度和平衡标记, bt 的高度为最高子树的高度加 1。若左、右子树的高度差大于 1, 则 $balance = 0$; 若左、右子树的高度差小于等于 1, 且左、右子树都平衡时, $balance = 1$, 否则 $balance = 0$ 。

算法如下:

```
void Judge_AVL(BiTree bt, int &balance, int &h) { // 本算法判断一个给定的二叉树是否为平衡二叉树
    int bl = 0, br = 0, hl = 0, hr = 0;           // 左、右子树的平衡标记和高度
    if (bt == NULL) {
        h = 0;                                     // 空树, 高度为 0
        balance = 1;
    }
    else if (bt->lchild == NULL && bt->rchild == NULL) { // 仅有根结点, 则高度为 1
```

```

        h = 1;
        balance = 1;
    }
    else {
        Judge_AVL(bt->lchild, bl, hl);           // 递归判断左子树
        Judge_AVL(bt->rchild, br, hr);           // 递归判断右子树
        h = (hl > hr ? hl : hr) + 1;             // 计算当前节点的高度
        if (abs(hl - hr) < 2) { // 若子树高度差的绝对值小于 2, 则看左、右子树是否都平衡
            balance = bl && br;    // &&为逻辑与, 即左、右子树都平衡时, 二叉树平衡
        }
        else
            balance = 0;
    }
}

```

4. 解答: 在一棵二叉排序树中, 最左下结点即为关键字最小的结点, 最右下结点即为关键字最大的结点, 本算法只要找出这两个结点即可, 而不需要比较关键字。算法如下:

```

KeyType MinKey(BSTNode *bt) {                // 求出二叉排序树中最小关键字结点
    while (bt->lchild != NULL)
        bt = bt->lchild;
    return bt->data;
}
KeyType MaxKey(BSTNode *bt) {                // 求出二叉排序树中最大关键字结点
    while (bt->rchild != NULL)
        bt = bt->rchild;
    return bt->data;
}

```

5. 解答: 由二叉排序树的性质可知, 右子树中所有的结点值均大于根结点值, 左子树中所有的结点值均小于根结点值。从大到小输出, 先遍历右子树, 再访问根结点, 后遍历左子树。算法如下:

```

void OutPut(BSTNode *bt, KeyType k) {
    // 本算法从大到小输出二叉排序树中所有值不小于 k 的关键字
    if (bt == NULL)
        return;
    if (bt->rchild != NULL)
        OutPut(bt->rchild, k);           // 递归输出右子树结点
    if (bt->data >= k)
        printf("%d", bt->data);         // 只输出大于等于 k 的结点值
    if (bt->lchild != NULL)
        OutPut(bt->lchild, k);           // 递归输出左子树的结点
}

```

6. 解答: 设二叉排序树的根结点为 *t, 根据结点存储的信息, 有以下几种情况:

(1) t->lchild 为空时, 情况如下:

①若 t->rchild 非空且 k=1, 则 *t 即为第 k 小的元素, 查找成功。

②若 t->rchild 非空且 k≠1, 则第 k 小的元素必在 *t 的右子树。

(2) 若 t->lchild 非空时, 情况如下:

①t->lchild->count=k-1, 气即为第 k 小的元素, 查找成功

②t->lchild->count>k-1, 第 k 小的元素必在 *t 的左子树, 继续到 *t 的左子树中查找。

③t->lchild->count<k-1, 第 k 小的元素必在右子树, 继续搜索右子树, 寻找第 k-(t->lchild->count+1) 小的元素。

对左右子树的搜索采用相同的规则, 递归实现的算法描述如下:

```
BSTNode *Search_Small(BSTNode *t, int k) {
    // 在以 t 为根的子树上寻找第 k 小的元素, 返回其所在结点的指针。k 从 1 开始计算。
    // 在树结点中增加一个 count 数据成员, 存储以该结点为根的子树的结点个数。
    if (k < 1 || k > t->count)
        return NULL;
    if (t->lchild == NULL) {
        if (k == 1)
            return t;
        else
            return Search_Small(t->rchild, k - 1);
    }
    else {
        if (t->lchild->count == k - 1)
            return t;
        if (t->lchild->count > k - 1)
            return Search_Small(t->lchild, k);
        if (t->lchild->count < k - 1)
            return Search_Small(t->rchild, k - (t->lchild->count + 1));
    }
}
```

最大查找长度取决于树的高度。由于二叉排序树是随机生成的, 其高度应是 $O(\log_2 n)$, 算法的时间复杂度为 $O(\log_2 n)$ 。

7.2.3 平衡二叉树

1. 在平衡二叉树的每个结点中增设一个域 lsize, 存储以该结点为根的左子树中的结点个数加 1。编写一个算法, 确定树中第 k ($k \geq 1$) 个结点的位置。

【答案解析】

1. 解析: 实际上, 增设的 lsize 域里面记入的就是以该结点为根的子树中该结点的次序。含有 lsize 域的结点定义如下:

```
typedef struct BTNode {
    int data;
    int lsize; //新增加的 lsize 域
    struct BTNode *lchild;
    struct BTNode *rchild;
} BTNode;
//算法描述如下:
BTNode *search_small(BTNode *t, int k) {
    if (t == NULL || k < 1)
        return NULL;
    if (t->lsize == k)
        return t;
    if (t->lsize > k)
        return search_small(t->lchild, k);
    else
        return search_small(t->rchild, k - t->lsize);
}
```

7.2.4 串和子串

1. 编写下列算法。

(1) 将串 str 中所有值为 ch1 的字符转换成 ch2 的字符, 如果 str 为空串, 或者串中不含值为 ch1

的字符, 则什么都不做。

(2) 实现串 str 的逆转函数, 如果 str 为空串, 则什么都不做。

(3) 删除 str 中值为 ch 的所有字符, 如果 str 为空串, 或者串中不含值为 ch 的字符, 则什么都不做。

(4) 从串 str 中的 pos 位置起, 求出与 substr 串匹配的子串的位置, 如果 str 为空串, 或者串中不含与 substr 匹配的子串, 则返回 -1 做标记。

2. 采用定长顺序存储表示串, 编写一个函数, 删除串中从下标为 i 的字符开始的 j 个字符, 如果下标为 i 的字符后没有足够的 j 个字符, 则有几个删除几个。

3. 采用顺序存储方式存储串, 编写一个函数, 将串 str1 中的下标 i 到下标 j 之间的字符 (包括 i 和 j 两个位置上的字符) 用 str2 串替换。

4. 编写一个函数, 计算一个子串在一个主串中出现的次数, 如果该子串不出现, 则返回 0。本题不需要考虑子串重叠, 如: 主串为 aaaa, 子串为 aaa, 考虑子串重叠结果为 2, 不考虑子串重叠结果为 1。

5. 构造串的链表结点数据结构 (每个结点内存储一个字符), 编写一个函数, 找出串 str1 中第一个不在 str2 中出现的字符。

【答案解析】

1. 解析: (1) 本题较为简单, 只需扫描 str 串, 发现值 chl 的字符用值为 ch2 的字符覆盖即可, 代码如下:

```
void trans ( Str& str, char chl, char ch2) {
    for(int i=0; i<str.length; ++i) {
        if(str.ch[i]==chl)
            str.ch[i]=ch2;
    }
}
```

(2) 本题也很简单, 只需用两个指针指向串首和串尾, 然后相向而行, 且交换其经过的字符, 两指针相遇时算法结束, 代码如下:

```
void swap (char& chl, char& ch2) {
    char temp=chl;
    chl=ch2;
    ch2=temp;
}
void invert ( Str& str) {
    int i=0;
    int j=str.length-1;
    while(i<j) {
        swap (str.ch[i], str.ch[j]);
        ++i;
        --j;
    }
}
```

(3) 解法一:

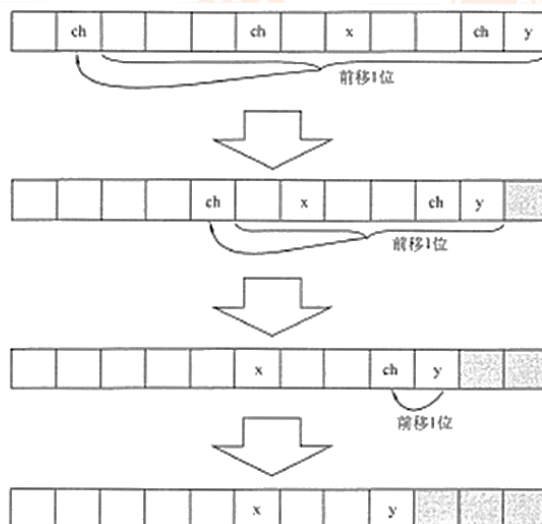
开辟一个新串空间, 将原串中未删除的字符复制到这片新空间上, 释放原串空间。算法实现代码如下:

```
void dell (Str & str , char ch) {
    if (str.length!=0) {
        int sum=0;
        int i;
        int j;
        for (i = 0; i < str.length ; ++i )           //求被删除字符的个数
            if(str.ch[i]==ch)
                ++sum;
        if(sum!=0) {
            char* temp_ch=(char*)malloc(sizeof(char)*(str.length-sum+1));
            for(i=0,j=0;i<str.length;++i)
                if(str.ch[i]!=ch)
                    temp_ch[j++] =str.ch[i] ;
            temp_ch[j]='\0';
            str.length=str.length-sum;
            free (str.ch) ;
            str.ch =temp_ch ;
        }
    }
}
```

解法二:

本题如果采用定长顺序串作为串的存储结构, 则可以通过移动覆盖元素实现删除, 算法描述为: 当发现值等于 ch 的字符时, 将其后所有的字符向前移动一个位置。算法实现代码如下:

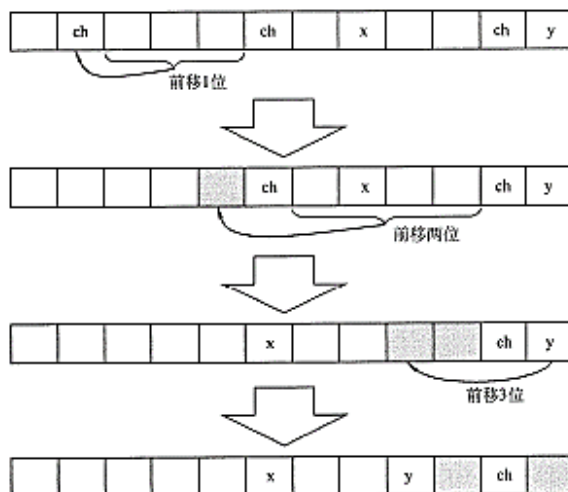
```
typedef struct{
    char ch[maxSize];
    int length ;
} Str2;
void del2 (Str2& str , char ch) {
    if(str.length!=0){
        for(int i=0;i<str.length;) {
            if(str.ch[i]==ch)
                for(int j=i; j<str.length-1;++ j)
                    str.ch[j]=str.ch[j+1];
            --str.length;
        }
    }
    else
        ++i;
    str.ch [str . length] = '\0 ' ;
}
}
```



题目图展示了解法二中的处理过程, 可以看出 x 字符被移动了两次, y 字符被移动了 3 次才到答案最终位置, 这显然是多余的操作 (图中灰色块表示每次移动腾出的空位置)。

改进方法: 只需将每个待删字符到后一个待删字符之间的子串前移 i 个位置即可, 其中 i 是目前扫描到的待删字符的个数。

改进算法如下图所示, 第一个 ch 到第二个 ch 之间的字符全部往前移动一个位置, 删除了第一个 ch (这里之所以说删除而不是覆盖, 是因为可能出现没有用其他字符覆盖掉 ch 的情况, 但 ch 一定会从当前串中删除), 并出现了一个空位; 此时将第二个 ch 到第三个 ch 之间的字符向前移动两个位置, 即可删除 ch 和空位上的字符; 以此类推, 直到处理完所有字符即可。



解法三:

```
void del3(Str2& str, char ch) {
    if(str.length!=0) {
        int ch_num=0;
        int i,j;
        i=0;
        while (str.ch[i] != '\0') {
            if (str.ch[i] == ch) {
                ++ch_num;
                for (j=i+1; str.ch[j]!=ch && str.ch[j]!='\0'; ++j)
                    str.ch[j-ch_num]=str.ch[j];
                i=j;
                --str.length;
            }
            else
                ++i;
        }
        str.ch[str.length]='\0';
    }
}
```

(4) 修改 KMP 算法, 使起始匹配检测位置从 pos 开始即可。假设 $next$ 数组已经构建完毕。具体代码如下:

```
int KMP(Str str, Str substr, int pos) {
    int i=pos, j=1;
    while (i <= str.length && j <= substr.length) {
        if (j==0 || str.ch[i]==substr.ch[j]) {
            ++i;
            ++j;
        }
    }
}
```

```

else
    j=next[j];
} //while
if ( j > substr.length )
    return i - substr.length;
else
    return -1;
}

```

2. 本题算法思想: 从下标为 $i+j$ 个字符开始, 将所有的字符都向前移动 j 个单位, 然后将字符串长度缩减为被删除字符的个数即可。算法实现代码如下:

```

void del(Str2& str, int i, int j) {
    if ( i < str.length && i >= 0 && j >= 0 ) {
        for(int k=i+j; k<str.length; ++k)
            str.ch[k-j]=str.ch[k];
        Str.length-= (str.length - i < j ? str.length - i : j);
        //调整串长, 注意需要考虑两种情况
        str.ch[str.length]='\0';
    }
}

```

3. 本题算法思想: 取 $str1$ 中 0 到 $i-1$ 位置上的子串 $str11$, 取 $str1$ 中 $j+1$ 到串尾的子串 $str12$, 最后连接 $str11$ 、 $str2$ 、 $str12$ 即可。算法实现代码如下:

```

/*串连接*/
int concat (Str& str, Str str1, Str str2) { //串的连接
    if (str.ch) { //释放原串空间
        free (str.ch);
        str.ch=NULL;
    }
    str.ch = (char*) malloc (sizeof(char) * (str1.length+str2.length+1));
    if ( str.ch == NULL )
        return false;
    int i=0;
    while(i < str1.length) {
        str.ch[i] = str1.ch[i];
        ++i;
    }
    int j=0;
    while(j<=str2.length) { //注意, 要连同 str2.ch 最后的 "\0" 一起复制
        Str.ch[i+j] = str2.ch[j];
        ++j;
    }
    str.length=str1.length+str2.length;
    return true;
}
/*求子串*/
int substring (Str& substr, Str str, int pos, int len) {
    if(pos<0 || pos>=str.length || len<0 || len>str.length-pos)
        return false;
    if (substr.ch) {
        free(substr.ch);
        substr.ch=NULL;
    }
    if (len==0) {
        substr.ch=NULL;
        substr.length=0;
    }
}

```



```

        return true;
    }
    else{
        substr.ch=(char*)malloc(sizeof(char)*(len+1));
        int i=pos ;
        int j=0;
        while(i<pos+len) {
            substr.ch[j]=str.ch[i];
            ++i;
            ++j;
        }
        substr.ch [ j ] ='\ 0';
        return true ;
    }
}
/*子串的替换*/
int stuff(Str& str1,Str str 2 , int i , int j) {
    Str strll ; strl.ch =NU LL; strll. length = 0;
    Str strl2 ; strl2.ch = NULL; strl2.length = 0;
    Str temp _str;
    if (!substring (strll , strl , 0, i ) )
        return 0;
    if (!substring (strl2 , strl , j+1 , strl.length-j-1) )
        return 0;
    if (!concat (temp _str, strll.,str2))
        return 0;
    if (!concat(strl ,temp _str , strl2) ) return 0;
    return 1;
}

```

4.本题为模式串匹配算法的扩展。在之前关于模式串匹配算法的讲解中,如果匹配成功,则算法结束,这里只需修改为匹配成功计数器加 1 并且继续进行匹配检测即可。算法代码如下:

```

int index (Str str, Str sub str) {
    int i=1,j=1,k=1,sum=0;
    while(i<=str.length) {
        if (str.ch[i]==substr.ch[j]) {
            ++i;
            ++j;
        }
        else{
            j=1;
            i=++ k ;
            //匹配失败, i 从主串下一位置开始, K 中记录了上一次的起始位置
        }
        if(j>substr.length) {
            j=1;
            ++sum;
        }
    }
    return sum;
}

```

当然本题也可以用 KMP 算法来实现,假设 next 数组已经求得,代码如下:

```

int KMP(Str str,Str substr){
    int i=1,j=1,sum=0;
    while ( i <=str . length ) {
        if(j==0|| str .ch [ i ] ==substr.ch [ j ] ) {
            ++i;

```

```

        ++j;
    }
    else
        j=next[j];
    if(j>substr.length) {
        j = 1;
        ++sum;
    }
}
return sum;
}

```

5. 本题算法思想: 从头到尾扫描 str1, 对于 str1 中的每一个结点判断是否在 str2 中出现, 若出现, 则继续扫描 str1 中的下一个字符, 否则返回当前检测字符。如果 str1 中的所有字符都在 str2 中出现, 则返回 '\0'。

结点数据结构构造比较简单, 只需把链表结点数据项类型改写成 char 型即可:

```

typedef struct SNode{
    char data;
    struct SNode *next;
} SNode;
本算法用不带头结点的单链表来表示串, 实现函数如下:
char findfirst(SNode* str1, SNode* str2) {
    for(SNode* p=str1; p!=NULL; p=p->next) {
        bool flag=false;
        for(SNode* q=str2; q!=NULL; q=q->next)
            if (p->data==q->data) {
                flag=true;
                break;
            }
        if(flag==false)
            return p->data;
    }
    return '\0';
}

```

8、排序

8.1 综合练习

(一) 选择题

- 用直接插入排序算法对下列 4 个表进行 (从小到大) 排序, 比较次数最少的是()。
A. 94,32,40,90,80,46,21,69 B. 21,32,46,40,80,69,90,94
C. 32,40,21,46,69,94,90,80 D. 90,69,80,46,21,32,94,40
- 对序列{15, 9, 7, 8, 20, -1, 4}用希尔排序方法排序, 经一趟后序列变为 {15, -1, 4, 8, 20, 9, 7}, 则该次采用的增量是()。
A. 1 B. 4 C. 3 D. 2
- 对下列关键字序列用快排进行排序时, 速度最快的情形是(), 速度最慢的情形是()。
A. {21, 25, 5, 17, 9, 23, 30} B. {25, 23, 30, 17, 21, 5, 9}
C. {21, 9, 17, 30, 25, 23, 5} D. {5, 9, 17, 21, 23, 25, 30}
- 下列序列中, () 可能是执行第一趟快速排序后所得到的序列。
I. {68, 11, 18, 69, 23, 93, 73} II. {68, 11, 69, 23, 18, 93, 73}
III. {93, 73, 68, 11, 69, 23, 18} IV. {68, 11, 69, 23, 18, 73, 93}
A. I、IV B. II、III C. III、IV D. 只有 IV
- 对关键码序列{23,17,72,60,25,8,68,71,52}进行堆排序, 输出两个最小关键码后的剩余堆是()。
A. {23,72,60,25,68,71,52} B. {23,25,52,60,71,72,68}
C. {71,25,23,52,60,72,68} D. {23,25,68,52,60,72,71}
- 设线性表中每个元素有两个数据项 k_1 和 k_2 , 现对线性表按以下规则进行排序: 先看数据项 k_1 , k_1 值小的元素在前, 大的元素在后; 在 k_1 值相同的情况下, 再看 k_2 , k_2 值小的在前, 大的元素在后。满足这种要求的排序方法是()。
A. 先按 k_1 进行直接插入排序, 再按 k_2 进行简单选择排序
B. 先按 k_2 进行直接插入排序, 再按 k_1 进行简单选择排序
C. 先按 k_1 进行简单选择排序, 再按 k_2 进行直接插入排序
D. 先按 k_2 进行简单选择排序, 再按 k_1 进行直接插入排序
- 设有 5 个初始归并段, 每个归并段有 20 个记录, 采用 5 路平衡归并排序, 若不采用败者树, 使用传统的顺序选出最小记录 (简单选择排序) 的方法, 总的比较次数是 (①); 若采用败者树最小的方法, 总的比较次数是 (②)。
A. 20 B. 300 C. 396 D. 500
- 在做 m 路平衡归并排序的过程中, 为实现输入/内部归并/输出的并行处理, 需要设置 (①) 个输入缓冲区和 (②) 个输出缓冲区。
① A. 2 B. m C. $2m-1$ D. $2m$
② A. 2 B. m C. $2m-1$ D. $2m$
- 对任意 7 个关键字进行基于比较的排序, 至少要进行 () 次关键字之间的两两比较。
A. 13 B. 14 C. 15 D. 6
- 下列排序算法中属于稳定排序的是 (①), 平均时间复杂度为 $O(n\log_2 n)$ 的是 (②), 在最好的情况下, 时间复杂度可以达到线性时间的有 (③)。
I. 冒泡排序 II. 堆排序 III. 选择排序 IV. 直接插入排序
V. 希尔排序 VI. 归并排序 VII. 快速排序

11. 用某种排序方法对线性表 {25,84,21,47,15,27,68,35,20} 进行排序时, 元素序列的变化情况如下:

- 1) 25,84,21,47,15,27,68,35,20 2) 20,15,21,25,47,27,68,35,84
3) 15,20,21,25,35,27,47,68,84 4) 15,20,21,25,27,35,47,68,84

则所采用的排序方法是()。

- A. 选择排序 B. 插入排序 C. 2 路归并排序 D. 快速排序

12. 置换-选择排序的作用是()。

- A. 用于生成外部排序的初始归并段
B. 完成将一个磁盘文件排序成有序文件的有效的外部排序算法
C. 生成的初始归并段的长度是内存工作区的 2 倍
D. 对外部排序中输入 / 归并 / 输出的并行处理

13. 最佳归并树在外部排序中的作用是()。

- A. 完成 m 路归并排序 B. 设计 m 路归并排序的优化方案
C. 产生初始归并段 D. 与锦标赛树的作用类似

【答案解析】

1.B. 首先, 越接近正序的序列, 比较次数应是越少的; 而越接近逆序, 比较次数越多。不难得出 B 和 C 是比较接近正序的, 然后分别判断两个序列的比较次数, 以 B 为例: 第一趟, 插入 32, 比较 1 次; 第二趟, 插入 46, 比较 1 次; 第三趟, 插入 40, 由于 40 比 46 小但比 32 大, 所以比较 2 次; 第四趟, 插入 80, 比较 1 次; 第五趟, 插入 69, 比较 2 次……共比较 9 次。同理求出 C 的比较次数为 11 次。故选 B。

2.B. 希尔排序将序列分成若干组, 记录只在组内进行交换。由观察可知, 经过一趟后 9 和 -1 交换, 7 和 4 交换, 可知增量为 4。

3.A、D. 由考点精析可知, 当每次的枢轴都把表等分为长度相近的两个子表时, 速度是最快的; 当表本身已经有序或逆序时, 速度最慢。选项 D 中的序列已按关键字排好序, 因此它是最慢的, 而 A 中第一趟枢轴值 21 将表划分为两个子表 {9, 17, 5} 和 {25, 23, 30}, 而后对两个子表划分时, 枢轴值再次将它们等分, 所以该序列是快速排序最优的情况, 速度最快。其他选项可以类似分析。

4.C. 显然, 若按从小到大排序, 则最终有序的序列是 {11, 18, 23, 68, 69, 73, 93}; 若按从大到小排序, 则最终有序的序列是 {93, 73, 69, 68, 23, 18, 11}。对比可知 I、II 中没有处于最终位置的元素, 故 I、II 都不可能。III 中 73 和 93 处于从大到小排序后的最终位置, 而且 73 将序列分割成大于 73 和小于 73 的两部分, 故 III 是有可能的。IV 中 73 和 93 处于从小到大排列后的最终位置, 73 也将序列分割成大于 73 和小于 73 的两部分。

5.D. 筛选法初始建堆为 {8, 17, 23, 52, 25, 72, 68, 71, 60}, 输出 8 后重建的堆为 {17, 25, 23, 52, 60, 72, 68, 71}, 输出 17 后重建的堆为 {23, 25, 68, 52, 60, 72, 71}。

6.D. 首先应确定 k_1 、 k_2 的排序顺序, 若先排 k_1 再排 k_2 , 则排序结果不符合题意, 排除选项 A、C。再考虑算法的稳定性, 当 k_2 排好序后, 再对 k_1 排序, 若对 k_1 排序采用的算法是不稳定的, 则对 k_1 相同而 k_2 不同的元素可能会改变相对次序, 从而不一定能满足题干中的条件“在 k_1 值相同的情况下, k_2 值小的元素在前, 大的元素在后”。直接插入排序算法是稳定的, 而简单选择排序算法是不稳定的, 故只能选 D。

7.C、B. ①不采用败者树时, 在 5 个记录中选出最小的需要做 4 次比较, 共有 100 个记录, 需要做 99 次选择最小记录的操作, 所以需要的比较次数为 $4 \times 99 = 396$, 故选 C。

②采用败者树时, 5 路归并意味着败者树的外结点有 5 个, 败者树的高度 $h = \lceil \log_2 5 \rceil = 3$ 。每次在参

加比较的记录中选择一个关键字最小的记录, 比较次数不超过 h , 共 100 个记录, 需要的比较次数不超过 $100 \times 3 = 300$, 故选 B。

8.D、A。在做 m 路平衡归并排序的过程中, 为实现输入/内部归并/输出的并行处理, 需要设置 $2m$ 个输入缓冲区和 2 个输出缓冲区, 以便在执行内部归并时, 能同时进行输入/输出操作。若仅设置 m 个输入缓冲区, 则仅能进行串行操作, 无法并行处理。注重理解 2 路归并排序并在解题中留心题眼是正确解答的关键。

9.A。对于任意序列进行基于比较的排序, 求最少的比较次数应考虑最坏情况。对任意 n 个关键字排序的比较次数至少为 $\lceil \log_2(n!) \rceil$ 。将 $n = 7$ 代入公式, 答案为 13。

上述公式证明如下(仅供有兴趣的同学参考): 在基于比较的排序方法中, 每次比较两个关键字后, 仅出现两种可能的转移。假设整个排序过程至少需要做 t 次比较, 则显然会有 2^t 种情况。由于 n 个记录共有 $n!$ 种不同的排列, 因而必须有 $n!$ 种不同的比较路径, 于是有 $2^t \geq n!$, 即 $t \geq \log_2(n!)$ 。考虑到 t 为整数, 故为 $\lceil \log_2(n!) \rceil$ 。

10.①I、IV、VI; ②II、VI、VII; ③I、IV。堆排序和归并排序在最坏情况下的时间复杂度与最好情况下的时间复杂度是同一数量级的, 都是 $O(n \log_2 n)$ 。

11.D。选择排序在每趟结束后可以确定一个元素的最终位置, 不对。插入排序, 第 i 趟后前 $i+1$ 个元素应该是有序的, 不对。第 2 趟 $\{20, 15\}$ 和 $\{21, 25\}$ 是反序的, 因此不是归并排序。快速排序每趟都将基准元素放在其最终位置, 然后以它为基准将序列划分为两个子序列。观察题中的排序过程, 可知是快速排序。

12.A。外部排序分为两个阶段: ①生成初始归并段; ②多路平衡归并。

置换-选择排序作用于第一阶段。首先需要将庞大的外存文件分割成若干个内部有序的子序列, 利用堆结构和内存缓冲区, 在读取记录的过程中动态地选择最小元素输出, 从而生成长度较长的初始有序子序列(归并段)。因此 A 正确。

B 错误: 外部排序由“生成初始段”和“归并”两部分组成。置换-选择排序仅完成第一部分, 不能单独完成整个文件的排序。

C 错误: 在随机数据下, 初始归并段平均长度约为内存工作区大小的 2 倍, 但在数据有序的情况下, 如数据正序, 归并段长度=文件总长, 大于 2 倍; 如数据逆序, 归并段长度 $\approx$ 内存工作区。

D 错误: 输入/输出/归并的并行处理是通过缓冲技术实现的, 与置换-选择排序无对应关系。

13.B。在外部排序的第二阶段(归并阶段), 需要将第一阶段生成的 m 个初始归并段合并成一个有序文件。最佳归并树本身不是一种新的排序算法, 也不直接执行归并操作, 它是一种方案, 用于确定最优的归并顺序, 从而使得归并过程中的总代价(I/O 次数)最小, 它本身不完成排序, 所以 A 错误, B 正确。

C: 错误。产生初始归并段是外部排序第一阶段的任务, 通常由置换-选择排序完成。最佳归并树用于第二阶段(归并阶段)。

D: 错误。锦标赛树(败者树), 用于在归并过程中, 从 k 个归并段的当前元素中快速选出最小值, 以提高内部比较效率。最佳归并树: 用于规划归并的顺序, 以减少总的 I/O 次数。

(二) 综合应用题

1. 荷兰国旗问题: 设有一个仅由红、白、蓝三种颜色的条块组成的条块序列, 请编写一个时间复杂度为 $O(n)$ 的算法, 使得这些条块按红、白、蓝的顺序排好, 即排成荷兰国旗图案。

2. 设初始归并段为(10, 15, 31), (9, 20), (22, 34, 37), (6, 15, 42), (12, 37), (84, 95)。试利用败者

树进行 m 路归并，手工执行选择最小的 5 个关键字的过程。

3. 给出 12 个初始归并段，其长度分别为 30, 44, 8, 6, 3, 20, 60, 18, 9, 62, 68, 85。现要做 4 路外归并排序，试画出表示归并过程的最佳归并树，并计算该归并树的带权路径长度 WPL。

4. 假设文件有 4500 个记录，在磁盘上每个块可放 75 个记录。计算机中用于排序的内存区可容纳 450 个记录。试问：

- 1) 可以建立多少个初始归并段？每个初始归并段有多少记录？存放于多少个块中？
- 2) 应采用几路归并？请写出归并过程及每趟需要读写磁盘的块数。

5. 假设内存缓冲区可容纳 4 个记录，设有一组记录为 10, 20, 15, 25, 12, 13, 21, 30, 8, 16, 10。请建立初始归并段（段内递增排序），列表写出求解过程。

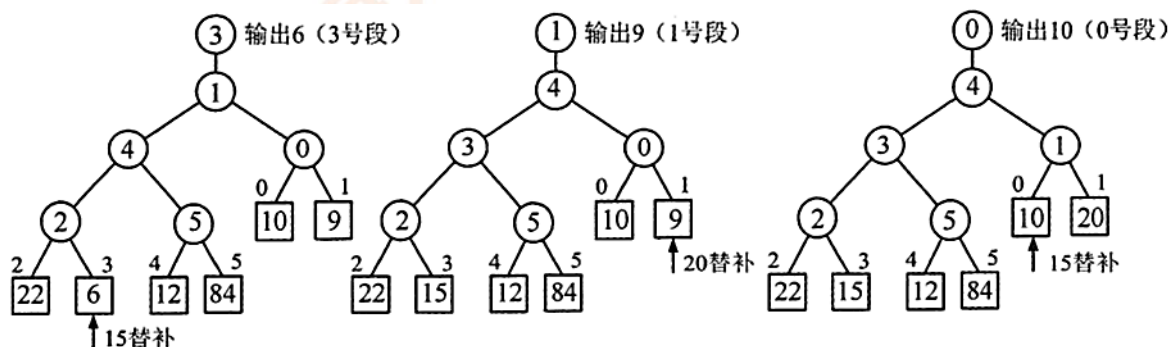
【答案解析】

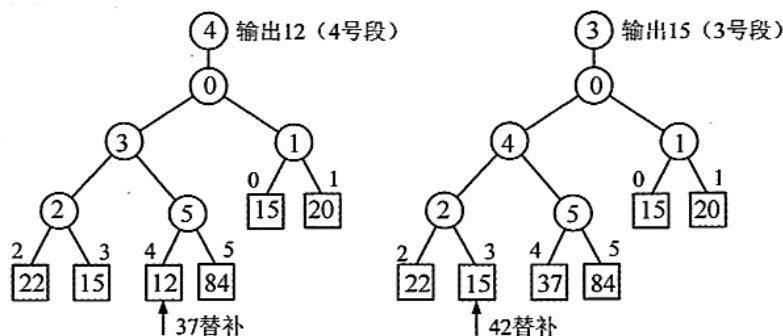
1. 解答：算法思想：顺序扫描线性表，将红色条块交换到线性表的最前面，蓝色条块交换到线性表的最后面。为此，设立三个指针，其中， j 为工作指针，表示当前扫描的元素， i 以前的元素全部为红色， K 以后的元素全部为蓝色。根据 j 所指示元素的颜色，决定将其交换到序列的前部或尾部。初始时 $i = 0$, $k = n - 1$ ，算法的实现如下：

```
typedef enum {RED, WHITE, BLUE} color;           // 设置枚举数组
void Flag_Arrange(color a[], int n) {
    int i = 0, j = 0, k = n - 1;
    while (j <= k)
        switch (a[j]) {
            case RED:
                Swap(a[i], a[j]); i++; j++; break; // 判断条块的颜色
                                                    // 红色，则和 i 交换
            case WHITE:
                j++; break;
            case BLUE:
                Swap(a[j], a[k]); k--;              // 蓝色，则和 k 交换
                                                    // 这里没有 j++ 语句以防止交换后 a[j] 仍为蓝色的情况
        }
}
```

例如，将元素值正数、负数和零排序为前面都是负数，接着是 0，最后是正数，也用同样的方法。思考：为什么 case RED 时不用考虑交换后 $a[j]$ 仍为红色，而 case BLUE 却需要考虑交换后 $a[j]$ 仍为蓝色？

2. 解答：做 6 路归并排序，选择最小的 5 个关键字的败者树如下图所示。





3. 解答: 设初始归并段个数 $n = 12$, 外归并路数 $k = 4$, 计算 $(n-1) \bmod (k-1) = 11 \bmod 3 = 2 \neq 0$, 说明不能做完全的 4 路归并, 因为多出了 2 个初始归并段, 必须添加 $k-2-1 = 1$ 个长度为 0 的空归并段, 才能构成严格的 4 路归并树, 即每次归并都有 K 个归并段参加归并。此时, 归并树的内结点应有 $(n-1+1)/(k-1) = 12/3 = 4$ 个, 如下图所示。

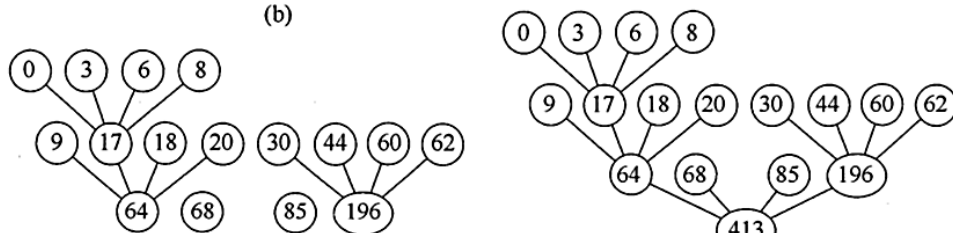
$WPL = (3+6+8) \times 3 + (9+18+20+30+44+60+62) \times 2 + (68+85) \times 1 = 51 + 486 + 153 = 690$ 。



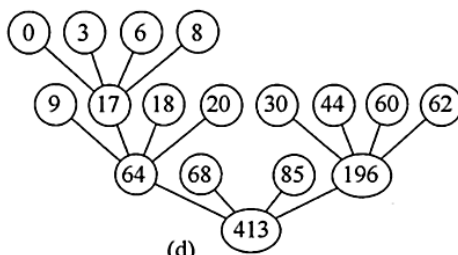
(a)



(b)



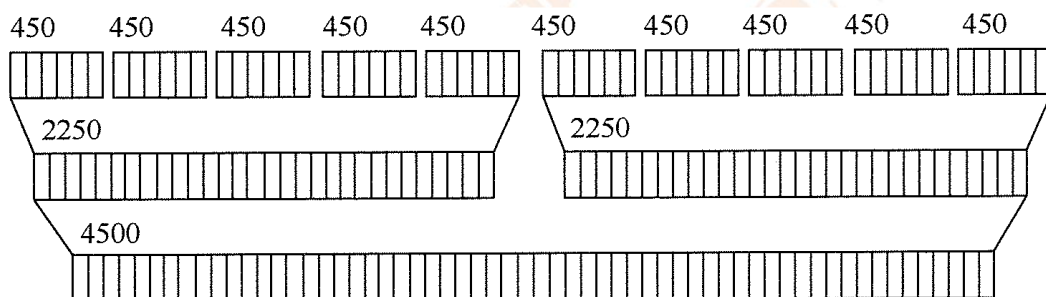
(c)



(d)

4. 解答: 1) 文件有 4500 个记录, 用于排序的内存区可容纳 450 个记录, 可建立的初始归并段有 $4500/450 = 10$ 个。每个初始归并段中有 450 个记录, 存于 $450/75 = 6$ 个块中。

2) 内存区可容纳 6 个块, 可建立 6 个缓冲区, 其中 5 个缓冲区用于输入, 1 个缓冲区用于输出, 因此可采用 5 路归并, 归并过程如下图所示。



共做了 2 趟归并, 每趟需要读 60 块、写 60 块。

5. 解: 初始归并段生成过程见下表。

步数	1	2	3	4	5	6	7	8	9	10	11
	10	12	13	21	21	21	16	16	16	16	

缓冲区内容	20	20	20	20	20	8	8	8			
	15	15	15	15	30	30	30	30			
	25	25	25	25	25	25	25	10	10		
输出结果	10	12	13	15	20	21	25	30	8	10	16
	第一初始归并段								第二初试归并段		

本题较简单，每次从缓冲区中选出最小且比当前归并段最大的关键字还要大的输出，空出来的位置读入新的关键字。

8.2 算法专题

8.2.1 插入排序算法

1. 给定一组关键字，创建一个带头结点的链表，设计一个直接插入排序算法，对这个单链表进行递增排序。

2. 设顺序表用数组 $A[]$ 表示，表中元素存储在数组下标 $1 \sim m+n$ 的范围内，前 m 个元素递增有序，后 n 个元素递增有序，设计一个算法，使得整个顺序表有序。

- 1) 给出算法的基本设计思想。
- 2) 根据设计思想，采用 C 或 C++ 描述算法，关键之处给出注释。
- 3) 说明你所设计算法的时间复杂度与空间复杂度。

【答案解析】

1. 分析：本题的排序思想和对用数组存储的关键字序列进行插入排序的情况一样，只不过是将对数组的操作改为对链表的操作。

具体算法如下：

```
void CreateLink(LNode *&h, char R[], int n) {           //建立链表存储结构
    int i;
    LNode *s, *t;
    h = (LNode *) malloc(sizeof(LNode));
    h->next = NULL;
    t = h;
    for(i=0; i<n; ++i) {
        s = (LNode *) malloc(sizeof(LNode));
        s->data = R[i];
        t->next = s;
        t = s;
    }
    t->next = NULL;
}

void Sort(LNode *h) {                                   //排序函数
    LNode *p, *pl, *q, *pre;
    if (h->next != NULL) {                               //链表中至少有一个结点
        p = h->next->next;                               // p 指向第二个关键字结点
        h->next->next = NULL;                             //产生只带一个结点的有序表
        while(p != NULL) {
            pre = h;                                     //pre 指向 q 的前驱结点
            q = pre->next;
            while (q != NULL && q->data < p->data) {
                //在有序表中找到一个结点 q，其 data 值刚好大于 p->data
                pre = q;
                q = q->next;
            }
            pre->next = p;
            p = p->next;
        }
    }
}
```



```

    }
    pl = p->next;
    p->next = pre->next;          //将 p 插入到 pre 之后
    pre->next = p;
    p = pl;
  }
}

```

2. 解答:

1) 算法的基本设计思想如下: 将数组 $A[1 \dots m+n]$ 视为一个已经过 m 趟插入排序的表, 则从 $m+1$ 趟开始, 将后 n 个元素依次插入前面的有序表中。

2) 算法的实现如下:

```

void Insert_Sort(ElemType A[], int m, int n) {
    int i, j;
    for (i = m + 1; i <= m + n; i++) {          // 依次将 A[m+1...m+n] 插入有序表
        A[0] = A[i];                             // 复制为哨兵
        for (j = i - 1; A[j] > A[0]; j--)         // 从后往前插入
            A[j + 1] = A[j];                     // 元素后移
        A[j + 1] = A[0];                         // 插入
    }
}

```

3) 时间复杂度由 m 和 n 共同决定, 从上面的算法不难看出, 在最坏情况下元素的比较次数为 $O(mn)$, 而元素移动的次数为 $O(mn)$, 所以时间复杂度为 $O(mn)$ 。

由于算法只用到了常数个辅助空间, 所以空间复杂度为 $O(1)$ 。

此外, 本题也可采用归并排序, 将 $A[1 \dots m]$ 和 $A[m+1 \dots m+n]$ 视为两个待归并的有序子序列, 算法的时间复杂度为 $O(m+n)$, 空间复杂度为 $O(m+n)$ 。

8.2.2 冒泡排序

1. 编写双向冒泡排序算法, 在正反两个方向交替进行扫描, 如第一趟把关键字最大的元素放在序列的最后面, 第二趟把关键字最小的元素放在序列的最前面, 或者第一趟把关键字最小的元素放在序列的最前面, 第二趟把关键字最大的元素放在序列的最后面, 如此反复进行。

【答案解析】

1. 分析: 本算法可以这样进行: 先从底向上从无序区冒了一个最小的关键字, 再从上向底从无序区冒出一个最大的关键字。

双向冒泡排序的算法代码如下:

```

void sort(int array[], int n) {
    int right = n - 1;
    int left = 0;
    bool flag = true;
    while(flag) {
        flag = false;
        for(i = left; i < right; ++i) {          //从左到右扫描, 最大的放到右边
            if(array[i] > array[i+1]) {
                swap(array[i], array[i+1]); //swap()是已定义的函数, 交换两个变量的值
                flag = true;
            }
        }
        // 从右到左扫描, 最小的放到左边
        for(i = right; i > left; --i) {
            if(array[i] < array[i-1]) {
                swap(array[i], array[i-1]);
                flag = true;
            }
        }
        left++;
        right--;
    }
}

```

```

    }
    --right; //从右到左扫描, 最小的放在左边
    for (j = right; j > left; --j) {
        if (array[j] < array[j-1]) {
            swap(array[j], array[j-1]);
            flag = true;
        }
    }
    ++left;
}
}
}

```

8.2.3 快速排序

1. 重新编写快速排序的划分算法, 使之每次选取的枢轴值 (哨兵) 都是随机地从当前子表中选择的。
2. 试编写一个算法, 使之能够在数组 $L[1..n]$ 中找出第 k 小的元素 (即从小到大排序后处于第 k 个位置的元素)。
3. 设有一个数组中存放了一个无序的关键序列 $K_1, K_2, \dots, K_n$ 。现要求将位放在将元素排序后的正确位置上, 试编写实现该功能的算法, 要求比较关键字的次数不超过 n 。

【答案解析】

1. 解答: 这类题目比较简单, 为方便起见, 可直接先随机地求出枢轴的下标, 然后将枢轴值与 $A[\text{low}]$ 交换, 而后的思想就与前面的划分算法一样。算法的实现如下:

```

int Partition2(ElemType A[], int low, int high) {
    int rand_Index = low + rand() % (high - low + 1);
    Swap(A[rand_Index], A[low]); // 将枢轴值交换到第一个元素
    ElemType pivot = A[low]; // 置当前表中的第一个元素为枢轴值
    int i = low; // 使得表 A[low...i] 中的所有元素小于 pivot, 初始为空表
    for (int j = low + 1; j <= high; j++) { // 从第二个元素开始寻找小于基准的元素
        if (A[j] < pivot) // 找到后, 交换到前面
            swap(A[++i], A[j]);
    }
    swap(A[i], A[low]); // 将基准元素插入最终位置
    return i; // 返回基准元素的位置
}

```

注意: 本题代码中的比较方法和考点精析中的两种方法均不相同, 请读者仔细模拟各种划分算法的执行步骤, 做到真正掌握。对于下一题, 读者可以尝试采用这三种比较方法来分别解决。

2. 解答: 显然, 本题最直接的做法是用排序算法对数组先进行从小到大的排序, 然后直接提取 $L(k)$ 便得到了第 k 小元素, 但其平均时间复杂度将达到 $O(n \log_2 n)$ 以上。此外, 还可采用小顶堆的方法, 每次堆顶元素都是最小值元素, 时间复杂度为 $O(n + k \log_2 n)$ 。下面介绍一个更精彩的算法, 它基于快速排序的划分操作。

这个算法的主要思想如下: 从数组 $L[1..n]$ 中选择枢轴 pivot (随机或直接取第一个) 进行和快速排序一样的划分操作后, 表 $L[1..n]$ 被划分为 $L[1..m-1]$ 和 $L[m+1..n]$, 其中 $L(m) = \text{pivot}$ 。讨论 m 与 K 的大小关系:

- 1) 当 $m = k$ 时, 显然 pivot 就是所要寻找的元素, 直接返回 pivot 即可。
- 2) 当 $m < k$ 时, 所要寻找的元素一定落在 $L[m+1 \dots n]$ 中, 因此可对 $L[m+1 \dots n]$ 递归地查找第 $k - m$ 小的元素。
- 3) 当 $m > k$ 时, 所要寻找的元素一定落在 $L[1 \dots m-1]$ 中, 因此可对 $L[1 \dots m-1]$ 递归地查找第 k 小的元素。

该算法的时间复杂度在平均情况下可以达到 $O(n)$, 而所占空间的复杂度则取决于划分的方法。算法的实现如下:

```
int kth_elem(int a[], int low, int high, int k) {
    int pivot = a[low];           // 选择枢轴值为数组的第一个元素
    int low_temp = low;           // 保存原始的 low 值
    int high_temp = high;         // 保存原始的 high 值
    while (low < high) {
        while (low < high && a[high] >= pivot) // 从右向左找到第一个小于枢轴值的元素
            --high;
        a[low] = a[high];          // 将该元素移动到 low 位置
        while (low < high && a[low] <= pivot) // 从左向右找到第一个大于枢轴值的元素
            ++low;
        a[high] = a[low];          // 将该元素移动到 high 位置
    }
    a[low] = pivot;                // 上面即为快速排序中的划分算法
    // 以下就是本算法思想中所述的内容
    if (low == k)                  // 由于与 k 相同, 直接返回 pivot 元素
        return a[low];
    else if (low > k)              // 在前一部分表中递归寻找
        return kth_elem(a, low_temp, low - 1, k);
    else                          // 在后一部分表中递归寻找
        return kth_elem(a, low + 1, high_temp, k);
}
```

3. 解答: 基本思想: 以凡为枢轴进行一趟快速排序。将快速排序算法改为以最后一个为枢轴先从前 向后再从后向前。算法的代码如下:

```
int Partition(ElemType K[], int n) {
    // 交换序列 K[1..n] 中的记录, 使枢轴到位, 并返回其所在位置
    int i = 1, j = n;             // 设置两个交替变量初值分别为 1 和 n
    ElemType pivot = K[j];        // 枢轴
    while (i < j) {               // 循环跳出条件
        while (i < j && K[i] <= pivot) // 从前往后找比枢轴大的元素
            i++;
        if (i < j)
            K[j] = K[i];          // 移动到右端
        while (i < j && K[j] >= pivot) // 从后往前找比枢轴小的元素
            j--;
        if (i < j)
            K[i] = K[j];          // 移动到左端
    } // while
    K[i] = pivot;                 // 枢轴存放在最终位置
    return i;                     // 返回存放枢轴的位置
}
```

8.2.4 计数排序

1. 有一种简单的排序算法叫作计数排序。这种算法对一个待排序表（用数组 $A[]$ 表示）进行排序，排序结果存储在另一个新的表中（用数组 $B[]$ 表示），表中关键字为 int 型。必须注意的是，表中所有待排序的关键字互不相同，计数排序算法针对表中的每个关键字，扫描待排序表一趟，统计表中有多少个关键字比该关键字小。假设对某一个关键字，统计出数值为 c ，那么这个关键字在新的有序表中的位置即为 c 。

- (1) 设计实现计数排序算法。
- (2) 对于有 n 个关键字的表，比较次数是多少？
- (3) 与简单选择排序相比，这种方法是否更好？为什么？

【答案解析】

1. 解:(1) 本题算法如下:

```
void countSort ( int A [ ], int B [ ], int n ) {
    int i, j, count;
    for(i=0; i<n; ++i) {
        count= 0;
        for(j=0; j<n; ++j)
            if (A [ j ] < A [ i ])
                ++count;
        B[count]=A[i];
    }
}
```

//本题条件特殊，数组从下标 0 开始存储
//用 count 统计 A 中小于 A[i] 的关键字个数

- (2) 对于 n 个关键字，每个关键字都要与 n 个关键字（含自身）进行比较，因此比较次数为 n^2
- (3) 简单选择排序比这种计数排序好，因为对有 n 个关键字的序列进行简单排序，只需要进行 $1+2+3+\dots+(n-1)=n(n-1)/2$ 次比较，并且可在原地进行排序（空间复杂度为常量级）。

8.2.5 简单选择排序

1. 编写一个算法，在基于单链表表示的待排序关键字序列上进行简单选择排序。

【答案解析】

1. 解答：算法的思想是：每趟在原始链表中摘下关键字最大的结点，把它插入结果链表的最前端。由于在原始链表中摘下的关键字越来越小，在结果链表前端插入的关键字也越来越小，因此最后形成的结果链表中的结点将按关键字非递减的顺序有序链接。

单链表的定义如第 2 章所述，假设它不带表头结点。

```
void selectSort(LinkedList& L) {
    LinkNode *h, *p, *q, *r, *s;
    L = NULL;
    while (h != NULL) {
        p = s = h; q = r = NULL;
        // 持续扫描原链表
        while (p != NULL) {
            // 指针 s 和 r 记忆最大结点和其前驱; p 为工作指针, q 为其前驱
            // 扫描原链表寻找最大结点 s
            if (p->data > s->data) {
                s = p; r = q;
            }
            // 找到更大的, 记忆它和它的前驱
            // 继续寻找
            q = p; p = p->link;
        }
    }
}
```



```

    if (s == h)
        h = h->link;           // 最大结点在原链表前端
    else
        r->link = s->link;       // 最大结点在原链表表内
        s->link = L; L = s;      // 结点 s 插入结果链前端
    }
}

```

8.2.6 堆排序

1. 试设计一个算法, 判断一个数据序列是否构成一个小根堆。

2. 已知由 $n-1$ 个关键字组成的序列 $(K_1, K_2, \dots, K_{n-1})$ 是大顶堆, 现在增加一个关键字 K_n , 要求将关键字序列 $(K_1, K_2, \dots, K_{n-1}, K_n)$ 重新调整为大顶堆。请完成以下要求:

- (1) 给出算法的基本设计思想。
- (2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。
- (3) 说明你所设计算法的时间复杂度。

【答案解析】

1. 解答: 将顺序表 $L[1 \dots n]$ 视为一个完全二叉树, 扫描所有分支结点, 遇到孩子结点的关键字小于根结点的关键字时返回 false, 扫描完后返回 true。算法的实现如下:

```

bool IsMinHeap(ElemType A[], int len) {
    if (len % 2 == 0) {           // len 为偶数, 有一个单分支结点
        if (A[len / 2] > A[len])  // 判断单分支结点
            return false;
        for (i = len / 2 - 1; i >= 1; i--) { // 判断所有双分支结点
            if (A[i] > A[2 * i] || A[i] > A[2 * i + 1])
                return false;
        }
    }
    else {                         // len 为奇数时, 没有单分支结点
        for (i = len / 2; i >= 1; i--) { // 判断所有双分支结点
            if (A[i] > A[2 * i] || A[i] > A[2 * i + 1])
                return false;
        }
    }
    return true;
}

```

2. 解答: (1) 基本设计思想: 从根结点的父母结点的标号 $[n/2]$ 开始向上, 对每个当前结点和左右子树进行调整。最开始的时候要判断 n 是左结点还是右结点, 之后的情况一定是左右结点都有。每次把当前结点的标号除以 2 则得到当前结点的父母结点的标号。

(2) 算法描述

```

#define n 100;                  // 宏定义 n 常量, 由用户自定义结点个数
int K[n];                       // 关键字序列
void heap() {
    int i = n / 2;              // 找到最后一个结点的父母结点
    if (n % 2 == 1)              // 当 n 是右结点时 {
        if (K[i] < K[n - 1] && K[n - 1] > K[n])
            swap(K[n - 1], K[i]); // swap() 实现交换两个元素
    }
}

```

```
    if (K[i] < K[n] && K[n - 1] < K[n])
        swap(K[n], K[i]);
}
else{                                //当 n 是左结点
    if (K[i] < K[n]) swap(K[n], K[i]);
    i = i / 2;                        // 依次向上调整
    while (i > 0) {
        if (K[i] < K[n - 1] && K[n - 1] > K[n])
            swap(K[n - 1], K[i]);
        if (K[i] < K[n] && K[n - 1] < K[n])
            swap(K[n], K[i]);
        i = i / 2;
    }
}
```

(3) 时间复杂度分析：在循环当中，我们可以看出每次都是对结点的父母结点进行调整，因此操作次数正好是树的高度，时间复杂度为 $O(\log_2 n)$ 。

9、多维数组和矩阵的压缩存储

9.1 综合练习

(一) 选择题

- 若将 n 阶上三角矩阵 A 按列优先级压缩存放在一维数组 $B[1..n(n+1)/2+1]$ 中, 则存放到 $B[k]$ 中的非零元素 $a_{ij} (1 \leq i, j \leq n)$ 的下标 i, j 与 k 的对应关系是 ()。
A. $i(i+1)/2 + j$ B. $i(i-1)/2 + j - 1$ C. $j(j-1)/2 + i$ D. $j(j-1)/2 + i - 1$
- 若将 n 阶下三角矩阵 A 按列优先顺序压缩存放在一维数组 $B[1..n(n+1)/2+1]$ 中, 则存放到 $B[k]$ 中的非零元素 $a_{ij} (1 \leq i, j \leq n)$ 的下标 i, j 与 k 的对应关系是 ()。
A. $(j-1)(2n-j+1)/2 + i - j$ B. $(j-1)(2n-j+2)/2 + i - j + 1$
C. $(j-1)(2n-j+2)/2 + i - j$ D. $(j-1)(2n-j+1)/2 + i - j - 1$

【答案解析】

1.C。按列优先存储, 故元素 a_{ij} 前面有 $j-1$ 列, 共有 $1+2+3+\dots+j-1=j(j-1)/2$ 个元素, 元素 a_{ij} 在第 j 列上是第 i 个元素, 数组 B 的下标是从 1 开始, 因此 $k=j(j-1)/2+i$ 。

2.B。按列优先存储, 故元素 a_{ij} 之前有 $j-1$ 列, 共有 $n+(n-1)+\dots+(n-j+2)=(j-1)(2n-j+2)/2$ 个元素, 元素 a_{ij} 是第 j 列上第 $i-j+1$ 个元素, 数组 B 下标从 1 开始, $k=(j-1)(2n-j+2)/2+i-j+1$ 。

(二) 综合应用题

- 设数组 $A[0, \dots, n-1]$ 的 n 个元素中有多个零元素, 设计一个算法, 将 A 中所有的非零元素依次移动到 A 数组的前端。

【答案解析】

1. 算法分析: 本算法总体思想是, 从左到右扫描整个数组, 当发现非零元素时, 使其尽可能与靠左边的零元素进行交换。下图所示为对一个输入样例的算法执行图。

算法实现代码如下:

```
void movelement(int A[], int n) {
    int i=-1, j, temp;
    for(j=0; j<n; ++j)
        if (A[j]!=0) {
            //A[j]为第 i 个不为 0 的元素
            ++i;
            if (i!=j) {
                //A[j]不在位置 i 上, 则 A[i] 与 A[j] 交换
                temp=A[i];
                A[i]=A[j];
                A[j]=temp;
            }
        }
}
```

初始值: $i=-1, j=0$

1	0	0	3	0	1	0	2
---	---	---	---	---	---	---	---

i j

$i=0, j=0$, i 等于 j , i 和 j 继续向前移动

1	0	0	3	0	1	0	2
---	---	---	---	---	---	---	---

i j

$i=1, j=3$, $A[i]$ 和 $A[j]$ 交换, i 和 j 继续向前移动

1	3	0	0	0	1	0	2
	i		j				

$i=2, j=5$, $A[i]$ 和 $A[j]$ 交换, i 和 j 继续向前移动

1	3	1	0	0	0	0	2
	i				j		

$i=3, j=7$, $A[i]$ 和 $A[j]$ 交换, 算法结束

1	3	1	2	0	0	0	0
		i					j

9.2 算法专题

9.2.1 数组运算

1. 关于浮点型数组 $A[0, \dots, n-1]$, 试设计实现下列运算的递归算法。

- (1) 求数组 A 中的最大值。
- (2) 求数组中 n 个数之和。
- (3) 求数组中 n 个数的平均值。

2. 输入一个按升序排序过的整数数组 $\{1, 2, 4, 7, 11, 15\}$ 以及一个整数数字 15, 我们可以从该数组中找到两个数字, 即 4 和 11, 使得 $4+11=15$ 。请实现一个时间上尽可能高效率的算法, 当输入一个已经按升序排序过的整数数组和一个整数数字, 在数组中查找两个数, 使得它们的和正好是输入的那个整数数字。如果有多对数字的和等于输入的整数数字, 输出任意一对即可。要求:

- (1) 给出算法的基本设计思想。
- (2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。
- (3) 说明你所设计算法的时间复杂度。

3. 在数组中, 某个数字减去它右边的数字得到一个数对之差。求所有数对之差的最大值。例如, 在数组 $\{2, 4, 1, 16, 7, 5, 11, 9\}$ 中, 数对之差的最大值是 11, 是 16 减去 5 的结果。

- (1) 给出算法的基本设计思想。
- (2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。
- (3) 说明你所设计算法的时间复杂度。

【答案解析】

1. (1) 算法分析: 如果数组长度为 1, 则可以直接返回最大值; 否则将数组 A 视为两部分, 即 $A[0]$ 和 $A[1, \dots, n-1]$ 。如果 $A[0]$ 大于 $A[1, \dots, n-1]$ 中的最大值, 则返回 $A[0]$, 反之按照上一步的方法递归地处理 $A[1]$ 和 $A[2, \dots, n-1]$ 。算法实现代码如下:

```
float findmax (float A [], int i, int j) {
    float max ;
    if(i==j)
        return A[i];
    else{
        max=findmax(A,i+1,j);
        if(A[i]>max)
            return A[i];
        else
```



```
        return max;
    }
}
```

在某函数体中调用函数 findmax ()求最大值的格式如下:

```
{
    ...
    findmax(A,0,n-1);
    ...
}
```

(2) 算法分析: 如果数组长度为 1, 则可以直接返回求和结果; 否则将数组 A 视为两部分, 即 A[0]和 A[1, ..., n-1], 递归地处理 A[1, ..., n-1]中 n-1 个数的和, 使之与 A[0]相加得到最终结果。算法实现代码如下:

```
float arraysum ( float A [], int i , int j ) {
    if(i==j)
        return A[i];
    else
        return A[i]+arraysum(A,i+1,j);
}
```

在某函数体中调用函数 arraysum ()求和的格式如下:

```
{
    ...
    arraysum(A,0,n-1);
    ...
}
```

(3) 算法分析: 如果数组长度为 1, 则可以直接返回平均值结果; 否则将数组 A 视为两部分, 即 A[0]和 A[1, ..., n-1], 递归地处理 A[1, ..., n-1]中 n-1 个数的平均值 tempavg, 通过(tempavg×(n-1)+A[0])/n 即可求得数组 A 中所有数的平均值。算法实现代码如下:

```
float array avg ( float A[],int i,int j) {
    if(i==j)
        return A[i];
    else
        return(A[i]+(j-i)*arrayavg(A,i+1,j))/(j-i+1);
}
```

在某函数体中调用函数 arrayavg()求平均值的格式如下:

```
{
    ...
    arrayavg(A,0,n-1);
    ...
}
```

2. (1) 基本设计思想: 如果我们不考虑时间复杂度, 最简单的想法是先在数组中固定一个数字, 再依次判断数组中剩下的 n-1 个数字与它的和是不是等于输入的数字。可惜这种思路需要的时间复杂度是 $O(n^2)$, 不满足题意要求。

假设现在随便在数组中找到两个数, 如果它们的和等于输入的数字, 那太好了, 我们找到了要找的两个数字; 如果小于输入的数字呢? 我们希望两个数字的和再大一点。由于数组已经排好序了, 我们是不是可以把较小的数字往后移动一个数字? 因为排在后面的数要大一些, 那么两个数字的和也要大一些, 就有可能等于输入的数字了; 同样, 当两个数的和大于输入的数字的时候, 我们把较大的数字往前移动, 因为排在数组前面的数字要小一些, 它们的和就有可能等于输入的数字了。

我们把前面的思路整理一下: 最初我们找到数组的第一个数字和最后一个数字。当两个数字的和大于输入的数字时, 把较大的数字往前移动; 当两个数字的和小于数字时, 把较小的数字往后移

动；当相等时，正合题意。

```
bool FindTwoNumbersWithSum(
    int data[],                // 已排序的数组
    unsigned int length,       // 已排序数组的长度
    int& num1,                 // 第一个数字 (输出参数)
    int& num2,                 // 第二个数字 (输出参数)
    int sum                    // 输入的整数数字 (目标和)
)
{
    bool found = false;        // 初始化找到标志为 false
    if (length < 1)             // 空数组将会出错
        return found;
    int ahead = length - 1;     // 初始化右指针为数组末尾
    int behind = 0;             // 初始化左指针为数组起始
    while (ahead > behind) {    // 当左指针小于右指针时继续循环
        int curSum = data[ahead] + data[behind]; // 计算当前两个指针指向元素的和
        // 如果两个数之和恰好等于输入的数字
        if (curSum == sum) {
            num1 = data[behind]; // 更新第一个数字
            num2 = data[ahead];  // 更新第二个数字
            found = true;        // 设置找到标志为 true
            break;               // 结束循环
        }
        // 如果两个数之和大于输入的数字，将大的数字减少 (右指针左移)
        else if (curSum > sum)
            ahead--;             // 右指针左移
        // 如果两个数之和小于输入的数字，将小的数字加大 (左指针右移)
        else
            behind++;            // 左指针右移
    }
    return found;               // 返回是否找到结果
}
```

(3) 时间复杂度分析：在 while 的循环中，每次根据 curSum 和 sum 之间的大小关系来决定改变 ahead 还是改变 behind。这个过程每次是 $O(1)$ 的。在整个算法流程中，因为 ahead 始终大于 behind 的，如果一个数被 ahead 扫过了，那么它不会被 behind 扫到，也不会被 ahead 再次扫到；同样的，如果一个数被 behind 扫过了，那么它将不会再被 ahead 或者 behind 扫到。所以循环最多执行 $n-1$ 次就会结束，故整个算法的时间复杂度为 $O(n)$ 。

3. 解法一：分治法。

把数组分成两个子数组，其实没有必要拿左边子数组中较小的数字去和右边子数组中较大的数字作减法。可以想象，数对之差的最大值只可能是下面三种情况之一：①被减数和减数都在第一个子数组中，即第一个子数组中的数对之差的最大值；②被减数和减数都在第二个子数组中，即第二个子数组中数对之差的最大值；③被减数在第一个子数组中，是第一个子数组的最大值。减数在第二个子数组中，是第二个子数组的最小值。这三个差值的最大者就是整个数组中数对之差的最大值。

在前面提到的三种情况中，得到第一个子数组的最大值和第二个子数组的最小值不是一件难事，但如何得到两个子数组中的数对之差的最大值？这其实是原始问题的子问题，可以递归地解决。下面是这种思路的参考代码：

```
int MaxDiffSolution1(int numbers[], unsigned length) {
```

```

    if (numbers == NULL || length < 2) // 检查输入是否合法: 数组不能为空且长度必须大于等于 2
        return 0;
    int max, min;
    return MaxDiffCore(numbers, numbers + length - 1, &max, &min); // 计算最大差值
}
// 辅助函数: 递归地计算数组中最大值和最小值之间的差值
int MaxDiffCore(int* start, int* end, int* max, int* min) {
    if (end == start) { // 基本情况: 如果数组只有一个元素, 则最大值和最小值相同, 差值为 0
        *max = *min = *start;
        return 0;
    }
    int* middle = start + (end - start) / 2; // 计算中间指针, 将数组分为左右两部分
    // 递归处理左半部分, 得到左半部分的最大值和最小值
    int maxLeft, minLeft;
    int leftDiff = MaxDiffCore(start, middle, &maxLeft, &minLeft);
    // 递归处理右半部分, 得到右半部分的最大值和最小值
    int maxRight, minRight;
    int rightDiff = MaxDiffCore(middle + 1, end, &maxRight, &minRight);
    // 计算跨越左右两部分的最大差值 (左半部分的最大值减去右半部分的最小值)
    int crossDiff = maxLeft - minRight;
    // 更新全局的最大值和最小值
    *max = (maxLeft > maxRight) ? maxLeft : maxRight;
    *min = (minLeft < minRight) ? minLeft : minRight;
    // 计算并返回最大差值: 取左半部分、右半部分和跨越部分的最大差值
    int maxDiff = (leftDiff > rightDiff) ? leftDiff : rightDiff;
    maxDiff = (maxDiff > crossDiff) ? maxDiff : crossDiff;
    return maxDiff;
}

```

在函数 MaxDiffCore 中, 我们先得到第一个子数组中的最大数对之差 leftDiff, 再得到第二个子数组中的最大数对之差 rightDiff。接下来用第一个子数组的最大值减去第二个子数组的最小值得到 crossDiff。这三者的最大值就是整个数组的最大数对之差。

解法二: 动态规划法。

定义 diff[i] 是以数组中第 i 个数字为减数的所有数对之差的最大值。也就是说对于任意 $h(h < i)$, $\text{diff}[i] \geq \text{number}[h] - \text{number}[i]$ 。diff[i] ($0 \leq i < n$) 的最大值就是整个数组最大的数对之差。

假设我们已求出 diff[i], 该怎么求 diff[i+1] 呢? 对于 diff[i], 肯定存在一个 $h(h < i)$, 满足 $\text{number}[h]$ 减去 $\text{number}[i]$ 之差是最大的, 也就是 $\text{number}[h]$ 应该是 $\text{number}[i]$ 之前的所有数字的最大值。当求 diff[i+1] 的时候, 需要找到第 i+1 个数字之前的最大太值。第 i+1 个数字之前的最大值只有两种可能: 这个最大值可能是第 i 个数字之前的最大值, 也可能这个最大值就是第 i 个数字。第 i+1 个数字之前的最大值肯定是这两者的较大者。我们只要拿第 i+1 个数字之前的最大值减去 $\text{number}[i+1]$, 就得到了 diff[i+1]。

```

int MaxDiffSolution2(int numbers[], unsigned length) {
    if (numbers == NULL || length < 2)
        return 0;
    int max = numbers[0]; // 第 i 个数之前的最大值
    int maxDiff = max - numbers[1]; // maxDiff 表示 diff[i-1]
    for (int i = 2; i < length; ++i) {
        if (numbers[i-1] > max) { // 第 i 个数和之前的最大值比较
            max = numbers[i-1];
            int currentDiff = max - numbers[i]; // currentDiff 表示 diff[i]
            if (currentDiff > maxDiff)
                maxDiff = currentDiff;
        }
    }
}

```

```

    }
}
return maxDiff;
}

```

上述两种解法, 虽然思路不同, 但时间复杂度都是 $O(n)$ (第一种解法的时间复杂度可以用递归公式表示为 $T(n)=2(n/2)+O(1)$, 总体的时间复杂度为 $O(n)$)。

暴力法: 让每一个数字逐个减去它右边的所有数字, 并通过比较得到数对之差的最大值。由于每个数字需要和它后面的 $O(n)$ 个数字作减法, 因此总的时间复杂度是 $O(n^2)$ 。

9.2.2 三元组

1. 简要介绍稀疏矩阵的三元组存储结构特点, 并实现稀疏矩阵的基本操作。
(1) 给定稀疏矩阵 A (int 型), 创建其三元组存储结构 B 。
(2) 查找给定元素 x 是否在矩阵中。
2. 假设稀疏矩阵 A 采用三元组表示, 编写一个函数, 计算其转置矩阵 B , 要求 B 也采用三元组表示。
3. 假设稀疏矩阵 A 和 B (两矩阵行列数对应相等) 都采用三元组表示, 编写一个函数, 计算 $C=A+B$, 要求 C 也采用三元组表示, 所有矩阵均为 int 型。
4. 假设稀疏矩阵 A 和 B (分别为 $m \times n$ 和 $n \times k$ 矩阵) 采用三元组表示, 编写一个函数, 计算 $C=A \times B$, 要求 C 也是采用三元组表示的稀疏矩阵。

【答案解析】

1. 答: 三元组存储结构是一种顺序结构, 因此也是一种顺序表, 表中的每个结点对应稀疏矩阵的一个非零元素, 其中包括 3 个字段, 分别为该元素的值、行下标和列下标。另外, 用第 0 行的第 1 个元素存储矩阵中非零元素的个数, 第 0 行的第 2 个元素存储矩阵的行数, 第 0 行的第 3 个元素存储矩阵的列数。

(1) 创建 A 的三元组存储结构, 代码如下:

```

void create (int A [ ] [maxSize], int m,int n, int B[][3]) {
    int i,j,k=1;
    for(i=0;i<m;++i)
        for(j=0;j<n;++j)
            if(A[i][j] !=0) {
                /*扫描 A 中的元素, 将不为零的元素相关信息存放在 B 中的对应位置*/
                B[k][0]=A[i][j];
                B[k][1]=i;
                B[k][2]=j;
                ++k;
            }
    B[0][0] = k-1;
    B[0][1] =m;
    B[0][2] =n; //存入非零元素个数
}

```

(2) 查找给定元素 x 是否在矩阵中, 代码如下:

```

int search(int B[][3],int x) {
    int i,t;

```



```

t= B[0] [0]; //非零元素个数
i=l;
while(i<=t && B[i] [0]!=x)
    i++;
if (i<=t)
    return 1;
else
    return 0;
}

```

2.算法分析: 根据给定一个稀疏矩阵构造三元组的方法可以知道, 三元组内元素是原矩阵按照行优先存储的结果, 因此实现三元组存储的转置运算不能简单地将其中各个元素的行下标和列下标互换, 正确的算法应该等效于互换三元组中各个元素的行下标与列下标, 并将其按照原矩阵列优先的顺序存储。假设原矩阵为 A , 具体的做法如下: 在 A 中首先找出第 1 列中的所有元素, 它们是转置矩阵第 1 行的非零元素, 并把它们依次放在转置矩阵的三元组数组 B 中。然后依次找出第 2 列中的所有元素, 把它们依次放在数组 B 中。按照同样的方法逐列进行, 直到找出第 $n-1$ 列的所有元素, 并把它们依次放在数组 B 中。算法实现代码如下:

```

void transpose(int A[][3],int B[][3]) {
    int p,q,col;
    B[0] [0] =A [0] [0];
    B[0] [1] =A [0] [2];
    B[0] [2] =A [0] [1];           //产生第 0 行的结果
    if (B[0] [0]>0) {
        q=1;
        for(col=0;col<B[0] [1];++col)
            for(p=1;p<=B[0] [0];++p)
                if (A[p] [2] ==col) {
                    B[q] [0]=A[p] [0];
                    B[q] [1]=A[p] [2];
                    B[q] [2]=A[p] [1];
                    ++q;
                }
    }
}

```

3.算法分析: 两矩阵相加规则为对应位置上的元素相加, 对于三元组存储结构下的矩阵 A 和 B , 假如当前需要将位置 (i,j) 上的元素 $a_{(i,j)}$ 和 $b_{(i,j)}$ 相加, 需要考虑三种不同的情况: $a_{(i,j)}$ 等于 0, $b_{(i,j)}$ 不等于 0; $a_{(i,j)}$ 不等于 0, $b_{(i,j)}$ 不等于 0; $a_{(i,j)}$ 不等于 0, $b_{(i,j)}$ 等于 0。

对应的可以按照以下步骤进行处理: 依次遍历 A 和 B 的行号和列号, 若 A 当前元素的行号等于 B 当前元素的行号, 则比较其列号, 将较小列的元素存入 C 中; 如果列号也相等, 则将对应的元素值相加后存入 C 中; 若 A 当前元素的行号小于 B 当前元素的行号, 则将 A 的元素存入 C 中; 若 A 当前元素的行号大于 B 当前元素的行号, 则将 B 的元素存入 C 中, 如此这样产生了 C 。算法实现代码如下:

```

void add(int A[ ][3], int B[ ][3], int C[ ][3]) {
    int i=l,j=l,k=l,m;
    while (i <=A [0] [0] && j <= B [0] [0])
        if (A [i] [1] == B [j] [1])
            /*若 A 当前元素的行号等于 B 当前元素的行号, 则比较其列号, 将较小列的元素存入 C
            中; 若列号也相等, 则将对应的元素值相加后存入 C 中*/
            if (A [i] [2] < B [j] [2]) {

```

```

        C[k][0]=A[i][0];
        C[k][1]=A[i][1];
        C[k][2]=A[i][2];
        ++k;
        ++i;
    }
    else if(A[i][2]>B[j][2]) {
        C[k][0]=B[j][0];
        C[k][1]=B[j][1];
        C[k][2]=B[j][2];
        ++k;
        ++j;
    }
    else{
        m=A[i][0]+B[j][0];
        if(m!=0) { //m 有可能为 0 , 不为 0 才添加到 C 中
            C[k][1]=B[j][1];
            C[k][2]=B[j][2];
            C[k][0]=m;
            ++k;
        }
        ++i;
        ++j;
    }
}
else if(A[i][1]<B[j][1]) {
    //若 A 当前元素的行号小于 B 当前元素的行号, 则将 A 的元素存入 C 中
    C[k][0]=A[i][0];
    C[k][1]=A[i][1];
    C[k][2]=A[i][2];
    ++k;
    ++i;
}
else{ //若 A 当前元素的行号大于 B 当前元素的行号, 则将 B 的元素存入 C 中
    C[k][0]=B[j][0];
    C[k][1]=B[j][1];
    C[k][2]=B[j][2];
    ++k;
    ++j;
}
//A 中有剩余元素, B 已经处理完毕, 将 A 中元素直接放入 C 中
while(i<=A[0][0]) {
    C[k][0]=A[i][0];
    C[k][1]=A[i][1];
    C[k][2]=A[i][2];
    ++k;
    ++i;
}
//B 中有剩余元素, A 已经处理完毕, 将 B 中元素直接放入 C 中
while(j<=B[0][0]) {
    C[k][0]=B[j][0];
    C[k][1]=B[j][1];
    C[k][2]=B[j][2];
    ++k;
    ++j;
}
C[0][0]=k-1; //产生第 0 行的结果

```

```

        C[0][1]=A[0][1];
        C[0][2]=A[0][2];
    }
}

```

4. 算法分析: 本题在原理上与普通的矩阵相乘算法没有太大区别, 其核心问题在于, 根据给出的行号和列号找出原矩阵中的对应元素值, 因此构造了一个函数 `getvalue()`, 当在三元组表示中找到时返回其元素值, 找不到说明原该位置处的元素值为 0, 因此返回 0。然后利用该函数计算出 `C` 的行号 `i` 和列号 `j` 处的元素值, 若该值不为 0, 则存入其三元组表示的矩阵中, 否则不存入。算法实现代码如下:

```

int getvalue ( int D[ ][ maxSize], int i, int j) {
//返回 D 对应的稀疏矩阵 A 中 (i,j) 位置上的值
    int k=1;
    while(k<=D[0][0] &&(D[k][1] !=i || D[k][2] !=j))
        k++;
    if (k<=D[0][0])
        return D[k][0];
    else
        return 0;
}

void mul ( int A[ ][3],int B[][3],int C[][3], int m, int n , int k) {
//矩阵相乘
    int i,j,l,p=1,s;
    for(i=0;i<m;++i)
        for (j=0;j<k;++j) {
            s=0;
            for(l=0;l<n;++l)
                s+=getvalue(A,i,l)*getvalue(B,l,j);
            if(s!=0){ //产生一个三元组元素
                C[p][1]=i;
                C[p][2]=j;
                C[p][0]=s;
                ++p;
            }
        }
    C[0][1]=m; //产生第 0 行的结果
    C[0][2]=k;
    C[0][0]=p-1;
}

```

9.2.3 矩阵

1. 设计一个算法, 对给定的一个整型 $m \times n$ 矩阵 `A`, 统计这个矩阵中具有下列特征的元素个数并输出它们的坐标及数值: 它们既是所在行中的最小值, 又是所在列中的最小值; 或者它们既是所在行中的最大值, 又是所在列中的最大值。假设矩阵中元素各不相同, 要求结果在处理过程中用输出语句输出。

【答案解析】

1. 算法分析: 本题较为简单, 只需找出某行上的最小值 (最大值), 然后判断其是否为对应列上的最小值 (最大值), 若是则输出。

```

void print min (int A[ ][ maxSize],int m,int n) {
    int i,j,k,min,minj; //用 minj 来记录第 i 行上最小值的列号
}

```

```

int flag;
for(i=0;i<m;++i) {
    min=A[i][0];
    minj=0;
    for(j=1;j<n;++j) //找出第 i 行上的最小值, 列号为 min j
        if(A[i][j]<min) {
            min=A[i][j];
            minj=j;
        }
    flag=1;
    for(k=0;k<m;++k) //判断 min 是否为 min j 列上的最小值
        if(min>A[k][minj]) {
            flag=0;
            break;
        }
    if(flag)
        cout<<min<<", ["<<i<<", "<<minj<<"]<<" ";
        //打印打最小值, 其格式为 min , [i,minj]
    }
    cout<<endl;
}

void printmax (int A[] [ maxSize] , int m, int n) {
    inti,J,k,max,maxj;
    int flag;
    for(i=0;i<m;++i) { //理处第 i 列
        max=A[i][0];
        maxj=0;
        for(j=1;j<n;++j) //找出第 i 行上的最大值, 列号为 ma x j
            if(A[i][j]>max) {
                max=A[i][j];
                maxj=j;
            }
        flag=1;
        for (k=0;k<m; ++ k) //判断 max 是否为 maxj 列上的最大值
            if(max<A[k][maxj]) {
                flag=0;
                break;
            }
        if (flag)
            cout<<max<<", [ "<<i<<", "<<maxj<<" J "<<" ";//打印最大值, 其格式为 max , [i,maxj]
        }
        cout<<endl ;
    }
}

```


第三部分：真题聚类篇

类型一：算法时间复杂度与空间复杂度

1、【2011.01】设 n 是描述问题规模的非负整数，下面程序片段的时间复杂度是_____。

```
x=2;
while (x<n/2)
    x = 2 * x;
```

A. $O(\log_2 n)$ B. $O(n)$ C. $O(n\log_2 n)$ D. $O(n^2)$

2、【2012.01】求整数 $n(n \geq 0)$ 阶乘的算法如下，其时间复杂度是_____。

```
int fact(int n) {
    if (n<=1) return 1;
    return n*fact(n-1);
}
```

A. $O(\log_2 n)$ B. $O(n)$ C. $(n\log_2 n)$ D. $O(n^2)$

3、【2013.01】已知两个长度分别为 m 和 n 的升序链表，若将它们合并为一个长度为 $m+n$ 的降序链表，则最坏情况下的时间复杂度是_____。

A. $O(n)$ B. $O(m*n)$ C. $O(\min(m,n))$ D. $O(\max(m,n))$

4、【2014.01】下列程序段的时间复杂度是_____。

```
count=0;
for(k=1;k<=n;k*=2)
    for(j=1;j<=n;j++)
        count++;
```

A. $O(\log_2 n)$ B. $O(n)$ C. $O(n\log_2 n)$ D. $O(n^2)$

5、【2017.01】下列函数的时间复杂度是_____。

```
int func (int n){
    int i=0, sum=0;
    while(sum< n) sum += ++ i;
    return i;
}
```

A. $O(\log_2 n)$ B. $O(n^{1/2})$ C. $O(n)$ D. $O(n\log_2 n)$

6、【2019.01】设 n 是描述问题规模的非负整数，下列程序段的时间复杂度是

```
x=0;
while(n>=(x+1)*(x+1))
    x=x+1;
```

A. $O(\log n)$ B. $O(n^{1/2})$ C. $O(n)$ D. $O(n^2)$

7、【2022.01】下列程序段的时间复杂度是 ()。

```
int sum=0;
for(int i=1;i<=n;i*=2)
    for(int j=1;j<i;j++)
        sum++;
```

A. $O(\log n)$ B. $O(n)$ C. $O(n\log n)$ D. $O(n^2)$

8、【2025.01】下列程序段的时间复杂度是：

```
int count=0,i,j;
for(i=1;i*i<=n;i++)
    for(j=1;j<=i;j++)
        count++;
```

A.O ($\log n$)

B.O (n)

C.O ($n \log n$)

D.O (n^2)

【答案解析】

1、A。本题对应于时间复杂度计算的 $\log$ 型，最内层循环语句是 $x=2*x$;

执行 1 次， $x = x * 2$

执行 2 次， $x = x * 2 * 2$

依次类推，执行 K 次， $j = K$ ， $x = x * 2 * 2 * \dots * 2$ ，一共乘了 K 个 2，于是 $\text{sum} = 2^K < n$ ；得到时间复杂度是 $O(\log_2 n)$ ， $T(n) = O(\log_2 n)$ 。

2、B。本题对应于时间复杂度计算的递归型，本算法的最内层语句是： $n * \text{fact}(n-1)$ ；每递归一次， n 减 1，共运行 $n-1$ 次，时间复杂度是 $O(n)$ 。递归运算，即算法中出现了调用自身的情形。递归的边界条件是 $n-1$ ，每调用一次 $\text{fact}()$ ，传入该层 $\text{fact}()$ 的参数值减 1。采用递归式来表示时间复杂度有

$$T(n) = \begin{cases} O(1) & n \leq 1 \\ T(n-1) + 1 & n > 1 \end{cases}$$

则 $T(n) = T(n-1) + 1 = T(n-2) + 2 = \dots = T(1) + n-1 = O(n)$ ，故时间复杂度为 $O(n)$ 。

3、D。 m 、 n 是两个升序链表，长度分别为 m 和 n 。两个升序链表合并，两两比较表中元素，每比较一次确定一个元素的位置。当一个链表比较结束后，将另一个链表的剩余元素插入即可。考虑两种极端情况：

(1) 最好的情况：其中一个表的元素的值比另外一个表的元素都大，此时比较次数最小，等于元素比较小的表的长度；

(2) 最坏的情况：两个链表中的元素依次进行比较，比较的次数最多是两个表的长度和。最坏的情况是两个链表中的元素依次进行比较，直到两个链表都到表尾，即每个元素都经过比较。但是不管哪种情况，其时间复杂度为 $O(m+n) \cong O(\max(m,n))$ 。

4、C。本题对应于时间复杂度计算的递归型。外层循环条件为 $k \leq n$ ，增量定义为 $k*=2$ ，可知循环次数为 $2^k \leq n$ ，即 $k \leq \log_2 n$ 。内层循环条件 $j \leq n$ 与外层循环的变量无关，每次循环 j 自增 1，每次内层循环都执行 n 次。对于嵌套循环，该段程序的时间复杂度 $T(n) = T_1(n)T_2(n) = O(n)O(\log_2 n) = O(n \log_2 n)$ ，选 C。

5、B。 $\text{sum} += ++i$ ；相当于 $++i$ ； $\text{sum} = \text{sum} + i$ ；。进行到第 k 趟循环， $\text{sum} = (1+k)*k/2$ 。显然需要进行 $O(n^{1/2})$ 趟循环，因此这也是该函数的时间复杂度。递归数列求和， sum 是 n^2 级别，所以复杂度为 $O(n^{1/2})$ 。

6、B。假设第 k 次循环终止，则第 k 次执行时， $(x+1)^2 > n$ ， x 的初始值为 0，第 k 次判断时， $x=k-1$ ，即 $k^2 > n$ ， $k > \sqrt{n}$ ，因此该程序段的时间复杂度为 $O(\sqrt{n})$ 。因此选 B。

7、B。本题为多层循环中嵌套循环指针相关类型题，题目中采用的是两重循环，外层循环用 K 表示执行的轮次，则有 $n = 2^K$ ，所以 $K = \log_2 n$ ；于是，

当 $K = 1$ 时，内层循环执行 1 次，

当 $K = 2$ 时，内层循环执行 2 次，

当 $K = 3$ 时，内层循环执行 4 次，

当 $K = \log_2 n$ 时，内存循环执行了 2^{K-1} 次。

于是内重循环次数是 $\underbrace{1 + 2 + 4 + \dots + 2^{K-1}}_{\log_2 n \text{ 项}} = 2^0 + 2^1 + \dots + 2^{\log_2 n - 1} = \frac{1 - 2^{\log_2 n}}{1 - 2} = n - 1 = O(n)$ 。

8、B。外层循环次数约为 $\sqrt{n}$ ；对每个 i ，内层循环运行 i 次，故总操作量为 $1 + 2 + \dots + \sqrt{n}$ ，约为 $(\sqrt{n})(\sqrt{n} + 1)/2$ ，即 $O(n)$ 。因此，整体时间复杂度为 $O(n)$ 。

类型二：线性表的顺序结构和操作

1、【2023.01】下列对顺序存储的有序表（长度为 n ）实现给定操作的算法中，平均时间复杂度为 $O(1)$ 的是（ ）

- A. 查找包含指定值元素的算法
B. 插入包含指定值元素的算法
C. 删除第 $i (1 \leq i \leq n)$ 个元素的算法
D. 获取第 $i (1 \leq i \leq n)$ 个元素的算法

【答案解析】

1、D。只要确定了存储线性表的起始位置，线性表中任一数据元素都可随机存取，所以线性表的顺序存储结构是一种随机存取的存储结构，因此获取第 i 个值的算法的时间复杂度为 $O(1)$ 。查找包含指定值元素的值需要从头到尾遍历顺序表直至找到目标元素，因此时间复杂度为 $O(n)$ 。插入包含指定值元素时，若在第 i 个位置插入，需要将第 i 个位置及其之后的元素都向后移动一个位置，若在其他位置插入那么时间复杂度为 $O(n)$ 。删除第 i 个元素时，需要将 i 后面的元素向前移动一个位置，因此时间复杂度为 $O(n)$ 。

类型三：链表的操作

1、【2016.02】已知一个带有表头结点的双向循环链表 L ，结点结构为，

prev	data	next
------	------	------

其中， $prev$ 和 $next$ 分别是指向其直接前驱和直接后继结点的指针。现要删除指针 p 所指的结点，正确的语句序列是_____。

- A. $p \rightarrow next \rightarrow prev = p \rightarrow prev$; $p \rightarrow prev \rightarrow next = p \rightarrow prev$; $free(p)$;
B. $p \rightarrow next \rightarrow prev = p \rightarrow next$; $p \rightarrow prev \rightarrow next = p \rightarrow next$; $free(p)$;
C. $p \rightarrow next \rightarrow prev = p \rightarrow next$; $p \rightarrow prev \rightarrow next = p \rightarrow prev$; $free(p)$;
D. $p \rightarrow next \rightarrow prev = p \rightarrow prev$; $p \rightarrow prev \rightarrow next = p \rightarrow next$; $free(p)$;

2、【2021.01】已知头指针 h 指向一个带头结点的非空单循环链表，结点结构为，

data	next
------	------

其中 $next$ 是指向直接后继结点的指针， p 是尾指针， q 是临时指针。现要删除该链表的第一个元素，正确的语句序列是

- A. $h \rightarrow next = h \rightarrow next \rightarrow next$; $q = h \rightarrow next$; $free(q)$;
B. $q = h \rightarrow next$; $h \rightarrow next = h \rightarrow next \rightarrow next$; $free(q)$;
C. $q = h \rightarrow next$; $h \rightarrow next = q \rightarrow next$; $if(p \neq q) p = h$; $free(q)$;
D. $q = h \rightarrow next$; $h \rightarrow next = q \rightarrow next$; $if(p == q) p = h$; $free(q)$;

3、【2023.02】现有非空双向链表 L ，其结点结构为：

prev	data	next
------	------	------

$prev$ 是指向前直接前驱结点的指针， $next$ 是指向直接后继结点的指针。若要在 L 中指针 p 所指向的结点（非尾结点）之后插入指针 s 指向的新结点，则在执行了语句序列：“ $s \rightarrow next = p \rightarrow next$;
 $p \rightarrow next = s$ ”后，下列语句序列中还需要执行的是（ ）

- A. $s \rightarrow next \rightarrow prev = p$; $s \rightarrow prev = p$;
B. $p \rightarrow next \rightarrow prev = s$; $s \rightarrow prev = p$;
C. $s \rightarrow prev = s \rightarrow next \rightarrow prev$; $s \rightarrow next \rightarrow prev = s$;
D. $p \rightarrow next \rightarrow prev = s \rightarrow prev$; $s \rightarrow next \rightarrow prev = p$;

- 4、【2024.01】已知带头结点的非空单链表 L 的头指针为 h，结点结构为，其中 next 是指向直接后继结点的指针。现有指针 p 和 q，若 p 指向 L 中非首且非尾的任意一个结点，则执行语句序列“ $q=p \rightarrow next; p \rightarrow next=q \rightarrow next; q \rightarrow next=h \rightarrow next; h \rightarrow next=q$ ”的结果是：
- A、把 p 所指结点后插入 q 所指结点
B、把 q 所指结点后插入 p 所指结点
C、将 p 所指结点移动到 L 的头结点之后
D、将 q 所指结点移动到 L 的头结点之后

【答案解析】

1、D。考察双向链表的删除，删除的过程中注意不要断开链表。

2、D。本题考查链表中结点的删除操作，带有头结点， $q=h \rightarrow next; h \rightarrow next=h \rightarrow next \rightarrow next (h \rightarrow next=q \rightarrow next)$ ；但是因为这是循环单链表，若通过判断确定该单循环链表只有一个结点，则该结点既是链表的第一个元素，也是最后一个元素，则删除该元素后，链表变成带头结点的空链表，尾指针即是头指针，此时 $if(p==q) p=h$ ，此时头尾指向头结点。

3、C。将 s 指向结点插入 p 指向结点之后需要修改指针使 p 指向结点作为 s 指向结点的前驱，再将 p 指向结点的后继结点作为 s 结点的后继结点。执行“ $s \rightarrow next=p \rightarrow next$ ”后，将 s 指向结点的 next 指针指向 p 指向结点的后继结点，“ $p \rightarrow next=s$ ”将 p 指向结点的 next 指针指向 s 结点。还需要将 s 结点的 prer 指针指向 p 结点的后继结点，将 s 指向结点的 prer 域指向 p 结点即“ $s \rightarrow next \rightarrow prer=p \rightarrow next; s \rightarrow prer=p$ ”，如图 1 所示。

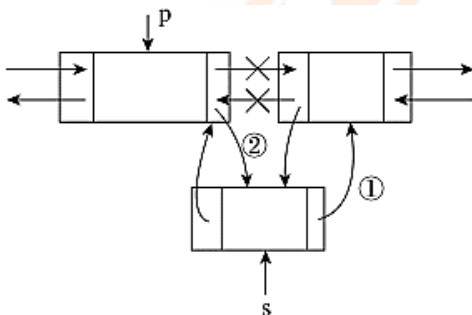


图 1

- 4、D。这段代码的功能是：把 q 指向的节点插入到 h 的后面。所以,选项 D 是正确的。
- 这段代码的解析如下：
- ```
q->next=p->next; // q 指向 p 的下一个节点
p->next=q->next; // p 跳过 q,直接指向 q 的下一个节点
q->next=h->next; // q 的下一个节点设置为 h 的下一个节点
h->next=q; // h 的下一个节点设置为 q
所以，这段代码的功能是把 q(原来是 p 的下一个节点)移动到链表的第二个位置(即 h 的一个结点)。
```

## 类型四：静态链表

- 1、【2016.01】已知表头元素为 c 的单链表在内存中的存储状态如下表所示。

| 地址    | 元素 | 链接地址  |
|-------|----|-------|
| 1000H | a  | 1010H |
| 1004H | b  | 100CH |



|       |   |       |
|-------|---|-------|
| 1008H | c | 1000H |
| 100CH | d | NULL  |
| 1010H | e | 1004H |
| 1014H |   |       |

现将f存放于1014H处并插入到单链表中,若f在逻辑上位于a和e之间,则a, e, f的“链接地址”依次是\_\_\_\_\_。

- A. 1010H, 1014H, 1004H      B. 1010H, 1004H, 1014H  
C. 1014H, 1010H, 1004H      D. 1014H, 1004H, 1010H

### 【答案解析】

1、D。链表应该是: a->f->e, 所以 a 的链接地址应该是 1014H, f 的链接地址应该是 1010H, e 的链接地址不变。

## 类型五: 线性表算法应用

### 1. 双指针策略

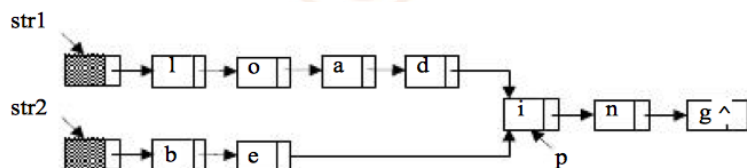
1、【2009.42】已知一个带有表头结点的单链表, 结点结构为:

| data | link |
|------|------|
|------|------|

假设该链表只给出了头指针 list。在不改变链表的前提下, 请设计一个尽可能高效的算法, 查找链表中 倒数第 k 个位置上的结点(k 为正整数)。若查找成功, 算法输出该结点的 data 域的值, 并返回 1; 否则, 只 返回 0。要求:

- (1) 描述算法的基本设计思想;
- (2) 描述算法的详细实现步骤;
- (3) 根据设计思想和实现步骤, 采用程序设计语言描述算法 (使用 C、C++语言实现), 关键之处请给出简要注释。

2、【2012.42】假定采用带头结点的单链表保存单词, 当两个单词有相同的后缀时, 则可共享相同的后缀存储空间, 例如, “loading”和“being”的存储映像如下图所示。



设 str1 和 str2 分别指向两个单词所在单链表的头结点, 链表结点结构为

| data | link |
|------|------|
|------|------|

请设计一个时间上尽可能高效的算法, 找出由 str1 和 str2 所指向两个链表共同后缀的起始位置 (如图中字符 i 所在结点的位置 p)。要求:

- (1) 给出算法的基本设计思想。
- (2) 根据设计思想, 采用 C 或 C++语言描述算法, 关键之处给出注释。
- (3) 说明你所设计算法的时间复杂度。

3、【2016.43】已知由  $n(n \geq 2)$  个正整数构成的集合  $A = \{a_k | 0 \leq k < n\}$ , 将其划分为两个不相交的子集  $A_1$  和  $A_2$ , 元素个数分别是  $n_1$  和  $n_2$ ,  $A_1$  和  $A_2$  中元素之和分别为  $S_1$  和  $S_2$ , 设计一个尽可能高效的

划分算法, 满足  $|n_1 - n_2|$  最小且  $|S_1 - S_2|$  最大。要求:

- (1) 给出算法的基本设计思想
- (2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。
- (3) 说明你所设计算法的平均时间复杂度和空间复杂度

### 【答案解析】

1、(1) 算法的基本设计思想: 问题的关键是设计一个尽可能高效的算法, 通过链表的一趟遍历, 找到倒数第  $k$  个结点的位置。算法的基本设计思想是: 定义两个指针变量  $p$  和  $q$ , 初始时均指向头结点的下一个结点 (链表的第一个结点)。 $p$  指针沿链表移动; 当  $p$  指针移动到第  $k$  个结点时,  $q$  指针开始与  $p$  指针同步移动; 当  $p$  指针移动到最后一个结点时,  $q$  指针所指示结点为倒数第  $k$  个结点。以上过程对链表仅进行一遍扫描。

(2) 算法的详细实现步骤:

- ①  $\text{count}=0$ ,  $p$  和  $q$  指向链表表头结点的下一个结点;
- ② 若  $p$  为空, 转⑤;
- ③ 若  $\text{count}$  等于  $k$ , 则  $q$  指向下一个结点; 否则,  $\text{count}=\text{count}+1$ ;
- ④  $p$  指向下一个结点, 转②;
- ⑤ 若  $\text{count}$  等于  $k$ , 则查找成功, 输出该结点的  $\text{data}$  域的值, 返回 1; 否则, 说明  $k$  值超过了线性表的长度, 查找失败, 返回 0;
- ⑥ 算法结束。

(3) 算法实现:

```
typedef int ElemType; // 链表数据的类型定义
typedef struct LNode{ // 链表结点的结构定义
 ElemType data; // 结点数据
 struct LNode *link; // 结点链接指针
} *LinkList;
// 查找链表 list 倒数第 k 个结点, 并输出该结点 data 域的值
int Search_k(LinkList list, int k){
 LinkList p = list->link, q = list->link; // 指针 p、q 指示第一个结点
 int count = 0;
 while(p != NULL){ // 遍历链表直到最后一个结点
 if(count < k) count++; // 计数, 若 count < k 只移动 p
 else { q = q->link; }
 p = p->link; // 之后让 p、q 同步移动
 } // while
 if(count < k) return 0; // 查找失败返回 0
 else { // 否则打印并返回 1
 printf("%d", q->data); return 1;
 }
} // Search_k
```

2、(1) 算法思想: 顺序遍历两个链表到尾结点时, 并不能保证两个链表同时到达尾结点。这是因为两个链表的长度不同。假设一个链表比另一个链表长  $k$  个结点, 我们先在长链表上遍历  $k$  个结点, 之后同步遍历两个链表。这样我们就能保证它们同时到达最后一个结点了。由于两个链表从第一个公共结点到链表的尾结点都是重合的。所以它们肯定同时到达第一个公共结点。于是得到算法思路: ① 遍历两个链表求的它们的长度  $L_1, L_2$ ; ② 比较  $L_1, L_2$ , 找出较长的链表, 并求

$L=[L1-L2]$ ; ③ 先遍历长链表的  $L$  各结点;

(2) 算法的 C 语言代码描述

```
typedef int ElemType; // 链表数据的类型定义
typedef struct LNode // 链表结点的结构定义
{
 ElemType data; // 结点数据
 struct LNode *link; // 结点链接指针
} *LinkList;
// 本算法实现线性时间内找到两个单链表的第一个公共结点
LinkList Search_First_Common(LinkList L1, LinkList L2) {
 int len1 = Length(L1), len2 = Length(L2);
 LinkList longList, shortList; // 分别指向较长和较短的链表
 if (len1 > len2) {
 longList = L1->next;
 shortList = L2->next;
 L = len1 - len2; // 表长之差
 } else {
 longList = L2->next;
 shortList = L1->next;
 L = len2 - len1; // 表长之差
 }
 while (L--)
 longList = longList->next;
 while (longList != NULL) {
 if (longList == shortList) // 同步寻找共同结点
 return longList;
 else {
 longList = longList->next;
 shortList = shortList->next;
 }
 } // while
 return NULL;
}
```

(3) 算法的时间复杂度为  $O(\text{len1} + \text{len2})$ , 空间复杂度为  $O(1)$ 。

### 3、(1) 算法的基本设计思想

由题意知, 将最小的  $(n/2)$  向下取整元素放在  $A1$  中, 其余的元素放在  $A2$  中, 分组结果即可满足题目要求。仿照快速排序的思想, 基于枢轴将  $n$  个整数划分为两个子集。根据划分后枢轴所处的位置  $i$  分别处理

① 若  $i = (n/2)$  向下取整, 则分组完成, 算法结束;

② 若  $i < (n/2)$  向下取整, 则枢轴及之前的所有元素均属于  $A1$ , 继续对  $i$  之后的元素进行划分;

③ 若  $i > (n/2)$  向下取整, 则枢轴及之后的所有元素均属于  $A2$ , 继续对  $i$  之前的元素进行划分;

基于该设计思想实现的算法, 毋须对全部元素进行全排序, 其平均时间复杂度是  $O(n)$ , 空间复杂度是  $O(1)$ 。

(2) 算法实现

```
int setPartition(int a[], int n) {
 int pivotkey, low = 0, low0 = 0, high = n-1, high0 = n-1,
 int flag = 1, k = n / 2;
 int s1 = 0, s2 = 0;
 while (flag) {
 pivotkey = a[low];
 while (low < high) {
```

```

 while(low<high && a[high]>=pivotkey) --high;
 if(low!=high) a[low] = a[high];
 while(low<high && a[low]<=pivotkey) ++low;
 if(low!=high) a[low] = a[high];
 }
 //end of while(low<high)
 a[low] = pivotkey;
 if(low == k-1)
 flag = 0;
 else{
 if(low<k-1){
 low0 = ++low;
 high = high0;
 }else{
 high0 = --high;
 low = low0;
 }
 }
}
for(int i = 0; i<k; i++) s1 += a[i];
for(int i = k; i<n; i++) s2 += a[i];
return s2 - s1;
}

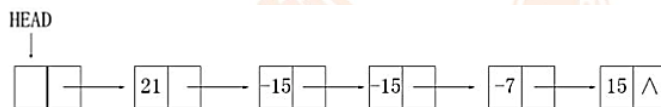
```

## 2. 哈希策略

1、【2013.41】已知一个整数序列  $A=(a_0, a_1, \dots, a_{n-1})$ , 其中  $0 \leq a_i \leq n (0 \leq i \leq n)$ 。若存在  $a_{p_1}=a_{p_2}=\dots=a_{p_m}=x$  且  $m>n/2 (0 \leq p_k < n, 1 \leq k \leq m)$ , 则称  $x$  为  $A$  的主元素。例如  $A=(0,5,5,3,5,7,5,5)$ , 则 5 为主元素; 又如  $A=(0,5,5,3,5,1,5,7)$ , 则  $A$  中没有主元素。假设  $A$  中的  $n$  个元素保存在一个一维数组中, 请设计一个尽可能高效的算法, 找出  $A$  的主元素。若存在主元素, 则输出该元素; 否则输出-1。要求:

- (1) 给出算法的基本设计思想。
- (2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。
- (3) 说明你所设计算法的时间复杂度和空间复杂度。

2、【2015.41】用单链表保存  $m$  个整数, 结点的结构为:  $[data][link]$ , 且  $|data| \leq n$  ( $n$  为正整数)。现要求设计一个时间复杂度尽可能高效的算法, 对于链表中  $data$  的绝对值相等的结点, 仅保留第一次出现的结点而删除其余绝对值相等的结点。例如, 若给定的单链表 head 如下:



则删除结点后的 head 为:



要求:

- (1) 给出算法的基本设计思想。
- (2) 使用 C 或 C++ 语言, 给出单链表结点的数据类型定义。
- (3) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。
- (4) 说明你所设计算法的时间复杂度和空间复杂度。



3、【2018.41】给定一个含  $n$  ( $n \geq 1$ ) 个整数的数组, 请设计一个在时间上尽可能高效的算法, 找出数组中未出现的最小正整数。例如, 数组  $\{-5, 3, 2, 3\}$  中未出现的最小正整数是 1; 数组  $\{1, 2, 3\}$  中未出现的最小正整数是 4。要求:

- (1) 给出算法的基本设计思想
- (2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。
- (3) 说明你所设计的算法的时间复杂度和空间复杂度

### 【答案解析】

1、(1) 算法思想: 算法的基本设计思想 算法的核心思想是用空间换时间。使用辅助数组记录链表中已出现的数值, 从而只需对链表进行 一趟扫描。因为  $|data| \leq n$ , 故辅助数组  $q$  的大小为  $n+1$ , 各元素的初值均为 0。依次扫描链表中的各结点, 同时检查  $q[|data|]$  的值, 如果为 0, 并令  $q[|data|]=1$ ; 否则,  $q[|data|] += 1$ 。

(2) 算法实现:

```
int findMajorData () {
int i,c,count=1;
int *q,m;
q=(int *)malloc(sizeof(int)*(n+1)); //申请 n+1 个位置的辅助空间
for(int i=0;i<n+1;i++) //数组元素初值置 0
 *(q+i)=0;
while(p->link!=NULL){
 if(*(q+m)==0) { //判断该结点的 data 是否已出现过
 *(q+m) = 1; //首次出现
 }else{ //重复出现
 *(q+m) += 1
 }
}
for(i=0;i<n;i++){
 if(*(q+i)>n/2){
 return i; //确认候选主元素
 }
}
return -1; //不存在主元素
}
```

(3) 时间复杂度是  $O(n)$ , 空间复杂度是  $O(n)$

2、(1) 算法的基本设计思想 算法的核心思想是用空间换时间。使用辅助数组记录链表中已出现的数值, 从而只需对链表进行 一趟扫描。 因为  $|data| \leq n$ , 故辅助数组  $q$  的大小为  $n+1$ , 各元素的初值均为 0。依次扫描链表中的各结点, 同 时检查  $q[|data|]$  的值, 如果为 0, 则保留该结点, 并令  $q[|data|]=1$ ; 否则, 将该结点从链表中删除。

(2) 使用 C 语言描述的单链表结点的数据类型定义

```
typedef struct node {
int data;
struct node *link;
}NODE;
Typedef NODE *PNODE;
```

(3) 算法实现

```
void func (PNODE h,int n){
PNODE p=h,r;
int *q,m;
```

```

q=(int *)malloc(sizeof(int)*(n+1)); //申请 n+1 个位置的辅助空间
for(int i=0;i<n+1;i++) //数组元素初值置 0
 *(q+i)=0;
while(p->link!=NULL){
 m=p->link->data>0? p->link->data:-p->link->data;
 if(*(q+m)==0) { //判断该结点的 data 是否已出现过
 *(q+m)=1; //首次出现
 p=p->link; //保留
 }else{ //重复出现
 r=p->link; //删除
 p->link=r->link;
 free(r);
 }
 p = p->link;
}
free(q);
}

```

(4) 所给算法的时间复杂度为  $O(m)$ , 空间复杂度为  $O(n)$ 。

3、(1) 给出算法的基本设计思想: 设要查找的数组中未出现的最小正整数为  $K$ 。 $K$  取值范围只能是  $[1, n+1]$ 。采用类似基数排序的思想, 分配一个数组  $B[n]$ , 用来标记  $A$  中是否出现了  $1 \sim n$  之间的正整数。从左至右依次扫描数组元素  $A[i]$  并标记数组  $B$ 。若  $A[i]$  是负数、零或是大于  $n$ , 则忽略该值; 否则, 根据计数排序的思想将  $B[A[i]-1]$  置为 1。标记完毕, 遍历数组  $B$ , 查找第一个值为 0 的元素, 其下标+1 即为目标元素  $K$ ; 找不到 0 时,  $K = n+1$ 。

(2) 算法实现:

```

int findMissMin(int A[], int n){
 int i, *B; //标记数组
 B = (int *)malloc(sizeof(int)*n); //分配空间
 memset(B, 0, sizeof(int)*n); //赋初值为 0
 for(i = 0; i < n; i++){
 if(A[i] > 0 && A[i] <= n) //若 A[i] 的值介于 1~n, 则标记数组 B
 B[A[i]-1] = 1;
 }
 for(i = 0; i < n; i++) //扫描计数数组, 找到目标值 K
 if(B[i] == 0) break;
 Return i+1; //返回结果
}

```

(3) 以上算法的时空复杂度都是  $O(n)$ 。

### 3. 单链表或者数组逆序

1、【2019.41】设线性表  $L=(a_1, a_2, a_3, \dots, a_{n-2}, a_{n-1}, a_n)$  采用带头结点的单链表保存, 链表中结点定义如下:

```

typedef struct node{
 int data;
 struct node * next;
} NODE;

```

请设计一个空间复杂度为  $O(1)$  且时间上尽可能高效的算法, 重新排列  $L$  中的各结点, 得到线性表  $L'=(a_1, a_n, a_2, a_{n-1}, a_{n-2}, \dots)$ 。

要求:

(1) 给出算法的基本设计思想。

- (2) 根据设计思想, 采用 c 或 c++ 语言描述算法, 关键之处给出注释。  
(3) 说明你所设计的算法的时间复杂度。

### 【答案解析】

1、(1) 算法的基本设计思想: 算法分 3 步完成。

第 1 步, 采用两个指针交替前行, 找到单链表的中间结点;

第 2 步, 将单链表的后半段结点原地逆置;

第 3 步, 从单链表前后两段中依次各取一个结点, 按要求重排。

(2) 算法实现:

```
void change_list(NODE * h){
 NODE *p,* q,* r,* s;
 p=q=h;
 while(q->next !=NULL){ //寻找中间结点
 p=p->next; //p 走一步
 q=q->next;
 if(q->next!=NULL) q=q->next; //q 走两步
 }
 q=p->next; //p 所指结点为中间结点, q 为后半段链表的首结点
 p->next=NULL;
 while(q!=NULL) { //将链表后半段逆置
 r=q->next;
 q->next= p->next;
 p->next=q
 q=r;
 }
 s=h->next; //s 指向前半段的第一个数据结点, 即插入点
 q=p->next; //q 指向后半段的第一个数据结点
 p->next=NULL;
 while(q!=NULL) { //将链表后半段的结点插入到指定位置
 r=q->next; //r 指向后半段的下一个结点
 q->next=s->next; //将 q 所指结点插入到 s 所指结点之后
 s->next=q;
 s=q->next; //s 指向前半段的下一个插入点
 q=r;
 }
}
```

(3) 算法的时间复杂度: 时间复杂度为  $O(n)$ 。

## 4. 单链表或者数组旋转

1、【2010.42】设将  $n$  ( $n>1$ ) 个整数存放于一维数组  $R$  中。试设计一个在时间和空间两方面都尽可能高效的算法。将  $R$  中保存的序列循环左移  $p$  ( $0<p<n$ ) 个位置, 即将  $R$  中的数据由  $(X_0, X_1, \dots, X_{n-1})$  变换为  $(X_p, X_{p+1}, \dots, X_{n-1}, X_0, X_1, \dots, X_{p-1})$ 。要求:

- (1) 给出算法的基本设计思想。  
(2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。  
(3) 说明你所设计算法的时间复杂度和空间复杂度。

### 【答案解析】

1、(1) 算法的基本设计思想: 可以将这个问题看做是把数组  $ab$  转换成数组  $ba$  ( $a$  代表数组的前  $p$  个元素,  $b$  代表数组中余下的  $n-p$  个元素), 先将  $a$  逆置得到  $a^{-1}b$ , 再将  $b$  逆置得到  $a^{-1}b^{-1}$ , 最后将整个  $a^{-1}b^{-1}$  逆置得到  $(a^{-1}b^{-1})^{-1}=ba$ 。设  $Reverse$  函数执行将数组元素逆置的操作, 对  $abcdefgh$  向左循环移动 3 ( $p=3$ ) 个位置的过程如下:

$Reverse(0,p-1)$ 得到  $cbadefgh$ ;

$Reverse(p,n-1)$ 得到  $cbahgfed$ ;

$Reverse(0,n-1)$ 得到  $defghabc$ ;

注:  $Reverse$  中, 两个参数分别表示数组中待转换元素的始末位置。

(2) 使用  $c$  语言描述算法如下:

```
void Reverse(int R[],int from,int to) {
 int i,temp;
 for(i = 0; i < (to-from+1)/2; i++){
 temp = R[from+i];
 R[from+i] = R[to-i];
 R[to-i] = temp;
 }
} //Reverse
void Converse(int R[],int n,int p){
 Reverse(R,0,p-1);
 Reverse(R,p,n-1);
 Reverse(R,0,n-1);
}
```

(3) 上述算法中三个  $Reverse$  函数的时间复杂度分别为  $O(p/2)$ 、 $O((n-p)/2)$  和  $O(n/2)$ , 故所设计的算法的时间复杂度为  $O(n)$ , 空间复杂度为  $O(1)$

## 类型六: 栈的基础概念和性质

1、【2009.02】设栈  $S$  和队列  $Q$  的初始状态均为空, 元素  $a,b,c,d,e,f,g$  依次进入栈  $S$ 。若每个元素出栈后立即进入队列  $Q$ , 且 7 个元素出队的顺序是  $b,d,c,f,e,a,g$ , 则栈  $S$  的容量至少是\_\_\_\_\_。

A. 1                      B. 2                      C. 3                      D. 4

2、【2010.01】若元素  $a、b、c、d、e、f$  依次进栈, 允许进栈、退栈操作交替进行, 但不允许连续三次进行退栈操作, 则不可能得到的出栈序列是\_\_\_\_\_。

A.  $dcebf a$               B.  $cbdaef$               C.  $bcaefd$               D.  $afedcb$

3、【2011.02】元素  $a, b, c, d, e$  依次进入初始为空的栈中, 若元素进栈后可停留、可出栈, 直到所有元素都出栈, 则在所有可能的出栈序列中, 以元素  $d$  开头的序列个数是\_\_\_\_\_。

A. 3                      B. 4                      C. 5                      D. 6

4、【2013.02】一个栈的入栈序列为  $1, 2, 3, \dots, n$ , 其出栈序列是  $p_1, p_2, p_3, \dots, p_n$ 。若  $p_2=3$ , 则  $p_3$  可能取值的个数是\_\_\_\_\_。

A.  $n-3$                       B.  $n-2$                       C.  $n-1$                       D. 无法确定

5、【2017.02】下列关于栈的叙述中, 错误的是\_\_\_\_\_。

I. 采用非递归方式重写递归程序时必须使用栈

II. 函数调用时, 系统要用栈保存必要的信息

III. 只要确定了入栈次序, 即可确定出栈次序

IV. 栈是一种受限的线性表, 允许在其两端进行操作

A. 仅I                      B. 仅I、II、III                      C. 仅I、III、IV                      D. 仅II、III、IV



6、【2018.02】现有队列 Q 与栈 S，初始时 Q 中的元素依次是 1, 2, 3, 4, 5, 6 (1 在队头)，S 为空。若仅允许下列 3 种操作：1)，出队并输出出队元素；2)，出队并将出队元素入栈；3) 出栈并输出出栈元素，则不能得到的输出序列是

- A. 1, 2, 5, 6, 4, 3                      B. 2, 3, 4, 5, 6, 1  
C. 3, 4, 5, 6, 1, 2                      D. 6, 5, 4, 3, 2, 1

7、【2020.02】对空栈 S 进行 Push 与 Pop 操作，入栈序列 a, b, c, d, e 经过 Push, Push, Pop, Push, Pop, Push, Push, Pop 操作后得到的出栈序列是

- A、b, a, c              B、b, a, e              C、b, c, a              D、b, c, e

8、【2022.02】有限符号集 S, in, out 均为 S 中所有元素的任意排列。对于初始为空的栈 ST，下列叙述正确的是 ( )

- A. 若 in 是 ST 的入栈序列，则不能判断 out 是否为其可能的出栈序列  
B. 若 out 是 ST 的出栈序列，则不能判断 in 是否为其可能的入栈序列  
C. 若 in 是 ST 的入栈序列，out 是对应 in 的出栈序列，则 in 与 out 一定不同  
D. 若 in 是 ST 的入栈序列，out 是对应 in 的出栈序列，则 in 与 out 可能互为倒序

### 【答案解析】

1、C。即出栈顺序是 b,d,c,f,e,a,g，结合入栈顺序 a,b,c,d,e,f,g，栈内元素最多为(a,c,d)，(a,e,f)。元素的出栈顺序是 b,d,c,f,e,a,g，可推出进栈出栈顺序为 Push(S,a)，Push(S,b)，Pop(S,b)，Push(S,c)，Push(S,d)，Pop(S,d)，Pop(S,c)，Push(S,e)，Push(S,f)，Pop(S,f)，Pop(S,e)，Pop(S,a)，Push(S,g)，Pop(S,g)。假设初始所需容量为 0，每做一次 Push 进行一次 "+1" 操作，每做一次 Pop 进行一次 "-1" 操作，记录容量的最大值为 3，所以选 C。

2、D。选项 A 可由 in, in, in, in, out, out, in, out, out, in, out, out 得到；选项 B 可由 in, in, in, out, out, in, out, out, in, out, in, out 得到；选项 C 可由 in, in, out, in, out, out, in, in, out, in, out, out 得到；选项 D 可由 in, out, in, in, in, in, in, out, out, out, out, out 得到，但题意要求不允许连续三次退栈操作，故 D 不可能得到。

3、B。出栈顺序必为 d\_c\_b\_a\_，e 的顺序不定，在任意一个“\_”上都有可能。d 为第 1 个出栈元素，则 d 之前的元素必定是进栈后在栈中停留。因而出栈顺序必为 d\_c\_b\_a\_，e 的顺序不定，在任意“\_”上都有可能，一共有 4 种可能。

4、C。元素 3 之后的 4, 5, ..., n 都是 p3 可取的数 (一直进栈直到该数入栈后马上出栈)。接下来分析 1 和 2: p1 只能是 3 之前入栈的数 (可能是 1 或 2)，当 p1=1 时，p3 可取 2；当 p1=2 时，p3 可取 1，故 p3 可能取除 3 之外的所有数，个数为 n-1。

5、C。采用非递归方式重写递归程序时不一定使用栈；只要确定了入栈次序，不确定出栈次序栈；是一种受限的线性表，不允许在其两端进行操作。I、III、IV 错误。I 的反例：计算斐波拉契数列迭代实现只需要一个循环即可实现。III 的反例：入栈序列为 1、2，进行如下操作 PUSH、PUSH、POP、POP，出栈次序为 2、1；进行如下操作 PUSH、POP、PUSH、POP，出栈次序为 1、2。IV，栈是一种受限的线性表，只允许在一端进行操作。因此 II 正确。

6、C。A 的操作顺序：①①②②①①③③。B 的操作顺序：②①①①①③③。D 的操作顺序：②②②②②①③③③③③③。对于 C：首先输出 3，说明 1 和 2 必须先依次入栈，而此后 2 肯定比 1 先输出，因此无法得到 1, 2 的输出顺序。

7、D。按题意，出入栈操作的过程如下：

| 操作 | 栈内元素 | 出栈元素 |
|----|------|------|
|----|------|------|

|      |     |   |
|------|-----|---|
| Push | a   |   |
| Push | ab  |   |
| Pop  | a   | b |
| Push | ac  |   |
| Pop  | a   | c |
| Push | ad  |   |
| Push | ade |   |
| Pop  | ad  | e |

故出栈序列为 b, c, e。

8、D。对于栈而言, 给定入栈序列, 只要判断给定的出栈序列是否为  $\frac{1}{n+1}C_{2n}^n$  (满足卡特兰数) 中的一个序列, 所以给定输入序列, 无法确定输出序列。

虽然给定输入序列, 无法确定输出序列, 但是我们可以根据入栈序列检查出栈序列是否符合栈先进后出的性质, 所以 A 选项错误。

类似于 A 中的分析, 给定出栈序列, 只要判断给定的入栈序列是否为  $\frac{1}{n+1}C_{2n}^n$  中的一个序列, 所以 B 错误。

关于 C 选项, 如果每个元素入栈后马上出栈, 得到的出栈序列与入栈序列完全相同, 所以 C 错误。

关于 D 选项, 如果序列元素全部入栈后再依次出栈, 得到的出栈序列与入栈序列完全相反, 所以 D 正确。

## 类型七: 栈的应用

1、【2012.02】已知操作符包括 '+', '-', '\*', '/', '(', 和 ')'. 将中缀表达式  $a+b-a*((c+d)/e-f)+g$  转换为等价的后缀表达式  $ab+acd+e/f-*-g+$  时, 用栈来存放暂时还不能确定运算次序的操作符, 若栈初始时空, 则转换过程中同时保存在栈中的操作符的最大个数是\_\_\_\_\_。

A. 5                      B. 7                      C. 8                      D. 11

2、【2014.02】假设栈初始为空, 将中缀表达式  $a/b+(c*d-e*f)/g$  转换为等价的后缀表达式的过程中, 当扫描到 f 时, 栈中的元素依次是\_\_\_\_\_。

A. + (\* -                      B. + (- \*                      C. / + (\* - \*                      D. / + - \*

3、【2015.01】已知程序如下:

```
int S(int n){
 return (n<=0)? 0 : s(n-1)+n;
}
void main(){
 cout<< S(1);
}
```

程序运行时使用栈来保存调用过程的信息, 自栈底到栈顶保存的信息依次对应的是\_\_\_\_\_。

A. main()→S(1)→S(0)                      B. S(0)→S(1)→main()  
C. main()→S(0)→S(1)                      D. S(1)→S(0)→main()

4、【2018.01】若栈 S1 中保存整数, 栈 S2 中保存运算符, 函数 F() 依次执行下述各步操作:

- (1) 从 S1 中依次弹出两个操作数 a 和 b;
- (2) 从 S2 中弹出一个运算符 op;
- (3) 执行相应的运算  $b \text{ op } a$ ;
- (4) 将运算结果压入 S1 中。

假定 S1 中的操作数依次是 5, 8, 3, 2 (2 在栈顶), S2 中的运算符依次是 \*, -, + (+ 在栈顶)。调用 3 次 F() 后, S1 栈顶保存的值是

A. -15

B. 15

C. -20

D. 20

5、【2024.02】与表达式  $x+y*(z-u)/v$  等价的后缀表达式是

A.  $xyzv-*/v/+$

B.  $xyzv-v/*+$

C.  $+x/*y-zuv$

D.  $+x*y/-zuv$

6、【2025.02】已知算法 A 用于检查字符串中各类括号是否匹配，A 执行过程中使用初始为空的栈保存遇到的括号，若栈的容量是 3，则下列选项中，A 不能处理的事

A.  $(a+[b+(c+d)/e+f])+g-h$

B.  $[a*((b+c)/(d-e)+f/g)-h]$

C.  $\sqrt{a}*(b-(c-d)*e/(f+g)-h)$

D.  $[a-(b+[c*(d+e)-f]+g+h)]$

### 【答案解析】

1、A。根据运算符的优先级入栈出栈即可。表达式求值是栈的典型应用。中缀表达式不仅依赖于运算符的优先级，而且要处理括号。后缀表达式的运算符在表达式的后面且没有括号，其形式已经包含了运算符的优先级。所以从中缀表达式转换到后缀表达式需要用运算符进行处理，使其包含运算符优先级的信息，从而转换为后缀表达式的形式。

2、B。根据运算符的优先级入栈出栈即可。将中缀表达式转换为后缀表达式的算法思想如下：从左向右开始扫描中缀表达式；遇到数字时，加入后缀表达式；遇到运算符时：a. 若为 ' ( '，入栈；b. 若为 ' ) '，则依次把栈中的运算符加入后缀表达式中，直到出现 ' ( '，从栈中删除 ' ( '；c. 若为除括号外的其他运算符，当其优先级高于除 ' ( ' 以外的栈顶运算符时，直接入栈；否则从栈顶开始，依次弹出比当前处理的运算符优先级高和优先级相等的运算符，直到一个比它优先级低的或者遇到了一个左括号为止。

在此，再给出中缀表达式转换为前缀或后缀表达式的手工做法，以上面给出的中缀表达式为例：第一步：按照运算符的优先级对所有的运算单位加括号。式子变成  $((a/b)/(((c*d)-(e*f))/g))$ 。第二步：转换为前缀或后缀表达式。前缀：把运算符移动到对应的括号前面，则变成  $+/(ab)/(-((cd)*(ef)g))$ 。把括号去掉： $+ab/-*cd*efg$  前缀式子出现。后缀：把运算符移动到对应的括号后面，则变成  $((ab)/(((cd)*(ef)*-g)/+)$ 。把括号去掉： $ab/cd*ef*-g/+$  后缀式子出现。当题目要求直接求前缀或后缀表达式时，这种方法会比上一种快捷得多。

3、A。递归调用函数时，在系统栈里保存的函数信息需满足先进后出的特点，依次调用了  $main()$ ， $S(1)$ ， $S(0)$ ，故栈底到栈顶的信息依次是  $main()$ ， $S(1)$ ， $S(0)$ 。

4、B。第一次调用：(1)从  $S1$  中弹出 2 和 3；(2)从  $S2$  中弹出 +；(3)执行  $3+2=5$ ；(4)将 5 压入  $S1$  中，第一次调用结束后  $S1$  中剩余 5,8,5 (5 在栈顶)， $S2$  中剩余 \*，- (- 在栈顶)。第二次调用：(1)从  $S1$  中弹出 5 和 8；(2)从  $S2$  中弹出 -；(3)执行  $8-5=3$ ；(4)将 3 压入  $S1$  中，第二次调用结束后  $S1$  中剩余 5,3 (3 在栈顶)， $S2$  中剩余 \*。第三次调用：(1)从  $S1$  弹出 3 和 5；(2)从  $S2$  中弹出 \*；(3)执行  $5*3=15$ ；(4)将 15 压入  $S1$  中，第三次调用结束后  $S1$  中仅剩余 15 (栈顶)， $S2$  为空。故选 B。

5、A。方法一：

将中缀表达式  $x+y*(z-u)/v$  转换为后缀表达式的过程如下：

①扫描整个中缀表达式。

②如果遇到操作数 (这里是  $x, y, z, u, v$ )，则直接输出 (添加到后缀表达式中)。

③如果遇到运算符 (这里是  $+, -, *, /$ )，则查看操作符栈顶的元素，如果栈顶运算符的优先级高于或等于当前运算符，则弹出栈顶运算符并输出，然后将当前运算符压入栈中。如果栈顶运算符的优先级低于当前运算符，或者栈为空，则直接将当前运算符压入栈中。

④如果遇到左括号 (这里是“(”)，则直接压入栈中。

⑤如果遇到右括号 (这里是“)”)，则依次弹出并输出栈顶的运算符，直到遇到左括号为止，然

后再弹出左括号（但不输出）。

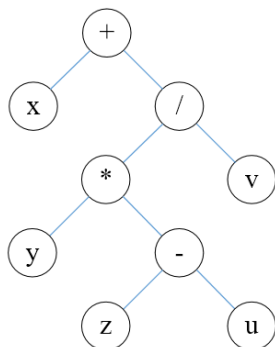
⑥重复步骤②-⑤，直到中缀表达式的所有字符都被扫描完。

⑦如果操作符栈中还有元素，则依次弹出并输出。

对于给定的表达式  $x+y*(z-u)/v$ ，其等价的后缀表达式按照运算的优先级顺序（括号，乘法和除法，加法和减法）来确定。所以，我们首先得到  $z$  和  $u$  的差  $zu$ ，然后将其与  $y$  相乘得到  $zyu*$ ，然后除以  $v$  得到  $zyu*/v$ ，最后加上  $x$  得到  $xyzu*/v+$ 。

方法二：

中缀表达式对应二叉树的中序遍历，根据中序遍历构建二叉树：



后缀表达式对应的是二叉树的后序遍历，根据后序遍历的结果得到  $xyzu*/v+$

6、D。对各表达式逐一检查括号的最大嵌套层数及匹配顺序。算法 A 的栈容量为 3，故当嵌套层数超过 3 时，就会发生栈溢出，无法继续处理。

在选项 D 中， $[ \rightarrow$  栈深度 1， $( \rightarrow$  栈深度 2， $[ \rightarrow$  栈深度 3， $( \rightarrow$  栈深度 4（此时超过栈的最大容量 3）。因此，一旦遇到第 4 层左括号，栈已无法继续压入，算法 A 不能处理该表达式。其余选项中，最大嵌套层数均不超过 3 或存在不同程度的匹配问题，但并未超出栈容量。

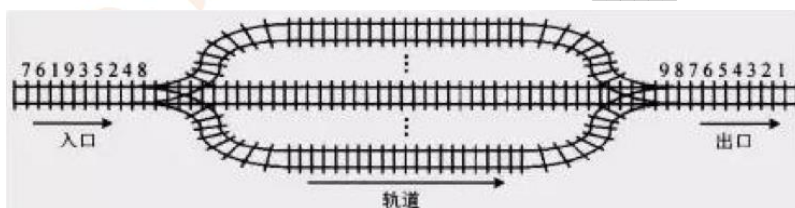
括号匹配通常使用栈来实现：遇到左括号则入栈，遇到右括号则出栈。若在扫描过程中某时刻需要入栈的左括号多于栈的最大容量，或出现不正确的匹配顺序，则算法无法处理。

## 类型八：队列的性质和应用

1、【2009.01】为解决计算机主机与打印机之间速度不匹配问题，通常设置一个打印数据缓冲区，主机将要输出的数据依次写入该缓冲区，而打印机则依次从该缓冲区中取出数据。该缓冲区的逻辑结构应该是\_\_\_\_\_。

A. 栈                      B. 队列                      C. 树                      D. 图

2、【2016.03】设有如下图所示的火车车轨，入口到出口之间有  $n$  条轨道，列车的行进方向均为从左至右，列车可驶入任意一条轨道。现有编号为 1~9 的 9 列列车，驶入的次序依次是 8, 4, 2, 5, 3, 9, 1, 6, 7。若期望驶出的次序依次为 1~9，则  $n$  至少是\_\_\_\_\_。



A. 2                      B. 3                      C. 4                      D. 5

3、【2010.02】某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素  $a$ 、 $b$ 、



c、d、e 依次入此队列后再进行出队操作, 则不可能得到的出队序列是\_\_\_\_\_。

- A. bacde      B. dbace      C. dbcae      D. ecbad

4、【2021.02】已知初始为空的队列 Q 的一端仅能进行入队操作, 另外一端既能进行入队操作又能进行出队操作。若 Q 的入队序列是 1, 2, 3, 4, 5, 则不能得到的出队序列是

- A. 5, 4, 3, 1, 2      B. 5, 3, 1, 2, 4  
C. 4, 2, 1, 3, 5      D. 4, 1, 3, 2, 5

5、【2019.42】设计一个队列, 要求满足: ①初始时队列为空; ②入队时, 允许增加队列占用空间; ③出队后, 出队元素所占用的空间可重复使用, 即整个队列所占用的空间只增不减; ④入队操作和出队操作的时间复杂度始终保持为  $O(1)$ 。请回答下列问题:

- (1) 该队列应该选择链式存储结构, 还是顺序存储结构?
- (2) 画出队列的初始状态, 并给出判断队空和队满的条件
- (3) 画出第一个元素入队后的队列状态。
- (4) 给出入队操作和出队操作的基本过程。

### 【答案解析】

1、B。缓冲区的概念出现在操作系统的设备管理中, 其特点是先进先出。缓冲区的作用是解决主机与打印机之间速度不匹配的问题, 而不应改变打印数据的顺序。若用栈, 先进入缓冲区的数据则要排队到最后才能打印, 显然不符题意, 故选 B。

2、C。在确保队列先进先出原则的前提下。根据题意具体分析: 入队顺序为 8、4、2、5、3、9、16、7, 出队顺序为 1-9。入口和出口之间有多个队列( $n$  条轨道), 且每个队列(轨道)可容纳多个元素(多列列车)。如此分析: 显然先入队的元素必须小于后入队的元素(否则, 如果 8 和 4 入同一队列, 8 在前 4 在后, 那么出队时只能是 8 在前 4 在后), 这样 8 入队列 1, 4 入队列 2, 2 入队列 3, 5 入队列 2(按照前面的原则“大的元素在小的元素后面”)也可以将 5 入队列 3, 但这时剩下的元素 3 就必须放到一个新的队列里面, 无法确保“至少”, 本应该是将 5 入队列 2, 再将 3 入队列 3, 不增加新队列的情况下, 可以满足题意“至少”的要求)。3 入队列 3, 9 入队列 1, 这时共占了 3 个队列, 后面还有元素 1, 直接再占用一个新的队列 4, 1 从队列 4 出队后, 剩下的元素 6 和 7 或者入队到队列 2 或者入队到队列 3(为简单起见我们不妨设  $n$  个队列的序号分别为 1、2、...、 $n$ ), 这样就可以满足题目的要求。综上, 共占用了 4 个队列。

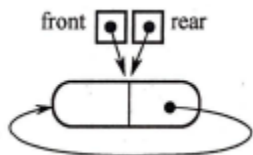
3、C。本题的队列实际上是一个输出受限的双端队列。A 操作: a 左(或右入)、b 左入、c 右入、d 右入、e 右入。B 操作: a 左入(或右入)、b 左入、c 右入、d 左入、e 右入。D 操作: a 左入(或右入)、b 左入、c 左入、d 右入、e 左入。C 操作: a 左入(或右入)、b 右入、因 d 未出, 此时只能进队, c 怎么进都不可能在 b 和 a 之间。

4、D。题目中输出受限的双端队列, A 选项, 1 左入右入都可, 2 右入, 3 左入, 4 左入, 5 左入, 可得到 5, 4, 3, 1, 2 的出队顺序。B 选项, 1 左入右入都可, 2 右入, 3 左入, 4 右入, 5 左入, 得到 5, 3, 1, 2, 4 的出队顺序。C 选项, 1 左入右入都可, 2 左入, 3 右入, 4 左入, 5 右入, 得到 4, 2, 1, 3, 5 的出队顺序。若 4 是第一个出队的元素, 那么至少 1, 2, 3, 4 已全部入队, 则队列 2 应该与 1 相邻, 出队时顺序也应该相邻, 所以选项 D 错误。

5、(1) 顺序存储无法满足要求的队列占用空间随着入队操作而增加。根据要求来分析: 要求①容易满足; 链式存储方便开辟新空间, 要求②容易满足; 对于要求③, 出队后的结点并不真正释放, 用队头指针指向新的队头结点, 新结点入队时, 有空闲结点则无须开辟新空间, 赋值到队尾后的第一个空闲结点即可, 然后用队尾指针指向新的队尾结点, 这就需要设计成一个首尾相接的循

环单链表, 类似于循环队列的思想。设置队头、队尾指针后, 链式队列的入队操作和出队操作的时间复杂度均为  $O(1)$ , 要求④可以满足。因此, 采用链式存储结构 (两段式单向循环链表), 队头指针为  $front$ , 队尾指针为  $rear$ 。

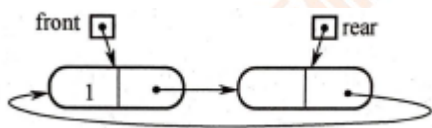
(2) 该循环链式队列的实现, 可以参考循环队列, 不同之处在于循环链式队列可以方便增加空间, 出队的结点可以循环利用, 入队时空间不够也可以动态增加。同样, 循环链式队列也要区分队满和队空的情况, 这里参考循环队列牺牲一个单元来判断。初始时, 创建只有一个空闲结点的循环单链表, 头指针  $front$  和尾指针  $rear$  均指向空闲结点, 如下图所示。



队空的判定条件:  $front = rear$ 。

队满的判定条件:  $front = rear \rightarrow next$ 。

(3) 插入第一个结点后的状态如下图所示。



(4) 操作的基本过程如下:

入队操作:

若  $(front == rear \rightarrow next)$  //队满  
则在  $rear$  后面插入一个新的空闲结点;  
入队元素保存到  $rear$  所指结点中;  
 $rear = rear \rightarrow next$ ;  
返回。

出队操作:

若  $(front == rear)$  //队空  
则出队失败, 返回;  
取  $front$  所指结点中的元素  $e$ ;  
 $front = front \rightarrow next$ ;  
返回  $e$ 。

## 类型九: 循环队列的概念和性质

1、【2011.03】已知循环队列存储在一维数组  $A[0 \dots n-1]$  中, 且队列非空时  $front$  和  $rear$  分别指向队头元素和队尾元素。若初始时队列为空, 且要求第 1 个进入队列的元素存储在  $A[0]$  处, 则初始时  $front$  和  $rear$  的值分别是\_\_\_\_\_。

- A. 0, 0                      B. 0,  $n-1$                       C.  $n-1$ , 0                      D.  $n-1$ ,  $n-1$

2、【2014.03】循环队列放在一维数组  $A[0 \dots M-1]$  中,  $end1$  指向队头元素,  $end2$  指向队尾元素的下一个位置。假设队列两端均可进行入队和出队操作, 队列中最多能容纳  $M-1$  个元素。初始 时为 空。下列判断队空和队满的条件中, 正确的是\_\_\_\_\_。

- A. 队空:  $end1 == end2$ ; 队满:  $end1 == (end2+1) \bmod M$   
B. 队空:  $end1 == end2$ ; 队满:  $end2 == (end1+1) \bmod (M-1)$   
C. 队空:  $end2 == (end1+1) \bmod M$ ; 队满:  $end1 == (end2+1) \bmod M$   
D. 队空:  $end1 == (end2+1) \bmod M$ ; 队满:  $end2 == (end1+1) \bmod (M-1)$

### 【答案解析】

1、B。根据题意, 第一个元素进入队列后存储在  $A[0]$  处, 此时  $front$  和  $rear$  值都为 0。入队时由于要执行  $(rear+1)\%n$  操作, 所以如果入队后指针指向 0, 则  $rear$  初值为  $n-1$ , 而由于第一个元素在  $A[0]$  中, 插入操作只改变  $rear$  指针, 所以  $front$  为 0 不变。注意: ①循环队列是指顺序存储的队列, 而不是指逻辑上的循环, 如循环单链表表示的队列不能称为循环队列。  $front$  和  $rear$  的初值并不是固定的。

2、A。  $end1$  指向队头元素, 可知出队的操作是先将  $A[end1]$  读数, 然后  $end1$  再加 1。  $end2$  指向队尾元素的后一个位置, 可知入队操作是先存数到  $A[end2]$ , 然后  $end2$  再加 1。若把  $A[0]$  储存第一个而  $end1$  指向的是队头元素, 队头元素在数组  $A$  中的下标为 0, 所以得知  $end1$  初值也为 0, 可知队空条件为  $end1=end2$ ; 然后考虑队列满时, 因为队列最多能容纳  $M-1$  个元素, 假设队列存储在下标为 0 到下标为  $M-2$  的  $M-1$  个区域, 队头为  $A[0]$ , 队尾为  $A[M-2]$ , 此时队列满, 考虑在这种情况下  $end1$  和  $end2$  的状态,  $end1$  指向队头元素, 可知  $end1=0$ ,  $end2$  指向队尾元素的后一个位置, 可知  $end2=M-2+1=M-1$ , 所以可知队满的条件为  $[end1-(end2+1)]\%M$ , 选 A。

## 类型十: 树和二叉树的概念与性质

1、【2009.05】已知一棵完全二叉树的第 6 层 (设根为第 1 层) 有 8 个叶结点, 则该完全二叉树的结点个数最多是\_\_\_\_\_。

- A. 39                      B. 52                      C. 111                      D. 119

2、【2010.05】在一棵度为 4 的树  $T$  中, 若有 20 个度为 4 的结点, 10 个度为 3 的结点, 1 个度为 2 的结点, 10 个度为 1 的结点, 则树  $T$  的叶结点个数是\_\_\_\_\_。

- A. 41                      B. 82                      C. 113                      D. 122

3、【2011.04】若一棵完全二叉树有 768 个结点, 则该二叉树中叶结点的个数是\_\_\_\_\_。

- A. 257                      B. 258                      C. 384                      D. 385

4、【2018.04】设一棵非空完全二叉树  $T$  的所有叶结点均位于同一层, 且每个非叶结点都有 2 个子结点。若  $T$  有  $k$  个叶结点, 则  $T$  的结点总数是

- A.  $2k-1$                       B.  $2k$                       C.  $k^2$                       D.  $2^k-1$

5、【2020.03】对于任意一棵高度为 5 且有 10 个结点的二叉树, 若采用顺序存储结构保存, 每个结点占 1 个存储单元 (仅存放结点的数据信息), 则存放该二叉树需要的存储单元数量至少是:

- A. 31                      B. 16                      C. 15                      D. 10

6、【2022.04】若三叉树  $T$  中有 244 个结点 (叶结点高度为 1), 则  $T$  的高度至少是 ( )

- A. 8                      B. 7                      C. 6                      D. 5

7、【2025.03】若二叉树的结点值均为正数, 采用顺序存储方式保存在数组  $R$  中, 用 -1 表示结点不存在, 则下列数组中, 不表示一棵二叉树的是:

- A.  $R[] = \{20, 15, 40, -1, -1, 35\}$                       B.  $R[] = \{15, 40, 10, 18, 35, -1, -1\}$   
C.  $R[] = \{15, 40, 10, -1, -1, -1, 23\}$                       D.  $R[] = \{17, 20, 35, -1, 18, 45, -1, -1, 19, 27\}$

8、【2016.42】如果一棵非空  $k(k \geq 2)$  叉树  $T$  中每个非叶结点都有  $k$  个孩子, 则称  $T$  为正则后  $k$  树。请回答下列问题并给出推导过程。

(1) 若  $T$  有  $m$  个非叶结点, 则  $T$  中的叶结点有多少个?

(2) 若  $T$  的高度为  $h$  (单结点的树  $h=1$ ), 则  $T$  的结点数最多为多少个? 最少为多少个?

【答案解析】

1、C。完全二叉树比满二叉树只是在最下面一层的右边缺少了部分叶结点，而最后一层之上是个满二叉树，并且只有最后两层有叶结点。第6层有叶结点则完全二叉树的高度可能为6或7，显然树高为7时结点更多。若第6层上有8个叶结点，则前六层为满二叉树，而第7层缺失了 $8 \times 2 = 16$ 个叶结点，故完全二叉树的结点个数最多为 $(2^7 - 1) - 16 = 111$ 个结点。

2、B。设树中度为 $i$  ( $i=0,1,2,3,4$ ) 的结点数分别为 $N_i$ ; 树中结点总数为 $N$ ，则树中各结点的度之和等于 $N-1$ ，即 $N = 1 + N_1 + 2 \times N_2 + 3 \times N_3 + 4 \times N_4 = N_0 + N_1 + N_2 + N_3 + N_4$ ，根据题设中的数据，即可得到 $N_0=82$ ，即树T的叶结点的个数是82。

3、C。 $N_0 + N_1 + N_2 = N_1 + 2 \times N_2 + 1 = 768$ ，且 $N_0 = N_2 + 1$ ，完全二叉树中，度为1的结点只有1个或者没有。因为本题中结点总数为偶数，所以有一个度为1的结点。解得 $N_2=383$ ， $N_0=384$ 。

4、A。非叶结点的度均为2，且所有叶结点都位于同一层的完全二叉树就是满二叉树。对于一棵高度为 $h$ 的满二叉树（空树 $h=0$ ），其最后一层全部是叶结点，数量为 $2^{h-1}$ ；总结点数为 $2^h - 1$ 。因此当 $2^{h-1}=k$ 时，可以得到 $2^h - 1 = 2k - 1$ 。

5、A。二叉树采用顺序存储时，用数组下标来表示结点之间的父子关系。对于一棵高度为5的二叉树，为了满足任意性，其1~5层的所有结点都要被存储起来，即考虑为一棵高度为5的满二叉树，总共需要存储单元的数量为 $1+2+1+2+4+8+16=31$ 。

6、C。给定三叉树，要让高度最少，构成高度为 $h$ 的满三叉树，根据等比数列求和公式，得到 $n = \frac{1 \times (1 - 3^h)}{1 - 3} = \frac{3^h - 1}{2}$ ，带入 $n=244$ ， $h=6$ 。

7、D。选项A、B、C通过下标逐层判断，均可形成合理的二叉树结构：

选项A中最大下标为5，对应父结点为2。

选项B、C中不存在“父结点是-1，却有孩子结点”的情况。

选项D中， $R[3]=-1$ ，表示下标3处无结点，但按顺序存储， $R[8]$ 和 $R[9]$ 应分别是 $R[3]$ 的右孩子和 $R[4]$ 的左孩子，结果却在下标8、9处出现了有效结点19、2。这样就出现了“父结点为空却有孩子”的矛盾，因此无法表示一棵二叉树。故正确答案为D。

在用数组顺序存储二叉树时，若一个结点位于下标 $i$ ，则其左、右孩子分别位于下标 $2i+1$ 和 $2i+2$ 处；用-1表示空结点。当某个父结点不存在却出现对应的孩子结点时，便无法构成有效的二叉树。

8、(1) 根据定义，正则 $k$ 叉树中仅含有两类结点：叶结点(个数记为 $n_0$ )和度为 $k$ 的分支结点(个数记为 $n_k$ )。树T中的结点总数 $n = n_0 + n_k = n_0 + m$ 。树中所含的边数 $e = n - 1$ ，这些边均为 $m$ 个度为 $k$ 的结点发出的，即 $e = m \times k$ 。整理得： $n_0 + m = m \times k + 1$ ，故 $n_0 = (k-1) \times m + 1$ 。

(2) 高度为 $h$ 的正则 $k$ 叉树T中，含最多结点的树形为：除第 $h$ 层外，第1到第 $h-1$ 层的结点都是度为 $k$ 的分支结点，而第 $h$ 层均为叶结点，即树是“满”树。此时第 $j$  ( $1 \leq j \leq h$ ) 层结点数为 $k^{j-1}$ ，结点总数 $M_1$ 为：

$$M_1 = \sum_{j=1}^h k^{j-1} = \frac{k^h - 1}{k - 1}$$

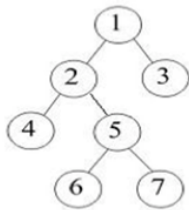
含最少结点的正则 $k$ 叉树的树形为：第1层只有根结点，第2到第 $h-1$ 层仅含1个分支结点和 $k-1$ 个叶结点，第 $h$ 层有 $k$ 个叶结点。即除根外第2到第 $h$ 层中每层的结点数均为 $k$ ，故T中所含结点总数 $M_2$ 为：

$$M_2 = 1 + (h-1) \times k$$



## 类型十一：二叉树的遍历

1、【2009.03】给定二叉树如右图所示。设  $N$  代表二叉树的根， $L$  代表根结点的左子树， $R$  代表根结点的右子树。若遍历后的结点序列是 3, 1, 7, 5, 6, 2, 4, 则其遍历方式是\_\_\_\_\_。



- A. LRN                      B. NRL                      C. RLN                      D. RNL

2、【2011.05】若一棵二叉树的前序遍历序列和后序遍历序列分别为 1, 2, 3, 4 和 4, 3, 2, 1, 则该二叉树的中序遍历序列不会是\_\_\_\_\_。

- A. 1, 2, 3, 4              B. 2, 3, 4, 1              C. 3, 2, 4, 1              D. 4, 3, 2, 1

3、【2012.03】若一棵二叉树的前序遍历序列为 a, e, b, d, c, 后序遍历序列为 b, c, d, e, a, 则根结点的孩子结点\_\_\_\_\_。

- A. 只有 e                      B. 有 e、b                      C. 有 e、c                      D. 无法确定

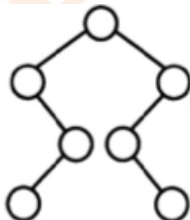
4、【2015.02】先序序列为 a, b, c, d 的不同二叉树的个数是\_\_\_\_\_。

- A. 13                      B. 14                      C. 15                      D. 16

5、【2017.04】要使一棵非空二叉树的先序序列与中序序列相同, 其所有非叶结点须满足的条件是\_\_\_\_\_。

- A. 只有左子树              B. 只有右子树              C. 结点的度均为 1              D. 结点的度均为 2

6、【2017.05】已知一棵二叉树的树形如下图所示, 其后序序列为 e, a, c, b, d, g, f, 树中与结点 a 同层的结点是\_\_\_\_\_。



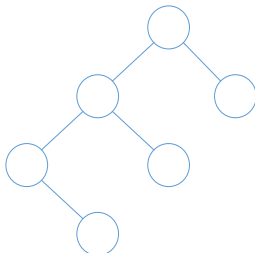
- A. c                      B. d                      C. f                      D. g

7、【2022.03】若结点  $p$  与  $q$  在二叉树  $T$  中的中序遍历序列中相邻, 且  $p$  在  $q$  之前, 则下列  $p$  与  $q$  的关系中, 不可能是 ( )

I.  $q$  是  $p$  的双亲; II.  $q$  是  $p$  的右孩子; III.  $q$  是  $p$  的右兄弟; IV.  $q$  是  $p$  的双亲的双亲

- A. 仅 I                      B. 仅 III                      C. 仅 II、III                      D. 仅 II、IV

8、【2023.05】已知一棵二叉树的树型如下图所示, 若其后序遍历序列为 f, d, b, e, c, a, 则其先 (前) 序遍历序列是 ( )



A. a, e, d, f, b, c

B. a, c, e, b, d, f

C. c, a, b, e, f, d

D. d, f, e, b, a, c

9、【2024.03】p、q 和 v 均为二叉树 T 中的结点，v 有两个孩子结点，T 的中序遍历列形如：“...，p，v，q，...”，则下列叙述中，正确的是

A. p 没有右孩子，q 没有左孩子

B. p 没有右孩子，q 有左孩子

C. p 有右孩子，q 没有左孩子

D. p 有右孩子，q 有左孩子

### 【答案解析】

1、D。分析遍历后的结点序列，可以看出根结点是在中间访问，而右子树结点在左子树之前，即遍历的方式是 RNL。本题考查的遍历方法并不是二叉树的 3 种基本遍历方法，对于考生而言，重要的是要掌握遍历的思想。

2、C。前序序列为 LRN，后序序列为 NLR，前序序列和后序序列刚好相反，所以不可能存在一个节点同时存在左右孩子。1 为根结点，且只有一个孩子，所以中序序列中，1 在序列首或者序列尾。其子树同理，可知 C 错误。

3、A。前序序列和后序序列不能唯一确定一棵二叉树，但可以确定二叉树中结点的祖先关系：当两个结点的前序序列为 XY 与后序序列为 YX 时，则 X 为 Y 的祖先。考虑前序序列~.e, b, d, c、后序序列 b, c, d, e，且，可知 a 为根结点，e 为 a 的孩子结点。此外，a 的孩子结点的前序序列~.b, d, c、后序序列 b, c, d, ~，可知 e 是 bed 的祖先，故根结点的孩子结点只有 e。故选 A。

4、B。根据二叉树前序遍历和中序遍历的递归算法中递归工作栈的状态变化得出：前序序列和中序序列的关系相当于以前序序列为入栈次序，以中序序列为出栈次序。因为前序序列和中序序列可以唯一地确定一棵二叉树，所以题意相当于“以序列 abcd 为入栈次序，则出栈序列的个数为?”，对于 n 个不同元素进栈，出栈序列的个数为  $\frac{1}{n+1}C_{2n}^n = 14$

5、B。先序序列是先父结点，接着左子树，然后右子树。中序序列是先左子树，接着父结点，然后右子树，递归进行。如果所有非叶结点只有右子树，先序序列和中序序列都是先父结点，然后右子树，递归进行，因此 B 正确。

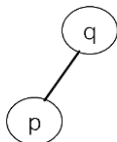
6、B。后序序列是先左子树，接着右子树，最后父结点，递归进行。根结点左子树的叶结点首先被访问，它是 e。接下来是它的父结点 a，然后是 a 的父结点 c。接着访问根结点的右子树。它的叶结点 b 首先被访问，然后是 b 的父结点 d，再者是 d 的父结点 g。最后是根结点 f。因此 d 与 a 同层，B 正确。

7、B。中序遍历的方式是左根右，且结点 p 与 q 在二叉树 T 中的中序遍历序列中相邻，且 p 在 q 之前，那么在中序遍历中，p 是 q 的前驱，q 是 p 的后继。

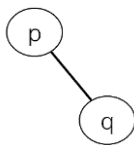
p 和 q 只存在父子和祖孙关系，不存在兄弟关系，因为如果 p 和 q 是兄弟节点，那么 p 和 q 不会相邻，他们之间的结点是 q 和 q 的父亲或者左孩子结点。

我们对其他几个可能做理论分析，并给出例子。

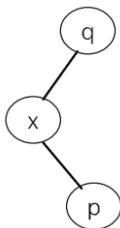
1. q 是 p 的双亲，此时 p 是 q 的左子树，且 p 是 q 的左子树的最右结点，p 没有右孩子时，I 的表述正确，如图所示。



II.  $q$  是  $p$  的双亲, 此时  $p$  是  $q$  的右子树, 且  $p$  是  $q$  的右子树的最右结点, II 的表述正确, 如图所示,



IV.  $q$  是  $p$  的双亲的双亲, 此时  $q$  是  $p$  的左孩子  $x$  (假设为  $x$ ) 的孩子结点, 且  $q$  是  $x$  的最右的孩子结点, 如图所示,



8. A. 先将此树从上至下, 从左至右依次编号 1, 2, 3, 4, 5, 6 如图 3 所示, 进行后序遍历, 然后与题目中结点对应可以得到每个结点的编号。将图 3 中树后序遍历得到序列 645231, 对应 f,d,b,e,c,a 得图 4。图 4 进行先序遍历得到 aedfbc, 因此选 A。

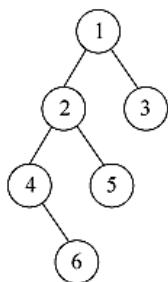


图 3

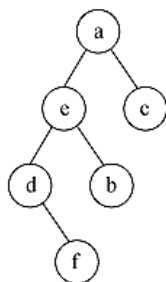


图 4

9. A. 方法一: 在二叉树的中序遍历中, 结点的顺序是“左孩子—父结点—右孩子”。所以对于给定的中序遍历... $p$ ,  $v$ ,  $q$ , ..., 我们可以得出以下结论:

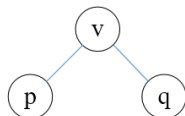
$P$  是  $V$  的左孩子, 因为  $P$  在  $V$  之前。

$Q$  是  $V$  的右孩子, 因为  $Q$  在  $V$  之后。

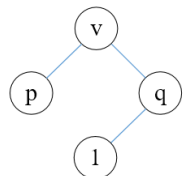
因此,  $P$  没有右孩子,  $Q$  没有左孩子。所以, 正确答案是选项 A。这是因为在中序遍历中, 如果一个结点后面紧接着另一个结点, 那么第一个结点就没有右孩子, 第二个结点就没有左孩子。

方法二: 构造特殊的二叉树:

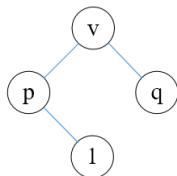
A.  $P$  没有右孩子,  $Q$  没有左孩子, 遍历结果为  $P, V, Q$ 。



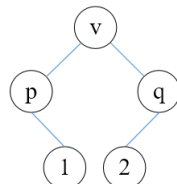
B.  $P$  没有右孩子,  $Q$  有左孩子, 遍历结果为  $P, V, 1, Q$ , 所以错误。



C.  $P$  有右孩子,  $Q$  没有左孩子, 遍历结果为  $P, 1, V, Q$ , 所以错误。

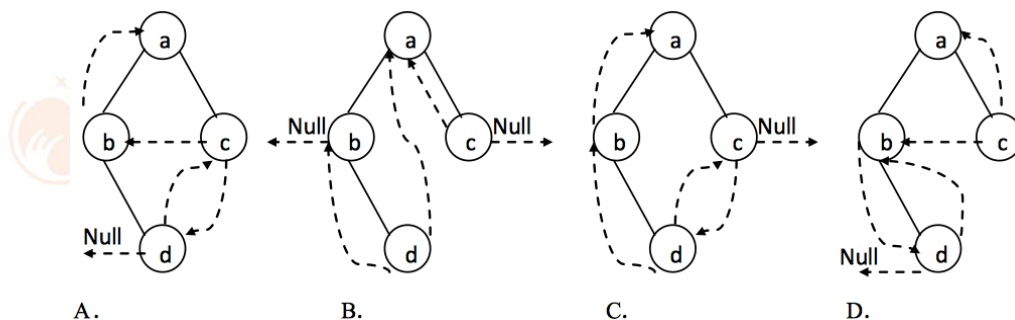


D. P 有右孩子, Q 有左孩子, 遍历结果为 P, 1, V, 2, Q, 所以错误。



## 类型十二：线索二叉树的构造和性质

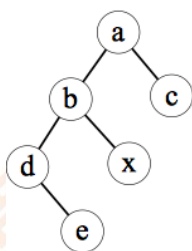
1、【2010.03】下列线索二叉树中（用虚线表示线索），符合后序线索树定义的是\_\_\_\_\_。



2、【2013.05】若 X 是后序线索二叉树中的叶结点，且 X 存在左兄弟结点 Y，则 X 的右线索指向的是\_\_\_\_\_。

- A. X 的父结点  
B. 以 Y 为根的子树的最左下结点  
C. X 的左兄弟结点 Y  
D. 以 Y 为根的子树的最右下结点

3、【2014.04】若对如下的二叉树进行中序线索化，则结点 x 的左、右线索指向的结点分别是\_\_\_\_\_。



- A. e、c  
B. e、a  
C. d、c  
D. b、a

### 【答案解析】

1、D。题中所给二叉树的后序序列为 d, b, c, a。结点 d 无前驱和左子树，左链域空，无右子树，右链域指向其后继结点 b；结点 b 无左子树，左链域指向其前驱结点 d；结点 c 无左子树，左链域指向其前驱结点 b，无右子树，右链域指向其后继结点 a。故选 D。

2、A。根据后续线索二叉树的定义，X 结点为叶子结点且有左兄弟，那么这个结点为右孩子点，利用后续遍历的方式可知 X 结点的后继是其父结点，即其右线索指向的是父结点。

3、D。线索二叉树的线索实际上指向的是相应遍历序列特定结点的前驱结点和后继结点，所以



先写出二叉树的中序遍历序列: debxac, 中序遍历中在 x 左边和右边的字符, 就是它在中序线索化的左、右线索, 即 b、a, 选 D。

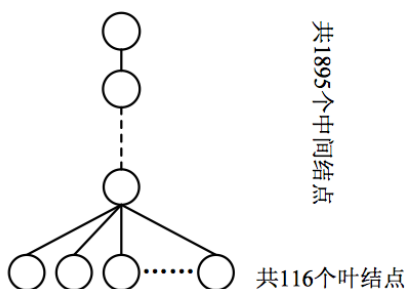
### 类型十三: 树、二叉树和森林的转换

- 1、【2009.06】将森林转换为对应的二叉树, 若在二叉树中, 结点 u 是结点 v 的父结点的父结点, 则在原来的森林中, u 和 v 可能具有的关系是\_\_\_\_\_。  
I. 父子关系 II. 兄弟关系 III. u 的父结点与 v 的父结点是兄弟关系  
A. 只有II B. I和II C. I和III D. I、II和III
- 2、【2011.06】已知一棵有 2011 个结点的树, 其叶结点个数为 116, 该树对应的二叉树中无右孩子的结点个数是\_\_\_\_\_。  
A. 115 B. 116 C. 1895 D. 1896
- 3、【2014.05】将森林 F 转换为对应的二叉树 T, F 中叶结点的个数等于\_\_\_\_\_。  
A. T 中叶结点的个数 B. T 中度为 1 的结点个数  
C. T 中左孩子指针为空的结点个数 D. T 中右孩子指针为空的结点个数
- 4、【2016.05】若森林 F 有 15 条边、25 个结点, 则 F 包含树的个数是\_\_\_\_\_。  
A. 8 B. 9 C. 10 D. 11
- 5、【2019.02】若将一棵树 T 转化为对应的二叉树 BT, 则下列对 BT 的遍历中, 其遍历序列与 T 的后根遍历序列相同的是  
A. 先序遍历 B. 中序遍历 C. 后序遍历 D. 按层遍历
- 6、【2020.04】已知森林 F 及与之对应的二叉树 T, 若 F 的先根遍历序列是 a, b, c, d, e, f, 中根遍历序列是 b, a, d, f, e, c 则 T 的后根遍历序列是:  
A、b, a, d, f, e, c B、b, d, f, e, c, a C、b, f, e, d, c, a D、f, e, d, c, b, a
- 7、【2021.04】某森林 F 对应的二叉树为 T, 若 T 的先序遍历序列是 a,b,d,c,e,g,f, 中序遍历序列是 b,d,a,e,g,e,f, 则 F 中树的棵数是  
A.1 B.2 C.3 D.4
- 8、【2025.04】下列关于二叉树及森林的叙述中, 正确的是:  
A. 完全二叉树中不存在度为 1 的结点 B. 任意一个森林都可以转换为一棵二叉树  
C. 二叉树的分支结点个数比叶结点个数少 D. 表达式树的根中保存的是最先计算的运算符 9

#### 【答案解析】

1、B。森林与二叉树的转换规则为“左孩子右兄弟”。在最后生成的二叉树中, 父子关系在对应森林关系中可能是兄弟关系或原本就是父子关系。情形 I: 若结点 v 是结点 u 的第二个孩子结点, 在转换时, 结点 v 就变成结点 u 第一个孩子的右孩子, 符合要求。情形 II: 结点 u 和 v 是兄弟结点的关系, 但二者之中还有一个兄弟结点 k, 则转换后, 结点 v 就变为结点 k 的右孩子, 而结点 k 则是结点 u 的右孩子, 符合要求。情形 III: 若结点 u 的父结点与 v 的父结点是兄弟关系, 则转换后, 结点 u 和 v 分别在两者最左父结点的两棵子树中, 不可能出现在同一条路径中。

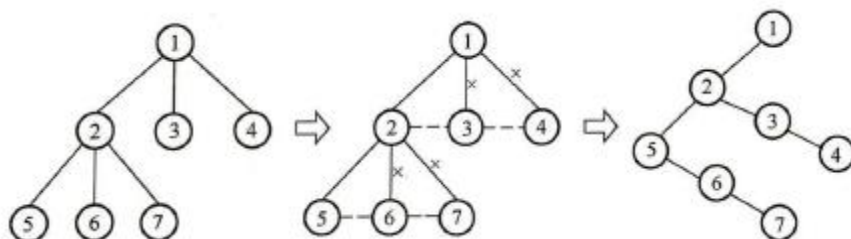
2、D。本题可采用特殊情况法解。设题意中的树是如下图所示的结构, 则对应的二叉树中仅有前 115 个叶结点有右孩子。



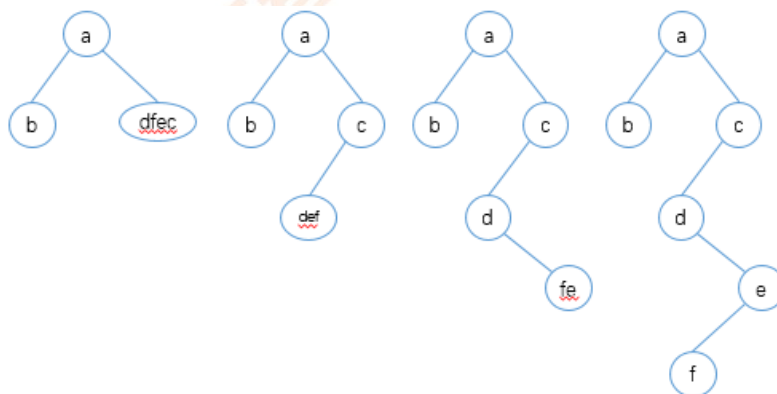
3、C。树转换成二叉树的规则是：左孩右兄。树的叶子结点没有孩子，所以是二叉树左孩子指针为空的结点。将森林转化为二叉树即相当于用孩子兄弟表示法表示森林。在变化过程中，原森林某结点的第一个孩子结点作为它的左子树，它的兄弟作为它的右子树。那么森林中的叶结点由于没有孩子结点，那么转化为二叉树时，该结点就没有左结点，所以 F 中叶结点的个数就等于 T 中左孩子指针为空的结点个数，选 C。此题还可以通过一些特例来排除 A、B、D 选项。

4、C。解法一：树有一个很重要的性质：在  $n$  个结点的树中有  $n-1$  条边，“那么对于每棵树，其结点数比边数多 1”。题中的森林中的结点数比边数多 10(即  $25-15=10$ )，显然共有 10 棵树。解法二：若考生再仔细分析可发现，此题也是考察图的某些方面的性质：生成树和生成森林。此时对于图的生成树有一个重要的性质：若图中顶点数为  $n$ ，则它的生成树含有  $n-1$  条边。对比解法一中树的性质，不难发现两种解法都利用到了“树中结点数比边数多 1”的性质，接下来的分析如解法一。

5、B。后根遍历树可分为两步：(1)从左到右访问双亲结点的每个孩子（转化为二叉树后就是先访问根结点再访问右子树）；(2)访问完所有孩子后再访问它们的双亲结点（转化为二叉树后就是先访问左子树再访问根结点），因此树 T 的后根遍历序列与其相应二叉树 BT 的中序遍历序列相同。对于此类题，采用特殊值法求解通常会更便捷，左下图树 T 转换为二叉树 BT 的过程如下图所示，树 T 的后序遍历序列显然和其相应二叉树 BT 的中序遍历序列相同，均为 5, 6, 7, 2, 3, 4, 1。因此选 B。

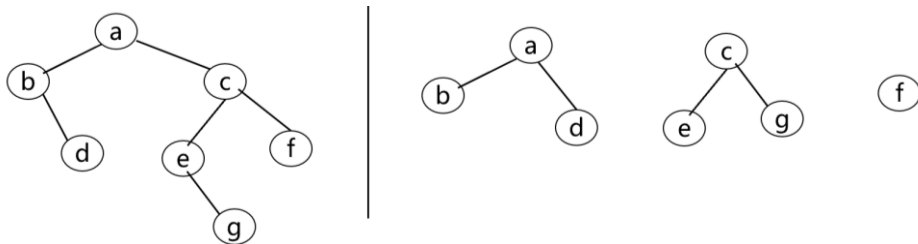


6、C。森林 F 的先根遍历序列对应其二叉树 T 的先序遍历序列，森林 F 的中根遍历序列对应其二叉树 T 的中序遍历序列。即 T 的先序遍历序列为 a, b, c, d, e, f，中序遍历序列为 b, a, d, f, e, c。根据二叉树 T 的先序序列和中序序列可以唯一确定它的结构，构造过程如下图：



可以得到二叉树 T 的后序序列为 b, f, e, d, c, a。

7、C。根据 T 的中序和先序遍历，可以构造出 T，如图所示，



根据二叉树和树的转换，有三个棵树。

5、B。完全二叉树的性质要求除了最后一层外，其余各层均被填满，但在最后一层可能只存在左子结点，从而出现度为 1 的结点。A 错误。

通过左孩子—右兄弟表示法，任意森林中的每棵树都可以看作二叉树的一部分，再将它们串联起来，即可转换成一棵二叉树。B 正确。

二叉树中分支结点（非叶结点）的个数与叶结点之间的关系取决于树的结构，不能一概认为“分支结点个数一定少于叶结点个数”。例如，在一个只有一个子结点的链式结构中，分支结点反而较多。

C 错误。

在表达式树（链式树）中，根结点一般保存的是整个表达式的主运算符，即最后计算的运算符，而非最先计算的运算符，D 错误。

## 类型十四：哈夫曼树的构造和性质

1、【2010.06】对  $n$  ( $n \geq 2$ ) 个权值均不相同的字符构造哈夫曼树。下列关于该哈夫曼树的叙述中，错误的是\_\_\_\_\_。

- A. 该树一定是一棵完全二叉树。
- B. 树中一定没有度为 1 的结点。
- C. 树中两个权值最小的结点一定是兄弟结点。
- D. 树中任一非叶结点的权值一定不小于下一层任一结点的权值。

2、【2013.04】已知三叉树 T 中 6 个叶结点的权分别是 2, 3, 4, 5, 6, 7, T 的带权（外部）路径长度最小是\_\_\_\_\_。

- A. 27
- B. 46
- C. 54
- D. 56

3、【2015.03】下列选项给出的是从根分别到达两个叶结点路径上的权值序列，能属于同一棵哈夫曼树的\_\_\_\_\_。

- A. 24, 10, 5 和 24, 10, 7
- B. 24, 10, 5 和 24, 12, 7
- C. 24, 10, 10 和 24, 14, 11
- D. 24, 10, 5 和 24, 14, 6

4、【2021.05】若某二叉树有 5 个叶结点，其权值分别为 10, 12, 16, 21, 30，则其最小的带权路径长度(WPL)是 ( )

- A. 89
- B. 200
- C. 208
- D. 289

5、【2023.04】在由 6 个字符组成的字符集 S 中，各字符出现的频次分别为 3, 4, 5, 6, 8, 10，为 S 构造的哈夫曼编码的加权平均长度为 ( )

- A. 2.4
- B. 2.5
- C. 2.67
- D. 2.75

### 【答案解析】

1、A。哈夫曼树为带权路径长度最小的二叉树，不一定是完全二叉树。哈夫曼树中没有度为 1

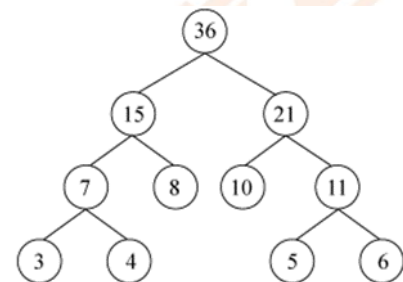
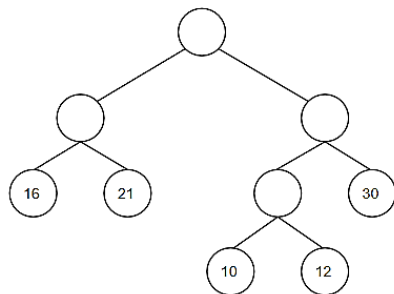
的结点, B 正确; 构造哈夫曼树时, 最先选取两个权值最小的结点作为左、右子树构造一棵新的二叉树, C 正确; 哈夫曼树中任意非叶结点 P 的权值为其左、右子树根结点权值之和, 其权值不小于其左、右子树根结点的权值, 在与结点 P 的左、右子树根结点处于同一层的结点中, 若存在权值大于结点 P 权值的结点 Q, 则结点 Q 的兄弟结点中权值较小的一个应该与结点 P 作为左、右子树构造新的二叉树。综上可知, 哈夫曼树中任意非叶结点的权值一定不小于下一层任意结点的权值。

2、B。将哈夫曼树的思想推广到三叉树的情形。为了构成严格的三叉树, 需添加权为 0 的虚叶结点, 对于严格的三叉树  $(n0-1)\%(3-1)=u=1\neq 0$ , 需要添加  $m-u-1=3-1-1$  个叶结点, 说明 7 个叶结点刚好可以构成一个严格的三叉树。利用三叉树的 6 个叶子结点的权构建最小带权生成树, 最小的带权路径长度为  $(2+3)*3+(4+5)*2+(6+7)*1=46$ 。

3、D。在哈夫曼树中, 左右孩子权值之和为父结点权值。仅以分析选项 A 为例: 若两个 10 分别属于两棵不同的子树, 根的权值不等于其孩子的权值和, 不符; 若两个 10 属同棵子树, 其权值不等于其两个孩子(叶结点)的权值和, 不符。B、C 选项的排除方法一样。

4、B。要求最小的 WPL, 即考察哈夫曼树的构造, 构造结果如右图所示, 因此  $MPL=2\times(16+21+30)+3\times(10+12)=200$ 。

5、B。先构造哈夫曼树如右图所示。哈夫曼树的平均编码长度是指使用哈夫曼编码时, 哈夫曼编码的总码长, 除以出现信息总字节数。因此, 哈夫曼编码的加权平均长度为  $(3*3+3*4+2*8+2*10+3*5+3*6)/(3+4+8+10+5+6)=2.5$ , 因此选 B。



## 类型十五：哈夫曼编码

1、【2014.06】5 个字符有如下 4 种编码方案, 不是前缀编码的是\_\_\_\_\_。

- A. 01, 0000, 0001, 001, 1      B. 011, 000, 001, 010, 1  
C. 000, 001, 010, 011, 100      D. 0, 100, 110, 1110, 1100

2、【2017.06】已知字符集 {a, b, c, d, e, f, g, h}, 若各字符的哈夫曼编码依次是 0100, 10, 0000, 0101, 001, 011, 11, 0001, 则编码序列 0100011001001011110101 的译码结果是\_\_\_\_\_。

- A. a c g a b f h      B. a d b a g b b      C. a f b e a g d      D. a f e e f g d

3、【2018.05】已知字符集 {a,b,c,d,e,f}, 若字符出现的次数分别为 6, 3, 8, 2, 10, 4, 则对应字符集中各字符的哈夫曼编码可能是

- A. 00, 1011, 01, 1010, 11, 100      B. 00, 100, 110, 000, 0010, 01  
C. 10, 1011, 11, 0011, 00, 010      D. 0011, 10, 11, 0010, 01, 000

4、【2019.03】对 n 个互不相同的符号进行哈夫曼编码。若生成的哈夫曼树共有 115 个结点, 则 n 的值是



A. 56

B. 57

C. 58

D. 60

5、【2022.05】对任意给定的含  $n$  ( $n > 2$ ) 个字符的有限集  $S$ , 用二叉树表示  $S$  的哈夫曼编码集和定长编码集, 分别得到二叉树  $T_1, T_2$ 。下列叙述中, 正确的是 ( )

A.  $T_1$  与  $T_2$  的结点数相同

B.  $T_1$  的高度大于  $T_2$  的高度

C. 出现频次不同的字符在  $T_1$  中处于不同层

D. 出现频次不同的字符在  $T_2$  中处于相同的层

6、【2025.05】设字符集  $S$  包含 7 个字符, 各字符出现的频次分别为 2, 3, 4, 6, 8, 10, 11。现为  $S$  中的各字符构造哈夫曼编码, 编码长度不小于 3 的字符个数是 ( )

A. 2

B. 3

C. 4

D. 5

7、【2012.41】设有 6 个有序表 A、B、C、D、E、F, 分别含有 10、35、40、50、60 和 200 个数据元素, 各表中元素按升序排列。要求通过 5 次两两合并, 将 6 个表最终合并成 1 个升序表, 并在最坏情况下比较的总次数达到最小。请问答下列问题。

(1) 给出完整的合并过程, 并求出最坏情况下比较的总次数。

(2) 根据你的合并过程, 描述  $n$  ( $n \geq 2$ ) 个不等长升序表的合并策略, 并说明理由。

8、【2020.42】若任一个字符的编码都不是其他字符编码的前缀, 则称这种编码具有前缀特性。现有某字符集 (字符个数  $\geq 2$ ) 的不等长编码, 每个字符的编码均为二进制的 0, 1 序列, 最长为  $L$  位, 且具有前缀特性。请回答下列问题:

(1) 哪种数据结构适宜保存上述具有前缀特性的不等长编码?

(2) 基于你所设计的数据结构, 简述从 0/1 串到字符串的译码过程

(3) 简述判定某字符集的不等长编码是否具有前缀特性的过程

### 【答案解析】

1、D。110 是 1100 的前缀。前缀编码的定义是在一个字符集中, 任何一个字符的编码都不是另一个字符编码的前缀。选项 D 中编码 110 是编码 1100 的前缀, 违反了前缀编码的规则, 所以 D 不是前缀编码。

2、D。哈夫曼编码是前缀编码, 各个编码的前缀各不相同, 因此直接拿编码序列与哈夫曼编码一一比对即可。序列可分割为 0100 011 001 001 011 11 0101, 译码结果是 afecfgd, 选项 D 正确。

3、A。构造一棵符合题意的哈夫曼树, 最后的根节点权重为 33, 令左子树为 0, 右子树为 1, 便可以得到答案 A。

4、C。 $n$  个符号构造哈夫曼树的过程中, 共新建了  $n-1$  个结点 (双分支结点), 因此哈夫曼树的结点总数为  $2n-1=115$ ,  $n$  的值为 58, 答案选 C。

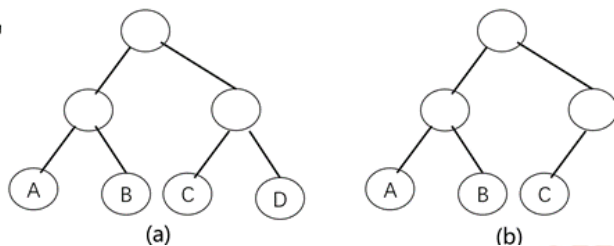
5、D。我们知道, 采用哈夫曼编码, 可以得到不等长且符合前缀特性的编码集。题目中给定了等长编码是一种简单且译码具有唯一性的编码方式, 这种编码方式的特点是每个字符的编码长度相同。题目中就是要分析哈夫曼编码和等长编码的特性。设哈夫曼编码构造的二叉树是  $T_1$ , 设等长编码的二叉树是  $T_2$ ,

根据哈夫曼树的性质, 假设叶子个数是  $n$  个, 那么度为 2 的结点个数是  $(n-1)$ , 总结点数是  $2n-1$  (哈夫曼树中没有度为 1 的结点), 但是等长编码中可能存在度为 1 的结点, 如图(b)和(c)。根据图(a)和(c),  $T_1$  的高度等于  $T_2$  的高度。根据图(a), 我们假设 A,B,C,D 四个点的权重 (频次) 相同, 那么他们处于同一个层次。

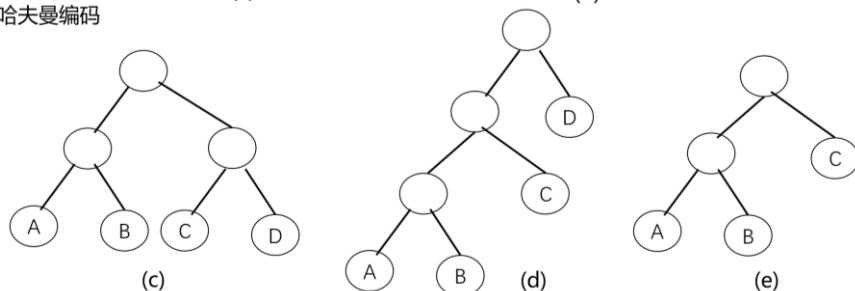
哈夫曼树考虑了词频, 需要用到前缀编码, 按照词频从大到小排序, 一般词频越高的编码长度

越短, 所以 D 正确。

等长编码



哈夫曼编码



6、D。将 6 个字符的频次按升序排列: 2,3,4,6, 8,10, 11。

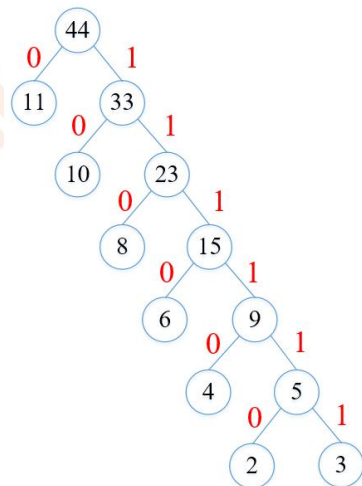
构造哈夫曼树步骤如下:

- ①合并最小的 2 和 3, 得到结点 55。
- ②合并 4 和 5, 得到结点 9。
- ③合并 6 和 9, 得到结点 15。
- ④合并 8 和 15, 得到结点 23。
- ⑤合并 10 和 23, 得到结点 33。
- ⑥合并 11 和 33, 得到根结点 44。

由树结构可得各叶结点的编码长度(深度):

结点 8 的深度为 3, 结点 6 的深度为 4, 结点 4 的深度为 5, 结点 2 的深度为 6, 结点 3 的深度为 6。

故编码长度不小于 3 的字符有 5 个, 选 D。



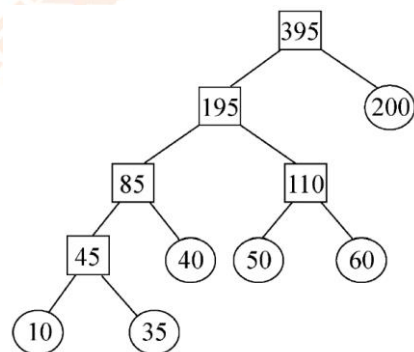
7、(1) 对于长度分别为  $m$ ,  $n$  的两个有序表的合并过程, 最坏情况下需要一直比较到两个表尾元素, 比较次数为  $m+n-1$  次。已知需要 5 次两两合并, 故可设总比较次数为  $X-5$ ,  $X$  就是以  $N$  个叶子结点表示升序表, 以升序表的表长表示结点权重, 构造的二叉树的带权路径长度。故只需设计方案使得  $X$  最小。这样受哈夫曼树和最佳归并树思想的启发, 设计哈夫曼树如右图:

这样, 最坏情况下比较的总次数为:

$$N = (10 + 35) \times 4 + (40 + 50 + 60) \times 3 + 200 - 5 = 825$$

(2)  $N$  ( $N \geq 2$ ) 个不等长升序表的合并策略: 以  $N$  个叶子结点表示升序表, 以升序表的表长表示结点权重, 构造哈夫曼树。合并时, 从深度最大的结点所代表的升序表开始合并, 依深度次序一直进行到根结点。

理由:  $N$  个有序表合并需要进行  $N-1$  次两两合并, 可设最坏情况下的比较总次数为  $X-N+1$ ,  $X$  就是以  $N$  个叶子结点表示升序表, 以升序表的表长表示结点权重, 构造的二叉树的带权路径长度。根据哈夫曼树的特点, 上述设计的比较次数是最小的。



8、(1) 使用一棵二叉树表示字符集中各字符的编码, 每个编码对应于从根开始到达某叶结点的一条路径, 路径长度等于编码位数, 路径到达叶结点中保存该编码对应的字符。

(2) 从左至右依次扫描 0/1 串中的各位。从根开始, 根据串中当前位沿当前结点的左子指针或右子指针下移, 直到移动到叶结点为止。输出叶结点中保存的字符, 然后从根开始重复这个过程, 直到扫描到 0/1 串结束, 译码完成。

(3) 二叉树既可用于保存各字符集的编码, 又可用于检测编码是否具有前缀特性。判定编码是否具有前缀特性的过程, 也是构建二叉树的过程。初始时, 二叉树中仅含有根节点, 其左子指针和右子指针均为空。

依次读入每个编码 C, 建立/寻找从根开始对应于该编码的一条路径, 过程如下:

对每个编码, 从左至右扫描 C 的各位, 根据 C 的当前位 (0 或 1) 沿结点的指针 (左子指针或右子指针) 向下移动, 当遇到空指针时, 创建新结点, 让空指针指向该新结点并继续移动, 沿指针移动的过程中, 可能遇到三种情况:

- 若遇到了叶结点 (非根), 则表明不具有前缀特性, 返回。
- 若在处理 C 的所有位的过程中, 均没有创建新结点, 则表明不具有前缀特性, 返回。
- 若在处理 C 的最后一个编码位时创建了新结点, 则继续验证下一个编码。

若所有编码均通过验证, 则编码具有前缀特性。

## 类型十六: 二叉树算法应用

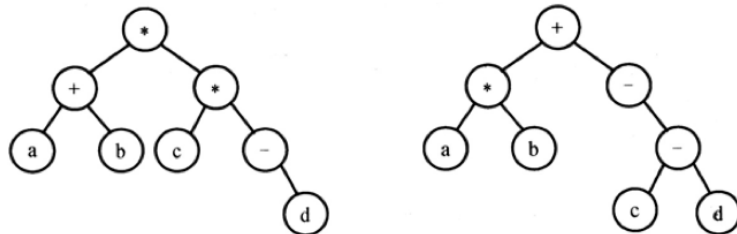
1、【2014.41】二叉树的带权路径长度(WPL)是二叉树中所有叶结点的带权路径长度之和。给定一棵二叉树 T, 采用二叉链表存储, 结点结构为:

| left | weight | right |
|------|--------|-------|
|------|--------|-------|

其中叶结点的 weight 域保存该结点的非负权值。设 root 为指向 T 的根结点的指针, 请设计求 T 的 WPL 的算法, 要求:

- (1) 给出算法的基本设计思想;
- (2) 使用 C 或 C++ 语言, 给出二叉树结点的数据类型定义;
- (3) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。

2、【2017.41】请设计一个算法, 将给定的表达式树 (二叉树) 转换为等价的中缀表达式 (通过括号反映操作符的计算次序) 并输出。例如, 当下列两棵表达式树作为算法的输入时:



输出的等价中缀表达式分别为  $(a+b)*(c*(-d))$  和  $(a*b)+(-(c-d))$ 。

二叉树结点定义如下:

```
typedef struct node
{
 char data[10]; //存储操作数或操作符
 struct node*left,*right;
}BTree;
```

要求:

- (1) 给出算法的基本设计思想。
- (2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。

### 【答案解析】

1、考查二叉树的带权路径长度, 二叉树的带权路径长度为每个叶子结点的深度与权值之积的总和, 可以使用先序遍历或层次遍历解决问题。

(1) 算法的基本设计思想:

基于先序递归遍历的算法思想是用一个 static 变量记录 wpl, 把每个结点的深度作为递归函数的一个参数传递, 算法步骤如下: 若该结点是叶子结点, 那么变量 wpl 加上该结点的深度与权值之积; 若该结点非叶子结点, 那么若左子树不为空, 对左子树调用递归算法, 若右子树不为空, 对右子树调用递归算法, 深度参数均为本结点的深度参数加一; 最后返回计算出的 wpl 即可。

(2) 二叉树结点的数据类型定义如下:

```
typedef struct BiTNode{
 int weight;
 struct BiTNode *lchild,*rchild;
}BiTNode,*BiTree;
```

(3) 算法代码如下:

基于先序遍历的算法:

```
int WPL(BiTree root){
 return wpl_PreOrder(root, 0);
}
int wpl_PreOrder(BiTree root, int deep){
 static int wpl = 0; // 定义一个 static 变量存储 wpl
 if(root->lchild == NULL && root->rchild == NULL) // 若为叶子结点, 累积 wpl
 wpl += deep*root->weight;
 if(root->lchild != NULL) // 若左子树不空, 对左子树递归遍历
 wpl_PreOrder(root->lchild, deep+1);
 if(root->rchild != NULL) // 若右子树不空, 对右子树递归遍历
 wpl_PreOrder(root->rchild, deep+1);
 return wpl;
}
```

2、(1) 算法的基本设计思想

表达式树的中序序列加上必要的括号即为等价的中缀表达式。可以基于二叉树的中序遍历策略得到所需的表达式。

表达式树中分支结点所对应的子表达式的计算次序, 由该分支结点所处的位置决定。为得到正确的中缀表达式, 需要在生成遍历序列的同时, 在适当位置增加必要的括号。显然, 表达式的最外层 (对应根结点) 及操作数 (对应叶结点) 不需要添加括号。

(2) 算法实现

```
void BtreeToE(BTree *root){
 BtreeToExp(root, 1); // 根的高度为 1
}
void BtreeToExp(BTree *root, int deep){
 if(root == NULL) return;
 else if(root->left == NULL && root->right == NULL) // 若为叶结点
 printf("%s", root->data); // 输出操作数
```



```

else{
 if(deep>1)
 printf("("); //若有子表达式则加 1 层括号
 BtreeToExp(root->left,deep+1);
 printf("%s",root->data); //输出操作符
 BtreeToExp(root->right,deep+1);
 if(deep>1)
 printf(")"); //若有子表达式则加 1 层括号
 }
}

```

## 类型十七：图的性质和概念

- 【2009.07】下列关于无向连通图特性的叙述中，正确的是\_\_\_\_\_。  
 I. 所有顶点的度之和为偶数      II. 边数大于顶点个数减 1      III. 至少有一个顶点的度为 1  
 A. 只有 I      B. 只有 II      C. I 和 II      D. I 和 III
- 【2010.07】若无向图  $G=(V,E)$  中含有 7 个顶点，要保证图  $G$  在任何情况下都是连通的，则需要的边数最少是\_\_\_\_\_。  
 A. 6      B. 15      C. 16      D. 21
- 【2011.08】下列关于图的叙述中，正确的是\_\_\_\_\_。  
 I. 回路是简单路径      II. 存储稀疏图，用邻接矩阵比邻接表更省空间  
 III. 若有向图中存在拓扑序列，则该图不存在回路  
 A. 仅 II      B. 仅 I、II      C. 仅 III      D. 仅 I、III
- 【2017.07】已知无向图  $G$  含有 16 条边，其中度为 4 的顶点个数为 3，度为 3 的顶点个数为 4，其他顶点的度均小于 3。图  $G$  所含的顶点个数至少是\_\_\_\_\_。  
 A. 10      B. 11      C. 13      D. 15
- 【2019.06】用有向无环图描述表达式  $(x+y)*((x+y)/x)$ ，需要的顶点个数至少是  
 A. 5      B. 6      C. 8      D. 9
- 【2022.06】对于无向图  $G=(V,E)$ ，下列选项中，正确的是 ( )  
 A. 当  $|V|>|E|$  时， $G$  一定是连通的      B. 当  $|V|<|E|$  时， $G$  一定是连通的  
 C. 当  $|V|=|E|-1$  时， $G$  一定是不连通的      D. 当  $|V|>|E|+1$  时， $G$  一定是不连通的
- 【2025.06】下列关于图的叙述中，正确的是 ( )  
 A. 有向图必存在入度为 0 的顶点。  
 B. 有向无环图的拓扑有序序列存在且唯一。  
 C. 各顶点的度均大于等于 2 的无向图必有回路。  
 D. 可用 BFS 算法求出带权图中每一对顶点的最短路径

### 【答案解析】

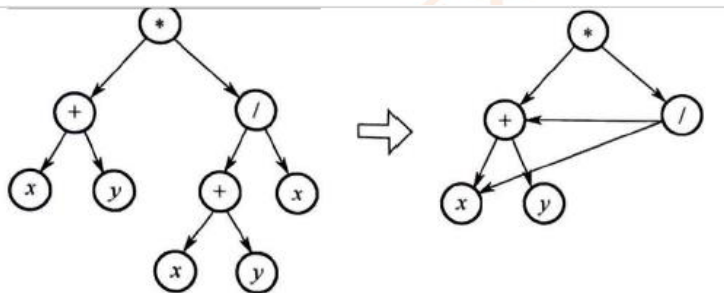
1、A 【解析】每条边都连接了两个结点，在计算顶点的度之和时每条边都被计算了两次（出度和入度），故所有顶点的度之和为边数的两倍，I 正确。 $n$  个顶点、 $n-1$  条边可以构成无向连通图，比如树，II 错误。顶点数为  $N(N \geq 1)$  的无向完全图中不存在度为 1 的顶点，III 错误。

2、C 【解析】要保证无向图  $G$  在任何情况下都是连通的，即任意变动图  $G$  中的边， $G$  始终保持连通，首先需要  $G$  的任意 6 个结点构成完全连通子图  $G'$ ，需  $n(n-1)/2=6*(6-1)/2=15$  条边，然后再添一条边将第 7 个结点与  $G'$  连接起来，共需 16 条边。

3、C 【解析】I.回路对应于路径，简单回路对应于简单路径;II. 刚好相反;III. 拓扑有序的必要条件。故选 C。

4、B 【解析】 $N_0+N_1+N_2+N_3+N_4=16+1$ ,  $4*N_4+3*N_3+x*N_2+y*N_1+1=N_0+N_1+N_2+N_3+N_4$ ,  
 $N_4=3$ ,  $N_3=4$ , 代入, 顶点个数至少为 11。无向图边数的两倍等于各顶点度数的总和。由于其他顶点的度均小于 3, 可以设它们的度都为 2, 设它们的数量是 X, 可列出方程  $4*3+3*4+2*x=16*2$ , 解得  $x=3$ 。  $4+3+3=11$ , B 正确

5、A 【解析】先将该表达式转换成有向二叉树, 注意到该二叉树中有些顶点是重复的, 为了节省存储空间, 可以去除重复的顶点(使顶点个数达到最少), 将有向二叉树去重转换成有向无环图, 如下图所示。答案选 A。



6、D 【解析】根据图的性质, 给定完全无向图,  $E=\frac{V*(V-1)}{2}$ , 此时是图中边数的最大值。而当  $E=V-1$  时, 是图能够连通的最小边数, 但是却无法保证图在任何情况下都连通, 所以 A 和 B, C 错误, 二者无法保证连通或者不连通。当图是生成树时, 满足  $E=V-1$ , 也就是  $V=E+1$ , 所以, 当  $|V|>|E|+1$  时, G 一定是不连通的, D 选项正确。

7、C。考虑一个有向环图, 每个顶点的入度均为 1, 故不存在入度为 0 的顶点。选项 A 错误。虽然有向无环图一定存在拓扑序列, 但通常不唯一, 因为可能存在多个入度为 0 的顶点可供选择。选项 B 错误。

对于无向图, 如果每个顶点的度都至少为 2, 那么不可能构成树(树中必定存在度为 1 的叶结点), 因此必含有回路。选项 C 正确。

BFS(广度优先搜索)算法只能求无权图或所有边权相同图的最短路径, 带权图需采用其他算法(如 Dijkstra 算法)。选项 D 错误。

## 类型十八: 图的存储

1、【2013.07】设图的邻接矩阵 A 如下所示。各顶点的度依次是\_\_\_\_\_。

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

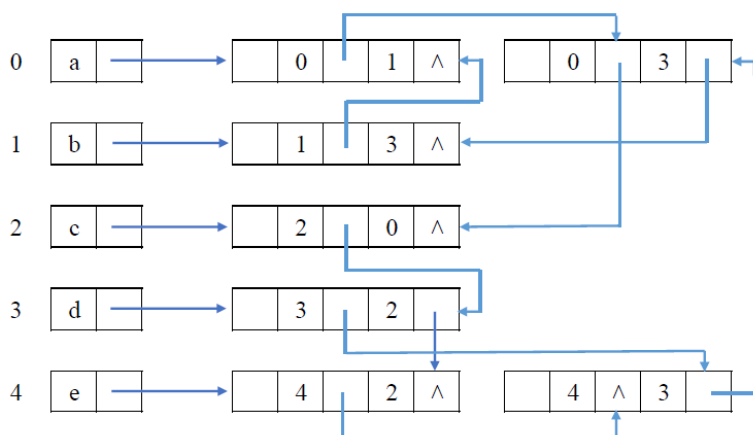
A. 1, 2, 1, 2

B. 2, 2, 1, 1

C. 3, 4, 2, 3

D. 4, 4, 2, 2

2、【2024.04】无向图  $G=(V, E)$  的邻接多重表如下图所示, 则 G 中顶点 b 与 d 的度分别是



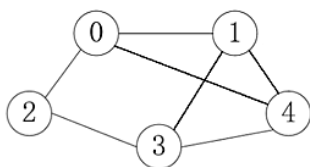
A.0, 2

B.2, 4

C.2, 5

D.3, 4

3、【2015.42】已知含有 5 个顶点的图 G 如下图所示。



请回答下列问题：

(1) 写出图 G 的邻接矩阵 A (行、列下标从 0 开始)。

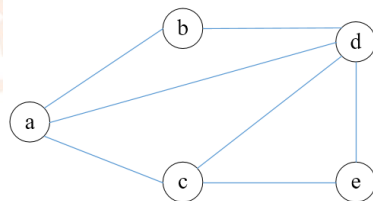
(2) 求  $A^2$ ，矩阵  $A^2$  中位于 0 行 3 列元素值的含义是什么？

(3) 若已知具有  $n$  ( $n \geq 2$ ) 个顶点的图的邻接矩阵为 B，则  $B^m$  ( $2 \leq m \leq n$ ) 中非零元素的含义是什么？

### 【答案解析】

1、C。邻接矩阵 A 为非对称矩阵，说明图是有向图，度为入度加出度之和。各顶点的度是矩阵中 此结点对应的行 (对应出度) 和列 (对应入度) 的非零元素之和。

2、B。邻接多重表不同于邻接链表，邻接链表是两个顶点确定一条边，邻接多重表是一个顶点定一条边，根据图，可以确定边的集合  $(0,1)$ ， $(0,3)$ ， $(1,3)$ ， $(2,0)$ ， $(3,2)$ ， $(4,2)$ ， $(4,3)$ ，根据边的集合和图示，还原无向图，则从右图中可以看出，b 的度为 2，d 的度为 4。



3、(1) 图 G 的邻接矩阵 A 如下：

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 0 \end{bmatrix}$$

(2)  $A^2$  如下：

$$A^2 = \begin{bmatrix} 3 & 1 & 0 & 3 & 1 \\ 1 & 3 & 2 & 1 & 2 \\ 0 & 2 & 0 & 0 & 2 \\ 3 & 1 & 0 & 3 & 1 \\ 1 & 2 & 0 & 1 & 3 \end{bmatrix}$$

0 行 3 列的元素值 3 表示从顶点 0 到顶点 3 之间长度为 2 的路径共有 3 条。

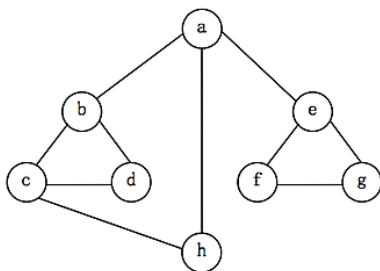
(3)  $B^m$  ( $2 \leq m \leq n$ ) 中位于  $i$  行  $j$  列 ( $0 \leq i, j \leq n-1$ ) 的非零元素的含义是: 图中从顶点  $i$  到顶点  $j$  长度为  $m$  的路径条数。

## 类型十九: 图的遍历

1、【2012.05】对有  $n$  个结点、 $e$  条边且使用邻接表存储的有向图进行广度优先遍历, 其算法时间复杂度是\_\_\_\_\_。

- A.  $O(n)$                       B.  $O(e)$                       C.  $O(n+e)$                       D.  $O(n \cdot e)$

2、【2013.08】若对如下无向图进行遍历, 则下列选项中, 不是广度优先遍历序列的是\_\_\_\_\_。

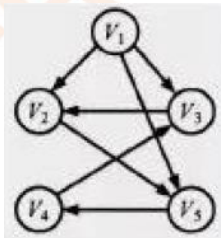


- A. h, c, a, b, d, e, g, f                      B. e, a, f, g, b, h, c, d  
C. d, b, c, a, h, e, f, g                      D. a, b, c, d, h, e, f, g

3、【2015.05】设有向图  $G=(V, E)$ , 顶点集  $V=\{V_0, V_1, V_2, V_3\}$ , 边集  $E=\{<v_0, v_1>, <v_0, v_2>, <v_0, v_3>, <v_1, v_3>\}$ 。若从顶点  $V_0$  开始对图进行深度优先遍历, 则可能得到的不同遍历序列个数是\_\_\_\_\_。

- A. 2                      B. 3                      C. 4                      D. 5

4、【2016.06】下列选项中, 不是下图深度优先搜索序列的是\_\_\_\_\_。



- A.  $V_1, V_5, V_4, V_3, V_2$                       B.  $V_1, V_3, V_2, V_5, V_4$   
C.  $V_1, V_2, V_5, V_4, V_3$                       D.  $V_1, V_2, V_3, V_4, V_5$

5、【2020.06】修改递归方式实现的图的深度优先搜索 (DFS) 算法, 将输出 (访问) 定点信息的语句移到退出递归前 (即执行输出语句后立刻退出递归)。采用修改后的算法遍历有向无环图  $G$ , 若输出结果中包含  $G$  中的全部顶点, 则输出的顶点序列是  $G$  的:

- A. 拓扑有序序列                      B. 逆拓扑有序序列  
C. 广度优先搜索序列                      D. 深度优先搜索序列

### 【答案解析】

1、C。广度优先遍历需要借助队列实现。邻接表的结构包括: 顶点表; 边表 (有向图为出边表)。当采用邻接表存储方式时, 在对图进行广度优先遍历时每个顶点均需入队一次 (顶点表遍历), 故时间复杂度为  $O(n)$ , 在搜索所有顶点的邻接点的过程中, 每条边至少访问一次 (出边表遍历), 故时间复杂度为  $O(e)$ , 算法总的时间复杂度为  $O(n+e)$ 。



2、D。只要掌握 DFS 和 BFS 的遍历过程,便能轻易解决。逐个代入,手工模拟,选项 D 是深度优先遍历,而不是广度优先遍历。

3、D。采用图的深度优先遍历,共 5 种可能:  $\langle V_0, V_1, V_3, V_2 \rangle$ ,  $\langle V_0, V_2, V_3, V_1 \rangle$ ,  $\langle V_0, V_2, V_1, V_3 \rangle$ ,  $\langle V_0, V_3, V_2, V_1 \rangle$ ,  $\langle V_0, V_3, V_1, V_2 \rangle$ , 选 D。

4、D。对于本题,只需按深度优先遍历的策略进行遍历即可。对于选项 A: 先访问  $V_i$ , 然后访问与  $V_i$  邻接且未被访问的任一顶点(满足的有  $V_1$  和  $V_s$ ), 此时访问  $V_s$ , 然后从  $V_s$  出发,访问与  $V_s$  邻接且未被访问的任一顶点(满足的只有  $V_4$ ), 然后从  $V_4$  出发,访问与  $V_4$  邻接且未被访问的任一顶点(满足的只有  $V_3$ ), 然后从  $V_3$  出发,访问与  $V_3$  邻接且未被访问的任一顶点(满足的只有  $V_2$ ), 结束遍历。选项 B 和 C 的分析方法与选项 A 相同,不再赘述。对于选项 D, 首先访问  $V_i$ , 然后从  $V_i$  出发,访问与  $V_i$  邻接且未被访问的任一顶点(满足的有  $V_2$ 、 $V_3$  和  $V_s$ ), 然后从  $V_2$  出发,访问与  $V_2$  邻接且未被访问的任一顶点(满足的只有  $V_s$ ), 按规则本应该访问  $V_s$ , 但选项 D 却访问  $V_3$ , 因此 D 错误。

5、B。DFS 是一个递归算法,在遍历过程中,先访问的顶点被压入栈底。设在图中有顶点  $V_i$ , 它有后继顶点  $V_j$ , 即存在边  $\langle V_i, V_j \rangle$ 。根据 DFS 的规则,  $V_i$  入栈后,必先遍历完其后继顶点后  $V_i$  才会出栈,也就是说  $V_i$  会在  $V_j$  之后出栈,在如题所指的过程中,  $V_i$  在  $V_j$  后打印。由于  $V_i$  和  $V_j$  具有任意性,从上面的规律可以看出,输出顶点的序列是逆拓扑有序序列。

## 类型二十: 最小生成树的构造算法和性质

1、【2012.08】下列关于最小生成树的说法中,正确的是\_\_\_\_\_。

I. 最小生成树的代价唯一

II. 权值最小的边一定会出现在所有的最小生成树中

III. 用普里姆 (Prim) 算法从不同顶点开始得到的最小生成树一定相同

IV. 普里姆算法和克鲁斯卡尔 (Kruskal) 算法得到的最小生成树总不相同

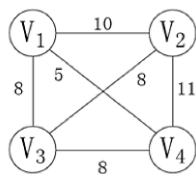
A. 仅 I

B. 仅 II

C. 仅 I、III

D. 仅 II、IV

2、【2015.06】求下面带权图的最小(代价)生成树时,可能是克鲁斯卡 (Kruskal) 算法第 2 次选中但不是普里姆 (Prim) 算法(从  $V_4$  开始)第 2 次选中的边是\_\_\_\_\_。



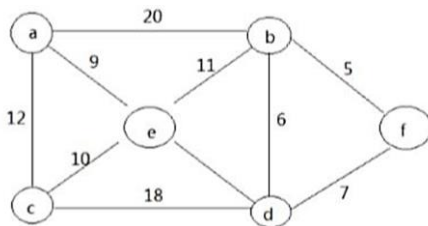
A.  $(V_1, V_3)$

B.  $(V_1, V_4)$

C.  $(V_2, V_3)$

D.  $(V_3, V_4)$

3、【2020.07】已知无向图 G 如下所示,使用克鲁斯卡尔 (Kruskal) 算法求图 G 的最小生成树,加入到最小生成树中的边依次是:



A.  $(b, f)(b, d)(a, e)(c, e)(b, e)$

B.  $(b, f)(b, d)(b, e)(a, e)(c, e)$

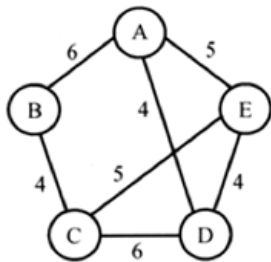
C.  $(a, e)(b, e)(c, e)(b, d)(b, f)$

D.  $(a, e)(c, e)(b, e)(b, f)(b, d)$

4、【2017.42】使用 Prim(普里姆)算法求带权连通图的最小(代价)生成树(MST)。请回答下列问题。

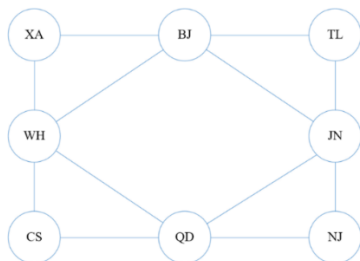
(1) 对下列图 G, 从顶点 A 开始求 G 的 MST, 依次给出按算法选出的边。

(2) 图 G 的 MST 是唯一的吗?



(3) 对任意的带权连通图, 满足什么条件时, 其 MST 是唯一的?

5、【2018.42】拟建设一个光通信骨干网络连通 BJ、CS、XA、QD、JN、NJ、TL 和 WH 等 8 个城市, 题图中无向边上的权值表示两个城市间备选光缆的铺设费用。



请回答下列问题。

(1) 仅从铺设费用角度出发, 给出所有可能的最经济的光缆铺设方案(用带权图表示), 并计算相应方案的总费用。

(2) 题 42 图可采用图的哪一种存储结构? 给出求解问题(1)所使用的算法名称。

(3) 假设每个城市采用一个路由器按(1)中得到的最经济方案组网, 主机 H1 直接连接在 TL 的路由器上, 主机 H2 直接连接在 BJ 的路由器上。若 H1 向 H2 发送一个 TTL = 5 的 IP 分组, 则 H2 是否可以收到该 IP 分组?

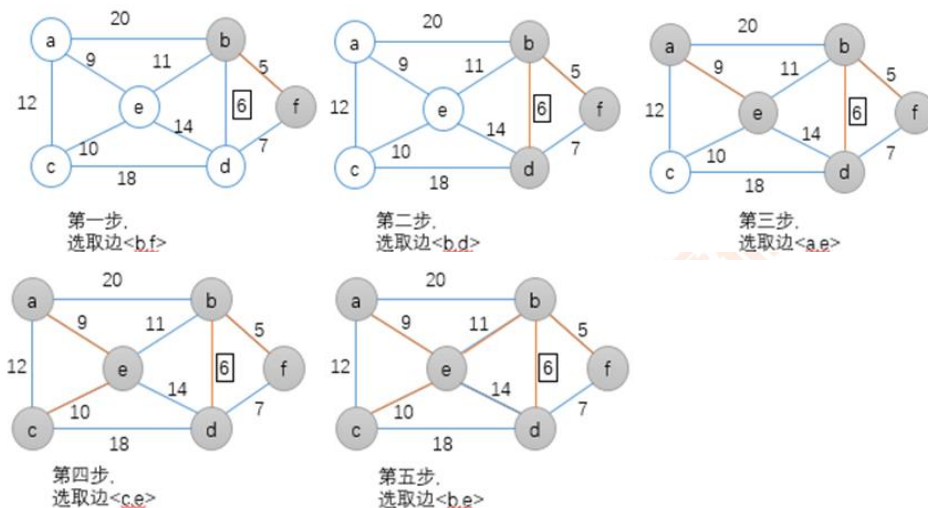
### 【答案解析】

1、A。最小生成树的树形可能不唯一(这是因为可能存在权值相同的边), 但是代价一定是唯一的, I 正确。如果权值最小的边有多条并且构成环状, 则总有权值最小的边将不出现在某棵最小生成树中, II 错误。设 N 个结点构成环, N-1 条边权值相等, 则从不同的顶点开始普里姆算法会得到 N-1 中不同的最小生成树, III 错误。当最小生成树唯一时(各边的权值不同), 普里姆算法和克鲁斯卡尔算法得到的最小生成树相同, IV 错误。

2、C。根据 Kruskal 算法和 Prim 算法计算最小生成树。从 V4 开始, Kruskal 算法选中的第一条边一定是权值最小的(V1, V4), B 错误。由于 V1 和 V4 已经可达, 第二条边含有 V1 和 V4 的权值为 8 的一定符合 Prim 算法, 排除 A、D。

3、A。Kruskal 算法: 按权值递增顺序依次选取 n-1 条边, 并保证这 n-1 条边不构成回路。初始构造一个仅含 n 个顶点的森林; 第一步, 选取权值最小的边(b, f)加入最小生成树; 第二步, 剩余边中权值最小的边为(b, d), 加入最小生成树, 第二步操作后权值最小的边(d, f)不能选, 因为会与之前已选取的边形成回路; 接下来依次选取权值 9、10、11 对应的边加入最小生成树, 此时 6 个顶点形成了一棵树, 最小生成树构造完成。按照上述过程, 加到最小生成树的边依次为(b, f), (b, d), (a,

e), (c, e), (b, e)。其生成过程如下所示:



4、(1) Prim 算法属于贪心策略。算法从一个任意的顶点开始, 一直长大到覆盖图中所有顶点为止。算法每一步在连接树集合  $S$  中顶点和其他顶点的边中, 选择一条使得树的总权重增加最小的边加入集合  $S$ 。当算法终止时,  $S$  就是最小生成树。

- ①  $S$  中顶点为  $A$ , 候选边为  $(A, D)$ ,  $(A, B)$ ,  $(A, E)$ , 选择  $(A, D)$  加入  $S$ 。
- ②  $S$  中顶点为  $A, D$ , 候选边为  $(A, B)$ ,  $(A, E)$ ,  $(D, E)$ ,  $(C, D)$ , 选择  $(D, E)$ , 加入  $s$ 。
- ③  $S$  中顶点为  $A, D, E$ , 候选边为  $(A, B)$ ,  $(C, D)$ ,  $(C, E)$ , 选择  $(C, E)$  加入  $S$ 。
- ④  $S$  中顶点为  $A, D, E, C$ , 候选边为  $(A, B)$ ,  $(B, C)$ , 选择  $(B, C)$  加入  $S$ 。
- ⑤  $S$  就是最小生成树。

依次选出的边为

$(A, D)$ ,  $(D, E)$ ,  $(C, E)$ ,  $(B, C)$

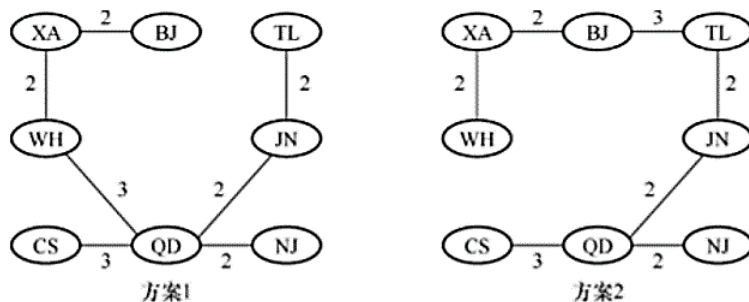
(2) 图  $G$  的 MST 是唯一的。

(3) 当带权连通图的任意一个环中所包含的边的权值均不相同时, 其 MST 是唯一的。

5、(1) 最经济的光缆铺设方案可以抽象为求最小生成树的问题。根据题图可以求得两种最经济的光缆建设方案, 如下所示。方案的总费用为 16。

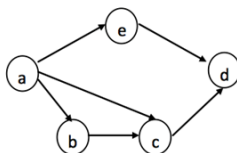
(2) 可采用邻接矩阵保存题 42 图。可使用 Prim 算法求解。

(3) 使用方案 1:  $H_2$  不能收到该 IP 分组;  
使用方案 2:  $H_2$  可以收到该 IP 分组。



## 类型二十一: AOE 网、拓扑排序和关键路径

1、【2010.08】对右图进行拓扑排序, 可以得到不同的拓扑序列的个数是\_\_\_\_\_。

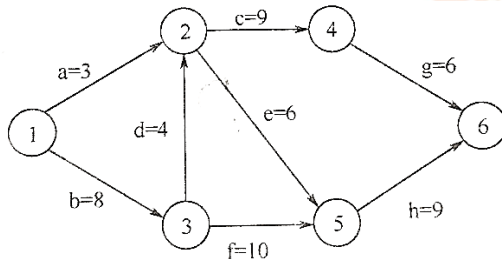


- A. 4                      B. 3                      C. 2                      D. 1

2、【2012.06】若用邻接矩阵存储有向图，矩阵中主对角线以下的元素均为零，则关于该图拓扑序列的结论是\_\_\_\_\_。

- A. 存在，且唯一                      B. 存在，且不唯一  
C. 存在，可能不唯一                      D. 无法确定是否存在

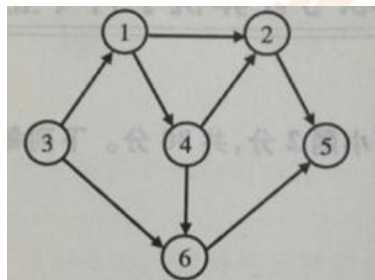
3、【2013.09】下列 AOE 网表示一项包含 8 个活动的工程。通过同时加快若干活动的进度可以缩短整个工程的工期。下列选项中，加快其进度就可以缩短工程工期的是\_\_\_\_\_。



- A. c 和 e                      B. d 和 e                      C. f 和 d                      D. f 和 h

4、【2014.07】对如下所示的有向图进行拓扑排序，得到的拓扑序列可能是\_\_\_\_\_。

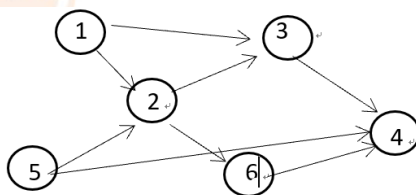
- A. 3, 1, 2, 4, 5, 6                      B. 3, 1, 2, 4, 6, 5  
C. 3, 1, 4, 2, 5, 6                      D. 3, 1, 4, 2, 6, 5



5、【2016.07】若将  $n$  个顶点  $e$  条弧的有向图采用邻接表存储，则拓扑排序算法的时间复杂度是\_\_\_\_\_。

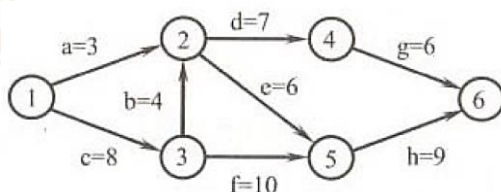
- A.  $O(n)$                       B.  $O(n+e)$                       C.  $O(n^2)$                       D.  $O(n \cdot e)$

6、【2018.07】下列选项中不是如下有向图的拓扑序列的是：



- A. 1, 5, 2, 3, 6, 4                      B. 5, 1, 2, 6, 3, 4  
C. 5, 1, 2, 3, 6, 4                      D. 5, 2, 1, 6, 3, 4

7、【2019.05】下图所示的 AOE 网表示一项包含 8 个活动的工程。活动 d 的最早开始时间和最迟开始时间分别是



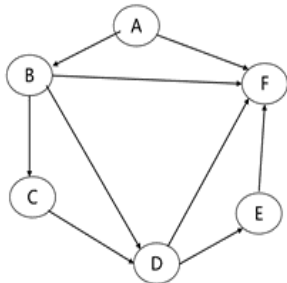
- A. 3 和 7                      B. 12 和 12                      C. 12 和 14                      D. 15 和 15

8、【2020.08】若使用 AOE 网估算工程进度，则下列叙述中正确的是：



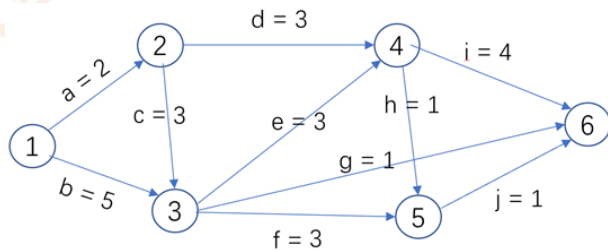
- A、关键路径是从原点到汇点边数最多的一条路径  
B、关键路径是从原点到汇点路径长度最长的路径  
C、增加任一关键活动的时间不会延长工程的工期  
D、缩短任一关键活动的时间将会缩短工程的工期

9、【2021.07】给定如下有向图，该图的拓扑有序序列的个数是



- A.1                      B.2                      C.3                      D.4

10、【2022.7】下图是一个有 10 个活动的 AOE 网，时间余量最大的活动是 ( )

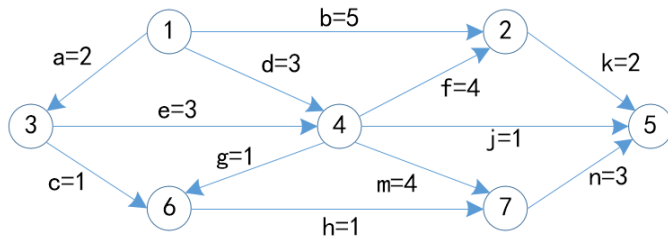


- A. e                      B. g                      C. h                      D. j

11、【2025.42】某工程包含 12 个活动，使用题 42 图所示的 AOE 网描述，图中各边上标注了活动及其持续时间。请回答下列问题（活动均用活动名表示）

- (1) 完成该工程的最短时间是多少？哪些活动是关键活动？（3 分）
- (2) 若从最短时间完成工程，则与活动 e 同时进行的活动可能有哪些？（3 分）
- (3) 时间余量最大的活动是哪个？其余时间余量是多少？（2 分）

(4) 假设工程从时刻 0 启动，因某种原因，活动 b 在时刻 6 开始。为了保证工程不延期，在其他活动持续时间均不变的情况下，b 的持续时间最多是多少？若改变 b 的持续时间，则压缩哪个活动的持续时间也能保证工程不延期？（2 分）



### 【答案解析】

1、B。根据题意可以得到 3 个不同的拓扑序列，分别为 aebcd、abced、abecd。

2、C。对角线以下元素均为零，表明只有顶点 i 到顶点 j ( $i < j$ ) 可能有边，而顶点 j 到顶点 i 一定没有边，即有向图是一个无环图，因此一定存在拓扑序列。对于拓扑序列是否唯一，试举一例：设有

向图的邻接矩阵  $\begin{pmatrix} 0 & 1 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$ , 则存在两个拓扑序列, 因此该图存在可能不唯一的拓扑序列。若题目中说对角线以上均为 1, 以下均为 0, 则拓扑序列唯一。

3、C。找出 AOE 网的全部关键路径为(b, d, c, g)、(b, d, e, h)和(b, f, h)。根据定义, 只有关键路径上的活动时间同时减少时, 才能缩短工期, 即正确选项中的两条路径必须涵盖在所有关键路径之中。利用关键路径算法可求出图中的关键路径共有三条: (b, d, c, g)、(b, d, e, h)和(b, f, h)。由此可知, 选项 A 和 B 中并不能包含(b, f, h)这条路径, 选项 C 中, 并不能包含(b, d, c, g)和(b, d, e, h)这两条路径, 只有 C 包含了所有的关键路径, 因此只有加快 f 和 d 的进度才能缩短工期。

4、D。按照拓扑排序规则排序即可。按照拓扑排序的算法, 每次都选择入度为 0 的结点从图中删去, 此图中一开始只有结点 3 的入度为 0; 删掉结点 3 后, 只有结点 1 的入度为 0; 删掉结点 1 后, 只有结点 4 的入度为 0; 删掉结点 4 后, 结点 2 和结点 6 的入度都为 0, 此时选择删去不同的结点, 会得出不同的拓扑序列, 分别处理完毕后可知可能的拓扑序列为 3, 1, 4, 2, 6, 5 和 3, 1, 4, 6, 2, 5, 选 D。

5、B。根据拓扑排序的规则, 输出每个顶点的同时还要删除以它为起点的边, 这样对各顶点和边都要进行遍历, 故拓扑排序的时间复杂度为  $O(n + e)$ 。

6、D。拓扑排序每次选取入度为 0 的结点输出, 经观察不难发现拓扑序列前两位一定是 1, 5 或 5, 1 (因为只有 1 和 5 的入度均为 0, 且其他结点都不满足仅有 1 或仅有 5 作为前驱)。因此 D 显然错误。

7、C。活动 d 的最早开始时间等于该活动弧的起点所表示的事件的最早发生时间, 活动 d 的最早开始时间等于事件 2 的最早发生时间  $\max\{a, b + c\} = \max\{3, 12\} = 12$ 。活动 d 的最迟开始时间等于该活动弧的终点所表示的事件的最迟发生时间与该活动所需时间之差, 先算出图中关键路径长度为 27 (对于不复杂的选择题, 找出所有路径计算长度), 那么事件 4 的最迟发生时间为  $\min\{27 - g\} = \min\{27 - 6\} = 21$ , 活动 d 的最迟开始时间为  $21 - d = 21 - 7 = 14$ 。

常规方法: 按照关键路径算法得到下表。

|         | $v_1$ | $v_2$ | $v_3$ | $v_4$ | $v_5$ | $v_6$ |
|---------|-------|-------|-------|-------|-------|-------|
| $ve(i)$ | 0     | 12    | 8     | 19    | 18    | 27    |
| $vl(i)$ | 0     | 12    | 8     | 21    | 18    | 27    |

|               | a | b | c | d  | e  | f | g  | h  |
|---------------|---|---|---|----|----|---|----|----|
| $e(i)$        | 0 | 8 | 0 | 12 | 12 | 8 | 19 | 18 |
| $l(i)$        | 9 | 8 | 0 | 14 | 12 | 8 | 21 | 18 |
| $l(i) - e(i)$ | 9 | 0 | 0 | 2  | 0  | 0 | 2  | 0  |

从表中可知, 活动 d 的最早开始时间和最迟开始时间分别为 12 和 14, 故选 C

8、B。关键路径是指权值之和最大而非边数最多的路径, 故选项 A 错误。选项 B 正确, 是关键路径的概念。无论是存在一条还是存在多条关键路径, 增加任一关键活动的时间都会延长工程的工期, 因为关键路径始终是权值之和最大的那条路径, 选项 C 错误。仅有一条关键路径时, 减少关键活动的时间会缩短工程的工期; 存在多条关键路径时, 缩短一条关键活动的时间不一定会缩短工程的工期, 缩短了路径长度的那条关键路径不一定还是关键路径, 选项 D 错误。

9、A。拓扑排序的方法: (1)在有向图中选一个没有前驱的顶点并且输出; (2)从图中删除该顶点和所有以它为尾的孤, 即删除所有与它有关的边; (3)重复上述两步, 直至全部顶点均已输出: 或者当图中不存在无前驱的顶点为止。根据此有向图, 可得到拓扑排序序列只有一个: 即: AECDEF。故本题答案为 A。

10、B。先正向 (从源点往汇点) 推导事件 (顶点) 的最早发生时间  $ve(i)$ 。再逆向 (从汇点往

源点) 推导事件 (顶点) 的最晚发生时间  $vl(i)$ 。

再正向 (从源点往汇点) 推导活动 (有向边) 的最早发生时间  $e(i)$ , 即有向边的起点的最早发生时间。再逆向 (从汇点往源点) 推导活动 (有向边) 的最晚发生时间  $l(i)$ , 即有向边的终点最晚发生时间减去弧长。

最后计算活动时间余量  $l(i)-e(i)$ , 计算表格如下所示,

|             | a | b | c | d | e | f | g  | h  | i | j  |
|-------------|---|---|---|---|---|---|----|----|---|----|
| $e(i)$      | 0 | 0 | 2 | 2 | 5 | 5 | 5  | 8  | 8 | 9  |
| $l(i)$      | 0 | 0 | 2 | 5 | 5 | 8 | 11 | 10 | 8 | 11 |
| $l(i)-e(i)$ | 0 | 0 | 0 | 3 | 0 | 3 | 6  | 2  | 0 | 2  |

很明显时间余量最大的是活动 g。

11、(1) 【答案】完成该工程的最短时间是 12。关键活动是 a, e, m, n。

【解析】求各事件的最早发生时间:

路径  $1 \rightarrow 2 \rightarrow 5$ , 持续时间= $b+k=5+2=7$

路径  $1 \rightarrow 4 \rightarrow 2 \rightarrow 5$ , 持续时间= $d+f+k=3+4+2=9$

路径  $1 \rightarrow 4 \rightarrow 5$ , 持续时间= $d+f+j=3+4+1=4$

路径  $1 \rightarrow 4 \rightarrow 7 \rightarrow 5$ , 持续时间= $d+m+n=3+4+3=10$

路径  $1 \rightarrow 4 \rightarrow 6 \rightarrow 7 \rightarrow 5$ , 持续时间= $d+g+h+n=3+1+1+3=8$

路径  $1 \rightarrow 3 \rightarrow 4 \rightarrow 2 \rightarrow 5$ , 持续时间= $a+e+f+k=2+3+4+2=11$

路径  $1 \rightarrow 3 \rightarrow 4 \rightarrow 5$ , 持续时间= $a+e+j=2+3+1=6$

路径  $1 \rightarrow 3 \rightarrow 4 \rightarrow 7 \rightarrow 5$ , 持续时间= $a+e+m+n=2+3+4+3=12$

路径  $1 \rightarrow 3 \rightarrow 4 \rightarrow 6 \rightarrow 7 \rightarrow 5$ , 持续时间= $a+e+g+h+n=2+3+1+1+3=10$

路径  $1 \rightarrow 3 \rightarrow 6 \rightarrow 7 \rightarrow 5$ , 持续时间= $a+c+h+n=2+1+1+3=7$

因此, 最早完成整个工程的时间为 12。

求个事件最晚发生时间, 从终点到起点反向推导。

$L[5]=12, L[2]=10, L[4]=5, L[7]=9, L[6]=8, L[3]=2, L[0]=0$ 。

完成该工程的最短时间是 12。关键路径上的活动需要满足最早开始时间和最晚开始时间相等 (且不能违背总时间), 因此这 4 个活动 a、e、m、n 为关键活动。

(2) 【答案】与 e 同时进行的活动可能有 b, c, d。

【解析】进行活动 e, 先要完成活动 a, 因此与活动 e 能同时进行的活动, 是从节点 1 或节点 3 出发。

路径  $1 \rightarrow 2$ , 进行活动 b;

路径  $1 \rightarrow 4$ , 进行活动 d

路径  $1 \rightarrow 3 \rightarrow 6$ , 进行活动 c

(3) 【答案】时间余量最大的活动是 j。时间余量是 6。

【解析】时间余量指的是活动的最晚开始时间与最早开始时间 (或最晚完成时间与最早完成时间) 之间的差值。

| 活动                     | a | b  | c | d | e | f  | g | h | j  | k  | m | n  |
|------------------------|---|----|---|---|---|----|---|---|----|----|---|----|
| 起点 u                   | 1 | 1  | 3 | 1 | 3 | 4  | 4 | 6 | 4  | 2  | 4 | 7  |
| 终点 v                   | 3 | 2  | 6 | 4 | 4 | 2  | 6 | 7 | 5  | 5  | 7 | 5  |
| 持续时间 t                 | 2 | 5  | 1 | 3 | 3 | 4  | 1 | 1 | 1  | 2  | 4 | 3  |
| $E[u]$                 | 0 | 0  | 2 | 0 | 2 | 5  | 5 | 6 | 5  | 9  | 5 | 9  |
| $L[v]$                 | 2 | 10 | 8 | 5 | 5 | 10 | 8 | 9 | 12 | 12 | 9 | 12 |
| 余量 = $L[v] - E[u] - t$ | 0 | 5  | 5 | 2 | 0 | 1  | 2 | 2 | 6  | 1  | 0 | 0  |

由上表可知, 时间余量最大的活动是 j, 余量为 6。

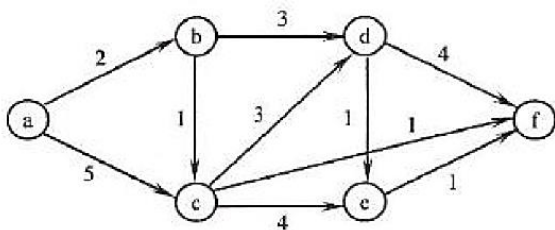
(4) b 的持续时间最多是 4。压缩 k 的持续时间。

①参与 b 活动的路径,  $1 \rightarrow 2 \rightarrow 5$ , 最晚时间  $L[2]=10$ , 活动 b 在时刻 6 开始, 若保证能够完工, 那么  $b = L[2] - 6 = 10 - 6 = 4$ 。所以 b 的持续时间最多是 4。

②参与 b 活动的路径,  $1 \rightarrow 2 \rightarrow 5$ , 这条路径进行的活动是 b 和 k, 活动 b 的时间不改变, 那么就要压缩活动 k 的持续时间。根据题目  $E[2] = 6 + 5 = 11$ ,  $k = 12 - 11 = 1$ 。

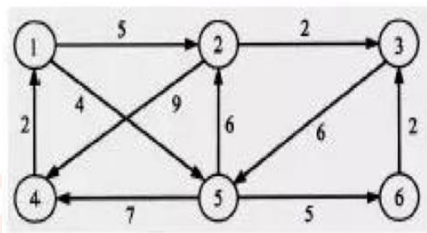
## 类型二十二：最短路径

1、【2012.07】对如下有向带权图, 若采用迪杰斯特拉 (Dijkstra) 算法求源点 a 到其他各顶点的最短路径, 则得到的第一条最短路径的目标顶点是 b, 第二条最短路径的目标顶点是 c, 后续得到的其余各最短路径的目标顶点依次是\_\_\_\_\_。



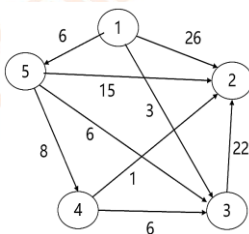
- A. d, e, f      B. e, d, f      C. f, d, e      D. f, e, d

2、【2016.08】使用迪杰斯特拉 (Dijkstra) 算法求下图中从顶点 1 到其他各顶点的最短路径, 依次得到的各最短路径的目标顶点是\_\_\_\_\_。



- A. 5, 2, 3, 6, 4      B. 5, 2, 3, 6, 4  
C. 5, 2, 4, 3, 6      D. 5, 2, 6, 3, 4

3、【2021.08】使用 Dijkstra 算法求下图中从顶点 1 到其余各顶点的最短路径, 将当前找到的从顶点 1 到顶点 2、3、4、5 的最短路径长度保存在数组 dist 中, 求出第二条最短路径后, dist 中的内容更新为 ( )



- A. 26, 3, 14, 6      B. 25, 3, 14, 6      C. 21, 3, 14, 6      D. 15, 3, 14, 6

4、【2023.06】已知无向连通图 G 中各边的权值均为 1。下列算法中, 一定能够求出图 G 中从某顶点到其余各个顶点最短路径的是 ( )

- I. 普里姆 (Prim) 算法      II. 克鲁斯卡尔 (Kruskal) 算法      III. 图的广度优先搜索算法  
A. 仅 I      B. 仅 III      C. I、II      D. I、II、III



5、【2009.41】带权图（权值非负，表示边连接的两顶点间的距离）的最短路径问题是找出从初始顶点到目标顶点之间的一条最短路径。假设从初始顶点到目标顶点之间存在路径，现有一种解决该问题的方法：

- ①设最短路径初始时仅包含初始顶点，令当前顶点  $u$  为初始顶点；
- ②选择离  $u$  最近且尚未在最短路径中的一个顶点  $v$ ，加入到最短路径中，修改当前顶点  $u=v$ ；
- ③重复步骤②，直到  $u$  是目标顶点时为止。

请问上述方法能否求得最短路径？若该方法可行，请证明之；否则，请举例说明。

### 【答案解析】

1、C。根据 Dijkstra 算法求解即可。从  $a$  到各顶点的最短路径的求解过程：

| 原点   | 第 1 趟    | 第 2 趟    | 第 3 趟      | 第 4 趟       | 第 5 趟         |
|------|----------|----------|------------|-------------|---------------|
| b    | (a,b)2   |          |            |             |               |
| c    | (a,c)5   | (a,b,c)3 |            |             |               |
| d    | $\infty$ | (a,b,c)5 | (a,b,d)5   | (a,b,d)5    |               |
| e    | $\infty$ | $\infty$ | (a,b,c,f)4 |             |               |
| f    | $\infty$ | $\infty$ | (a,b,c,e)7 | (a,b,c,e)7  | (a,b,d,e)6    |
| 集合 S | {a,b}    | {a,b,c}  | {a,b,c,f}  | {a,b,c,f,d} | {a,b,c,f,d,e} |

后续目标顶点依次为  $f,d,e$ 。对于 A，若下一个顶点为  $d$ ，路径  $a,b,d$  的长度 5，而  $a,b,c,f$  的长度仅为 4，显然错误。同理可以排除 B。将  $f$  加入集合  $S$  后，采用上述的方法也可以排除 D。

2、B。根据 Dijkstra 算法，从顶点 1 到其余各顶点的最短路径如下表所示，

| 顶点   | 第一趟      | 第二趟          | 第三趟          | 第四趟          | 第五趟           |
|------|----------|--------------|--------------|--------------|---------------|
| 2    | 5(V1-V2) | 5(V1-V2)     |              |              |               |
| 3    | $\infty$ | $\infty$     | 7(V1-V2-V3)  |              |               |
| 4    | $\infty$ | 11(V1-V5-V4) | 11(V1-V5-V4) | 11(V1-V5-V4) |               |
| 5    | 4(V1-V5) |              |              |              |               |
| 6    | $\infty$ | 9(V1-V5-V6)  | 9(V1-V5-V6)  | 9(V1-V5-V6)  |               |
| 集合 S | {1,5}    | {1,5,2}      | {1,5,2,3}    | {1,5,2,3,6}  | {1,5,2,3,6,4} |

3、C。从 1 出发，第一条最短路径是  $1 \rightarrow 3$ ；第二条最短路径是  $1 \rightarrow 5$ ；此时 3 和 5 的最短路径已经找到。

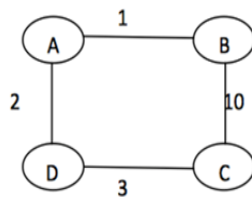
于是 4 的 dist 是： $1 \rightarrow 5 \rightarrow 4$  距离是 14；于是 2 的 dist 是： $1 \rightarrow 5 \rightarrow 2$  距离是 21。

所以 dist 是 21，3，14，6；

4、B。无向连通图  $G$  中各边的权值均为 1 可以将  $G$  图视为无权图，可以利用广度优先遍历 BFS 寻找单源最短路径。普里姆算法，克鲁斯卡尔算法用来构造最小生成树，因此选 B。

5、该方法不一定能（或不能）求得最短路径。

例如，对于右图所示的带权图，如果按照题中的原则，从  $A$  到  $C$  的最短路径是  $A \rightarrow B \rightarrow C$ ，事实上其最短路径是  $A \rightarrow D \rightarrow C$ 。



## 类型二十三：图算法和综合应用

### 1. 图的基本算法

1、【2021.41】已知无向连通图  $G$  由顶点集  $V$  和边集  $E$  组成， $|E| > 0$ ，当  $G$  中度为奇数的顶点个数为不大于 2 的偶数时， $G$  存在包含所有边且长度为  $|E|$  的路径（称为 EL 路径）。设图  $G$  采用邻接矩阵存储，类型定义如下

```
typedef struct t{ // 图的定义
```

```
Int numVertices, numEdges; // 图中实际的顶点数和边数
char VerticesList[MAXV] ; // 顶点表。MAXV 为已定义常量
Int Edge[MAXV][MAXV]; //邻接矩阵
} MGraph;
```

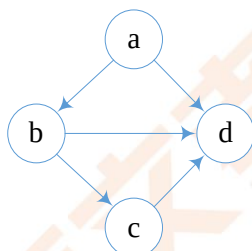
请设计算法: int IsExistEL(MGraph G), 判断 G 是否存在 EL 路径, 若存在, 则返回 1, 否则, 返回 0。要求:

- (1) 给出算法的基本设计思想。
- (2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。
- (3) 说明你所设计算法的时间复杂度和空间复杂度。

2、【2023.41】已知有向图 G 采用邻接矩阵存储, 类型定义如下:

```
typedef struct //图的类型定义
{
 int numVertices, numEdges; //图的顶点数和有向边数
 char VerticesList[MAXV]; //顶点表, MAXV 为以定义常量
 int Edge[MAXV][MAXV]; //邻接矩阵
} MGraph;
```

将图中出度大于入度的顶点成为 K 顶点, 例如题 41 图中, 顶点 a 和 b 为 k 顶点。



题 41 图

请设计算法: int printVertices (MGraph G), 对给定的任意非空有向图 G, 输出 G 中所有的 K 顶点, 并返回 K 顶点的个数。

- (1) 给出算法的基本设计思想。
- (2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。

3、【2024.41】2023 年 10 月 26 日, 神州十七号咱人飞船发射取得圆满成功, 再次彰显了中国航天事业的辉煌成就。载人航天工程是包含众多子工程的复杂系统工程, 为了保证工程的有序开展, 需要明确各子工程的前导工程, 以协调各子工程的实施。该问题可以简化、抽象为有向图的拓扑序列问题。已知有向图 G 采用邻接矩阵存储, 类型定义如下:

```
typedef struct //图的类型定义
{
 int numVertices, numEdges; //图的顶点数和有向边数
 char VerticesList[MAXV]; //顶点表, MAXV 为以定义常量
 int Edge[MAXV][MAXV]; //邻接矩阵
} MGraph;
```

请设计算法: int uniquely(MGraph G), 判定 G 是否存在唯一的拓扑序列, 若是则返回 1, 否则返回 0。

要求:

- (1) 给出算法的基本设计思想 (4 分)。
- (2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释 (9 分)。

### 【答案解析】

#### 1、(1) 算法的基本设计思想

对于采用邻接矩阵存储的无向图，邻接矩阵每一行(列)中非零元素的个数为本行(列)对应顶点的度。可以依次计算连通图  $G$  中各顶点的度，并记录度为奇数的顶点个数，若个数为 0 或 2，则返回 1，否则返回 0。

#### (2) 算法实现

//采用邻接矩阵存储，判断图是否存在 EL 路径

```
int IsExistEL(MGraph G) {
 int degree, i, j, count=0;
 for(i=0;i<G.numVertices;i++){
 degree=0;
 for(j=0;j<G.numVertices;j++){ //依次计算各个顶点的度
 degree+= G.Edge[i][j];
 }
 if(degree%2 != 0) //对度为奇数的顶点计数
 count++;
 }
 if(count== 0 || count ==2) //存在 EL 路径，返回 1
 return 1;
 else //不存在 EL 路径，返回 0
 return 0;
}
```

(3) 算法的时间复杂度时间复杂度是  $O(n^2)$ ，空间复杂度是  $O(1)$ 。

2、(1) 算法思想：遍历有向图中所有顶点，并统计各顶点的出度和入度，输出出度大于入度的  $K$  顶点， 并使用变量  $count$  累计  $K$  顶点的总数。

计算顶点  $i$  的出度：遍历邻接矩阵的  $i$  行元素，即  $Edge[i][0] \sim Edge[i][numVertex-1]$ ，统计非零元素个数，即为顶点  $i$  的出度。

计算顶点  $i$  的入度：遍历邻接矩阵的  $i$  列元素， 即  $Edge[0][i] \sim Edge[numVertex-1][i]$ ，统计非零元素个数，即为顶点  $i$  的入度

#### (2) 算法实现：

```
int printVertices(MGraph G) {
 int count=0; //K 顶点的总数
 for(int i=0;i<G.numVertex;i++) {
 int outDegree=0; //顶点 i 的出度
 int inDegree=0; //顶点 i 的入度
 for(int j=0;j<G.numVertex;j++) {
 if(G.Edge[i][j] >0) outDegree++;
 }
 for(int j=0;j<G.numVertex;j++) {
 if(G.Edge[j][i] >0) inDegree++; //循环两次方便理解
 }
 if(outDegree>inDegree){ //顶点 i 的出度大于入度
 printf("%c\n", G.Vertex List[i]); //输出顶点 i
 count++; //累加 K 顶点总数
 }
 }
 return count; //返回 K 顶点总数
}
```

### 3、(1) 算法的基本设计思想

建立图 G 各顶点的入度表 `degree[]`。

选择入度为 0 的顶点 `v`，将 `v` 的所有邻接点的入度减 1，重复执行这个过程。若每次选中的入度为 0 的顶点有且仅有一个，且共进行了 `G.numVertices` 次，则意味着存在唯一的拓扑序列，返回 1，否则不存在拓扑序列，或存在多个拓扑序列，返回 0。

### (2) 用 C 语言描述的算法

图中的代码内容如下：

```
int uniquely(MGraph G) {
 int* degree,i,j,count=0,in0=-1,prev_in0;
 degree=(int*)malloc(G.numVertices*sizeof(int));
 for(j=0;j<G.numVertices;j++){
 degree[j]=0;
 for(i=0;i<G.numVertices;i++){
 degree[j]+=G.Edge[i][j];
 if(degree[j]==0) {
 if(in0==-1)in0=j;
 else in0=-2;
 }
 }
 }
 while(in0>0) {
 count++;
 prev_in0=in0;
 in0=-1;
 for(j=0;j<G.numVertices;j++){
 if(G.Edge[prev_in0][j]>0) {
 if(--degree[j]==0) {
 if(in0==-1)in0=j;
 else in0=-2;
 }
 }
 }
 }
 free(degree);
 if(count==G.numVertices)return 1;
 else return 0;
}
```

//判定有向图是否存在唯一的拓扑序列  
// 动态分配数组存储各个顶点的入度  
//计算各顶点的入度  
// 初始化顶点的入度为 0  
// 遍历图中的所有边，累加入度  
// 如果找到入度为 0 的顶点  
// 入度为 0 的顶点  
// 有多个入度为 0 的顶点  
// 处理入度为 0 的顶点  
// 每次找到一个入度为 0 的顶点就增加计数  
// 记录当前入度为 0 的顶点  
// 重置当前入度为 0 的顶点标识  
//从入度为 0 的顶点删除边  
//如果该邻接点存在边  
//邻接点入度值减 1  
//入度为 0 的顶点  
//有多个入度为 0 的顶点  
//释放内存空间  
//存在唯一拓扑序列  
//不存在唯一拓扑序列

## 2. 图的应用

1、【2011.41】已知有 6 个顶点（顶点编号为 0~5）的有向带权图 G，其邻接矩阵 A 为上三角矩阵，按行为主序（行优先）保存在如下的一维数组中。

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 4 | 6 | ∞ | ∞ | ∞ | 5 | ∞ | ∞ | ∞ | 4 | 3 | ∞ | ∞ | 3 | 3 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

要求：

- (1) 写出图 G 的邻接矩阵 A。
- (2) 画出有向带权图 G。
- (3) 求图 G 的关键路径，并计算该关键路径的长度。

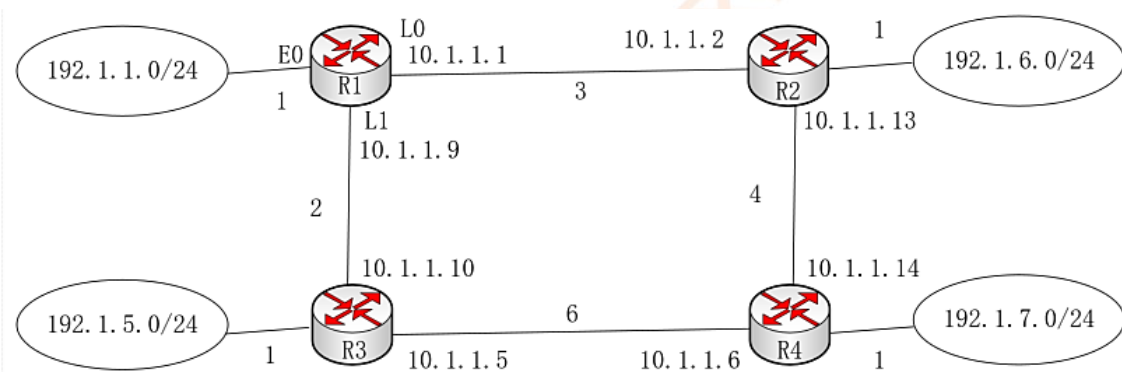
2、【2014.42】某网络中的路由器运行 OSPF 路由协议，题 42 表是路由器 R1 维护的主要链路状态信息(LSI)，题 42 图是根据题 42 表及 R1 的接口名构造出来的网络拓扑。

|  |          |          |          |          |    |
|--|----------|----------|----------|----------|----|
|  | R1 的 LSI | R2 的 LSI | R3 的 LSI | R4 的 LSI | 备注 |
|--|----------|----------|----------|----------|----|



|        |        |              |              |              |              |                  |
|--------|--------|--------------|--------------|--------------|--------------|------------------|
| Router |        | 10.1.1.1     | 10.1.1.2     | 10.1.1.5     | 10.1.1.6     | 标识路由器的 IP 地址     |
| Link1  | ID     | 10.1.1.2     | 10.1.1.1     | 10.1.1.6     | 10.1.1.5     | 所连路由器的 Route ID  |
|        | IP     | 10.1.1.1     | 10.1.1.2     | 10.1.1.5     | 10.1.1.6     | Link1 的本地 IP 地址  |
|        | Metric | 3            | 3            | 6            | 6            | Link1 的费用        |
| Link2  | ID     | 10.1.1.5     | 10.1.1.6     | 10.1.1.1     | 10.1.1.2     | 所连路由器的 Router ID |
|        | IP     | 10.1.1.9     | 10.1.1.13    | 10.1.1.10    | 10.1.1.14    | Link2 的本地 IP 地址  |
|        | Metric | 2            | 4            | 2            | 4            | Link2 的费用        |
| Net1   | Prefix | 192.1.1.0/24 | 192.1.6.0/24 | 192.1.5.0/24 | 192.1.7.0/24 | 直连网络 Net1 的网络前缀  |
|        | Metric | 1            | 1            | 1            | 1            | 到达直连网络 Net1 的费用  |

题 42 表 R1 所维护的 LSI



题 42 图 R1 构造的网络拓扑

请回答下列问题。

- (1) 本题中的网络可抽象为数据结构中的哪种逻辑结构?
- (2) 针对题 42 表中的内容, 设计合理的链式存储结构, 以保存题 42 表中的链路状态信息(LSI)。要求给出链式存储结构的数据类型定义, 并画出对应题 42 表的链式存储结构示意图(示意图中可仅以 ID 标识结点)。
- (3) 按照迪杰斯特拉(Dijkstra)算法的策略, 依次给出 R1 到达题 42 图中子网 192.1.x.x 的最短路径及费用。

### 【答案解析】

1、此题考察的知识点是图的存储以及关键路径求解的综合知识。

(1) 由题可以画出待定上三角矩阵的结构图如下(图中“?”待定元素)

$$\begin{bmatrix}
 0 & ? & ? & ? & ? & ? \\
 \infty & 0 & ? & ? & ? & ? \\
 \infty & \infty & 0 & ? & ? & ? \\
 \infty & \infty & \infty & 0 & ? & ? \\
 \infty & \infty & \infty & \infty & 0 & ? \\
 \infty & \infty & \infty & \infty & \infty & 0
 \end{bmatrix}$$

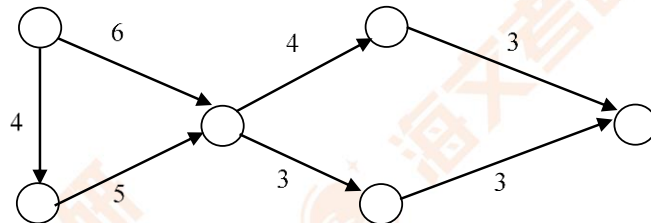
可以看出, 第一行至第五行主对角线上方的元素分别 5、4、3、2、1 个, 由此可以画出压缩存储数组中的元素所属行的情况, 如下图所示:

|     |   |          |          |          |     |          |          |          |   |     |          |          |   |     |
|-----|---|----------|----------|----------|-----|----------|----------|----------|---|-----|----------|----------|---|-----|
| 4   | 6 | $\infty$ | $\infty$ | $\infty$ | 5   | $\infty$ | $\infty$ | $\infty$ | 4 | 3   | $\infty$ | $\infty$ | 3 | 3   |
| 第一行 |   |          |          |          | 第二行 |          |          |          |   | 第三行 |          | 第四行      |   | 第五行 |

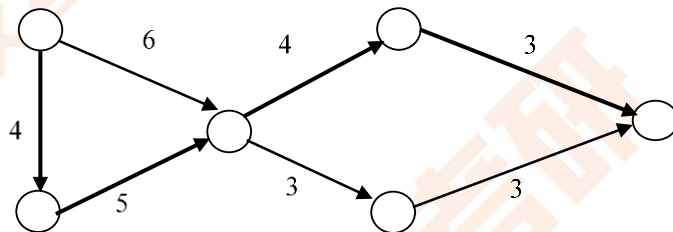
将各元素填入各行即得邻接矩阵: (2 分)

$$A = \begin{bmatrix} 0 & 4 & 6 & \infty & \infty & \infty \\ \infty & 0 & 5 & \infty & \infty & \infty \\ \infty & \infty & 0 & 4 & 3 & \infty \\ \infty & \infty & \infty & 0 & \infty & 3 \\ \infty & \infty & \infty & \infty & 0 & 3 \\ \infty & \infty & \infty & \infty & \infty & 0 \end{bmatrix}$$

(2) 根据第一步所得矩阵 A 容易做出有向带权图 G, 如下: (2 分)



(3) 下图中粗线箭头所标识的 4 个活动组成 G 的关键路径 (3 分)



由上图容易求得图的关键路径长度为:  $4+5+4+3=16$ 。

2、考察在给出具体模型时, 数据结构的应用。该题很多考生乍看之下以为是网络的题目, 其实题本身并没有涉及太多的网络知识点, 只是应用了网络的模型, 实际上考察的还是数据结构的内容。

(1) 图, 题中给出的是一个简单的网络拓扑图, 可以抽象为无向图。

(2) 链式存储结构的如下图所示

弧结点的两种基本形态

| Flag=1 | Next |
|--------|------|
| ID     |      |
| IP     |      |
| Metric |      |

| Flag=2 | Next |
|--------|------|
| Prefix |      |
| Mask   |      |
| Metric |      |

表头结点  
结构示意

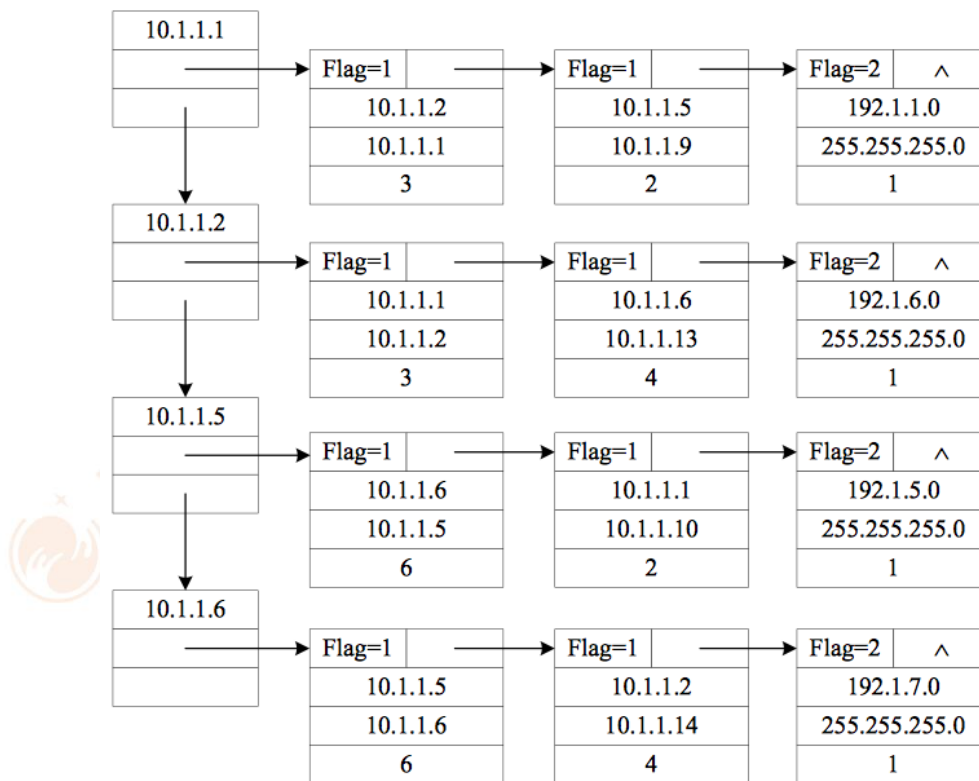
|          |
|----------|
| RouterID |
| LN_link  |
| Next     |

其数据类型定义如下:

```
typedef struct{
 unsigned int ID,IP;
}LinkNode; //Link 的结构
typedef struct{
 unsigned int Prefix, Mask;
}NetNode; //Net 的结构
typedef struct Node{
 int Flag; //Flag=1 为 Link; Flag=2 为 Net
 union{
 LinkNode Lnode;
 NetNode Nnode
 }LinkORNet;
 unsigned int Metric;
 struct Node *next;
}ArcNode; //弧结点
```

```
typedef struct HNode{
 unsigned int RouterID;
 ArcNode *LN_link;
 Struct HNode *next;
}HNODE; //表头结点
```

对应题 42 表的链式存储结构示意图如下。



(3) 计算结果如下表所示。

|      | 目的网络         | 路径                    | 代价 (费用) |
|------|--------------|-----------------------|---------|
| 步骤 1 | 192.1.1.0/24 | 直接到达                  | 1       |
| 步骤 2 | 192.1.5.0/24 | R1→R3→192.1.5.0/24    | 3       |
| 步骤 3 | 192.1.6.0/24 | R1→R2→192.1.6.0/24    | 4       |
| 步骤 4 | 192.1.7.0/24 | R1→R2→R4→192.1.7.0/24 | 8       |

## 类型二十四：顺序查找

1、【2013.42】设包含 4 个数据元素的集合  $S = \{ "do", "for", "repeat", "while" \}$ ，各元素的查找概率依次为： $p_1=0.35$ ， $p_2=0.15$ ， $p_3=0.15$ ， $p_4=0.35$ 。将  $S$  保存在一个长度为 4 的顺序表中，采用折半查找法，查找成功时的平均查找长度为 2.2。请回答：

(1) 若采用顺序存储结构保存  $S$ ，且要求平均查找长度更短，则元素应如何排列？应使用何种查找方法？查找成功时的平均查找长度是多少？

(2) 若采用链式存储结构保存  $S$ ，且要求平均查找长度更短，则元素应如何排列？应使用何种查找方法？查找成功时的平均查找长度是多少？

### 【答案解析】

1、【解析】(1) 采用顺序存储结构，数据元素按其查找概率降序排列。采用顺序查找方法。查找成功时的平均查找长度  $= 0.35 \times 1 + 0.35 \times 2 + 0.15 \times 3 + 0.15 \times 4 = 2.1$ 。

(2) 采用链式存储结构, 数据元素按其查找概率降序排列, 构成单链表。采用顺序查找方法。查找成功时的平均查找长度  $= 0.35 \times 1 + 0.35 \times 2 + 0.15 \times 3 + 0.15 \times 4 = 2.1$ 。

## 类型二十五：分块查找

1、【2025.07】已知查找表中有 400 个元素, 查找每个元素的概率相同, 采用分块查找法进行查找, 且均匀分布。若采用顺序查找法确定元素所在的块, 且块内也采用顺序查找法, 则查找成功时平均比较次数为:

- A. 8                      B. 10                      C. 20                      D. 25

### 【答案解析】

1、C。设每块包含  $n$  个元素, 则块数  $t = 400 / n$ 。

块查找: 采用顺序查找, 平均比较次数为  $(t + 1) / 2$ , 即  $(400/n + 1) / 2$ 。

块内查找: 采用顺序查找, 平均比较次数为  $(n + 1) / 2$ 。

因此, 总平均比较次数:

$$T(n) = (400/n + 1 + n + 1) / 2 = (400/n + n + 2) / 2 = 200/n + n/2 + 1$$

为使  $T(n)$  最小, 令导数  $T'(n) = -200/n^2 + 1/2 = 0$ ,  $n^2 = 400$ , 即  $n = 20$ 。

故当每块包含 20 个元素时, 查找效率最高。

## 类型二十六：折半树的构造和查找

1、【2010.09】已知一个长度为 16 的顺序表  $L$ , 其元素按关键字有序排列。若采用折半查找法查找一个  $L$  中不存在的元素, 则关键字的比较次数最多的是\_\_\_\_\_。

- A. 4                      B. 5                      C. 6                      D. 7

2、【2015.07】下列选项中, 不能构成折半查找中关键字比较序列的是\_\_\_\_\_。

- A. 500, 200, 450, 180                      B. 500, 450, 200, 180  
C. 180, 500, 200, 450                      D. 180, 200, 500, 450

3、【2016.09】在有  $n(n > 1000)$  个元素的升序数组  $A$  中查找关键字  $x$ 。查找算法的伪代码如下所示。  
 $k=0$ ;

while( $k < n$  且  $A[k] < x$ )

$k=k+3$ ;

if( $k < n$  且  $A[k-1] = x$ ) 查找成功;

else if( $k-1 < n$  且  $A[k-1] == x$ ) 查找成功;

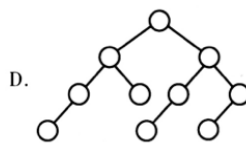
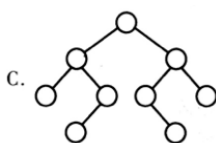
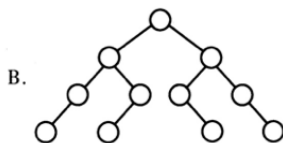
else if( $k-2 < n$  且  $A[k-2] == x$ ) 查找成功;

else 查找失败;

本算法与折半查找算法相比, 有可能具有更少比较次数的情形是\_\_\_\_\_。

- A. 当  $x$  不在数组中                      B. 当  $x$  接近数组开头处  
C. 当  $x$  接近数组结尾处                      D. 当  $x$  位于数组中间位置

4、【2017.08】下列二叉树中, 可能成为折半查找判定树(不含外部结点)的是\_\_\_\_\_。





- 5、【2023.08】对含 600 个元素的有序顺序表进行折半查找，关键字间的比较次数最多是 ( )  
A. 9                      B. 10                      C. 30                      D. 300
- 6、【2024.05】下列数据结构中，不适合直接使用折半查找的是  
I.有序链表；II.无序数组；III.有序静态链表；IV.无序静态链表。  
A.仅 I、III              B.仅 II、IV              C.仅 II、III、IV              D.I、II、III、IV

### 【答案解析】

1、B。折半查找法在查找不成功时和给定值进行关键字的比较次数最多为树的高度，即 $(\log_2 n)$ 向下取整+1，在本题中， $n=16$ ，故比较次数最多为 5。

2、A。180 不应在 200 的右边。

3、B。由于 k 是从 0 开始，如果 x 位于数组开头处，则可很快命中。折半需要从中间开始比较。

4、A。折半查找判定树实际上是一棵二叉排序树，它的中序序列是一个有序序列。可以在树结点上依次填上相应的元素，符合折半查找规则的树即是所求。B 选项 4、5 相加除 2 向上取整，7、8 相加除 2 向下取整，矛盾。C 选项，3、4 相加除 2 向上取整，6、7 相加除 2 向下取整，矛盾。D 选项，1、10 相加除 2 向下取整，6、7 相加除 2 向上取整，矛盾。A 符合折半查找规则，因此 A 正确。

5、B。若元素少可以作图推断，600 个元素作图比较复杂，因此优先利用公式法，若采用折半查找顺序表中不存在得元素，最大比较次数为对应二叉搜索树得高度，若有  $n$  个结点，二叉搜索树得高度为 $\lceil \log_2(n+1) \rceil$ ，将 600 带入得高度为 10。

6、D。折半查找(也称为二分查找)是一种在有序数组中查找特定元素的搜索算法。查找过程从组的中间元素开始，如果中间元素正是要查找的元素，则搜索过程结束；如果某一部分根据确定不包含该元素，则不需要再对其进行搜索，直接在另一部分进行查找。

①折半查找需要随机访问元素，因此它适用于数组和静态链表，但不适用于链表，因为链表不支持高效的随机访问。

②折半查找需要在有序的数据结构中进行，因此它不适用于无序的数组或链表。

所以，不适合直接使用折半查找的是：

I.有序链表：因为链表不支持高效的随机访问。

II.无序数组：折半查找需要在有序的数据结构中进行。

III.有序静态链表：虽然静态链表支持随机访问，但是在实际应用中，静态链表的使用并不广泛，因此在大多数情况下，不会在静态链表上使用折半查找。

IV.无序静态链表：因为折半查找需要在有序的数据结构中进行。

因此，正确答案是 D。

## 类型二十七：BST 的查找、插入、删除和判定

1、【2011.07】对于下列关键字序列，不可能构成某二叉排序树中一条查找路径的序列是\_\_\_\_\_。

- A. 95, 22, 91, 24, 94, 71                      B. 92, 20, 91, 34, 88, 35  
C. 21, 89, 77, 29, 36, 38                      D. 12, 25, 71, 68, 33, 34

2、【2013.6】在任意一棵非空二叉排序树  $T_1$  中，删除某结点  $v$  之后形成二叉排序树  $T_2$ ，再将  $v$  插入  $T_2$  形成二叉排序树  $T_3$ 。下列关于  $T_1$  与  $T_3$  的叙述中，正确的是\_\_\_\_\_。

I. 若  $v$  是  $T_1$  的叶结点，则  $T_1$  与  $T_3$  不同

II. 若  $v$  是  $T_1$  的叶结点, 则  $T_1$  与  $T_3$  相同

III. 若  $v$  不是  $T_1$  的叶结点, 则  $T_1$  与  $T_3$  不同

IV. 若  $v$  不是  $T_1$  的叶结点, 则  $T_1$  与  $T_3$  相同

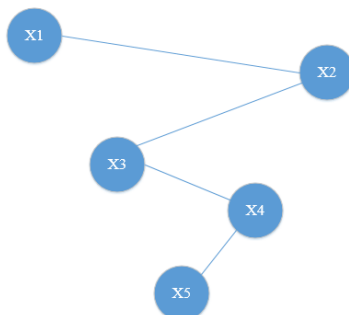
A. 仅 I、III

B. 仅 I、IV

C. 仅 II、III

D. 仅 II、IV

3、【2018.6】已知二叉排序树如下图所示, 元素间应满足的大小关系是:



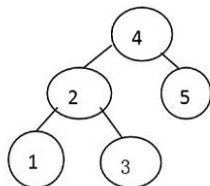
A.  $x_1 < x_2 < x_5$

B.  $x_1 < x_4 < x_5$

C.  $x_3 < x_5 < x_4$

D.  $x_4 < x_3 < x_5$

4、【2020.05】下列给定的关键字输入序列中, 不能生成如下二叉排序树的是:



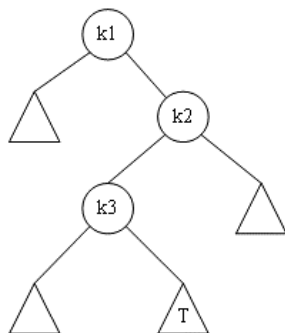
A. 4, 5, 2, 1, 3

B. 4, 5, 1, 2, 3

C. 4, 2, 5, 3, 1

D. 4, 2, 1, 3, 5

5、【2024.07】一棵二叉搜索树如图 7 图所示,  $k_1$ 、 $k_2$ 、 $k_3$  分别是对应结点中保存的关键字。子树  $T$  的任一结点中保存的关键字  $x$  满足的是



A.  $x < k_1$

B.  $x > k_2$

C.  $k_1 < x < k_3$

D.  $k_3 < x < k_2$

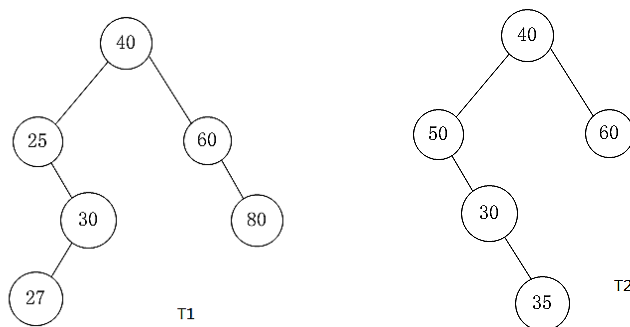
6、【2022.41】已知非空二叉树  $T$  的结点值均为正整数, 采用顺序存储方式保存, 数据结构定义如下:

```

typedef struct{
 int SqBiTNode[MAX_SIZE]; //MAX_SIZE 为已定义常量
 int ElemNum; //保存二叉树结点值的数组
 //实际占用的数组元素个数
} SqBiTree;

```

$T$  中不存在的结点在数组  $SqBiTNode$  中用 -1 来表示。例如, 对于下图所示的两棵非空二叉树  $T_1$  和  $T_2$ ,



T<sub>1</sub>的存储结果如下:

T<sub>1</sub>.SqBiTNode

|    |    |    |    |    |    |    |    |    |    |  |  |
|----|----|----|----|----|----|----|----|----|----|--|--|
| 40 | 25 | 60 | -1 | 30 | -1 | 80 | -1 | -1 | 27 |  |  |
|----|----|----|----|----|----|----|----|----|----|--|--|

T<sub>1</sub>.ElemNum=10

T<sub>2</sub>的存储结果如下:

T<sub>2</sub>.SqBiTNode

|    |    |    |    |    |    |    |    |    |    |    |  |  |
|----|----|----|----|----|----|----|----|----|----|----|--|--|
| 40 | 50 | 60 | -1 | 30 | -1 | -1 | -1 | -1 | -1 | 35 |  |  |
|----|----|----|----|----|----|----|----|----|----|----|--|--|

T<sub>2</sub>.ElemNum=11

请设计一个尽可能高效的算法,判定一棵采用这种方式存储的二叉树是否为二叉搜索树,若是,则返回 true,否则,返回 false。

要求:

- (1) 给出算法的基本设计思想
- (2) 根据设计思想,采用 C 或 C++ 语言描述算法,关键之处给出注释。

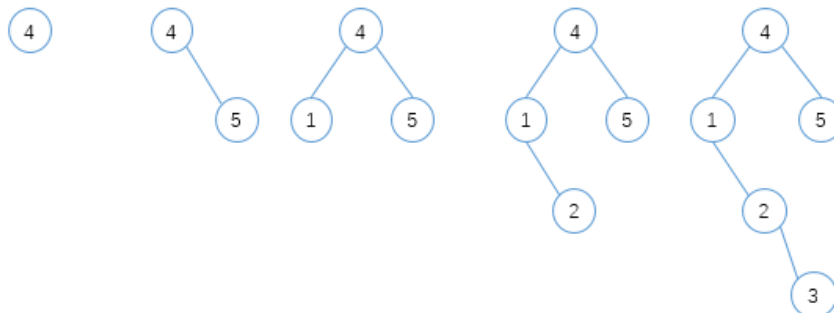
### 【答案解析】

1、A。选项 A 中,当查到 91 后再向 24 查找,说明这一条路径之后查找的数都要比 91 小,后面的 94 就错了。

2、C。在一棵二叉排序树中删除一个结点后再将此结点插入到二叉排序树中,如果删除的结点是叶子结点,那么在插入结点后,后来的二叉排序树与删除结点之前相同。如果删除的结点不是叶子结点,那么再插入这个结点后,后来的二叉树可能发生变化,不完全相同。

3、C。根据二叉排序树的特性:中序遍历 (LNR) 得到的是一个递增序列。图中二叉排序树的中序遍历序列为 X<sub>1</sub>, X<sub>3</sub>, X<sub>5</sub>, X<sub>4</sub>, X<sub>2</sub>, 可知 X<sub>3</sub> < X<sub>5</sub> < X<sub>4</sub>。

4、B。每个选项都逐一验证,选项 B 生成二叉排序树的过程如下:



5、D。二叉搜索树 (Binary Search Tree), 也被称为二叉查找树或二叉排序树, 是一种特殊的二叉树。它或者是一棵空树, 或者是具有以下性质的二叉树:

- ① 若它的左子树不空, 则左子树上所有结点的值均小于它的根结点的值;
- ② 若它的右子树不空, 则右子树上所有结点的值均大于它的根结点的值;

③它的左、右子树也分别为二叉搜索树。

二叉搜索树作为一种经典的数据结构,它既有链表的快速插入与删除操作的特点,又有数组快速查找的优势。因此,它在文件系统和数据库系统中被广泛应用,用于进行高效率的排序与检索操作。

X 在 K1 的右子树上,所以 X 大于 K1;

X 在 K2 的左子树上,所以 X 小于 K2;

X 在 K3 的右子树上,所以 X 大于 K3。

#### 6、(1) 算法的基本设计思想

对于采用顺序储存方法保存的二叉树,根节点在 SqBiTNode[0]中;当某节点保存在 SqBiTNode[1]中时,若有左孩子,则其值保存在 SqBiTNode[2i+1]中;若有右孩子,则其值保存在 SqBiTNode[2i+2]中;若有双亲结点,则其值保存在 SqBiTNode[(i-1)/2]中。

二叉搜索树需要满足的条件是:任意结点值大于其左子树中的全部结点值,小于其右子树中的全部结点值。中序遍历二叉搜索树得到一个升序序列。

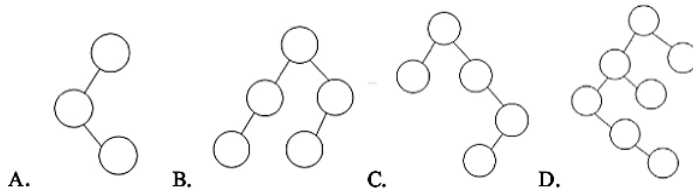
使用整形变量 val 记录中序遍历过程中已遍历结点的最大值,初值为一个负整数。若当前遍历的结点值小于等于 val,则算法返回 false,否则,将 val 的值更新为当前结点的值。

#### (2) 算法实现

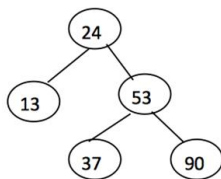
```
#define false 0
#define true 1
typedef int bool
bool judgeInOrderBST (SqBiTNode bt,int k,int* val) { //初始调用时 k 的值是
 if(k < bt.ElemNum && bt.SqBiTNode[k] != -1) {
 if(!judgeInOrderBST(bt, *k+1,val)) return false;
 if(bt.SqBiTNode[k] <= *val) return false;
 *val=bt.SqBiTNode[k]
 if(!judgeInOrderBST(bt, 2*k+2, val)) return false;
 }
 return true;
}
```

## 类型二十八: AVL 树的性质和调整

1、【2009.04】下列二叉排序树中,满足平衡二叉树定义的是\_\_\_\_\_。



2、【2010.04】在下图所示的平衡二叉树中,插入关键字 48 后得到一棵新平衡二叉树。在新平衡二叉树中,关键字 37 所在结点的左、右子结点中保存的关键字分别是\_\_\_\_\_。



A. 13, 48

B. 24, 48

C. 24, 53

D. 24, 90



3、【2012.04】若平衡二叉树的高度为 6, 且所有非叶结点的平衡因子均为 1, 则该平衡二叉树的结点总数为\_\_\_\_\_。

- A. 10                      B. 20                      C. 32                      D. 33

4、【2013.03】若将关键字 1, 2, 3, 4, 5, 6, 7 依次插入到初始为空的平衡二叉树 T 中, 则 T 中平衡因子为 0 的分支结点的个数是\_\_\_\_\_。

- A. 0                      B. 1                      C. 2                      D. 3

5、【2015.04】现有一棵无重复关键字的平衡二叉树 (AVL 树), 对其进行中序遍历可得到一个降序序列。下列关于该平衡二叉树的叙述中, 正确的是\_\_\_\_\_。

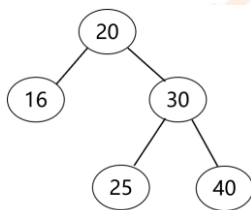
- A. 根结点的度一定为 2                      B. 树中最小元素一定是叶结点  
C. 最后插入的元素一定是叶结点                      D. 树中最大元素一定是无左子树

6、【2019.04】在任意一棵非空平衡二叉树(AVL 树)T1 中, 删除某结点 v 之后形成平衡二叉树 T2, 再将 v 插入 T2 形成平衡二叉树 T3。下列关于 T1 与 T3 的叙述中, 正确的是 ( )

- I.若 v 是 T1 的叶结点, 则 T1 与 T3 可能不相同  
II.若 v 不是 T1 的叶结点, 则 T1 与 T3 一定不相同  
III.若 v 不是 T1 的叶结点, 则 T1 与 T3 一定相同

- A.仅 I                      B.仅 II                      C.仅 I、II                      D.仅 I、III

7、【2021.06】给定平衡二叉树如下图所示, 插入关键字 23 后, 根中的关键字是



- A.16                      B.20                      C.23                      D.25

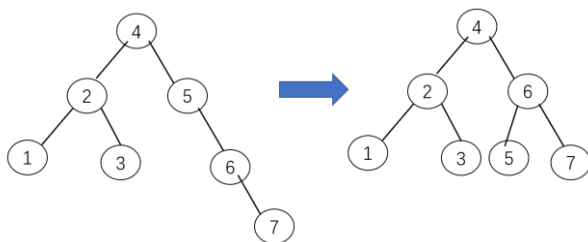
### 【答案解析】

1、B。根据平衡二叉树的定义有, 任意结点的左、右子树高度差的绝对值不超过 1。而其余 3 个选项均可以找到不符合该条件的结点。在做题过程中, 如果答案不太明显, 可以把每个非叶结点的平衡因子都写出来再进行判断。

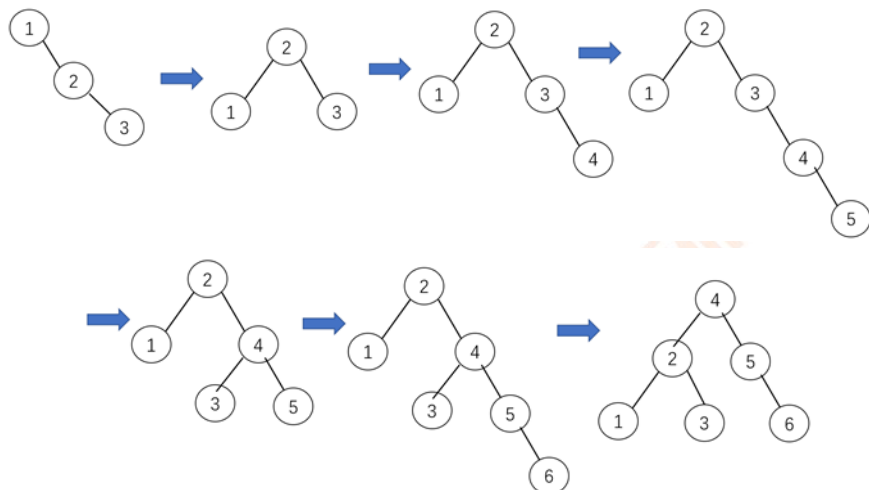
2、C。根据平衡二叉树的定义有, 任意结点的左、右子树高度差的绝对值不超过 1。而其余 3 个选项均可以找到不符合该条件的结点。在做题过程中, 如果答案不太明显, 可以把每个非叶结点的平衡因子都写出来再进行判断。插入 48 之后, 该二叉树根结点的平衡因子由 -1 变为 -2, 失去平衡, 需进行两次旋转 (先左旋后右旋) 操作。

3、B。所有非叶结点的平衡因子均为 1, 即平衡二叉树满足平衡的最少结点情况。对于高度为 N、左右子树的高度分别为 N-1 和 N-2、所有非叶结点的平衡因子均为 1 的平衡二叉树, 总结点数的公式为:  $C_N = C_{N-1} + C_{N-2} + 1$ ,  $C_1 = 1$ ,  $C_2 = 2$ ,  $C_3 = 2 + 1 + 1 = 4$ , 可推出  $C_6 = 20$ 。

4、D。利用 7 个关键字构建平衡二叉树 T, 平衡因子为 0 的分支结点个数为 3, 构建的平衡

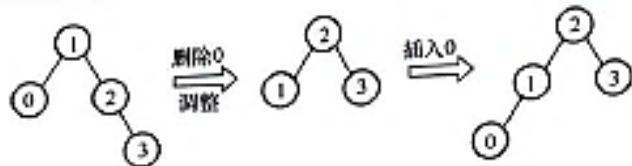


二叉树如下图所示。

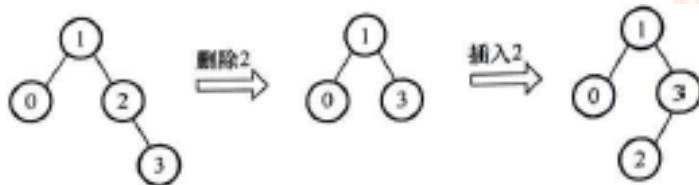


5、D。因对其进行中序遍历可得到一个降序序列，所以是左大右小。所以树中最大元素一定是无左子树。只有两个结点的平衡二叉树的根结点的度为 1，A 错误。中序遍历后可以得到一个降序序列，树中最大元素一定无左子树（可能有右子树），因此不一定是叶结点，B 错误。最后插入的结点可能会导致平衡调整，而不一定是叶结点，C 错误。

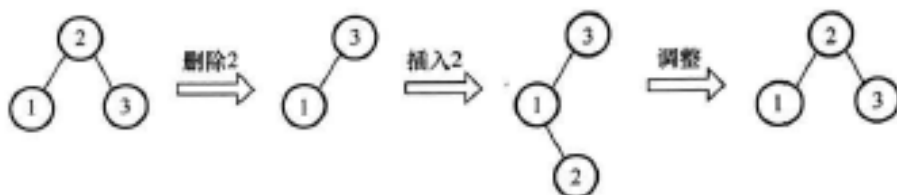
6、A。在非空平衡二叉树中插入结点，在失去平衡调整前，一定插入在叶结点的位置。若删除的是T1的叶结点，则删除后平衡二叉树不会失去平衡，即不会发生调整，再插入此结点得到的二叉平衡树T2与T3 相同；若删除后平衡二叉树失去平衡而发生调整，再插入结点得到的二叉平衡树T3与T1可能不同。I 正确。例如，如下图所示，删除结点0，平衡二叉树失衡调整，再插入结点0后，平衡二叉树和以前不同。对于比较绝对的说法II 和III，通常只需举出反例即可。



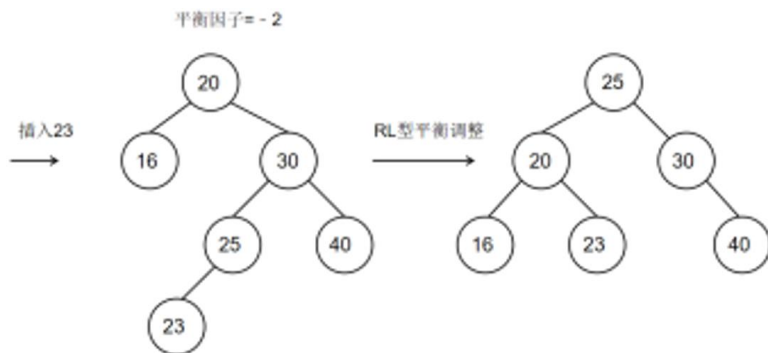
若删除的是 T1 的非叶结点，且删除和插入操作均没有导致平衡二叉树的调整（这时可以首先想到删除的结点只有一个孩子的情况），则该结点从非叶结点变成了叶结点，T1 与 T3 显然不同。例如，如下图所示，删除结点 2，用右孩子结点 3 填补，再插入结点 2，平衡二叉树和以前不同。



若删除的是 T1 的非叶结点，且删除和插入操作后导致了平衡二叉树的调整，则该结点有可能通过旋转后继续变成非叶结点，T1 与 T3 相同。例如，如下图所示，删除结点 2，用右孩子结点 3 填补，再插入结点 2，平衡二叉树失衡调整，调整后的平衡二叉树和以前相同。



7、D。将关键字 23 插入作为关键字 25 所在结点的左孩子后, 该二叉树失衡, 经过平衡调整后, 该平衡二叉树变为如下图所示结构。根节点中的关键字为 25。



## 类型二十九: B-树和 B+树的性质

- 【2009.08】下列叙述中, 不符合  $m$  阶 B 树定义要求的是\_\_\_\_\_。
  - 根节点最多有  $m$  棵子树
  - 所有叶结点都在同一层上
  - 各结点内关键字均升序或降序排列
  - 叶结点之间通过指针链接
- 【2013.10】在一株高度为 2 的 5 阶 B 树中, 所含关键字的个数最少是\_\_\_\_\_。
  - 5
  - 7
  - 8
  - 14
- 【2014.09】在一棵具有 15 个关键字的 4 阶 B 树中, 含关键字的结点个数最多是\_\_\_\_\_。
  - 5
  - 6
  - 10
  - 15
- 【2016.10】B+树不同于 B-树的特点之一是\_\_\_\_\_。
  - 能支持顺序查找
  - 结点中含有关键字
  - 根结点至少有两个分支
  - 所有叶结点都在同一层上
- 【2017.09】下列应用中, 适合使用 B+树的是\_\_\_\_\_。
  - 编译器中的词法分析
  - 关系数据库系统中的索引
  - 网络中的路由表快速查找
  - 操作系统的磁盘空闲块管理
- 【2018.08】高度为 5 的三阶 B 树含有的关键字个数至少是:
  - 15
  - 31
  - 62
  - 242
- 【2021.09】在一棵高度为 3 的 3 阶 B 树中, 根为第 1 层, 若第 2 层中有 4 个关键字, 则该树的结点个数最多是
  - 11
  - 10
  - 9
  - 8
- 【2023.07】下面关于非空 B 树的叙述中, 正确的是 ( )
  - 插入操作可能会增加树的高度;
  - 删除操作一定会导致叶节点的变化
  - 查找某关键字总是要查找到叶节点;
  - 插入的新关键字最终位于叶节点中
  - 仅 I
  - I、II
  - III、IV
  - I、II、IV
- 【2025.08】给定 7 个不同的关键字, 能够构造的不同 4 阶 B 树的个数最多是:
  - 7
  - 8
  - 9
  - 10

### 【答案解析】

1、D。选项 A、B 和 C 都是 B-树的特点, 而选项 D 则是 B+树的特点。注意区别 B-树和 B+树各自的特点。

2、A。一棵高度为 2 的 5 阶 B 树，根结点只有到达 5 个关键字的时候才能产生分裂，成为高度为 2 的 B 树。对于 5 阶 B 树，根结点只有达到 5 个关键字时才能产生分裂，成为高度为 2 的 B 树，因此 高度为 2 的 5 阶 B 树所含关键字的个数最少是 5。

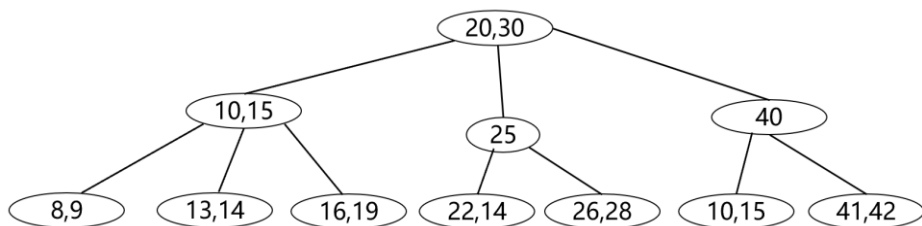
3、D。关键字数量不变，要求结点数量最多，那么即每个结点中含关键字的数量最少。根据 4 阶 B 树的定义，根结点最少含 1 个关键字，非根结点中至少含  $\lceil 4/2 \rceil - 1 = 1$  个关键字，所以每个结点中，关键字数量最少都为 1 个，即每个结点都有 2 个分支，类似与排序二叉树，而 15 个结点正好以构造一个 4 层的 4 阶 B 树，使得叶结点全在第四层，符合 B 树定义，因此选 D。

4、A。B+树的特点是将也结点按关键字的大小顺序排列，并且将相邻的叶结点按大小顺序相互链接起来，因此支持顺序查找。

5、B。B+树是应文件系统所需而产生的 B-树的变形，前者比后者更加适用于实际应用中的操作系统的文件索引和数据库索引，因为前者磁盘读写代价更低，查询效率更加稳定。编译器中的 词法分析使用有穷自动机和语法树。网络中的路由表快速查找主要靠高速缓存、路由表压缩技术和快速查找算法。系统一般使用空闲空间链表管理磁盘空闲块。所以 B 正确。

6、B。m 阶 B 树的基本性质：根结点以外的非叶结点最少含有  $\lceil m/2 \rceil - 1$  个关键字，代入  $m=3$  得到每个非叶结点中至少包含 1 个关键字，而根结点含有 1 个关键字，因此所有非叶结点都有两个孩子。此时其树形与  $h=5$  的满二叉树相同，可求得关键字最少为 31 个。

7、A。根据题意，可以构造如图所示的 2-3B 树，结点个数是 11 个。



8、B。插入关键字可能会导致上溢，上溢到根节点树的高度会增加。删除关键字一定会有叶节点发生变化。B 树所有结点均可以存储关键字，若查找得关键字位于非叶结点，则不可能查找到叶节点。插入新关键字，插入后进行调整，那么插入的关键字可能不在叶节点中，因此选 B。

9、C。7 个关键字构成 4 阶 B 树的可能情况，可分为两类：

① 高度为 2 的 B 树（根为内部结点，叶结点在同一层）：

设根中有  $r$  个关键字，则根有  $r+1$  个叶结点。叶结点中关键字总数为  $7-r$ 。

当  $r=1$  时，根有 2 个叶结点，要求两个叶结点各含 3 个关键字（因为每个叶结点最多 3 个关键字），唯一方案。

当  $r=2$  时，根有 3 个叶结点，需使叶结点关键字个数之和为 5，且每个叶结点至少 1 个、至多 3 个关键字。正好有 6 种正整数分解方式：(1,1,3)、(1,2,2)、(1,3,1)、(2,1,2)、(2,2,1)、(3,1,1)，故有 6 种方案。

当  $r=3$  时，根有 4 个叶结点，叶结点中关键字个数之和为 4，唯一方案即每个叶结点各 1 个关键字。

因此，高度为 2 的 B 树共有  $1+6+1=8$  种。

② 高度为 3 的 B 树：

必须为“最小”B 树，即各内部结点都取最小关键字个数。构造时：根取 1 个关键字（有 2 个叶结点），各内部结点取 1 个关键字（各有 2 个叶结点），叶结点均取 1 个关键字。

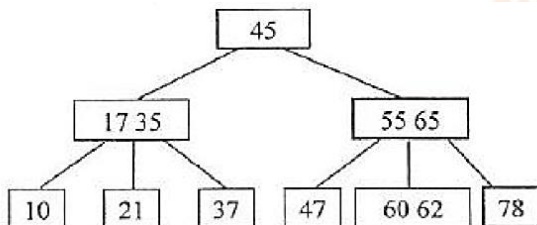
这样总关键字个数为：1（根）+ 2（第二层）+ 4（叶结点）= 7，方案唯一。



总计不同 B 树个数 =  $8 + 1 = 9$ 。

### 类型三十：B-树的插入和删除

1、【2012.09】设有一棵 3 阶 B 树，如下图所示。删除关键字 78 得到一棵新 B 树，其最右叶结点所含的关键字是\_\_\_\_\_。

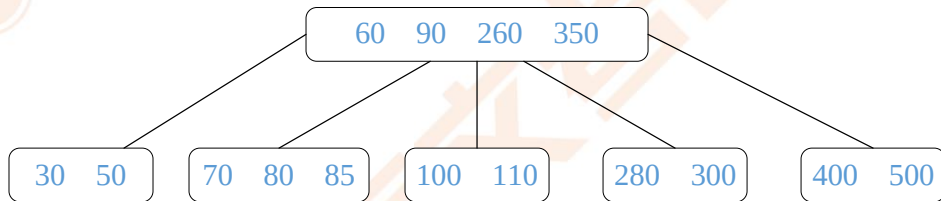


- A. 60                      B. 60, 62                      C. 62, 65                      D. 65

2、【2020.10】堆中的次大值一定在根的下一层。依次将关键字 5, 6, 9, 13, 8, 2, 12, 15 插入初始为空的 4 阶 B 树后，根节点中包含的关键字是：

- A、8                      B、6, 9                      C、8, 13                      D、9, 12

3、【2022.08】在下图所示的 5 阶 B 树 T 中，删除关键字 260 之后需要进行必要的调整，得到新的 B 树 T1。下列选项中，不可能是 T1 根结点中关键字字序列的是（ ）。

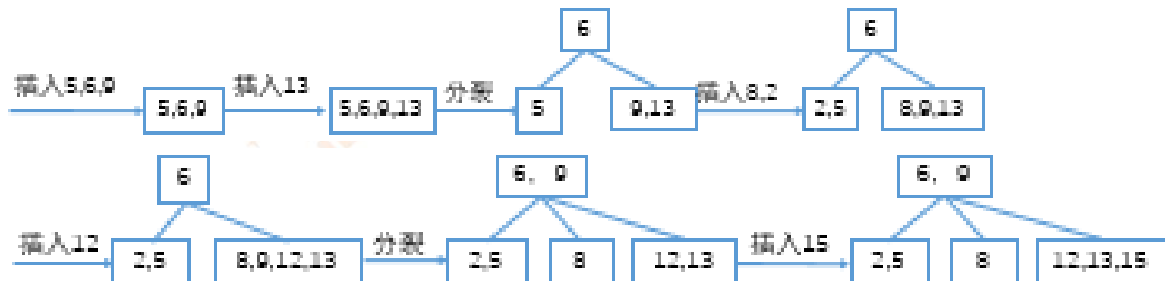


- A. 60, 90, 280                      B. 60, 90, 350                      C. 60, 85, 110, 350                      D. 60, 90, 110, 350

#### 【答案解析】

1、D。对于图中所示的 3 阶 B 树，被删关键字 78 所在结点在删除前的关键字个数 =  $3/2$  (向上取整) - 1，且其左兄弟结点的关键字个数 =  $2 \geq 3/2$  (向上取整)，属于“兄弟够借”的情况，则需把该结点的左兄弟结点中最大的关键字上移到双亲结点中，同时把双亲结点中大于上移关键字的关键字下移到要删除关键字的结点中，这样就达到了新的平衡。

2、B。解析：一个 4 阶 B 树的任意非叶结点至多含有  $m-1=3$  个关键字，在关键字依次插入的过程中，会导致结点的不断分裂，插入过程如下所示。



得到根节点包含的关键字为 6, 9。

3、D。本题考察不可能，删除非叶子结点中的关键字：如果删除结点不在叶子结点中，设关键字为 N，我们就用 N 的子孙节点的关键字代替 N，且用最接近 N 的关键字来代替 N，具体说有

两种情况:

(1)用 N 的最右边的子树代替 N, 那么是最右边子树的最大的关键字, 设为 K;

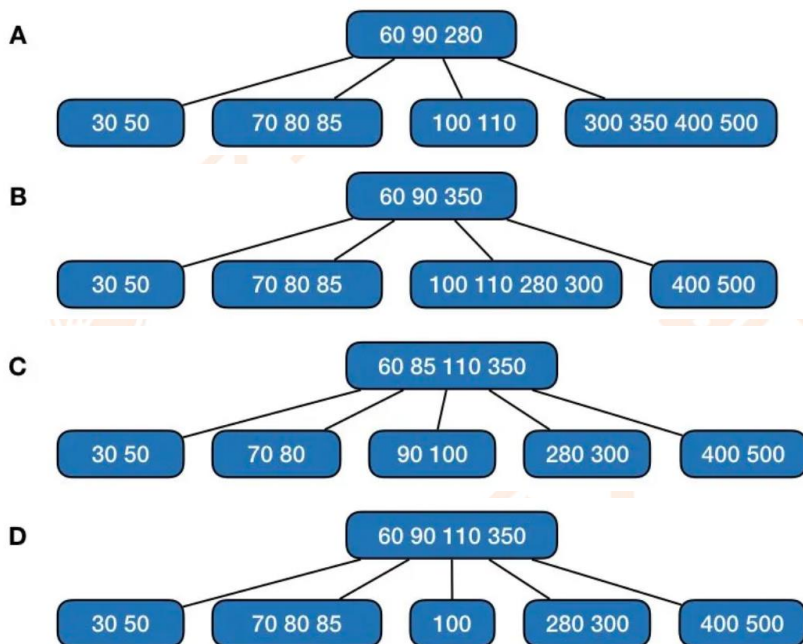
(2)用 N 的最左边的子树代替 N, 那么是最左边子树的最小关键字, 设为 K;

分析 A 选项, 用 280 代替 260, 那么 300 所在的结点中的关键字就不够了, 将 350 放下去, 构成如图 A 的情形。

分析 B 选项, 不用孩子代替 260, 那将 100, 110, 280, 300 合在一起, 构成如图 B 的情形。

分析 C 选项, 用 110 代替 260, 那此时 100 所在的结点中的关键字就不够了, 让 100 去借他的做兄弟, 将 90 拉下来, 将 85 放到根中, 构成如图 C 的情形。

分析 D 选项, 用 110 代替 260, 那此时 100 所在的结点中的关键字就不够了, 构成如图 D 的情形。



### 类型三十一：哈希函数构造和冲突解决方法

1、【2011.09】为提高散列 (Hash) 表的查找效率, 可以采取的正确措施是\_\_\_\_\_。

I. 增大装填 (载) 因子; II. 设计冲突 (碰撞) 少的散列函数

III. 处理冲突 (碰撞) 时避免产生聚集 (堆积) 现象

A. 仅I                      B. 仅II                      C. 仅I、II                      D. 仅II、III

2、【2014.08】用哈希 (散列) 方法处理冲突 (碰撞) 时可能出现堆积 (聚集) 现象, 下列选项中, 会受堆积现象直接影响的是\_\_\_\_\_。

A. 存储效率                      B. 散列函数                      C. 装填 (装载) 因子                      D. 平均查找长度

3、【2018.09】现有长度为 7、初始为空的散列表 HT, 散列函数  $H(k) = k \% 7$ , 用线性探测再散列法解决冲突。将关键字 22, 43, 15 依次插入到 HT 后, 查找成功的平均查找长度是

A. 1.5                      B. 1.6                      C. 2                      D. 3

4、【2019.08】现有长度为 11 且初始为空的散列表 HT, 散列函数是  $H(\text{key}) = \text{key} \% 11$ , 采用线性探查 (线性探测再散列) 法解决冲突。将关键字序列 87, 40, 30, 6, 11, 22, 98, 20 依次插入到 HT 后, HT 查找失败的平均查找长度是

A. 4                      B. 5.25                      C. 6                      D. 6.29

5、【2022.09】下列因素中,影响散列(哈希)方法平均查找长度的是( )

I、装填因子; II、散列函数; III、冲突解决方法

A. 仅 I、II B. 仅 I、III C. 仅 II、III D. I、II、III

6、【2023.09】现有长度为 5、初始为空的散列表 HT, 散列函数  $H(k) = (k+4) \% 5$ , 用线性探查再散列法解决冲突。若将关键字序列 2022, 12, 25 依次插入 HT 中, 然后删除关键字 25, 则 HT 中查找失败的平均查找长度为( )

A. 1 B. 1.6 C. 1.8 D. 2.2

7、【2025.09】下列关于散列方法处理冲突的叙述中, 正确的是:

- A. 只要散列表不满, 线性探查再散列一定能找到一个空闲位置。
- B. 只要散列表不满, 二次探查再散列一定能找到一个空闲位置。
- C. 线性探查再散列处理的冲突, 一定是发生在同义词之间的冲突。
- D. 二次探查再散列处理的冲突, 一定是发生在非同义词之间的冲突。

8、【2010.41】将关键字序列 (7,8,30,11,18,9,14) 散列存储到散列表中。散列表的存储空间是一个下标从 0 开始的一维数组, 散列函数为:  $H(\text{key}) = (\text{key} * 3) \text{MOD } 7$ , 处理冲突采用线性探测再散列法, 要求装填(载)因子为 0.7。

(1) 请画出所构造的散列表。

(2) 分别计算等概率情况下查找成功和查找不成功的平均查找长度。

9、【2024.12】将关键字序列 20, 3, 11, 18, 9, 14, 7 依次存储到初始值为空、长度为 11 的散列表 HT 中, 散列函数  $H(\text{key}) = (\text{key} * 3) \% 11$ 。  $H(\text{key})$  计算机出初始散列地址为  $H_0$ , 发生冲突时探查地址序列是  $H_1, H_2, H_3, \dots$ , 其中,  $H_k = (H_0 + k^2) \% 11, k=1, 2, 3, \dots$ 。

请回答下列问题。

(1) 画出所构造的 HT, 并计算 HT 的装填因子 (6 分)

(2) 给出在 HT 中查找关键字 14 的关键字比较序列 (2 分)

(3) 在 HT 中查找关键字 8, 确认查找失败时的散列地址是多少? (2 分)

### 【答案解析】

1、D。Hash 表的查找效率取决于散列函数、处理冲突的方法和装填因子。显然, 冲突的产生概率与装填因子(表中记录数与表长之比)的大小成正比, 即装填得越满越容易发生冲突, I 错误。II 显然正确。采用合适的处理冲突的方式避免产生聚集现象, 也将提高查找效率, 例如用拉链法解决冲突时就不存在聚集现象, 用线性探测法解决冲突时易引起聚集现象, III 正确。

2、D。产生堆积现象, 即产生了冲突, 它对存储效率、散列函数和装填因子均不会有影响, 而平均查找长度会因为堆积现象而增大。

3、C。根据题意, 得到的 HT 如下:

|   |    |    |    |   |   |   |
|---|----|----|----|---|---|---|
| 0 | 1  | 2  | 3  | 4 | 5 | 6 |
|   | 22 | 43 | 15 |   |   |   |

因此,  $ASL_{\text{成功}} = (1 + 2 + 3) / 3 = 2$ 。

4、C。采用线性探查法计算每个关键字的存放情况, 如下表所示。

|      |    |    |    |    |    |    |   |    |   |   |    |
|------|----|----|----|----|----|----|---|----|---|---|----|
| 散列地址 | 0  | 1  | 2  | 3  | 4  | 5  | 6 | 7  | 8 | 9 | 10 |
| 关键字  | 98 | 22 | 30 | 87 | 11 | 40 | 6 | 20 |   |   |    |

由于  $H(\text{key}) = 0 \sim 6$ , 查找失败时可能对应的地址有 7 个, 对于计算出地址为 0 的关键字 Key0, 只

有比较完 0~8 号地址后才能确定该关键字不在表中, 比较次数为 9; 对于计算出地址为 1 的关键字 key1, 只有比较完 1~8 号地址后才能确定该关键字不在表中, 比较次数为 8; 以此类推。需要特别注意的是, 散列函数不可能计算出地址 7, 因此有

ASL 失败 =  $(9 + 8 + 7 + 6 + 5 + 4 + 3) / 7 = 6$ 。

5、D。属于记忆性的概念题, 装填因子、散列函数、冲突解决方法都会影响平均查找长度。

6、C。根据关键字,

$H(2022) = (2022 + 4) \% 5 = 1$

$H(12) = (12 + 4) \% 5 = 1$

$H(5) = (25 + 4) \% 5 = 4$

得到散列表为

| 地址   | 0 | 1    | 2  | 3 | 4  |
|------|---|------|----|---|----|
| 元素   |   | 2022 | 12 |   | 25 |
| 比较次数 |   | 1    | 2  |   | 1  |

当删除 25 时, 开放定址法不能做物理删除, 只能逻辑删除, 即 25 位置标记为删除, 这是因为同一个故列位置可能是同义词, 也可能不是同义词, 则  $ASL_{\text{失败}} = (1 + 3 + 2 + 1 + 2) / 5 = 1.8$  次

7、A。线性探查的探查序列为:

$h(k), [h(k)+1] \bmod m, [h(k)+2] \bmod m$  只要散列表不满, 就会遍历整个表, 必能找到一个空闲位置。A 正确。

二次探查的探查序列为:

$h(k), [h(k)+1^2] \bmod m, [h(k)+2^2] \bmod m$

不保证能遍历所有位置, 即使散列表不满, 也可能找不到空闲位置。B 错误。

线性在探测过程中, 冲突既可能发生在原本哈希地址相同的同义词之间, 也可能在探测过程中产生, 因此不能说冲突一定发生在同义词之间。C 错误。

二次探查处理的冲突并非一定发生在非同义词之间, 若两个关键字初始哈希值相同, 二次探查也会处理它们之间的冲突。D 错误。

8、(1) 由装载因子 0.7, 数据总数为 7, 得一维数组大小为  $7/0.7=10$ , 数组下标为 0~9。所构造的散列函数值如下所示:

| key    | 7 | 8 | 30 | 11 | 18 | 9 | 14 |
|--------|---|---|----|----|----|---|----|
| H(key) | 0 | 3 | 6  | 5  | 5  | 6 | 0  |

采用线性探测再散列法处理冲突, 所构造的散列表为:

| 地址  | 0 | 1  | 2 | 3 | 4 | 5  | 6  | 7  | 8 | 9 |
|-----|---|----|---|---|---|----|----|----|---|---|
| 关键字 | 7 | 14 |   | 8 |   | 11 | 30 | 18 | 9 |   |

(2) 查找成功时, 是根据每个元素查找次数来计算平均长度, 在等概率的情况下, 各关键字的查找次数为:

| key | 7 | 8 | 30 | 11 | 18 | 9 | 14 |
|-----|---|---|----|----|----|---|----|
| 次数  | 1 | 1 | 1  | 1  | 3  | 3 | 2  |

故,  $ASL_{\text{成功}} = \text{查找次数} / \text{元素个数} = (1+2+1+1+1+3+3)/7 = 12/7$

这里要特别防止惯性思维。查找失败时, 是根据查找失败位置计算平均次数, 根据散列函数 MOD 7, 初始只可能在 0~6 的位置。等概率情况下, 查找 0~6 位置查找失败的查找次数为:

| H(key) | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|--------|---|---|---|---|---|---|---|
| 次数     | 3 | 2 | 1 | 2 | 1 | 5 | 4 |

9、(1) 对于关键字 20, 计算  $(20 \times 3) \% 11 = 60 \% 11 = 5$ 。所以 20 的散列地址是 5。



对于关键字 3, 计算  $(3 \times 3) \% 11 = 9 \% 11 = 9$ 。所以 3 的散列地址是 9。

对于关键字 11, 计算  $(11 \times 3) \% 11 = 33 \% 11 = 0$ 。所以 11 的散列地址是 0。

对于关键字 18, 计算  $(18 \times 3) \% 11 = 54 \% 11 = 10$ 。所以 18 的散列地址是 10。

对于关键字 9, 计算  $(9 \times 3) \% 11 = 27 \% 11 = 5$ 。但是地址 5 已经被关键字 20 占用, 所以需要使用二次探查来找到一个新的地址。计算  $(5 + 12) \% 11 = 6 \% 11 = 6$ 。所以 9 的散列地址是 6。

对于关键字 14, 计算  $(14 \times 3) \% 11 = 42 \% 11 = 9$ 。但是地址 9 已经被关键字 3 占用, 所以需要使用二次探查来找到一个新的地址。计算  $(9 + 12) \% 11 = 10 \% 11 = 10$ 。但是地址 10 也被 8 占用了, 再次使用二次探查来找到一个新的地址。计算  $(9 + 2^2) \% 11 = 13 \% 11 = 2$ 。所以 14 的散列地址是 2。

对于关键字 7, 计算  $(7 \times 3) \% 11 = 21 \% 11 = 10$ 。但是地址 10 已经被关键字 8 占用, 所以需要使用二次探查来找到一个新的地址。计算  $(10 + 1^2) \% 11 = 11 \% 11 = 0$ 。但是, 地址 0 已经被关键字 1 占用, 所以需要继续使用二次探查。计算  $(10 + 2^2) \% 11 = 14 \% 11 = 3$ 。所以 7 的散列地址是 3。

| 散列地址 | 0  | 1 | 2  | 3 | 4 | 5  | 6 | 7 | 8 | 9 | 10 |
|------|----|---|----|---|---|----|---|---|---|---|----|
| 关键字  | 11 |   | 14 | 7 |   | 20 | 9 |   |   | 3 | 18 |
| 冲突次数 | 1  |   | 3  | 3 |   | 1  | 2 |   |   | 1 | 1  |

最后, 计算散列表的填充因子, 即散列表中已经被填充的位置的数量除以散列表的总长度。在这个例子中, 填充因子是  $7/11$ 。

(2) ①首先, 计算 14 的散列地址:  $H(14) = ((14 \times 3) \% 11 = 42 \% 11 = 9$ 。

②查看散列表中索引为 9 的位置, 发现那里存储的是关键字 3, 而不是 14。这意味着遇到了冲突。

③由于使用的是二次探查, 所以计算下一个散列地址:  $H_1 = (H_0 + 1^2) \% 11 = (9 + 1) \% 11 = 10$ 。发现那里存储的是关键字 18, 而不是 14。这意味着又遇到了冲突。

④所以继续计算下一个散列地址:  $H_2 = (H_0 + 2^2) \% 11 = (9 + 4) \% 11 = 2$ 。发现那里存储的正是要找的关键字 14。

(3) ①首先, 计算 8 的散列地址:  $H(8) = (8 \times 3) \% 11 = 24 \% 11 = 2$ 。

②查看散列表中索引为 2 的位置, 发现那里存储的是关键字 18, 而不是 8。这意味着遇到了冲突。

③由于使用的是二次探查, 所以计算下一个散列地址:  $H_1 = (H_0 + 1^2) \% 11 = (2 + 1) \% 11 = 3$ 。

④查看散列表中索引为 3 的位置, 发现那里存储的是关键字 7, 而不是 8。这意味着又遇到了冲突。

⑤继续使用二次探查, 计算下一个散列地址:  $H_2 = (H_0 + 2^2) \% 11 = (2 + 4) \% 11 = 6$ 。

⑥查看散列表中索引为 6 的位置, 发现那里存储的是关键字 9, 而不是 8。这意味着又遇到了冲突。

⑦继续使用二次探查, 计算下一个散列地址:  $H_3 = (H_0 + 3^2) \% 11 = (2 + 9) \% 11 = 0$ 。

⑧查看散列表中索引为 0 的位置, 发现那里存储的是关键字 11, 而不是 8。这意味着又遇到了冲突。

⑨继续使用二次探查, 计算下一个散列地址:  $H_4 = (H_0 + 4^2) \% 11 = (2 + 16) \% 11 = 7$ 。

⑩我们查看散列表中索引为 7 的位置, 发现那里是空的, 所以可以确认查找失败, 散列地址是 7。

## 类型三十二: 串与 KMP 算法

1、【2015.08】已知字符串 S 为“abaabaabacacaabaabcc”, 模式串 t 为“abaabc”。采用 KMP 算法进行匹配, 第一次出现“失配”(s[i]≠t[j]) 时, i=j=5, 则下次开始匹配时, i 和 j 的值分别是\_\_\_\_\_。

A. i=1, j=0      B. i=5, j=0      C. i=5, j=2      D. i=6, j=2

2、【2019.09】设主串 T=“abaabaabacabaabc”, 模式串 S=“abaabc”, 采用 KMP 算法进行模式匹配, 到匹配成功时为止, 在匹配过程中进行的单个字符间的比较次数足

A. 9      B. 10      C. 12      D. 15

3、【2024.06】 KMP 算法使用修正后的 next 数组进行模式匹配, 模式串  $S=aabaab$ , 当主串中某字符与  $S$  中某字符失配时,  $S$  将向右滑动的最长距离是

- A.5                      B.4                      C.3                      D.2

【答案解析】

1、C。由题中“失配  $s[i] \neq t[j]$  时,  $i=j=5$ ”, 可知题中的主串和模式串的位序都是从 0 开始的(意灵活应变)。按照 next 数组生成算法, 对于  $t$  有:

| 编号   | 0  | 1 | 2 | 3 | 4 | 5 |
|------|----|---|---|---|---|---|
| t    | A  | b | a | a | b | C |
| Next | -1 | 0 | 0 | 1 | 1 | 2 |

依据 KMP 算法“当失配时,  $i$  不变,  $j$  回退到  $next[j]$  的位置并重新比较”, 当失配  $s[i] \neq t[j]$  时,  $i=j=5$ , 由上表不难得出  $next[j]=next[5]=2$ (位序从 0 开始)。从而最后结果应为:  $i=5$ ( $i$  保持不变),  $j=2$ 。

2、B。假设位序都是从 0 开始的, 按照 next 数组生成算法, 对于  $S$  有

| 编号   | 0  | 1 | 2 | 3 | 4 | 5 |
|------|----|---|---|---|---|---|
| S    | a  | b | a | a | b | c |
| Next | -1 | 0 | 0 | 1 | 1 | 2 |

根据 KMP 算法, 第一趟连续比较 6 次, 在模式串的 5 号位和主串的 5 号位匹配失败, 模式串的下一个比较位置为  $next[5]$ , 即下一次比较从模式串的 2 号位和主串的 5 号位开始, 然后直到模式串 5 号位和主串 8 号位匹配, 第二趟比较 4 次, 模式串匹配成功。单个字符的比较次数为 10 次, 所以选 B。

3、A。假设有一个主串  $T$ , 在  $T$  中查找模式串  $S$ , 从  $T$  的第一个字符和  $S$  的第一个字符开始比较:

如果  $T$  的当前字符和  $S$  的当前字符相同, 那么我们就将两者的索引都向右移动一位, 继续比较下一个字符;

如果  $T$  的当前字符和  $S$  的当前字符不同, 那么我们就查看  $next$  在  $S$  当前字符位置前一个位置的值(记作  $k$ )。然后将  $S$  向右滑动  $k$  个位置。这样,  $S$  的前  $k$  个字符(即最长的相同前缀)就会与  $T$  的当前字符对齐。

重复步骤 1 和步骤 2, 直到  $S$  的所有字符与  $T$  的某个子串完全匹配, 或者  $T$  的所有字符都被扫描完。

计算 nextval 数组: nextval 数组是对 next 数组的优化。当模式串中的第  $i$  位与主串失配时, 如果模式串中的第  $next[i]$  位与第  $i$  位相同, 那么显然在下一轮匹配中, 仍然会失配。因此, 我们需要再次将模式串向右滑动一个单位, 也就是说让  $j=next[next[i]]$ ...直到模式串中的第  $next[j]$  位与第  $j$  位不同为止。对于模式串  $S: "aabaab"$ , 我们可以根据其 next 数组计算出 nextval 数组:

$nextval[0]=next[0]=-1$ , 因为  $S[0]$  之前没有字符

| 下标:      | 0  | 1 | 2 | 3 | 4 | 5 |
|----------|----|---|---|---|---|---|
| 数据:      | a  | a | b | a | a | b |
| Next=    | -1 | 0 | 1 | 0 | 1 | 2 |
| Nextval= | -1 |   |   |   |   |   |

$next[1]=0$ , 且位置 0 为 a, 位置 1 也为 a, 所以  $nextval[1]=nextval[0]=-1$ 。

| 下标:      | 0  | 1  | 2 | 3 | 4 | 5 |
|----------|----|----|---|---|---|---|
| 数据:      | a  | a  | b | a | a | b |
| Next=    | -1 | 0  | 1 | 0 | 1 | 2 |
| Nextval= | -1 | -1 |   |   |   |   |

$next[2]=1$ , 但是位置 1 为 a, 位置 2 为 b, 二者不等, 所以  $nextval[2]=nextval[2]=1$ 。

|          |    |    |   |   |   |   |
|----------|----|----|---|---|---|---|
| 下标:      | 0  | 1  | 2 | 3 | 4 | 5 |
| 数据:      | a  | a  | b | a | a | b |
| Next=    | -1 | 0  | 1 | 0 | 1 | 2 |
| Nextval= | -1 | -1 | 1 |   |   |   |

next[3]=0, 位置 0 为 a, 位置 3 也为 a, 所以 nextval[3]=nextval[0]=-1。

|          |    |    |   |    |   |   |
|----------|----|----|---|----|---|---|
| 下标:      | 0  | 1  | 2 | 3  | 4 | 5 |
| 数据:      | a  | a  | b | a  | a | b |
| Next=    | -1 | 0  | 1 | 0  | 1 | 2 |
| Nextval= | -1 | -1 | 1 | -1 |   |   |

next[4]=0, 位置 0 为 a, 位置 4 也为 a, 所以 nextval[4]=nextval[0]=-1。

|          |    |    |   |    |    |   |
|----------|----|----|---|----|----|---|
| 下标:      | 0  | 1  | 2 | 3  | 4  | 5 |
| 数据:      | a  | a  | b | a  | a  | b |
| Next=    | -1 | 0  | 1 | 0  | 1  | 2 |
| Nextval= | -1 | -1 | 1 | -1 | -1 |   |

next[5]=2, 位置 2 为 b, 位置 5 也为 b, 所以 nextval[5]=nextval[2]=1。

|          |    |    |   |    |    |   |
|----------|----|----|---|----|----|---|
| 下标:      | 0  | 1  | 2 | 3  | 4  | 5 |
| 数据:      | a  | a  | b | a  | a  | b |
| Next=    | -1 | 0  | 1 | 0  | 1  | 2 |
| Nextval= | -1 | -1 | 1 | -1 | -1 | 1 |
| 滑动距离     | 1  | 2  | 1 | 4  | 5  | 4 |

滑动距离=下标-netval, 所以 S 将向右滑动的最长距离是 5。

### 类型三十三：查找算法

1、【2022.42】现有  $n$  ( $n > 100\,000$ ) 个数保存在一维数组  $M$  中, 需要查找  $M$  中最小的 10 个数。请回答下列问题。

- (1) 设计一个完成上述查找任务的算法, 尽可能查找次数最小, 给出算法思想 (不用写出具体的程序)
- (2) 写出所给算法的时间复杂度与空间复杂度

#### 【答案解析】

- 1、(1) 定义含 10 个元素的大根堆  $H$ , 元素值均为该堆元素类型能表示的最大数  $MAX$ 。  
For  $M$  中的每个元素  $S$   
If ( $s < H$  的堆顶元素) 删除堆顶元素并将  $s$  插入  $H$  中; 当数据全部扫描完毕, 堆  $H$  中保存的就是最小的 10 个数。
- (2) 算法平均情况下的时间复杂度是  $O(n)$ , 空间复杂度是  $O(1)$ 。

### 类型三十四：插入类排序

1、【2012.11】对一待排序序列分别进行折半插入排序和直接插入排序, 两者之间可能的不同之处是\_\_\_\_\_。

- A. 排序的总趟数
- B. 元素的移动次数
- C. 使用辅助空间的数量
- D. 元素之间的比较次数

2、【2014.10】用希尔排序方法对一个数据序列进行排序时, 若第 1 趟排序结果为 9, 1, 4, 13, 7, 8, 20, 23, 15, 则该趟排序采用的增量 (间隔) 可能是\_\_\_\_\_。

- A. 2                      B. 3                      C. 4                      D. 5

3、【2015.11】希尔排序的组内排序采用的是\_\_\_\_\_。

- A. 直接插入排序      B. 折半插入排序      C. 快速排序      D. 归并排序

4、【2018.10】对初始数据序列(8,3,9,11,2,1,4,7,5,10,6)进行希尔排序。若第一趟排序结果为(1,3,7,5,2,6,4,9,11,10,8),第二趟排序结果为(1,2,6,4,3,7,5,8,11,10,9),则两趟排序采用的增量(间隔)依次是:

- A.3, 1                      B.3, 2                      C.5, 2,                      D.5, 3

5、【2022.11】对数据进行排序时,若采用直接插入排序而不采用快速排序,则可能的原因是( )

I.大部分元素已有序; II.待排序元素数量很少

III.要求空间复杂度为  $O(1)$

IV.要求排序算法是稳定的

- A. 仅I、II                      B. 仅II、IV                      C. 仅I、II、IV                      D. I、II、III、IV

6、【2025.11】对含9个关键字的初始序列进行排序,若序列的变化情况如下图表所示,则下列排序算法中,采用的是:

|           |                                       |
|-----------|---------------------------------------|
| 初试序列      | 5, 25, 40, 30, 10, 20, 45, 15, 35, 35 |
| 第1趟排序后的序列 | 5, 25, 40, 30, 10, 20, 45, 15, 35, 40 |
| 第2趟排序后的序列 | 5, 10, 15, 25, 20, 30, 40, 35, 45, 40 |

- A. 希尔排序                      B. 基数排序                      C. 归并排序                      D. 折半插入排序

### 【答案解析】

1、D。折半插入排序与直接插入排序都是将待插入元素插入前面的有序子表,区别是:确定当前记录在前面有序子表中的位置时,直接插入排序是采用顺序查找法,而折半插入排序是采用折半查找法。排序的总趟数取决于元素个数  $n$ ,两者都是  $n-1$  趟。元素的移动次数都取决于初试序列,两者相同。使用辅助空间的数量也都是  $O(1)$ 。折半插入排序的比较次数与序列初态无关,为  $O(n\log 2n)$ ;直接插入排序的比较次数与序列初态有关,为  $O(n) \sim O(n^2)$ 。

2、B。首先,第二个元素为1,是整个序列中的最小元素,所以可知该希尔排序为从小到大排序。然后考虑增量问题,若增量为2,第1+2个元素4明显比第1个元素9要大,A排除;若增量为3,第八  $i+3$ 、 $i+6$  个元素都为有序序列( $i=1, 2, 3$ ),符合希尔排序的定义;若增量为4,第1个元素9比第1+4个元素7要大,C排除;若增量为5,第1个元素9比第1+5个元素8要大,D排除,选B。

3、A。希尔排序的思想是:先将待排元素序列分割成若干子序列(由相隔某个“增量”的元素组成),分别进行直接插入排序,然后依次缩减增量再进行排序,待整个序列中的元素基本有序(增量足够小)时,再对全体元素进行一次直接插入排序。

4、D。初始序列: 8, 3, 9, 11, 2, 1, 4, 7, 5, 10, 6  
第一趟分组: 8, 1, 6; 3, 4; 9, 7; 11, 5; 2, 10; 间隔为5, 排序后组内递增。  
第二趟分组: 1, 5, 4, 10; 3, 2, 9, 8; 7, 6, 11; 间隔为3, 排序后组内递增。  
因此答案为D。

5、D。为了对本题进行分析,我们用表格总结直接插入排序和快速排序的核心要点,通过表格的分析,四个原因都是正确原因。

|      | 直接插入排序                                     | 快速排序                                                         |
|------|--------------------------------------------|--------------------------------------------------------------|
| 算法思想 | 其基本操作是将一条记录插入到已排好的有序表中,从而得到一个新的、记录数量增1的有序表 | 快速排序在每一轮挑选一个基准元素,并让其他比它大的元素移动到数列一边,比它小的元素移动到数列的另一边,从而把数列拆解成了 |



|       |             |               |
|-------|-------------|---------------|
|       |             | 两个部分。         |
| 时间复杂度 | $O(n^2)$    | $O(n \log n)$ |
| 空间复杂度 | $O(1)$      | $O(\log n)$   |
| 稳定性   | 稳定的算法       | 不稳定的算法        |
| 存储需求  | 连续或者链式均可    | 顺序结构          |
| 其他    | 元素越有序, 算法越快 | 元素越有序, 算法越慢   |

6、A. 初始序列: 5, 25, 40, 30, 10, 20, 45, 15, 35。

第 1 趟排序后的序列: 5, 10, 20, 30, 15, 35, 45, 25, 40。这表明排序过程中, 部分元素之间发生了跨越较大距离的比较和移动, 不是简单的相邻元素的交换。这正是希尔排序中使用较大增量进行分组插入排序的特点。

第 2 趟排序后的序列: 5, 10, 15, 25, 20, 30, 40, 35, 45, 显示出排序过程逐步趋向有序的趋势。

综合来看, 该排序过程符合希尔排序利用较大增量分组排序后逐步减小增量进行整体插入排序的特性, 而不符合基数排序、归并排序或折半 (即二分) 插入排序的处理方式。

## 类型三十五: 交换类排序

1、【2010.10】采用递归方式对顺序表进行快速排序。下列关于递归次数的叙述中, 正确的是\_\_\_\_\_。

- A. 递归次数与初始数据的排列次序无关。
- B. 每次划分后, 先处理较长的分区可以减少递归次数。
- C. 每次划分后, 先处理较短的分区可以减少递归次数。
- D. 递归次数与每次划分后得到的分区的处理顺序无关。

2、【2011.10】为实现快速排序算法, 待排序序列宜采用的存储方式是\_\_\_\_\_。

- A. 顺序存储
- B. 散列存储
- C. 链式存储
- D. 索引存储

3、【2014.11】下列选项中, 不可能是快速排序第 2 趟排序结果的是\_\_\_\_\_。

- A. 2, 3, 5, 4, 6, 7, 9
- B. 2, 7, 5, 6, 4, 3, 9
- C. 3, 2, 5, 4, 7, 6, 9
- D. 4, 2, 3, 5, 7, 6, 9

4、【2019.10】排序过程中, 对尚未确定最终位置的所有元素进行一遍处理称为一“趟”。下列序列中, 不可能是快速排序第二趟结果的是

- A. 5, 2, 16, 12, 28, 60, 32, 72
- B. 2, 16, 5, 28, 12, 60, 32, 72
- C. 2, 12, 16, 5, 28, 32, 72, 60
- D. 5, 2, 12, 28, 16, 32, 72, 60

5、【2023.11】使用快速排序算法对数据进行升序排序, 若经过一次划分后得到的数据序列是 68, 11, 70, 23, 80, 77, 48, 81, 93, 88, 则该次划分的枢轴是 ( )

- A. 11
- B. 70
- C. 80
- D. 81

6、【2024.08】使用快速排序算法对含  $n$  ( $n \geq 3$ ) 个元素的数组  $M$  进行排序, 若第一趟排序将  $M$  中除枢轴外的  $n-1$  个元素划分为均不为空的  $P$  和  $Q$  两块, 则下列叙述中, 正确的是 ( )。

- A.  $P$  与  $Q$  块间有序
- B.  $P$  与  $Q$  均块内有序
- C.  $P$  和  $Q$  的元素个数大致相等
- D.  $P$  中和  $Q$  中均不存在相等的元素

### 【答案解析】

1、D. 递归次数和各元素的初始排列有关。如果每次划分后的分区比较平衡, 则递归次数少; 如果分区不平衡, 递归次数多。递归次数与处理顺序是没有关系的。快速排序的递归次数与元素的初始排列有关。若每次划分后分区比较平衡, 则递归次数少; 若划分后分区不平衡, 则递归次数多。

但快速排序的递归次数与分区处理顺序无关,即先处理较长的分区或先处理较短的分区都不影响递归次数。此外,可以形象地把快速排序的递归调用过程用一个二叉树描述,先处理较长或较短分区,可以想象为交换某一递归结点处的左右子树,这并不会影响树中的分支数。

2、A。内部排序采用顺序存储结构。快排是内部排序算法,要求必须采用顺序结构进行存储。但是,这里出一个问题:如果使用单链表实现快快排?

3、C。快排的阶段性排序结果的特点是,第  $i$  趟完成时,会有  $i$  个以上的数出现在它最终将要出现的位置,即它左边的数都比它小,它右边的数都比它大。题目问第二趟排序的结果,即要找不存在 2 个这样的数的选项。A 选项中 2、3、6、7、9 均符合,所以 A 排除;B 选项中,2、9 均符合,所以 B 排除;D 选项中 5、9 均符合,所以 D 选项排除;最后看 C 选项,只有 9 一个数符合,所以 C 不可能是快速排序第二趟的结果。

4、D。要理解清楚排序过程中一“趟”的含义,题干也进行了解释。一个初始无序序列,所有元素都没有确定最终位置,对所有元素做一次(称为一趟)快速排序后一个元素确定最终位置,且将原序列划分成了前后两块,此时前后两块子表是无序的。按“一趟”的解释——对尚未确定最终位置的所有元素都处理一遍才是一趟,所以此时要对前后两块子表各做一次快速排序才是一“趟”快速排序,如果只对一块子表进行了排序,而未处理另一块子表,就不能算是完整的一趟。

选项 A, 第一趟匹配 72, 只余一块无序序列,第二趟匹配 28, 选项 A 可能。选项 B, 第一趟匹配 2, 第二趟匹配 72, 选项 B 可能。选项 c, 第一趟匹配 2, 第二趟匹配 28 或 32, 选项 C 可能。选项 D, 无论先匹配 12 还是先匹配 32, 都会将序列分成两块,那么第二趟必须有两个元素匹配, 所以选项 D 不可能。故选 D。

5、D。快速排序每次可以确定枢轴元素的最终位置,经过依次排序后,枢轴左边的元素都小于枢轴元素,枢轴元素右边的元素都大于枢轴元素。由此可判断此次划分的枢轴为 81, 81 左边所有元素都小于 81, 81 右边元素都大于 81。

6、A。快速排序算法的基本思想是,通过一趟排序将待排记录分割成独立的两部分,其中一部分记录的关键字均比另一部分的关键字小,然后分别对这两部分继续进行排序,以达到整个序列有序的目标。

在这个问题中,在第一趟排序后,将数组 M 除枢轴外的  $N-1$  个元素划分为均不空的 P 和 Q 两块。对于这种情况,可以得出以下结论:

选项 A: P 与 Q 块间有序:这是正确的。在快速排序中,选择一个枢轴,然后将所有小于枢轴的元素放在枢轴的左边(即 P 块),将所有大于枢轴的元素放在枢轴的右边(即 Q 块)。因此, P 块中的所有元素都小于 Q 块中的所有元素,所以 P 与 Q 块间是有序的。

选项 B: P 与 Q 均块内有序:这是错误的。在快速排序的一趟排序过程中,只是根据是否小于或大于枢轴来划分元素,而并没有对 P 块或 Q 块内部的元素进行排序。因此,虽然 P 块和 Q 块间有序,但 P 块和 Q 块内部可能是无序的。

选项 C: P 和 Q 的元素个数大致相等:这是错误的。在快速排序中, P 块和 Q 块的元素个数取决于选取的枢轴。如果枢轴恰好是中位数,那么 P 块和 Q 块的元素个数会大致相等。但如果枢轴偏离中位数,那么 P 块和 Q 块的元素个数可能会有很大的不同。

选项 D: P 和 Q 中均不存在相等的元素:这是错误的。在快速排序中,只是将元素划分为小于枢轴的部分和大于枢轴的部分。如果原数组中存在相等的元素,那么在划分后的 P 块或 Q 块中仍然可能存在相等的元素。

## 类型三十六：选择类排序

- 1、【2009.09】已知关键字序列 5, 8, 12, 19, 28, 20, 15, 22 是小根堆（最小堆），插入关键字 3，调整后得到的小根堆是\_\_\_\_\_。  
A. 3, 5, 12, 8, 28, 20, 15, 22, 19      B. 3, 5, 12, 19, 20, 15, 22, 8, 28  
C. 3, 8, 12, 5, 20, 15, 22, 28, 19      D. 3, 12, 5, 8, 28, 20, 15, 22, 19
- 2、【2011.11】已知序列 25, 13, 10, 12, 9 是大根堆，在序列尾部插入新元素 18，将其再调整为大根堆，调整过程中元素之间进行的比较次数是\_\_\_\_\_。  
A. 1      B. 2      C. 4      D. 5
- 3、【2015.10】已知小根堆为 8, 15, 10, 21, 34, 16, 12，删除关键字 8 之后需重建堆，在此过程中，关键字之间的比较次数是\_\_\_\_\_。  
A. 1      B. 2      C. 3      D. 4
- 4、【2018.11】在将数据序列 (6, 1, 5, 9, 8, 4, 7) 建成大根堆时，正确的序列变化过程是  
A. 6, 1, 7, 9, 8, 4, 5 → 6, 9, 7, 1, 8, 4, 5 → 9, 6, 7, 1, 8, 4, 5 → 9, 8, 7, 1, 6, 4, 5  
B. 6, 9, 5, 1, 8, 4, 7 → 6, 9, 7, 1, 8, 4, 5 → 9, 6, 7, 1, 8, 4, 5 → 9, 8, 7, 1, 6, 4, 5  
C. 6, 9, 5, 1, 8, 4, 7 → 9, 6, 5, 1, 8, 4, 7 → 9, 6, 7, 1, 8, 4, 5 → 9, 8, 7, 1, 6, 4, 5  
D. 6, 1, 7, 9, 8, 4, 5 → 7, 1, 6, 9, 8, 4, 5 → 7, 9, 6, 1, 8, 4, 5 → 9, 7, 6, 1, 8, 4, 5 → 9, 8, 6, 1, 7, 4, 5
- 5、【2020.09】下列关于大根堆（至少含 2 个元素）的叙述中正确的是：  
I. 可以将堆看成一棵完全二叉树；II. 可采用顺序存储方式保存堆；  
III. 可以将堆看成一棵二叉排序树；IV. 堆中的次大值一定在根的下一层  
A. I      B. II      C. I, II      D. I, II, III
- 6、【2021.11】将关键字 6, 9, 1, 5, 8, 4, 7 依次插入到初始为空的大根堆 H 中，得到的 H 是  
A. 9, 8, 7, 6, 5, 4, 1      B. 9, 8, 7, 5, 6, 1, 4  
C. 9, 8, 7, 5, 6, 4, 1      D. 9, 6, 7, 5, 8, 4, 1
- 7、【2024.09】已知关键字序列 28, 22, 20, 19, 8, 12, 15, 5 是大根堆（最大堆），对该堆进行两次删除操作后，得到的新堆是  
A. 20, 19, 15, 12, 8, 5      B. 20, 19, 15, 5, 8, 12  
C. 20, 19, 12, 15, 8, 5      D. 20, 19, 8, 12, 15, 5

### 【答案解析】

1、A。根据小根堆调整规则调整即可。插入关键字 3 时，先将其放在小顶堆的末端，再将该关键字向上进行调整，调整后的小顶堆序列为 3, 5, 12, 8, 28, 20, 15, 22, 19。

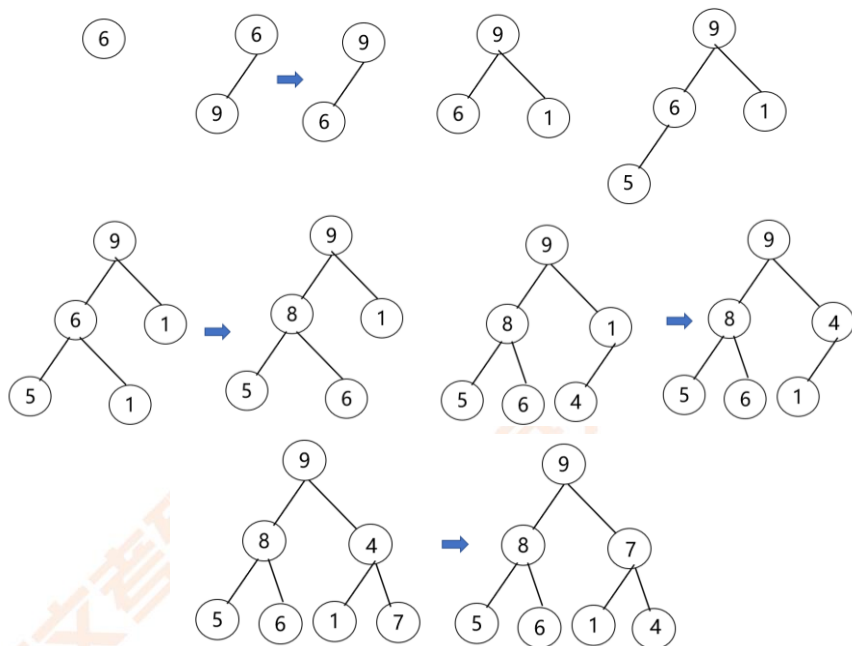
2、B。首先与 10 比较，交换位置，再与 25 比较，不交换位置。比较了二次。

3、C。删除 8 后，将 12 移动到堆顶，第一次是 15 和 10 比较，第二次是 10 和 12 比较并交换，第三次还需要比较 12 和 16，故比较次数为 3 次。

4、A。要熟练掌握建堆、堆的调整方法，从序列末尾开始向前遍历。首先根据所给出的数据建立对应的树。对节点进行从下至上，从右至左依次比较，将最大的节点移动到根节点处。

5、C。这是一道简单的概念题。堆是一棵完全树，采用一维数组存储，故 I 正确，II 正确。大根堆只要求根结点值大于左右孩子值，并不要求左右孩子值有序，III 错误。堆的定义是递归的，所以其左右子树也是大根堆，所以堆的次大值一定是其左孩子或右孩子，IV 正确。

6、B。本题是一边插入，一边调整。如图所示



其对应的序列是 9, 8, 7, 5, 6, 1, 4

7、B。大根堆（最大堆）是一种特殊的完全二叉树，它的每个结点的值都大于或等于其子结点的值。在大根堆中，最大的元素总是位于根结点。

对大根堆进行删除操作时，总是删除根结点，也就是堆中最大的元素。删除操作的步骤如下：

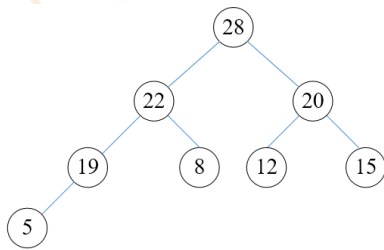
①移除根结点。

②将堆的最后一个元素移动到根结点的位置。

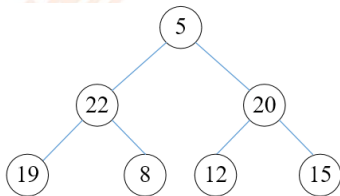
③从根结点开始，将新的根结点与其子结点进行比较，如果根结点小于任何一个子结点，就将根结点与最大的子结点交换。

④重复步骤③，直到根结点大于其所有子结点，或者已经没有子结点。

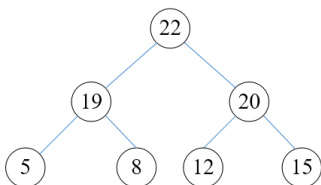
现有的堆状态：



删除 28 后的堆状态：

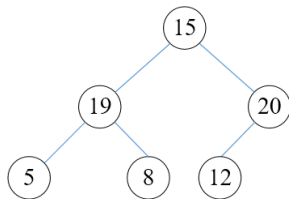


重新调整为大根堆：

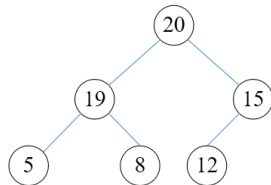




删除 22 后的堆状态:



重新调整为大根堆:



所以答案为 B。

## 类型三十七：基数类排序

1、【2013.11】对给定的关键字序列 110, 119, 007, 911, 114, 120, 122 进行基数排序, 则第 2 趟分配收集后得到的关键字序列是\_\_\_\_\_。

- A. 007, 110, 119, 114, 911, 120, 122      B. 007, 110, 119, 114, 911, 122, 120  
C. 007, 110, 911, 114, 119, 120, 122      D. 110, 120, 911, 122, 114, 007, 119

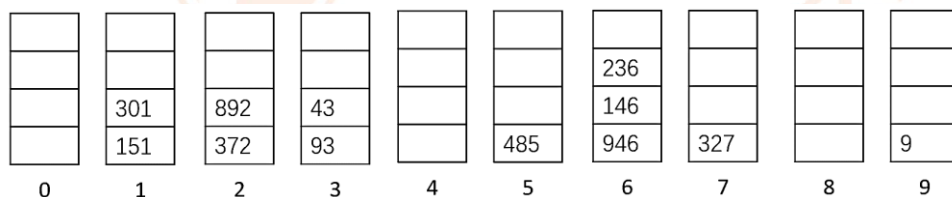
2、【2021.10】设数组  $S=[93,946,372,9,146,151,301,485,236,327,43,892]$ , 采用最低位优先(LSD)基数排序将 S 排列成升序序列。第 1 趟分配、收集后, 元素 372 之前、之后紧邻的元素分别是

- A. 43, 892      B. 236, 301      C. 301, 892      D. 485, 301

### 【答案解析】

1、C。基数排序的第 1 趟排序是按照个位数字来排序的, 第 2 趟排序是按照十位数字的大小进行排序的, 答案是 C 选项。

2、C。按照 LSD 的方法, 进行一趟分配的结果如图所示,



按照分配的结果进行回收, 得到[151,301,372,892,93,43,485,946,146,236,327,9]

## 类型三十八：归并类排序

1、【2019.11】设外存上有 120 个初始归并段, 进行 12 路归并时, 为实现最佳归并, 需要补充的虚段个数是

- A. 1      B. 2      C. 3      D. 4

2、【2022.10】使用二路归并排序对含  $n$  个元素的数组  $M$  进行排序时, 对二路归并操作的功能

- A. 将两个有序表合并为一个新的有序表  
B. 将  $M$  划分为两部分, 两部分的元素个数大致相等

- C. 将  $M$  划分为  $n$  个部分, 每个部分中仅含所有一个元素  
 D. 将  $M$  划分为两部分, 一部分元素的值均小于另一部分元素的值  
 3、【2024.10】现有由关键字组成的 3 个有序序列 (3, 5)、(7, 9) 和 (6), 若按从左至右的次序选择有序序列进行二路归并排序, 则关键字之间的总比较次数是  
 A. 3                      B. 4                      C. 5                      D. 6

### 【答案解析】

1、B。在 12 路归并树中只存在度为 0 和度为 12 的结点, 设度为 0 的结点数、度为 12 的结点数和要补充的结点数分别为  $n_0$ ,  $n_{12}$ ,  $n_{\text{补}}$ , 则有  $n_0 = 120 + n_{\text{补}}$ ,  $n_0 = (12 - 1)n_{12} + 1$ , 可得  $n_{12} = (120 - 1 + n_{\text{补}}) / (12 - 1)$ 。由于结点数  $n_{12}$  为整数, 所以  $n_{\text{补}}$  是使上式整的最小整数, 求得  $n_{\text{补}} = 2$ , 所以答案选 B。

2、A。将含有  $n$  个记录的原始序列看做  $n$  个长度 1 的子序列。从第一个子序列开始, 将相邻的两个子序列合并, 得到长度为 2 或 1 (最后只剩下 1 个子序列的情况) 的子序列。如此重复成对归并, 直到序列的长度等于原始序列的长度时为止。基本过程: 两个有序序列的合并。

归并排序中的归并 merge 操作就是将两个有序表合并成一个表。所以 A 选项正确。

B 错误, 这是快速排序的调优, 当枢轴进行平衡划分的时候, 即两部分元素数量大致相等, 快速排序的性能是最好的。

C 错误, 归并排序不一定要分成两部分, 使得最小归并段只有一个元素。其实最小归并段可以包含多个元素。

D 错误, 这是快速排序的性质, 枢轴将数组分成左右两侧, 如果所有元素互不相同, 调整后枢轴左侧元素小于枢轴右侧元素。

3、C。首先, 将 (3,5) 和 (7,9) 进行归并, 得到新的有序序列 (3,5,7,9)。在这个过程中, 需要进行 2 次比较: 首先比较 3 和 7, 然后比较 5 和 7, 5 已经到末尾, 所以不需要继续比较。然后, 我们将新的有序序列 (3,5,7,9) 和 (6) 进行归并。在这个过程中, 需要进行 3 次比较: 首先比较 3 和 6, 然后比较 5 和 6, 最后比较 6 和 7, 6 已经到末尾, 所以不需要继续比较。因此, 关键字之间的总比较次数是  $2+3=5$ 。所以, 正确答案是 C。

## 类型三十九: 外部排序

1、【2024.11】在外排序中, 利用败者树对初始为升序的归并段进行多路归并, 败者树中记录“冠军”的结点保存的是

- A、最大关键字                      B、最小关键字  
 C、最大关键字所在的归并段号                      D、最小关键字所在的归并段号

2、【2023.42】对含有  $n$  ( $n > 0$ ) 个记录的文件进行外部排序, 采用置换-选择排序生成初始归并段时需要使用一个工作区, 工作区中能保存  $m$  个记录。请回答下列问题。

(1) 若文件中含有 19 个记录, 其关键字依次是 51, 94, 37, 92, 14, 63, 15, 99, 48, 56, 23, 60, 31, 17, 43, 8, 90, 166, 100, 当  $m=4$  时, 可生成几个初始归并段? 各是什么?

(2) 对任意的  $m$  ( $n \gg m > 0$ ), 生成的第一个初始归并段的长度最大值和最小值分别是多少?

### 【答案解析】

1、D。败者树是一种树形数据结构, 常用于多路归并排序和其他需要高效获取最小 (或最大)

元素的场景。败者树的主要思想是“胜者负，败者胜”，即在每一轮比较中，胜者进入下一轮比较，败者则被保存在相应的内部结点中。

败者树的特点如下：

1. 每个非叶子结点存储的是其所有子结点中“败者”的值。
2. 树的根结点存储的是所有元素中的“胜者”（即最小或最大元素）。

构建败者树的步骤如下：

1. 首先，将所有元素作为叶子结点，然后进行两两比较，将胜者保存到父结点，胜者进入下一轮比较。

2. 然后，继续进行两两比较，直到所有元素都比较完，此时树的根结点存储的就是所有元素中的胜者。

在败者树中，获取最小（或最大）元素的操作非常高效，只需要访问根结点即可。当需要插入或删除元素时，只需要调整一部分结点，而不需要调整整棵树，因此效率也很高。

在外排序中，利用败者树进行多路归并时，败者树中记录“冠军”的结点保存的是最小关键字所在的归并段号。因为在多路归并排序中，总是希望能够快速找到所有归并段中的最小元素，以便将其输出到排序结果中。败者树的设计正是为了实现这个目标：在每一轮比较中，总是将最小元素（即“冠军”）留在树根，而将“败者”保存在内部结点中。

2、（1）解析 (1) 由于  $m=4$ ，因此一个工作区可以保存 4 个记录，按照置换-选择排序的方法求归并段过程如表所示。

| FO                                      | WA          | FI                                                        |
|-----------------------------------------|-------------|-----------------------------------------------------------|
|                                         |             | 51,94,37,92,14,63,15,99,48,56,23,60,31,17,43,8,90,166,100 |
|                                         | 51,94,37,92 | 14,63,15,99,48,56,23,60,31,17,43,8,90,166,100             |
| 37                                      | 51,94,92    | 14,63,15,99,48,56,23,60,31,17,43,8,90,166,100             |
| 37                                      | 51,94,14,92 | 63,15,99,48,56,23,60,31,17,43,8,90,166,100                |
| 37,51                                   | 94,14,92    | 63,15,99,48,56,23,60,31,17,43,8,90,166,100                |
| 37,51                                   | 63,94,14,92 | 15,99,48,56,23,60,31,17,43,8,90,166,100                   |
| 37,51,63                                | 94,14,92    | 15,99,48,56,23,60,31,17,43,8,90,166,100                   |
| 37,51,63                                | 15,94,14,92 | 99,48,56,23,60,31,17,43,8,90,166,100                      |
| 37,51,63,92                             | 15,94,14    | 99,48,56,23,60,31,17,43,8,90,166,100                      |
| 37,51,63,92                             | 15,94,14,99 | 48,56,23,60,31,17,43,8,90,166,100                         |
| 37,51,63,92,94                          | 15,14,99    | 48,56,23,60,31,17,43,8,90,166,100                         |
| 37,51,63,92,94                          | 15,48,14,99 | 56,23,60,31,17,43,8,90,166,100                            |
| 37,51,63,92,94,99                       | 15,48,14    | 56,23,60,31,17,43,8,90,166,100                            |
| 37,51,63,92,94,99,*                     | 15,48,14,56 | 23,60,31,17,43,8,90,166,100                               |
| 37,51,63,92,94,99,*14                   | 15,48,56    | 23,60,31,17,43,8,90,166,100                               |
| 37,51,63,92,94,99,*14                   | 15,48,23,56 | 60,31,17,43,8,90,166,100                                  |
| 37,51,63,92,94,99,*14,15                | 48,23,56    | 60,31,17,43,8,90,166,100                                  |
| 37,51,63,92,94,99,*14,15                | 60,48,23,56 | 31,17,43,8,90,166,100                                     |
| 37,51,63,92,94,99,*14,15,23             | 60,48,56    | 31,17,43,8,90,166,100                                     |
| 37,51,63,92,94,99,*14,15,23             | 60,48,31,56 | 17,43,8,90,166,100                                        |
| 37,51,63,92,94,99,*14,15,23,31          | 60,48,56    | 17,43,8,90,166,100                                        |
| 37,51,63,92,94,99,*14,15,23,31          | 60,48,17,56 | 43,8,90,166,100                                           |
| 37,51,63,92,94,99,*14,15,23,31,48       | 60,17,56    | 43,8,90,166,100                                           |
| 37,51,63,92,94,99,*14,15,23,31,48       | 60,43,17,56 | 8,90,166,100                                              |
| 37,51,63,92,94,99,*14,15,23,31,48,56    | 60,43,17    | 8,90,166,100                                              |
| 37,51,63,92,94,99,*14,15,23,31,48,56    | 60,43,17,8  | 90,166,100                                                |
| 37,51,63,92,94,99,*14,15,23,31,48,56,60 | 43,17,8     | 90,166,100                                                |
| 37,51,63,92,94,99,*14,15,23,31,48,56,60 | 90,43,17,8  | 166,100                                                   |

|                                                                 |             |         |
|-----------------------------------------------------------------|-------------|---------|
| 37,51,63,92,94,99,*,14,15,23,31,48,56,60,90                     | 43,17,8     | 166,100 |
| 37,51,63,92,94,99,*,14,15,23,31,48,56,60,90                     | 166,43,17,8 | 100     |
| 37,51,63,92,94,99,*,14,15,23,31,48,56,60,90,166                 | 43,17,8     | 100     |
| 37,51,63,92,94,99,*,14,15,23,31,48,56,60,90,166,*               | 100,43,17,8 |         |
| 37,51,63,92,94,99,*,14,15,23,31,48,56,60,90,166,*,8             | 100,43,17   |         |
| 37,51,63,92,94,99,*,14,15,23,31,48,56,60,90,166,*,8,17          | 100,43      |         |
| 37,51,63,92,94,99,*,14,15,23,31,48,56,60,90,166,*,8,17,43       | 100         |         |
| 37,51,63,92,94,99,*,14,15,23,31,48,56,60,90,166,*,8,17,43,100,* |             |         |

由表可知,可生成3个初始归并段,分别是①37,51,63,92,94,99; ②14,15,23,31,48,56,60,90,166; ③8,17,43,100。

(2) 最大可能长度为  $n$ , 即刚进入 WA 的  $m$  个关键字一定能够被输出, 每次进入 WA 的元素刚好大于刚输入 FO 中的元素。最小可能长度为  $m$ , 即刚进入 WA 的  $m$  个关键字一定能够被输出, 而后续进入 WA 中的元素均无法输出到第一个归并段, WA 中的最小值大于 FI 中的最大值。

## 类型四十：排序的比较分析

1、【2009.10】若数据元素序列 11, 12, 13, 7, 8, 9, 23, 4, 5 是采用下列排序方法之一得到的第二趟排序后的结果, 则该排序算法只能是\_\_\_\_\_。

A. 起泡排序      B. 插入排序      C. 选择排序      D. 二路归并排序

2、【2010.11】对一组数据(2, 12, 16, 88, 5, 10)进行排序, 若前三趟排序结果如下:

第一趟排序结果: 2, 12, 16, 5, 10, 88

第二趟排序结果: 2, 12, 5, 10, 16, 88

第三趟排序结果: 2, 5, 10, 12, 16, 88 则采用的排序方法可能是\_\_\_\_\_。

A. 起泡排序      B. 希尔排序      C. 归并排序      D. 基数排序

3、【2012.10】在内部排序过程中, 对尚未确定最终位置的所有元素进行一遍处理称为一趟排序。下列排序方法中, 每一趟排序结束都至少能够确定一个元素最终位置的方法是\_\_\_\_\_。

I. 简单选择排序; II. 希尔排序; III. 快速排序; IV. 堆排序; V. 二路归并排序

A. 仅 I、III、IV      B. 仅 I、III、V      C. 仅 II、III、IV      D. 仅 III、IV、V

4、【2015.09】下列排序算法中, 元素的移动次数与关键字的初始排列次序无关的是\_\_\_\_\_。

A. 直接插入排序      B. 起泡排序      C. 基数排序      D. 快速排序

5、【2016.11】对 10TB 的数据文件进行排序, 应使用的方法是\_\_\_\_\_。

A. 希尔排序      B. 堆排序      C. 快速排序      D. 归并排序

6、【2017.10】在内部排序时, 若选择了归并排序而没有选择插入排序, 则可能的理由是\_\_\_\_\_。

I. 归并排序的程序代码更短; II. 归并排序的占用空间更少; III. 归并排序的运行效率更高

A. 仅 II      B. 仅 III      C. 仅 I、II      D. 仅 I、III

7、【2017.11】下列排序方法中, 若将顺序存储更换为链式存储, 则算法的时间效率会降低的是\_\_\_\_\_。

I. 插入排序      II. 选择排序      III. 起泡排序      IV. 希尔排序      V. 堆排序



- A. 仅I、II                      B. 仅II、III                      C. 仅III、IV                      D. 仅IV、V
- 8、【2019.07】选择一个排序算法时,除算法的时空效率外,下列因素中,还需要考虑的是  
I.数据的规模 II.数据的存储方式  
III.算法的稳定性 IV.数据的初始状态  
A.仅III                      B.仅 I、II                      C.仅II、III、IV                      D. I、II、III、IV
- 9、【2020.11】对大部分元素已有序的数组进行排序时,直接插入排序比简单选择排序效率更高,其原因是:  
I、直接插入排序过程中元素之间的比较次数更少  
II、直接插入排序过程中所需要的辅助空间更少  
III、直接插入排序过程中元素的移动次数更少  
A、I                      B、III                      C、I、II                      D、I、II、III
- 10、【2023.10】下列排序算法中,不稳定的是( )  
I. 希尔排序; II. 归并排序; III. 快速排序; IV. 堆排序; V. 基数排序  
A. I、II                      B. II、V                      C. I、III、IV                      D. III、IV、V
- 11、【2025.10】下列排序算法中,最坏情况下元素移动次数最少的是:  
A. 起泡排序                      B. 直接插入排序                      C. 快速排序                      D. 简单选择排序

#### 【答案解析】

1、B。冒泡和选择排序每趟之后都会有一个元素被放置在最终位置上。 $\{11, 12\}$ 和 $\{4, 5\}$ 并不是最终位置,因此排除冒泡和选择。二路归并算法之后应该是每4个元素有序。但是 $\{11, 12, 13, 7\}$ 并不有序,因此排除二路归并算法。解答本题需要对各种排序算法的特点极为清楚。对于冒泡排序和选择排序,每一趟都能确定一个元素的最终位置,而题目中,前2个元素和后2个元素均不是最小或最大的2个元素并按序排列。选项D中的2路归并排序,第一趟排序结束都可以得到若干个有序子序列,而此时序列中并没有两两元素有序排列。插入排序在每趟排序后能确定前面的若干元素是有序的,而此时第二趟排序后,序列的前三个元素是有序的,符合其特征。

2、A。题中所给的三趟排序过程中,每一趟排序是从前往后依次比较,使最大值“沉底”,符合冒泡排序的特点。看第一趟可知仅有88被移到最后。

- 如果是希尔排序,则12, 88, 10应变为10, 12, 88。因此排除希尔排序。
- 如果是归并排序,则长度为2的子序列是有序的。因此可排除归并排序。
- 如果是基数排序,则16, 5, 10应变为10, 5, 16。因此排除基数排序。

3、A。对于I,简单选择排序每次选择未排序列中的最小元素放入其最终位置。对于II,希尔排序每次是对划分的子表进行排序,得到局部有序的结果,所以不能保证每一趟排序结束都能确定一个元素的最终位置。对于III,快速排序每一趟排序结束后都将枢轴元素放到最终位置。对于IV,堆排序属于选择排序,每次都大根堆的根结点与表尾结点交换,确定其最终位置。对于V,二路归并排序每趟对子表进行两两归并从而得到若干个局部有序的结果,但无法确定最终位置。

4、C。直接插入排序、起泡排序、快速排序元素的移动次数都与关键字的初始排列次序有关。

5、D。外部排序指待排序文件较大,内存一次性放不下,需存放在外部介质中。外部排序通常采用归并排序法。选项A、B、C都是内部排序的方法。

6、B。归并排序的时间复杂度更低,运行效率更高。归并排序代码比选择插入排序更复杂,前者空间复杂度是 $O(n)$ ,后者是 $O(1)$ 。但是前者时间复杂度是 $O(n\log n)$ ,后者是 $O(n^2)$ 。所以B正确。

7、D. 希尔排序需要多次分割链表进行直接插入排序, 堆排序需要多次调整堆。这些操作都不利于在链表上进行。插入排序、选择排序、起泡排序原本时间复杂度是  $O(n^2)$ , 更换为链式存储后的时间复杂度 还是  $O(n^2)$ 。希尔排序和堆排序都利用了顺序存储的随机访问特性, 而链式存储不支持这种性质, 所以时间复杂度会增加, 因此选 D。

8、D。当数据规模较小时可选择复杂度为  $O(n^2)$  的简单排序方法, 当数据规模较大时应选择复杂度为  $O(n\log_2 n)$  的排序方法, 当数据规模大到内存无法放下时需选择外部排序方法, I 正确。数据的存储方式主要分为顺序存储和链式存储, 有些排序方法 (如堆排序) 只能用于顺序存储方式, II 正确。若对数据稳定性有要求, 则不能选择不稳定的排序方法, III 显然正确。当数据初始基本有序时, 直接插入排序的效率最高, 冒泡排序和直接插入排序的时间复杂度都是  $O(n)$ , 而归并排序的时间复杂度依旧是  $O(n\log_2 n)$ , IV 正确。所以选 D。

9、A。考虑较极端的情况, 对于有序数组, 直接插入排序的比较次数为  $n-1$ , 简单选择排序的比较次数始终为  $1+2+\dots+n-1=n(n-1)/2$ , I 正确。两种排序方法的辅助空间都是  $O(1)$ , 无差别, II 错误。初始有序时, 移动次数均为 0; 对于通常情况, 直接插入排序每趟插入都需要依次向后挪位, 而简单选择排序只需与找到的最小元素交换位置, 后者的移动次数少很多, III 错误。

10、C。稳定的排序有: 插入排序、冒泡排序、归并排序、基数排序。

不稳定的排序有: 选择排序、希尔排序、快速排序、堆排序。

11、D。起泡排序在最坏情况下需要频繁相邻交换, 移动次数非常多, 时间复杂度为  $O(n^2)$ ; 直接插入排序在逆序排列 (最坏情况) 时, 每个元素都要向后移动多次, 总移动次数也为  $O(n^2)$ ; 快速排序在最坏情况下比较次数较高, 因分区操作需要多次交换, 移动次数也不少, 时间复杂度为  $O(n^2)$ ; 简单选择排序在排序过程中, 每次从剩余元素中选择最小值, 然后交换到当前位置, 无论数据如何, 其交换次数固定为  $n-1$ , 移动次数最少。

## 类型十四一：排序算法题专题

1、【2021.42】已知某排序算法如下:

```
void cmpCountSort(int a[], int b[], int n) {
 int i, j, *count;
 count = (int *)malloc(sizeof(int) * n); // C++语言: count=new int[n];
 for(i=0; i<n; i++) count[i]=0;
 for(i=0; i<n-1; i++)
 for(j=i+1; j<n; j++)
 if(a[i]<a[j]) count[j]++;
 else count[i]++;
 for(i=0; i<n; i++) b[count[i]]=a[i];
 free(count); // C++语言: delete count;
}
```

请回答下列问题。

- (1) 若有  $\text{int } a[] = \{25, -10, 25, 10, 11, 19\}$ ,  $b[6]$ ; , 则调用  $\text{cmpCountSort}(a, b, 6)$  后数组  $b$  中的内容是什么?
- (2) 若  $a$  中含有  $n$  个元素, 则算法执行过程中, 元素之间的比较次数是多少?
- (3) 该算法是稳定的吗? 若是, 则阐述理由; 否则, 修改为稳定排序算法。

### 【答案解析】

1、(1) 算法运行结果如下

|         | 0   | 1   | 2         | 3  | 4         | 5  |
|---------|-----|-----|-----------|----|-----------|----|
| a       | 25  | -10 | <u>25</u> | 10 | 11        | 19 |
| 排列结果: b | -10 | 10  | 11        | 19 | <u>25</u> | 25 |
| count   | 5   | 0   | 4         | 1  | 2         | 3  |

其中, a 为原始数组; b 数组用于存放最终的排序结果; count[i] 的大概含义是, 在数组 a 中, 不大于 a[i] 的元素有几个,

则, 调用 couCountSort(a, b, n) 后数组  $b = \{-10, 10, 11, 19, 25, 25\}$

(2) 从 for 循环的逻辑可知, 第一趟排序, 用 a[0] 与 a[i~n-1] 比较; 第二趟排序, 用 a[1] 与 a[2~n-1] 比较; 第三趟排序, 用 a[2] 与 a[3~n-1] 比较.....总共进行 n-1 趟, 总的关键字比较次数为  $(n-1)+(n-2)+(n-3)+\dots+1=n*(n-1)/2$ , 则, 比较次数  $n*(n-1)/2$

(3) 如上图所示, 用第一小问给的例子验证就能发现算法是不稳定的, 则, 该排序算法不稳定, 只需将 if (a[i]<a[j]) 改为 if (a[i]≤a[j]) 即可保证排序结果稳定。

需要将程序中的 if 语句修改如下:

```
if(a[i] <= a[j])
 count[j]++;
else
 count[i] ++;
```

## 类型四十二：数组运算和矩阵压缩

1、【2016.4】有一个 100 阶的三对角矩阵 M, 其元素  $m_{i,j} (1 \leq i \leq 100, 1 \leq j \leq 100)$  按行优先次序压缩存入下标从 0 开始的一维数组 IV 中。元素  $m_{30,30}$  在 N 中的下标是\_\_\_\_\_。

A. 86                      B. 87                      C. 88                      D. 89

2、【2017.3】适用于压缩存储稀疏矩阵的两种存储结构是\_\_\_\_\_。

A. 三元组表和十字链表                      B. 三元组表和邻接矩阵  
C. 十字链表和二叉链表                      D. 邻接矩阵和十字链表

3、【2018.03】设有一个 12\*12 的对称矩阵 M, 将其上三角部分的元素  $M_{i,j} (1 \leq i \leq j \leq 12)$  按行优先存入 C 语言的一维数组 N 中, 元素  $M_{6,6}$  在 N 中的下标是

A.50                      B.51                      C.55                      D.66

4、【2020.01】将一个 10\*10 对称矩阵 M 的上三角部分的元素  $m_{ij} (1 \leq i \leq j \leq 10)$  按列优先存入 C 语言的一维数组 N 中, 元素  $m_{7,2}$  在 N 中的下标是:

A、15                      B、16                      C、22                      D、23

5、【2021.03】已知二维数组 A 按行优先方式存储, 每个元素占用 1 个存储单元。若元素 A[0][0] 的存储地址是 100, A[3][3] 的存储地址是 220 则元素 A[5][5] 的存储地址是

A.295                      B.300                      C.301                      D.306

6、【2021.03】若采用三元组表存储结构存储稀疏矩阵 M, 则除三元组表外, 下列数据中还需要保存的是 ( )

I. M 的行数; II. M 中包含非零元素的行数; III. M 的列数; IV. M 中包含非零元素的列数

A. I、III                      B. I、IV                      C. II、IV                      D. I、II、III、IV

7、【2011.42】一个长度为 L ( $L \geq 1$ ) 的升序序列 S, 处在第  $(L/2)$  向上取整个位置的数称为 S 的中位数。例如, 若序列  $S_1 = (11, 13, 15, 17, 19)$ , 则  $S_1$  的中位数是 15, 两个序列的中位数是含它们所有元素

的升序序列的中位数。例如,若  $S_2 = (2, 4, 6, 8, 20)$ , 则  $S_1$  和  $S_2$  的中位数是 11。现在有两个等长升序序列  $A$  和  $B$ , 试设计一个在时间和空间两方面都尽可能高效的算法, 找出两个序列  $A$  和  $B$  的中位数。要求:

- (1) 给出算法的基本设计思想。
- (2) 根据设计思想, 采用 C 或 C++ 或 JAVA 语言描述算法, 关键之处给出注释。
- (3) 说明你所设计算法的时间复杂度和空间复杂度。

8、【2020.41】定义三元组  $(a, b, c)$  ( $a, b, c$  均为正数) 的距离  $D = |a-b| + |b-c| + |c-a|$ 。给定 3 个非空整数集合  $S_1, S_2, S_3$ , 按升序分别存储在 3 个数组中。请设计一个尽可能高效的算法, 计算并输出所有可能的三元组  $(a, b, c)$  ( $a \in S_1, b \in S_2, c \in S_3$ ) 中的最小距离。例如  $S_1 = \{-1, 0, 9\}, S_2 = \{-25, -10, 10, 11\}, S_3 = \{2, 9, 17, 30, 41\}$  则最小距离为 2, 相应的三元组为  $(9, 10, 9)$  要求:

- (1) 给出算法的基本设计思想;
- (2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释;
- (3) 说明你所设计算法的时间复杂度和空间复杂度。

9、【2025.41】设有两个长度均为  $n$  的一维整型数组  $A$  和  $res$ , 对数组  $A$  中的每个元素  $A[i]$ , 计算  $A[i]$  与  $A[j]$  ( $0 \leq i \leq j \leq n-1$ ) 乘积的最大值, 并将其保存到  $res[i]$  中。例如, 若  $A = \{1, 4, -9, 6\}$ , 则得到  $res = \{6, 24, 81, 36\}$ 。现给定数组  $A$ , 请设计一个时间和空间上尽可能高效的算法  $calMulMax$ 。求  $res$  中各元素的值。函数原型为: `void calMulMax (int A[], int res[], int n)`。要求:

- (1) 给出算法的基本设计思想。(4 分)
- (2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。(7 分)
- (3) 说明你所设计算法的时间复杂度和空间复杂度。(2 分)

### 【答案解析】

1、B。画出三对角矩阵, 观察规律, 第一行有 2 个元素, 剩下的在  $m_{30, 30}$  之前的 28 行中每行都有 3 个元素, 而  $m_{30, 30}$  之前仅有一个元素,  $m_{30, 29}$ , 所以  $m_{30, 30}$  在数组  $N$  中的下标是:  $2 + 28 \times 3 + 2 - 1 = 87$ 。

2、A。三元组表的结点存储了行  $row$ 、列  $col$ 、值  $value$  三种信息, 主要用来存储稀疏矩阵的一种数据结构。十字链表将行单链表和列单链表结合起来存储稀疏矩阵。邻接矩阵空间复杂度达  $O(n^2)$ , 不适于存储稀疏矩阵。二叉链表又名左孩子右兄弟表示法, 可用于表示树或森林。因此 A 正确。

3、A。数组  $N$  的下标从 0 开始, 第一个元素  $m_{1, 1}$  对应存入  $n_0$ 。矩阵  $M$  的第一行有 12 个元素, 第二行有 11 个, 第三行有 10 个, 第四行有 9 个, 第五行有 8 个, 所以  $m_{6, 6}$  是第  $12 + 11 + 10 + 9 + 8 + 1 = 51$  个元素, 下标应为 50, 故选 A。

4、C。上三角矩阵按列优先存储, 先存储仅 1 个元素的第一列, 再存储有 2 个元素的第二列, 以此类推。 $m_{7, 2}$  位于左下角, 对应右上角的元素为  $m_{2, 7}$ , 在  $m_{2, 7}$  之前存有

第 1 列: 1

第 2 列: 2

.....

第 6 列: 6

第 7 列: 1

前面共存有  $1 + 2 + 3 + 4 + 5 + 6 + 1 = 22$  个元素 (数组下标范围为 0~21), 注意数组下标从 0



开始, 故  $m_2, 7$  在数组  $N$  中的下标为 22, 即  $m_{7, 2}$  在数组  $N$  中的下标为 22。

5、B。按照行优先存储, 假设二维数组是  $A[m][n]$ , 即是每一行  $n$  个元素, 一共  $m$  行;  
 $[3][3] = A[0][0] + 1 * 3 * n + 3 * 1 = 220$ ; 且  $A[0][0] = 100$ ; 可以得到  $n = 39$   
 $[5][5] = A[0][0] + 1 * 5 * n + 5 * 1 = 100 + 5 * 39 + 5 = 300$   
 另外的方法, 计算  $A[3][3]$  到  $A[5][5]$  之间的元素个数, 再加上  $A[3][3]$  的存储地址即可。

6、A。三元组从左到右存储依次为行号, 列号, 元素值存储稀疏矩阵的非 0 元素。因此还需要保存的是  $M$  的行数 (I),  $M$  的列数 (III) 和非 0 元素。

7、(1) 算法的基本设计思想如下。分别求出序列  $A$  和  $B$  的中位数, 设为  $a$  和  $b$ , 求序列  $A$  和  $B$  的中位数过程如下: 1) 若  $a=b$ , 则  $a$  或  $b$  即为所求中位数, 算法结束。2) 若  $a < b$ , 则舍弃序列  $A$  中较小的一半, 同时舍弃序列  $B$  中较大的一半, 要求舍弃的长度相等; 3) 若  $a > b$ , 则舍弃序列  $A$  中较大的一半, 同时舍弃序列  $B$  中较小的一半, 要求舍弃的长度相等; 在保留的两个升序序列中, 重复过程 1)、2)、3), 直到两个序列中只含一个元素时为止, 较小者即为所求的中位数。

算法的实现如下:

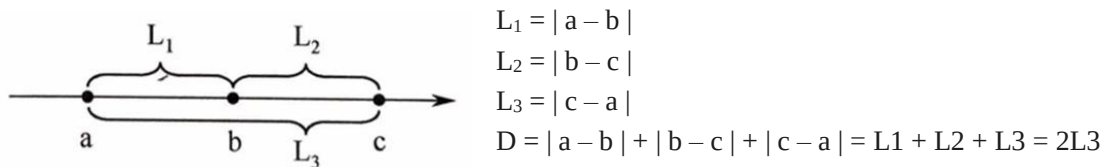
```
int M_Search(int A[], int B[], int n){
 int s1 = 0, d1 = n-1, m1, s2 = 1, d2 = n-1, m2; //分别表示序列 A 和 B 的首位数、末位数和中位数
 while(s1 != d1 || s2 != d2){
 m1 = (s1+d1)/2; m2 = (s2+d2)/2;
 if(A[m1] == B[m2]) return A[m1]; //满足条件 1)
 if(A[m1] < B[m2]){ //满足条件 2
 if((s1+d1) % 2 == 0) { //若元素个数为奇数
 s1 = m1; //舍弃 A 中间点以前的部分且保留中间点
 d2 = m2; //舍弃 B 中间点以后的部分且保留中间点
 } else { //元素个数为偶数
 s1 = m1+1; //舍弃 A 中间点及中间点以前部分
 d2 = m2; //舍弃 B 中间点以后部分且保留中间点
 }
 } else { //满足条件 3
 if((s1+d1) % 2 == 0) { //若元素个数为奇数
 d1 = m1; //舍弃 A 中间点以后的部分且保留中间点
 s2 = m2; //舍弃 B 中间点以前的部分且保留中间点
 } else { //元素个数为偶数
 d1 = m1 + 1; //舍弃 A 中间点以后部分且保留中间点
 s2 = m2; //舍弃 B 中间点及中间点以前部分
 }
 }
 }
 return A[s1] < B[s2] ? A[s1] : B[s2];
}
```

(3) 算法的时间复杂度为  $O(\log_2 n)$ , 空间复杂度为  $O(1)$ 。

8、解答: 分析。由  $D = |a-b| + |b-c| + |c-a| \geq 0$  得:

①当  $a=b=c$  时, 距离最小。

②其余情况。不失一般性, 假设  $a \leq b \leq c$ , 观察下面的数轴:



由D的表达式可知,事实上决定D大小的关键是a和c之间的距离,于是问题就可以简化为每次固定c找一个a使得 $L_3 = |c - a|$ 最小。

(1) 算法的基本设计思想

①使用 $D_{\min}$ 记录所有已处理过的三元组的最小距离,初值为一个足够大的整数。

②集合 $S_1$ 、 $S_2$ 和 $S_3$ 分别保存在数组A、B、C中,数组的下标变量 $i=j=k=0$ ,当 $i < |S_1|$ 、 $j < |S_2|$ 且 $k < |S_3|$ 时, ( $|S|$ 表示集合S中的元素个数),循环执行a)~c)。

a) 计算  $(A[i], B[j], C[k])$  的距离D; (计算D)

b) 若 $D < D_{\min}$ , 则 $D_{\min} = D$ ; (更新D)

c) 将 $A[i], B[j], C[k]$ 中最小值的下标+1;(对照分析:最小值为a,最大值为c,这里c不变而更新a,试图寻找更小距离D)

③输出 $D_{\min}$ ,结束。

(2) 算法实现:

```
#define INT_MAX 0x7fffffff
int abs_(int a){ //计算绝对值
 if (a < 0) return -a;
 else return a;
}
bool xls_min(int a,int b,int c){ //a是否为三个数中的最小值
 if (a <= b && a <= c) return true;
 return false;
}
int findMinofTrip(int A[],int n,int B[],int m,int c[],int p){
 //D_min用于记录三元组的最小距离, 初值赋为INT_MAX
 int i=0,j=0,k=0,D_min=INT_MAX,D;
 while(i < n && j < m && k < p && D_min > 0){
 D=abs_(A[i]-B[j])+abs_(B[j]-C[k])+abs_(C[k]-A[i]); //计算D
 if(D < D_min) D_min=D; //更新D
 if(xls_min(A[i],B[j],C[k])) i++; //更新a
 else if(xls_min(B[j],C[k],A[i])) j++;
 else k++;
 }
 Return D_min
}
```

(3) 设 $n=(|S_1|+|S_2|+|S_3|)$ ,时间复杂度为 $O(n)$ ,空间复杂度为 $O(1)$ 。

9、(1) **此算法的核心思想是:** 对于每个位置  $i$ , 找到一个在  $[i, n-1]$  区间内的值, 使得  $A[i] * X$  的结果最大。

当  $A[i] \geq 0$  时, 为了使乘积最大, 应选择右侧最大的数;

当  $A[i] < 0$  时, 负数乘以最小的数 (可能是更小的负数) 才能得到更大的正积;

因此使用动态维护的 Max 和 Min 来记录从当前位置到末尾的最大值和最小值;

**算法的基本设计思想:**

从后向前扫描一趟数组 A, 对每个  $A[i] (0 \leq i \leq n-1)$ , 分别找到从  $A[n-1]$  到  $A[i]$  中的最大值 Max 和最小值 Min, 然后分以下情况处理:

① 若  $A[i] \geq 0$ , 则  $A[i]$  与 Max 相乘;

② 若  $A[i] < 0$ , 则  $A[i]$  与 Min 相乘;

相乘的结果保存在  $res[i]$  中。

## (2) 算法实现

```
void calMulMax(int A[], int res[], int n)
{
 int i, Max, Min;
 Max = Min = A[n-1]; // 初始化最大值和最小值为数组最后一个元素
 for(i = n-1; i >= 0; i--) // 从数组末尾开始向前遍历
 {
 // 更新从位置 i 到 n-1 范围内的最大值
 if(A[i] > Max)
 Max = A[i];
 // 更新从位置 i 到 n-1 范围内的最小值
 else if(A[i] < Min)
 Min = A[i];
 // 根据当前元素 A[i] 的正负性决定与哪个值相乘:
 if(A[i] >= 0)
 res[i] = A[i] * Max; // 非负数乘以右侧最大值
 else
 res[i] = A[i] * Min; // 负数乘以右侧最小值 (使结果尽可能大)
 }
}
```

(3) 算法的时间复杂度为  $O(n)$ ；空间复杂度为  $O(1)$ 。

**您的满意是我们唯一和全部的追求**



**万学教育**  
微信公众号



**海文神龙考研**  
微信公众号



集团总部地址：北京市海淀区中关村大街数码大厦 28 层

万学主网：[www.wanxue.cn](http://www.wanxue.cn)

本书版权属于北京万学教育科技有限公司所有，违者必究